ALL_CODE.md

FILE: app/Enums/GiftTypeEnum.php

```
<?php

namespace App\Enums;

enum GiftTypeEnum: string
{
    case Lion = 'lion';
    case Flower = 'flower';
    case Star = 'star';
    case Heart = 'heart';

    // Optionally, add a method to get the display name
    public function getDisplayName(): string
    {
        return ucfirst($this->value);
    }

    // Get the price associated with each gift
    public function getPrice(): float
    {
        switch ($this) {
            case self::Lion:
                return 100.00;
            case self::Flower:
                return 50.00;
            case self::Star:
                return 200.00;
            case self::Heart:
                return 150.00;
            default:
                return 0.00;
        }
    }
}
```

FILE: app/Enums/QuestionTypeEnum.php

```
<?php
namespace App\Enums;

class QuestionTypeEnum
{
    const TEXT = 'text';
    const TEXTAREA = 'textarea';
    const SELECT = 'select';
    const RADIO = 'radio';
    const CHECKBOX = 'checkbox';

    public static function getValues()
    {
        return [
            self::TEXT,
            self::TEXTAREA,
            self::SELECT,
            self::RADIO,
            self::CHECKBOX,
        ];
    }
```

```
}
```

FILE: app/Events/GroupDeleted.php

```
<?php

namespace App\Events;

use App\Http\Resources\GroupResource;
use Illuminate\Broadcasting\Channel;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Broadcasting\PresenceChannel;
use Illuminate\Broadcasting\PrivateChannel;
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;
use Illuminate\Contracts\Broadcasting\ShouldBroadcastNow;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;

class GroupDeleted implements ShouldBroadcastNow
{
    use Dispatchable, InteractsWithSockets, SerializesModels;

    public int $id;
    public string $name;

    /**
     * Create a new event instance.
     *
     * @param int $id
     * @param string $name
     * @return void
     */
    public function __construct(int $id, string $name)
    {
        $this->id = $id;
        $this->name = $name;
    }

    /**
     * Get the channels the event should broadcast on.
     *
     * @return array<int, \Illuminate\Broadcasting\Channel>
     */
    public function broadcastOn(): array
    {
        return [
            // Using PrivateChannel with a dynamic channel name for group deletion
            new PrivateChannel('group.deleted.' . $this->id),
        ];
    }

    /**
     * Optionally, you can broadcast additional data like name, but it's usually good practice
     * to broadcast only necessary data for your frontend to consume.
     */
    public function broadcastWith(): array
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
        ];
    }
}
```

FILE: app/Events/OrderStatusUpdated.php

```
<?php
namespace App\Events;

use App\Models\Order;
use Illuminate\Broadcasting\Channel;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Broadcasting\PresenceChannel;
use Illuminate\Broadcasting\BroadcastManager;
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;
use Illuminate\Contracts\Broadcasting\ShouldBroadcastNow;


class OrderStatusUpdated implements ShouldBroadcastNow
{
    use Dispatchable, InteractsWithSockets, SerializesModels;

    public $order; // The order data you want to broadcast

    public function __construct(Order $order)
    {
        $this->order = $order;
    }

    public function broadcastOn()
    {
        return new Channel('order-status.' . $this->order->id);
    }

    public function broadcastWith()
    {
        return [
            'orderId' => $this->order->id,
            'status' => $this->order->status,
        ];
    }
}
```

FILE: app/Events/SocketMessage.php

```
<?php

namespace App\Events;

use App\Http\Resources\MessageResource;
use App\Models\Message;
use Illuminate\Broadcasting\Channel;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Broadcasting\PresenceChannel;
use Illuminate\Broadcasting\PrivateChannel;
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;
use Illuminate\Contracts\Broadcasting\ShouldBroadcastNow;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;

class SocketMessage implements ShouldBroadcastNow
{
    use Dispatchable, InteractsWithSockets, SerializesModels;
```

```
    public Message $message;

    /**
     * Create a new event instance.
     *
     * @param Message $message
     * @return void
     */
    public function __construct(Message $message)
    {
        $this->message = $message;
    }

    /**
     * The data to broadcast with the event.
     *
     * @return array
     */
    public function broadcastWith(): array
    {
        // Make sure the MessageResource is properly wrapping the message
        return [
            'message' => new MessageResource($this->message),
        ];
    }

    /**
     * Get the channels the event should broadcast on.
     *
     * @return array<int, \Illuminate\Broadcasting\Channel>
     */
    public function broadcastOn(): array
    {
        $m = $this->message;
        $channels = [];

        // If the message belongs to a group, broadcast to the group channel
        if ($m->group_id) {
            $channels[] = new PrivateChannel('message.group.' . $m->group_id);
        } else {
            // For private messages, ensure the user-to-user channel is created correctly
            $userIds = collect([$m->sender_id, $m->receiver_id])->sort()->implode('-');
            $channels[] = new PrivateChannel('message.user.' . $userIds);
        }

        return $channels;
    }
}
```

FILE: app/Events/StreamChatMessageSent.php

```
<?php

namespace App\Events;

use App\Models\Stream;
use App\Models\StreamMessage;
use Illuminate\Broadcasting\Channel;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Broadcasting\BroadcastManager;
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;
use Illuminate\Contracts\Broadcasting\ShouldBroadcastNow;
```

```
class StreamChatMessageSent implements ShouldBroadcastNow
{
    use Dispatchable, InteractsWithSockets, SerializesModels;

    public $streamMessage; // The StreamMessage data you want to broadcast

    /**
     * Create a new event instance.
     *
     * @param StreamMessage $streamMessage
     */
    public function __construct(StreamMessage $streamMessage)
    {
        $this->streamMessage = $streamMessage;
    }

    /**
     * Get the channels the event should broadcast on.
     *
     * @return Channel
     */
    public function broadcastOn()
    {
        // Broadcasting on a channel related to the stream
        return new Channel('stream-chat.' . $this->streamMessage->stream_id);
    }

    /**
     * The data to broadcast with the event.
     *
     * @return array
     */
    public function broadcastWith()
    {
        return [
            'message' => $this->streamMessage->message,
            'creator' => [
                'id' => $this->streamMessage->creator_id,
                'first_name' => $this->streamMessage->creator->first_name,
                'last_name' => $this->streamMessage->creator->last_name,
            ],
            'viewer' => [
                'id' => $this->streamMessage->viewer_id,
                'first_name' => $this->streamMessage->viewer->first_name,
                'last_name' => $this->streamMessage->viewer->last_name,
            ],
            'stream_id' => $this->streamMessage->stream_id,
        ];
    }
}
```

FILE: app/Http/Controllers/Auth/AuthenticatedSessionController.php

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\Request;
use Illuminate\Http\Response;
use Illuminate\Support\Facades\Auth;
```

```
class AuthenticatedSessionController extends Controller
{
    /**
     * Handle an incoming authentication request.
     */
    public function store(LoginRequest $request): Response
    {
        $request->authenticate();

        $request->session()->regenerate();

        return response()->noContent();
    }

    /**
     * Destroy an authenticated session.
     */
    public function destroy(Request $request): Response
    {
        Auth::guard('web')->logout();

        $request->session()->invalidate();

        $request->session()->regenerateToken();

        return response()->noContent();
    }
}
```

FILE: app/Http/Controllers/Auth/EmailVerificationNotificationController.php

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;

class EmailVerificationNotificationController extends Controller
{
    /**
     * Send a new email verification notification.
     */
    public function store(Request $request): JsonResponse|RedirectResponse
    {
        if ($request->user()->hasVerifiedEmail()) {
            return redirect()->intended('/dashboard');
        }

        $request->user()->sendEmailVerificationNotification();

        return response()->json(['status' => 'verification-link-sent']);
    }
}
```

FILE: app/Http/Controllers/Auth/NewPasswordController.php

```
<?php
```

```
namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Auth\Events\PasswordReset;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Password;
use Illuminate\Support\Str;
use Illuminate\Validation\Rules;
use Illuminate\Validation\ValidationException;

class NewPasswordController extends Controller
{
    /**
     * Handle an incoming new password request.
     *
     * @throws \Illuminate\Validation\ValidationException
     */
    public function store(Request $request): JsonResponse
    {
        $request->validate([
            'token' => ['required'],
            'email' => ['required', 'email'],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        // Here we will attempt to reset the user's password. If it is successful we
        // will update the password on an actual user model and persist it to the
        // database. Otherwise we will parse the error and return the response.
        $status = Password::reset(
            $request->only('email', 'password', 'password_confirmation', 'token'),
            function ($user) use ($request) {
                $user->forceFill([
                    'password' => Hash::make($request->string('password')),
                    'remember_token' => Str::random(60),
                ])->save();

                event(new PasswordReset($user));
            }
        );

        if ($status != Password::PASSWORD_RESET) {
            throw ValidationException::withMessages([
                'email' => [__($status)],
            ]);
        }

        return response()->json(['status' => __($status)]);
    }
}
```

FILE: app/Http/Controllers/Auth/PasswordResetLinkController.php

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Password;
use Illuminate\Validation\ValidationException;
```

```
class PasswordResetLinkController extends Controller
{
    /**
     * Handle an incoming password reset link request.
     *
     * @throws \Illuminate\Validation\ValidationException
     */
    public function store(Request $request): JsonResponse
    {
        $request->validate([
            'email' => ['required', 'email'],
        ]);

        // We will send the password reset link to this user. Once we have attempted
        // to send the link, we will examine the response then see the message we
        // need to show to the user. Finally, we'll send out a proper response.
        $status = Password::sendResetLink(
            $request->only('email')
        );

        if ($status != Password::RESET_LINK_SENT) {
            throw ValidationException::withMessages([
                'email' => [__($status)],
            ]);
        }

        return response()->json(['status' => __($status)]);
    }
}
```

FILE: app/Http/Controllers/Auth/RegisteredUserController.php

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Auth\Events\Registered;
use Illuminate\Http\Request;
use Illuminate\Http\Response;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules;

class RegisteredUserController extends Controller
{
    /**
     * Handle an incoming registration request.
     *
     * @throws \Illuminate\Validation\ValidationException
     */
    public function store(Request $request): Response
    {
        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'lowercase', 'email', 'max:255', 'unique:'.User::class],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->string('password')),
```

```
        ]);

        event(new Registered($user));

        Auth::login($user);

        return response()->noContent();
    }
}
```

FILE: app/Http/Controllers/Auth/VerifyEmailController.php

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Auth\Events\Verified;
use Illuminate\Foundation\Auth\EmailVerificationRequest;
use Illuminate\Http\RedirectResponse;

class VerifyEmailController extends Controller
{
    /**
     * Mark the authenticated user's email address as verified.
     */
    public function __invoke(EmailVerificationRequest $request): RedirectResponse
    {
        if ($request->user()->hasVerifiedEmail()) {
            return redirect()->intended(
                config('app.frontend_url').'/dashboard?verified=1'
            );
        }

        if ($request->user()->markEmailAsVerified()) {
            event(new Verified($request->user()));
        }

        return redirect()->intended(
            config('app.frontend_url').'/dashboard?verified=1'
        );
    }
}
```

FILE: app/Http/Controllers/AuthController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Validator;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Log;

class AuthController extends Controller
{
    // Signup method
    public function signup(Request $request)
```

```
    {
        // Validate the request
        $validator = Validator::make($request->all(), [
            'first_name' => 'required|string|max:255',
            'last_name' => 'required|string|max:255',
            'email' => 'required|string|email|unique:users,email|max:255',
            'phone_number' => 'nullable|string|max:15', // Optional field for phone number
            'ip_address' => 'nullable|ip', // Optional field for IP address
            'password' => 'required|string|min:6',
            'language' => 'nullable|string|max:2', // Optional language code (e.g., 'en', 'fr')
        ]);

        // Handle validation failure
        if ($validator->fails()) {
            Log::warning('Signup validation failed', $validator->errors()->toArray());
            return response()->json(['message' => $validator->errors()->first()], 400);
        }

        try {
            // Create the user
            $user = User::create([
                'first_name' => $request->first_name,
                'last_name' => $request->last_name,
                'email' => $request->email,
                'phone_number' => $request->phone_number,
                'ip_address' => $request->ip_address,
                'language' => $request->language ?? 'en', // Default to 'en' if not provided
                'password' => Hash::make($request->password),
            ]);

            // Create a new token for the user
            $token = $user->createToken('token-name')->plainTextToken;

            // Return all user details along with the token
            return response()->json([
                'message' => 'User registered successfully',
                'user' => $user,
                'token' => $token
            ], 201);
        } catch (\Exception $error) {
            Log::error("Signup error: ", ['error' => $error->getMessage()]);
            return response()->json(['message' => 'Server error'], 500);
        }
    }

    // Login method
    public function login(Request $request)
    {
        // Validate the request
        $validator = Validator::make($request->all(), [
            'email' => 'required|string|email|max:255',
            'password' => 'required|string',
        ]);

        // Handle validation failure
        if ($validator->fails()) {
            Log::warning('Login validation failed', $validator->errors()->toArray());
            return response()->json(['message' => $validator->errors()->first()], 400);
        }

        // Attempt to log the user in
        $credentials = $request->only('email', 'password');
        Log::info('Login attempt', $credentials);

        // Find the user by email first
        $user = User::where('email', $request->email)->first();
```

```
        if (!$user || !Hash::check($request->password, $user->password)) {
            Log::warning('Invalid login credentials', $credentials);
            return response()->json(['message' => 'Invalid credentials'], 400);
        }

        // Log the user in
        Auth::login($user);

        // Generate the token
        $token = $user->createToken('token-name')->plainTextToken;

        // Optionally, you can log the successful login
        Log::info('User logged in successfully', ['user_id' => $user->id]);

        // Return all user details along with the token
        return response()->json([
            'message' => 'Logged in successfully',
            'user' => $user,
            'token' => $token
        ]);
    }

    // Logout method
    public function logout(Request $request)
    {
        try {

            // Optionally, log out session-based authentication
            Auth::logout();

            // Log the logout action

            return response()->json(['message' => 'Logged out successfully']);
        } catch (\Exception $error) {
            Log::error("Logout error: ", ['error' => $error->getMessage()]);
            return response()->json(['message' => 'Server error'], 500);
        }
    }

}
```

FILE: app/Http/Controllers/BlockController.php

```
<?php

namespace App\Http\Controllers;

use Agence104\LiveKit\RoomServiceClient;
use Agence104\LiveKit\AccessToken;
use Agence104\LiveKit\AccessTokenOptions;
use Agence104\LiveKit\VideoGrant;
use Illuminate\Http\Request;
use App\Models\User;
use App\Models\Block;
use Illuminate\Support\Facades\Log; // Import the Log facade

class BlockController extends Controller
{
    private $roomService;

    public function __construct()
    {
        $this->roomService = new RoomServiceClient(
```

```
            env('LIVEKIT_URL'),
            env('LIVEKIT_API_KEY'),
            env('LIVEKIT_API_SECRET')
        );
    }

    // Block a user
    public function blockUser(Request $request, $id)
    {
        $self = $request->user(); // Get the authenticated user from the request

        Log::info("User {$self->id} is attempting to block user {$id}");

        if ($self->id == $id) {
            Log::warning("User {$self->id} tried to block themselves.");
            return response()->json(['error' => 'You cannot block yourself'], 400);
        }

        $blockedUser = User::find($id);

        if (!$blockedUser) {
            Log::error("User {$self->id} tried to block a non-existing user {$id}");
            return response()->json(['error' => 'User not found'], 404);
        }

        // Check if already blocked
        $existingBlock = Block::where('blocker_id', $self->id)
            ->where('blocked_id', $blockedUser->id)
            ->first();

        if ($existingBlock) {
            Log::info("User {$self->id} has already blocked user {$id}");
            return response()->json(['error' => 'Already blocked'], 400);
        }

        // Block the user
        $block = Block::create([
            'blocker_id' => $self->id,
            'blocked_id' => $blockedUser->id,
        ]);

        Log::info("User {$self->id} successfully blocked user {$id}");

        // Remove user from the LiveKit room if they're in
        try {
            $this->roomService->removeParticipant($self->id, $id);
            Log::info("Removed user {$id} from the LiveKit room.");
        } catch (\Exception $e) {
            Log::warning("User {$id} was not in the room or failed to remove from room.");
        }

        return response()->json($block);
    }

    // Unblock a user
    public function unblockUser(Request $request, $id)
    {
        $self = $request->user(); // Get the authenticated user from the request

        Log::info("User {$self->id} is attempting to unblock user {$id}");

        $blockedUser = User::find($id);

        if (!$blockedUser) {
            Log::error("User {$self->id} tried to unblock a non-existing user {$id}");
            return response()->json(['error' => 'User not found'], 404);
```

```
        }

        // Check if the user is blocked
        $block = Block::where('blocker_id', $self->id)
            ->where('blocked_id', $blockedUser->id)
            ->first();

        if (!$block) {
            Log::info("User {$self->id} has not blocked user {$id}, so cannot unblock.");
            return response()->json(['error' => 'Not blocked'], 400);
        }

        // Unblock the user
        $block->delete();

        Log::info("User {$self->id} successfully unblocked user {$id}");

        return response()->json(['message' => 'User unblocked successfully']);
    }

    // Check if a user is blocked by the authenticated user
    public function isBlockedByUser(Request $request, $id)
    {
        $self = $request->user(); // Get the authenticated user from the request

        Log::info("User {$self->id} is checking if user {$id} is blocked.");

        $blockedUser = User::find($id);

        if (!$blockedUser) {
            Log::error("User {$self->id} tried to check block status of a non-existing user {$id}");
            return response()->json(['error' => 'User not found'], 404);
        }

        // Check if blocked
        $existingBlock = Block::where('blocker_id', $blockedUser->id)
            ->where('blocked_id', $self->id)
            ->first();

        Log::info("User {$self->id} checked block status of user {$id}: " . ($existingBlock !== null ? 'Blocked' : 'Not Blocked'));

        return response()->json($existingBlock !== null);
    }

    // Get all blocked users
    public function getBlockedUsers(Request $request)
    {
        $self = $request->user(); // Get the authenticated user from the request

        Log::info("User {$self->id} is requesting their list of blocked users.");

        $blockedUsers = Block::where('blocker_id', $self->id)
            ->with('blocked')
            ->get();

        Log::info("User {$self->id} has " . $blockedUsers->count() . " blocked users.");

        return response()->json($blockedUsers);
    }
}
```

FILE: app/Http/Controllers/CartController.php

```
<?php
```

```
namespace App\Http\Controllers;

use App\Models\Product;
use App\Models\Order;
use App\Models\OrderItem;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Illuminate\Support\Facades\Auth;

class CartController extends Controller
{

    /**
     * Add product to the user's cart.
     */
    public function addToCart(Request $request)
    {
        try {
            $user = $request->user();  // Get the authenticated user

            $productId = $request->input('productId');
            $orderId = $request->input('orderId');
            $product = Product::find($productId);  // Correct method: find()

            if (!$product) {
                return response()->json(['message' => 'Product not found'], 404);
            }

            $pricePerProduct = $product->price;

            // If no orderId is provided or the order doesn't exist, create a new order
            $order = Order::find($orderId);

            if (!$order) {
                // Create a new order if it doesn't exist
                $order = new Order([
                    'user_id' => $user->id,
                    'status' => 'pending',  // Set the initial status, modify as needed
                    'order_date' => now(),
                    'total_amount' => 0,  // Start with a total amount of 0
                ]);

                // Optionally generate a tracking number (can be done here or later)
                $order->tracking_number = strtoupper('TRACK-' . uniqid());

                // Save the new order
                $order->save();
            }

            // Check if the product is already in the cart (OrderItem exists for this order and product)
            $existingItem = OrderItem::where('orders_id', $order->id)
                                    ->where('products_id', $productId)
                                    ->first();

            // If the product is already in the cart
            if ($existingItem) {
                Log::info('Existing Item', $existingItem->toArray());

                // Store the previous total amount before updating the quantity
                $previousTotalAmount = $existingItem->total_amount;

                // Increment the quantity by 1
                $existingItem->quantity += 1;
                $existingItem->total_amount = $existingItem->quantity * $pricePerProduct;  // Recalculate total amount for the item

                // Save the updated order item
```

```
            $existingItem->save();

            // Update the total amount of the order by adding the difference between the new total and the old total
            $order->total_amount += $existingItem->total_amount - $previousTotalAmount;
        } else {
            // If the product is not in the cart, add it
            $newOrderItem = new OrderItem([
                'orders_id' => $order->id,
                'products_id' => $productId,
                'quantity' => 1,
                'total_amount' => $pricePerProduct
            ]);
            $newOrderItem->save();  // Save the new order item

            // Update the total amount of the order
            $order->total_amount += $pricePerProduct;
        }

        // Save the updated order
        $order->save();

        // Retrieve all order items for this order
        $allOrderItems = OrderItem::where('orders_id', $order->id)->with('product')->get();
        $totalAmount = $allOrderItems->sum(function ($item) {
            return $item->total_amount;  // Assuming each order item has a total_amount field
        });
        return response()->json([
            'orderItems' => $allOrderItems,
            'orderId' => $order->id,  // Return the updated order, including the tracking number and total amount
            'totalAmount' => $totalAmount,
        ]);
    } catch (\Exception $e) {
        return response()->json(['message' => 'Server error', 'error' => $e->getMessage()], 500);
    }
}


public function removeCartItem(Request $request)
{
    try {
        // Get the authenticated user
        $user = $request->user();
        $orderId = $request->input('orderId');
        $productId = $request->input('productId');
        // Find the order associated with the provided orderId
        $order = Order::find($orderId);

        if (!$order) {
            return response()->json(['message' => 'Order not found'], 404);
        }

        // Check if the order belongs to the authenticated user
        if ($order->user_id !== $user->id) {
            return response()->json(['message' => 'You are not authorized to modify this order'], 403);
        }

        // Find the specific order item for the provided productId
        $orderItem = OrderItem::where('orders_id', $orderId)
                            ->where('products_id', $productId)
                            ->first();

        if (!$orderItem) {
            return response()->json(['message' => 'Product not found in cart'], 404);
        }

        // Store the total amount of the item being deleted
```

```
        $itemTotalAmount = $orderItem->total_amount;

        // Delete the order item from the cart
        $orderItem->delete();

        // Update the total amount of the order
        $order->total_amount -= $itemTotalAmount;

        // Save the updated order
        $order->save();

        // Retrieve the updated order items
        $allOrderItems = OrderItem::where('orders_id', $orderId)->with('product')->get();
        $totalAmount = $allOrderItems->sum(function ($item) {
            return $item->total_amount;  // Assuming each order item has a total_amount field
        });
        return response()->json([
            'orderId' => $order->id,
            'orderItems' => $allOrderItems,
            'totalAmount' => $totalAmount,

        ]);
    } catch (\Exception $e) {
        return response()->json(['message' => 'Server error', 'error' => $e->getMessage()], 500);
    }
}

/**
 * Update the quantity of a product in the user's cart by incrementing or decrementing.
 */
public function updateOrderItemQuantity(Request $request)
{
    try {
        // Get the authenticated user
        $user = $request->user();
        $orderId = $request->input('orderId');
        $productId = $request->input('productId');
        $isIncrement = $request->input('isIncrement'); // true for increment, false for decrement

        // Ensure 'isIncrement' is either true or false
        if ($isIncrement === null) {
            return response()->json(['message' => 'isIncrement flag is required'], 400);
        }

        // Find the order associated with the provided orderId
        $order = Order::find($orderId);

        if (!$order) {
            return response()->json(['message' => 'Order not found'], 404);
        }

        // Check if the order belongs to the authenticated user
        if ($order->user_id !== $user->id) {
            return response()->json(['message' => 'You are not authorized to modify this order'], 403);
        }

        // Find the specific order item for the provided productId
        $orderItem = OrderItem::where('orders_id', $orderId)
                            ->where('products_id', $productId)
                            ->first();

        if (!$orderItem) {
            return response()->json(['message' => 'Product not found in cart'], 404);
        }

        // Get the price per product for recalculating the total amount
```

```
            $product = Product::find($productId);
            if (!$product) {
                return response()->json(['message' => 'Product not found'], 404);
            }

            $pricePerProduct = $product->price;

            // Store the previous total amount before updating the quantity
            $previousTotalAmount = $orderItem->total_amount;

            // Adjust the quantity based on the 'isIncrement' flag
            if ($isIncrement) {
                $orderItem->quantity += 1; // Increase quantity by 1
            } else {
                if ($orderItem->quantity > 1) {
                    $orderItem->quantity -= 1; // Decrease quantity by 1, but never go below 1
                } else {
                    return response()->json(['message' => 'Quantity cannot be less than 1'], 400);
                }
            }

            // Recalculate the total amount for the item
            $orderItem->total_amount = $orderItem->quantity * $pricePerProduct;

            // Save the updated order item
            $orderItem->save();

            // Update the total amount of the order by adding the difference between the new total and the old total
            $order->total_amount += $orderItem->total_amount - $previousTotalAmount;

            // Save the updated order
            $order->save();

            // Retrieve the updated order items
            $allOrderItems = OrderItem::where('orders_id', $orderId)->with('product')->get();
            $totalAmount = $allOrderItems->sum(function ($item) {
                return $item->total_amount;  // Assuming each order item has a total_amount field
            });
            return response()->json([
                'orderId' => $order->id,
                'orderItems' => $allOrderItems,
                'totalAmount' => $totalAmount,

            ]);
        } catch (\Exception $e) {
            return response()->json(['message' => 'Server error', 'error' => $e->getMessage()], 500);
        }
    }

}```

## FILE: app/Http/Controllers/ChatBotController.php
```php
<?php
namespace App\Http\Controllers;

use App\Models\Comment;
use App\Models\Product;
use App\Models\Post;
use App\Models\Coupon;
use App\Models\ChatBot;
use App\Models\User;
use App\Models\Role;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Illuminate\Support\Facades\Validator;
```
```

```
class ChatBotController extends Controller
{

    /**
     * Gather necessary information for the AI chatbot.
     *
     * @return \Illuminate\Http\Response
     */
    public function gatherChatbotContext()
    {
        // Fetch latest comments for each post (if needed)
        $comments = Comment::with('post', 'user')->get(10);

        // Fetch products data
        $products = Product::with('supplier', 'categories', 'coupons')->get(10);

        // Fetch posts and their associated comments and likes
        $posts = Post::with(['author', 'comments', 'likes'])->get(10);

        // Fetch coupons (products and users associated with them)
        $coupons = Coupon::with('users', 'products')->get(10);

        // Fetch roles and permissions (assuming this is relevant to your context)
        $roles = Role::with('permissions')->get(10);

        // Optional: You can combine all the data into a single structure to make it easy to pass
        $contextData = [
            'comments' => $comments,
            'products' => $products,
            'posts' => $posts,
            'coupons' => $coupons,
            'roles' => $roles,
        ];

        // Return the gathered data (or format as necessary)
        return response()->json($contextData);
    }

    // Store a new chat
    public function storeChat(Request $request)
    {
        $userId = $request->user()->id;  // Get the currently authenticated user's ID
        $text = $request->input('text');

        // Log incoming request data
        Log::info('storeChat method called', ['user_id' => $userId, 'text' => $text]);

        // Validate incoming request
        $validator = Validator::make($request->all(), [
            'text' => 'required|string|max:255',
        ]);

        if ($validator->fails()) {
            Log::warning('Validation failed', ['errors' => $validator->errors()]);
            return response()->json($validator->errors(), 400);
        }

        try {
            // CREATE A NEW CHAT
            $chat = new ChatBot([
                'user_id' => $userId,
                'history' => [
                    [
                        'role' => 'user',
                        'parts' => [['text' => $text]]
```

```
                ]
            ]
        ]);
        $chat->save();

        Log::info('Chat created successfully', ['chat_id' => $chat->id]);

        return response()->json($chat);
    } catch (\Exception $e) {
        Log::error('Error creating chat', ['error' => $e->getMessage()]);
        return response()->json(['message' => 'Error creating chat!'], 500);
    }
}

// Fetch all chats for the user
public function getUserChats(Request $request)
{
    $userId = $request->user()->id; // Get the currently authenticated user's ID

    // Log user chat fetching
    Log::info('Fetching user chats', ['user_id' => $userId]);

    try {
        $userChats = User::find($userId)->chats;
        Log::info('User chats fetched successfully', ['user_chats_count' => $userChats->count()]);
        return response()->json($userChats, 200);
    } catch (\Exception $e) {
        Log::error('Error fetching user chats', ['error' => $e->getMessage()]);
        return response()->json(['message' => 'Error fetching user chats!'], 500);
    }
}

// Fetch a specific chat by ID
public function getChatById(Request $request, $id)
{
    $userId = $request->user()->id; // Get the currently authenticated user's ID

    Log::info('Fetching chat by ID', ['user_id' => $userId, 'chat_id' => $id]);

    try {
        $chat = ChatBot::where('id', $id)->where('user_id', $userId)->first();

        if (!$chat) {
            Log::warning('Chat not found', ['chat_id' => $id]);
            return response()->json(['message' => 'Chat not found!'], 404);
        }

        Log::info('Chat fetched successfully', ['chat_id' => $id]);
        return response()->json($chat, 200);
    } catch (\Exception $e) {
        Log::error('Error fetching chat', ['error' => $e->getMessage(), 'chat_id' => $id]);
        return response()->json(['message' => 'Error fetching chat!'], 500);
    }
}

// Update a chat's history
public function updateChat(Request $request, $id)
{
    $userId = $request->user()->id; // Get the currently authenticated user's ID
    $question = $request->input('question');
    $answer = $request->input('answer');

    Log::info('Updating chat history', ['user_id' => $userId, 'chat_id' => $id, 'question' => $question, 'answer' => $answer]);

    $newItems = [
        ...(isset($question) ? [
```

```
                [
                    'role' => 'user',
                    'parts' => [['text' => $question]]
                ]
            ] : []),
            [
                'role' => 'model',
                'parts' => [['text' => $answer]]
            ]
        ];

        try {
            $chat = ChatBot::where('id', $id)->where('user_id', $userId)->first();

            if (!$chat) {
                Log::warning('Chat not found', ['chat_id' => $id]);
                return response()->json(['message' => 'Chat not found!'], 404);
            }

            // Update the chat history
            $chat->history = array_merge($chat->history, $newItems);
            $chat->save();

            Log::info('Chat history updated successfully', ['chat_id' => $id]);
            return response()->json(['message' => 'Chat updated successfully!'], 200);
        } catch (\Exception $e) {
            Log::error('Error updating chat history', ['error' => $e->getMessage(), 'chat_id' => $id]);
            return response()->json(['message' => 'Error updating chat!'], 500);
        }
    }
}
```

FILE: app/Http/Controllers/ConnectionRequestController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\ConnectionRequest;
use App\Models\Notification;
use App\Models\User;
use App\Models\UserConnection; // Include the UserConnection model
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;

class ConnectionRequestController extends Controller
{
    public function sendConnectionRequest(Request $request, $userId)
    {
        $senderId = $request->user()->id;
        Log::info('Sender ID sending connection request: ' . $senderId);

        if ($senderId == $userId) {
            return response()->json(['message' => "You can't send a request to yourself"], 400);
        }

        if (UserConnection::where('user_id', $senderId)->where('connection_id', $userId)->exists()) {
            return response()->json(['message' => "You are already connected"], 400);
        }

        $existingRequest = ConnectionRequest::where('sender_id', $senderId)
            ->where('recipient_id', $userId)
            ->where('status', 'pending')
            ->first();
```

```
        if ($existingRequest) {
            return response()->json(['message' => "A connection request already exists"], 400);
        }

        $newRequest = ConnectionRequest::create([
            'sender_id' => $senderId,
            'recipient_id' => $userId,
            'status' => 'pending',
        ]);

        return response()->json(['message' => "Connection request sent successfully"], 201);
    }


    public function acceptConnectionRequest(Request $request, $requestId)
    {
        $userId = $request->user()->id;

        // Log the incoming request and user ID
        Log::info("User {$userId} is attempting to accept connection request {$requestId}");

        $connectionRequest = ConnectionRequest::findOrFail($requestId);

        if ($connectionRequest->recipient_id !== $userId) {
            // Log unauthorized access attempt
            Log::warning("User {$userId} attempted to accept a connection request that was not intended for them (Request ID:
{$requestId})");

            return response()->json(['message' => "Not authorized to accept this request"], 403);
        }

        if ($connectionRequest->status !== 'pending') {
            // Log that the request has already been processed
            Log::info("Connection request {$requestId} has already been processed (status: {$connectionRequest->status}) by user
{$userId}");

            return response()->json(['message' => "This request has already been processed"], 400);
        }

        // Log when the request is being accepted
        Log::info("User {$userId} has accepted connection request {$requestId}");

        $connectionRequest->status = 'accepted';
        $connectionRequest->save();

        // Log creation of connections for both users
        UserConnection::create([
            'user_id' => $connectionRequest->sender_id,
            'connection_id' => $userId,
        ]);
        Log::info("Connection created between user {$connectionRequest->sender_id} and user {$userId}");

        UserConnection::create([
            'user_id' => $userId,
            'connection_id' => $connectionRequest->sender_id,
        ]);
        Log::info("Connection created between user {$userId} and user {$connectionRequest->sender_id}");

        // Log notification creation
        Notification::create([
            'recipient_id' => $connectionRequest->sender_id,
            'type' => 'connectionAccepted',
            'related_user' => $userId,
        ]);
        Log::info("Notification sent to user {$connectionRequest->sender_id} regarding connection acceptance from user {$userId}");
```

```
        return response()->json(['message' => "Connection accepted successfully"]);
    }


    public function rejectConnectionRequest(Request $request, $requestId)
    {
        $userId = $request->user()->id;
        $connectionRequest = ConnectionRequest::findOrFail($requestId);

        if ($connectionRequest->recipient_id !== $userId) {
            return response()->json(['message' => "Not authorized to reject this request"], 403);
        }

        if ($connectionRequest->status !== 'pending') {
            return response()->json(['message' => "This request has already been processed"], 400);
        }

        $connectionRequest->status = 'rejected';
        $connectionRequest->save();

        return response()->json(['message' => "Connection request rejected"]);
    }

    public function getConnectionRequests(Request $request)
    {
        $userId = $request->user()->id;
        Log::info('User ID fetching connection requests: ' . $userId);

        $requests = ConnectionRequest::where('recipient_id', $userId)
            ->where('status', 'pending')
            ->with('sender:id,first_name,last_name,phone_number,email,language,profile_picture,banner_img,headline,about')
            ->get();

        return response()->json($requests);
    }

    public function getUserConnections(Request $request)
    {
        $userId = $request->user()->id;

        // Fetch connections using the UserConnection model
        $connections = UserConnection::where('user_id', $userId)
            ->with('connection:id,first_name,last_name,phone_number,email,language,profile_picture,banner_img,headline,about') // Assuming
you have the relationship set in UserConnection
            ->get();

        return response()->json($connections);
    }

    public function removeConnection(Request $request, $userId)
    {
        $myId = $request->user()->id;

        // Remove connections using the UserConnection model
        UserConnection::where('user_id', $myId)->where('connection_id', $userId)->delete();
        UserConnection::where('user_id', $userId)->where('connection_id', $myId)->delete();

        return response()->json(['message' => "Connection removed successfully"]);
    }

    public function getConnectionStatus(Request $request, $targetUserId)
    {
        $currentUserId = $request->user()->id;

        if (UserConnection::where('user_id', $currentUserId)->where('connection_id', $targetUserId)->exists()) {
```

```
            return response()->json(['status' => 'connected']);
        }

        $pendingRequest = ConnectionRequest::where(function ($query) use ($currentUserId, $targetUserId) {
            $query->where('sender_id', $currentUserId)
                  ->where('recipient_id', $targetUserId);
        })->orWhere(function ($query) use ($currentUserId, $targetUserId) {
            $query->where('sender_id', $targetUserId)
                  ->where('recipient_id', $currentUserId);
        })->where('status', 'pending')->first();

        if ($pendingRequest) {
            return response()->json(['status' => $pendingRequest->sender_id == $currentUserId ? 'pending' : 'received', 'requestId' =>
$pendingRequest->id]);
        }

        return response()->json(['status' => 'not_connected']);
    }
}
```

FILE: app/Http/Controllers/Controller.php

```
<?php

namespace App\Http\Controllers;

abstract class Controller
{
    //
}
```

FILE: app/Http/Controllers/CouponController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Coupon;
use App\Models\ProductCoupon;
use App\Models\UserCoupon;
use App\Models\Order;
use App\Models\OrderItem;
use App\Models\Product;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Log;

class CouponController extends Controller
{
    /**
     * Get the active coupon for the authenticated user.
     */
    public function getMyCoupon(Request $request)
    {
        try {
            $user = $request->user();  // Get the authenticated user
            $productIds = $request->productIds;  // Product IDs from the request
            Log::info('Product Ids', ['product id'=>$productIds]);

            if (empty($productIds)) {
```

```
            return response()->json(['message' => 'No products in the cart.'], 400);
        }

        // Find coupons applicable for the products in the user's cart
        $validCoupons = ProductCoupon::whereIn('products_id', $productIds)
            ->whereIn('coupons_id', function ($query) use ($user) {
                $query->select('coupons_id')
                    ->from('users_coupons')
                    ->where('users_id', $user->id);
            })
            ->get();

        if ($validCoupons->isEmpty()) {
            return response()->json(['message' => 'No valid coupons for the selected products.'], 400);
        }

        // Return the coupons with their details
        $coupons = Coupon::whereIn('id', $validCoupons->pluck('coupons_id'))->get();

        return response()->json($coupons);

    } catch (\Exception $e) {
        return response()->json(['message' => 'Server error', 'error' => $e->getMessage()], 500);
    }
}

/**
 * Validate and apply the coupon for the authenticated user.
 */
public function applyCoupon(Request $request)
{
    try {
        $user = $request->user();  // Get the authenticated user
        $couponId = $request->couponId;  // Coupon code from the request
        $productIds = $request->productIds;  // Product IDs from the request
        $orderId = $request->orderId;  // Order ID (if applicable)

        if (empty($productIds)) {
            return response()->json(['message' => 'No products in the cart.'], 400);
        }

        // Validate if the coupon exists
        $coupon = Coupon::where('id', $couponId)->first();

        if (!$coupon) {
            return response()->json(['message' => 'Coupon code is invalid.'], 400);
        }

        // Check if the coupon is associated with the products in the cart
        $productCoupon = ProductCoupon::whereIn('products_id', $productIds)
            ->where('coupons_id', $coupon->id)
            ->first();

        if (!$productCoupon) {
            return response()->json(['message' => 'Coupon is not valid for the selected products.'], 400);
        }

        // Now, let's apply the coupon to the user's cart and calculate the total amount
        $order = Order::find($orderId);

        if (!$order || $order->user_id !== $user->id) {
            return response()->json(['message' => 'Order not found or unauthorized.'], 400);
        }

        // Get the order items for the products in the cart
        $orderItems = OrderItem::where('orders_id', $order->id)->whereIn('products_id', $productIds)->with('product')->get();
```

```
            if ($orderItems->isEmpty()) {
                return response()->json(['message' => 'No matching products found in the order.'], 400);
            }

            // Calculate the total amount
            $totalAmount = $orderItems->sum(function ($item) {
                return $item->total_amount;  // Assuming each order item has a total_amount field
            });

            // Calculate the discount based on coupon type
            $discountAmount = 0;
            $discountAmount = ($totalAmount * $coupon->discount) / 100;

            $totalAfterDiscount = max(0, $totalAmount - $discountAmount);

            UserCoupon::where('users_id', $user->id)
                ->where('coupons_id', $coupon->id)
                ->delete(); // Delete the coupon from the user's active coupons

            // Return the updated order details with discount applied
            return response()->json([
                'order_id' => $order->id,
                'total_amount' => $totalAmount,
                'discount_amount' => $discountAmount,
                'coupon' => $coupon,
                'total_after_discount' => $totalAfterDiscount,
                'order_items' => $orderItems,
            ]);

        } catch (\Exception $e) {
            return response()->json(['message' => 'Server error', 'error' => $e->getMessage()], 500);
        }
    }

    /**
     * Store a new coupon, check if any product has the same discount,
     * and assign the coupon to the products and the authenticated user.
     */
    public function create(Request $request)
    {
        // Validate the incoming request data
        $validated = $request->validate([
            'discount' => 'required|numeric',
            'expiration_date' => 'required|date',
        ]);

        Log::info('Creating new coupon', ['discount' => $validated['discount'], 'expiration_date' => $validated['expiration_date']]);

        // Check if a product already has a coupon with the same discount
        $existingProducts = Product::whereHas('coupons', function ($query) use ($validated) {
            $query->where('discount', $validated['discount']);
        })->get();

        if ($existingProducts->isNotEmpty()) {
            Log::info('Found products with the same discount', ['discount' => $validated['discount'], 'products' => $existingProducts-
>pluck('id')]);
        }

        // Start a database transaction to ensure atomicity
        DB::beginTransaction(); // Start transaction for atomic operations

        try {
            // Create the new coupon
            $coupon = Coupon::create([
                'discount' => $validated['discount'],
```

```
                'expiration_date' => $validated['expiration_date'],
            ]);

            Log::info('Coupon created successfully', ['coupon_id' => $coupon->id]);

            // If any products already have the same discount, assign the new coupon to them
            foreach ($existingProducts as $product) {
                $product->coupons()->attach($coupon->id);
                Log::info('Coupon assigned to product', ['coupon_id' => $coupon->id, 'product_id' => $product->id]);
            }

            // Assign the coupon to the authenticated user (use request()->user()->id)
            $userId = $request->user()->id;  // Get the authenticated user's ID
            $coupon->users()->attach($userId);

            Log::info('Coupon assigned to user', ['coupon_id' => $coupon->id, 'user_id' => $userId]);

            DB::commit(); // Commit the transaction

            return response()->json($coupon, 201);
        } catch (\Exception $e) {
            DB::rollBack(); // Rollback transaction on error

            // Log the error
            Log::error('Error creating coupon', [
                'error' => $e->getMessage(),
                'stack_trace' => $e->getTraceAsString()
            ]);

            return response()->json([
                'message' => 'There was an error creating the coupon.',
                'error' => $e->getMessage(),
            ], 500);
        }
    }
    /**
     * Get the coupons associated with the authenticated user.
     */
    public function getAllCoupons(Request $request)
    {
        // Get the authenticated user using the request()->user() helper
        $user = $request->user();

        // Log the action for tracking purposes
        Log::info('Fetching coupons for user', ['user_id' => $user->id]);

        // Fetch the user's coupons
        $coupons = $user->coupons;  // Assuming the relationship 'coupons' is defined in the User model

        if ($coupons->isEmpty()) {
            Log::info('No coupons found for user', ['user_id' => $user->id]);
            return response()->json([
                'message' => 'You have no coupons.',
                'coupons' => [],
            ], 200);
        }

        Log::info('Coupons found for user', ['user_id' => $user->id, 'coupons' => $coupons->pluck('id')]);

        // Return the coupons in a response
        return response()->json($coupons, 200);
    }
}
```

FILE: app/Http/Controllers/DeliveryController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Shipment;
use App\Models\RouteOptimization;
use App\Models\RouteDetail;
use App\Models\RouteCondition;
use App\Models\Vehicle;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log; // Import the Log facade
use Carbon\Carbon; // For measuring the time taken
use Illuminate\Support\Facades\Validator;
use App\Events\OrderStatusUpdated;

class DeliveryController extends Controller
{
    /**
     * Get all shipments with related route optimizations, route details, and route conditions using where clauses,
     * but only include shipments with non-empty route optimizations.
     *
     * @return \Illuminate\Http\Response
     */
    public function getShipmentsWithRouteDetails()
    {
        // Log the beginning of the request with the timestamp
        Log::info('Fetching shipments with route details and optimizations started at: ' . Carbon::now()->toDateTimeString());

        // Record the start time for performance tracking
        $startTime = microtime(true);

        try {
            // Get all shipments
            $shipments = Shipment::all();

            // Log the number of shipments retrieved
            Log::info('Number of shipments retrieved: ' . $shipments->count());

            // Initialize an array to hold the data with relationships
            $shipmentsWithDetails = [];

            foreach ($shipments as $shipment) {
                // Fetch related route optimizations for the shipment using where clause
                $routeOptimizations = RouteOptimization::where('shipments_id', $shipment->id)->get();

                // Check if there are any related route optimizations for this shipment
                if ($routeOptimizations->isEmpty()) {
                    // Skip shipments that have no route optimizations
                    continue;
                }

                $routeOptimizationsWithDetails = [];

                foreach ($routeOptimizations as $routeOptimization) {
                    // Fetch related route details using where clause
                    $routeDetails = RouteDetail::where('route_optimization_id', $routeOptimization->id)->get();

                    $routeDetailsWithConditions = [];

                    foreach ($routeDetails as $routeDetail) {
                        // Fetch the related route condition using where clause
                        $routeCondition = RouteCondition::where('id', $routeDetail->route_condition_id)->first();

                        // Add route details along with route condition to the list
                        $routeDetailsWithConditions[] = [
```

```
                        'routeDetail' => $routeDetail,
                        'routeCondition' => $routeCondition
                    ];
                }

                // Add the route details and conditions to the route optimization
                $routeOptimizationsWithDetails[] = [
                    'routeOptimization' => $routeOptimization,
                    'routeDetails' => $routeDetailsWithConditions
                ];
            }

            // Add the shipment with its related route optimizations and route details
            $shipmentsWithDetails[] = [
                'shipment' => $shipment,
                'routeOptimizations' => $routeOptimizationsWithDetails
            ];
        }

        // Log the shipment details (warning: this can be large data)
        // Option 1: Log the entire shipments with relationships
        Log::info('Shipments data with relationships: ' . json_encode($shipmentsWithDetails));

        // Option 2: Log only a summary of the shipments (e.g., just ids or key attributes)
        // Log::info('Sample shipment data: ' . json_encode(array_column($shipmentsWithDetails, 'shipment.id')));

        // Log the success message and the time taken to fetch the shipments
        $endTime = microtime(true);
        $executionTime = round($endTime - $startTime, 2); // in seconds
        Log::info("Shipments fetched successfully. Time taken: {$executionTime} seconds.");

        // Return the shipments data with their relationships
        return response()->json($shipmentsWithDetails);

    } catch (\Exception $e) {
        // Log any errors that occur
        Log::error('Error fetching shipments: ' . $e->getMessage());

        // Log the full stack trace of the error for debugging
        Log::error('Error stack trace: ' . $e->getTraceAsString());

        // Return an error response
        return response()->json(['error' => 'An error occurred while fetching shipments.'], 500);
    }
}

/**
 * Assign a vehicle to a route detail.
 *
 * @param  \Illuminate\Http\Request  $request
 * @param  int  $routeDetailId
 * @return \Illuminate\Http\Response
 */
public function assignVehicle(Request $request, $routeDetailId)
{
    // Validate the incoming request
    $validator = Validator::make($request->all(), [
        'vehicle_id' => 'required|exists:vehicles,id', // Ensure the vehicle exists
    ]);

    if ($validator->fails()) {
        return response()->json([
            'status' => 'error',
            'message' => $validator->errors(),
        ], 422);
    }
```

```
    // Find the RouteDetail by ID
    $routeDetail = RouteDetail::find($routeDetailId);

    if (!$routeDetail) {
        return response()->json([
            'status' => 'error',
            'message' => 'Route detail not found.',
        ], 404);
    }

    // Check if the route detail already has a vehicle assigned
    if ($routeDetail->vehicle_id && $routeDetail->vehicle_id != $request->vehicle_id) {
        return response()->json([
            'status' => 'error',
            'message' => 'This route detail already has a vehicle assigned.',
        ], 400);
    }

    // Assign the vehicle to the route detail
    $routeDetail->vehicle_id = $request->vehicle_id;
    $routeDetail->save();

    // Check if all RouteDetails for the associated RouteOptimization have a vehicle assigned
    $routeOptimization = $routeDetail->routeOptimization; // Get the associated RouteOptimization
    $allRouteDetailsHaveVehicle = $routeOptimization->routeDetails->every(function ($detail) {
        return !is_null($detail->vehicle_id); // Check if vehicle_id is not null for each RouteDetail
    });

    if ($allRouteDetailsHaveVehicle) {
        // If all RouteDetails have a vehicle assigned, update the Order status
        $shipment = $routeOptimization->shipment; // Get the associated Shipment
        $order = $shipment->order; // Get the associated Order

        // Update the Order status to 'on delivery'
        $order->status = 'On Delivery';
        $order->save();
        event(new OrderStatusUpdated($order));
    }

    return response()->json([
        'status' => 'success',
        'message' => 'Vehicle assigned successfully.',
        'data' => $routeDetail,
    ]);
}


/**
 * Get the vehicle for the authenticated user.
 *
 * @return \Illuminate\Http\Response
 */
public function getVehicleForAuthenticatedUser(Request $request)
{
    // Ensure the user is authenticated
    $user = $request->user();

    // Check if the user is authenticated
    if (!$user) {
        return response()->json([
            'status' => 'error',
            'message' => 'Unauthenticated user.',
        ], 401);
    }
```

```
        // Fetch the vehicle associated with the authenticated user (driver_id).
        $vehicle = Vehicle::where('driver_id', $user->id)->first();

        if (!$vehicle) {
            return response()->json([
                'status' => 'error',
                'message' => 'No vehicle found for this user.',
            ], 404);
        }

        // Return the vehicle details in the response
        return response()->json([
            'status' => 'success',
            'vehicle' => $vehicle,
        ], 200);
    }
}
```

FILE: app/Http/Controllers/EditorController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Log;
use App\Integrations\files\CloudinaryImageClient; // Use the custom Cloudinary client
use App\Integrations\files\CloudinaryVideoClient; // Use the custom Cloudinary client
use Exception;
class EditorController extends Controller
{

    protected $cloudinaryClient;
    protected $cloudinaryVideoClient;


    public function __construct()
    {
        // Initialize custom Cloudinary image client
        $this->cloudinaryClient = new CloudinaryImageClient();

        $this->cloudinaryVideoClient = new CloudinaryVideoClient();

    }

    /**
     * Upload an image to Cloudinary.
     *
     * @param \Illuminate\Http\Request $request
     * @return \Illuminate\Http\JsonResponse
     */
    public function uploadImage(Request $request)
    {
        // Validate the incoming request data
        $validatedData = $request->validate([
            'image' => 'required|file|mimes:jpg,jpeg,png,gif,webp',
        ]);

        if ($request->hasFile('image')) {
            try {
                // Retrieve the image file
                $uploadedFile = $request->file('image');
                $mimeType = $uploadedFile->getClientMimeType();
```

```
            $base64Contents = base64_encode($uploadedFile->getContent());

            // Upload image using custom Cloudinary client
            $uploadResult = $this->cloudinaryClient->uploadImage($mimeType, $base64Contents);

            // If the upload is successful, return the response
            if ($uploadResult->isSuccess) {
                Log::info('result', [
                    'result' => $uploadResult,
                ]);
                return response()->json(['success' => $uploadResult],200);

            } else {
                throw new \Exception("Cloudinary image upload failed: " . $uploadResult->msg);
            }
        } catch (\Exception $e) {
            Log::error('Error uploading image to Cloudinary', [
                'error_message' => $e->getMessage(),
                'image' => $uploadedFile->getClientOriginalName()
            ]);
            return response()->json(['error' => 'Image upload failed'], 500);
        }
    }

    return response()->json(['error' => 'No image file provided'], 400);
}

/**
 * Upload a video to Cloudinary.
 *
 * @param \Illuminate\Http\Request $request
 * @return \Illuminate\Http\JsonResponse
 */


public function uploadVideo(Request $request)
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'video' => 'required|file|mimes:mp4,mkv,avi,webm|max:2048000', // max 2GB
    ]);

    if ($request->hasFile('video')) {
        $file = $validatedData['video'];

        $uploadResult = $this->cloudinaryVideoClient->uploadVideo($file);

        if ($uploadResult->isSuccess) {
            return response()->json(['success' => $uploadResult], 200);
        } else {
            return response()->json(['error' => $uploadResult->msg], 500);
        }
    }

    return response()->json(['error' => 'No video file provided'], 400);
}



/**
 * Remove the background from an image via Cloudinary.
 *
 * @param  \Illuminate\Http\Request  $request
 * @return \Illuminate\Http\JsonResponse
 */
public function removeBackground(Request $request)
```

```
    {
        // Validate the incoming request data
        $validatedData = $request->validate([
            'activeImage' => 'required|string',
            'format' => 'required|string',
        ]);

        $activeImage = $validatedData['activeImage'];
        $format = $validatedData['format'];

        // Log incoming request details
        Log::info('Incoming request to remove background', [
            'activeImage' => $activeImage,
            'format' => $format,
        ]);

        // Cloudinary configuration from .env file
        $cloudName = env('CLOUDINARY_CLOUD_NAME');
        $apiKey = env('CLOUDINARY_API_KEY');
        $apiSecret = env('CLOUDINARY_API_SECRET');

        // Generate the URL for background removal
        $form = explode($format, $activeImage);
        $pngConvert = $form[0] . "png"; // Convert image to PNG format
        $parts = explode("/upload/", $pngConvert);
        $removeUrl = $parts[0] . "/upload/e_background_removal/" . $parts[1];

        // Log the generated remove URL
        Log::info('Generated remove URL', ['removeUrl' => $removeUrl]);

        // Poll the URL to check if the image is processed
        $maxAttempts = 20;
        $delay = 1; // Delay between each attempt in seconds
        $isProcessed = false;

        try {
            for ($attempt = 0; $attempt < $maxAttempts; $attempt++) {
                // Log each attempt
                Log::info('Attempt ' . ($attempt + 1) . ': Checking image processing for URL: ' . $removeUrl);

                // Check if the image is processed
                $response = Http::get($removeUrl);

                if ($response->ok()) {
                    $isProcessed = true;
                    Log::info("Image processed successfully for URL: {$removeUrl}");
                    break;
                }

                // Delay before rechecking
                sleep($delay);
            }

            if (!$isProcessed) {
                // Log the timeout and return a response
                Log::error('Image processing timed out', [
                    'removeUrl' => $removeUrl,
                    'attempts' => $maxAttempts,
                ]);
                return response()->json([
                    'error' => 'Image processing timed out.',
                ], 408); // HTTP 408 Request Timeout
            }

            // Return the processed image URL
            Log::info('Returning processed image URL', ['removeUrl' => $removeUrl]);
```

```
        return response()->json([
            'success' => $removeUrl,
        ], 200);

    } catch (Exception $e) {
        // Log the exception with full details
        Log::error('Error during background removal process', [
            'exception' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
            'removeUrl' => $removeUrl,
        ]);

        // Return a generic error response
        return response()->json([
            'error' => 'An error occurred while processing the image.',
        ], 500); // HTTP 500 Internal Server Error
    }
}

/**
 * Replace the background of an image via Cloudinary.
 *
 * @param  \Illuminate\Http\Request  $request
 * @return \Illuminate\Http\JsonResponse
 */
public function replaceBackground(Request $request)
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'activeImage' => 'required|string',
        'prompt' => 'nullable|string',
    ]);

    $activeImage = $validatedData['activeImage'];
    $prompt = $validatedData['prompt'];

    // Log incoming request details
    Log::info('Incoming request to replace background', [
        'activeImage' => $activeImage,
        'prompt' => $prompt,
    ]);

    // Generate the URL for background replacement
    $parts = explode("/upload/", $activeImage);
    $bgReplaceUrl = $prompt
        ? "{$parts[0]}/upload/e_gen_background_replace:prompt_" . urlencode($prompt) . "/{$parts[1]}"
        : "{$parts[0]}/upload/e_gen_background_replace/{$parts[1]}";

    // Log the generated replace URL
    Log::info('Generated background replace URL', ['bgReplaceUrl' => $bgReplaceUrl]);

    // Poll the URL to check if the image is processed
    $maxAttempts = 20;
    $delay = 1; // Delay between each attempt in seconds
    $isProcessed = false;

    try {
        for ($attempt = 0; $attempt < $maxAttempts; $attempt++) {
            // Log each attempt
            Log::info('Attempt ' . ($attempt + 1) . ': Checking image processing for URL: ' . $bgReplaceUrl);

            // Check if the image is processed
            $response = Http::get($bgReplaceUrl);

            if ($response->ok()) {
                $isProcessed = true;
```

```
                Log::info("Image processed successfully for URL: {$bgReplaceUrl}");
                break;
            }

            // Delay before rechecking
            sleep($delay);
        }

        if (!$isProcessed) {
            // Log the timeout and return a response
            Log::error('Image processing timed out', [
                'bgReplaceUrl' => $bgReplaceUrl,
                'attempts' => $maxAttempts,
            ]);
            return response()->json([
                'error' => 'Image processing timed out.',
            ], 408); // HTTP 408 Request Timeout
        }

        // Return the processed image URL
        Log::info('Returning processed image URL', ['bgReplaceUrl' => $bgReplaceUrl]);
        return response()->json([
            'success' => $bgReplaceUrl,
        ], 200);

    } catch (Exception $e) {
        // Log the exception with full details
        Log::error('Error during background replacement process', [
            'exception' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
            'bgReplaceUrl' => $bgReplaceUrl,
        ]);

        // Return a generic error response
        return response()->json([
            'error' => 'An error occurred while processing the image.',
        ], 500); // HTTP 500 Internal Server Error
    }
}

/**
 * Extract objects from an image via Cloudinary.
 *
 * @param  \Illuminate\Http\Request  $request
 * @return \Illuminate\Http\JsonResponse
 */
public function extractImage(Request $request)
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'prompts' => 'required|array',
        'prompts.*' => 'required|string',
        'activeImage' => 'required|string',
        'multiple' => 'nullable|boolean',
        'mode' => 'nullable|in:default,mask',
        'invert' => 'nullable|boolean',
        'format' => 'required|string',
    ]);

    $prompts = $validatedData['prompts'];
    $activeImage = $validatedData['activeImage'];
    $multiple = $validatedData['multiple'] ?? false;
    $mode = $validatedData['mode'] ?? 'default';
    $invert = $validatedData['invert'] ?? false;
    $format = $validatedData['format'];
```

```
        // Log incoming request details
        Log::info('Incoming request to extract image', [
            'activeImage' => $activeImage,
            'prompts' => $prompts,
            'multiple' => $multiple,
            'mode' => $mode,
            'invert' => $invert,
            'format' => $format,
        ]);

        // Generate the URL for extraction
        $form = explode($format, $activeImage);
        $pngConvert = $form[0] . "png"; // Convert image to PNG format
        $parts = explode("/upload/", $pngConvert);

        // Build the extract parameters string
        $extractParams = 'prompt_(' . implode(';', array_map('urlencode', $prompts)) . ')';
        if ($multiple) $extractParams .= ';multiple_true';
        if ($mode === 'mask') $extractParams .= ';mode_mask';
        if ($invert) $extractParams .= ';invert_true';

        $extractUrl = $parts[0] . '/upload/e_extract:' . $extractParams . '/' . $parts[1];

        // Log the generated extract URL
        Log::info('Generated extract URL', ['extractUrl' => $extractUrl]);

        // Poll the URL to check if the image is processed
        $maxAttempts = 20;
        $delay = 1; // Delay between each attempt in seconds
        $isProcessed = false;

        try {
            for ($attempt = 0; $attempt < $maxAttempts; $attempt++) {
                // Log each attempt
                Log::info('Attempt ' . ($attempt + 1) . ': Checking image processing for URL: ' . $extractUrl);


                // Check if the image is processed
                $response = Http::get($extractUrl);

                if ($response->ok()) {
                    $isProcessed = true;
                    Log::info("Image processed successfully for URL: {$extractUrl}");
                    break;
                }

                // Delay before rechecking
                sleep($delay);
            }

            if (!$isProcessed) {
                // Log the timeout and return a response
                Log::error('Image processing timed out', [
                    'extractUrl' => $extractUrl,
                    'attempts' => $maxAttempts,
                ]);
                return response()->json([
                    'error' => 'Image processing timed out.',
                ], 408); // HTTP 408 Request Timeout
            }

            // Return the processed image URL
            Log::info('Returning processed image URL', ['extractUrl' => $extractUrl]);
            return response()->json([
                'success' => $extractUrl,
            ], 200);
```

```
    } catch (Exception $e) {
        // Log the exception with full details
        Log::error('Error during image extraction process', [
            'exception' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
            'extractUrl' => $extractUrl,
        ]);

        // Return a generic error response
        return response()->json([
            'error' => 'An error occurred while processing the image.',
        ], 500); // HTTP 500 Internal Server Error
    }
}

/**
 * Generate the fill transformation URL and check if the image is processed.
 *
 * @param  \Illuminate\Http\Request  $request
 * @return \Illuminate\Http\JsonResponse
 */
public function genFill(Request $request)
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'activeImage' => 'required|string',
        'aspect' => 'required|string',
        'width' => 'required|string',
        'height' => 'required|string',
    ]);

    $activeImage = $validatedData['activeImage'];
    $aspect = $validatedData['aspect'];
    $width = $validatedData['width'];
    $height = $validatedData['height'];

    // Log incoming request data for debugging
    Log::info('Incoming request to generate fill URL', [
        'activeImage' => $activeImage,
        'aspect' => $aspect,
        'width' => $width,
        'height' => $height,
    ]);

    // Generate the fill transformation URL
    $parts = explode('/upload/', $activeImage);
    $fillUrl = "{$parts[0]}/upload/ar_{$aspect},b_gen_fill,c_pad,w_{$width},h_{$height}/{$parts[1]}";

    // Log the generated fill URL
    Log::info('Generated fill URL', ['fillUrl' => $fillUrl]);

    // Poll the URL to check if the image is processed
    $maxAttempts = 20;
    $delay = 1; // Delay between each attempt in seconds
    $isProcessed = false;

    try {
        for ($attempt = 0; $attempt < $maxAttempts; $attempt++) {
            // Log each attempt for debugging
            Log::info('Attempt ' . ($attempt + 1) . ': Checking image processing for URL: ' . $fillUrl);


            // Check if the image is processed
            $response = Http::get($fillUrl);
```

```
            if ($response->ok()) {
                $isProcessed = true;
                Log::info("Image processed successfully for URL: {$fillUrl}");
                break;
            }

            // Delay before rechecking
            sleep($delay);
        }

        if (!$isProcessed) {
            // Log the timeout and return a response
            Log::error('Image processing failed', [
                'fillUrl' => $fillUrl,
                'attempts' => $maxAttempts,
            ]);
            return response()->json([
                'error' => 'Image processing failed.',
            ], 408); // HTTP 408 Request Timeout
        }

        // Return the processed image URL
        Log::info('Returning processed image URL', ['fillUrl' => $fillUrl]);
        return response()->json([
            'success' => $fillUrl,
        ], 200);

    } catch (Exception $e) {
        // Log the exception with full details
        Log::error('Error during image processing', [
            'exception' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
            'fillUrl' => $fillUrl,
        ]);

        // Return a generic error response
        return response()->json([
            'error' => 'An error occurred while processing the image.',
        ], 500); // HTTP 500 Internal Server Error
    }
}

/**
 * Remove the background from an image via Cloudinary using a custom prompt.
 *
 * @param  \Illuminate\Http\Request  $request
 * @return \Illuminate\Http\JsonResponse
 */
public function genRemove(Request $request)
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'prompt' => 'required|string',
        'activeImage' => 'required|string',
    ]);

    $prompt = $validatedData['prompt'];
    $activeImage = $validatedData['activeImage'];

    // Log incoming request data for debugging
    Log::info('Incoming request for image background removal', [
        'prompt' => $prompt,
        'activeImage' => $activeImage,
    ]);

    // Generate the URL for image background removal
```

```
        $parts = explode("/upload/", $activeImage);
        $removeUrl = "{$parts[0]}/upload/e_gen_remove:{$prompt}/{$parts[1]}";

        // Log the generated remove URL
        Log::info('Generated remove URL', ['removeUrl' => $removeUrl]);

        // Poll the URL to check if the image is processed
        $maxAttempts = 20;
        $delay = 1; // Delay between each attempt in seconds
        $isProcessed = false;

        try {
            for ($attempt = 0; $attempt < $maxAttempts; $attempt++) {
                // Log each attempt for debugging
                Log::info('Attempt ' . ($attempt + 1) . ': Checking image processing for URL: ' . $removeUrl);

                // Check if the image is processed
                $response = Http::get($removeUrl);

                if ($response->ok()) {
                    $isProcessed = true;
                    Log::info("Image processed successfully for URL: {$removeUrl}");
                    break;
                }

                // Delay before rechecking
                sleep($delay);
            }

            if (!$isProcessed) {
                // Log the timeout and return a response
                Log::error('Image processing failed', [
                    'removeUrl' => $removeUrl,
                    'attempts' => $maxAttempts,
                ]);
                return response()->json([
                    'error' => 'Image processing failed.',
                ], 408); // HTTP 408 Request Timeout
            }

            // Return the processed image URL
            Log::info('Returning processed image URL', ['removeUrl' => $removeUrl]);
            return response()->json([
                'success' => $removeUrl,
            ], 200);

        } catch (Exception $e) {
            // Log the exception with full details
            Log::error('Error during image processing', [
                'exception' => $e->getMessage(),
                'stack' => $e->getTraceAsString(),
                'removeUrl' => $removeUrl,
            ]);

            // Return a generic error response
            return response()->json([
                'error' => 'An error occurred while processing the image.',
            ], 500); // HTTP 500 Internal Server Error
        }
    }

    /**
     * Recolor an image via Cloudinary.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return \Illuminate\Http\JsonResponse
```

```
     */
    public function recolorImage(Request $request)
    {
        // Validate the incoming request data
        $validatedData = $request->validate([
            'tag' => 'required|string',
            'color' => 'required|string',
            'activeImage' => 'required|string',
        ]);

        $tag = $validatedData['tag'];
        $color = $validatedData['color'];
        $activeImage = $validatedData['activeImage'];

        // Log the incoming request for debugging
        Log::info('Incoming request for image recoloring', [
            'tag' => $tag,
            'color' => $color,
            'activeImage' => $activeImage,
        ]);

        // Generate the URL for image recoloring
        $parts = explode("/upload/", $activeImage);
        $recolorUrl = "{$parts[0]}/upload/e_gen_recolor:{$tag};{$color}/{$parts[1]}";

        // Log the generated recolor URL
        Log::info('Generated recolor URL', ['recolorUrl' => $recolorUrl]);

        // Poll the URL to check if the image is processed
        $maxAttempts = 20;
        $delay = 1; // Delay between each attempt in seconds
        $isProcessed = false;

        try {
            for ($attempt = 0; $attempt < $maxAttempts; $attempt++) {
                // Log each attempt for debugging
                Log::info('Attempt ' . ($attempt + 1) . ': Checking image processing for URL: ' . $recolorUrl);

                // Check if the image is processed
                $response = Http::get($recolorUrl);

                if ($response->ok()) {
                    $isProcessed = true;
                    Log::info("Image processed successfully for URL: {$recolorUrl}");
                    break;
                }

                // Delay before rechecking
                sleep($delay);
            }

            if (!$isProcessed) {
                // Log the timeout and return a response
                Log::error('Image processing failed', [
                    'recolorUrl' => $recolorUrl,
                    'attempts' => $maxAttempts,
                ]);
                return response()->json([
                    'error' => 'Image processing failed.',
                ], 408); // HTTP 408 Request Timeout
            }

            // Return the processed image URL
            Log::info('Returning processed image URL', ['recolorUrl' => $recolorUrl]);
            return response()->json([
                'success' => $recolorUrl,
```

```
        ], 200);

    } catch (Exception $e) {
        // Log the exception with full details
        Log::error('Error during image processing', [
            'exception' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
            'recolorUrl' => $recolorUrl,
        ]);

        // Return a generic error response
        return response()->json([
            'error' => 'An error occurred while processing the image.',
        ], 500); // HTTP 500 Internal Server Error
    }
}

/**
 * Crop a video via Cloudinary.
 *
 * @param  \Illuminate\Http\Request  $request
 * @return \Illuminate\Http\JsonResponse
 */
public function cropVideo(Request $request)
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'activeVideo' => 'required|string',
        'aspect' => 'required|string',
        'height' => 'required|string',
    ]);

    $activeVideo = $validatedData['activeVideo'];
    $aspect = $validatedData['aspect'];
    $height = $validatedData['height'];

    // Log the incoming request for debugging
    Log::info('Incoming request for video cropping', [
        'activeVideo' => $activeVideo,
        'aspect' => $aspect,
        'height' => $height,
    ]);

    // Generate the URL for video cropping
    $parts = explode("/upload/", $activeVideo);
    $fillUrl = "{$parts[0]}/upload/ar_{$aspect},c_fill,g_auto,h_{$height}/{$parts[1]}";

    // Log the generated crop URL
    Log::info('Generated video crop URL', ['fillUrl' => $fillUrl]);

    // Poll the URL to check if the video is processed
    $maxAttempts = 20;
    $delay = 1; // Delay between each attempt in seconds
    $isProcessed = false;

    try {
        for ($attempt = 0; $attempt < $maxAttempts; $attempt++) {
            // Log each attempt for debugging
            Log::info('Attempt ' . ($attempt + 1) . ': Checking image processing for URL: ' . $fillUrl);

            // Check if the video is processed
            $response = Http::get($fillUrl);

            if ($response->ok()) {
                $isProcessed = true;
                Log::info("Video processed successfully for URL: {$fillUrl}");
```

```
                break;
            }

            // Delay before rechecking
            sleep($delay);
        }

        if (!$isProcessed) {
            // Log the timeout and return a response
            Log::error('Video processing failed', [
                'fillUrl' => $fillUrl,
                'attempts' => $maxAttempts,
            ]);
            return response()->json([
                'error' => 'Video processing failed.',
            ], 408); // HTTP 408 Request Timeout
        }

        // Return the processed video URL
        Log::info('Returning processed video URL', ['fillUrl' => $fillUrl]);
        return response()->json([
            'success' => $fillUrl,
        ], 200);

    } catch (Exception $e) {
        // Log the exception with full details
        Log::error('Error during video processing', [
            'exception' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
            'fillUrl' => $fillUrl,
        ]);

        // Return a generic error response
        return response()->json([
            'error' => 'An error occurred while processing the video.',
        ], 500); // HTTP 500 Internal Server Error
    }
}

/**
 * Initiate transcription for a video and check the transcription status.
 *
 * @param  \Illuminate\Http\Request  $request
 * @return \Illuminate\Http\JsonResponse
 */
public function initiateTranscription(Request $request)
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'publicId' => 'required|string',
    ]);

    $publicId = $validatedData['publicId'];

    // Cloudinary configuration from .env file
    $cloudName = env('CLOUDINARY_CLOUD_NAME');
    $apiKey = env('CLOUDINARY_API_KEY');
    $apiSecret = env('CLOUDINARY_API_SECRET');

    // Log the request for debugging
    Log::info('Initiating transcription for video', ['publicId' => $publicId]);

    try {
        // Initiate transcription via Cloudinary API
        $response = Http::withBasicAuth($apiKey, $apiSecret)->post(
            "https://api.cloudinary.com/v1_1/{$cloudName}/video/upload/{$publicId}",
```

```
            [
                'resource_type' => 'video',
                'raw_convert' => 'google_speech',
            ]
        );

        if (!$response->successful()) {
            return response()->json(['error' => 'Failed to initiate transcription.'], 500);
        }

        // Poll for transcription status
        $maxAttempts = 20;
        $delay = 2000; // 2 seconds delay
        $status = 'pending';

        for ($attempt = 0; $attempt < $maxAttempts; $attempt++) {
            $status = $this->checkTranscriptionStatus($publicId);

            Log::info('Attempt ' . ($attempt + 1) . ': Transcription status - ' . $status);

            if ($status === 'complete') {
                $subtitledVideoUrl = $this->generateSubtitledVideoUrl($publicId);
                return response()->json([
                    'success' => 'Transcription completed',
                    'subtitledVideoUrl' => $subtitledVideoUrl,
                ]);
            } elseif ($status === 'failed') {
                return response()->json(['error' => 'Transcription failed'], 500);
            }

            // Delay before checking again
            usleep($delay * 1000); // Convert delay to microseconds
        }

        return response()->json(['error' => 'Transcription timed out'], 408);
    } catch (Exception $e) {
        Log::error('Error in transcription process', ['exception' => $e->getMessage()]);
        return response()->json(['error' => 'Error in transcription process: ' . $e->getMessage()], 500);
    }
}

/**
 * Check the status of transcription.
 *
 * @param string $publicId
 * @return string
 */
private function checkTranscriptionStatus(string $publicId): string
{
    try {
        $response = Http::withBasicAuth(env('CLOUDINARY_API_KEY'), env('CLOUDINARY_API_SECRET'))
            ->get("https://api.cloudinary.com/v1_1/" . env('CLOUDINARY_CLOUD_NAME') . "/resources/video/{$publicId}");

        if ($response->successful()) {
            $result = $response->json();
            if (isset($result['info']['raw_convert']['google_speech'])) {
                return $result['info']['raw_convert']['google_speech']['status'];
            }
        }
        return 'pending'; // Default if status is not available
    } catch (Exception $e) {
        Log::error('Error checking transcription status', ['exception' => $e->getMessage()]);
        return 'pending'; // Assume pending if an error occurs
    }
}
```

```
    /**
     * Generate the URL for the subtitled video.
     *
     * @param string $publicId
     * @return string
     */
    private function generateSubtitledVideoUrl(string $publicId): string
    {
        return cloudinary_url($publicId, [
            'resource_type' => 'video',
            'transformation' => [
                [
                    'overlay' => [
                        'resource_type' => 'subtitles',
                        'public_id' => "{$publicId}.transcript",
                    ],
                ],
                ['flags' => 'layer_apply'],
            ],
        ]);
    }
}
```

FILE: app/Http/Controllers/FeedbackFormController.php

```
<?php

namespace App\Http\Controllers;
use Illuminate\Support\Facades\Log; // Ensure you include this at the top of your file
use App\Enums\QuestionTypeEnum;
use App\Http\Requests\StoreFeedbackFormAnswerRequest;
use App\Http\Resources\FeedbackFormResource;
use App\Models\FeedbackForm;
use App\Models\Order;
use App\Http\Requests\StoreFeedbackFormRequest;
use App\Http\Requests\UpdateFeedbackFormRequest;
use App\Models\FeedbackFormAnswer;
use App\Models\FeedbackFormQuestion;
use App\Models\FeedbackFormQuestionAnswer;
use Illuminate\Support\Arr;
use Illuminate\Support\Facades\Validator;
use Illuminate\Support\Str;
use Illuminate\Validation\Rule;
use Symfony\Component\HttpFoundation\Request;

class FeedbackFormController extends Controller
{

    public function getOrderWithFeedbackForms($orderId)
    {
        // Fetch all feedback forms associated with the order
        $feedbackForms = FeedbackForm::where('order_id', $orderId)->get(); // Use get() to retrieve all feedback forms
        Log::info('Feedback Forms Fetched', ['feedback forms' => $feedbackForms->toArray()]);

        if ($feedbackForms->isEmpty()) {
            return response()->json([
                'message' => 'No feedback forms found for this order'
            ], 404);
        }

        // Return the order details along with the feedback forms as a resource collection
        return response()->json([
            'feedback_forms' => FeedbackFormResource::collection($feedbackForms) // Return a collection of feedback forms
        ]);
```

```
    }


    /**
     * Store a newly created resource in storage.
     *
     * @param  \App\Http\Requests\StoreFeedbackFormRequest  $request
     * @return \Illuminate\Http\Response
     */
    public function storeFeedbackForm(StoreFeedbackFormRequest $request)
    {
        $data = $request->validated();

        $feedbackForm = FeedbackForm::create($data);

        // Create new questions
        foreach ($data['questions'] as $question) {
            $question['feedback_form_id'] = $feedbackForm->id;
            $this->createQuestion($question);
        }

        return new FeedbackFormResource($feedbackForm);
    }

    /**
     * Update the specified resource in storage.
     *
     * @param  \App\Http\Requests\UpdateFeedbackFormRequest  $request
     * @param  \App\Models\FeedbackForm  $feedbackForm
     * @return \Illuminate\Http\Response
     */
    public function updateFeedbackForm(UpdateFeedbackFormRequest $request, FeedbackForm $feedbackForm)
    {
        $data = $request->validated();

        // Update FeedbackForm in the database
        $feedbackForm->update($data);

        // Get ids as plain array of existing questions
        $existingIds = $feedbackForm->questions()->pluck('id')->toArray();
        // Get ids as plain array of new questions
        $newIds = Arr::pluck($data['questions'], 'id');
        // Find questions to delete
        $toDelete = array_diff($existingIds, $newIds);
        //Find questions to add
        $toAdd = array_diff($newIds, $existingIds);

        // Delete questions by $toDelete array
        FeedbackFormQuestion::destroy($toDelete);

        // Create new questions
        foreach ($data['questions'] as $question) {
            if (in_array($question['id'], $toAdd)) {
                $question['feedback_form_id'] = $feedbackForm->id;
                $this->createQuestion($question);
            }
        }

        // Update existing questions
        $questionMap = collect($data['questions'])->keyBy('id');
        foreach ($feedbackForm->questions as $question) {
            if (isset($questionMap[$question->id])) {
                $this->updateQuestion($question, $questionMap[$question->id]);
            }
        }
```

```
        return new FeedbackFormResource($feedbackForm);
    }

    /**
     * Create a question and return
     *
     * @param $data
     * @return mixed
     * @throws \Illuminate\Validation\ValidationException
     * @author Zura Sekhniashvili <zurasekhniashvili@gmail.com>
     */
    private function createQuestion($data)
    {
        Log::info('Creating question with data:', $data);

        if (is_array($data['data'])) {
            $data['data'] = json_encode($data['data']);
        }

        Log::info('Creating question with data:', $data);


        $validator = Validator::make($data, [
            'question' => 'required|string',
            'type' => 'required|in:' . implode(',', QuestionTypeEnum::getValues()),
            'description' => 'nullable|string',
            'data' => 'present',
            'feedback_form_id' => 'exists:App\Models\FeedbackForm,id'
        ]);

        return FeedbackFormQuestion::create($validator->validated());
    }

    /**
     * Update a question and return true or false
     *
     * @param \App\Models\FeedbackFormQuestion $question
     * @param                                  $data
     * @return bool
     * @throws \Illuminate\Validation\ValidationException
     * @author Zura Sekhniashvili <zurasekhniashvili@gmail.com>
     */
    private function updateQuestion(FeedbackFormQuestion $question, $data)
    {
        if (is_array($data['data'])) {
            $data['data'] = json_encode($data['data']);
        }
        $validator = Validator::make($data, [
            'id' => 'exists:App\Models\FeedbackFormQuestion,id',
            'question' => 'required|string',
            'type' => 'required|in:' . implode(',', QuestionTypeEnum::getValues()),
            'description' => 'nullable|string',
            'data' => 'present',
        ]);

        return $question->update($validator->validated());
    }

    public function storeAnswer(StoreFeedbackFormAnswerRequest $request, $id)
    {
        $validated = $request->validated();

        $feedbackFormAnswer = FeedbackFormAnswer::create([
            'feedback_form_id' => $id,
            'start_date' => date('Y-m-d H:i:s'),
            'end_date' => date('Y-m-d H:i:s'),
```

```
        ]);

        foreach ($validated['answers'] as $questionId => $answer) {
            $question = FeedbackFormQuestion::where(['id' => $questionId, 'feedback_form_id' => $id])->get();
            if (!$question) {
                return response("Invalid question ID: \"$questionId\"", 400);
            }

            $data = [
                'feedback_form_question_id' => $questionId,
                'feedback_form_answer_id' => $feedbackFormAnswer->id,
                'answer' => is_array($answer) ? json_encode($answer) : $answer
            ];

            $questionAnswer = FeedbackFormQuestionAnswer::create($data);
        }

        return response("", 201);
    }
}
```

FILE: app/Http/Controllers/FileController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Services\CloudinaryService;

class FileController extends Controller
{
    protected $cloudinaryService;

    public function __construct(CloudinaryService $cloudinaryService)
    {
        $this->cloudinaryService = $cloudinaryService;
    }

    public function upload(Request $request)
    {
        $request->validate([
            'file' => 'required|file|mimes:jpeg,png,jpg,gif,webp,mp4|max:10240', // max file size 10MB
        ]);

        $file = $request->file('file');

        // Upload to Cloudinary
        $response = $this->cloudinaryService->uploadImage($file->getPathname());

        if (isset($response['error'])) {
            return response()->json(['error' => $response['error']], 500);
        }

        // Successfully uploaded file to Cloudinary
        return response()->json([
            'message' => 'File uploaded successfully',
            'url' => $response['secure_url'],
            'public_id' => $response['public_id']
        ]);
    }
}
```

FILE: app/Http/Controllers/FollowController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Follow;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log; // Import the Log facade

class FollowController extends Controller
{
    // Follow a user
    public function followUser(Request $request, $id)
    {
        $self = $request->user(); // Get the authenticated user from the request

        Log::info("User {$self->id} is attempting to follow user {$id}");

        // Find the user to follow
        $userToFollow = User::find($id);

        if (!$userToFollow) {
            Log::error("User {$self->id} tried to follow a non-existing user {$id}");
            return response()->json(['error' => 'User not found'], 404);
        }

        // Ensure a user cannot follow themselves
        if ($self->id === $userToFollow->id) {
            Log::warning("User {$self->id} tried to follow themselves.");
            return response()->json(['error' => 'Cannot follow yourself'], 400);
        }

        // Check if already following
        $existingFollow = Follow::where('follower_id', $self->id)
            ->where('following_id', $userToFollow->id)
            ->first();

        if ($existingFollow) {
            Log::info("User {$self->id} is already following user {$id}");
            return response()->json(['error' => 'Already following'], 400);
        }

        // Create the follow relationship
        $follow = Follow::create([
            'follower_id' => $self->id,
            'following_id' => $userToFollow->id,
        ]);

        Log::info("User {$self->id} successfully followed user {$id}");

        // You can add revalidation or cache clearing here if needed

        return response()->json(['message' => 'Followed successfully', 'follow' => $follow]);
    }

    // Unfollow a user
    public function unfollowUser(Request $request, $id)
    {
        $self = $request->user(); // Get the authenticated user from the request

        Log::info("User {$self->id} is attempting to unfollow user {$id}");

        // Find the user to unfollow
```

```
        $userToUnfollow = User::find($id);

        if (!$userToUnfollow) {
            Log::error("User {$self->id} tried to unfollow a non-existing user {$id}");
            return response()->json(['message' => 'User not found'], 404);
        }

        // Ensure a user cannot unfollow themselves
        if ($self->id === $userToUnfollow->id) {
            Log::warning("User {$self->id} tried to unfollow themselves.");
            return response()->json(['message' => 'Cannot unfollow yourself'], 400);
        }

        // Check if already following
        $existingFollow = Follow::where('follower_id', $self->id)
            ->where('following_id', $userToUnfollow->id)
            ->first();

        if (!$existingFollow) {
            Log::info("User {$self->id} is not following user {$id}.");
            return response()->json(['message' => 'Not following'], 400);
        }

        // Delete the follow relationship
        $existingFollow->delete();

        Log::info("User {$self->id} successfully unfollowed user {$id}");

        // You can add revalidation or cache clearing here if needed

        return response()->json(['message' => 'Unfollowed successfully']);
    }

    // Check if the user is following another user
    public function isFollowingUser(Request $request, $id)
    {
        $self = $request->user(); // Get the authenticated user from the request

        Log::info("User {$self->id} is checking if they are following user {$id}");

        $userToCheck = User::find($id);

        if (!$userToCheck) {
            Log::error("User {$self->id} tried to check follow status of a non-existing user {$id}");
            return response()->json(['error' => 'User not found'], 404);
        }

        // Check if the current user is following the given user
        $existingFollow = Follow::where('follower_id', $self->id)
            ->where('following_id', $userToCheck->id)
            ->first();

        Log::info("User {$self->id} checked follow status of user {$id}: " . ($existingFollow !== null ? 'Following' : 'Not Following'));

        return response()->json($existingFollow !== null);
    }

    // Get the list of users that the current user is following
    public function getFollowedUsers(Request $request)
    {
        $self = $request->user(); // Get the authenticated user from the request

        Log::info("User {$self->id} is requesting their list of followed users.");

        $followedUsers = Follow::where('follower_id', $self->id)
            ->whereHas('following', function ($query) use ($self) {
```

```
                $query->whereDoesntHave('blocks', function ($blockQuery) use ($self) {
                    $blockQuery->where('blocker_id', $self->id);
                });
            })
            ->with('following')
            ->get();

        Log::info("User {$self->id} has " . $followedUsers->count() . " followed users.");

        return response()->json($followedUsers);
    }
}
```

FILE: app/Http/Controllers/GroupController.php

```
<?php
namespace App\Http\Controllers;

use App\Jobs\DeleteGroupJob;
use App\Models\Group;
use App\Http\Requests\StoreGroupRequest;
use App\Http\Requests\UpdateGroupRequest;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class GroupController extends Controller
{
    /**
     * Store a newly created resource in storage.
     */
    public function store(StoreGroupRequest $request): JsonResponse
    {
        // Validate the request data
        $data = $request->validated();
        $user_ids = $data['user_ids'] ?? [];

        // Create the group
        $group = Group::create($data);

        // Attach users to the group, including the current authenticated user
        $group->users()->attach(array_unique([$request->user()->id, ...$user_ids]));

        // Return a JSON response with the created group data
        return response()->json(['message' => 'Group created successfully']);
    }

    /**
     * Update the specified resource in storage.
     */
    public function update(UpdateGroupRequest $request, $groupId): JsonResponse
    {
        $group = Group::find($groupId);

        // Validate the request data
        $data = $request->validated();
        $user_ids = $data['user_ids'] ?? [];

        // Update the group with the new data
        $group->update($data);

        // Remove all current users and attach the new ones
        $group->users()->detach();
        $group->users()->attach(array_unique([$request->user()->id, ...$user_ids]));
```

```
        // Return a JSON response confirming the update
        return response()->json(['message' => 'Group updated successfully']);
    }

    /**
     * Remove the specified resource from storage.
     */
    public function destroy(Request $request, $groupId): JsonResponse
    {
        // Retrieve the group based on the groupId
        $group = Group::find($groupId);

        // If the group doesn't exist, return a 404 error
        if (!$group) {
            return response()->json(['message' => 'Group not found'], 404);
        }

        // Check if the authenticated user is the owner of the group
        if ($group->owner_id !== $request->user()->id) {
            // If the user is not the owner, return a 403 Forbidden response
            return response()->json([
                'message' => 'You are not authorized to delete this group'
            ], 403);
        }

        // Dispatch the group deletion job with a delay
        DeleteGroupJob::dispatch($group)->delay(now()->addSeconds(10));

        // Return a JSON response indicating that the deletion was scheduled
        return response()->json(['message' => 'Group delete was scheduled and will be deleted soon']);
    }
}
```

FILE: app/Http/Controllers/LicensePlateController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Support\Facades\Log;
use Illuminate\Support\Facades\Http;
use Symfony\Component\HttpFoundation\File\Exception\FileException;
use Illuminate\Http\Request;

class LicensePlateController extends Controller
{
    // Define the URL of the FastAPI server
    protected $fastApiUrl = 'http://localhost:5000/recognise'; // Replace with your FastAPI URL

    // Function to handle the image upload and call FastAPI
    public function processLicensePlate(Request $request)
    {
        // Increase the max execution time to 1 hour (3600 seconds)
        set_time_limit(3600); // Set to 1 hour

        // Log the incoming request
        Log::info('Received a request to process a license plate image.', [
            'request_data' => $request->all(),
        ]);

        // Validate the incoming request to ensure a file is present
        $request->validate([
            'image' => 'required|image|mimes:jpeg,png,jpg|max:10240', // 10MB max image size
        ]);
```

```
        // Get the uploaded file from the request
        $image = $request->file('image');

        try {
            // Convert the image to base64 string
            $imageData = base64_encode(file_get_contents($image->getRealPath()));

            // Log the base64 image data (or part of it) for debugging
            Log::info('Converted image to base64 for sending to FastAPI.', [
                'image_name' => $image->getClientOriginalName(),
                'image_base64' => substr($imageData, 0, 50),  // Log only part of the base64 string
            ]);

            // Send the base64-encoded image to FastAPI with a longer timeout (1 hour)
            $response = Http::timeout(3600)->post($this->fastApiUrl, [
                'image' => $imageData, // Send base64 encoded image
            ]);

            // Log the response from FastAPI
            Log::info('Response from FastAPI server:', [
                'response_status' => $response->status(),
                'response_body' => $response->body(),
            ]);

            // Check if the request to FastAPI was successful
            if ($response->successful()) {
                $data = $response->json();
                return response()->json([
                    'success' => true,
                    'license_plate_results' => $data,
                ]);
            } else {
                return response()->json(['success' => false, 'message' => 'Error from FastAPI server']);
            }
        } catch (FileException $e) {
            Log::error('File upload error.', ['exception' => $e->getMessage()]);
            return response()->json(['success' => false, 'message' => 'File upload error: ' . $e->getMessage()]);
        } catch (\Exception $e) {
            Log::error('General error during license plate processing.', ['exception' => $e->getMessage()]);
            return response()->json(['success' => false, 'message' => 'Error: ' . $e->getMessage()]);
        }
    }
}
```

FILE: app/Http/Controllers/MailController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Mail;
use Illuminate\Support\Facades\Log;
use App\Mail\OrderConfirmationMail;

class MailController extends Controller
{
    public function sendEmail(Request $request)
    {
        // Log incoming request data
        Log::info('Received order email request', [
            'cart' => $request->input('cart'),
            'orderId' => $request->input('orderId'),
```

```
            'routeDetails' => $request->input('routeDetails'),
            'totalAmount' => $request->input('totalAmount'),
            'discountAmount' => $request->input('discountAmount'),
            'totalAfterDiscount' => $request->input('totalAfterDiscount'),
            'orderStatus' => $request->input('orderStatus'),
            'totalDistance' => $request->input('totalDistance'),
        ]);

        // Validate the incoming request
        $validatedData = $request->validate([
            'cart' => 'required|array',
            'orderId' => 'required|numeric',
            'routeDetails' => 'required|array',
            'totalAmount' => 'required|numeric',
            'discountAmount' => 'required|numeric',
            'totalAfterDiscount' => 'required|numeric',
            'orderStatus' => 'required|string',
            'totalDistance' => 'required|numeric',
        ]);

        // Extract variables from request
        $cart = $validatedData['cart'];
        $orderId = $validatedData['orderId'];
        $routeDetails = $validatedData['routeDetails'];
        $totalAmount = $validatedData['totalAmount'];
        $discountAmount = $validatedData['discountAmount'];
        $totalAfterDiscount = $validatedData['totalAfterDiscount'];
        $orderStatus = $validatedData['orderStatus'];
        $totalDistance = $validatedData['totalDistance'];

        // Log extracted data after validation
        Log::info('Validated order data', [
            'cart' => $cart,
            'orderId' => $orderId,
            'routeDetails' => $routeDetails,
            'totalAmount' => $totalAmount,
            'discountAmount' => $discountAmount,
            'totalAfterDiscount' => $totalAfterDiscount,
            'orderStatus' => $orderStatus,
            'totalDistance' => $totalDistance,
        ]);

        // Format the cart items
        $cartItems = array_map(function ($item) {
            return [
                'productName' => $item['product']['name'],
                'quantity' => $item['quantity'],
                'price' => number_format($item['total_amount'], 2),
                'image' => $item['product']['image'],
                'totalAmount' => number_format($item['total_amount'], 2)
            ];
        }, $cart);

        // Format route details
        $routeDetailsFormatted = array_map(function ($route) {
            return [
                'routeName' => $route['route_name'],
                'supplier' => $route['supplier_name'] ?? 'N/A',
                'warehouse' => $route['warehouse_name_1'] ?? 'N/A',
                'destination' => $route['end_location'],
                'estimatedTime' => $route['estimated_time'],
                'distance' => $route['distance']
            ];
        }, $routeDetails);

        // Prepare summary information
```

```
        $orderSummary = [
            'orderId' => $orderId ?: 'N/A',
            'totalAmount' => number_format($totalAmount, 2),
            'discountAmount' => $discountAmount > 0 ? number_format($discountAmount, 2) : '0.00',
            'totalAfterDiscount' => number_format($totalAfterDiscount, 2),
            'orderStatus' => $orderStatus ?: 'Pending',
            'totalDistance' => $totalDistance ?: '0',
        ];

        // Log order summary before sending email
        Log::info('Order summary prepared', $orderSummary);

        // Return the structured order data
        $orderData = [
            'orderSummary' => $orderSummary,
            'cartItems' => $cartItems,
            'routeDetails' => $routeDetailsFormatted,
        ];

        // Send the order confirmation email
        try {
            Mail::to('nguyendanghieu0608@gmail.com') // Replace with the recipient's email
                ->send(new OrderConfirmationMail($orderData));

            Log::info('Email sent successfully to nguyen...@gmail.com');
            return response()->json(['message' => 'Order processed and email sent successfully'], 200);
        } catch (\Exception $e) {
            // Log the error if email sending fails
            Log::error('Error sending email', [
                'error' => $e->getMessage(),
                'stack' => $e->getTraceAsString(),
            ]);
            return response()->json(['error' => 'Error sending email: ' . $e->getMessage()], 500);
        }
    }
}
```

FILE: app/Http/Controllers/MapController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class MapController extends Controller
{
    //
}
```

FILE: app/Http/Controllers/MessageController.php

```
<?php

namespace App\Http\Controllers;

use App\Events\SocketMessage;
use App\Http\Requests\StoreMessageRequest;
use App\Http\Resources\MessageResource;
use App\Models\Conversation;
use App\Models\Group;
```

```
use App\Models\Message;
use App\Models\MessageAttachment;
use Illuminate\Support\Str;
use Illuminate\Support\Facades\Storage;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Carbon\Carbon;

class MessageController extends Controller
{

    public static function getConversationsForSidebar(Request $request): JsonResponse
    {
        $user = $request->user();

        // Query for group-to-user conversations with eager loading of users
        $queryForGroupToUser = Group::select(['groups.*', 'messages.message as last_message', 'messages.created_at as last_message_date'])
            ->join('group_users', 'group_users.group_id', '=', 'groups.id')
            ->leftJoin('messages', 'messages.id', '=', 'groups.last_message_id')
            ->where('group_users.user_id', $user->id)
            ->with('users') // Ensure the users relationship is loaded
            ->orderBy('messages.created_at', 'desc')
            ->orderBy('groups.name');

        // Query for user-to-user conversations
        $userId = $user->id;
        $queryForUserToUser = Conversation::select([
                'conversations.id',
                'messages.message as last_message',
                'messages.created_at as last_message_date',
                'u1.first_name as user1_first_name',
                'u1.last_name as user1_last_name',
                'u2.first_name as user2_first_name',
                'u2.last_name as user2_last_name',
                'u2.profile_picture'
            ])
            ->leftJoin('users as u1', 'u1.id', '=', 'conversations.user_id1')
            ->leftJoin('users as u2', 'u2.id', '=', 'conversations.user_id2')
            ->leftJoin('messages', 'messages.id', '=', 'conversations.last_message_id')
            ->where(function ($query) use ($userId) {
                $query->where('conversations.user_id1', $userId)
                    ->orWhere('conversations.user_id2', $userId);
            })
            ->orderByDesc('messages.created_at');

        // Map group conversations
        $groupConversations = $queryForGroupToUser->get()->map(function ($group) {
            return [
                'id' => $group->id,
                'name' => $group->name,
                'description' => $group->description,
                'is_group' => true,
                'is_user' => false,
                'owner_id' => $group->owner_id,
                'users' => $group->users,
                'user_ids' => $group->users->pluck('id'),
                'last_message' => $group->last_message,
                'last_message_date' => $group->last_message_date ? (Carbon::parse($group->last_message_date)->toIso8601String())  : null,
// Format the date correctly
            ];
        });

        // Map user-to-user conversations
        $userToUserConversations = $queryForUserToUser->get()->map(function ($conversation) use ($userId) {
            // Determine which user is the logged-in user and get the other user's details
```

```
            if ($conversation->user_id1 === $userId) {
                $otherUserFirstName = $conversation->user2_first_name;
                $otherUserLastName = $conversation->user2_last_name;
                $otherUserProfilePicture = $conversation->profile_picture;
            } else {
                $otherUserFirstName = $conversation->user1_first_name;
                $otherUserLastName = $conversation->user1_last_name;
                $otherUserProfilePicture = $conversation->profile_picture;
            }

            return [
                'id' => $conversation->id,
                'name' => $otherUserFirstName . ' ' . $otherUserLastName,
                'profile_picture' => $otherUserProfilePicture,
                'is_group' => false,
                'is_user' => true,
                'last_message' => $conversation->last_message,
                'last_message_date' => $conversation->last_message_date ?(Carbon::parse($conversation->last_message_date)-
>toIso8601String()) : null, // Format the date correctly
            ];
        });

        // Merge both conversations and sort by last message date
        $mergedConversations = $groupConversations->merge($userToUserConversations)->sortByDesc('last_message_date');

        return response()->json($mergedConversations);
    }


    public function store(Request $request, StoreMessageRequest $messageRequest): JsonResponse
    {
        // Validate and extract data from the request
        $data = $messageRequest->validated();
        $data['sender_id'] = request()->user()->id; // Set sender ID

        $receiverId = $data['receiver_id'] ?? null;
        $groupId = $data['group_id'] ?? null;
        $messageContent = $data['message']; // The actual message content
        $files = $data['attachments'] ?? []; // The attachments

        // Create a new message instance
        $message = Message::create($data);

        // Handle file attachments
        $attachments = [];
        if ($files) {
            foreach ($files as $file) {
                $directory = 'attachments/' . Str::random(32);
                Storage::makeDirectory($directory);

                $attachmentModel = [
                    'message_id' => $message->id,
                    'name' => $file->getClientOriginalName(),
                    'mime' => $file->getClientMimeType(),
                    'size' => $file->getSize(),
                    'path' => $file->store($directory, 'public'),
                ];

                $attachment = MessageAttachment::create($attachmentModel);
                $attachments[] = $attachment;
            }
            $message->attachments = $attachments;
        }

        // Update or create a conversation
        if ($receiverId) {
```

```
        // Find the conversation by user_id1 and user_id2, or create it if it doesn't exist
        $conversation = Conversation::where(function ($query) use ($receiverId) {
            $query->where('user_id1', request()->user()->id)
                ->where('user_id2', $receiverId);
        })->orWhere(function ($query) use ($receiverId) {
            $query->where('user_id1', $receiverId)
                ->where('user_id2', request()->user()->id);
        })->first();

        if ($conversation) {
            // If conversation exists, update the last_message_id
            $conversation->update([
                'last_message_id' => $message->id,
            ]);
        } else {
            // Create a new conversation if it doesn't exist
            Conversation::create([
                'user_id1' => request()->user()->id,
                'user_id2' => $receiverId,
                'last_message_id' => $message->id,
            ]);
        }
    }

    // Update the group if the groupId is provided
    if ($groupId) {
        // Use updateOrCreate to update or create the group with the last_message_id
        Group::updateOrCreate(
            ['id' => $groupId], // search conditions (looking for group by ID)
            ['last_message_id' => $message->id] // values to update (set the last_message_id to the new message ID)
        );
    }

    // Dispatch the message event for broadcasting
    SocketMessage::dispatch($message);

    // Return the created message as a response, formatted with MessageResource
    return response()->json(new MessageResource($message));
}

/**
 * Get messages by user or group
 */
public function getMessages(Request $request, $id): JsonResponse
{
    // Check if the ID belongs to a user or group by some identifier (e.g., `is_user` in the request)
    $isUser = $request->query('isUser', false); // You can set 'isUser' as a query parameter

    if ($isUser) {
        // Fetch messages between the current user and the specified user
        $messages = Message::where('sender_id', request()->user()->id)
            ->where('receiver_id', $id)
            ->orWhere('sender_id', $id)
            ->where('receiver_id', request()->user()->id)
            ->latest()
            ->paginate(10);
    } else {
        // Fetch messages for a group
        $messages = Message::where('group_id', $id)
            ->latest()
            ->paginate(10);
    }

    return response()->json(MessageResource::collection($messages));
}
```

```
    /**
     * Load older messages based on the provided message
     */
    public function loadOlder(Request $request, Message $message): JsonResponse
    {
        // Load older messages that are older than the given message
        if ($message->group_id) {
            $messages = Message::where('created_at', '<', $message->created_at)
                ->where('group_id', $message->group_id)
                ->latest()
                ->paginate(10);
        } else {
            $messages = Message::where('created_at', '<', $message->created_at)
                ->where(function ($query) use ($message) {
                    $query->where('sender_id', $message->sender_id)
                        ->where('receiver_id', $message->receiver_id)
                        ->orWhere('sender_id', $message->receiver_id)
                        ->where('receiver_id', $message->sender_id);
                })
                ->latest()
                ->paginate(10);
        }

        // Return the messages as a resource
        return response()->json(MessageResource::collection($messages));
    }


    /**
     * Remove the specified message from storage.
     */
    public function deleteMessage(Request $request, $messageId): JsonResponse
    {
        $message = Message::find($messageId);
        // Check if the user is the sender of the message
        if ($message->sender_id !== request()->user()->id) { // Updated here
            return response()->json(['message' => 'Forbidden'], 403);
        }

        $group = null;
        $conversation = null;
        // Check if the message is part of a group or conversation
        if ($message->group_id) {
            $group = Group::where('last_message_id', $message->id)->first();
        } else {
            $conversation = Conversation::where('last_message_id', $message->id)->first();
        }

        // Delete the message
        $message->delete();

        // Get the new last message for the group or conversation
        $lastMessage = null;
        if ($group) {
            // Repopulate $group with the latest database data
            $group = Group::find($group->id);
            $lastMessage = $group->lastMessage;
        } else if ($conversation) {
            $conversation = Conversation::find($conversation->id);
            $lastMessage = $conversation->lastMessage;
        }

        // Return the last message or null
        return response()->json($lastMessage ? new MessageResource($lastMessage) : null);
    }
```

```
}
```

FILE: app/Http/Controllers/NotificationController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Notification;
use Illuminate\Http\Request;

class NotificationController extends Controller
{
    public function getUserNotifications(Request $request)
    {
        try {
            $notifications = Notification::where('recipient_id', $request->user()->id)
                ->orderBy('created_at', 'desc')
                ->with(['relatedUser:id,first_name,last_name,phone_number,email,language,profile_picture,banner_img,headline,about',
'relatedPost:id,content,image'])
                ->get();

            return response()->json($notifications, 200);
        } catch (\Exception $error) {
            \Log::error("Error in getUserNotifications controller: ", ['error' => $error]);
            return response()->json(['message' => 'Internal server error'], 500);
        }
    }

    public function markNotificationAsRead(Request $request, $id)
    {
        try {
            $notification = Notification::where('id', $id)
                ->where('recipient_id', $request->user()->id)
                ->firstOrFail();

            $notification->read = true;
            $notification->save();

            return response()->json($notification);
        } catch (\Exception $error) {
            \Log::error("Error in markNotificationAsRead controller: ", ['error' => $error]);
            return response()->json(['message' => 'Internal server error'], 500);
        }
    }

    public function deleteNotification(Request $request, $id)
    {
        try {
            $notification = Notification::where('id', $id)
                ->where('recipient_id', $request->user()->id)
                ->firstOrFail();

            $notification->delete();

            return response()->json(['message' => 'Notification deleted successfully']);
        } catch (\Exception $error) {
            \Log::error("Error in deleteNotification controller: ", ['error' => $error]);
            return response()->json(['message' => 'Server error'], 500);
        }
    }
}
```

FILE: app/Http/Controllers/OrderController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Order;
use Illuminate\Http\Request;

class OrderController extends Controller
{
    // Method to get the order status by order ID
    public function getOrderStatusById($orderId)
    {
        try {
            // Find the order by ID
            $order = Order::find($orderId);

            // If the order doesn't exist, return a 404 response
            if (!$order) {
                return response()->json(['error' => 'Order not found.'], 404);
            }

            // Return the order status
            return response()->json($order->status, 200);

        } catch (\Exception $e) {
            // Log any exceptions and return a 500 error response
            return response()->json(['error' => 'An error occurred while fetching the order status.'], 500);
        }
    }
}
```

FILE: app/Http/Controllers/PaymentController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Order;
use App\Models\Shipment;
use App\Models\Payment;
use App\Models\RouteOptimization;
use App\Models\RouteDetail;
use App\Models\OrderItem;
use App\Models\Warehouse;
use App\Models\FeedbackForm;
use App\Models\FeedbackFormQuestion;
use App\Models\FeedbackFormAnswer;
use App\Models\FeedbackFormQuestionAnswer;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Log;
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Str;
use Carbon\Carbon;
use Illuminate\Support\Facades\Cache;
use App\Models\RouteCondition;
$trackingNumber = 'TRACK-' . strtoupper(Str::random(10));  // Generate a random tracking number

class PaymentController extends Controller
{
```

```
    protected $mapboxApiKey;

    // Injecting the necessary keys and URLs
    public function __construct()
    {
        // Set your Mapbox API key here
        $this->mapboxApiKey = env('MAPBOX_API_KEY');  // Updated to the correct Mapbox token name

    }


    /**
     * Geocode an address using Mapbox Geocoding API
     *
     * @param string $address
     * @return array|null
     */
    public function geocodeAddress($address)
    {
        // Check if the address is valid (non-empty)
        if (empty($address)) {
            Log::error('Geocoding failed, empty address', ['address' => $address]);
            return null;  // or return some error message if necessary
        }

        // Log the received address for debugging
        Log::info('Geocoding address received', ['address' => $address]);

        // Check if the address is already cached
        $cacheKey = 'geocode_' . md5($address);  // Create a unique cache key based on the address

        // Try to get the geocoded data from cache
        $result = Cache::get($cacheKey);

        if ($result) {
            // If the result is cached, return it directly
            Log::info('Returning geocoded address from cache', ['address' => $address]);
            return $result;
        }

        // If the result is not cached, perform the geocoding API call
        $url = 'https://api.mapbox.com/geocoding/v5/mapbox.places/' . urlencode($address) . '.json';

        // Perform geocoding API call
        $response = Http::get($url, [
            'access_token' => $this->mapboxApiKey,
            'limit' => 1,  // Adjust the limit to control how many results you want
        ]);

        // Check if the request was successful
        if ($response->successful()) {
            $result = $response->json()['features'][0] ?? null;

            if ($result) {
                // Extract the geocoded coordinates (longitude, latitude)
                $longitude = $result['geometry']['coordinates'][0];
                $latitude = $result['geometry']['coordinates'][1];

                // Check if the coordinates fall within the valid bounds for Mapbox
                if (!$this->isValidCoordinates($latitude, $longitude)) {
                    // If not valid, adjust them (optional: you could set a default fallback location or adjust to nearest valid point)
                    Log::warning('Coordinates are out of Mapbox bounds, adjusting coordinates.', [
                        'latitude' => $latitude,
                        'longitude' => $longitude,
                    ]);
```

```
                // Adjust the coordinates to fit within Mapbox bounds
                // Latitude between -85 and 85
                // Longitude between -180 and 180
                $latitude = max(min($latitude, 85), -85);
                $longitude = max(min($longitude, 180), -180);

                // Optionally, log the adjustment
                Log::info('Adjusted coordinates to be within Mapbox bounds.', [
                    'adjusted_latitude' => $latitude,
                    'adjusted_longitude' => $longitude,
                ]);

                // Update the result coordinates to the adjusted ones
                $result['geometry']['coordinates'] = [$longitude, $latitude];
            }

            // Cache the result for 24 hours (or any duration you prefer)
            Cache::put($cacheKey, $result, now()->addHours(24));

            return $result;  // Return the geocoded result
        }
    } else {
        // Handle API failure
        Log::error('Mapbox Geocoding failed', [
            'address' => $address,
            'response_status' => $response->status(),
            'response_body' => $response->body(),
        ]);
    }

    return null;  // Return null if no result
}


/**
 * Check if coordinates are within valid Mapbox bounds.
 *
 * @param float $latitude
 * @param float $longitude
 * @return bool
 */
private function isValidCoordinates($latitude, $longitude)
{
    // Valid latitude range: -85 to 85
    // Valid longitude range: -180 to 180
    return $latitude >= -85 && $latitude <= 85 && $longitude >= -180 && $longitude <= 180;
}


/**
 * Get the distance between two coordinates using Mapbox Directions API
 *
 * @param array $originLatLng (lat, lon)
 * @param array $destinationLatLng (lat, lon)
 * @return float|null  Distance in kilometers
 */

public function getDistance($originLatLng, $destinationLatLng)
{
    // Extract latitude and longitude from the input arrays
    $originLat = $originLatLng['center'][1];  // Latitude is at index 1
    $originLon = $originLatLng['center'][0];  // Longitude is at index 0

    $destinationLat = $destinationLatLng['center'][1];  // Latitude is at index 1
    $destinationLon = $destinationLatLng['center'][0];  // Longitude is at index 0
```

```
        // Convert degrees to radians
        $originLat = deg2rad($originLat);
        $originLon = deg2rad($originLon);
        $destinationLat = deg2rad($destinationLat);
        $destinationLon = deg2rad($destinationLon);

        // Haversine formula
        $dLat = $destinationLat - $originLat;
        $dLon = $destinationLon - $originLon;

        $a = sin($dLat / 2) * sin($dLat / 2) +
            cos($originLat) * cos($destinationLat) *
            sin($dLon / 2) * sin($dLon / 2);

        $c = 2 * atan2(sqrt($a), sqrt(1 - $a));

        // Earth radius in kilometers (mean radius = 6,371 km)
        $R = 6371;

        // Calculate the distance
        $distance = $R * $c;

        // Return the distance in kilometers
        return $distance;
    }



    /**
 * Handle the payment processing through an AJAX request.
 *
 * @param Request $request
 * @return \Illuminate\Http\JsonResponse
 */
public function processPayment(Request $request)
{
    // Validate the payment data coming from frontend
    $validatedData = $request->validate([
        'orderId' => 'required|exists:orders,id',
        'payment_method' => 'required|string',
        'gateway' => 'required|string',
        'currency' => 'required|string',
    ]);

    // Retrieve the order
    $order = Order::findOrFail($validatedData['orderId']);

    // Check if the user is authorized to make this payment (for security reasons)
    if ($order->user_id !== Auth::id()) {
        return response()->json(['error' => 'Unauthorized access'], 403);
    }

    // Create the payment record
    $payment = new Payment();
    $payment->order_id = $order->id;
    $payment->total_amount = $order->total_amount;
    $payment->paid_amount = $order->total_amount;
    $payment->due_amount = $order->total_amount - $payment->paid_amount;
    $payment->payment_method = $validatedData['payment_method']; // 'credit_card', 'paypal', etc.
    $payment->gateway = $validatedData['gateway']; // 'stripe', 'razorpay', etc.
    $payment->currency = $validatedData['currency'];
    $payment->payment_status = 'success'; // Assuming success if no issues
    $payment->payment_date = now(); // Use the current date and time
    $payment->providers_id = 1; // Example for provider ID, this may need to be dynamic
    $payment->save();
```

```
    // Update the order status to 'Paid'
    $order->status = 'Paid';
    $order->save();

    // Fetch the related order items with product and supplier details
    $results = OrderItem::where('orders_id', $validatedData['orderId'])
        ->with(['product.supplier']) // Eager load product and supplier
        ->get()
        ->map(function ($orderItem) {
            $product = $orderItem->product;
            $supplier = $product ? $product->supplier : null;

            return [
                'supplier' => $supplier ? $supplier : null,
            ];
        });

    // Get a collection of suppliers from the order items
    $suppliers = $results->pluck('supplier')->filter();

    foreach ($suppliers as $supplier) {
        // Check if the FeedbackForm already exists for the supplier and the order
        $existingFeedbackForm = FeedbackForm::where('order_id', $validatedData['orderId'])
            ->where('user_id', $supplier->id)
            ->first();

        // Log the current state for debugging
        Log::info('Existing Feedback Form', ['existing_feedback_form' => $existingFeedbackForm]);

        if (!$existingFeedbackForm) {
            // Find the FeedbackForm for the current supplier (assuming it exists)
            $feedbackForm = FeedbackForm::where('user_id', $supplier->id)->first();

            if ($feedbackForm) {
                // Create a copy (replicate) of the existing FeedbackForm
                $newFeedbackForm = $feedbackForm->replicate();
                Log::info('New Feedback Form', ['new_feedback_form' => $newFeedbackForm]);

                // Set the new order_id
                $newFeedbackForm->order_id = $validatedData['orderId'];

                // Set the timestamps to the current time
                $newFeedbackForm->created_at = Carbon::now();
                $newFeedbackForm->updated_at = Carbon::now();

                // Save the new FeedbackForm with the updated order_id
                $newFeedbackForm->save();

                // Replicate FeedbackFormQuestion
                $feedbackFormQuestions = FeedbackFormQuestion::where('feedback_form_id', $feedbackForm->id)->get();
                foreach ($feedbackFormQuestions as $question) {
                    $newQuestion = $question->replicate();
                    $newQuestion->feedback_form_id = $newFeedbackForm->id; // Associate with the new feedback form
                    $newQuestion->save();

                    // Replicate FeedbackFormAnswer
                    $feedbackFormAnswers = FeedbackFormAnswer::where('feedback_form_id', $feedbackForm->id)->get();
                    foreach ($feedbackFormAnswers as $answer) {
                        $newAnswer = $answer->replicate();
                        $newAnswer->feedback_form_id = $newFeedbackForm->id; // Associate with the new feedback form
                        $newAnswer->save();

                        // Replicate FeedbackFormQuestionAnswer
                        $questionAnswers = FeedbackFormQuestionAnswer::where([
                            'feedback_form_question_id' => $question->id,
                            'feedback_form_answer_id' => $answer->id
```

```
                        ])->get();

                        foreach ($questionAnswers as $qa) {
                            $newQa = $qa->replicate();
                            $newQa->feedback_form_question_id = $newQuestion->id;
                            $newQa->feedback_form_answer_id = $newAnswer->id;
                            $newQa->save();
                        }
                    }
                }
            }
        } else {
            Log::info('Skipping Feedback Form Creation', ['supplier_id' => $supplier->id, 'order_id' => $validatedData['orderId']]);
        }
    }

    // Return a successful response with the payment details
    return response()->json(['success' => 'Payment processed successfully', 'payment' => $payment]);
}


    public function prepareDelivery(Request $request)
    {


        // Validate input data
        Log::info('Validating input data for order and user location.');
        $validatedData = $request->validate([
            'orderId' => 'required|exists:orders,id',
            'userLocation' => 'required'
        ]);

        $orderId = $validatedData['orderId'];
        $userLocation = $validatedData['userLocation'];

        Log::info('Input validated.', ['order_id' => $orderId, 'user_location' => $userLocation]);

        // Step 1: Get all supplier locations for the given order
        Log::info('Fetching supplier locations for order ID: ' . $orderId);
            // Single query to fetch order items with related product and supplier data
        $results = OrderItem::where('orders_id', $orderId)
        ->with(['product.supplier']) // Eager load product and supplier
        ->get()
        ->map(function ($orderItem) {
            $product = $orderItem->product;
            $supplier = $product ? $product->supplier : null;

            return [
                'product' => $product,
                'supplier_location' => $supplier ? $supplier : null,
            ];
        });

        // Separate the results into two variables: one for supplier locations, one for products
        $supplierLocations = $results->pluck('supplier_location')->filter();
        $productsInOrder = $results->pluck('product')->filter();

        Log::info('Found suppliers for the order.', ['suppliers' => $supplierLocations->toArray()]);

        $warehouses = Warehouse::whereHas('inventory', function ($query) use ($productsInOrder) {
            // Filter warehouses that have inventory for any of the products in the order
            $query->whereIn('products_id', $productsInOrder->pluck('id'));
        })->get();

        Log::info('Fetching warehouses with inventory for the products in the order.', [
            'warehouses' => $warehouses->toArray()
```

```
        ]);
        Log::info('Warehouses loaded for route calculations.', ['warehouses_count' => $warehouses->count()]);

        // Initialize variables
        $routeDetails = [];
        $routeDetailModels = [];
        $totalDistance = 0;
        $timeSum = 0;  // Track cumulative estimated time
        $nearestWarehouses = [];  // Array to store the nearest warehouses
        $shipment = Shipment::create([
            'providers_id' => 1,
            'orders_id' => $orderId,
        ]);
        $shipmentId = $shipment->id;
        $routeOptimization = RouteOptimization::create([
            'shipments_id' => $shipmentId,
            'total_distance' => 0,
        ]);
        $routeIds = [];
        $nowDate = Carbon::now();
        // Step 3: For each supplier, calculate the nearest warehouse
        Log::info('Calculating the nearest warehouse for each supplier.');

        foreach ($supplierLocations as $supplier) {
            Log::info('Processing supplier: ' . $supplier->first_name . $supplier->last_name);
            $supplierLocation = $supplier->getPrimaryAddress();


            // Step 3.1: Geocode the supplier location using OpenStreetMap
            $supplierLatLng = $this->geocodeAddress($supplierLocation);
            if (!$supplierLatLng) {
                Log::warning('Supplier location geocoding failed.', ['supplier_name' => $supplier->first_name . $supplier->last_name]);
                continue;
            }

            // Step 3.2: Find the nearest warehouse
            $nearestWarehouse = $warehouses->map(function ($warehouse) use ($supplierLatLng) {
                $warehouseLatLng = $this->geocodeAddress($warehouse->location);
                if ($warehouseLatLng) {
                    $distance = $this->getDistance($supplierLatLng, $warehouseLatLng);
                    return [
                        'warehouse' => $warehouse,
                        'distance' => $this->parseDistance($distance)
                    ];
                }
                return null;
            })->filter()->sortBy('distance')->first();  // Get the nearest warehouse to this supplier

            if (!$nearestWarehouse) {
                Log::warning('No warehouse found for supplier.', ['supplier_name' => $supplier->first_name . $supplier->last_name]);
                continue;
            }

            $nearestWarehouses[] = $nearestWarehouse;

            Log::info('Nearest warehouse found for supplier.', [
                'warehouse_name' => $nearestWarehouse['warehouse']->warehouse_name,
                'distance' => $nearestWarehouse['distance']
            ]);

            // Step 4: Calculate estimated time for this route and update timeSum
            $estimatedTimeVariable = $this->estimateTime($nearestWarehouse['distance'], $timeSum,$nowDate);
            Log::info('Estimated time for supplier-to-warehouse route.', ['estimated_time' => $estimatedTimeVariable]);
            $timeSum = $estimatedTimeVariable;

            // Step 5: Store route details for the supplier to warehouse
```

```
            $routeDetails[] = [
                'route_name' => 'Supplier ' . $supplier->first_name . $supplier->last_name . ' to ' . $nearestWarehouse['warehouse']-
>warehouse_name,
                'supplier_name' => $supplier->first_name . $supplier->last_name,
                'warehouse_name_1' => $nearestWarehouse['warehouse']->warehouse_name,
                'distance' => $nearestWarehouse['distance'],
                'start_location' => $supplierLocation,
                'end_location' => $nearestWarehouse['warehouse']->location,
                'estimated_time' => $estimatedTimeVariable,
            ];

            // Create RouteDetail records for each route optimization
            Log::info('Creating RouteDetail records for route optimization.');
            $routeDetail = RouteDetail::create([
                'route_optimization_id' => $routeOptimization->id,  // This will be updated later
                'route_name' => 'Supplier ' . $supplier->first_name . $supplier->last_name . ' to ' . $nearestWarehouse['warehouse']-
>warehouse_name,
                'supplier_name' => $supplier->first_name . $supplier->last_name,
                'warehouse_name_1' => $nearestWarehouse['warehouse']->warehouse_name,
                'distance' => $nearestWarehouse['distance'],
                'start_location' => $supplierLocation,
                'end_location' => $nearestWarehouse['warehouse']->location,
                'start_latitude' => $this->geocodeAddress($supplierLocation)['center'][1],
                'start_longitude' => $this->geocodeAddress($supplierLocation)['center'][0],
                'end_latitude' => $this->geocodeAddress($nearestWarehouse['warehouse']->location)['center'][1],
                'end_longitude' => $this->geocodeAddress($nearestWarehouse['warehouse']->location)['center'][0],
                'estimated_time' => $estimatedTimeVariable,
            ]);

            $routeDetailModels[] = $routeDetail;

            // Update total distance
            $totalDistance += $nearestWarehouse['distance'];
            Log::info('Updated total distance after supplier to warehouse.', ['total_distance' => $totalDistance]);
        }

        $nearestWarehouses = collect($nearestWarehouses)->unique(function ($item) {
            return $item['warehouse']->warehouse_name;
        })->values()->all();

        Log::info('Nearest warehouses', ['warehouses' => $nearestWarehouses]);

        // Step 6: Sum distances between the nearest warehouses
        Log::info('Summing distances between nearest warehouses.');

        for ($i = 0; $i < count($nearestWarehouses) - 1; $i++) {
            $currentWarehouse = $nearestWarehouses[$i];
            $nextWarehouse = $nearestWarehouses[$i + 1];

            $currentWarehouseLatLng = $this->geocodeAddress($currentWarehouse['warehouse']->location);
            $nextWarehouseLatLng = $this->geocodeAddress($nextWarehouse['warehouse']->location);

            Log::info('current warehouse', ['warehouse' => $nearestWarehouse['warehouse']->location]);
            Log::info('next warehouse', ['warehouse' => $nextWarehouse]);

            // Step 6.1: Calculate the distance between the current warehouse and the next warehouse
            $distanceBetweenWarehouses = $this->getDistance(
                $currentWarehouseLatLng,
                $nextWarehouseLatLng,
            );

            Log::info('distance', ['distance' => $distanceBetweenWarehouses]);

            $distanceBetweenWarehouses = $this->parseDistance($distanceBetweenWarehouses);

            // Step 6.2: Update total distance
```

```
            $totalDistance += $distanceBetweenWarehouses;

            $estimatedTimeVariable = $this->estimateTime($distanceBetweenWarehouses, $timeSum, $nowDate);
            Log::info('Estimated time for supplier-to-warehouse route.', ['estimated_time' => $estimatedTimeVariable]);
            $timeSum = $estimatedTimeVariable;
            // Step 6.3: Create a RouteDetail for the warehouse-to-warehouse leg
            $routeDetail = RouteDetail::create([
                'route_optimization_id' => $routeOptimization->id,
                'route_name' => 'Warehouse ' . $currentWarehouse['warehouse']->warehouse_name . ' to ' . $nextWarehouse['warehouse']-
>warehouse_name,
                'warehouse_name_1' => $currentWarehouse['warehouse']->warehouse_name,
                'warehouse_name_2' => $nextWarehouse['warehouse']->warehouse_name,
                'distance' => $distanceBetweenWarehouses,
                'start_location' => $currentWarehouse['warehouse']->location,
                'end_location' => $nextWarehouse['warehouse']->location,
                'start_latitude' => $this->geocodeAddress($currentWarehouse['warehouse']->location)['center'][1],
                'start_longitude' => $this->geocodeAddress($currentWarehouse['warehouse']->location)['center'][0],
                'end_latitude' => $this->geocodeAddress($nextWarehouse['warehouse']->location)['center'][1],
                'end_longitude' => $this->geocodeAddress($nextWarehouse['warehouse']->location)['center'][0],
                'estimated_time' => $estimatedTimeVariable,
            ]);


            $routeDetails[] = [
                'route_name' => 'Warehouse ' . $currentWarehouse['warehouse']->warehouse_name . ' to ' . $nextWarehouse['warehouse']-
>warehouse_name,
                'warehouse_name_1' => $currentWarehouse['warehouse']->warehouse_name,
                'warehouse_name_2' => $nextWarehouse['warehouse']->warehouse_name,
                'distance' => $distanceBetweenWarehouses,
                'start_location' => $currentWarehouse['warehouse']->location,
                'end_location' => $nextWarehouse['warehouse']->location,
                'estimated_time' => $estimatedTimeVariable,
            ];

            $routeDetailModels[] = $routeDetail;

            Log::info('Distance between warehouses:', [
                'from_warehouse' => $currentWarehouse['warehouse']->warehouse_name,
                'to_warehouse' => $nextWarehouse['warehouse']->warehouse_name,
                'distance' => $distanceBetweenWarehouses,
            ]);
        }

        // Step 7: Calculate the nearest warehouse to the user's location
        Log::info('Calculating nearest warehouse to the user location.');
        $nearestWarehouseToUser = $warehouses->map(function ($warehouse) use ($userLocation) {

            $warehouseLatLng = $this->geocodeAddress($warehouse->location);
            $userLatLng = $this->geocodeAddress($userLocation);

            $distance = $this->getDistance($warehouseLatLng, $userLatLng);
            return [
                'warehouse' => $warehouse,
                'distance' => $this->parseDistance($distance)
            ];
        })->sortBy('distance')->first();  // Nearest warehouse to the user

        Log::info('Nearest warehouse to user location:', [
            'warehouse_name' => $nearestWarehouseToUser['warehouse']->warehouse_name,
            'distance' => $nearestWarehouseToUser['distance']
        ]);

        // Step 8: Calculate estimated time for the second route (warehouse to user)
        $estimatedTimeVariable2 = $this->estimateTime($nearestWarehouseToUser['distance'], $timeSum,$nowDate);
        Log::info('Estimated time for warehouse-to-user route.', ['estimated_time' => $estimatedTimeVariable2]);
        $timeSum = $estimatedTimeVariable2;
```

```
        // Create a RouteDetail for warehouse-to-user route
        $routeDetail = RouteDetail::create([
            'route_optimization_id' => $routeOptimization->id,
            'route_name' => 'Warehouse ' . $nearestWarehouseToUser['warehouse']->warehouse_name . ' to User',
            'warehouse_name_1' => $nearestWarehouseToUser['warehouse']->warehouse_name,
            'distance' => $nearestWarehouseToUser['distance'],
            'start_location' => $nearestWarehouseToUser['warehouse']->location,
            'end_location' => $userLocation,
            'start_latitude' => $this->geocodeAddress($nearestWarehouseToUser['warehouse']->location)['center'][1],
            'start_longitude' => $this->geocodeAddress($nearestWarehouseToUser['warehouse']->location)['center'][0],
            'end_latitude' => $this->geocodeAddress($userLocation)['center'][1],
            'end_longitude' => $this->geocodeAddress($userLocation)['center'][0],
            'estimated_time' => $estimatedTimeVariable2,
        ]);

        $routeDetails[] = [
            'route_name' => 'Warehouse ' . $nearestWarehouseToUser['warehouse']->warehouse_name . ' to you',
            'warehouse_name_1' => $nearestWarehouseToUser['warehouse']->warehouse_name,
            'distance' => $nearestWarehouseToUser['distance'],
            'start_location' => $nearestWarehouseToUser['warehouse']->location,
            'end_location' => $userLocation,
            'estimated_time' => $estimatedTimeVariable2,
        ];

        $routeDetailModels[] = $routeDetail;


        // Add the distance from the warehouse to the user
        $totalDistance += $nearestWarehouseToUser['distance'];

        // Step 10: Create the Shipment record
        Log::info('Creating shipment record.');
        $trackingNumber = 'TRACK-' . strtoupper(Str::random(10));  // Generate a random tracking number

        $shipment->update([
            'status' => 'in_transit',
            'estimated_arrival' => $estimatedTimeVariable2,
            'origin' => $supplierLocations->first()->location,
            'destination' => $userLocation,
            'shipment_method' => 'standard',
            'tracking_number' => $trackingNumber,
            'last_updated' => now(),
            'total_amount' => $totalDistance / 100,
        ]);

        Log::info('Updating route optimization record.');
        $routeOptimization->update([
            'total_distance' => $totalDistance
        ]);

        // Step 13: Return the results
        Log::info('Delivery preparation complete, returning response.');
        return response()->json([
            'route_details' => $routeDetails,
            'total_distance' => $totalDistance,
        ]);
    }


    // Method to parse distance to the required format
    private function parseDistance($distance)
    {
        Log::info('Parsing distance.', ['raw_distance' => $distance]);
        return floatval($distance);  // Example: Return the parsed distance (adjust according to Google Maps API response)
```

```
    }

    // Method to estimate the time for the route
    private function estimateTime($distance, $previousTime, $nowDate)
    {
        Log::info('Estimating time for the route.', ['distance' => $distance, 'previous_time' => $previousTime]);
        $timePerKm = 10;  // Example: 10 minutes per km
        $estimatedTime = $distance * $timePerKm;
        if ($previousTime instanceof Carbon) {
            $combinedTime = $this->carbonToInt($previousTime, $nowDate) + (int) $estimatedTime;
            return $nowDate->addMinutes($combinedTime);
        } else {
            $combinedTime = $previousTime + (int) $estimatedTime;
            return $nowDate->addMinutes($combinedTime);

        }
    }
    private function carbonToInt(Carbon $carbonDate,$nowDate)
    {

        // Calculate the difference between now and the given Carbon instance
        // You can use diffInMinutes, diffInSeconds, or other methods based on your requirement
        $diffInMinutes = $nowDate->diffInMinutes($carbonDate);

        // Return the result as an integer (in minutes)
        return (int) $diffInMinutes;
    }

    public function getRouteDetailsByOrderId($orderId)
    {
        // Log the beginning of the request with the timestamp
        Log::info('Fetching route details for order ID ' . $orderId . ' started at: ' . Carbon::now()->toDateTimeString());

        // Record the start time for performance tracking
        $startTime = microtime(true);

        try {
            // Fetch the shipments based on the orders_id
            $shipments = Shipment::where('orders_id', $orderId)->get();

            if ($shipments->isEmpty()) {
                return response()->json(['error' => 'No shipments found for this order ID.'], 404);
            }

            // Log the number of shipments retrieved
            Log::info('Number of shipments found for order ID ' . $orderId . ': ' . $shipments->count());

            // Initialize an array to hold the route details for the shipments
            $routeDetailsForShipments = [];

            foreach ($shipments as $shipment) {
                // Fetch related route optimizations for the shipment
                $routeOptimizations = RouteOptimization::where('shipments_id', $shipment->id)->get();

                // Check if there are any related route optimizations for this shipment
                if ($routeOptimizations->isEmpty()) {
                    continue; // Skip shipments that have no route optimizations
                }

                foreach ($routeOptimizations as $routeOptimization) {
                    // Fetch related route details for each route optimization
                    $routeDetails = RouteDetail::where('route_optimization_id', $routeOptimization->id)->get();

                    foreach ($routeDetails as $routeDetail) {
                        // Add the route details to the response array
                        $routeDetailsForShipments[] = $routeDetail;
```

```
                    }
                }
            }

            // Log the success message and the time taken to fetch the route details
            $endTime = microtime(true);
            $executionTime = round($endTime - $startTime, 2); // in seconds
            Log::info("Route details fetched successfully. Time taken: {$executionTime} seconds.");

            // Return only the route details
            return response()->json($routeDetailsForShipments);

        } catch (\Exception $e) {
            // Log any errors that occur
            Log::error('Error fetching route details: ' . $e->getMessage());
            Log::error('Error stack trace: ' . $e->getTraceAsString());

            // Return an error response
            return response()->json(['error' => 'An error occurred while fetching route details.'], 500);
        }
    }

}
```

FILE: app/Http/Controllers/PostController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Post;
use App\Models\Notification;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Facades\Log;
use App\Integrations\files\CloudinaryImageClient; // Use the custom Cloudinary client

class PostController extends Controller
{

    protected $cloudinaryClient;

    public function __construct()
    {
        // Initialize custom Cloudinary image client
        $this->cloudinaryClient = new CloudinaryImageClient();
    }
    public function getFeedPosts(Request $request)
    {
        try {
            $posts = Post::with([
                'author:id,first_name,last_name,phone_number,email,language,profile_picture,banner_img,headline,about',
                'comments.user:id,first_name,last_name,phone_number,email,language,profile_picture,banner_img,headline,about',
                'likes'
            ])
            ->orderBy('created_at', 'desc')
            ->get();


            return response()->json($posts, 200);
        } catch (\Exception $error) {
            Log::error("Error in getFeedPosts controller: ", ['error' => $error]);
            return response()->json(['message' => 'Server error'], 500);
        }
```

```
    }

    public function createPost(Request $request)
    {
        try {
            // Validate the incoming request
            $request->validate([
                'content' => 'required|string',
                'image' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
            ]);

            $imageUrl = null;

            // Check if an image file is provided
            if ($request->hasFile('image')) {
                $uploadedFile = $request->file('image');
                $mimeType = $uploadedFile->getClientMimeType();
                $base64Contents = base64_encode($uploadedFile->getContent());

                // Upload image to Cloudinary using the custom Cloudinary client
                $uploadResult = $this->cloudinaryClient->uploadImage($mimeType, $base64Contents);

                // Check the upload result
                if ($uploadResult->isSuccess) {
                    $imageUrl = $uploadResult->secureUrl;
                    Log::info('Post image uploaded to Cloudinary', ['image_url' => $imageUrl]);
                } else {
                    // If upload failed, throw an exception with the message from Cloudinary
                    throw new \Exception("Cloudinary image upload failed: " . $uploadResult->msg);
                }
            }

            // Create the new post in the database
            $newPost = Post::create([
                'author_id' => $request->user()->id,
                'content' => $request->input('content'),
                'image' => $imageUrl,
            ]);

            // Return the newly created post along with its author relationship
            return response()->json($newPost->load('author'), 201);
        } catch (\Exception $error) {
            // Log the exception for debugging
            Log::error("Error in createPost controller: ", [
                'error' => $error->getMessage(),
                'stack' => $error->getTraceAsString(),
            ]);

            // Return a generic server error response
            return response()->json(['message' => 'Server error'], 500);
        }
    }


    public function deletePost(Request $request, $id)
    {
        try {
            $post = Post::findOrFail($id);

            if ($post->author_id !== $request->user()->id) {
                return response()->json(['message' => 'You are not authorized to delete this post'], 403);
            }

            // Delete the image from storage if it exists
            if ($post->image) {
                $imagePath = str_replace('/storage/', '', $post->image);
```

```
                Storage::disk('public')->delete($imagePath);
            }

            $post->delete();

            return response()->json(['message' => 'Post deleted successfully'], 200);
        } catch (\Exception $error) {
            Log::error("Error in deletePost controller: ", ['error' => $error]);
            return response()->json(['message' => 'Server error'], 500);
        }
    }

    public function getPostById(Request $request, $id)
    {
        try {
            $post = Post::with(['author:id,first_name,last_name,phone_number,email,language,profile_picture,banner_img,headline,about',
'comments.user:id,first_name,last_name,phone_number,email,language,profile_picture,banner_img,headline,about'])
                ->findOrFail($id);

            return response()->json($post, 200);
        } catch (\Exception $error) {
            Log::error("Error in getPostById controller: ", ['error' => $error]);
            return response()->json(['message' => 'Server error'], 500);
        }
    }

    public function createComment(Request $request, $id)
    {
        try {
            $request->validate(['content' => 'required|string']);
            $post = Post::findOrFail($id);

            $comment = $post->comments()->create([
                'user_id' => $request->user()->id,
                'content' => $request->input('content'),
            ]);

            // Create a notification if the comment owner is not the post owner
            if ($post->author_id !== $request->user()->id) {
                Notification::create([
                    'recipient_id' => $post->author_id,
                    'type' => 'comment',
                    'related_user' => $request->user()->id,
                    'related_post' => $id,
                ]);
            }

            return response()->json($comment-
>load('user:id,first_name,last_name,phone_number,email,language,profile_picture,banner_img,headline,about'), 201);
        } catch (\Exception $error) {
            Log::error("Error in createComment controller: ", ['error' => $error]);
            return response()->json(['message' => 'Server error'], 500);
        }
    }

    public function likePost(Request $request, $id)
    {
        try {
            $post = Post::findOrFail($id);
            $userId = $request->user()->id;

            // Get the current likes for the post
            $likes = $post->likes()->pluck('user_id')->toArray();

            if (in_array($userId, $likes)) {
                // If the user already liked, remove like
```

```
                $post->likes()->sync(array_diff($likes, [$userId]));
            } else {
                // Otherwise, add like
                $post->likes()->sync([...$likes, $userId]);

                // Create a notification if the post owner is not the user who liked
                if ($post->author_id !== $userId) {
                    Notification::create([
                        'recipient_id' => $post->author_id,
                        'type' => 'like',
                        'related_user' => $userId,
                        'related_post' => $id,
                    ]);
                }
            }

            return response()->json($post->load('likes'), 200);
        } catch (\Exception $error) {
            Log::error("Error in likePost controller: ", ['error' => $error]);
            return response()->json(['message' => 'Server error'], 500);
        }
    }

}
```

FILE: app/Http/Controllers/ProductController.php

```
<?php
namespace App\Http\Controllers;

use App\Models\Product;
use App\Models\Category;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\Log;

class ProductController extends Controller
{
    public function fetchProductById($id)
    {
        try {
            // Fetch product along with its categories and supplier (using eager loading)
            $product = Product::with(['categories', 'supplier'])
                ->findOrFail($id); // This will throw a ModelNotFoundException if the product is not found

            return response()->json($product, 200); // Return the product data as JSON response
        } catch (ModelNotFoundException $e) {
            // Return a 404 error if product not found
            return response()->json([
                'error' => 'Product not found'
            ], 404);
        }
    }
    /**
     * Get all products
     */
    public function getAllProducts()
    {
        try {
            $products = Product::all();
            return response()->json(['products' => $products]);
        } catch (\Exception $e) {
            return response()->json(['message' => 'Server error', 'error' => $e->getMessage()], 500);
        }
```

```
    }

    /**
     * Get featured products from cache or DB
     */
    public function getFeaturedProducts()
    {
        try {
            // First check Redis cache
            $featuredProducts = Cache::get('featured_products');

            if ($featuredProducts) {
                return response()->json(json_decode($featuredProducts));
            }

            // If not in Redis, fetch from database
            $featuredProducts = Product::where('isFeatured', true)->get();

            if ($featuredProducts->isEmpty()) {
                return response()->json(['message' => 'No featured products found'], 404);
            }

            // Store in Redis for future requests
            Cache::put('featured_products', $featuredProducts->toJson(), now()->addMinutes(10));

            return response()->json($featuredProducts);
        } catch (\Exception $e) {
            return response()->json(['message' => 'Server error', 'error' => $e->getMessage()], 500);
        }
    }


    /**
     * Get recommended products (random sample)
     */
    public function getRecommendedProducts()
    {
        try {
            $products = Product::inRandomOrder()->limit(4)->get();
            return response()->json($products);
        } catch (\Exception $e) {
            return response()->json(['message' => 'Server error', 'error' => $e->getMessage()], 500);
        }
    }

    /**
     * Get products by category
     */

     public function getProductsByCategory(Request $request)
    {
        try {
            // Get the categories array from the request input
            $categoriesInput = $request->input('categories');

            // Check if the categories input is provided and is an array
            if (empty($categoriesInput) || !is_array($categoriesInput)) {
                // Log the error
                Log::warning('Categories input is empty or not an array');

                // Return a response with a specific message for invalid categories input
                return response()->json(['message' => 'Categories input must be a non-empty array'], 400);
            }

            // Log the incoming categories input
            Log::info('Searching for products by categories', ['categories' => $categoriesInput]);
```

```
        // Normalize the category input (lowercase all category names)
        $categoriesInput = array_map('strtolower', $categoriesInput);

        // Search for categories where the category_name or description contains any of the provided categories (case-insensitive)
        $categories = Category::where(function ($query) use ($categoriesInput) {
            foreach ($categoriesInput as $category) {
                $query->orWhereRaw('LOWER(category_name) LIKE ?', ['%' . $category . '%'])
                    ->orWhereRaw('LOWER(description) LIKE ?', ['%' . $category . '%']);
            }
        })->get();

        // Log the found categories
        Log::info('Categories found', ['categories' => $categories->pluck('category_name')]);

        // Check if any categories were found
        if ($categories->isEmpty()) {
            Log::warning('No categories found', ['categories' => $categoriesInput]);
            return response()->json(['message' => 'No categories found for the given search terms'], 404);
        }

        // Collect all products related to these categories using the pivot table
        $products = $categories->flatMap(function ($category) {
            return $category->products; // Fetch related products for each category
        });

        // Log the found products
        Log::info('Products found for categories', ['products' => $products->pluck('name')]);

        // Return the unique products (to avoid duplicates)
        return response()->json(['products' => $products->unique('id')]);

    } catch (\Exception $e) {
        // Log the general error
        Log::error('Error fetching products by categories', [
            'error' => $e->getMessage(),
            'stack' => $e->getTraceAsString()
        ]);

        // Handle any potential errors
        return response()->json(['message' => 'Server error', 'error' => $e->getMessage()], 500);
    }
}



/**
 * Toggle the featured status of a product
 */
public function toggleFeaturedProduct($id)
{
    try {
        $product = Product::findOrFail($id);
        $product->isFeatured = !$product->isFeatured;
        $product->save();

        // Update cache after toggling featured status
        $this->updateFeaturedProductsCache();

        return response()->json($product);
    } catch (\Exception $e) {
        return response()->json(['message' => 'Server error', 'error' => $e->getMessage()], 500);
    }
}

/**
```

```
     * Update featured products cache
     */
    private function updateFeaturedProductsCache()
    {
        try {
            $featuredProducts = Product::where('isFeatured', true)->get();
            Cache::put('featured_products', $featuredProducts->toJson(), now()->addMinutes(10));
        } catch (\Exception $e) {
            \Log::error('Error updating featured products cache: ' . $e->getMessage());
        }
    }
}
```

FILE: app/Http/Controllers/ProfileController.php

```
<?php

namespace App\Http\Controllers;

use App\Http\Requests\ProfileUpdateRequest;
use Illuminate\Contracts\Auth\MustVerifyEmail;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Redirect;
use Inertia\Inertia;
use Inertia\Response;

class ProfileController extends Controller
{
    /**
     * Display the user's profile form.
     */
    public function edit(Request $request): Response
    {
        return Inertia::render('Profile/Edit', [
            'mustVerifyEmail' => $request->user() instanceof MustVerifyEmail,
            'status' => session('status'),
        ]);
    }

    /**
     * Update the user's profile information.
     */
    public function update(ProfileUpdateRequest $request): RedirectResponse
    {
        $request->user()->fill($request->validated());

        if ($request->user()->isDirty('email')) {
            $request->user()->email_verified_at = null;
        }

        $request->user()->save();

        return Redirect::route('profile.edit');
    }

    /**
     * Delete the user's account.
     */
    public function destroy(Request $request): RedirectResponse
    {
        $request->validate([
            'password' => ['required', 'current-password'],
```

```
        ]);

        $user = $request->user();

        Auth::logout();

        $user->delete();

        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return Redirect::to('/');
    }
}
```

FILE: app/Http/Controllers/RatingController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Rating;
use App\Models\Product;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;

class RatingController extends Controller
{
    // Fetch ratings for a specific product
    public function getProductRatings($productId)
    {
        // Retrieve all ratings for a product
        $ratings = Rating::where('products_id', $productId)->get();

        return response()->json($ratings);
    }

    // Store a new rating/feedback for a product
    public function storeRating(Request $request, $productId)
    {
        // Validate input
        $validator = Validator::make($request->all(), [
            'rating_value' => 'required|integer|min:1|max:5',
            'feedback' => 'nullable|string|max:1000',
        ]);

        if ($validator->fails()) {
            return response()->json(['error' => $validator->errors()], 400);
        }

        // Store the rating
        $rating = new Rating();
        $rating->rating_value = $request->input('rating_value');
        $rating->feedback = $request->input('feedback');
        $rating->products_id = $productId;
        $rating->shipments_id = $request->input('shipments_id'); // Assuming you have a shipment_id if applicable
        $rating->date_created = now(); // Add the current timestamp
        $rating->save();

        return response()->json(['message' => 'Rating submitted successfully', 'rating' => $rating]);
    }
}
```

FILE: app/Http/Controllers/StreamController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log; // Import the Log facade
use App\Models\Stream;
use App\Models\Block;
use App\Models\User;
use App\Models\Follow;
use Agence104\LiveKit\RoomServiceClient;
use Agence104\LiveKit\IngressServiceClient;
use App\Integrations\files\CloudinaryImageClient; // Use the custom Cloudinary client
use Livekit\IngressAudioOptions;
use Livekit\IngressVideoOptions;
use Livekit\IngressVideoEncodingPreset;
use Livekit\TrackSource;
use Livekit\IngressInput;
use Livekit\IngressAudioEncodingPreset;


class StreamController extends Controller
{
    protected $cloudinaryClient;
    protected $roomService;
    protected $ingressServiceClient; // Changed to IngressServiceClient

    public function __construct()
    {
        $this->roomService = new RoomServiceClient(
            env('LIVEKIT_URL'),
            env('LIVEKIT_API_KEY'),
            env('LIVEKIT_API_SECRET')
        );
        $this->cloudinaryClient = new CloudinaryImageClient();

        $this->ingressServiceClient = new IngressServiceClient(env('LIVEKIT_URL'),env('LIVEKIT_API_KEY'),env('LIVEKIT_API_SECRET')); //
Updated to IngressServiceClient
    }

    public function createStream(Request $request)
    {
        $self = $request->user(); // Get the authenticated user
        Log::info("User {$self->id} is attempting to create a new stream");

        // Validate the request input for title, thumbnail (file upload), and chat settings
        try {
            $validatedData = $request->validate([
                'title' => 'required|string|max:255',
                'thumbnail' => 'nullable|image|mimes:jpeg,png,jpg,gif,webp|max:2048',  // Ensure the thumbnail is a valid image and limit
file size
                'isChatEnabled' => 'nullable|string|in:true,false', // Validate as a string with 'true' or 'false'
                'isChatDelayed' => 'nullable|string|in:true,false', // Validate as a string with 'true' or 'false'
                'isChatFollowersOnly' => 'nullable|string|in:true,false', // Validate as a string with 'true' or 'false'
            ]);
        } catch (\Illuminate\Validation\ValidationException $e) {
            Log::error("Validation failed for user {$self->id}. Errors: " . json_encode($e->errors()));
            return response()->json(['message' => 'Invalid input data'], 422);
        }

        $title = $validatedData['title'];

        // Handle the thumbnail upload if present
```

```
        $thumbnailPath = 'default-thumbnail.jpg';  // Default thumbnail if no file is uploaded
        if ($request->hasFile('thumbnail')) {
            $uploadedFile = $request->file('thumbnail'); // Get the uploaded file

            // Get MIME type and content of the uploaded file
            $mimeType = $uploadedFile->getClientMimeType();
            $base64Contents = base64_encode($uploadedFile->get());

            try {
                // Upload image using custom Cloudinary client (ensure $this->cloudinaryClient is configured properly)
                $uploadResult = $this->cloudinaryClient->uploadImage($mimeType, $base64Contents);

                // If the upload is successful, we get the secure URL
                if ($uploadResult->isSuccess) {
                    $thumbnailPath = $uploadResult->secureUrl; // Use the Cloudinary URL
                    Log::info('Product image uploaded to Cloudinary', ['image_url' => $thumbnailPath]);
                } else {
                    throw new \Exception("Cloudinary image upload failed: " . $uploadResult->msg);
                }
            } catch (\Exception $e) {
                Log::error('Error uploading image to Cloudinary for user ' . $self->id . ': ' . $e->getMessage() . "\n" . $e-
>getTraceAsString());
                return response()->json(['message' => 'Failed to upload thumbnail image'], 500);
            }
        }

        // Set the correct input type (replace 0 with the appropriate value)
        $inputType = IngressInput::RTMP_INPUT;  // Assuming 1 is for regular video and audio input
        $name = $self->first_name . ' ' . $self->last_name;
        $roomName = $self->id;  // The room name will be the user ID (this could be customized)
        $participantName = $self->first_name . ' ' . $self->last_name;
        $participantIdentity = $self->id;
        $bypassTranscoding = false;  // Set to false if you don't want to bypass transcoding

        // Create the audio configuration using the Livekit\IngressAudioOptions class
        $audioConfig = new IngressAudioOptions([
            'source' => TrackSource::MICROPHONE, // Audio source (MICROPHONE constant is an integer)
            'preset' => IngressAudioEncodingPreset::OPUS_STEREO_96KBPS, // OPUS audio encoding preset for stereo audio at 96 kbps
        ]);
        // Video and audio configurations
        $videoConfig = new IngressVideoOptions([
            'source' => TrackSource::CAMERA,  // Video source (CAMERA constant is an integer)
            'preset' => IngressVideoEncodingPreset::H264_1080P_30FPS_3_LAYERS, // Video encoding preset (this should also be an integer)
        ]);



        // Create the ingress using the updated IngressServiceClient
        try {
            $ingress = $this->ingressServiceClient->createIngress(
                $inputType,
                $name,
                $roomName,
                $participantIdentity,
                $participantName,
                $audioConfig,  // Pass the audio configuration
                $videoConfig,  // Pass the video configuration
                $bypassTranscoding,  // Pass the bypass transcoding flag
            );
        } catch (\Exception $e) {
            Log::error("Failed to create ingress for user {$self->id}: " . $e->getMessage() . "\n" . $e->getTraceAsString());
            return response()->json(['message' => 'Failed to create stream ingress.'], 500);
        }


        // Check if ingress was successfully created
```

```
        if (!$ingress || !$ingress->getUrl() || !$ingress->getStreamKey()) {
            Log::error("Ingress creation failed for user {$self->id}. Ingress data: " . json_encode($ingress));
            return response()->json(['message' => 'Failed to create ingress'], 500);
        }


        // Extract chat settings from the validated data and convert them to boolean
        $isChatEnabled = filter_var($validatedData['isChatEnabled'] ?? 'true', FILTER_VALIDATE_BOOLEAN); // Default to true if not
provided
        $isChatDelayed = filter_var($validatedData['isChatDelayed'] ?? 'false', FILTER_VALIDATE_BOOLEAN); // Default to false if not
provided
        $isChatFollowersOnly = filter_var($validatedData['isChatFollowersOnly'] ?? 'false', FILTER_VALIDATE_BOOLEAN); // Default to false
if not provided

        // Store the stream information in the database
        try {
            $stream = Stream::create([
                'user_id' => $self->id,
                'title' => $title,  // Use the title from the request input
                'thumbnail' => $thumbnailPath,  // Save the Cloudinary URL or default
                'ingressId' => $ingress->getIngressId(),  // Use the getter method for ingressId
                'serverUrl' => $ingress->getUrl(),  // Use the getter method for url
                'streamKey' => $ingress->getStreamKey(),  // Use the getter method for streamKey
                'isLive' => true,
                'isChatEnabled' => $isChatEnabled,
                'isChatDelayed' => $isChatDelayed,
                'isChatFollowersOnly' => $isChatFollowersOnly,
            ]);
            // Eager load the 'user' relationship for the created stream
            $stream->load('user');
        } catch (\Exception $e) {
            Log::error("Error saving stream data to the database for user {$self->id}: " . $e->getMessage() . "\n" . $e-
>getTraceAsString());
            return response()->json(['message' => 'Failed to save stream data'], 500);
        }

        Log::info("Stream created successfully for user {$self->id}, stream ID: {$stream->id}");

        return response()->json($stream);
    }


    public function stopStream(Request $request)
    {
        $self = $request->user(); // Get the authenticated user
        $streamId = $request->input('stream_id'); // Get the stream_id from the request, if available

        Log::info("User {$self->id} is attempting to stop a stream");

        // Determine the stream to stop
        $stream = null;

        if ($streamId) {
            // If stream_id is provided, find the stream by stream_id
            $stream = Stream::find($streamId);
            if (!$stream) {
                Log::warning("Stream with ID {$streamId} not found.");
                return response()->json(['message' => 'Stream not found'], 400);
            }
        } else {
            // If stream_id is not provided, find the active stream for the authenticated user
            $stream = Stream::where('user_id', $self->id)->where('isLive', true)->first();
            if (!$stream) {
                Log::warning("User {$self->id} does not have an active stream.");
                return response()->json(['message' => 'No active stream found'], 400);
            }
```

```
        }

        try {
            // Mark the stream as not live
            $stream->isLive = false;
            $stream->save();

            // Delete the ingress associated with the stream
            $this->ingressServiceClient->deleteIngress($stream->ingressId);

            Log::info("Stream stopped successfully for user {$self->id}, Stream ID: {$stream->id}");

            return response()->json(['message' => 'Stream stopped successfully']);
        } catch (\Exception $e) {
            Log::error("Error stopping stream for user {$self->id}, Stream ID: {$stream->id}. Error: " . $e->getMessage());
            return response()->json(['message' => 'Failed to stop stream', 'error' => $e->getMessage()], 500);
        }
    }

    // Method to search streams based on a term
    public function searchStreams(Request $request)
    {
        try {
            $term = $request->input('term', ''); // Get search term, default to empty string
            $userId = $request->user()->id; // Get authenticated user's ID

            Log::info("User {$userId} is searching streams with term: '{$term}'.");

            // Initialize query
            $query = Stream::query();

            if ($userId) {
                // If the user is authenticated, exclude their own streams
                $query->where('user_id', '!=', $userId);
            }

            // Apply search filters
            $query->where(function ($q) use ($term) {
                $q->where('title', 'like', '%' . $term . '%')
                  ->orWhereHas('user', function ($q) use ($term) {
                      $q->where('first_name', 'like', '%' . $term . '%')
                        ->orWhere('last_name', 'like', '%' . $term . '%');
                  });
            });

            // Apply ordering by isLive status (desc) and updatedAt timestamp (desc)
            $streams = $query->with(['user' => function ($query) {
                    $query->select('id', 'first_name', 'last_name'); // Include user details
                }])
                ->orderBy('is_live', 'desc')
                ->orderBy('updated_at', 'desc')
                ->get([
                    'id',
                    'user_id',
                    'title',
                    'is_live',
                    'thumbnail',
                    'updated_at',
                ]);

            Log::info("Stream search completed for user {$userId}, found " . count($streams) . " results.");

            return response()->json($streams);
        } catch (\Exception $e) {
            Log::error("An error occurred during stream search: {$e->getMessage()}");
            return response()->json(['message' => 'Internal error: ' . $e->getMessage()], 500);
```

```
        }
    }

    // Method to get recommended users
    public function getRecommendedUsers(Request $request)
    {
        try {
            $self = $request->user();
            $userId = $self ? $self->id : null;

            Log::info("User {$userId} is requesting recommended users.");

            // Initialize users array
            $users = [];

            if ($userId) {
                // Get all users except the authenticated one
                $users = User::where('id', '!=', $userId)
                    ->with(['stream' => function ($query) {
                        $query->select('isLive');
                    }])
                    ->orderBy('created_at', 'desc')
                    ->get();

                // Get the ids of the users the authenticated user is following
                $followingIds = Follow::where('follower_id', $userId)
                    ->pluck('following_id')
                    ->toArray();

                // Get the ids of the users that have blocked the authenticated user
                $blockedIds = Block::where('blocker_id', $userId)
                    ->pluck('blocked_id')
                    ->toArray();

                // Filter out users that are followed or blocked by the authenticated user
                $users = $users->filter(function ($user) use ($followingIds, $blockedIds) {
                    return !in_array($user->id, $followingIds) && !in_array($user->id, $blockedIds);
                });

                Log::info("Recommended users filtered for user {$userId}, found " . count($users) . " users.");
            } else {
                // If no user is authenticated, return all users
                $users = User::with(['stream' => function ($query) {
                    $query->select('isLive');
                }])
                    ->orderBy('created_at', 'desc')
                    ->get();

                Log::info("No user authenticated, returning all recommended users, found " . count($users) . " users.");
            }

            return response()->json($users);
        } catch (\Exception $e) {
            Log::error("An error occurred while fetching recommended users: {$e->getMessage()}");
            return response()->json(['message' => 'Internal error: ' . $e->getMessage()], 500);
        }
    }

    // Method to fetch streams for the authenticated user or all streams if not authenticated
    public function getStreams(Request $request)
    {
        try {
            $self = $request->user();
            $userId = $self ? $self->id : null;

            Log::info("User {$userId} is requesting streams.");
```

```
        // If the user is authenticated, fetch streams excluding the blocked ones
        if ($userId) {
            $streams = Stream::whereDoesntHave('user.blocks', function ($query) use ($userId) {
                $query->where('blocked_id', $userId);
            })
            ->with(['user'])  // Eager load the user relationship to avoid N+1 issues
            ->orderBy('isLive', 'desc')
            ->orderBy('updated_at', 'desc')
            ->get();
            Log::info("Streams fetched for authenticated user {$userId}.");
        } else {
            // If no user is authenticated, fetch all streams
            $streams = Stream::with(['user'])  // Eager load user relationship
                ->orderBy('isLive', 'desc')
                ->orderBy('updated_at', 'desc')
                ->get();
            Log::info("Streams fetched for guest user.");
        }

        return response()->json($streams);
    } catch (\Exception $e) {
        Log::error("An error occurred while fetching streams: {$e->getMessage()}");
        return response()->json(['message' => 'Something went wrong: ' . $e->getMessage()], 500);
    }
}

// Method to update the stream for the authenticated user
public function updateStream(Request $request)
{
    try {
        // Get the authenticated user from the request
        $user = $request->user();

        Log::info("User {$user->id} is attempting to update their stream.");

        // Fetch the stream for the authenticated user
        $selfStream = Stream::where('user_id', $user->id)->first();

        if (!$selfStream) {
            Log::warning("Stream not found for user {$user->id}.");
            return response()->json(['message' => 'Stream not found'], 404);
        }

        // Validate and prepare the data
        $validData = $request->only([
            'thumbnail',
            'title',
            'isChatEnabled',
            'isChatFollowersOnly',
            'isChatDelayed',
        ]);

        // Log the data being updated
        Log::info("User {$user->id} is updating the stream with data: ", $validData);

        // Update the stream in the database
        $updatedStream = $selfStream->update($validData);

        if (!$updatedStream) {
            Log::error("Failed to update the stream for user {$user->id}.");
            return response()->json(['message' => 'Failed to update the stream'], 500);
        }

        Log::info("Stream updated successfully for user {$user->id}.");
```

```
            return response()->json(['message' => 'Stream updated successfully', 'stream' => $selfStream]);
        } catch (\Exception $e) {
            // Log the exception error
            Log::error("An error occurred while updating stream for user {$request->user()->id}: {$e->getMessage()}");

            return response()->json(['message' => 'Internal Error'], 500);
        }
    }
}
```

FILE: app/Http/Controllers/StreamMessageController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\StreamMessage;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;
use Illuminate\Support\Facades\Log; // Add the Log facade for logging
use App\Jobs\BroadcastStreamMessage; // Import the job

class StreamMessageController extends Controller
{
    /**
     * Send a message to a viewer in a stream.
     *
     * @param Request $request
     * @return \Illuminate\Http\Response
     */
    public function sendMessage(Request $request)
    {
        // Log the incoming request data
        Log::info('sendMessage called', ['request' => $request->all()]);

        // Validate the incoming request
        $validator = Validator::make($request->all(), [
            'message' => 'required|string|max:255',
            'creator_id' => 'required|exists:users,id',
            'viewer_id' => 'required|exists:users,id',
            'stream_id' => 'required|exists:streams,id',
        ]);

        // Log the validation status
        if ($validator->fails()) {
            Log::warning('sendMessage validation failed', [
                'errors' => $validator->errors()
            ]);
            return response()->json([
                'success' => false,
                'errors' => $validator->errors(),
            ], 400);
        }

        // Log successful validation
        Log::info('sendMessage validation passed');

        // Create a new StreamMessage
        $streamMessage = StreamMessage::create([
            'message' => $request->message,
            'creator_id' => $request->creator_id,
            'viewer_id' => $request->viewer_id,
            'stream_id' => $request->stream_id,
        ]);
```

```
        $streamMessage = StreamMessage::with(['creator', 'viewer', 'stream'])->find($streamMessage->id);

        dispatch(new BroadcastStreamMessage($streamMessage));

        // Log the created message
        Log::info('sendMessage created new message', ['message' => $streamMessage]);

        // Return the created message
        return response()->json($streamMessage);
    }

    /**
     * Get all messages sent by a specific creator.
     *
     * @param Request $request
     * @return \Illuminate\Http\Response
     */
    public function getMessagesByCreator(Request $request, $creatorId)
    {

        // Get all messages sent by the creator (creator_id)
        $messages = StreamMessage::where('creator_id', $creatorId)
        ->with(['creator', 'viewer'])  // Eager load both creator and viewer relationships
        ->get();

        // Log the retrieved messages
        Log::info('getMessagesByCreator retrieved messages', ['messages' => $messages]);

        // Return the retrieved messages
        return response()->json($messages);
    }
}
```

FILE: app/Http/Controllers/StripeController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Stripe\Stripe;
use Stripe\PaymentIntent;
use Log; // Import the Log facade

class StripeController extends Controller
{
    // Constructor to set your Stripe Secret Key
    public function __construct()
    {
        Stripe::setApiKey(env('STRIPE_SECRET_KEY'));
    }

    // Create a payment intent
    public function createPaymentIntent(Request $request)
    {
        try {
            // Log incoming request data
            Log::info('Creating payment intent', [
                'amount' => $request->amount,
                'currency' => $request->currency,
            ]);

            // You can pass dynamic amount from the front-end
```

```
        $amount = (int) ($request->amount * 100); // Convert to cents (e.g., 275.79 -> 27579)
        $currency = $request->currency;

        // Create the PaymentIntent
        $paymentIntent = PaymentIntent::create([
            'amount' => $amount,
            'currency' => $currency, // or another currency
            'metadata' => ['integration_check' => 'accept_a_payment'],
        ]);

        // Log the successful creation of the payment intent
        Log::info('Payment intent created successfully', [
            'paymentIntentId' => $paymentIntent->id,
            'clientSecret' => $paymentIntent->client_secret,
        ]);

        // Return the client secret to the front-end
        return response()->json([
            'clientSecret' => $paymentIntent->client_secret
        ]);
    } catch (\Exception $e) {
        // Log the exception if there is any error
        Log::error('Error creating payment intent', [
            'error_message' => $e->getMessage(),
            'error_stack' => $e->getTraceAsString(), // Optionally log the stack trace
        ]);

        // Return error response
        return response()->json(['error' => $e->getMessage()], 500);
    }
}

public function createPaymentIntentGift(Request $request)
{
    try {
        // Log incoming request data
        Log::info('Creating payment intent', [
            'amount' => $request->amount,
            'currency' => $request->currency,
        ]);

        // You can pass dynamic amount from the front-end
        $amount = (int) ($request->amount * 100); // Convert to cents (e.g., 275.79 -> 27579)
        $currency = $request->currency;

        // Create the PaymentIntent
        $paymentIntent = PaymentIntent::create([
            'amount' => $amount,
            'currency' => $currency, // or another currency
            'metadata' => ['integration_check' => 'accept_a_payment'],
        ]);

        // Log the successful creation of the payment intent
        Log::info('Payment intent created successfully', [
            'paymentIntentId' => $paymentIntent->id,
            'clientSecret' => $paymentIntent->client_secret,
        ]);

        // Return the client secret to the front-end
        return response()->json([
            'clientSecret' => $paymentIntent->client_secret
        ]);
    } catch (\Exception $e) {
        // Log the exception if there is any error
        Log::error('Error creating payment intent', [
            'error_message' => $e->getMessage(),
```

```
                'error_stack' => $e->getTraceAsString(), // Optionally log the stack trace
            ]);

            // Return error response
            return response()->json(['error' => $e->getMessage()], 500);
        }
    }
}
```

FILE: app/Http/Controllers/TwoFactorController.php

```
<?php
namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;

class TwoFactorController extends Controller
{
    public function sendCode(Request $request)
    {
        // Use request()->user() to get the logged-in user
        $user = $request->user();

        // Check if the user has a phone number
        if (!$user->phone_number) {
            return response()->json(['message' => 'Phone number is not set.'], 400);
        }

        // Send the 2FA code to the user's phone number
        $user->sendTwoFactorCode();

        return response()->json(['message' => 'Verification code sent to your phone.']);
    }

    public function verifyCode(Request $request)
    {
        // Validate the incoming request to ensure the code is numeric and 6 digits long
        $request->validate([
            'code' => 'required|numeric|digits:6',
        ]);

        // Use request()->user() to get the logged-in user
        $user = $request->user();
        $code = $request->input('code');

        // Verify the entered code
        if ($user->verifyTwoFactorCode($code)) {
            return response()->json(['message' => 'Code verified successfully.']);
        }

        return response()->json(['message' => 'Invalid or expired code.'], 400);
    }
}
```

FILE: app/Http/Controllers/UserController.php

```
<?php

namespace App\Http\Controllers;
```

```
use App\Models\User;
use App\Models\UserConnection;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Facades\Validator;

class UserController extends Controller
{
    public function getSuggestedConnections(Request $request)
    {
        try {
            // Ensure user is authenticated
            $currentUser = $request->user();

            // Fetch all users except the current user
            $allUsers = User::where('id', '!=', $currentUser->id)
                ->select('id',
'first_name','last_name','phone_number','email','language','profile_picture','banner_img','headline','about')
                ->limit(30)
                ->get();

            return response()->json($allUsers);
        } catch (\Exception $error) {
            \Log::error("Error in getSuggestedConnections controller: ", ['error' => $error]);
            return response()->json(['message' => 'Server error'], 500);
        }
    }
    public function updateHeadlineAndAbout(Request $request)
    {
        // Validate the incoming request data
        $validator = Validator::make($request->all(), [
            'headline' => 'nullable|string|max:255',
            'about' => 'nullable|string|max:1000',
        ]);

        // If validation fails, return an error response
        if ($validator->fails()) {
            return response()->json([
                'success' => false,
                'errors' => $validator->errors(),
            ], 400);
        }

        // Get the currently authenticated user
        $user = $request->user();

        // Update the user's headline and about attributes
        $user->update([
            'headline' => $request->input('headline', $user->headline),
            'about' => $request->input('about', $user->about),
        ]);

        // Return a success response
        return response()->json($user);
    }

    public function setLanguage(Request $request)
    {
        // Validate the incoming request data
        $validator = Validator::make($request->all(), [
            'language' => 'required|string|size:2', // ISO 639-1 codes should be exactly 2 characters long
        ]);

        // If validation fails, return an error response
        if ($validator->fails()) {
            return response()->json([
```

```
                'success' => false,
                'errors' => $validator->errors(),
            ], 400);
        }

        // Get the currently authenticated user
        $user = $request->user();

        // Update the user's language attribute
        $user->update([
            'language' => $request->input('language', $user->language),
        ]);

        // Return a success response
        return response()->json($user);
    }

}
```

FILE: app/Http/Controllers/UserProfileController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\User;
use App\Models\UserAddress;
use App\Models\Payment;
use App\Models\Invoice;
use App\Models\OrderItem;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Auth;
use App\Http\Controllers\Controller;
use Illuminate\Validation\Rule;
use Illuminate\Support\Facades\Log;

class UserProfileController extends Controller
{

    // Update user details (first name, last name, email, etc.)
    public function updateDetails(Request $request)
    {
        $user = $request->user();

        // Validate the request data
        $validated = $request->validate([
            'first_name' => 'required|string|max:255',
            'last_name' => 'required|string|max:255',
            'email' => [
                'required',
                'email',
                'max:255',
                Rule::unique('users')->ignore($user->id),
            ],
            'phone_number' => 'nullable|string|max:20',
            'language' => 'nullable|string|max:10',
        ]);

        // Update user details
        $user->update($validated);

        // Return updated user profile as JSON
```

```
        return response()->json($user);
    }

    // Update the user's password
    public function updatePassword(Request $request)
    {
        // Validate the password data
        $validated = $request->validate([
            'currentPassword' => 'required|string',
            'newPassword' => 'required|string|min:8', // Must be confirmed
        ]);

        $user = $request->user();

        // Check if the current password matches
        if (!Hash::check($validated['currentPassword'], $user->password)) {
            return response()->json(['error' => 'Current password is incorrect'], 400);
        }

        // Update the password
        $user->update([
            'password' => Hash::make($validated['newPassword']),
        ]);

        // Return success response
        return response()->json(['message' => 'Password updated successfully']);
    }

    /**
     * Display a listing of the user's addresses.
     *
     * @return \Illuminate\Http\Response
     */
    public function getUserAddresses(Request $request)
    {
        // Get the authenticated user
        $user = $request->user();

        // Get all the addresses for the authenticated user
        $addresses = $user->addresses;  // Assuming the relationship method on the User model is `addresses`

        // Return the addresses as a JSON response
        return response()->json($addresses);
    }

    /**
     * Store a newly created address for the authenticated user.
     *
     * @param \Illuminate\Http\Request $request
     * @return \Illuminate\Http\Response
     */
    public function storeUserAddress(Request $request)
    {
        // Validate the incoming request
        $validated = $request->validate([
            'address_line1' => 'required|string|max:255',
            'address_line2' => 'nullable|string|max:255',
            'city' => 'required|string|max:100',
            'state' => 'required|string|max:100',
            'postal_code' => 'required|string|max:20',
            'country' => 'required|string|max:100',
            'is_primary' => 'nullable|boolean',
        ]);

        // Get the authenticated user
        $user = $request->user();
```

```
        // Create the address for the user
        $address = $user->addresses()->create($validated);

        // Return a response with the created address
        return response()->json($address, 201);
    }

    /**
     * Update the specified address for the authenticated user.
     *
     * @param \Illuminate\Http\Request $request
     * @param int $id
     * @return \Illuminate\Http\Response
     */
    public function updateUserAddress(Request $request, $id)
    {
        // Validate the incoming request
        $validated = $request->validate([
            'address_line1' => 'required|string|max:255',
            'address_line2' => 'nullable|string|max:255',
            'city' => 'required|string|max:100',
            'state' => 'required|string|max:100',
            'postal_code' => 'required|string|max:20',
            'country' => 'required|string|max:100',
            'is_primary' => 'nullable|boolean',
        ]);

        // Get the authenticated user
        $user = $request->user();

        // Find the address by ID, ensuring it belongs to the authenticated user
        $address = $user->addresses()->findOrFail($id);

        // Update the address
        $address->update($validated);

        // Return a response with the updated address
        return response()->json($address);
    }

    public function getCurrentUser(Request $request)
{
    // Log the incoming request to verify who is making the request
    Log::info('getCurrentUser request started', ['user_id' => $request->user()->id]);

    // Get the authenticated user
    $user = $request->user();

    // Log user information before eager loading
    Log::info('Authenticated user fetched', [
        'user_id' => $user->id,
        'user_email' => $user->email,
        'user_roles' => $user->roles->pluck('role_name', 'description'),  // Pluck role names and descriptions
    ]);

    // Eager load the related roles and permissions
    $userWithDetails = $user->load([
        'roles.permissions',       // Eager load roles and their associated permissions
    ]);

    // Log after eager loading
    Log::info('User data with roles and permissions loaded', [
        'roles_count' => $userWithDetails->roles->count(),
    ]);
```

```
    // Format the roles and permissions for frontend expectations
    $userWithDetails->roles = $userWithDetails->roles->map(function ($role) {
        Log::info('Mapping role', ['role_name' => $role->role_name, 'description' => $role->description]);
        return [
            'role_name' => $role->role_name,
            'description' => $role->description, // Include role description
            'permissions' => $role->permissions->map(function ($permission) {
                Log::info('Mapping permission', ['permission_name' => $permission->permission_name, 'description' => $permission-
>description]);
                return [
                    'permission_name' => $permission->permission_name,
                    'description' => $permission->description, // Include permission description
                ];
            }),
        ];
    });

    // Log the roles and permissions after mapping
    Log::info('Roles and permissions after mapping', ['roles' => $userWithDetails->roles]);

    // Format the orders and include order_items, products, and payment for frontend expectations
    $userWithDetails->orders = $userWithDetails->orders->map(function ($order) {
        Log::info('Mapping order', [
            'order_id' => $order->id,
            'order_status' => $order->status,
            'order_total_amount' => $order->total_amount,
        ]);

        // Query the order items separately using the order_id
        $orderItems = OrderItem::where('orders_id', $order->id)->get();

        $formattedOrderItems = $orderItems->map(function ($orderItem) {
            Log::info('Mapping order item', [
                'order_item_id' => $orderItem->id,
                'quantity' => $orderItem->quantity,
                'total_amount' => $orderItem->total_amount,
                'product_id' => $orderItem->product_id,
            ]);

            // Include the product details for this order item
            $product = $orderItem->product;
            $productDetails = [
                'product_id' => $product->id,
                'product_name' => $product->name,
                'image' => $product->image,
                'description'=> $product->description,
                'price' => $product->price,
                'isFeatured' => $product->isFeatured,
                'categories' => $product->categories->map(function ($category) {
                    return [
                        'category_name' => $category->category_name,
                        'category_description' => $category->description,
                    ];
                }),
            ];

            return [
                'order_item_id' => $orderItem->id,
                'quantity' => $orderItem->quantity,
                'total_amount' => $orderItem->total_amount,
                'product' => $productDetails, // Include product details
            ];
        });

        // Query the associated payment for the order separately using order_id
        $paymentData = Payment::where('order_id', $order->id)->first();
```

```
        $formattedPaymentData = $paymentData ? [
            'payment_id' => $paymentData->id,
            'total_amount' => $paymentData->total_amount,
            'paid_amount' => $paymentData->paid_amount,
            'due_amount' => $paymentData->due_amount,
            'payment_status' => $paymentData->payment_status,
            'payment_method' => $paymentData->payment_method,
            'payment_date' => $paymentData->payment_date,
            'gateway' => $paymentData->gateway,
            'currency' => $paymentData->currency,
        ] : null;
        Log::info('Payment Data', ['payment' => $paymentData]);

        return [
            'id' => $order->id,
            'status' => $order->status,
            'order_date' => $order->order_date,
            'total_amount' => $order->total_amount,
            'tracking_number' => $order->tracking_number,
            'order_items' => $formattedOrderItems, // Include order items in the order data
            'payment' => $formattedPaymentData,    // Include associated payment for the order
        ];
    });

    // Log orders after mapping
    Log::info('Orders after mapping', ['orders' => $userWithDetails->orders]);

    // Map invoices with associated payments
    $invoices = Invoice::all()->map(function ($invoice) {
        Log::info('Mapping invoice', [
            'invoice_id' => $invoice->id,
            'customer_name' => $invoice->customer_name,
            'total_amount' => $invoice->total_amount,
            'paid_amount' => $invoice->paid_amount,
            'payment_status' => $invoice->payment_status,
        ]);

        // Query the associated payment using payment_id
        $paymentData = Payment::where('id', $invoice->payments_id)->first();

        $formattedPaymentData = $paymentData ? [
            'payment_id' => $paymentData->id,
            'payment_status' => $paymentData->payment_status,
            'payment_method' => $paymentData->payment_method,
            'total_amount' => $paymentData->total_amount,
            'paid_amount' => $paymentData->paid_amount,
            'due_amount' => $paymentData->due_amount,
        ] : null;

        return [
            'invoice_id' => $invoice->id,
            'customer_name' => $invoice->customer_name,
            'total_amount' => $invoice->total_amount,
            'paid_amount' => $invoice->paid_amount,
            'due_amount' => $invoice->due_amount,
            'payment_status' => $invoice->payment_status,
            'payment_method' => $invoice->payment_method,
            'created_date' => $invoice->created_date,
            'discount' => $invoice->discount,
            'currency' => $invoice->currency,
            'payment' => $formattedPaymentData, // Include associated payment data
        ];
    });

    // Log invoices after mapping
```

```
    Log::info('Invoices after mapping', ['invoices' => $invoices]);

    // Log final response
    Log::info('Returning response with user details', [
        'user' => [
            'id' => $userWithDetails->id,
            'first_name' => $userWithDetails->first_name,
            'last_name' => $userWithDetails->last_name,
            'email' => $userWithDetails->email,
            'phone_number' => $userWithDetails->phone_number,
            'language' => $userWithDetails->language,
        ],
        'roles' => $userWithDetails->roles,
        'permissions' => $userWithDetails->roles->pluck('permissions')->flatten(),
        'orders' => $userWithDetails->orders,
        'invoices' => $invoices,
        'payments' => $userWithDetails->payments,
    ]);

    return response()->json([
        'user' => [
            'id' => $userWithDetails->id,
            'first_name' => $userWithDetails->first_name,
            'last_name' => $userWithDetails->last_name,
            'email' => $userWithDetails->email,
            'phone_number' => $userWithDetails->phone_number,
            'language' => $userWithDetails->language,
        ],
        'roles' => $userWithDetails->roles,
        'permissions' => $userWithDetails->roles->pluck('permissions')->flatten(),
        'orders' => $userWithDetails->orders,
        'invoices' => $invoices,
    ]);
}




}
```

FILE: app/Http/Controllers/VehicleManagerController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Vehicle;
use App\Models\Order;
use App\Models\Shipment;
use App\Models\Route;
use App\Events\OrderStatusUpdated;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Illuminate\Support\Facades\Validator;

class VehicleManagerController extends Controller
{
    /**
     * Get all vehicles with their associated vehicle management for the authenticated vehicle manager.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return \Illuminate\Http\JsonResponse
     */
    public function getVehiclesWithVehicleManagement(Request $request)
```

```
    {
        // Retrieve the authenticated user
        $user = $request->user();

        // Log user info
        Log::info('VehicleManagerController@getVehiclesWithVehicleManagement - Authenticated user info', [
            'user_id' => $user->id,
            'user_name' => $user->name,
            'user_email' => $user->email,
        ]);

        // Log the incoming request data
        Log::info('VehicleManagerController@getVehiclesWithVehicleManagement - Incoming Request Data', [
            'request_data' => $request->all(),
        ]);

        // Get all vehicles associated with the authenticated user, along with their vehicle management data
        try {
            $vehicles = Vehicle::with('vehicleManagement') // Eager load the vehicleManagement relationship
                ->whereHas('vehicleManagement', function ($query) use ($user) {
                    // Ensure that the vehicle management is associated with the current user
                    $query->where('users_id', $user->id);
                })
                ->get();

            // Log the vehicles retrieved
            Log::info('VehicleManagerController@getVehiclesWithVehicleManagement - Vehicles Retrieved', [
                'vehicles_count' => $vehicles->count(),
                'vehicle_ids' => $vehicles->pluck('id')->toArray(),
            ]);
        } catch (\Exception $e) {
            // Log error in case of failure
            Log::error('VehicleManagerController@getVehiclesWithVehicleManagement - Error retrieving vehicles', [
                'error_message' => $e->getMessage(),
                'user_id' => $user->id,
            ]);

            // Return a response indicating error
            return response()->json(['error' => 'Failed to retrieve vehicles.'], 500);
        }

        // Return the response with the vehicles along with their vehicle management
        return response()->json($vehicles);
    }

    /**
     * Update vehicle and its associated vehicle management for the authenticated vehicle manager.
     *
     * @param  \Illuminate\Http\Request  $request
     * @param  int  $vehicleId
     * @return \Illuminate\Http\JsonResponse
     */
    public function updateVehicleWithManagement(Request $request, $vehicleId)
    {
        // Retrieve the authenticated user
        $user = $request->user();

        // Log user info
        Log::info('VehicleManagerController@updateVehicleWithManagement - Authenticated user info', [
            'user_id' => $user->id,
            'user_name' => $user->name,
            'user_email' => $user->email,
        ]);

        // Log the incoming request data
        Log::info('VehicleManagerController@updateVehicleWithManagement - Incoming Request Data', [
```

```
            'request_data' => $request->all(),
        ]);

        // Find the vehicle by ID
        $vehicle = Vehicle::with('vehicleManagement')->find($vehicleId);

        if (!$vehicle) {
            // Log if the vehicle is not found
            Log::error('VehicleManagerController@updateVehicleWithManagement - Vehicle not found', [
                'vehicle_id' => $vehicleId,
                'user_id' => $user->id,
            ]);
            return response()->json(['error' => 'Vehicle not found.'], 404);
        }

        // Check if the vehicle is associated with the authenticated user's vehicle management
        if ($vehicle->vehicleManagement->users_id !== $user->id) {
            // Log unauthorized access
            Log::warning('VehicleManagerController@updateVehicleWithManagement - Unauthorized access attempt', [
                'vehicle_id' => $vehicleId,
                'user_id' => $user->id,
            ]);
            return response()->json(['error' => 'Unauthorized access.'], 403);
        }

        // Validate the incoming data (all fields)
        $validator = Validator::make($request->all(), [
            // Vehicle fields
            'license_plate' => 'required|string|max:255',
            'type' => 'required|string|max:255',
            'status' => 'required|string|max:255',
            'driver_id' => 'nullable|exists:users,id',
            'capacity' => 'nullable|numeric',
            'fuel_capacity' => 'nullable|numeric',
            'current_location' => 'nullable|string|max:255',
            'last_serviced' => 'nullable|date',
            'last_fuel_refill' => 'nullable|date',
            'last_location_update' => 'nullable|date',
            'mileage' => 'nullable|numeric',
            'maintenance_logs' => 'nullable|string',
            'fuel_interval' => 'nullable|numeric',
            'fuel_type' => 'nullable|string|max:50',
            'vin' => 'nullable|string|max:50',
            'brand' => 'nullable|string|max:255',
            'model' => 'nullable|string|max:255',
            'year_of_manufacture' => 'nullable|numeric',

            // Vehicle management fields
            'vehicle_management' => 'nullable|array', // Expecting nested data for vehicle management
            'vehicle_management.fuel_consumption' => 'nullable|numeric',
            'vehicle_management.distance_traveled' => 'nullable|numeric',
            'vehicle_management.maintenance_status' => 'nullable|string|max:255',
            'vehicle_management.last_maintenance_date' => 'nullable|date',
            'vehicle_management.maintenance_schedule' => 'nullable|string|max:255',
            'vehicle_management.maintenance_cost' => 'nullable|numeric',
        ]);

        // Log validation errors if they occur
        if ($validator->fails()) {
            Log::warning('VehicleManagerController@updateVehicleWithManagement - Validation Failed', [
                'errors' => $validator->errors(),
                'user_id' => $user->id,
            ]);
            return response()->json(['error' => 'Validation failed.', 'details' => $validator->errors()], 422);
        }
```

```
        // Log successful validation
        Log::info('VehicleManagerController@updateVehicleWithManagement - Validation Passed', [
            'user_id' => $user->id,
        ]);

        // Update the vehicle details
        try {
            // Log before updating the vehicle
            Log::info('VehicleManagerController@updateVehicleWithManagement - Updating Vehicle', [
                'vehicle_id' => $vehicleId,
                'new_data' => $request->only([
                    'license_plate', 'type', 'status', 'driver_id', 'capacity', 'fuel_capacity',
                    'current_location', 'last_serviced', 'last_fuel_refill', 'last_location_update',
                    'mileage', 'maintenance_logs', 'fuel_interval', 'fuel_type', 'vin', 'brand', 'model',
                    'year_of_manufacture'
                ]),
            ]);

            // Update the Vehicle model with all the fields from the request
            $vehicle->update([
                'license_plate' => $request->license_plate,
                'type' => $request->type,
                'status' => $request->status,
                'driver_id' => $request->driver_id,
                'capacity' => $request->capacity,
                'fuel_capacity' => $request->fuel_capacity,
                'current_location' => $request->current_location,
                'last_serviced' => $request->last_serviced,
                'last_fuel_refill' => $request->last_fuel_refill,
                'last_location_update' => $request->last_location_update,
                'mileage' => $request->mileage,
                'maintenance_logs' => $request->maintenance_logs,
                'fuel_interval' => $request->fuel_interval,
                'fuel_type' => $request->fuel_type,
                'vin' => $request->vin,
                'brand' => $request->brand,
                'model' => $request->model,
                'year_of_manufacture' => $request->year_of_manufacture,
            ]);

            // Log the update for vehicle management
            if ($request->has('vehicle_management')) {
                Log::info('VehicleManagerController@updateVehicleWithManagement - Updating Vehicle Management', [
                    'vehicle_id' => $vehicleId,
                    'new_vehicle_management_data' => $request->vehicle_management,
                ]);

                $vehicleManagement = $vehicle->vehicleManagement;

                $vehicleManagement->update([
                    'fuel_consumption' => $request->vehicle_management['fuel_consumption'] ?? $vehicleManagement->fuel_consumption,
                    'distance_traveled' => $request->vehicle_management['distance_traveled'] ?? $vehicleManagement->distance_traveled,
                    'maintenance_status' => $request->vehicle_management['maintenance_status'] ?? $vehicleManagement->maintenance_status,
                    'last_maintenance_date' => $request->vehicle_management['last_maintenance_date'] ?? $vehicleManagement-
>last_maintenance_date,
                    'maintenance_schedule' => $request->vehicle_management['maintenance_schedule'] ?? $vehicleManagement-
>maintenance_schedule,
                    'maintenance_cost' => $request->vehicle_management['maintenance_cost'] ?? $vehicleManagement->maintenance_cost,
                ]);
            }

            // Handle orders and shipment maintenance check
            $orders = Order::all();
            foreach ($orders as $order) {
                $shipment = Shipment::where('orders_id', $order->id)->first();
                if ($shipment) {
```

```
                    $routes = Route::where('shipments_id', $shipment->id)->get();
                    $isAllMaintenance = true;
                    $vehicle = null;
                    foreach ($routes as $route) {
                        $vehicle = Vehicle::find($route->vehicles_id);
                        $vehicleManagement = $vehicle->vehicleManagement;
                        $isAllMaintenance = $isAllMaintenance && ($vehicleManagement->maintenance_status === 'Delivery Maintenance
Checked');
                        if($vehicleManagement->maintenance_status === 'Delivery Maintenance Checked'){
                            $vehicle->status = 'Delivery Maintenance Checked';
                            $vehicle->save();
                        }
                    }

                    if ($isAllMaintenance) {
                        $order->status = 'Delivery Maintenance Checked';
                        $order->save();
                        event(new OrderStatusUpdated($order));
                    }
                }
            }

            // Log the successful update
            Log::info('VehicleManagerController@updateVehicleWithManagement - Vehicle updated successfully', [
                'vehicle_id' => $vehicleId,
                'user_id' => $user->id,
            ]);
        } catch (\Exception $e) {
            // Log any errors during the update
            Log::error('VehicleManagerController@updateVehicleWithManagement - Error updating vehicle', [
                'error_message' => $e->getMessage(),
                'vehicle_id' => $vehicleId,
                'user_id' => $user->id,
            ]);
            return response()->json(['error' => 'Failed to update vehicle.'], 500);
        }

        // Return the updated vehicle along with its management details
        return response()->json($vehicle->load('vehicleManagement'));
    }
}
```

FILE: app/Http/Controllers/ViewerController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\User;
use Illuminate\Support\Facades\Log;
use Agence104\LiveKit\AccessToken;
use Agence104\LiveKit\AccessTokenOptions;

use Agence104\LiveKit\VideoGrant;
use App\Models\Block;

class ViewerController extends Controller
{
    // Method to create a viewer token for the LiveKit room
    public function createViewerToken(Request $request, $hostIdentity)
    {
        try {
            // Log the incoming request for viewer token creation
```

```
        Log::info("Received request to create viewer token for hostIdentity: {$hostIdentity}");

        // Get the authenticated user or create a guest user if not authenticated
        $self = $request->user();

        // Log whether the user is authenticated or a guest
        if (!$self) {
            Log::info("No authenticated user found. Creating a guest user.");
            $self = (object)[
                'id' => (string) \Str::uuid(),  // Generate a new UUID for guest
                'first_name' => 'Guest',  // Random guest first name
                'last_name' => rand(100000, 999999),  // Random guest last name
            ];
            Log::info("Guest user created with ID {$self->id}");
        } else {
            Log::info("Authenticated user: {$self->first_name} {$self->last_name} with ID {$self->id}.");
        }

        // Log the attempt to find the host user
        Log::info("Attempting to find the host user with ID {$hostIdentity}");

        // Find the host user by their identity (id)
        $host = User::where('id', $hostIdentity)->first();

        if (!$host) {
            Log::warning("Host with ID {$hostIdentity} not found.");
            return response()->json(['error' => 'Host not found'], 404);
        }

        Log::info("Found host with ID {$host->id} and name {$host->first_name} {$host->last_name}");

        // Check if the authenticated user is blocked by the host
        Log::info("Checking if user {$self->id} is blocked by host {$host->id}");
        if ($this->isBlockedByUser($host->id, $self->id)) {
            Log::warning("User {$self->id} is blocked by host {$host->id}.");
            return response()->json(['error' => 'User is blocked'], 403);
        }

        Log::info("User {$self->id} is not blocked by host {$host->id}");

        // Determine if the user is the host
        $isHost = $self->id === $host->id;
        Log::info("User {$self->id} is " . ($isHost ? "the host" : "a viewer"));

        // Log the LiveKit token creation
        Log::info("Creating LiveKit AccessToken for user {$self->id}");

        // Create the LiveKit AccessToken
        $tokenOptions = new AccessTokenOptions();
        $tokenOptions->setIdentity($isHost ? 'Host-' . $self->id : $self->id);
        $tokenOptions->setName($self->first_name . ' ' . $self->last_name);

        // Create the LiveKit AccessToken with the token options
        $token = new AccessToken(env('LIVEKIT_API_KEY'), env('LIVEKIT_API_SECRET'));
        $token->init($tokenOptions);

        if ($token) {
            Log::info("AccessToken successfully created for user {$self->id}");
        } else {
            Log::error("Failed to create AccessToken for user {$self->id}");
            return response()->json(['error' => 'Failed to create AccessToken'], 500);
        }

        // Add the VideoGrant to the token
        Log::info("Creating VideoGrant for user {$self->id} for room {$host->id}");
        $videoGrant = new VideoGrant();
```

```
            $videoGrant->setRoomJoin()
                        ->setRoomName($host->id)
                        ->setCanPublish(false)  // Viewer cannot publish
                        ->setCanPublishData(true);

            // Log the state of the video grant before adding it
            Log::info("VideoGrant details: " . json_encode($videoGrant->getData()));

            // Check if the AccessToken class has a method to accept grants
            if (method_exists($token, 'setGrant')) {
                Log::info("Adding VideoGrant to AccessToken.");
                $token->setGrant($videoGrant);  // Add the VideoGrant to the token
                Log::info("VideoGrant successfully added to AccessToken.");
            } else {
                Log::error("Failed to add VideoGrant: setGrant method does not exist on AccessToken.");
                return response()->json(['error' => 'Failed to add VideoGrant'], 500);
            }

            // Log the final token before returning
            Log::info("Generated viewer token for user {$self->id} for room {$host->id}. Token: " . $token->toJwt());

            // Return the JWT token
            return response()->json($token->toJwt());
        } catch (\Exception $e) {
            // Log the exception with the detailed error message and stack trace
            Log::error("Error creating viewer token for host {$hostIdentity}: " . $e->getMessage(), ['exception' => $e]);
            return response()->json(['message' => 'Internal error: ' . $e->getMessage()], 500);
        }
    }

    // Method to check if the user is blocked by another user
    private function isBlockedByUser($userId, $selfId)
    {
        try {
            // Log the block status check attempt
            Log::info("Checking block status: Is user {$selfId} blocked by {$userId}?");

            // Check if the self user is blocked by the specified user
            $isBlocked = Block::where('blocker_id', $userId)
                        ->where('blocked_id', $selfId)
                        ->exists();

            // Log whether the user is blocked
            if ($isBlocked) {
                Log::info("User {$selfId} is blocked by {$userId}.");
            } else {
                Log::info("User {$selfId} is not blocked by {$userId}.");
            }

            return $isBlocked;
        } catch (\Exception $e) {
            // Log the error in checking block status
            Log::error("Error checking block status: " . $e->getMessage(), ['exception' => $e]);
            return false;
        }
    }
}
```

FILE: app/Http/Controllers/WarehouseController.php

```
<?php

namespace App\Http\Controllers;
```

```
use App\Models\Order;
use App\Models\OrderItem;
use App\Models\Product;
use App\Models\Category;
use App\Models\Inventory;
use App\Models\Warehouse;
use App\Models\VehicleManagement;
use App\Models\Shipment;
use App\Models\Route;

use Carbon\Carbon;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use App\Integrations\files\CloudinaryImageClient; // Use the custom Cloudinary client
use App\Events\OrderStatusUpdated;

class WarehouseController extends Controller
{
    protected $cloudinaryClient;

    public function __construct()
    {
        // Initialize custom Cloudinary image client
        $this->cloudinaryClient = new CloudinaryImageClient();
    }

    public function createInventory(Request $request)
    {
        // Log the incoming request data
        Log::info('Create Inventory request received', [
            'user_id' => $request->user()->id,
            'product_details' => $request->input('product'),
            'inventory_details' => $request->input('inventory')
        ]);

        // Get the user's warehouses
        $userWarehouses = Warehouse::where('users_id', $request->user()->id)->get();

        // If no warehouses are found for the user, return an error
        if ($userWarehouses->isEmpty()) {
            Log::warning('No warehouses found for the user', ['user_id' => $request->user()->id]);
            return response()->json(['message' => 'No warehouses found for this user.'], 400);
        }

        // Select the first warehouse (default)
        $defaultWarehouse = $userWarehouses->first();
        Log::info('Selected default warehouse', ['warehouse_id' => $defaultWarehouse->id]);

        try {
            // Validate the incoming request
            $validated = $request->validate([
                'product' => [
                    'name' => ['required', 'string'],
                    'description' => ['required', 'string'],
                    'price' => ['required', 'numeric'],
                    'image' => ['nullable', 'image'],
                    'category' => ['nullable', 'string'],
                    'isFeatured' => ['nullable', 'boolean'],  // Default to false if not provided
                ],
                'inventory' => [
                    'stock' => ['required', 'numeric'],
                    'weightPerUnit' => ['required', 'numeric'],
                ]
            ]);

            // Log the validated product and inventory data
```

```
        Log::info('Validated product and inventory data', [
            'product' => $validated['product'],
            'inventory' => $validated['inventory']
        ]);

        // Handle image upload to Cloudinary
        $imageUrl = '';
        if ($request->hasFile('product.image')) {
            try {
                // Retrieve the image file
                $uploadedFile = $request->file('product.image');
                $mimeType = $uploadedFile->getClientMimeType();
                $base64Contents = base64_encode($uploadedFile->getContent());

                // Upload image using custom Cloudinary client
                $uploadResult = $this->cloudinaryClient->uploadImage($mimeType, $base64Contents);

                // If the upload is successful, we get the secure URL
                if ($uploadResult->isSuccess) {
                    $imageUrl = $uploadResult->secureUrl;
                    Log::info('Product image uploaded to Cloudinary', ['image_url' => $imageUrl]);
                } else {
                    throw new \Exception("Cloudinary image upload failed: " . $uploadResult->msg);
                }
            } catch (\Exception $e) {
                Log::error('Error uploading image to Cloudinary', [
                    'error_message' => $e->getMessage(),
                    'image' => $uploadedFile->getClientOriginalName()
                ]);
                throw $e;  // Re-throw the exception after logging it
            }
        }

        // Create Product (First, handle the product)
        $product = Product::create([
            'name' => $validated['product']['name'],
            'description' => $validated['product']['description'],
            'price' => $validated['product']['price'],
            'image' => $imageUrl,
            'category' => $validated['product']['category'],
            'isFeatured' => $validated['product']['isFeatured'] == true,
        ]);
        Log::info('Product created successfully', ['product_id' => $product->id, 'product_name' => $product->name]);

        // Create Inventory (After creating the product)
        $inventory = Inventory::create([
            'stock' => $validated['inventory']['stock'],
            'last_updated' => Carbon::now(),
            'weight_per_unit' => $validated['inventory']['weightPerUnit'],
            'products_id' => $product->id,  // Link inventory to the newly created product
            'warehouses_id' => $defaultWarehouse->id,  // Use the first warehouse from the user's warehouses
        ]);
        Log::info('Inventory created and linked to product', ['inventory_id' => $inventory->id, 'product_id' => $product->id]);

        // Return both the created product and inventory
        return response()->json(['product' => $product, 'inventory' => $inventory], 201);
    } catch (\Exception $e) {
        // Log any exception that occurs during the process
        Log::error('Error creating product and inventory', [
            'user_id' => $request->user()->id,
            'error_message' => $e->getMessage(),
            'stack_trace' => $e->getTraceAsString(),
        ]);
        return response()->json(['message' => 'Server error', 'error' => $e->getMessage()], 500);
    }
}
```

```
/**
 * Get the total weight of inventory and capacity for the warehouse owned by the authenticated user.
 *
 * @param Request $request
 * @return \Illuminate\Http\JsonResponse
 */
public function getTotalInventoryWeightForUserWarehouse(Request $request)
{
    try {
        // Get the warehouse managed by the current user (assuming only one warehouse)
        $warehouse = Warehouse::where('users_id', $request->user()->id)->first();

        // Check if the user has a warehouse
        if (!$warehouse) {
            return response()->json(['error' => 'No warehouse found for this user.'], 404);
        }

        // Get all inventories for the user's warehouse
        $inventories = Inventory::where('warehouses_id', $warehouse->id)
            ->with('product') // Eager load product to get weight_per_unit
            ->get();

        // Calculate total weight for the warehouse
        $totalWeight = 0;
        foreach ($inventories as $inventory) {
            // Calculate weight for this inventory (stock * weight_per_unit)
            $totalWeight += $inventory->stock * $inventory->weight_per_unit;
        }

        // Prepare the response data
        $responseData = [
            'warehouse_name' => $warehouse->warehouse_name,
            'total_weight' => $totalWeight,
            'capacity' => $warehouse->capacity // Include warehouse capacity in the response
        ];

        // Return the total weight and warehouse capacity
        return response()->json($responseData);

    } catch (\Exception $e) {
        // Log the exception and return a generic error message
        \Log::error('Error occurred while calculating total inventory weight: ' . $e->getMessage());

        return response()->json(['error' => 'An error occurred while processing your request.'], 500);
    }
}

/**
* Get aggregated expense data (total cost by category or product).
*
* @param  Request  $request
* @return \Illuminate\Http\JsonResponse
*/
public function filterExpenses(Request $request)
{
    try {
        // Log incoming request parameters
        Log::info('Incoming request data:', $request->all());

        // Validate the incoming request for date filters (removed category validation)
        $validated = $request->validate([
            'start_date' => 'nullable|date',
            'end_date' => 'nullable|date',
        ]);
```

```
        $startDate = $validated['start_date'] ?? null;
        $endDate = $validated['end_date'] ?? null;

        // Log the validated filter values (category has been removed)
        Log::info('Validated filter values:', [
            'start_date' => $startDate,
            'end_date' => $endDate,
        ]);

        // Get the warehouses managed by the current user
        $userWarehouses = Warehouse::where('users_id', $request->user()->id)->get();

        if ($userWarehouses->isEmpty()) {
            return response()->json(['error' => 'No warehouses found for this user.'], 404);
        }

        // Get all product IDs from the inventories of these warehouses
        $productIdsInWarehouse = Inventory::whereIn('warehouses_id', $userWarehouses->pluck('id'))
            ->pluck('products_id')
            ->unique(); // Get unique product IDs

        // Log the product IDs from the warehouse inventory
        Log::info('Products in the managed warehouses:', $productIdsInWarehouse->toArray());

        // Start by querying the orders and joining with order items
        $ordersQuery = OrderItem::query()
            ->with(['order'])  // eager load relationships
            ->whereIn('products_id', $productIdsInWarehouse) // Filter by product IDs in the warehouse inventory
            ->when($startDate, function ($query) use ($startDate) {
                return $query->whereHas('order', function ($query) use ($startDate) {
                    $query->whereDate('order_date', '>=', Carbon::parse($startDate));
                });
            })
            ->when($endDate, function ($query) use ($endDate) {
                return $query->whereHas('order', function ($query) use ($endDate) {
                    $query->whereDate('order_date', '<=', Carbon::parse($endDate));
                });
            });

        // Log the SQL query before executing
        Log::info('Executing query for order items:', [
            'query' => $ordersQuery->toSql(),
            'bindings' => $ordersQuery->getBindings(),
        ]);

        // Get the order items
        $orderItems = $ordersQuery->get();
        Log::info('Fetched order items:', ['count' => $orderItems->count()]);

        // Aggregate the data without filtering by category
        $aggregatedData = collect($orderItems)->map(function ($orderItem) {
            $product = $orderItem->product;

            return [
                'product_id' => $product->id,
                'product_name' => $product->name,
                'amount' => $orderItem->total_amount,
                'order_date' => $orderItem->order->order_date,  // Using the order's date here
            ];
        });

        // Log the structure of aggregatedData before processing further
        Log::info('Aggregated Data Structure:', $aggregatedData->toArray());

        // Now we can sum the amounts by product (instead of category) if needed
        $aggregatedData = $aggregatedData->groupBy('product_name')->map(function ($items) {
```

```
            Log::info('Processing group:', [
                'product_name' => $items->first()['product_name'] ?? 'N/A',
                'item_count' => $items->count(),
            ]);

            // Check if product_name exists before accessing it
            if (isset($items->first()['product_name'])) {
                $totalAmount = $items->map(function ($item) {
                    return $item['amount'];
                })->sum();

                return [
                    'product_name' => $items->first()['product_name'],
                    'total_amount' => $totalAmount,
                ];
            } else {
                return null; // Handle missing product_name
            }
        })->filter(); // Remove any null values

        // Log the aggregated data before returning
        Log::info('Aggregated data after grouping and summing:', $aggregatedData->toArray());

        // Prepare the response format
        $response = $aggregatedData->map(function ($item) {
            return [
                'name' => $item['product_name'],
                'amount' => $item['total_amount'],
            ];
        });

        // Log the final response data
        Log::info('Response data:', $response->toArray());

        return response()->json($response);

    } catch (\Exception $e) {
        // Log any exceptions that occur during the execution
        Log::error('Error occurred in filterExpenses method:', [
            'message' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
        ]);

        // Return a generic error response
        return response()->json(['error' => 'An error occurred while processing your request.'], 500);
    }
}

public function getInventoriesForWarehouse(Request $request)
{
    try {
        // Log incoming request parameters
        Log::info('Request received in getInventoriesForWarehouse', [
            'user_id' => $request->user()->id,
            'product_name' => $request->input('product_name'),
            'category' => $request->input('category')
        ]);

        // Get the warehouses managed by the current user
        $userWarehouses = Warehouse::where('users_id', $request->user()->id)->get();

        // Log the warehouses fetched
        Log::info('Warehouses fetched for user', [
            'user_warehouses_count' => $userWarehouses->count(),
            'warehouses' => $userWarehouses->pluck('id')->toArray()
        ]);
```

```
        if ($userWarehouses->isEmpty()) {
            return response()->json(['error' => 'No warehouses found for this user.'], 404);
        }

        // Start building the query to get inventories
        $query = Inventory::whereIn('warehouses_id', $userWarehouses->pluck('id'))
            ->with(['product', 'product.categories'])  // Eager load product and its categories
            ->distinct('products_id'); // Get unique products by product_id

        // Apply filters if provided in the request
        if ($request->has('product_name')) {
            $query->whereHas('product', function ($query) use ($request) {
                $query->where('name', 'like', '%' . $request->input('product_name') . '%');
            });
        }

        // Filter by category if provided
        if ($request->has('category')) {
            $query->whereHas('product.categories', function ($query) use ($request) {
                $query->where('category_name', 'like', '%' . $request->input('category') . '%');
            });
        }

        // Log the final query before executing
        Log::info('Query built for fetching products in warehouse', [
            'query' => $query->toSql(),
            'bindings' => $query->getBindings()
        ]);

        // Execute the query
        $productsInWarehouse = $query->get();

        // Log the result of the query
        Log::info('Products fetched from warehouse', [
            'products_count' => $productsInWarehouse->count(),
            'products' => $productsInWarehouse->pluck('product.name')->toArray()
        ]);

        // If no products found, return a 404 response
        if ($productsInWarehouse->isEmpty()) {
            return response()->json(['error' => 'No products found for the warehouses with the given filters.'], 404);
        }

        // Return the products with their categories
        return response()->json($productsInWarehouse->map(function ($inventory) {
            return [
                'product' => $inventory->product,
                'categories' => $inventory->product->categories, // Add product categories to the response
                'stock' => $inventory->stock,
                'weight_per_unit' => $inventory->weight_per_unit,
                'warehouse_id' => $inventory->warehouses_id
            ];
        }));

    } catch (\Exception $e) {
        // Log any exceptions that occur during the execution
        Log::error('Error occurred in getInventoriesForWarehouse method:', [
            'message' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
        ]);

        // Return a generic error response
        return response()->json(['error' => 'An error occurred while processing your request.'], 500);
    }
}
```

```
public function updateInventory(Request $request)
{
    try {
        // Validate incoming data to ensure we have required fields
        $validated = $request->validate([
            'inventory_updates' => 'required|array',
            'inventory_updates.*.inventory_id' => 'required|exists:inventory,id',
            'inventory_updates.*.stock' => 'nullable|integer|min:0',
            'inventory_updates.*.weight_per_unit' => 'nullable|numeric|min:0',
        ]);

        // Loop through the updates and apply them
        $updatedInventories = [];

        foreach ($validated['inventory_updates'] as $update) {
            // Find the inventory by its ID and eager load the product
            $inventory = Inventory::with('product')->find($update['inventory_id']);

            if (!$inventory) {
                return response()->json(['error' => 'Inventory not found.'], 404);
            }

            // Update the fields that are provided (stock and/or weight_per_unit)
            if (isset($update['stock'])) {
                $inventory->stock = $update['stock'];
            }

            if (isset($update['weight_per_unit'])) {
                $inventory->weight_per_unit = $update['weight_per_unit'];
            }

            // Update the 'last_updated' timestamp
            $inventory->last_updated = now();
            $inventory->save(); // Save the updated inventory

            // The product data is already eager loaded, so we can directly access it
            $product = $inventory->product;

            // Add the updated inventory and product details to the response
            $updatedInventories[] = [
                'inventory_id' => $inventory->id,
                'stock' => $inventory->stock,
                'weight_per_unit' => $inventory->weight_per_unit,
                'last_updated' => $inventory->last_updated,
                'product' => [
                    'id' => $product->id,
                    'name' => $product->name,
                    'sku' => $product->sku, // If SKU is needed, otherwise remove
                    'price' => $product->price, // If price is needed, otherwise remove
                ]
            ];
        }

        return response()->json([
            'message' => 'Inventory updated successfully.',
            'updated_inventories' => $updatedInventories
        ]);

    } catch (\Exception $e) {
        // Log any exceptions that occur during the execution
        Log::error('Error occurred in updateInventory method:', [
            'message' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
```

```
        ]);

        // Return a generic error response
        return response()->json(['error' => 'An error occurred while updating the inventory.'], 500);
    }
}

/**
 * Get an aggregated expense report with total shipment amounts,
 * total maintenance costs, and total product expenses grouped by day.
 *
 * @param  Request  $request
 * @return \Illuminate\Http\JsonResponse
 */
public function lineChart(Request $request)
{
    try {
        Log::info('Incoming expense report request data:', $request->all());
        $validated = $request->validate([
            'start_date' => 'nullable|date',
            'end_date' => 'nullable|date|after_or_equal:start_date',
        ]);
        $startDate = $validated['start_date'] ?? null;
        $endDate = $validated['end_date'] ?? null;
        Log::info('Validated filter values:', [
            'start_date' => $startDate,
            'end_date' => $endDate,
        ]);

        // Get the warehouses managed by the current user
        $userWarehouses = Warehouse::where('users_id', $request->user()->id)->get();
        if ($userWarehouses->isEmpty()) {
            return response()->json(['error' => 'No warehouses found for this user.'], 404);
        }

        // Get all product IDs from the inventories of these warehouses
        $productIdsInWarehouse = Inventory::whereIn('warehouses_id', $userWarehouses->pluck('id'))
            ->pluck('products_id')
            ->unique();

        Log::info('Products in the managed warehouses:', $productIdsInWarehouse->toArray());

        // Query order items
        $orderItemsQuery = OrderItem::query()->with(['order'])
            ->whereIn('products_id', $productIdsInWarehouse)
            ->when($startDate, function ($query) use ($startDate) {
                return $query->whereHas('order', function ($query) use ($startDate) {
                    $query->whereDate('order_date', '>=', Carbon::parse($startDate));
                });
            })
            ->when($endDate, function ($query) use ($endDate) {
                return $query->whereHas('order', function ($query) use ($endDate) {
                    $query->whereDate('order_date', '<=', Carbon::parse($endDate));
                });
            });

        Log::info('Executing query for order items:', [
            'query' => $orderItemsQuery->toSql(),
            'bindings' => $orderItemsQuery->getBindings(),
        ]);

        $orderItems = $orderItemsQuery->get();
        Log::info('Fetched order items:', ['count' => $orderItems->count()]);

        // Aggregate product data (group by order date)
        $aggregatedProductData = collect($orderItems)->flatMap(function ($orderItem) {
```

```
            $product = $orderItem->product;
            $categoryNames = $product->categories->pluck('category_name')->toArray();
            return collect($categoryNames)->map(function ($categoryName) use ($orderItem, $product) {
                return [
                    'category_name' => $categoryName,
                    'product_id' => $product->id,
                    'product_name' => $product->name,
                    'amount' => $orderItem->total_amount,
                    'order_date' => $orderItem->order->order_date, // This is the date to group by
                ];
            });
        })->filter();

        // Aggregate the product data by order date and sum the amounts per date
        $aggregatedProductData = $aggregatedProductData->groupBy('order_date')->map(function ($items) {
            return [
                'date' => $items->first()['order_date'],  // Use the order_date as the "date" key
                'total_product_expenses' => $items->sum('amount'),
            ];
        });

        Log::info('Aggregated product expenses data:', $aggregatedProductData->toArray());

        // Query maintenance costs by date
        $maintenanceCostsQuery = VehicleManagement::query()
            ->selectRaw('DATE(last_maintenance_date) as date, SUM(maintenance_cost) as total_maintenance_cost')
            ->when($startDate, function ($query) use ($startDate) {
                return $query->whereDate('last_maintenance_date', '>=', Carbon::parse($startDate));
            })
            ->when($endDate, function ($query) use ($endDate) {
                return $query->whereDate('last_maintenance_date', '<=', Carbon::parse($endDate));
            })
            ->groupBy('date')
            ->orderBy('date')
            ->get();

        Log::info('Maintenance cost data:', $maintenanceCostsQuery->toArray());

        // Query shipments by date
        $shipmentsQuery = Shipment::query()
            ->selectRaw('DATE(created_at) as date, SUM(total_amount) as total_shipment_amount')
            ->when($startDate, function ($query) use ($startDate) {
                return $query->whereDate('created_at', '>=', Carbon::parse($startDate));
            })
            ->when($endDate, function ($query) use ($endDate) {
                return $query->whereDate('created_at', '<=', Carbon::parse($endDate));
            })
            ->groupBy('date')
            ->orderBy('date')
            ->get();

        Log::info('Shipment data:', $shipmentsQuery->toArray());

        // Combine the data from product expenses, maintenance costs, and shipment amounts
        $combinedData = collect();

        // Add shipments data
        $shipmentsQuery->each(function ($shipment) use ($combinedData) {
            $combinedData->put($shipment->date, [
                'date' => $shipment->date,
                'total_shipment_amount' => $shipment->total_shipment_amount,
                'total_maintenance_cost' => 0,
                'total_product_expenses' => 0,
            ]);
        });
```

```
        // Add maintenance data
        $maintenanceCostsQuery->each(function ($maintenance) use ($combinedData) {
            $combinedData->put($maintenance->date, [
                'date' => $maintenance->date,
                'total_shipment_amount' => $combinedData->get($maintenance->date)['total_shipment_amount'] ?? 0,
                'total_maintenance_cost' => $maintenance->total_maintenance_cost,
                'total_product_expenses' => $combinedData->get($maintenance->date)['total_product_expenses'] ?? 0,
            ]);
        });

        // Add product expense data
        $aggregatedProductData->each(function ($item) use ($combinedData) {
            $combinedData->put($item['date'], [
                'date' => $item['date'],
                'total_shipment_amount' => $combinedData->get($item['date'])['total_shipment_amount'] ?? 0,
                'total_maintenance_cost' => $combinedData->get($item['date'])['total_maintenance_cost'] ?? 0,
                'total_product_expenses' => $item['total_product_expenses'],
            ]);
        });

        // Format the response
        $response = $combinedData->values();

        // Return the aggregated data as a JSON response
        return response()->json($response);
    } catch (\Exception $e) {
        Log::error('Error occurred in lineChart method:', [
            'message' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
        ]);
        return response()->json(['error' => 'An error occurred while processing your request.'], 500);
    }
}


/**
 * Get aggregated warehouse count by country.
 *
 * @param  Request  $request
 * @return \Illuminate\Http\JsonResponse
 */
public function getGeography(Request $request)
{
    try {
        // Log the incoming request (optional)
        Log::info('Incoming geography request data:', $request->all());

        // Fetch all warehouses and their locations
        $warehouses = Warehouse::all();

        // Aggregate warehouse count by country (extracted from the location field)
        $mappedLocations = $warehouses->reduce(function ($acc, $warehouse) {
            // Extract the country from the location field
            $location = $warehouse->location;

            // Assume the country is the part after the last comma
            $country = $this->extractCountryFromLocation($location);

            // Initialize country if it doesn't exist in accumulator
            if (!isset($acc[$country])) {
                $acc[$country] = 0;
            }

            // Increment the count for the country
            $acc[$country]++;
            return $acc;
```

```
        }, []);

        // Format the result into an array of objects with 'id' as country and 'value' as count
        $formattedLocations = collect($mappedLocations)->map(function ($count, $country) {
            return [
                'id' => $country,  // Country name (or code)
                'value' => $count  // Number of warehouses in this country
            ];
        })->values(); // Reset array keys

        // Return the formatted response
        return response()->json($formattedLocations, 200);

    } catch (\Exception $e) {
        // Log the exception error
        Log::error('Error occurred in getGeography method:', [
            'message' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
        ]);

        // Return a generic error response
        return response()->json(['message' => 'An error occurred while fetching data.'], 500);
    }
}

/**
 * Extract country from a location string.
 *
 * @param  string  $location
 * @return string
 */
private function extractCountryFromLocation($location)
{
    // Split the location by commas
    $parts = explode(',', $location);

    // The country should be the last part of the array
    $country = trim(end($parts));

    // Return the country
    return $country;
}

public function getOrdersByUserWarehouse(Request $request)
{
    try {
        // Log incoming request parameters
        Log::info('Incoming request data:', $request->all());

        // Get the warehouses managed by the current user
        $userWarehouses = Warehouse::where('users_id', $request->user()->id)->get();

        if ($userWarehouses->isEmpty()) {
            return response()->json(['error' => 'No warehouses found for this user.'], 404);
        }

        // Get all product IDs from the inventories of these warehouses
        $productIdsInWarehouse = Inventory::whereIn('warehouses_id', $userWarehouses->pluck('id'))
            ->pluck('products_id')
            ->unique(); // Get unique product IDs

        // Log the product IDs from the warehouse inventory
        Log::info('Products in the managed warehouses:', $productIdsInWarehouse->toArray());

        // Start by querying the orders and joining with order items
        $ordersQuery = OrderItem::query()
```

```
            ->with(['order'])  // eager load relationships
            ->whereIn('products_id', $productIdsInWarehouse) // Filter by product IDs in the warehouse inventory
            ->whereHas('order', function ($query) {
                // Filter orders that are marked as 'Paid'
                $query->where('status', 'Paid');
            });

        // Log the SQL query before executing
        Log::info('Executing query for order items:', [
            'query' => $ordersQuery->toSql(),
            'bindings' => $ordersQuery->getBindings(),
        ]);

        // Get the order items
        $orderItems = $ordersQuery->get();
        Log::info('Fetched order items:', ['count' => $orderItems->count()]);

        // Aggregate the data without filtering by category
        $response = collect($orderItems)->map(function ($orderItem) {
            return $orderItem->order;  // Return the entire order associated with the order item
        });

        // Log the response data before returning
        Log::info('Response data:', $response->toArray());

        return response()->json($response);

    } catch (\Exception $e) {
        // Log any exceptions that occur during the execution
        Log::error('Error occurred in getOrdersByUserWarehouse method:', [
            'message' => $e->getMessage(),
            'stack' => $e->getTraceAsString(),
        ]);

        // Return a generic error response
        return response()->json(['error' => 'An error occurred while processing your request.'], 500);
    }
}

public function updateOrderStatus(Request $request, $orderId)
{
    try {
        // Log incoming request parameters
        Log::info('Incoming request data for updating order status:', $request->all());

        // Validate the incoming request to ensure the new status is provided
        $validated = $request->validate([
            'status' => 'required|string', // List the possible statuses
        ]);

        $newStatus = $validated['status'];

        // Get the order by ID
        $order = Order::find($orderId);

        if (!$order) {
            return response()->json(['error' => 'Order not found.'], 404);
        }

        // Get the warehouses that belong to the user
        $userWarehouses = Warehouse::where('users_id', $request->user()->id)->get();
        if ($userWarehouses->isEmpty()) {
            return response()->json(['error' => 'No warehouses found for this user.'], 404);
        }

        // Get all product IDs from the inventories of these warehouses
```

```
            $productIdsInWarehouse = Inventory::whereIn('warehouses_id', $userWarehouses->pluck('id'))
                ->pluck('products_id')
                ->unique();

            Log::info('Products in the managed warehouses:', $productIdsInWarehouse->toArray());

            // Query order items for the given order to ensure they belong to this order and their products exist in the managed
warehouses
            $orderItems = OrderItem::where('orders_id', $orderId) // Ensure these items are for the correct order
                ->whereIn('products_id', $productIdsInWarehouse) // Ensure the products are available in the user's warehouses
                ->get();

            if ($orderItems->isEmpty()) {
                return response()->json(['error' => 'No valid order items found in the managed warehouses for this order.'], 404);
            }

            // Update the status of the order
            $order->status = $newStatus;
            $order->save();

            // Log the status update
            Log::info('Order status updated successfully:', [
                'order_id' => $orderId,
                'new_status' => $newStatus,
            ]);
            event(new OrderStatusUpdated($order));

            // Return the updated order details
            return response()->json([
                'message' => 'Order status updated successfully.',
                'order' => $order,
            ]);

        } catch (\Exception $e) {
            // Log any exceptions that occur during the execution
            Log::error('Error occurred in updateOrderStatus method:', [
                'message' => $e->getMessage(),
                'stack' => $e->getTraceAsString(),
            ]);

            // Return a generic error response
            return response()->json(['error' => 'An error occurred while processing your request.'], 500);
        }
    }


}
```

FILE: app/Http/Middleware/CustomCors.php

```
<?php
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class CustomCors
{
    /**
     * Handle an incoming request.
     *
     * @param  \Illuminate\Http\Request  $request
     * @param  \Closure  $next
```

```
     * @return \Illuminate\Http\Response
     */
    public function handle(Request $request, Closure $next)
    {
        $response = $next($request);

        // Specify the allowed origin (e.g., frontend running at http://localhost:5173)
        // You can also use environment variables for dynamic origin configuration in production
        $allowedOrigin = 'http://localhost:5173';

        $response->headers->set('Access-Control-Allow-Origin', $allowedOrigin);  // Allow only specific origin
        $response->headers->set('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS'); // Allowed methods
        $response->headers->set('Access-Control-Allow-Headers', 'Content-Type, X-Requested-With, Authorization, Origin'); // Allowed
headers
        $response->headers->set('Access-Control-Allow-Credentials', 'true'); // Allow credentials (cookies, etc.)
        $response->headers->set('Access-Control-Max-Age', '3600000'); // Cache pre-flight response for 1 hour

        // Handle OPTIONS request (pre-flight requests)
        if ($request->getMethod() === 'OPTIONS') {
            return response()->json([], Response::HTTP_OK);
        }

        return $response;
    }
}
```

FILE: app/Http/Middleware/EnsureEmailIsVerified.php

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Contracts\Auth\MustVerifyEmail;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureEmailIsVerified
{
    /**
     * Handle an incoming request.
     *
     * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response)  $next
     */
    public function handle(Request $request, Closure $next): Response
    {
        if (! $request->user() ||
            ($request->user() instanceof MustVerifyEmail &&
            ! $request->user()->hasVerifiedEmail())) {
            return response()->json(['message' => 'Your email address is not verified.'], 409);
        }

        return $next($request);
    }
}
```

FILE: app/Http/Middleware/RoleRoute.php

```
<?php

namespace App\Http\Middleware;
```

```
use Closure;
use Illuminate\Http\Request;
use App\Models\UserRole;
use App\Models\Role;
use Illuminate\Support\Facades\Log; // Import the Log facade

class RoleRoute
{
    public function handle(Request $request, Closure $next, ...$roles)
    {
        // Log the incoming request details (e.g., user IP, requested route, etc.)
        Log::info('RoleRoute Middleware triggered', [
            'user_ip' => $request->ip(),
            'requested_route' => $request->path(),
            'roles_required' => $roles,
        ]);

        // Get the authenticated user
        $user = $request->user();

        // Check if the user exists
        if (!$user) {
            Log::warning('User not authenticated', [
                'requested_route' => $request->path(),
                'user_ip' => $request->ip(),
            ]);
            return response()->json(['message' => 'User not authenticated'], 401);
        }

        // Log the user's roles
        $userRoleIds = UserRole::where('users_id', $user->id)->pluck('roles_id');
        Log::info('User roles retrieved', [
            'user_id' => $user->id,
            'roles_ids' => $userRoleIds,
        ]);

        // Fetch the role names from the Role table
        $userRoles = Role::whereIn('id', $userRoleIds)
            ->pluck('role_name')
            ->map(function ($roleName) {
                return strtolower($roleName);  // Convert role names to lowercase
            })
            ->toArray();

        // Convert all provided roles to lowercase for case-insensitive comparison
        $roles = array_map('strtolower', $roles);

        // Log the comparison between user's roles and required roles
        Log::info('Checking if user has required roles', [
            'user_roles' => $userRoles,
            'roles_required' => $roles,
        ]);

        // Check if the user has one of the required roles (case-insensitive)
        if (array_intersect($roles, $userRoles)) {
            Log::info('Access granted', [
                'user_id' => $user->id,
                'user_roles' => $userRoles,
                'requested_route' => $request->path(),
            ]);
            return $next($request);
        }

        // Log when access is denied
        Log::warning('Access denied - Unauthorized role', [
```

```
            'user_id' => $user->id,
            'user_roles' => $userRoles,
            'requested_route' => $request->path(),
            'roles_required' => $roles,
        ]);

        // Return response if the user doesn't have the required role
        return response()->json(['message' => 'Access denied - Unauthorized role'], 403);
    }
}
```

FILE: app/Http/Middleware/TranslateResponse.php

```
<?php
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use App\Services\RecursiveTranslation;
use Illuminate\Support\Facades\Log;

class TranslateResponse
{
    protected $translator;

    public function __construct(RecursiveTranslation $translator)
    {
        $this->translator = $translator;
    }

    /**
     * Handle an incoming request and translate the response.
     *
     * @param \Illuminate\Http\Request $request
     * @param \Closure $next
     * @return mixed
     */
    public function handle(Request $request, Closure $next)
    {
        // Log that the translation process is starting
        Log::info('Translation middleware started for user: ' . ($request->user() ? $request->user()->id : 'Guest'));

        // Get the response after the request is handled
        $response = $next($request);

        // Get the authenticated user's language, defaulting to 'en' if not set
        $userLanguage = $request->user() && $request->user()->language ? $request->user()->language : 'en';

        // Log the detected user language
        Log::info('Detected language for translation: ' . $userLanguage);

        // Check if the response is a JSON response
        if ($response instanceof \Illuminate\Http\JsonResponse) {
            // Log the content before translation (if not too large)
            Log::info('Original response content: ', (array) $response->getData(true));

            $content = $response->getData(true); // Get the response data as an array
            // Translate the content recursively
            $translatedContent = $this->translator->translate($content, $userLanguage);

            // Log the translated content (you may want to be cautious logging sensitive data)
            Log::info('Translated response content: ', (array) $translatedContent);

            // Update the response with the translated content
```

```
            $response->setData($translatedContent);
        }

        // Log when the translation process is complete
        Log::info('Translation middleware completed for user: ' . ($request->user() ? $request->user()->id : 'Guest'));

        return $response;
    }
}
```

FILE: app/Http/Requests/Auth/LoginRequest.php

```
<?php

namespace App\Http\Requests\Auth;

use Illuminate\Auth\Events\Lockout;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Str;
use Illuminate\Validation\ValidationException;

class LoginRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'email' => ['required', 'string', 'email'],
            'password' => ['required', 'string'],
        ];
    }

    /**
     * Attempt to authenticate the request's credentials.
     *
     * @throws \Illuminate\Validation\ValidationException
     */
    public function authenticate(): void
    {
        $this->ensureIsNotRateLimited();

        if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember'))) {
            RateLimiter::hit($this->throttleKey());

            throw ValidationException::withMessages([
                'email' => __('auth.failed'),
            ]);
        }

        RateLimiter::clear($this->throttleKey());
```

```
    }

    /**
     * Ensure the login request is not rate limited.
     *
     * @throws \Illuminate\Validation\ValidationException
     */
    public function ensureIsNotRateLimited(): void
    {
        if (! RateLimiter::tooManyAttempts($this->throttleKey(), 5)) {
            return;
        }

        event(new Lockout($this));

        $seconds = RateLimiter::availableIn($this->throttleKey());

        throw ValidationException::withMessages([
            'email' => trans('auth.throttle', [
                'seconds' => $seconds,
                'minutes' => ceil($seconds / 60),
            ]),
        ]);
    }

    /**
     * Get the rate limiting throttle key for the request.
     */
    public function throttleKey(): string
    {
        return Str::transliterate(Str::lower($this->input('email')).'|'.$this->ip());
    }
}
```

FILE: app/Http/Requests/ProfileUpdateRequest.php

```
<?php

namespace App\Http\Requests;

use App\Models\User;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class ProfileUpdateRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, \Illuminate\Contracts\Validation\Rule|array|string>
     */
    public function rules(): array
    {
        return [
            'name' => ['string', 'max:255'],
            'email' => ['email', 'max:255', Rule::unique(User::class)->ignore($this->user()->id)],
        ];
    }
}
```

FILE: app/Http/Requests/StoreFeedbackFormAnswerRequest.php

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreFeedbackFormAnswerRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     *
     * @return bool
     */
    public function authorize()
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, mixed>
     */
    public function rules()
    {
        return [
            'answers' => 'required|array'
        ];
    }
}
```

FILE: app/Http/Requests/StoreFeedbackFormRequest.php

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreFeedbackFormRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     *
     * @return bool
     */
    public function authorize()
    {
        return true;
    }

    protected function prepareForValidation()
    {
        $this->merge([
            'user_id' => $this->user()->id
        ]);
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, mixed>
```

```
     */
    public function rules()
    {
        return [
            'user_id' => 'exists:users,id',
            'questions' => 'array',
        ];
    }
}
```

FILE: app/Http/Requests/StoreGroupRequest.php

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreGroupRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'description' => ['nullable', 'string'],
            'user_ids' => ['nullable', 'array'],
            'user_ids.*' => ['integer', 'exists:users,id'],
        ];
    }

    public function validated($key = null, $default = null)
    {
        $validated = parent::validated($key, $default);
        $validated['owner_id'] = $this->user()->id;
        return $validated;
    }
}
```

FILE: app/Http/Requests/StoreMessageRequest.php

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreMessageRequest extends FormRequest
{
```

```
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'message' => 'nullable|string',
            'group_id' => 'required_without:receiver_id|nullable|exists:groups,id',
            'receiver_id' => 'required_without:group_id|nullable|exists:users,id',
            'attachments' => 'nullable|array|max:10',
            'attachments.*' => 'file|max:1024000',
        ];
    }
}
```

FILE: app/Http/Requests/UpdateFeedbackFormRequest.php

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class UpdateFeedbackFormRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     *
     * @return bool
     */
    public function authorize()
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, mixed>
     */
    public function rules()
    {
        return [
            'user_id' => 'exists:users,id',
            'questions' => 'array'
        ];
    }
}
```

FILE: app/Http/Requests/UpdateGroupRequest.php

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class UpdateGroupRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'description' => ['nullable', 'string'],
            'user_ids' => ['nullable', 'array'],
            'user_ids.*' => ['integer', 'exists:users,id'],
        ];
    }
}
```

FILE: app/Http/Resources/FeedbackFormAnswerResource.php

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class FeedbackFormAnswerResource extends JsonResource
{
    /**
     * Transform the resource into an array.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return array|\Illuminate\Contracts\Support\Arrayable|\JsonSerializable
     */
    public function toArray($request)
    {
        return [
            'id' => $this->id,
            'feedbackform' => new FeedbackFormResource($this->feedbackform),
            'end_date' => $this->end_date
        ];
    }
}
```

FILE: app/Http/Resources/FeedbackFormQuestionResource.php

```php
<?php
```

```
namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class FeedbackFormQuestionResource extends JsonResource
{
    /**
     * Transform the resource into an array.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return array|\Illuminate\Contracts\Support\Arrayable|\JsonSerializable
     */
    public function toArray($request)
    {
        return [
            'id' => $this->id,
            'type' => $this->type,
            'question' => $this->question,
            'description' => $this->description,
            'data' => json_decode($this->data),
        ];
    }
}
```

FILE: app/Http/Resources/FeedbackFormResource.php

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;
use Illuminate\Support\Facades\URL;

class FeedbackFormResource extends JsonResource
{
    /**
     * Transform the resource into an array.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return array|\Illuminate\Contracts\Support\Arrayable|\JsonSerializable
     */
    public function toArray($request)
    {
        return [
            'id' => $this->id,
            'questions' => FeedbackFormQuestionResource::collection($this->questions)
        ];
    }
}
```

FILE: app/Http/Resources/GroupResource.php

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class GroupResource extends JsonResource
{
```

```
    /**
     * Transform the resource into an array.
     *
     * @return array<string, mixed>
     */
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'description' => $this->description,
            'owner' => new UserResource($this->owner),
            'users' => UserResource::collection($this->users),
        ];
    }
}
```

FILE: app/Http/Resources/MessageAttachmentResource.php

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;
use Illuminate\Support\Facades\Storage;

class MessageAttachmentResource extends JsonResource
{
    public static $wrap = false;

    /**
     * Transform the resource into an array.
     *
     * @return array<string, mixed>
     */
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'message_id' => $this->message_id,
            'name' => $this->name,
            'mime' => $this->mime,
            'size' => $this->size,
            'url' => Storage::url($this->path),
            'created_at' => $this->created_at,
            'updated_at' => $this->updated_at,
        ];
    }
}
```

FILE: app/Http/Resources/MessageResource.php

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class MessageResource extends JsonResource
```

```
{

    public static $wrap = false;

    /**
     * Transform the resource into an array.
     *
     * @return array<string, mixed>
     */
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'message' => $this->message,
            'sender_id' => $this->sender_id,
            'receiver_id' => $this->receiver_id,
            'sender' => new UserResource($this->sender),
            'group_id' => $this->group_id,
            'attachments' => MessageAttachmentResource::collection($this->attachments),
            'created_at' => $this->created_at,
            'updated_at' => $this->updated_at,
        ];
    }
}
```

FILE: app/Http/Resources/UserResource.php

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class UserResource extends JsonResource
{
    public static $wrap = false;

    /**
     * Transform the resource into an array.
     *
     * @return array<string, mixed>
     */
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'first_name' => $this->first_name,
            'last_name' => $this->last_name,
            'phone_number' => $this->phone_number,
            'email' => $this->email,
            'language' => $this->language,
            'profile_picture' => $this->profile_picture,
            'banner_img' => $this->banner_img,
            'headline' => $this->headline,
            'about' => $this->about,
        ];
    }
}
```

FILE: app/Integrations/Files/CloudinaryImageClient.php

```
<?php

namespace App\Integrations\Files;

use Cloudinary\Api\Upload\UploadApi;
use Cloudinary\Configuration\Configuration;
use Illuminate\Support\Facades\Config;
use Illuminate\Support\Facades\Log;
use Illuminate\Support\Str; // Make sure this is at the top of the file if you want to use Str::uuid()

class CloudinaryImageClient
{
    protected $cloudinaryInstance;

    public function __construct()
    {
        // Retrieve Cloudinary credentials from the environment file
        $cloudName = env('CLOUDINARY_CLOUD_NAME');
        $apiKey = env('CLOUDINARY_API_KEY');
        $apiSecret = env('CLOUDINARY_API_SECRET');

        // Check if the necessary configuration values are available
        if (!$cloudName || !$apiKey || !$apiSecret) {
            Log::error("Cloudinary configuration is missing in the environment file.");
            throw new \Exception("Cloudinary configuration is missing in the environment file.");
        }

        // Set up Cloudinary instance with your configuration
        $this->cloudinaryInstance = Configuration::instance([
            'cloud' => [
                'cloud_name' => $cloudName,
                'api_key' => $apiKey,
                'api_secret' => $apiSecret
            ],
            'url' => [
                'secure' => true
            ]
        ]);
    }

    public function uploadImage(string $mimeType, string $base64FileContents): CloudinaryResults
    {
        Log::debug("uploadImage called", [
            'mimeType' => $mimeType,
            'base64_length' => strlen($base64FileContents),
            'base64_preview' => substr($base64FileContents, 0, 30) . '...' // Only log part to avoid massive output
        ]);

        // Format the file URI for Cloudinary upload
        $formattedUploadUri = self::createUri($mimeType, $base64FileContents);
        Log::debug("Formatted upload URI", [
            'uri_sample' => substr($formattedUploadUri, 0, 50) . '...' // Partial preview
        ]);

        try {
            // Instantiate Cloudinary UploadApi
            $uploadClient = new UploadApi();
            Log::debug("UploadApi client instantiated.");

            // Upload the image
            $results = $uploadClient->upload($formattedUploadUri, ['resource_type' => 'image']);
            Log::info("Cloudinary upload completed", [
                'results' => $results
            ]);
            $results['original_filename'] = (string) Str::uuid();
```

```
            // Create success result object
            $successResult = CloudinaryResults::success(
                $results['secure_url'],
                $results['url'],
                $results['width'],
                $results['height'],
                $results['original_filename'],
                $results['public_id'],
                $results['format'],
                $results['resource_type'],
                $results['tags']
            );

            Log::debug("Returning success result", [
                'secure_url' => $results['secure_url'],
                'url' => $results['url'],
                'dimensions' => $results['width'] . 'x' . $results['height'],
                'filename' => $results['original_filename'],
                'public_id' => $results['public_id'],
                'format' => $results['format'],
                'resource_type' => $results['resource_type'],
                'tags' => $results['tags']
            ]);

            return $successResult;

        } catch (\Exception $e) {
            Log::error("Exception during Cloudinary upload", [
                'error_message' => $e->getMessage(),
                'trace' => $e->getTraceAsString()
            ]);

            $failedResult = CloudinaryResults::failed($e->getMessage());

            Log::debug("Returning failure result", [
                'error_message' => $e->getMessage()
            ]);

            return $failedResult;
        }
    }


    /**
     * Utility function to create a data URI from base64 file contents.
     *
     * @param string $mimeType Example: image/jpeg
     * @param string $base64FileContents The base64-encoded file contents
     * @return string The formatted data URI (e.g., ' ...')
     */
    private static function createUri(string $mimeType, string $base64FileContents): string
    {
        return 'data:' . $mimeType . ';base64,' . $base64FileContents;
    }
}
```

FILE: app/Integrations/Files/CloudinaryResults.php

```
<?php

namespace App\Integrations\Files;

class CloudinaryResults
```

```
{
    public bool $isSuccess;
    public string $insecureUrl;
    public string $secureUrl;
    public string $msg = '';

    public ?int $width = null;
    public ?int $height = null;
    public ?string $name = null;
    public ?string $publicId = null;
    public ?string $format = null;
    public ?string $resourceType = null;
    public ?array $tags = null;


    public static function success(
        string $secureUrl,
        string $insecureUrl,
        int $width,
        int $height,
        string $name,
        string $publicId,
        string $format,
        string $resourceType,
        array $tags
    ): CloudinaryResults {
        $results = new CloudinaryResults();
        $results->isSuccess = true;
        $results->secureUrl = $secureUrl;
        $results->insecureUrl = $insecureUrl;
        $results->width = $width;
        $results->height = $height;
        $results->name = $name;
        $results->publicId = $publicId;
        $results->format = $format;
        $results->resourceType = $resourceType;
        $results->tags = $tags;


        return $results;
    }

    public static function failed(string $msg): CloudinaryResults
    {
        $results = new CloudinaryResults();
        $results->isSuccess = false;
        $results->msg = $msg;
        return $results;
    }
}
```

FILE: app/Integrations/Files/CloudinaryVideoClient.php

```
<?php
namespace App\Integrations\Files;

use Cloudinary\Api\Upload\UploadApi;
use Cloudinary\Configuration\Configuration;
use Illuminate\Support\Facades\Log;
use Illuminate\Support\Str;

class CloudinaryVideoClient
{
    protected $cloudinaryInstance;
```

```
    public function __construct()
    {
        $cloudName = env('CLOUDINARY_CLOUD_NAME');
        $apiKey = env('CLOUDINARY_API_KEY');
        $apiSecret = env('CLOUDINARY_API_SECRET');

        if (!$cloudName || !$apiKey || !$apiSecret) {
            Log::error("Cloudinary configuration is missing in the environment file.");
            throw new \Exception("Cloudinary configuration is missing in the environment file.");
        }

        $this->cloudinaryInstance = Configuration::instance([
            'cloud' => [
                'cloud_name' => $cloudName,
                'api_key' => $apiKey,
                'api_secret' => $apiSecret
            ],
            'url' => [
                'secure' => true
            ]
        ]);
    }

    /**
     * Upload video to Cloudinary
     *
     * @param  \Illuminate\Http\UploadedFile  $file
     * @return CloudinaryVideoResults
     */
    public function uploadVideo($file): CloudinaryVideoResults
    {
        Log::debug("uploadVideo called");

        try {
            // Use the UploadApi to upload the video file
            $uploadClient = new UploadApi();

            // Upload video using the file path
            $uploadResult = $uploadClient->upload(
                $file->getRealPath(), // The file path
                [
                    'resource_type' => 'video', // We are uploading a video
                    'folder' => 'my_videos', // Optional: Upload to 'my_videos' folder
                ]
            );

            Log::info("Cloudinary video upload completed", ['results' => $uploadResult]);

            // Map the results to CloudinaryVideoResults
            $videoResults = CloudinaryVideoResults::success(
                $uploadResult['secure_url'],
                $uploadResult['url'],
                $uploadResult['width'] ?? 0,
                $uploadResult['height'] ?? 0,
                $uploadResult['original_filename'],
                $uploadResult['public_id'],
                $uploadResult['format'],
                $uploadResult['resource_type'],
                $uploadResult['tags'] ?? [],
                $uploadResult['poster'] ?? null // Optional poster
            );

            return $videoResults;

        } catch (\Exception $e) {
```

```
            Log::error("Exception during Cloudinary video upload", [
                'error_message' => $e->getMessage(),
                'trace' => $e->getTraceAsString()
            ]);

            return CloudinaryVideoResults::failed($e->getMessage());
        }
    }
}
```

FILE: app/Integrations/Files/CloudinaryVideoResults.php

```
<?php

namespace App\Integrations\Files;

class CloudinaryVideoResults
{
    public bool $isSuccess;
    public string $insecureUrl;
    public string $secureUrl;
    public string $msg = '';

    public ?int $width = null;
    public ?int $height = null;
    public ?string $name = null;
    public ?string $publicId = null;
    public ?string $format = null;
    public ?string $resourceType = null;
    public ?array $tags = null;
    public ?string $posterUrl = null; // ✅ added for video poster

    public static function success(
        string $secureUrl,
        string $insecureUrl,
        ?int $width,
        ?int $height,
        string $name,
        string $publicId,
        string $format,
        string $resourceType,
        array $tags,
        ?string $posterUrl = null // ✅ optional poster parameter
    ): CloudinaryVideoResults {
        $results = new CloudinaryVideoResults();
        $results->isSuccess = true;
        $results->secureUrl = $secureUrl;
        $results->insecureUrl = $insecureUrl;
        $results->width = $width;
        $results->height = $height;
        $results->name = $name;
        $results->publicId = $publicId;
        $results->format = $format;
        $results->resourceType = $resourceType;
        $results->tags = $tags;
        $results->posterUrl = $posterUrl;

        return $results;
    }

    public static function failed(string $msg): CloudinaryVideoResults
    {
        $results = new CloudinaryVideoResults();
        $results->isSuccess = false;
```

```
        $results->msg = $msg;
        return $results;
    }
}
```

FILE: app/Jobs/BroadcastStreamMessage.php

```
<?php
namespace App\Jobs;

use App\Events\StreamChatMessageSent;
use App\Models\StreamMessage;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;
use Illuminate\Support\Facades\Log;

class BroadcastStreamMessage implements ShouldQueue
{
    use Queueable, InteractsWithQueue, SerializesModels;

    public $streamMessage;

    /**
     * Create a new job instance.
     *
     * @param StreamMessage $streamMessage
     */
    public function __construct(StreamMessage $streamMessage)
    {
        $this->streamMessage = $streamMessage;
    }

    /**
     * Execute the job.
     *
     * @return void
     */
    public function handle()
    {
        try {
            // Broadcast the message
            broadcast(new StreamChatMessageSent($this->streamMessage));
            Log::info("Stream message broadcasted: {$this->streamMessage->id}");
        } catch (\Exception $e) {
            Log::error("Error broadcasting stream message: {$this->streamMessage->id}", ['error' => $e->getMessage()]);
        }
    }
}
```

FILE: app/Jobs/DeleteGroupJob.php

```
<?php

namespace App\Jobs;

use App\Events\GroupDeleted;
use App\Models\Group;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
```

```
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;

class DeleteGroupJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    /**
     * The group instance.
     *
     * @var Group
     */
    public Group $group;

    /**
     * Create a new job instance.
     */
    public function __construct(Group $group)
    {
        $this->group = $group;
    }

    /**
     * Execute the job.
     */
    public function handle(): void
    {
        // Get group details
        $id = $this->group->id;
        $name = $this->group->name;

        // Detach any associated users
        $this->group->users()->detach();

        // Delete all associated messages in one query to avoid loading them into memory
        $this->group->messages()->delete();

        // Set last_message_id to null before deleting the group
        $this->group->last_message_id = null;
        $this->group->save();

        // Finally, delete the group
        $this->group->delete();

        // Dispatch the group deleted event after deletion
        GroupDeleted::dispatch($id, $name);
    }
}
```

FILE: app/Mail/OrderConfirmationMail.php

```
<?php

namespace App\Mail;

use Illuminate\Bus\Queueable;
use Illuminate\Mail\Mailable;
use Illuminate\Queue\SerializesModels;

class OrderConfirmationMail extends Mailable
{
    use Queueable, SerializesModels;
```

```
    public $orderData;

    /**
     * Create a new message instance.
     *
     * @param array $orderData
     * @return void
     */
    public function __construct($orderData)
    {
        $this->orderData = $orderData;
    }

    /**
     * Build the message.
     *
     * @return $this
     */
    public function build()
    {
        return $this->view('emails.order_confirmation') // View path for the email
            ->with('orderData', $this->orderData)
            ->subject('Order Confirmation - ' . $this->orderData['orderSummary']['orderId']);
    }
}
```

FILE: app/Models/Block.php

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Block extends Model
{
    use HasFactory;

    protected $table = 'blocks';

    // Relationships
    public function blocker()
    {
        return $this->belongsTo(User::class, 'blocker_id');
    }

    public function blocked()
    {
        return $this->belongsTo(User::class, 'blocked_id');
    }
    // Define the inverse of the 'blocks' relationship

}
```

FILE: app/Models/Category.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
```

```
use Illuminate\Database\Eloquent\Model;

class Category extends Model
{
    use HasFactory;

    protected $table = 'categories'; // Table name

    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Indicates if the model should manage created_at and updated_at timestamps

    protected $fillable = [
        'category_name', 'description'
    ];

    // Define relationships if needed (for example, categories can have many products)
    // public function products() {
    //     return $this->hasMany(Product::class);
    // }
    public function products()
    {
        return $this->belongsToMany(Product::class, 'products_categories', 'categories_id', 'products_id');
    }
}
```

FILE: app/Models/Chatbot.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Chatbot extends Model
{
    use HasFactory;

    // Define the table name (optional if following Laravel's convention)
    protected $table = 'chatbots';

    // Define the fillable fields (for mass assignment)
    protected $fillable = [
        'user_id',
        'history',
    ];

    // Specify that the history field is a JSON type
    protected $casts = [
        'history' => 'array',  // Automatically cast to an array
    ];

    // Optionally, you can define relationships or other methods here
}
```

FILE: app/Models/Comment.php

```
<?php
namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Comment extends Model
{
    use HasFactory;

    protected $fillable = [
        'post_id', // Foreign key to Post
        'user_id', // Foreign key to User
        'content',
    ];

    public function post()
    {
        return $this->belongsTo(Post::class);
    }

    public function user()
    {
        return $this->belongsTo(User::class); // Assumes you have a User model
    }
}
```

FILE: app/Models/ConnectionRequest.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class ConnectionRequest extends Model
{
    use HasFactory;

    protected $fillable = [
        'sender_id', // Changed for clarity
        'recipient_id', // Changed for clarity
        'status',
    ];

    protected $attributes = [
        'status' => 'pending', // Default status for new requests
    ];

    // Define relationships
    public function sender()
    {
        return $this->belongsTo(User::class, 'sender_id'); // Specify foreign key for clarity
    }

    public function recipient()
    {
        return $this->belongsTo(User::class, 'recipient_id'); // Specify foreign key for clarity
    }
}
```

FILE: app/Models/Conversation.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Conversation extends Model
{
    use HasFactory;

    protected $fillable = [
        'user_id1',
        'user_id2',
        'last_message_id',
    ];

    public function lastMessage()
    {
        return $this->belongsTo(Message::class, 'last_message_id');
    }

    public function user1()
    {
        return $this->belongsTo(User::class, 'user_id1');
    }

    public function user2()
    {
        return $this->belongsTo(User::class, 'user_id2');
    }

}
```

FILE: app/Models/Coupon.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Coupon extends Model
{
    use HasFactory;

    // Define the table name if it doesn't follow Laravel's naming convention
    protected $table = 'coupons';

    // If needed, define the columns that are fillable
    protected $fillable = [
        'discount',
        'expiration_date',
    ];

    // You can also define any specific logic for this model, if necessary.

    public function users()
    {
        return $this->belongsToMany(User::class, 'users_coupons', 'coupons_id', 'users_id')
            ->withTimestamps();  // Include timestamps if needed
    }
```

```
    public function products()
    {
        return $this->belongsToMany(Product::class, 'products_coupons', 'coupons_id', 'products_id')
                    ->withTimestamps();
    }
}
```

FILE: app/Models/FeedbackForm.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class FeedbackForm extends Model
{
    use HasFactory;

    protected $fillable = ['user_id', 'created_at', 'updated_at','order_id'];

    public function questions()
    {
        return $this->hasMany(FeedbackFormQuestion::class);
    }

    public function answers()
    {
        return $this->hasMany(FeedbackFormAnswer::class);
    }
}
```

FILE: app/Models/FeedbackFormAnswer.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class FeedbackFormAnswer extends Model
{
    use HasFactory;

    const CREATED_AT = null;
    const UPDATED_AT = null;

    protected $fillable = ['feedback_form_id', 'start_date', 'end_date'];

    public function feedbackForm()
    {
        return $this->belongsTo(FeedbackForm::class);
    }
}
```

FILE: app/Models/FeedbackFormQuestion.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class FeedbackFormQuestion extends Model
{
    use HasFactory;

    protected $fillable = ['id', 'type', 'question', 'description', 'data', 'feedback_form_id'];

}
```

FILE: app/Models/FeedbackFormQuestionAnswer.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class FeedbackFormQuestionAnswer extends Model
{
    use HasFactory;

    protected $fillable = ['feedback_form_question_id', 'feedback_form_answer_id', 'answer'];
}
```

FILE: app/Models/Follow.php

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Follow extends Model
{
    use HasFactory;

    protected $table = 'follows';

    // Disable the default timestamps if not needed, otherwise leave them on.
    public $timestamps = true;

    // Relationships
    public function follower()
    {
        return $this->belongsTo(User::class, 'follower_id');
    }

    public function following()
    {
        return $this->belongsTo(User::class, 'following_id');
    }
}
```

FILE: app/Models/Group.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Group extends Model
{
    use HasFactory;

    protected $fillable = [
        'name',
        'description',
        'owner_id',
        'last_message_id',
    ];

    public function users()
    {
        return $this->belongsToMany(User::class, 'group_users');
    }

    public function messages()
    {
        return $this->hasMany(Message::class);
    }

    public function owner()
    {
        return $this->belongsTo(User::class);
    }

    public function lastMessage()
    {
        return $this->belongsTo(Message::class, 'last_message_id');
    }

}
```

FILE: app/Models/Inventory.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Inventory extends Model
{
    use HasFactory;

    protected $table = 'inventory'; // Table name

    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Indicates if the model should manage created_at and updated_at timestamps

    protected $fillable = [
```

```
        'stock', 'last_updated', 'weight_per_unit', 'products_id', 'warehouses_id'
    ];

    // Define the relationship to the Product model (inventory belongs to a product)
    public function product()
    {
        return $this->belongsTo(Product::class, 'products_id');
    }

    // Define the relationship to the Warehouse model (inventory belongs to a warehouse)
    public function warehouse()
    {
        return $this->belongsTo(Warehouse::class, 'warehouses_id');
    }
}
```

FILE: app/Models/Invoice.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Invoice extends Model
{
    use HasFactory;

    protected $table = 'invoices'; // Table name
    protected $primaryKey = 'id'; // Primary key column
    public $timestamps = true; // Automatically manage 'created_at' and 'updated_at'

    protected $fillable = [
        'customer_name', 'created_date', 'payment_status', 'payment_method', 'description',
        'shipping_cost', 'total_amount', 'paid_amount', 'due_amount', 'currency',
        'discount', 'payments_id'
    ];

    // Relationships

    /**
     * Get the payment associated with the invoice.
     */
    public function payment()
    {
        return $this->belongsTo(Payment::class, 'payments_id');
    }
}
```

FILE: app/Models/Like.php

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Like extends Model
{
    use HasFactory;
```

```
    protected $fillable = [
        'post_id',
        'user_id',
    ];

    // Define relationships
    public function post()
    {
        return $this->belongsTo(Post::class);
    }

    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

FILE: app/Models/LocationHistory.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class LocationHistory extends Model
{
    use HasFactory;

    protected $table = 'location_histories'; // Table name
    public $timestamps = true; // Automatically manage 'created_at' and 'updated_at'

    // Define the fillable fields
    protected $fillable = [
        'vehicle_id', 'location', 'latitude', 'longitude', 'timestamp'
    ];

    /**
     * Get the vehicle associated with this location history.
     */
    public function vehicle()
    {
        return $this->belongsTo(Vehicle::class); // A location history belongs to one vehicle
    }
}
```

FILE: app/Models/Message.php

```
<?php

namespace App\Models;

use App\Observers\MessageObserver;
use Illuminate\Database\Eloquent\Attributes\ObservedBy;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

#[ObservedBy([MessageObserver::class])]
class Message extends Model
```

```
{
    use HasFactory;

    protected $fillable = [
        'message',
        'sender_id',
        'group_id',
        'receiver_id',
    ];

    public function sender()
    {
        return $this->belongsTo(User::class, 'sender_id');
    }

    public function receiver()
    {
        return $this->belongsTo(User::class, 'receiver_id');
    }

    public function group()
    {
        return $this->belongsTo(Group::class);
    }

    public function attachments()
    {
        return $this->hasMany(MessageAttachment::class);
    }
}
```

FILE: app/Models/MessageAttachment.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class MessageAttachment extends Model
{
    use HasFactory;

    protected $fillable = [
        'message_id',
        'name',
        'path',
        'mime',
        'size',
    ];
}
```

FILE: app/Models/Notification.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
```

```
class Notification extends Model
{
    use HasFactory;

    protected $fillable = [
        'recipient_id',  // Changed to recipient_id for clarity
        'type',
        'related_user', // Changed for clarity
        'related_post', // Changed for clarity
        'read',
    ];

    protected $casts = [
        'read' => 'boolean', // Cast read to boolean
    ];

    protected $attributes = [
        'read' => false, // Default to unread
    ];

    // Define notification types as constants
    const TYPES = [
        'like',
        'comment',
        'connectionAccepted',
    ];

    // Validation method for notification types
    public static function isValidType($type)
    {
        return in_array($type, self::TYPES);
    }

    // Define relationships
    public function recipient()
    {
        return $this->belongsTo(User::class, 'recipient_id'); // Specify foreign key for clarity
    }

    public function relatedUser()
    {
        return $this->belongsTo(User::class, 'related_user'); // Specify foreign key for clarity
    }

    public function relatedPost()
    {
        return $this->belongsTo(Post::class, 'related_post'); // Specify foreign key for clarity
    }
}
```

FILE: app/Models/Order.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Order extends Model
{
    use HasFactory;
```

```
    protected $table = 'orders'; // Table name

    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Indicates if the model should manage created_at and updated_at timestamps

    protected $fillable = [
        'status', 'order_date', 'user_id', 'tracking_number', 'total_amount'
    ];

    // Define the relationship to the User model (order belongs to a user)
    public function user()
    {
        return $this->belongsTo(User::class, 'user_id');
    }

    public function orderItems()
    {
        return $this->hasMany(OrderItem::class);
    }

}
```

FILE: app/Models/OrderItem.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class OrderItem extends Model
{
    use HasFactory;

    // Table name
    protected $table = 'order_items'; // Note that Laravel will use the plural form "order_items"
    protected $primaryKey = 'id';  // Default primary key

    // Define the columns that can be mass-assigned
    protected $fillable = [
        'quantity', 'total_amount', 'orders_id', 'products_id'
    ];

    // Relationship to Order
    public function order()
    {
        return $this->belongsTo(Order::class, 'orders_id');
    }

    // Relationship to Product
    public function product()
    {
        return $this->belongsTo(Product::class, 'products_id');
    }
}
```

FILE: app/Models/Payment.php

```
<?php
```

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Payment extends Model
{
    use HasFactory;

    protected $table = 'payments'; // Table name

    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Indicates if the model should manage created_at and updated_at timestamps

    protected $fillable = [
        'total_amount', 'paid_amount', 'due_amount', 'payment_method', 'payment_status', 'payment_date', 'gateway', 'currency',
'order_id', 'providers_id'
    ];

    // Define the relationship to the Order model (payment belongs to an order)
    public function order()
    {
        return $this->belongsTo(Order::class, 'order_id');
    }

    // Define the relationship to the Provider model (payment belongs to a provider)
    public function provider()
    {
        return $this->belongsTo(Provider::class, 'providers_id');
    }
}
```

FILE: app/Models/Permission.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Permission extends Model
{
    use HasFactory;

    protected $table = 'permissions'; // Table name

    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Indicates if the model should manage created_at and updated_at timestamps

    protected $fillable = [
        'permission_name', 'description'
    ];

    // Define any relationships if needed (for example, permissions can be associated with roles or users in some cases)

    public function roles()
    {
        return $this->belongsToMany(Role::class, 'roles_permissions', 'permissions_id', 'roles_id');
    }
}
```

FILE: app/Models/Post.php

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    use HasFactory;

    protected $fillable = [
        'author_id',
        'content',
        'image',
    ];

    protected $attributes = [
        'image' => '',
    ];

    public function author()
    {
        return $this->belongsTo(User::class, 'author_id');
    }

    public function comments()
    {
        return $this->hasMany(Comment::class, 'post_id');
    }

    // public function likes()
    // {
    //     return $this->hasMany(Like::class, 'post_id');
    // }
    public function likes()
    {
        return $this->belongsToMany(User::class, 'likes', 'post_id', 'user_id');
    }

    public function likeCount()
    {
        return $this->likes()->count();
    }
}
```

FILE: app/Models/Product.php

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    use HasFactory;

    protected $table = 'products'; // Table name
```

```
    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Indicates if the model should manage created_at and updated_at timestamps

    protected $fillable = [
        'name', 'price', 'description', 'supplier_id', 'isFeatured', 'image' // Add 'image' here
    ];

    // Define the relationship to the User model (supplier of the product)
    public function supplier()
    {
        return $this->belongsTo(User::class, 'supplier_id');
    }

    // Define the relationship to the Category model
    public function categories()
    {
        return $this->belongsToMany(Category::class, 'products_categories', 'products_id', 'categories_id');
    }

    // Define the relationship to the Coupon model
    public function coupons()
    {
        return $this->belongsToMany(Coupon::class, 'products_coupons', 'products_id', 'coupons_id')
                    ->withTimestamps();
    }
}
```

FILE: app/Models/ProductCategory.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class ProductCategory extends Model
{
    use HasFactory;
    // Since it's a pivot table, we often don’t need the default timestamps.
    public $timestamps = false;

    // Define the table name (optional since Laravel can infer it).
    protected $table = 'products_categories';

    // Define the primary key for this table (composite primary key is not natively supported by Eloquent).
    // But you can manually handle the composite key logic, or just rely on the pivot table's simplicity.
    protected $primaryKey = ['products_id', 'categories_id'];

    // Disable auto-increment as this is not an autoincrementing table.
    public $incrementing = false;

    // If needed, you can define the fillable attributes for mass assignment.
    protected $fillable = ['products_id', 'categories_id'];

    // Define relationships, but typically, pivot models don’t have additional methods.
}
```

FILE: app/Models/ProductCoupon.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class ProductCoupon extends Model
{
    use HasFactory;
    // Specify the table name (optional if following naming convention)
    protected $table = 'products_coupons';

    // Disable Eloquent timestamps if you don't want them (optional)
    public $timestamps = true;

    // Define the fillable attributes (if any)
    protected $fillable = [
        'products_id',
        'coupons_id',
    ];

    // Define the relationship with Product
    public function product()
    {
        return $this->belongsTo(Product::class, 'products_id');
    }

    // Define the relationship with Coupon
    public function coupon()
    {
        return $this->belongsTo(Coupon::class, 'coupons_id');
    }
}
```

FILE: app/Models/Provider.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Provider extends Model
{
    use HasFactory;

    protected $table = 'providers'; // Table name

    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Indicates if the model should manage created_at and updated_at timestamps

    protected $fillable = [
        'type', 'terms_of_service'
    ];

    // You can define any relationships if necessary, depending on your business logic
}
```

FILE: app/Models/Rating.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Rating extends Model
{
    use HasFactory;

    protected $table = 'ratings'; // Table name

    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Laravel will automatically manage the 'created_at' and 'updated_at' columns

    protected $fillable = [
        'rating_value', 'feedback', 'date_created', 'shipments_id', 'products_id'
    ];

    // Define relationships

    /**
     * Get the shipment that owns the rating.
     */
    public function shipment()
    {
        return $this->belongsTo(Shipment::class, 'shipments_id');
    }

    /**
     * Get the product that owns the rating.
     */
    public function product()
    {
        return $this->belongsTo(Product::class, 'products_id');
    }
}
```

FILE: app/Models/Role.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Role extends Model
{
    use HasFactory;

    protected $table = 'roles'; // Table name

    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Laravel will automatically manage the 'created_at' and 'updated_at' columns

    protected $fillable = [
```

```
        'role_name', 'description'
    ];

    // Define relationships (if needed)
    public function users()
    {
        return $this->belongsToMany(User::class, 'users_roles', 'roles_id', 'users_id');
    }

    public function permissions()
    {
        return $this->belongsToMany(Permission::class, 'roles_permissions', 'roles_id', 'permissions_id');
    }
}
```

FILE: app/Models/RolePermission.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class RolePermission extends Model
{
    use HasFactory;
    // Since this is a pivot table, we don’t need timestamps.
    public $timestamps = false;

    // Define the table name (optional, can be inferred by Laravel).
    protected $table = 'roles_permissions';

    // Define the composite primary key (Laravel doesn’t natively support composite keys).
    protected $primaryKey = ['permissions_id', 'roles_id'];

    // Disable auto-incrementing, as this table doesn't have an auto-incrementing primary key.
    public $incrementing = false;

    // You can define fillable attributes here if needed.
    protected $fillable = ['permissions_id', 'roles_id'];
}
```

FILE: app/Models/RouteCondition.php

```
<?php
// app/Models/RouteCondition.php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class RouteCondition extends Model
{
    use HasFactory;

    protected $fillable = [
        'weather',
        'road_condition',
        'traffic_condition',
```

```
        'has_accident',
        'accident_description',
        'road_closure_description',
        'reported_at',
    ];

}
```

FILE: app/Models/RouteDetail.php

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class RouteDetail extends Model
{
    use HasFactory;

    protected $fillable = [
        'route_optimization_id',
        'route_name',
        'supplier_name',
        'warehouse_name_1',
        'warehouse_name_2',
        'start_location',
        'end_location',
        'start_latitude',
        'start_longitude',
        'end_latitude',
        'end_longitude',
        'estimated_time',
        'distance',
        'route_condition_id',
        'vehicle_id',
    ];

    /**
     * Define the relationship with the RouteOptimization model.
     */


    public function routeOptimization()
    {
        return $this->belongsTo(RouteOptimization::class);
    }
    public function routeCondition()
    {
        return $this->belongsTo(RouteCondition::class, 'route_condition_id');
    }
    public function vehicle()
    {
        return $this->belongsTo(Vehicle::class, 'vehicle_id');
    }
}
```

FILE: app/Models/RouteOptimization.php

```
<?php
namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class RouteOptimization extends Model
{
    use HasFactory;

    protected $fillable = [
        'shipments_id',
        'total_distance',
    ];

    /**
     * Define the relationship with the RouteDetail model.
     */
    public function routeDetails()
    {
        return $this->hasMany(RouteDetail::class);
    }

    /**
     * Define the relationship with the Shipment model.
     */
    public function shipment()
    {
        return $this->belongsTo(Shipment::class,'shipments_id');
    }
}
```

FILE: app/Models/Shipment.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Shipment extends Model
{
    use HasFactory;

    protected $table = 'shipments'; // Table name

    protected $primaryKey = 'id'; // Primary key column (composite key handled via migration)

    public $timestamps = true; // Laravel will automatically manage the 'created_at' and 'updated_at' columns

    protected $fillable = [
        'status', 'estimated_arrival', 'actual_arrival', 'origin', 'destination',
        'shipment_method', 'tracking_number', 'last_updated', 'total_amount',
        'providers_id', 'orders_id'
    ];

    // Define relationships

    /**
     * Get the provider that owns the shipment.
     */
    public function provider()
    {
        return $this->belongsTo(Provider::class, 'providers_id');
    }
```

```
    /**
     * Get the order that owns the shipment.
     */
    public function order()
    {
        return $this->belongsTo(Order::class, 'orders_id');
    }
    /**
     * Define the relationship with the RouteOptimization model.
     */
    public function routeOptimizations()
    {
        return $this->hasMany(RouteOptimization::class);
    }
}
```

FILE: app/Models/Stream.php

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Stream extends Model
{
    use HasFactory;

    protected $table = 'streams';

    protected $fillable = [
        'title',
        'thumbnail',
        'ingressId',
        'serverUrl',
        'streamKey',
        'isLive',
        'isChatEnabled',
        'isChatDelayed',
        'isChatFollowersOnly',
        'user_id',
    ];

    // Relationship with User model
    public function user()
    {
        return $this->belongsTo(User::class, 'user_id');
    }

    // Relationship with StreamMessage model
    public function streamMessages()
    {
        return $this->hasMany(StreamMessage::class, 'stream_id');
    }

    // Relationship with StreamGift model
    public function gifts()
    {
        return $this->hasMany(StreamGift::class, 'stream_id'); // A Stream can have many gifts
    }
}
```

FILE: app/Models/StreamGift.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use App\Enums\GiftTypeEnum;

class StreamGift extends Model
{
    use HasFactory;

    // Table name (optional, Laravel will guess it)
    protected $table = 'stream_gifts';

    // Define fillable attributes
    protected $fillable = [
        'owner_id',
        'gift_type',
        'price', // Add the price attribute to the fillable array
        'stream_id', // The foreign key referencing the stream
    ];

    // Relationship with the User model (assuming owner is a user)
    public function owner()
    {
        return $this->belongsTo(User::class, 'owner_id');
    }

    // Relationship with the Stream model
    public function stream()
    {
        return $this->belongsTo(Stream::class, 'stream_id'); // A StreamGift belongs to a Stream
    }

    // Accessor to get human-readable gift name
    public function getGiftNameAttribute()
    {
        return ucfirst(str_replace('_', ' ', $this->gift_type));
    }

    // Set price based on gift type
    public function setPriceBasedOnGiftType()
    {
        switch ($this->gift_type) {
            case GiftTypeEnum::Lion->value:
                return 100.00; // Price for 'lion' gift
            case GiftTypeEnum::Flower->value:
                return 50.00; // Price for 'flower' gift
            case GiftTypeEnum::Star->value:
                return 200.00; // Price for 'star' gift
            case GiftTypeEnum::Heart->value:
                return 150.00; // Price for 'heart' gift
            default:
                return 0.00; // Default price
        }
    }

    // Override the saving method to set the price before saving the gift
    public static function boot()
    {
        parent::boot();
```

```
        static::creating(function ($streamGift) {
            $streamGift->price = $streamGift->setPriceBasedOnGiftType();
        });
    }
}
```

FILE: app/Models/StreamMessage.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class StreamMessage extends Model
{
    use HasFactory;

    protected $fillable = [
        'message',
        'creator_id', // Creator (sender) ID
        'viewer_id',  // Viewer (receiver) ID
        'stream_id',  // Foreign key to the Stream model
    ];

    // Creator is the sender of the message
    public function creator()
    {
        return $this->belongsTo(User::class, 'creator_id');
    }

    // Viewer is the receiver of the message
    public function viewer()
    {
        return $this->belongsTo(User::class, 'viewer_id');
    }

    // Relationship with the stream this message belongs to
    public function stream()
    {
        return $this->belongsTo(Stream::class, 'stream_id');
    }
}```

## FILE: app/Models/User.php
```php
<?php
namespace App\Models;

use Illuminate\Foundation\Auth\User as Authenticatable;  // Use Authenticatable
use Laravel\Sanctum\HasApiTokens;
use Illuminate\Database\Eloquent\Factories\HasFactory; // Correct namespace for HasFactory
use Twilio\Rest\Client; // Import Twilio Client

class User extends Authenticatable  // Extend Authenticatable
{
    use HasFactory, HasApiTokens;

    protected $table = 'users';

    protected $primaryKey = 'id';

    public $timestamps = true;
```
```

```
    protected $fillable = [
        'first_name',
        'last_name',
        'phone_number',
        'ip_address',
        'email',
        'password',
        'two_factor_enabled',
        'last_login',
        'language',
        'is_admin',
        'last_password_change',
        'profile_picture',
        'banner_img',
        'headline',
        'about',
        'two_factor_code',
        'two_factor_code_created_at',
    ];

    protected $casts = [
        'two_factor_enabled' => 'boolean',
        'is_admin' => 'boolean',
        'last_login' => 'datetime',
        'last_password_change' => 'datetime',
        'two_factor_code_created_at' => 'datetime',
    ];

    // Relationships and other methods as usual
    public function roles()
    {
        return $this->belongsToMany(Role::class, 'users_roles', 'users_id', 'roles_id');
    }

    // User can have many orders
    public function orders()
    {
        return $this->hasMany(Order::class);
    }

    public function coupons()
    {
        return $this->belongsToMany(Coupon::class, 'users_coupons', 'users_id', 'coupons_id')->withTimestamps();
    }

    public function addresses()
    {
        return $this->hasMany(UserAddress::class, 'users_id');
    }

    public function primaryAddress()
    {
        // Corrected: Return a relationship instance
        return $this->hasOne(UserAddress::class, 'users_id')->where('is_primary', true);
    }


    // Get the user's primary address as a concatenated string
    public function getPrimaryAddress()
    {
        $address = $this->primaryAddress;

        if ($address) {
            // Combine the address components into a single string
            $addressString = $address->address_line1;
```

```
        if ($address->address_line2) {
            $addressString .= ', ' . $address->address_line2;
        }

        $addressString .= ', ' . $address->city;
        $addressString .= ', ' . $address->state;
        $addressString .= ' ' . $address->postal_code;
        $addressString .= ', ' . $address->country;

        return $addressString; // Return the combined address string
    }

    return null;  // No address found
}

// Define relationships
public function posts()
{
    return $this->hasMany(Post::class, 'author_id'); // Posts authored by this user
}

// Define a relationship for connections
public function connections()
{
    return $this->hasMany(UserConnection::class, 'user_id');
}

public function chats()
{
    return $this->hasMany(ChatBot::class, 'user_id');
}

// Define a relationship for connected users
public function connectedUsers()
{
    return $this->belongsToMany(User::class, 'user_connections', 'user_id', 'connection_id');
}

// Add a connection
public function addConnection($userId)
{
    if (!$this->connectedUsers()->where('connection_id', $userId)->exists()) {
        $this->connectedUsers()->attach($userId); // Use the relationship to attach
    }
}

// Remove a connection
public function removeConnection($userId)
{
    if ($this->connectedUsers()->where('connection_id', $userId)->exists()) {
        $this->connectedUsers()->detach($userId); // Use the relationship to detach
    }
}

// Get all connections
public function getConnections()
{
    return $this->connectedUsers; // Returns a collection of connected User models
}


public function groups()
{
    return $this->belongsToMany(Group::class, 'group_users');
}
```

```
    // New relationships for following and blocking
    // New relationships for following and blocking
    public function following()
    {
        return $this->belongsToMany(User::class, 'follows', 'follower_id', 'followed_id')
            ->withTimestamps()
            ->as('follow');
    }

    public function followedBy()
    {
        return $this->belongsToMany(User::class, 'follows', 'followed_id', 'follower_id')
            ->withTimestamps()
            ->as('follow');
    }

    public function blocking()
    {
        return $this->belongsToMany(User::class, 'blocks', 'blocker_id', 'blocked_id')
            ->withTimestamps()
            ->as('block');
    }

    public function blockedBy()
    {
        return $this->belongsToMany(User::class, 'blocks', 'blocked_id', 'blocker_id')
            ->withTimestamps()
            ->as('block');
    }

    // Add a follow relationship
    public function follow($userId)
    {
        if (!$this->following()->where('followed_id', $userId)->exists()) {
            $this->following()->attach($userId);
        }
    }

    // Remove a follow relationship
    public function unfollow($userId)
    {
        if ($this->following()->where('followed_id', $userId)->exists()) {
            $this->following()->detach($userId);
        }
    }

    // Add a block relationship
    public function block($userId)
    {
        if (!$this->blocking()->where('blocked_id', $userId)->exists()) {
            $this->blocking()->attach($userId);
        }
    }

    // Remove a block relationship
    public function unblock($userId)
    {
        if ($this->blocking()->where('blocked_id', $userId)->exists()) {
            $this->blocking()->detach($userId);
        }
    }

    // Add a stream relationship if needed
    public function stream()
    {
        return $this->hasMany(Stream::class, 'user_id');
```

```
    }

    // Define the "blocks" relationship to the Block model (where a user is blocking others)
    public function blocks()
    {
        return $this->hasMany(Block::class, 'blocker_id'); // assuming 'blocker_id' is the foreign key in the 'blocks' table
    }

    // Send 2FA code to user's phone number
    public function sendTwoFactorCode()
    {
        // Generate a random 6-digit code
        $code = rand(100000, 999999);

        // Save code and timestamp to the user
        $this->two_factor_code = $code;
        $this->two_factor_code_created_at = now();
        $this->save();

        // Send code via SMS using Twilio
        $this->sendSms($this->phone_number, "Your verification code is: $code");

        return $code;
    }

    // Verify the code entered by the user
    public function verifyTwoFactorCode($code)
    {
        if ($this->two_factor_code && $this->two_factor_code_created_at && now()->diffInMinutes($this->two_factor_code_created_at) <= 10)
{
            // If the code matches and is not older than 10 minutes
            if ($this->two_factor_code == $code) {
                // Clear the code after successful verification
                $this->two_factor_code = null;
                $this->two_factor_code_created_at = null;
                $this->save();
                return true;
            }
        }

        return false; // Invalid code or expired
    }

    // Send SMS via Twilio
    private function sendSms($to, $message)
    {
        $sid = env('TWILIO_SID');
        $token = env('TWILIO_AUTH_TOKEN');
        $from = env('TWILIO_PHONE_NUMBER');

        $twilio = new Client($sid, $token);

        $twilio->messages->create(
            $to, // The user's phone number
            [
                'from' => $from, // Your Twilio phone number
                'body' => $message
            ]
        );
    }

}
```

FILE: app/Models/UserAddress.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class UserAddress extends Model
{
    use HasFactory;

    // Define the table name if it's not plural of the model name
    protected $table = 'user_addresses';

    // Set the primary key if it's not the default 'id'
    protected $primaryKey = 'id';

    // Enable timestamps if you want to track created_at and updated_at
    public $timestamps = true;

    // Define the fillable fields
    protected $fillable = [
        'users_id',      // Foreign key for the user
        'address_line1', // Address line 1
        'address_line2', // Address line 2 (optional)
        'city',          // City
        'state',         // State
        'postal_code',   // Postal code
        'country',       // Country
        'is_primary',    // Whether the address is the primary one for the user
    ];

    // Define the relationship with the User model
    public function user()
    {
        return $this->belongsTo(User::class, 'users_id');
    }
}
```

FILE: app/Models/UserConnection.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class UserConnection extends Model
{
    use HasFactory;

    protected $fillable = [
        'user_id',
        'connection_id',
    ];

    // Define relationships
    public function user()
    {
        return $this->belongsTo(User::class, 'user_id');
    }
```

```
    public function connection()
    {
        return $this->belongsTo(User::class, 'connection_id');
    }
}
```

FILE: app/Models/UserCoupon.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class UserCoupon extends Model
{
    use HasFactory;
    // Since it's a pivot table, we don’t need timestamps.
    public $timestamps = true;

    // Define the table name (optional, can be inferred by Laravel).
    protected $table = 'users_coupons';

    // Define the composite primary key (Laravel doesn’t natively support composite keys).
    protected $primaryKey = ['users_id', 'coupons_id'];

    // Disable auto-incrementing, as this table doesn't have an auto-incrementing primary key.
    public $incrementing = false;

    // Define the attributes that are mass assignable (for security).
    protected $fillable = ['users_id', 'coupons_id'];
}
```

FILE: app/Models/UserRole.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class UserRole extends Model
{
    use HasFactory;

    // Since this is a pivot table, we often don’t need the default timestamps.
    public $timestamps = false;

    // Define the table name (optional, but can be useful for clarity).
    protected $table = 'users_roles';

    // Disable auto-increment, as this is not an autoincrementing table.
    public $incrementing = false;

    // Define the attributes that are mass assignable (for security).
    protected $fillable = ['roles_id', 'users_id'];

    // Define the relationship with User
```

```
    public function user()
    {
        return $this->belongsTo(User::class, 'users_id');
    }

    // Define the relationship with Role
    public function role()
    {
        return $this->belongsTo(Role::class, 'roles_id');
    }
}
```

FILE: app/Models/Vehicle.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Vehicle extends Model
{
    use HasFactory;

    protected $table = 'vehicles'; // Table name

    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Automatically manage 'created_at' and 'updated_at'

    protected $fillable = [
        'license_plate', 'type', 'driver_id', 'capacity', 'fuel_capacity',
        'current_location', 'last_serviced', 'status', 'last_fuel_refill',
        'last_location_update', 'mileage', 'maintenance_logs', 'vehicle_management_id',
        'fuel_interval', 'fuel_type', 'vin', // Existing fields
        'brand', 'model', 'year_of_manufacture' // New fields
    ];

    // Relationships

    /**
     * Get the driver (user) associated with the vehicle.
     */
    public function driver()
    {
        return $this->belongsTo(User::class, 'driver_id');
    }

    /**
     * Get the vehicle management record associated with the vehicle.
     */
    public function vehicleManagement()
    {
        return $this->belongsTo(VehicleManagement::class, 'vehicle_management_id');
    }
}
```

FILE: app/Models/VehicleManagement.php

```
<?php
```

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class VehicleManagement extends Model
{
    use HasFactory;

    protected $table = 'vehicle_management'; // Table name

    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Indicates if the model should manage created_at and updated_at timestamps

    protected $fillable = [
        'fuel_consumption',
        'distance_traveled',
        'maintenance_status',
        'last_maintenance_date',
        'maintenance_schedule',
        'maintenance_cost',
        'users_id',
    ];

    // Define the relationship to the User model (user who owns the vehicle)
    public function user()
    {
        return $this->belongsTo(User::class, 'users_id');
    }
}
```

FILE: app/Models/Warehouse.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Warehouse extends Model
{
    use HasFactory;

    protected $table = 'warehouses'; // Table name

    protected $primaryKey = 'id'; // Primary key column

    public $timestamps = true; // Indicates if the model should manage created_at and updated_at timestamps

    protected $fillable = [
        'warehouse_name', 'location', 'capacity', 'available_space', 'users_id'
    ];

    // Define the relationship to the User model (user who owns the warehouse)
    public function user()
    {
        return $this->belongsTo(User::class, 'users_id');
    }
    // Define the relationship to the Inventory model (a warehouse has many inventory items)
    public function inventory()
    {
```

```
        return $this->hasMany(Inventory::class, 'warehouses_id'); // 'warehouses_id' is the foreign key in the 'inventory' table
    }
}
```

FILE: app/Observers/MessageObserver.php

```
<?php

namespace App\Observers;

use App\Models\Conversation;
use App\Models\Group;
use App\Models\Message;
use Illuminate\Support\Facades\Storage;


class MessageObserver
{
    // listen when message is deleted
    public function deleting(Message $message)
    {
        // Iterate over the message's attachments and delete them from file system
        $message->attachments->each(function ($attachment) {
            // Delete attachment file from file system saved on public disk
            $dir = dirname($attachment->path);
            Storage::disk('public')->deleteDirectory($dir);
        });

        // delete all attachments related to the message from the database
        $message->attachments()->delete();

        // Update Group and Conversation's last message if the message is the last message
        if ($message->group_id) {
            $group = Group::where('last_message_id', $message->id)->first();

            if ($group) {
                $prevMessage = Message::where('group_id', $message->group_id)
                    ->where('id', '!=', $message->id)
                    ->latest()
                    ->limit(1)
                    ->first();

                if ($prevMessage) {
                    $group->last_message_id = $prevMessage->id;
                    $group->save();
                }
            }

        } else {
            $conversation = Conversation::where('last_message_id', $message->id)->first();

            // if the message is the last message in the conversation
            if ($conversation) {
                $prevMessage = Message::where(function ($query) use ($message) {
                    $query->where('sender_id', $message->sender_id)
                        ->where('receiver_id', $message->receiver_id)
                        ->orWhere('sender_id', $message->receiver_id)
                        ->where('receiver_id', $message->sender_id);
                })
                    ->where('id', '!=', $message->id)
                    ->latest()
                    ->limit(1)
                    ->first();
```

```
                if ($prevMessage) {
                    $conversation->last_message_id = $prevMessage->id;
                    $conversation->save();
                }
            }
        }
    }
}
```

FILE: app/Providers/AppServiceProvider.php

```
<?php

namespace App\Providers;

use Illuminate\Auth\Notifications\ResetPassword;
use Illuminate\Support\ServiceProvider;
use App\Services\GoogleMapsService;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     */
    public function register(): void
    {
        //
    }

    /**
     * Bootstrap any application services.
     */
    public function boot(): void
    {
        ResetPassword::createUrlUsing(function (object $notifiable, string $token) {
            return config('app.frontend_url')."/password-reset/$token?email={$notifiable->getEmailForPasswordReset()}";
        });
        $this->app->singleton(GoogleMapsService::class, function ($app) {
            return new GoogleMapsService();
        });
    }
}
```

FILE: app/Providers/HorizonServiceProvider.php

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\Gate;
use Laravel\Horizon\Horizon;
use Laravel\Horizon\HorizonApplicationServiceProvider;

class HorizonServiceProvider extends HorizonApplicationServiceProvider
{
    /**
     * Bootstrap any application services.
     */
    public function boot(): void
    {
        parent::boot();
```

```
        // Horizon::routeSmsNotificationsTo('15556667777');
        // Horizon::routeMailNotificationsTo('example@example.com');
        // Horizon::routeSlackNotificationsTo('slack-webhook-url', '#channel');
    }

    /**
     * Register the Horizon gate.
     *
     * This gate determines who can access Horizon in non-local environments.
     */
    protected function gate(): void
    {
        Gate::define('viewHorizon', function ($user) {
            return in_array($user->email, [
                //
            ]);
        });
    }
}
```

FILE: app/Services/CloudinaryService.php

```
<?php

namespace App\Services;

use Cloudinary\Api\Upload\UploadApi;
use Cloudinary\Api\Resources\ExplicitResource;
use Cloudinary\Cloudinary;

class CloudinaryService
{
    protected $cloudinary;

    public function __construct()
    {
        $this->cloudinary = new Cloudinary([
            'cloud' => [
                'cloud_name' => env('CLOUDINARY_CLOUD_NAME'),
                'api_key'    => env('CLOUDINARY_API_KEY'),
                'api_secret' => env('CLOUDINARY_API_SECRET'),
            ],
            'url' => [
                'secure' => true, // Ensures secure URLs for uploaded media
            ]
        ]);
    }

    /**
     * Upload an image to Cloudinary.
     *
     * @param  string $filePath
     * @param  array  $options
     * @return array
     */
    public function uploadImage($filePath, $options = [])
    {
        try {
            // Upload the file to Cloudinary
            $response = $this->cloudinary->uploadApi()->upload($filePath, $options);
            return $response;
        } catch (\Exception $e) {
            return ['error' => $e->getMessage()];
```

```
        }
    }

    /**
     * Upload a video to Cloudinary.
     *
     * @param  string $filePath
     * @param  array  $options
     * @return array
     */
    public function uploadVideo($filePath, $options = [])
    {
        try {
            $response = $this->cloudinary->uploadApi()->upload($filePath, array_merge($options, ['resource_type' => 'video']));
            return $response;
        } catch (\Exception $e) {
            return ['error' => $e->getMessage()];
        }
    }

    /**
     * Delete a media item from Cloudinary.
     *
     * @param  string $publicId
     * @return array
     */
    public function deleteMedia($publicId)
    {
        try {
            $response = $this->cloudinary->api()->deleteResources($publicId);
            return $response;
        } catch (\Exception $e) {
            return ['error' => $e->getMessage()];
        }
    }
}
```

FILE: app/Services/RecursiveTranslation.php

```
<?php

namespace App\Services;

use Stichoza\GoogleTranslate\GoogleTranslate;

class RecursiveTranslation
{
    protected $translator;

    public function __construct()
    {
        $this->translator = new GoogleTranslate();
    }

    /**
     * Recursively translate the input text into the target language.
     *
     * @param mixed $input
     * @param string $targetLanguage
     * @return mixed
     */
    public function translate($input, $targetLanguage)
    {
        // If the input is a string, translate it
```

```
        if (is_string($input)) {
            return $this->translateText($input, $targetLanguage);
        }

        // If the input is an array, loop through its elements and translate
        if (is_array($input)) {
            foreach ($input as $key => $item) {
                $input[$key] = $this->translate($item, $targetLanguage);
            }
            return $input;
        }

        // If the input is an object, loop through its properties and translate
        if (is_object($input)) {
            foreach (get_object_vars($input) as $property => $value) {
                $input->$property = $this->translate($value, $targetLanguage);
            }
            return $input;
        }

        // If it's neither a string, array, nor object, return the input
        return $input;
    }

    /**
     * Translate a single string of text.
     *
     * @param string $text
     * @param string $targetLanguage
     * @return string
     */
    private function translateText($text, $targetLanguage)
    {
        try {
            // Set the target language for translation
            $this->translator->setTarget($targetLanguage);
            // Translate the text
            return $this->translator->translate($text);
        } catch (\Exception $e) {
            // Handle errors gracefully
            return "Translation failed: " . $e->getMessage();
        }
    }
}
```

FILE: bootstrap/app.php

```
<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        channels: __DIR__.'/../routes/channels.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        $middleware->api(prepend: [
            \Laravel\Sanctum\Http\Middleware\EnsureFrontendRequestsAreStateful::class,
```

```
        ]);

        $middleware->alias([
            'verified' => \App\Http\Middleware\EnsureEmailIsVerified::class,
            'role' => \App\Http\Middleware\RoleRoute::class,
            'custom_cors' => \App\Http\Middleware\CustomCors::class, // Add this line
            'translation' => \App\Http\Middleware\TranslateResponse::class,
        ]);

        //
    })
    ->withExceptions(function (Exceptions $exceptions) {
        //
    })->create();
```

FILE: bootstrap/providers.php

```
<?php

return [
    App\Providers\AppServiceProvider::class,
    App\Providers\HorizonServiceProvider::class,
];
```

FILE: config/app.php

```
<?php

return [

    /*
    |--------------------------------------------------------------------------
    | Application Name
    |--------------------------------------------------------------------------
    |
    | This value is the name of your application, which will be used when the
    | framework needs to place the application's name in a notification or
    | other UI elements where an application name needs to be displayed.
    |
    */

    'name' => env('APP_NAME', 'Laravel'),

    /*
    |--------------------------------------------------------------------------
    | Application Environment
    |--------------------------------------------------------------------------
    |
    | This value determines the "environment" your application is currently
    | running in. This may determine how you prefer to configure various
    | services the application utilizes. Set this in your ".env" file.
    |
    */

    'env' => env('APP_ENV', 'production'),

    /*
    |--------------------------------------------------------------------------
    | Application Debug Mode
    |--------------------------------------------------------------------------
    |
```

```
    | When your application is in debug mode, detailed error messages with
    | stack traces will be shown on every error that occurs within your
    | application. If disabled, a simple generic error page is shown.
    |
    */

    'debug' => (bool) env('APP_DEBUG', false),

    /*
    |--------------------------------------------------------------------------
    | Application URL
    |--------------------------------------------------------------------------
    |
    | This URL is used by the console to properly generate URLs when using
    | the Artisan command line tool. You should set this to the root of
    | the application so that it's available within Artisan commands.
    |
    */

    'url' => env('APP_URL', 'http://localhost'),

    /*
    |--------------------------------------------------------------------------
    | Application Timezone
    |--------------------------------------------------------------------------
    |
    | Here you may specify the default timezone for your application, which
    | will be used by the PHP date and date-time functions. The timezone
    | is set to "UTC" by default as it is suitable for most use cases.
    |
    */

    'timezone' => env('APP_TIMEZONE', 'UTC'),

    /*
    |--------------------------------------------------------------------------
    | Application Locale Configuration
    |--------------------------------------------------------------------------
    |
    | The application locale determines the default locale that will be used
    | by Laravel's translation / localization methods. This option can be
    | set to any locale for which you plan to have translation strings.
    |
    */

    'locale' => env('APP_LOCALE', 'en'),

    'fallback_locale' => env('APP_FALLBACK_LOCALE', 'en'),

    'faker_locale' => env('APP_FAKER_LOCALE', 'en_US'),

    /*
    |--------------------------------------------------------------------------
    | Encryption Key
    |--------------------------------------------------------------------------
    |
    | This key is utilized by Laravel's encryption services and should be set
    | to a random, 32 character string to ensure that all encrypted values
    | are secure. You should do this prior to deploying the application.
    |
    */

    'cipher' => 'AES-256-CBC',

    'key' => env('APP_KEY'),
```

```
    'previous_keys' => [
        ...array_filter(
            explode(',', env('APP_PREVIOUS_KEYS', ''))
        ),
    ],

    /*
    |--------------------------------------------------------------------------
    | Maintenance Mode Driver
    |--------------------------------------------------------------------------
    |
    | These configuration options determine the driver used to determine and
    | manage Laravel's "maintenance mode" status. The "cache" driver will
    | allow maintenance mode to be controlled across multiple machines.
    |
    | Supported drivers: "file", "cache"
    |
    */

    'maintenance' => [
        'driver' => env('APP_MAINTENANCE_DRIVER', 'file'),
        'store' => env('APP_MAINTENANCE_STORE', 'database'),
    ],

];
```

FILE: config/auth.php

```
<?php

return [

    /*
    |--------------------------------------------------------------------------
    | Authentication Defaults
    |--------------------------------------------------------------------------
    |
    | This option defines the default authentication "guard" and password
    | reset "broker" for your application. You may change these values
    | as required, but they're a perfect start for most applications.
    |
    */

    'defaults' => [
        'guard' => env('AUTH_GUARD', 'web'),
        'passwords' => env('AUTH_PASSWORD_BROKER', 'users'),
    ],

    /*
    |--------------------------------------------------------------------------
    | Authentication Guards
    |--------------------------------------------------------------------------
    |
    | Next, you may define every authentication guard for your application.
    | Of course, a great default configuration has been defined for you
    | which utilizes session storage plus the Eloquent user provider.
    |
    | All authentication guards have a user provider, which defines how the
    | users are actually retrieved out of your database or other storage
    | system used by the application. Typically, Eloquent is utilized.
    |
    | Supported: "session"
    |
    */
```

```
    'guards' => [
        'web' => [
            'driver' => 'session',
            'provider' => 'users',
        ],

        'api' => [
            'driver' => 'jwt',  // Use JWT driver for API authentication
            'provider' => 'users',
        ],
    ],


    /*
    |--------------------------------------------------------------------------
    | User Providers
    |--------------------------------------------------------------------------
    |
    | All authentication guards have a user provider, which defines how the
    | users are actually retrieved out of your database or other storage
    | system used by the application. Typically, Eloquent is utilized.
    |
    | If you have multiple user tables or models you may configure multiple
    | providers to represent the model / table. These providers may then
    | be assigned to any extra authentication guards you have defined.
    |
    | Supported: "database", "eloquent"
    |
    */

    'providers' => [
        'users' => [
            'driver' => 'eloquent',
            'model' => env('AUTH_MODEL', App\Models\User::class),
        ],

        // 'users' => [
        //     'driver' => 'database',
        //     'table' => 'users',
        // ],
    ],

    /*
    |--------------------------------------------------------------------------
    | Resetting Passwords
    |--------------------------------------------------------------------------
    |
    | These configuration options specify the behavior of Laravel's password
    | reset functionality, including the table utilized for token storage
    | and the user provider that is invoked to actually retrieve users.
    |
    | The expiry time is the number of minutes that each reset token will be
    | considered valid. This security feature keeps tokens short-lived so
    | they have less time to be guessed. You may change this as needed.
    |
    | The throttle setting is the number of seconds a user must wait before
    | generating more password reset tokens. This prevents the user from
    | quickly generating a very large amount of password reset tokens.
    |
    */

    'passwords' => [
        'users' => [
            'provider' => 'users',
            'table' => env('AUTH_PASSWORD_RESET_TOKEN_TABLE', 'password_reset_tokens'),
```

```
            'expire' => 60,
            'throttle' => 60,
        ],
    ],

    /*
    |---------------------------------------------------------------------------
    | Password Confirmation Timeout
    |---------------------------------------------------------------------------
    |
    | Here you may define the amount of seconds before a password confirmation
    | window expires and users are asked to re-enter their password via the
    | confirmation screen. By default, the timeout lasts for three hours.
    |
    */

    'password_timeout' => env('AUTH_PASSWORD_TIMEOUT', 10800),

];
```

FILE: config/broadcasting.php

```
<?php

return [

    /*
    |---------------------------------------------------------------------------
    | Default Broadcaster
    |---------------------------------------------------------------------------
    |
    | This option controls the default broadcaster that will be used by the
    | framework when an event needs to be broadcast. You may set this to
    | any of the connections defined in the "connections" array below.
    |
    | Supported: "reverb", "pusher", "ably", "redis", "log", "null"
    |
    */

    'default' => env('BROADCAST_CONNECTION', 'null'),

    /*
    |---------------------------------------------------------------------------
    | Broadcast Connections
    |---------------------------------------------------------------------------
    |
    | Here you may define all of the broadcast connections that will be used
    | to broadcast events to other systems or over WebSockets. Samples of
    | each available type of connection are provided inside this array.
    |
    */

    'connections' => [

        'reverb' => [
            'driver' => 'reverb',
            'key' => env('REVERB_APP_KEY'),
            'secret' => env('REVERB_APP_SECRET'),
            'app_id' => env('REVERB_APP_ID'),
            'options' => [
                'host' => env('REVERB_HOST'),
                'port' => env('REVERB_PORT', 443),
                'scheme' => env('REVERB_SCHEME', 'https'),
                'useTLS' => env('REVERB_SCHEME', 'https') === 'https',
```

```
            ],
            'client_options' => [
                // Guzzle client options: https://docs.guzzlephp.org/en/stable/request-options.html
            ],
        ],

        'pusher' => [
            'driver' => 'pusher',
            'key' => env('PUSHER_APP_KEY'),
            'secret' => env('PUSHER_APP_SECRET'),
            'app_id' => env('PUSHER_APP_ID'),
            'options' => [
                'cluster' => env('PUSHER_APP_CLUSTER'),
                'host' => env('PUSHER_HOST') ?: 'api-'.env('PUSHER_APP_CLUSTER', 'mt1').'.pusher.com',
                'port' => env('PUSHER_PORT', 443),
                'scheme' => env('PUSHER_SCHEME', 'https'),
                'encrypted' => true,
                'useTLS' => env('PUSHER_SCHEME', 'https') === 'https',
            ],
            'client_options' => [
                // Guzzle client options: https://docs.guzzlephp.org/en/stable/request-options.html
            ],
        ],

        'ably' => [
            'driver' => 'ably',
            'key' => env('ABLY_KEY'),
        ],

        'log' => [
            'driver' => 'log',
        ],

        'null' => [
            'driver' => 'null',
        ],

    ],

];
```

FILE: config/cache.php

```
<?php

use Illuminate\Support\Str;

return [

    /*
    |--------------------------------------------------------------------------
    | Default Cache Store
    |--------------------------------------------------------------------------
    |
    | This option controls the default cache store that will be used by the
    | framework. This connection is utilized if another isn't explicitly
    | specified when running a cache operation inside the application.
    |
    */

    'default' => env('CACHE_STORE', 'database'),

    /*
    |--------------------------------------------------------------------------
```

```
    | Cache Stores
    |--------------------------------------------------------------------------
    |
    | Here you may define all of the cache "stores" for your application as
    | well as their drivers. You may even define multiple stores for the
    | same cache driver to group types of items stored in your caches.
    |
    | Supported drivers: "array", "database", "file", "memcached",
    |                    "redis", "dynamodb", "octane", "null"
    |
    */

    'stores' => [

        'array' => [
            'driver' => 'array',
            'serialize' => false,
        ],

        'database' => [
            'driver' => 'database',
            'connection' => env('DB_CACHE_CONNECTION'),
            'table' => env('DB_CACHE_TABLE', 'cache'),
            'lock_connection' => env('DB_CACHE_LOCK_CONNECTION'),
            'lock_table' => env('DB_CACHE_LOCK_TABLE'),
        ],

        'file' => [
            'driver' => 'file',
            'path' => storage_path('framework/cache/data'),
            'lock_path' => storage_path('framework/cache/data'),
        ],

        'memcached' => [
            'driver' => 'memcached',
            'persistent_id' => env('MEMCACHED_PERSISTENT_ID'),
            'sasl' => [
                env('MEMCACHED_USERNAME'),
                env('MEMCACHED_PASSWORD'),
            ],
            'options' => [
                // Memcached::OPT_CONNECT_TIMEOUT => 2000,
            ],
            'servers' => [
                [
                    'host' => env('MEMCACHED_HOST', '127.0.0.1'),
                    'port' => env('MEMCACHED_PORT', 11211),
                    'weight' => 100,
                ],
            ],
        ],

        'redis' => [
            'driver' => 'redis',
            'connection' => env('REDIS_CACHE_CONNECTION', 'cache'),
            'lock_connection' => env('REDIS_CACHE_LOCK_CONNECTION', 'default'),
        ],

        'dynamodb' => [
            'driver' => 'dynamodb',
            'key' => env('AWS_ACCESS_KEY_ID'),
            'secret' => env('AWS_SECRET_ACCESS_KEY'),
            'region' => env('AWS_DEFAULT_REGION', 'us-east-1'),
            'table' => env('DYNAMODB_CACHE_TABLE', 'cache'),
            'endpoint' => env('DYNAMODB_ENDPOINT'),
        ],
```

```
        'octane' => [
            'driver' => 'octane',
        ],

    ],

    /*
    |--------------------------------------------------------------------------
    | Cache Key Prefix
    |--------------------------------------------------------------------------
    |
    | When utilizing the APC, database, memcached, Redis, and DynamoDB cache
    | stores, there might be other applications using the same cache. For
    | that reason, you may prefix every cache key to avoid collisions.
    |
    */

    'prefix' => env('CACHE_PREFIX', Str::slug(env('APP_NAME', 'laravel'), '_').'_cache_'),

];
```

FILE: config/cors.php

```
<?php

return [

    /*
    |--------------------------------------------------------------------------
    | Cross-Origin Resource Sharing (CORS) Configuration
    |--------------------------------------------------------------------------
    |
    | Here you may configure your settings for cross-origin resource sharing
    | or "CORS". This determines what cross-origin operations may execute
    | in web browsers. You are free to adjust these settings as needed.
    |
    | To learn more: https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
    |
    */

    'paths' => ['*'],

    'allowed_methods' => ['*'],

    'allowed_origins' => [env('FRONTEND_URL', 'http://localhost:5173'),'http://localhost:5174',
'http://localhost:5175','http://localhost:5176','http://localhost:5177','http://localhost:5178' ],

    'allowed_origins_patterns' => [],

    'allowed_headers' => ['*'],

    'exposed_headers' => [],

    'max_age' => 0,

    'supports_credentials' => true,

];
```

FILE: config/database.php

```php
<?php

use Illuminate\Support\Str;

return [

    /*
    |--------------------------------------------------------------------------
    | Default Database Connection Name
    |--------------------------------------------------------------------------
    |
    | Here you may specify which of the database connections below you wish
    | to use as your default connection for database operations. This is
    | the connection which will be utilized unless another connection
    | is explicitly specified when you execute a query / statement.
    |
    */

    'default' => env('DB_CONNECTION', 'sqlite'),

    /*
    |--------------------------------------------------------------------------
    | Database Connections
    |--------------------------------------------------------------------------
    |
    | Below are all of the database connections defined for your application.
    | An example configuration is provided for each database system which
    | is supported by Laravel. You're free to add / remove connections.
    |
    */

    'connections' => [

        'sqlite' => [
            'driver' => 'sqlite',
            'url' => env('DB_URL'),
            'database' => env('DB_DATABASE', database_path('database.sqlite')),
            'prefix' => '',
            'foreign_key_constraints' => env('DB_FOREIGN_KEYS', true),
            'busy_timeout' => null,
            'journal_mode' => null,
            'synchronous' => null,
        ],

        'mysql' => [
            'driver' => 'mysql',
            'url' => env('DB_URL'),
            'host' => env('DB_HOST', '127.0.0.1'),
            'port' => env('DB_PORT', '3306'),
            'database' => env('DB_DATABASE', 'laravel'),
            'username' => env('DB_USERNAME', 'root'),
            'password' => env('DB_PASSWORD', ''),
            'unix_socket' => env('DB_SOCKET', ''),
            'charset' => env('DB_CHARSET', 'utf8mb4'),
            'collation' => env('DB_COLLATION', 'utf8mb4_unicode_ci'),
            'prefix' => '',
            'prefix_indexes' => true,
            'strict' => true,
            'engine' => null,
            'options' => extension_loaded('pdo_mysql') ? array_filter([
                PDO::MYSQL_ATTR_SSL_CA => env('MYSQL_ATTR_SSL_CA'),
            ]) : [],
        ],

        'mariadb' => [
```

```
            'driver' => 'mariadb',
            'url' => env('DB_URL'),
            'host' => env('DB_HOST', '127.0.0.1'),
            'port' => env('DB_PORT', '3306'),
            'database' => env('DB_DATABASE', 'laravel'),
            'username' => env('DB_USERNAME', 'root'),
            'password' => env('DB_PASSWORD', ''),
            'unix_socket' => env('DB_SOCKET', ''),
            'charset' => env('DB_CHARSET', 'utf8mb4'),
            'collation' => env('DB_COLLATION', 'utf8mb4_unicode_ci'),
            'prefix' => '',
            'prefix_indexes' => true,
            'strict' => true,
            'engine' => null,
            'options' => extension_loaded('pdo_mysql') ? array_filter([
                PDO::MYSQL_ATTR_SSL_CA => env('MYSQL_ATTR_SSL_CA'),
            ]) : [],
        ],

        'pgsql' => [
            'driver' => 'pgsql',
            'url' => env('DB_URL'),
            'host' => env('DB_HOST', '127.0.0.1'),
            'port' => env('DB_PORT', '5432'),
            'database' => env('DB_DATABASE', 'laravel'),
            'username' => env('DB_USERNAME', 'root'),
            'password' => env('DB_PASSWORD', ''),
            'charset' => env('DB_CHARSET', 'utf8'),
            'prefix' => '',
            'prefix_indexes' => true,
            'search_path' => 'public',
            'sslmode' => 'prefer',
        ],

        'sqlsrv' => [
            'driver' => 'sqlsrv',
            'url' => env('DB_URL'),
            'host' => env('DB_HOST', 'localhost'),
            'port' => env('DB_PORT', '1433'),
            'database' => env('DB_DATABASE', 'laravel'),
            'username' => env('DB_USERNAME', 'root'),
            'password' => env('DB_PASSWORD', ''),
            'charset' => env('DB_CHARSET', 'utf8'),
            'prefix' => '',
            'prefix_indexes' => true,
            // 'encrypt' => env('DB_ENCRYPT', 'yes'),
            // 'trust_server_certificate' => env('DB_TRUST_SERVER_CERTIFICATE', 'false'),
        ],

    ],

    /*
    |--------------------------------------------------------------------------
    | Migration Repository Table
    |--------------------------------------------------------------------------
    |
    | This table keeps track of all the migrations that have already run for
    | your application. Using this information, we can determine which of
    | the migrations on disk haven't actually been run on the database.
    |
    */

    'migrations' => [
        'table' => 'migrations',
        'update_date_on_publish' => true,
    ],
```

```
    /*
    |--------------------------------------------------------------------------
    | Redis Databases
    |--------------------------------------------------------------------------
    |
    | Redis is an open source, fast, and advanced key-value store that also
    | provides a richer body of commands than a typical key-value system
    | such as Memcached. You may define your connection settings here.
    |
    */

    'redis' => [

        'client' => env('REDIS_CLIENT', 'phpredis'),

        'options' => [
            'cluster' => env('REDIS_CLUSTER', 'redis'),
            'prefix' => env('REDIS_PREFIX', Str::slug(env('APP_NAME', 'laravel'), '_').'_database_'),
        ],

        'default' => [
            'url' => env('REDIS_URL'),
            'host' => env('REDIS_HOST', '127.0.0.1'),
            'username' => env('REDIS_USERNAME'),
            'password' => env('REDIS_PASSWORD'),
            'port' => env('REDIS_PORT', '6379'),
            'database' => env('REDIS_DB', '0'),
        ],

        'cache' => [
            'url' => env('REDIS_URL'),
            'host' => env('REDIS_HOST', '127.0.0.1'),
            'username' => env('REDIS_USERNAME'),
            'password' => env('REDIS_PASSWORD'),
            'port' => env('REDIS_PORT', '6379'),
            'database' => env('REDIS_CACHE_DB', '1'),
        ],

    ],

];
```

FILE: config/filesystems.php

```
<?php

return [

    /*
    |--------------------------------------------------------------------------
    | Default Filesystem Disk
    |--------------------------------------------------------------------------
    |
    | Here you may specify the default filesystem disk that should be used
    | by the framework. The "local" disk, as well as a variety of cloud
    | based disks are available to your application for file storage.
    |
    */

    'default' => env('FILESYSTEM_DISK', 'local'),

    /*
    |--------------------------------------------------------------------------
```

```
    | Filesystem Disks
    |--------------------------------------------------------------------------
    |
    | Below you may configure as many filesystem disks as necessary, and you
    | may even configure multiple disks for the same driver. Examples for
    | most supported storage drivers are configured here for reference.
    |
    | Supported drivers: "local", "ftp", "sftp", "s3"
    |
    */

    'disks' => [

        'local' => [
            'driver' => 'local',
            'root' => storage_path('app/private'),
            'serve' => true,
            'throw' => false,
        ],

        'public' => [
            'driver' => 'local',
            'root' => storage_path('app/public'),
            'url' => env('APP_URL').'/storage',
            'visibility' => 'public',
            'throw' => false,
        ],

        's3' => [
            'driver' => 's3',
            'key' => env('AWS_ACCESS_KEY_ID'),
            'secret' => env('AWS_SECRET_ACCESS_KEY'),
            'region' => env('AWS_DEFAULT_REGION'),
            'bucket' => env('AWS_BUCKET'),
            'url' => env('AWS_URL'),
            'endpoint' => env('AWS_ENDPOINT'),
            'use_path_style_endpoint' => env('AWS_USE_PATH_STYLE_ENDPOINT', false),
            'throw' => false,
        ],

    ],

    /*
    |--------------------------------------------------------------------------
    | Symbolic Links
    |--------------------------------------------------------------------------
    |
    | Here you may configure the symbolic links that will be created when the
    | `storage:link` Artisan command is executed. The array keys should be
    | the locations of the links and the values should be their targets.
    |
    */

    'links' => [
        public_path('storage') => storage_path('app/public'),
    ],

];
```

FILE: config/horizon.php

```
<?php

use Illuminate\Support\Str;
```

```
return [

    /*
    |--------------------------------------------------------------------------
    | Horizon Domain
    |--------------------------------------------------------------------------
    |
    | This is the subdomain where Horizon will be accessible from. If this
    | setting is null, Horizon will reside under the same domain as the
    | application. Otherwise, this value will serve as the subdomain.
    |
    */

    'domain' => env('HORIZON_DOMAIN'),

    /*
    |--------------------------------------------------------------------------
    | Horizon Path
    |--------------------------------------------------------------------------
    |
    | This is the URI path where Horizon will be accessible from. Feel free
    | to change this path to anything you like. Note that the URI will not
    | affect the paths of its internal API that aren't exposed to users.
    |
    */

    'path' => env('HORIZON_PATH', 'horizon'),

    /*
    |--------------------------------------------------------------------------
    | Horizon Redis Connection
    |--------------------------------------------------------------------------
    |
    | This is the name of the Redis connection where Horizon will store the
    | meta information required for it to function. It includes the list
    | of supervisors, failed jobs, job metrics, and other information.
    |
    */

    'use' => 'default',

    /*
    |--------------------------------------------------------------------------
    | Horizon Redis Prefix
    |--------------------------------------------------------------------------
    |
    | This prefix will be used when storing all Horizon data in Redis. You
    | may modify the prefix when you are running multiple installations
    | of Horizon on the same server so that they don't have problems.
    |
    */

    'prefix' => env(
        'HORIZON_PREFIX',
        Str::slug(env('APP_NAME', 'laravel'), '_').'_horizon:'
    ),

    /*
    |--------------------------------------------------------------------------
    | Horizon Route Middleware
    |--------------------------------------------------------------------------
    |
    | These middleware will get attached onto each Horizon route, giving you
    | the chance to add your own middleware to this list or change any of
    | the existing middleware. Or, you can simply stick with this list.
```

```
    |
    */

    'middleware' => ['web'],

    /*
    |--------------------------------------------------------------------------
    | Queue Wait Time Thresholds
    |--------------------------------------------------------------------------
    |
    | This option allows you to configure when the LongWaitDetected event
    | will be fired. Every connection / queue combination may have its
    | own, unique threshold (in seconds) before this event is fired.
    |
    */

    'waits' => [
        'redis:default' => 60,
    ],

    /*
    |--------------------------------------------------------------------------
    | Job Trimming Times
    |--------------------------------------------------------------------------
    |
    | Here you can configure for how long (in minutes) you desire Horizon to
    | persist the recent and failed jobs. Typically, recent jobs are kept
    | for one hour while all failed jobs are stored for an entire week.
    |
    */

    'trim' => [
        'recent' => 60,
        'pending' => 60,
        'completed' => 60,
        'recent_failed' => 10080,
        'failed' => 10080,
        'monitored' => 10080,
    ],

    /*
    |--------------------------------------------------------------------------
    | Silenced Jobs
    |--------------------------------------------------------------------------
    |
    | Silencing a job will instruct Horizon to not place the job in the list
    | of completed jobs within the Horizon dashboard. This setting may be
    | used to fully remove any noisy jobs from the completed jobs list.
    |
    */

    'silenced' => [
        // App\Jobs\ExampleJob::class,
    ],

    /*
    |--------------------------------------------------------------------------
    | Metrics
    |--------------------------------------------------------------------------
    |
    | Here you can configure how many snapshots should be kept to display in
    | the metrics graph. This will get used in combination with Horizon's
    | `horizon:snapshot` schedule to define how long to retain metrics.
    |
    */
```

```
    'metrics' => [
        'trim_snapshots' => [
            'job' => 24,
            'queue' => 24,
        ],
    ],

    /*
    |--------------------------------------------------------------------------
    | Fast Termination
    |--------------------------------------------------------------------------
    |
    | When this option is enabled, Horizon's "terminate" command will not
    | wait on all of the workers to terminate unless the --wait option
    | is provided. Fast termination can shorten deployment delay by
    | allowing a new instance of Horizon to start while the last
    | instance will continue to terminate each of its workers.
    |
    */

    'fast_termination' => false,

    /*
    |--------------------------------------------------------------------------
    | Memory Limit (MB)
    |--------------------------------------------------------------------------
    |
    | This value describes the maximum amount of memory the Horizon master
    | supervisor may consume before it is terminated and restarted. For
    | configuring these limits on your workers, see the next section.
    |
    */

    'memory_limit' => 64,

    /*
    |--------------------------------------------------------------------------
    | Queue Worker Configuration
    |--------------------------------------------------------------------------
    |
    | Here you may define the queue worker settings used by your application
    | in all environments. These supervisors and settings handle all your
    | queued jobs and will be provisioned by Horizon during deployment.
    |
    */

    'defaults' => [
        'supervisor-1' => [
            'connection' => 'redis',
            'queue' => ['default'],
            'balance' => 'auto',
            'autoScalingStrategy' => 'time',
            'maxProcesses' => 1,
            'maxTime' => 0,
            'maxJobs' => 0,
            'memory' => 128,
            'tries' => 1,
            'timeout' => 60,
            'nice' => 0,
        ],
    ],

    'environments' => [
        'production' => [
            'supervisor-1' => [
                'maxProcesses' => 10,
```

```
                'balanceMaxShift' => 1,
                'balanceCooldown' => 3,
            ],
        ],

        'local' => [
            'supervisor-1' => [
                'maxProcesses' => 3,
            ],
        ],
    ],
];
```

FILE: config/jwt.php

```
<?php

/*
 * This file is part of jwt-auth.
 *
 * (c) Sean Tymon <tymon148@gmail.com>
 *
 * For the full copyright and license information, please view the LICENSE
 * file that was distributed with this source code.
 */

return [

    /*
    |--------------------------------------------------------------------------
    | JWT Authentication Secret
    |--------------------------------------------------------------------------
    |
    | Don't forget to set this in your .env file, as it will be used to sign
    | your tokens. A helper command is provided for this:
    | `php artisan jwt:secret`
    |
    | Note: This will be used for Symmetric algorithms only (HMAC),
    | since RSA and ECDSA use a private/public key combo (See below).
    |
    */

    'secret' => env('JWT_SECRET'),

    /*
    |--------------------------------------------------------------------------
    | JWT Authentication Keys
    |--------------------------------------------------------------------------
    |
    | The algorithm you are using, will determine whether your tokens are
    | signed with a random string (defined in `JWT_SECRET`) or using the
    | following public & private keys.
    |
    | Symmetric Algorithms:
    | HS256, HS384 & HS512 will use `JWT_SECRET`.
    |
    | Asymmetric Algorithms:
    | RS256, RS384 & RS512 / ES256, ES384 & ES512 will use the keys below.
    |
    */

    'keys' => [

        /*
```

```
        |--------------------------------------------------------------------------
        | Public Key
        |--------------------------------------------------------------------------
        |
        | A path or resource to your public key.
        |
        | E.g. 'file://path/to/public/key'
        |
        */

        'public' => env('JWT_PUBLIC_KEY'),

        /*
        |--------------------------------------------------------------------------
        | Private Key
        |--------------------------------------------------------------------------
        |
        | A path or resource to your private key.
        |
        | E.g. 'file://path/to/private/key'
        |
        */

        'private' => env('JWT_PRIVATE_KEY'),

        /*
        |--------------------------------------------------------------------------
        | Passphrase
        |--------------------------------------------------------------------------
        |
        | The passphrase for your private key. Can be null if none set.
        |
        */

        'passphrase' => env('JWT_PASSPHRASE'),

    ],

    /*
    |--------------------------------------------------------------------------
    | JWT time to live
    |--------------------------------------------------------------------------
    |
    | Specify the length of time (in minutes) that the token will be valid for.
    | Defaults to 1 hour.
    |
    | You can also set this to null, to yield a never expiring token.
    | Some people may want this behaviour for e.g. a mobile app.
    | This is not particularly recommended, so make sure you have appropriate
    | systems in place to revoke the token if necessary.
    | Notice: If you set this to null you should remove 'exp' element from 'required_claims' list.
    |
    */

    'ttl' => env('JWT_TTL', 60),

    /*
    |--------------------------------------------------------------------------
    | Refresh time to live
    |--------------------------------------------------------------------------
    |
    | Specify the length of time (in minutes) that the token can be refreshed
    | within. I.E. The user can refresh their token within a 2 week window of
    | the original token being created until they must re-authenticate.
    | Defaults to 2 weeks.
    |
```

```
    | You can also set this to null, to yield an infinite refresh time.
    | Some may want this instead of never expiring tokens for e.g. a mobile app.
    | This is not particularly recommended, so make sure you have appropriate
    | systems in place to revoke the token if necessary.
    |
    */

    'refresh_ttl' => env('JWT_REFRESH_TTL', 20160),

    /*
    |--------------------------------------------------------------------------
    | JWT hashing algorithm
    |--------------------------------------------------------------------------
    |
    | Specify the hashing algorithm that will be used to sign the token.
    |
    */

    'algo' => env('JWT_ALGO', Tymon\JWTAuth\Providers\JWT\Provider::ALGO_HS256),

    /*
    |--------------------------------------------------------------------------
    | Required Claims
    |--------------------------------------------------------------------------
    |
    | Specify the required claims that must exist in any token.
    | A TokenInvalidException will be thrown if any of these claims are not
    | present in the payload.
    |
    */

    'required_claims' => [
        'iss',
        'iat',
        'exp',
        'nbf',
        'sub',
        'jti',
    ],

    /*
    |--------------------------------------------------------------------------
    | Persistent Claims
    |--------------------------------------------------------------------------
    |
    | Specify the claim keys to be persisted when refreshing a token.
    | `sub` and `iat` will automatically be persisted, in
    | addition to the these claims.
    |
    | Note: If a claim does not exist then it will be ignored.
    |
    */

    'persistent_claims' => [
        // 'foo',
        // 'bar',
    ],

    /*
    |--------------------------------------------------------------------------
    | Lock Subject
    |--------------------------------------------------------------------------
    |
    | This will determine whether a `prv` claim is automatically added to
    | the token. The purpose of this is to ensure that if you have multiple
    | authentication models e.g. `App\User` & `App\OtherPerson`, then we
```

```
    | should prevent one authentication request from impersonating another,
    | if 2 tokens happen to have the same id across the 2 different models.
    |
    | Under specific circumstances, you may want to disable this behaviour
    | e.g. if you only have one authentication model, then you would save
    | a little on token size.
    |
    */

    'lock_subject' => true,

    /*
    |--------------------------------------------------------------------------
    | Leeway
    |--------------------------------------------------------------------------
    |
    | This property gives the jwt timestamp claims some "leeway".
    | Meaning that if you have any unavoidable slight clock skew on
    | any of your servers then this will afford you some level of cushioning.
    |
    | This applies to the claims `iat`, `nbf` and `exp`.
    |
    | Specify in seconds - only if you know you need it.
    |
    */

    'leeway' => env('JWT_LEEWAY', 0),

    /*
    |--------------------------------------------------------------------------
    | Blacklist Enabled
    |--------------------------------------------------------------------------
    |
    | In order to invalidate tokens, you must have the blacklist enabled.
    | If you do not want or need this functionality, then set this to false.
    |
    */

    'blacklist_enabled' => env('JWT_BLACKLIST_ENABLED', true),

    /*
    | --------------------------------------------------------------------------
    | Blacklist Grace Period
    | --------------------------------------------------------------------------
    |
    | When multiple concurrent requests are made with the same JWT,
    | it is possible that some of them fail, due to token regeneration
    | on every request.
    |
    | Set grace period in seconds to prevent parallel request failure.
    |
    */

    'blacklist_grace_period' => env('JWT_BLACKLIST_GRACE_PERIOD', 0),

    /*
    |--------------------------------------------------------------------------
    | Cookies encryption
    |--------------------------------------------------------------------------
    |
    | By default Laravel encrypt cookies for security reason.
    | If you decide to not decrypt cookies, you will have to configure Laravel
    | to not encrypt your cookie token by adding its name into the $except
    | array available in the middleware "EncryptCookies" provided by Laravel.
    | see https://laravel.com/docs/master/responses#cookies-and-encryption
    | for details.
```

```
    |
    | Set it to true if you want to decrypt cookies.
    |
    */

    'decrypt_cookies' => false,

    /*
    |--------------------------------------------------------------------------
    | Providers
    |--------------------------------------------------------------------------
    |
    | Specify the various providers used throughout the package.
    |
    */

    'providers' => [

        /*
        |-----------------------------------------------------------------------
        | JWT Provider
        |-----------------------------------------------------------------------
        |
        | Specify the provider that is used to create and decode the tokens.
        |
        */

        'jwt' => Tymon\JWTAuth\Providers\JWT\Lcobucci::class,

        /*
        |-----------------------------------------------------------------------
        | Authentication Provider
        |-----------------------------------------------------------------------
        |
        | Specify the provider that is used to authenticate users.
        |
        */

        'auth' => Tymon\JWTAuth\Providers\Auth\Illuminate::class,

        /*
        |-----------------------------------------------------------------------
        | Storage Provider
        |-----------------------------------------------------------------------
        |
        | Specify the provider that is used to store tokens in the blacklist.
        |
        */

        'storage' => Tymon\JWTAuth\Providers\Storage\Illuminate::class,

    ],

];
```

FILE: config/logging.php

```
<?php

use Monolog\Handler\NullHandler;
use Monolog\Handler\StreamHandler;
use Monolog\Handler\SyslogUdpHandler;
use Monolog\Processor\PsrLogMessageProcessor;
```

```
return [

    /*
    |--------------------------------------------------------------------------
    | Default Log Channel
    |--------------------------------------------------------------------------
    |
    | This option defines the default log channel that is utilized to write
    | messages to your logs. The value provided here should match one of
    | the channels present in the list of "channels" configured below.
    |
    */

    'default' => env('LOG_CHANNEL', 'stack'),

    /*
    |--------------------------------------------------------------------------
    | Deprecations Log Channel
    |--------------------------------------------------------------------------
    |
    | This option controls the log channel that should be used to log warnings
    | regarding deprecated PHP and library features. This allows you to get
    | your application ready for upcoming major versions of dependencies.
    |
    */

    'deprecations' => [
        'channel' => env('LOG_DEPRECATIONS_CHANNEL', 'null'),
        'trace' => env('LOG_DEPRECATIONS_TRACE', false),
    ],

    /*
    |--------------------------------------------------------------------------
    | Log Channels
    |--------------------------------------------------------------------------
    |
    | Here you may configure the log channels for your application. Laravel
    | utilizes the Monolog PHP logging library, which includes a variety
    | of powerful log handlers and formatters that you're free to use.
    |
    | Available drivers: "single", "daily", "slack", "syslog",
    |                    "errorlog", "monolog", "custom", "stack"
    |
    */

    'channels' => [

        'stack' => [
            'driver' => 'stack',
            'channels' => explode(',', env('LOG_STACK', 'single')),
            'ignore_exceptions' => false,
        ],

        'single' => [
            'driver' => 'single',
            'path' => storage_path('logs/laravel.log'),
            'level' => env('LOG_LEVEL', 'debug'),
            'replace_placeholders' => true,
        ],

        'daily' => [
            'driver' => 'daily',
            'path' => storage_path('logs/laravel.log'),
            'level' => env('LOG_LEVEL', 'debug'),
            'days' => env('LOG_DAILY_DAYS', 14),
            'replace_placeholders' => true,
```

```
        ],

        'slack' => [
            'driver' => 'slack',
            'url' => env('LOG_SLACK_WEBHOOK_URL'),
            'username' => env('LOG_SLACK_USERNAME', 'Laravel Log'),
            'emoji' => env('LOG_SLACK_EMOJI', ':boom:'),
            'level' => env('LOG_LEVEL', 'critical'),
            'replace_placeholders' => true,
        ],

        'papertrail' => [
            'driver' => 'monolog',
            'level' => env('LOG_LEVEL', 'debug'),
            'handler' => env('LOG_PAPERTRAIL_HANDLER', SyslogUdpHandler::class),
            'handler_with' => [
                'host' => env('PAPERTRAIL_URL'),
                'port' => env('PAPERTRAIL_PORT'),
                'connectionString' => 'tls://'.env('PAPERTRAIL_URL').':'.env('PAPERTRAIL_PORT'),
            ],
            'processors' => [PsrLogMessageProcessor::class],
        ],

        'stderr' => [
            'driver' => 'monolog',
            'level' => env('LOG_LEVEL', 'debug'),
            'handler' => StreamHandler::class,
            'formatter' => env('LOG_STDERR_FORMATTER'),
            'with' => [
                'stream' => 'php://stderr',
            ],
            'processors' => [PsrLogMessageProcessor::class],
        ],

        'syslog' => [
            'driver' => 'syslog',
            'level' => env('LOG_LEVEL', 'debug'),
            'facility' => env('LOG_SYSLOG_FACILITY', LOG_USER),
            'replace_placeholders' => true,
        ],

        'errorlog' => [
            'driver' => 'errorlog',
            'level' => env('LOG_LEVEL', 'debug'),
            'replace_placeholders' => true,
        ],

        'null' => [
            'driver' => 'monolog',
            'handler' => NullHandler::class,
        ],

        'emergency' => [
            'path' => storage_path('logs/laravel.log'),
        ],

    ],

];
```

FILE: config/mail.php

```
<?php
```

```
return [

    /*
    |--------------------------------------------------------------------------
    | Default Mailer
    |--------------------------------------------------------------------------
    |
    | This option controls the default mailer that is used to send all email
    | messages unless another mailer is explicitly specified when sending
    | the message. All additional mailers can be configured within the
    | "mailers" array. Examples of each type of mailer are provided.
    |
    */

    'default' => env('MAIL_MAILER', 'log'),

    /*
    |--------------------------------------------------------------------------
    | Mailer Configurations
    |--------------------------------------------------------------------------
    |
    | Here you may configure all of the mailers used by your application plus
    | their respective settings. Several examples have been configured for
    | you and you are free to add your own as your application requires.
    |
    | Laravel supports a variety of mail "transport" drivers that can be used
    | when delivering an email. You may specify which one you're using for
    | your mailers below. You may also add additional mailers if needed.
    |
    | Supported: "smtp", "sendmail", "mailgun", "ses", "ses-v2",
    |            "postmark", "resend", "log", "array",
    |            "failover", "roundrobin"
    |
    */

    'mailers' => [

        'smtp' => [
            'transport' => 'smtp',
            'scheme' => env('MAIL_SCHEME'),
            'url' => env('MAIL_URL'),
            'host' => env('MAIL_HOST', '127.0.0.1'),
            'port' => env('MAIL_PORT', 2525),
            'username' => env('MAIL_USERNAME'),
            'password' => env('MAIL_PASSWORD'),
            'timeout' => null,
            'local_domain' => env('MAIL_EHLO_DOMAIN', parse_url(env('APP_URL', 'http://localhost'), PHP_URL_HOST)),
        ],

        'ses' => [
            'transport' => 'ses',
        ],

        'postmark' => [
            'transport' => 'postmark',
            // 'message_stream_id' => env('POSTMARK_MESSAGE_STREAM_ID'),
            // 'client' => [
            //     'timeout' => 5,
            // ],
        ],

        'resend' => [
            'transport' => 'resend',
        ],

        'sendmail' => [
```

```
            'transport' => 'sendmail',
            'path' => env('MAIL_SENDMAIL_PATH', '/usr/sbin/sendmail -bs -i'),
        ],

        'log' => [
            'transport' => 'log',
            'channel' => env('MAIL_LOG_CHANNEL'),
        ],

        'array' => [
            'transport' => 'array',
        ],

        'failover' => [
            'transport' => 'failover',
            'mailers' => [
                'smtp',
                'log',
            ],
        ],

        'roundrobin' => [
            'transport' => 'roundrobin',
            'mailers' => [
                'ses',
                'postmark',
            ],
        ],

    ],

    /*
    |--------------------------------------------------------------------------
    | Global "From" Address
    |--------------------------------------------------------------------------
    |
    | You may wish for all emails sent by your application to be sent from
    | the same address. Here you may specify a name and address that is
    | used globally for all emails that are sent by your application.
    |
    */

    'from' => [
        'address' => env('MAIL_FROM_ADDRESS', 'hello@example.com'),
        'name' => env('MAIL_FROM_NAME', 'Example'),
    ],

];
```

FILE: config/queue.php

```
<?php

return [

    /*
    |--------------------------------------------------------------------------
    | Default Queue Connection Name
    |--------------------------------------------------------------------------
    |
    | Laravel's queue supports a variety of backends via a single, unified
    | API, giving you convenient access to each backend using identical
    | syntax for each. The default queue connection is defined below.
    |
```

```
    */

    'default' => env('QUEUE_CONNECTION', 'database'),

    /*
    |--------------------------------------------------------------------------
    | Queue Connections
    |--------------------------------------------------------------------------
    |
    | Here you may configure the connection options for every queue backend
    | used by your application. An example configuration is provided for
    | each backend supported by Laravel. You're also free to add more.
    |
    | Drivers: "sync", "database", "beanstalkd", "sqs", "redis", "null"
    |
    */

    'connections' => [

        'sync' => [
            'driver' => 'sync',
        ],

        'database' => [
            'driver' => 'database',
            'connection' => env('DB_QUEUE_CONNECTION'),
            'table' => env('DB_QUEUE_TABLE', 'jobs'),
            'queue' => env('DB_QUEUE', 'default'),
            'retry_after' => (int) env('DB_QUEUE_RETRY_AFTER', 90),
            'after_commit' => false,
        ],

        'beanstalkd' => [
            'driver' => 'beanstalkd',
            'host' => env('BEANSTALKD_QUEUE_HOST', 'localhost'),
            'queue' => env('BEANSTALKD_QUEUE', 'default'),
            'retry_after' => (int) env('BEANSTALKD_QUEUE_RETRY_AFTER', 90),
            'block_for' => 0,
            'after_commit' => false,
        ],

        'sqs' => [
            'driver' => 'sqs',
            'key' => env('AWS_ACCESS_KEY_ID'),
            'secret' => env('AWS_SECRET_ACCESS_KEY'),
            'prefix' => env('SQS_PREFIX', 'https://sqs.us-east-1.amazonaws.com/your-account-id'),
            'queue' => env('SQS_QUEUE', 'default'),
            'suffix' => env('SQS_SUFFIX'),
            'region' => env('AWS_DEFAULT_REGION', 'us-east-1'),
            'after_commit' => false,
        ],

        'redis' => [
            'driver' => 'redis',
            'connection' => env('REDIS_QUEUE_CONNECTION', 'default'),
            'queue' => env('REDIS_QUEUE', 'default'),
            'retry_after' => (int) env('REDIS_QUEUE_RETRY_AFTER', 90),
            'block_for' => null,
            'after_commit' => false,
        ],

    ],

    /*
    |--------------------------------------------------------------------------
    | Job Batching
```

```
    |--------------------------------------------------------------------------
    |
    | The following options configure the database and table that store job
    | batching information. These options can be updated to any database
    | connection and table which has been defined by your application.
    |
    */

    'batching' => [
        'database' => env('DB_CONNECTION', 'sqlite'),
        'table' => 'job_batches',
    ],

    /*
    |--------------------------------------------------------------------------
    | Failed Queue Jobs
    |--------------------------------------------------------------------------
    |
    | These options configure the behavior of failed queue job logging so you
    | can control how and where failed jobs are stored. Laravel ships with
    | support for storing failed jobs in a simple file or in a database.
    |
    | Supported drivers: "database-uuids", "dynamodb", "file", "null"
    |
    */

    'failed' => [
        'driver' => env('QUEUE_FAILED_DRIVER', 'database-uuids'),
        'database' => env('DB_CONNECTION', 'sqlite'),
        'table' => 'failed_jobs',
    ],

];
```

FILE: config/reverb.php

```
<?php

return [

    /*
    |--------------------------------------------------------------------------
    | Default Reverb Server
    |--------------------------------------------------------------------------
    |
    | This option controls the default server used by Reverb to handle
    | incoming messages as well as broadcasting message to all your
    | connected clients. At this time only "reverb" is supported.
    |
    */

    'default' => env('REVERB_SERVER', 'reverb'),

    /*
    |--------------------------------------------------------------------------
    | Reverb Servers
    |--------------------------------------------------------------------------
    |
    | Here you may define details for each of the supported Reverb servers.
    | Each server has its own configuration options that are defined in
    | the array below. You should ensure all the options are present.
    |
    */
```

```
    'servers' => [

        'reverb' => [
            'host' => env('REVERB_SERVER_HOST', '0.0.0.0'),
            'port' => env('REVERB_SERVER_PORT', 8080),
            'hostname' => env('REVERB_HOST'),
            'options' => [
                'tls' => [],
            ],
            'max_request_size' => env('REVERB_MAX_REQUEST_SIZE', 10_000),
            'scaling' => [
                'enabled' => env('REVERB_SCALING_ENABLED', false),
                'channel' => env('REVERB_SCALING_CHANNEL', 'reverb'),
                'server' => [
                    'url' => env('REDIS_URL'),
                    'host' => env('REDIS_HOST', '127.0.0.1'),
                    'port' => env('REDIS_PORT', '6379'),
                    'username' => env('REDIS_USERNAME'),
                    'password' => env('REDIS_PASSWORD'),
                    'database' => env('REDIS_DB', '0'),
                ],
            ],
            'pulse_ingest_interval' => env('REVERB_PULSE_INGEST_INTERVAL', 15),
            'telescope_ingest_interval' => env('REVERB_TELESCOPE_INGEST_INTERVAL', 15),
        ],

    ],

    /*
    |--------------------------------------------------------------------------
    | Reverb Applications
    |--------------------------------------------------------------------------
    |
    | Here you may define how Reverb applications are managed. If you choose
    | to use the "config" provider, you may define an array of apps which
    | your server will support, including their connection credentials.
    |
    */

    'apps' => [

        'provider' => 'config',

        'apps' => [
            [
                'key' => env('REVERB_APP_KEY'),
                'secret' => env('REVERB_APP_SECRET'),
                'app_id' => env('REVERB_APP_ID'),
                'options' => [
                    'host' => env('REVERB_HOST'),
                    'port' => env('REVERB_PORT', 443),
                    'scheme' => env('REVERB_SCHEME', 'https'),
                    'useTLS' => env('REVERB_SCHEME', 'https') === 'https',
                ],
                'allowed_origins' => ['*'],
                'ping_interval' => env('REVERB_APP_PING_INTERVAL', 60),
                'activity_timeout' => env('REVERB_APP_ACTIVITY_TIMEOUT', 30),
                'max_message_size' => env('REVERB_APP_MAX_MESSAGE_SIZE', 10_000),
            ],
        ],

    ],

];
```

FILE: config/sanctum.php

```
<?php

use Laravel\Sanctum\Sanctum;

return [

    /*
    |---------------------------------------------------------------------------
    | Stateful Domains
    |---------------------------------------------------------------------------
    |
    | Requests from the following domains / hosts will receive stateful API
    | authentication cookies. Typically, these should include your local
    | and production domains which access your API via a frontend SPA.
    |
    */

    'stateful' => explode(',', env('SANCTUM_STATEFUL_DOMAINS', sprintf(
        '%s%s%s',
        'localhost,localhost:3000,127.0.0.1,127.0.0.1:3000,127.0.0.1:8000,::1',
        Sanctum::currentApplicationUrlWithPort(),
        env('FRONTEND_URL') ? ','.parse_url(env('FRONTEND_URL'), PHP_URL_HOST) : ''
    ))),

    /*
    |---------------------------------------------------------------------------
    | Sanctum Guards
    |---------------------------------------------------------------------------
    |
    | This array contains the authentication guards that will be checked when
    | Sanctum is trying to authenticate a request. If none of these guards
    | are able to authenticate the request, Sanctum will use the bearer
    | token that's present on an incoming request for authentication.
    |
    */

    'guard' => ['web'],

    /*
    |---------------------------------------------------------------------------
    | Expiration Minutes
    |---------------------------------------------------------------------------
    |
    | This value controls the number of minutes until an issued token will be
    | considered expired. This will override any values set in the token's
    | "expires_at" attribute, but first-party sessions are not affected.
    |
    */

    'expiration' => null,

    /*
    |---------------------------------------------------------------------------
    | Token Prefix
    |---------------------------------------------------------------------------
    |
    | Sanctum can prefix new tokens in order to take advantage of numerous
    | security scanning initiatives maintained by open source platforms
    | that notify developers if they commit tokens into repositories.
    |
    | See: https://docs.github.com/en/code-security/secret-scanning/about-secret-scanning
    |
    */
```

```
    'token_prefix' => env('SANCTUM_TOKEN_PREFIX', ''),

    /*
    |---------------------------------------------------------------------------
    | Sanctum Middleware
    |---------------------------------------------------------------------------
    |
    | When authenticating your first-party SPA with Sanctum you may need to
    | customize some of the middleware Sanctum uses while processing the
    | request. You may change the middleware listed below as required.
    |
    */

    'middleware' => [
        'authenticate_session' => Laravel\Sanctum\Http\Middleware\AuthenticateSession::class,
        'encrypt_cookies' => Illuminate\Cookie\Middleware\EncryptCookies::class,
        'validate_csrf_token' => Illuminate\Foundation\Http\Middleware\ValidateCsrfToken::class,
    ],

];
```

FILE: config/services.php

```
<?php

return [

    /*
    |---------------------------------------------------------------------------
    | Third Party Services
    |---------------------------------------------------------------------------
    |
    | This file is for storing the credentials for third party services such
    | as Mailgun, Postmark, AWS and more. This file provides the de facto
    | location for this type of information, allowing packages to have
    | a conventional file to locate the various service credentials.
    |
    */

    'postmark' => [
        'token' => env('POSTMARK_TOKEN'),
    ],

    'ses' => [
        'key' => env('AWS_ACCESS_KEY_ID'),
        'secret' => env('AWS_SECRET_ACCESS_KEY'),
        'region' => env('AWS_DEFAULT_REGION', 'us-east-1'),
    ],

    'resend' => [
        'key' => env('RESEND_KEY'),
    ],

    'slack' => [
        'notifications' => [
            'bot_user_oauth_token' => env('SLACK_BOT_USER_OAUTH_TOKEN'),
            'channel' => env('SLACK_BOT_USER_DEFAULT_CHANNEL'),
        ],
    ],
    'stripe' => [
    'secret' => env('STRIPE_SECRET_KEY'),
    ],
    'cloudinary' => [
```

```
        'cloud_name' => env('CLOUDINARY_CLOUD_NAME'),
        'api_key'    => env('CLOUDINARY_API_KEY'),
        'api_secret' => env('CLOUDINARY_API_SECRET'),
    ],




];
```

FILE: config/session.php

```
<?php

use Illuminate\Support\Str;

return [

    /*
    |--------------------------------------------------------------------------
    | Default Session Driver
    |--------------------------------------------------------------------------
    |
    | This option determines the default session driver that is utilized for
    | incoming requests. Laravel supports a variety of storage options to
    | persist session data. Database storage is a great default choice.
    |
    | Supported: "file", "cookie", "database", "apc",
    |            "memcached", "redis", "dynamodb", "array"
    |
    */

    'driver' => env('SESSION_DRIVER', 'database'),

    /*
    |--------------------------------------------------------------------------
    | Session Lifetime
    |--------------------------------------------------------------------------
    |
    | Here you may specify the number of minutes that you wish the session
    | to be allowed to remain idle before it expires. If you want them
    | to expire immediately when the browser is closed then you may
    | indicate that via the expire_on_close configuration option.
    |
    */

    'lifetime' => env('SESSION_LIFETIME', 120),

    'expire_on_close' => env('SESSION_EXPIRE_ON_CLOSE', false),

    /*
    |--------------------------------------------------------------------------
    | Session Encryption
    |--------------------------------------------------------------------------
    |
    | This option allows you to easily specify that all of your session data
    | should be encrypted before it's stored. All encryption is performed
    | automatically by Laravel and you may use the session like normal.
    |
    */

    'encrypt' => env('SESSION_ENCRYPT', false),

    /*
```

```
    |--------------------------------------------------------------------------
    | Session File Location
    |--------------------------------------------------------------------------
    |
    | When utilizing the "file" session driver, the session files are placed
    | on disk. The default storage location is defined here; however, you
    | are free to provide another location where they should be stored.
    |
    */

    'files' => storage_path('framework/sessions'),

    /*
    |--------------------------------------------------------------------------
    | Session Database Connection
    |--------------------------------------------------------------------------
    |
    | When using the "database" or "redis" session drivers, you may specify a
    | connection that should be used to manage these sessions. This should
    | correspond to a connection in your database configuration options.
    |
    */

    'connection' => env('SESSION_CONNECTION'),

    /*
    |--------------------------------------------------------------------------
    | Session Database Table
    |--------------------------------------------------------------------------
    |
    | When using the "database" session driver, you may specify the table to
    | be used to store sessions. Of course, a sensible default is defined
    | for you; however, you're welcome to change this to another table.
    |
    */

    'table' => env('SESSION_TABLE', 'sessions'),

    /*
    |--------------------------------------------------------------------------
    | Session Cache Store
    |--------------------------------------------------------------------------
    |
    | When using one of the framework's cache driven session backends, you may
    | define the cache store which should be used to store the session data
    | between requests. This must match one of your defined cache stores.
    |
    | Affects: "apc", "dynamodb", "memcached", "redis"
    |
    */

    'store' => env('SESSION_STORE'),

    /*
    |--------------------------------------------------------------------------
    | Session Sweeping Lottery
    |--------------------------------------------------------------------------
    |
    | Some session drivers must manually sweep their storage location to get
    | rid of old sessions from storage. Here are the chances that it will
    | happen on a given request. By default, the odds are 2 out of 100.
    |
    */

    'lottery' => [2, 100],
```

```
    /*
    |--------------------------------------------------------------------------
    | Session Cookie Name
    |--------------------------------------------------------------------------
    |
    | Here you may change the name of the session cookie that is created by
    | the framework. Typically, you should not need to change this value
    | since doing so does not grant a meaningful security improvement.
    |
    */

    'cookie' => env(
        'SESSION_COOKIE',
        Str::slug(env('APP_NAME', 'laravel'), '_').'_session'
    ),

    /*
    |--------------------------------------------------------------------------
    | Session Cookie Path
    |--------------------------------------------------------------------------
    |
    | The session cookie path determines the path for which the cookie will
    | be regarded as available. Typically, this will be the root path of
    | your application, but you're free to change this when necessary.
    |
    */

    'path' => env('SESSION_PATH', '/'),

    /*
    |--------------------------------------------------------------------------
    | Session Cookie Domain
    |--------------------------------------------------------------------------
    |
    | This value determines the domain and subdomains the session cookie is
    | available to. By default, the cookie will be available to the root
    | domain and all subdomains. Typically, this shouldn't be changed.
    |
    */

    'domain' => env('SESSION_DOMAIN'),

    /*
    |--------------------------------------------------------------------------
    | HTTPS Only Cookies
    |--------------------------------------------------------------------------
    |
    | By setting this option to true, session cookies will only be sent back
    | to the server if the browser has a HTTPS connection. This will keep
    | the cookie from being sent to you when it can't be done securely.
    |
    */

    'secure' => env('SESSION_SECURE_COOKIE'),

    /*
    |--------------------------------------------------------------------------
    | HTTP Access Only
    |--------------------------------------------------------------------------
    |
    | Setting this value to true will prevent JavaScript from accessing the
    | value of the cookie and the cookie will only be accessible through
    | the HTTP protocol. It's unlikely you should disable this option.
    |
    */
```

```
    'http_only' => env('SESSION_HTTP_ONLY', true),

    /*
    |--------------------------------------------------------------------------
    | Same-Site Cookies
    |--------------------------------------------------------------------------
    |
    | This option determines how your cookies behave when cross-site requests
    | take place, and can be used to mitigate CSRF attacks. By default, we
    | will set this value to "lax" to permit secure cross-site requests.
    |
    | See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie#samesitesamesite-value
    |
    | Supported: "lax", "strict", "none", null
    |
    */

    'same_site' => env('SESSION_SAME_SITE', 'lax'),

    /*
    |--------------------------------------------------------------------------
    | Partitioned Cookies
    |--------------------------------------------------------------------------
    |
    | Setting this value to true will tie the cookie to the top-level site for
    | a cross-site context. Partitioned cookies are accepted by the browser
    | when flagged "secure" and the Same-Site attribute is set to "none".
    |
    */

    'partitioned' => env('SESSION_PARTITIONED_COOKIE', false),

];
```

FILE: database/factories/CategoryFactory.php

```
<?php
namespace Database\Factories;

use App\Models\Category;
use Illuminate\Database\Eloquent\Factories\Factory;

class CategoryFactory extends Factory
{
    protected $model = Category::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        // Predefined real category names and descriptions
        $categories = [
            ['category_name' => 'Technology', 'description' => 'Latest advancements in technology and gadgets'],
            ['category_name' => 'Health & Wellness', 'description' => 'Tips and information on staying healthy'],
            ['category_name' => 'Business', 'description' => 'Insights on entrepreneurship and business trends'],
            ['category_name' => 'Education', 'description' => 'Resources and knowledge for personal growth'],
            ['category_name' => 'Fashion', 'description' => 'Trends and styles in the fashion world'],
            // Add more categories as needed
        ];

        // Randomly pick a category from the list
```

```
        $category = $categories[array_rand($categories)];

        return [
            'category_name' => $category['category_name'],
            'description' => $category['description'],
        ];
    }
}
```

FILE: database/factories/ChatbotFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Chatbot;
use Illuminate\Database\Eloquent\Factories\Factory;

class ChatbotFactory extends Factory
{
    protected $model = Chatbot::class;

    public function definition()
    {
        return [
            'user_id' => 1, // Assigning user_id as 1
            'history' => $this->generateHistory(), // Call the method to generate the history
        ];
    }

    // Helper method to generate the history structure with more consistent and varied data
    private function generateHistory()
    {
        // Predefined patterns for 'user' and 'model' roles
        $history = [
            // Initial greeting
            [
                'role' => 'user',
                'parts' => [
                    ['text' => 'Hi, I need help with my account.']
                ],
            ],
            [
                'role' => 'model',
                'parts' => [
                    ['text' => 'Hello! How can I assist you today?']
                ],
            ],
            // Account-related issue
            [
                'role' => 'user',
                'parts' => [
                    ['text' => 'I’m having trouble logging in.']
                ],
            ],
            [
                'role' => 'model',
                'parts' => [
                    ['text' => 'I see. Are you receiving any error messages?']
                ],
            ],
            [
                'role' => 'user',
                'parts' => [
```

```
                    ['text' => 'Yes, it says my password is incorrect.']
                ],
            ],
            [
                'role' => 'model',
                'parts' => [
                    ['text' => 'Got it. Have you tried resetting your password yet?']
                ],
            ],
            // Resetting password flow with a slight variance
            [
                'role' => 'user',
                'parts' => [
                    ['text' => 'I tried, but I didn’t get the reset email.']
                ],
            ],
            [
                'role' => 'model',
                'parts' => [
                    ['text' => 'Hmm, that’s strange. Please check your spam folder.']
                ],
            ],
            // Offering assistance with further issues
            [
                'role' => 'user',
                'parts' => [
                    ['text' => 'I checked, and it’s not there. What should I do next?']
                ],
            ],
            [
                'role' => 'model',
                'parts' => [
                    ['text' => 'I can help you with that. Let me escalate the issue.']
                ],
            ],
            [
                'role' => 'user',
                'parts' => [
                    ['text' => 'Thanks for your help!']
                ],
            ],
            [
                'role' => 'model',
                'parts' => [
                    ['text' => 'You’re welcome! Let me know if you need anything else.']
                ],
            ],
        ];

        return $history; // Return the predefined, varied conversation history
    }
}
```

FILE: database/factories/CommentFactory.php

```
<?php
namespace Database\Factories;

use App\Models\Comment;
use App\Models\Post;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class CommentFactory extends Factory
```

```
{
    protected $model = Comment::class;

    public function definition()
    {
        $comments = [
            'This product is amazing! Exactly as described, will definitely buy again.',
            'Great quality and fast shipping, I am very happy with my purchase.',
            'The product didn’t meet my expectations. It arrived damaged.',
            'I love this item! It works perfectly and was easy to use.',
            'Not worth the price, I found a better alternative elsewhere.',
            'Shipping took longer than expected, but the product is great.',
            'Excellent customer service, the seller was very helpful with my inquiries.',
            'I am very satisfied with this purchase, highly recommend it!',
            'It’s okay, but I expected more for the price.',
            'The product is good but has some minor defects.',
        ];

        return [
            'post_id' => Post::factory(), // Creates a new post for the comment
            'user_id' => User::factory(), // Creates a new user for the comment
            'content' => $this->faker->randomElement($comments), // Picks a random comment from the array
        ];
    }
}
```

FILE: database/factories/ConnectionRequestFactory.php

```
<?php

namespace Database\Factories;

use App\Models\ConnectionRequest;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class ConnectionRequestFactory extends Factory
{
    protected $model = ConnectionRequest::class;

    public function definition()
    {
        return [
            'sender_id' => User::factory(), // Creates a new user for the sender
            'recipient_id' => User::factory(), // Creates a new user for the recipient
            'status' => $this->faker->randomElement(['pending', 'accepted', 'rejected']),
        ];
    }
}
```

FILE: database/factories/CouponFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Coupon;
use Illuminate\Database\Eloquent\Factories\Factory;

class CouponFactory extends Factory
{
```

```
    protected $model = Coupon::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'discount' => $this->faker->numberBetween(5, 50), // Random discount percentage between 5 and 50
            'expiration_date' => $this->faker->dateTimeBetween('now', '+1 year'), // Random expiration date within the next year
        ];
    }
}
```

FILE: database/factories/FeedbackFormAnswerFactory.php

```
<?php

namespace Database\Factories;

use App\Models\FeedbackFormAnswer;
use App\Models\FeedbackFormQuestionAnswer;

use Illuminate\Database\Eloquent\Factories\Factory;

class FeedbackFormAnswerFactory extends Factory
{
    protected $model = FeedbackFormAnswer::class;

    public function definition()
    {
        return [
            'start_date' => $this->faker->dateTimeBetween('now', '+1 month'),
            'end_date' => $this->faker->dateTimeBetween('+1 month', '+2 months'),
            'feedback_form_id' => null, // Set later
        ];
    }
}
```

FILE: database/factories/FeedbackFormFactory.php

```
<?php

namespace Database\Factories;

use App\Models\FeedbackForm;
use App\Models\FeedbackFormQuestion;
use App\Models\FeedbackFormAnswer;
use App\Models\FeedbackFormQuestionAnswer;

use Illuminate\Database\Eloquent\Factories\Factory;

class FeedbackFormFactory extends Factory
{
    protected $model = FeedbackForm::class;

    public function definition()
    {
        // Randomly select a tech company
```

```
        return [
            'user_id' => \App\Models\User::factory(), // Assuming you have a User factory
            'order_id' => \App\Models\Order::factory(), // Assuming you have a User factory
            'created_at' => now(),
            'updated_at' => now(),
        ];
    }

    public function configure()
    {
        return $this->afterCreating(function (FeedbackForm $FeedbackForm) {
            // Create associated FeedbackFormQuestions
            FeedbackFormQuestion::factory()
                ->count(3) // Change count as needed
                ->create(['feedback_form_id' => $FeedbackForm->id])
                ->each(function ($question) use ($FeedbackForm) {
                    // Create associated FeedbackFormAnswers for each question
                    $answer = FeedbackFormAnswer::factory()
                        ->create(['feedback_form_id' => $FeedbackForm->id]);

                    // Now create FeedbackFormQuestionAnswer
                    FeedbackFormQuestionAnswer::factory()
                        ->create([
                            'feedback_form_question_id' => $question->id,
                            'feedback_form_answer_id' => $answer->id,
                            'answer' => $this->faker->sentence, // Optionally add an answer
                        ]);
                });
        });
    }

}
```

FILE: database/factories/FeedbackFormQuestionAnswerFactory.php

```
<?php
namespace Database\Factories;

use App\Models\FeedbackFormQuestionAnswer;
use App\Models\FeedbackFormQuestion;
use App\Models\FeedbackFormAnswer;
use Illuminate\Database\Eloquent\Factories\Factory;

class FeedbackFormQuestionAnswerFactory extends Factory
{
    protected $model = FeedbackFormQuestionAnswer::class;

    public function definition()
    {
        // Define possible answers for feedback questions.
        $answers = [
            'Very satisfied with my purchase. Everything was perfect!',
            'Satisfied with the product, but shipping took longer than expected.',
            'The product didn’t match the description. I am disappointed.',
            'The product is good, but the packaging was damaged.',
            'Customer service was excellent, and the product quality is great!',
            'Not satisfied. The item was faulty, and the return process was confusing.',
            'Great value for money, will buy again.',
            'I had issues with the payment process, but the product itself is fine.',
            'The quality of the product exceeded my expectations.',
            'The product was as described, but the delivery service could be better.',
            'Very disappointed, I will not be purchasing again.',
            'The return process was easy, but the product didn’t work for me.',
```

```
            'Good product, but I wish there were more color options.',
            'Product was damaged on arrival, very dissatisfied.',
            'The product is okay but not worth the price.',
            'Excellent product and the delivery was very quick!',
            'I am happy with my purchase but had difficulty navigating the website.',
            'The product broke after one use, I want a refund.',
            'Not happy with the customer service response time.',
            'The product is exactly what I was looking for, would recommend.',
        ];

        return [
            'feedback_form_question_id' => FeedbackFormQuestion::factory(), // Create a question if it doesn't exist
            'feedback_form_answer_id' => FeedbackFormAnswer::factory(), // Create an answer if it doesn't exist
            'answer' => $this->faker->randomElement($answers), // Randomly pick an answer from the predefined list
        ];
    }
}
```

FILE: database/factories/FeedbackFormQuestionFactory.php

```
<?php

namespace Database\Factories;

use App\Models\FeedbackFormQuestion;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Str;

class FeedbackFormQuestionFactory extends Factory
{
    protected $model = FeedbackFormQuestion::class;

    // Array mapping questions to their descriptions and answers for an e-commerce platform
    private $feedbackQuestions = [
        'How would you rate the speed of delivery?' => [
            'description' => 'Please provide your feedback on the delivery time and whether it met your expectations.',
            'answers' => [
                'The delivery was quick and arrived ahead of schedule.',
                'The delivery took longer than expected, and I was disappointed.',
                'The delivery was on time, and I am satisfied with the service.',
            ],
        ],
        'How satisfied are you with the condition of the product upon arrival?' => [
            'description' => 'Share your experience with the product’s condition when it was delivered.',
            'answers' => [
                'The product arrived in perfect condition, exactly as described.',
                'The product was damaged during shipping, and I had to return it.',
                'The product was slightly damaged but still usable.',
            ],
        ],
        'How easy was it to place your order on our website?' => [
            'description' => 'Provide feedback on the ease of the ordering process on our website.',
            'answers' => [
                'The website was very easy to navigate, and I had no issues placing my order.',
                'I had some trouble finding the product I wanted and checking out.',
                'The order process was confusing, and I had to contact customer service.',
            ],
        ],
        'Were the product details accurate on the website?' => [
            'description' => 'Tell us whether the product descriptions, images, and specifications matched what you received.',
            'answers' => [
                'The product was exactly as described on the website.',
                'The product did not match the description; I was disappointed with the mismatch.',
                'Some aspects of the product description were inaccurate or unclear.',
```

```
            ],
        ],
        'How satisfied are you with the packaging of your order?' => [
            'description' => 'Please share your feedback regarding the packaging of your product.',
            'answers' => [
                'The product was well-packaged and protected during transit.',
                'The packaging was poor, and the product was slightly damaged.',
                'The packaging was sufficient but could have been improved.',
            ],
        ],
        'How would you rate the communication regarding your order status?' => [
            'description' => 'Provide feedback on how well we kept you informed about your order status and any updates.',
            'answers' => [
                'I received timely updates, and I was always informed about my order status.',
                'I did not receive enough communication and had to check manually.',
                'I was not informed about any delays or issues with my order.',
            ],
        ],
        'How satisfied are you with the quality of the product?' => [
            'description' => 'Give feedback on the quality of the product you received.',
            'answers' => [
                'The product exceeded my expectations and was of high quality.',
                'The product was not as expected and felt low quality.',
                'The product was of acceptable quality, but not as good as I had hoped.',
            ],
        ],
        'How likely are you to recommend this product to others?' => [
            'description' => 'Share whether you would recommend this product to friends or family.',
            'answers' => [
                'I would highly recommend this product to others.',
                'I would not recommend this product due to poor quality.',
                'I might recommend this product depending on the needs of the person.',
            ],
        ],
        'Was the product available when you wanted to purchase it?' => [
            'description' => 'Tell us if you were able to find and order the product easily.',
            'answers' => [
                'The product was readily available and I had no issues purchasing it.',
                'The product was out of stock, and I had to wait longer for delivery.',
                'The product was listed but later marked as unavailable.',
            ],
        ],
        'How do you rate the variety of products on our website?' => [
            'description' => 'Provide feedback on the range of products available on our site.',
            'answers' => [
                'There is a wide variety of products, and I found everything I needed.',
                'The product selection was limited, and I could not find what I wanted.',
                'The variety of products is good, but there could be more options.',
            ],
        ],
        'How easy was the return process if your order didn’t meet expectations?' => [
            'description' => 'Share your experience with our return process.',
            'answers' => [
                'The return process was straightforward and quick.',
                'The return process was complicated and took too long.',
                'The return process was acceptable, but it could be improved.',
            ],
        ],
        'How did you find the pricing of the product?' => [
            'description' => 'Tell us if you think the product was priced fairly based on its quality and the competition.',
            'answers' => [
                'The product was priced fairly and matched the quality.',
                'The product was overpriced compared to similar products.',
                'The product was a great deal considering its quality.',
            ],
        ],
```

```
        'Did you experience any issues during checkout?' => [
            'description' => 'Please share if you faced any issues while completing your order.',
            'answers' => [
                'The checkout process was smooth and easy.',
                'I encountered some issues during checkout but eventually completed the order.',
                'I had significant issues during checkout and had to contact support.',
            ],
        ],
        'How do you feel about the shipping costs?' => [
            'description' => 'Tell us if you think the shipping fees were reasonable for the service provided.',
            'answers' => [
                'The shipping costs were reasonable for the delivery time and service.',
                'The shipping was too expensive, especially for standard delivery.',
                'I was happy with the free shipping offer and did not mind the price.',
            ],
        ],
        'How would you rate our customer service if you contacted them?' => [
            'description' => 'Provide feedback on any interaction with our customer service team.',
            'answers' => [
                'Customer service was very helpful and resolved my issue quickly.',
                'Customer service was slow to respond and didn’t fully resolve my issue.',
                'I did not need to contact customer service.',
            ],
        ],
        'How would you rate the overall ordering experience on our website?' => [
            'description' => 'Give an overall rating of your experience with our online store.',
            'answers' => [
                'The overall experience was excellent, and I will shop here again.',
                'The experience was okay, but there are a few things that could be improved.',
                'I did not have a good experience and may not shop here again.',
            ],
        ],
        'How satisfied are you with the ease of finding products on our website?' => [
            'description' => 'Please share if you found it easy to search for and find the products you wanted.',
            'answers' => [
                'The website’s search and filters made it easy to find what I was looking for.',
                'The search functionality was okay, but it could be improved.',
                'I had trouble finding what I was looking for on the website.',
            ],
        ],
        'Did the product meet your expectations based on the website images and descriptions?' => [
            'description' => 'Let us know if the product matched your expectations after viewing it online.',
            'answers' => [
                'The product was exactly as shown and described on the website.',
                'The product didn’t match the online description or image.',
                'Some aspects of the product were different from what I expected based on the description.',
            ],
        ],
        'How would you rate your overall satisfaction with the product delivery?' => [
            'description' => 'Please share your thoughts on the overall delivery process.',
            'answers' => [
                'I am very satisfied with the delivery service.',
                'I was dissatisfied with the delivery and had issues with the service.',
                'The delivery was acceptable, but it could be improved.',
            ],
        ],
        'Would you consider purchasing from us again in the future?' => [
            'description' => 'Let us know if you would consider buying from our website again based on this experience.',
            'answers' => [
                'Yes, I will definitely shop here again.',
                'I might consider shopping here again depending on future experiences.',
                'I would not consider shopping here again based on my current experience.',
            ],
        ],
    ];
```

```
    public function definition()
    {
        // Randomly select a question and its corresponding description and answers
        $question = $this->faker->randomElement(array_keys($this->feedbackQuestions));
        $description = $this->feedbackQuestions[$question]['description'];
        $answers = $this->feedbackQuestions[$question]['answers'];

        // Randomly decide if 'data' will be text or an options array
        $dataType = $this->faker->randomElement(['text','select', 'radio', 'checkbox','textarea' ]);

        if ($dataType != 'text' && $dataType != 'textarea') {
            // Create options based on the answers for the question
            $options = [
                'options' => array_map(function ($answer) {
                    return [
                        'uuid' => (string) Str::uuid(),
                        'text' => $answer,
                    ];
                }, $answers),
            ];
            $data = json_encode($options); // Encode the array as a JSON string
        } else {
            $data = $this->faker->text; // Simple text (if needed for some questions)
        }

        return [
            'type' => $dataType, // Keep this if you have different types
            'question' => $question, // Random job question
            'description' => $description, // Corresponding job description
            'data' => $data,
            'feedback_form_id' => null, // Set later
        ];
    }
}
```

FILE: database/factories/GroupFactory.php

```
<?php
namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;
use App\Models\User;

class GroupFactory extends Factory
{
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        // Predefined set of group names and descriptions for e-commerce.
        $groupNames = [
            'Electronics',
            'Fashion',
            'Home & Living',
            'Beauty & Personal Care',
            'Sports & Outdoors',
            'Toys & Games',
            'Books & Media',
```

```
            'Health & Wellness',
            'Automotive',
            'Food & Groceries'
        ];

        $groupDescriptions = [
            'Explore a wide range of electronics, including the latest gadgets and devices.',
            'Trendy fashion items, clothing, accessories, and shoes for every season.',
            'Furniture, home decor, and appliances to help you create the perfect home.',
            'Top-quality beauty and personal care products to keep you feeling your best.',
            'Everything you need for your outdoor adventures and fitness goals.',
            'Toys, games, and activities for children of all ages.',
            'A vast collection of books, movies, and music to entertain and educate.',
            'Products for your health and well-being, from supplements to fitness equipment.',
            'Car accessories, parts, and tools for maintaining your vehicle.',
            'Fresh food, snacks, and groceries delivered to your doorstep.'
        ];

        return [
            'name' => $this->faker->randomElement($groupNames), // Pick a random group name from the predefined list
            'description' => $this->faker->randomElement($groupDescriptions), // Pick a random description
            'owner_id' => User::inRandomOrder()->first()->id, // Assign a random user as the owner
        ];
    }
}
```

FILE: database/factories/InventoryFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Inventory;
use App\Models\Product;
use App\Models\Warehouse;
use Illuminate\Database\Eloquent\Factories\Factory;

class InventoryFactory extends Factory
{
    protected $model = Inventory::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'stock' => $this->faker->numberBetween(10, 500), // Random stock between 10 and 500 units
            'last_updated' => $this->faker->dateTimeThisYear(), // Random last updated date within this year
            'weight_per_unit' => $this->faker->randomFloat(2, 0.5, 10), // Random weight per unit between 0.5 and 10 kg
            'products_id' => Product::factory(), // Associate with a random product using the Product factory
            'warehouses_id' => Warehouse::factory(), // Associate with a random warehouse using the Warehouse factory
        ];
    }
}
```

FILE: database/factories/InvoiceFactory.php

```
<?php
```

```
namespace Database\Factories;

use App\Models\Invoice;
use App\Models\Payment;
use Illuminate\Database\Eloquent\Factories\Factory;

class InvoiceFactory extends Factory
{
    protected $model = Invoice::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'customer_name' => $this->faker->name(), // Random customer name
            'created_date' => $this->faker->dateTimeThisYear(), // Random date within this year
            'payment_status' => $this->faker->randomElement(['paid', 'unpaid', 'pending']), // Random payment status
            'payment_method' => $this->faker->randomElement(['credit_card', 'paypal', 'bank_transfer']), // Random payment method
            'description' => $this->faker->sentence(), // Random description
            'shipping_cost' => $this->faker->randomFloat(2, 5, 50), // Random shipping cost between 5 and 50
            'total_amount' => $this->faker->randomFloat(2, 50, 500), // Random total amount between 50 and 500
            'paid_amount' => $this->faker->randomFloat(2, 10, 500), // Random paid amount between 10 and 500
            'due_amount' => $this->faker->randomFloat(2, 0, 500), // Random due amount between 0 and 500
            'currency' => $this->faker->randomElement(['USD', 'EUR', 'GBP']), // Random currency
            'discount' => $this->faker->randomFloat(2, 0, 50), // Random discount between 0 and 50
            'payments_id' => Payment::factory(), // Associate with a payment using the Payment factory
        ];
    }
}
```

FILE: database/factories/LikeFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Like;
use App\Models\User;
use App\Models\Post;
use Illuminate\Database\Eloquent\Factories\Factory;

class LikeFactory extends Factory
{
    protected $model = Like::class;

    public function definition()
    {
        return [
            'user_id' => User::factory(), // Creates a new user for each like
            'post_id' => Post::factory(), // Creates a new post for each like, adjust as needed
        ];
    }
}
```

FILE: database/factories/LocationHistoryFactory.php

```php
<?php

namespace Database\Factories;

use App\Models\LocationHistory;
use App\Models\Vehicle;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Str;

class LocationHistoryFactory extends Factory
{
    protected $model = LocationHistory::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'vehicle_id' => Vehicle::factory(), // Associate a random vehicle
            'location' => $this->faker->address, // Random address
            'latitude' => $this->faker->latitude(), // Random latitude
            'longitude' => $this->faker->longitude(), // Random longitude
            'timestamp' => $this->faker->dateTimeThisYear(), // Random timestamp this year
        ];
    }

}
```

FILE: database/factories/MessageFactory.php

```php
<?php
namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;
use App\Models\User;
use App\Models\Group;

class MessageFactory extends Factory
{
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        // Predefined set of real message content related to e-commerce scenarios
        $messages = [
            'Hi, I have a question about the product I ordered. Is it compatible with model XYZ?',
            'My order hasn’t arrived yet. Can you provide an update on the delivery status?',
            'I would like to return an item I purchased. Could you please assist me with the process?',
            'Can you help me track my order? I ordered it over a week ago and haven’t received any updates.',
            'I received the wrong product. Please let me know how to return it.',
            'The quality of the item doesn’t match the description. I would like a refund.',
            'I received my package, but it was damaged. What steps do I need to take for a replacement?',
            'I would like to know when this item will be back in stock. Can you provide any information?',
            'Can you recommend similar products? I’m not satisfied with my current purchase.',
            'Thank you for the fast shipping! I’m very happy with the product.',
            'I love this product! It’s exactly what I was looking for.',
            'How can I change my shipping address for an existing order?',
            'Is there any way to expedite my order? I need it urgently.',
```

```
            'Do you have a size guide for your clothing? I’m unsure about which size to select.',
            'I’m not able to apply my discount code. Could you assist with that?',
            'Can I cancel my order? I made a mistake while ordering.',
            'The product was great, but I didn’t like the packaging. Can you improve it?',
        ];

        // Randomly pick a message from the predefined list
        $messageContent = $this->faker->randomElement($messages);

        // Logic for determining sender and receiver, or group
        $senderId = $this->faker->randomElement([0, 1]);
        if ($senderId === 0) {
            $senderId = $this->faker
                ->randomElement(User::where('id', '!=', 1)
                    ->pluck('id')->toArray());
            $receiverId = 1;
        } else {
            $receiverId = $this->faker->randomElement(User::pluck('id')->toArray());
        }

        $groupId = null;
        if ($this->faker->boolean(50)) {
            $groupId = $this->faker->randomElement(Group::pluck('id')->toArray());
            // Select group by group_id
            $group = Group::find($groupId);
            $senderId = $this->faker->randomElement($group->users->pluck('id')->toArray());
            $receiverId = null;
        }

        return [
            'sender_id' => $senderId,
            'receiver_id' => $receiverId,
            'group_id' => $groupId,
            'message' => $messageContent,  // Using the real message content
            'created_at' => $this->faker->dateTimeBetween('-1 year', 'now'),
        ];
    }
}
```

FILE: database/factories/NotificationFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Notification;
use App\Models\Post;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class NotificationFactory extends Factory
{
    protected $model = Notification::class;

    public function definition()
    {
        return [
            'recipient_id' => User::factory(), // Creates a new user for the recipient
            'type' => $this->faker->randomElement(['like', 'comment', 'connectionAccepted']),
            'related_user' => User::factory(),
            'related_post' => Post::factory(),
            'read' => $this->faker->boolean(),
        ];
    }
```

```
}
```

FILE: database/factories/OrderFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Order;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class OrderFactory extends Factory
{
    protected $model = Order::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'status' => $this->faker->randomElement(['Paid','Pending', 'Confirmed', 'Packed', 'Delivery Scheduled', 'Delivery Maintenance
Checked','On Delivery', 'Delivered']), // Random status
            'order_date' => $this->faker->dateTimeThisYear(), // Random date within this year
            'user_id' => User::factory(), // Associate with a user using the User factory
            'tracking_number' => $this->faker->unique()->numerify('TRACK-#######'), // Random tracking number
            'total_amount' => $this->faker->randomFloat(2, 10, 500), // Random total amount between 10 and 500
        ];
    }
}
```

FILE: database/factories/OrderItemFactory.php

```
<?php

namespace Database\Factories;

use App\Models\OrderItem;
use App\Models\Order;
use App\Models\Product;
use Illuminate\Database\Eloquent\Factories\Factory;

class OrderItemFactory extends Factory
{
    protected $model = OrderItem::class;

    public function definition()
    {
        return [
            // Generate fake data for the columns
            'orders_id' => \App\Models\Order::factory(),  // Foreign key to the Order table
            'products_id' => \App\Models\Product::factory(),  // Foreign key to the Product table
            'quantity' => $this->faker->numberBetween(1, 10),
            'total_amount' => $this->faker->randomFloat(2, 5, 100),
        ];
    }
}
```

FILE: database/factories/PaymentFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Payment;
use App\Models\Order;
use App\Models\Provider;
use Illuminate\Database\Eloquent\Factories\Factory;

class PaymentFactory extends Factory
{
    protected $model = Payment::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'total_amount' => $this->faker->randomFloat(2, 50, 1000), // Total amount between 50 and 1000
            'paid_amount' => $this->faker->randomFloat(2, 0, 1000), // Paid amount, up to total amount
            'due_amount' => function (array $attributes) {
                return max(0, $attributes['total_amount'] - $attributes['paid_amount']);
            },
            'payment_method' => $this->faker->randomElement(['credit_card', 'paypal', 'bank_transfer', 'cash']),
            'payment_status' => $this->faker->randomElement(['pending', 'completed', 'failed', 'refunded']),
            'payment_date' => $this->faker->dateTimeThisYear(), // Random date within this year
            'gateway' => $this->faker->company, // Random company name as the payment gateway
            'currency' => $this->faker->randomElement(['USD', 'EUR', 'GBP', 'INR']), // Random currency
            'order_id' => Order::factory(), // Associate with an order using the Order factory
            'providers_id' => Provider::factory(), // Associate with a provider using the Provider factory
        ];
    }
}
```

FILE: database/factories/PermissionFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Permission;
use Illuminate\Database\Eloquent\Factories\Factory;

class PermissionFactory extends Factory
{
    protected $model = Permission::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'permission_name' => '',
            'description' => '',
```

```
        ];
    }

    /**
     * Return all predefined permissions for the application.
     *
     * @return array<int, array{permission_name: string, description: string}>
     */
    public static function allPermissions(): array
    {
        return [
            // ------------------------
            // 1. Administration
            // ------------------------
            ['permission_name' => 'Manage Users', 'description' => 'Full access to user management: view and update profiles, change
passwords, set language preferences.'],
            ['permission_name' => 'Manage Products', 'description' => 'Full control over products: create, update, delete, and manage
featured status.'],
            ['permission_name' => 'Manage Inventory', 'description' => 'Full warehouse and inventory management: view, update, create,
analyze inventory; manage orders, expenses; process license plates.'],
            ['permission_name' => 'Manage Vehicles', 'description' => 'Full vehicle and delivery management: manage vehicles, assign
routes, monitor shipments.'],
            ['permission_name' => 'Manage Cart Coupons Payments', 'description' => 'Full cart, coupon, and payment management: process
payments, manage coupons, handle delivery preparation.'],
            ['permission_name' => 'Manage Social Messaging', 'description' => 'Full social and messaging management: connections,
notifications, posts, groups, chat, chatbot, emails.'],
            ['permission_name' => 'Manage Live Streaming', 'description' => 'Full live streaming and LiveKit management: create streams,
manage viewers, send messages, manage access tokens.'],
            ['permission_name' => 'Manage Media Editor Tools', 'description' => 'Full media and editor tools access: image/video editing,
background manipulation, transcription, uploads.'],
            ['permission_name' => 'Manage Two Factor Authentication', 'description' => 'Two-factor authentication management.'],

            // ------------------------
            // 2. Warehouse Manager
            // ------------------------
            ['permission_name' => 'Warehouse Manage Inventory', 'description' => 'Access all warehouse and inventory management
functionalities: view, create, update inventories, track orders, manage expenses, process license plates.'],
            ['permission_name' => 'Warehouse Limited User Management', 'description' => 'Limited access to user profile management.'],

            // ------------------------
            // 3. Vehicle Manager
            // ------------------------
            ['permission_name' => 'Vehicle Manage', 'description' => 'Manage vehicles: view and update vehicles with management
details.'],

            // ------------------------
            // 4. Delivery Man / Driver
            // ------------------------
            ['permission_name' => 'Delivery View Shipments', 'description' => 'Access shipments: view shipment details and assigned
vehicle.'],
            ['permission_name' => 'Delivery Manage Routes', 'description' => 'Can manage route assignments for deliveries.'],

            // ------------------------
            // 5. Customer
            // ------------------------
            ['permission_name' => 'Customer Manage Profile', 'description' => 'Manage personal profile and addresses.'],
            ['permission_name' => 'Customer Manage Cart Payments Coupons', 'description' => 'Manage cart, payments, and coupons.'],
            ['permission_name' => 'Customer View Products', 'description' => 'Access to view products, ratings, and recommendations.'],
            ['permission_name' => 'Customer Social Features', 'description' => 'Participate in social features: posts, comments, likes,
connections, notifications, block/follow.'],
            ['permission_name' => 'Customer Messaging Chatbot', 'description' => 'Access messaging and chatbot interactions.'],
            ['permission_name' => 'Customer Live Stream', 'description' => 'Participate in live streaming: own streams, view streams,
receive viewer tokens.'],
            ['permission_name' => 'Customer Media Tools', 'description' => 'Access basic media and editor tools for personal use.'],

            // ------------------------
```

```
            // 6. ProductSaler
            // ------------------------
            ['permission_name' => 'Productsaler Feedback Forms', 'description' => 'Manage feedback forms for orders.'],
            ['permission_name' => 'Productsaler Manage Coupons Visibility', 'description' => 'Manage coupons and assist in product
visibility.'],

            // ------------------------
            // 7. Customer Support Staff
            // ------------------------
            ['permission_name' => 'Support Read Products', 'description' => 'Read access to products.'],
            ['permission_name' => 'Support Read Ratings', 'description' => 'Read access to ratings.'],
            ['permission_name' => 'Support Read Posts', 'description' => 'Read access to posts.'],
            ['permission_name' => 'Support Read Social', 'description' => 'Read access to social connections and notifications.'],
        ];
    }
    /**
     * Return permissions suitable for a given role
     */
    public static function permissionsForRole(string $roleName): array
    {
        $all = self::allPermissions();

        return match ($roleName) {
            'Administration' => $all, // Admin gets all
            'WarehouseManager' => array_filter($all, fn($p) => str_contains($p['permission_name'], 'Warehouse') ||
str_contains($p['permission_name'], 'Manage Profile')),
            'VehicleManager' => array_filter($all, fn($p) => str_contains($p['permission_name'], 'Vehicle')),
            'DeliveryDriver', 'DeliveryMan' => array_filter($all, fn($p) => str_contains($p['permission_name'], 'Delivery')),
            'Customer' => array_filter($all, fn($p) => str_starts_with($p['permission_name'], 'Customer')),
            'ProductSaler' => array_filter($all, fn($p) => str_starts_with($p['permission_name'], 'Customer') ||
str_contains($p['permission_name'], 'Productsaler')),
            'CustomerSupportStaff' => array_filter($all, fn($p) => str_starts_with($p['permission_name'], 'Support')),
            default => [],
        };
    }
}
```

FILE: database/factories/PostFactory.php

```
<?php
namespace Database\Factories;

use App\Models\Post;
use App\Models\User;
use App\Models\Comment;
use App\Models\Like;
use Illuminate\Database\Eloquent\Factories\Factory;

class PostFactory extends Factory
{
    protected $model = Post::class;

    // Predefined image URLs (these could represent products or promotions)
    private static $imageUrls = [
        'https://cdn.thewirecutter.com/wp-content/media/2024/11/runningshoes-2048px-09522.jpg?auto=webp&quality=75&width=1024',
        'https://m.media-amazon.com/images/I/61V98P7+jiL.jpg',
        'https://down-sg.img.susercontent.com/file/2520047a3506c1dc94f41677d129f540',
        'https://cdn.thewirecutter.com/wp-content/media/2024/05/white-sneaker-2048px-9320.jpg?
auto=webp&quality=75&crop=1.91:1&width=1200',
        'https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTbzCINXMUbqCfb_MnCjyooBavt2dYK-gVrBQ&s',
        'https://assets.vogue.com/photos/5891e0ebb482c0ea0e4db2a8/4:3/w_2560%2Cc_limit/02-lestrange.jpg',
        'https://i.ebayimg.com/images/g/sTgAAOSwinJk3YuY/s-l1200.jpg',
        'https://marilyn.nz/wp-content/uploads/2021/12/Morilee-Wedding-Dress-5960-Feature.webp'
    ];
```

```
    // Predefined content for e-commerce posts, such as product promotions, announcements, or sales
    private static $postContent = [
        'Check out our new arrivals! These sneakers are perfect for your daily runs. Get yours today with 10% off!',
        'Limited time offer: Buy one, get one free on all summer dresses! Don\'t miss out on this amazing deal.',
        'Hurry up! Our biggest sale of the year is happening right now. Get up to 50% off on selected items.',
        'Have you seen our latest collection of kitchen gadgets? They\'re now available at discounted prices!',
        'The best time to upgrade your wardrobe is now. Shop our winter collection and save up to 30%.',
        'Introducing our new smart watch that helps track your health! Get it now with an exclusive discount.',
        'Customer review: "These headphones are amazing! The sound quality is top-notch, and the comfort is unbeatable!"',
        'Our most popular shoes are now back in stock! Get yours before they run out again!',
        'Want to look stylish for less? Don\'t miss out on our Black Friday sale with huge discounts!',
        'We’ve added new products to our beauty collection. Explore the latest skincare items on sale!',
        'Get ready for the holidays with our seasonal promotions! Gift cards are now available for purchase!',
    ];

    public function definition()
    {
        return [
            'author_id' => User::factory(), // Creates a new user for the author (could be a business or admin)
            'content' => $this->getRandomContent(), // Randomly selected real post content
            'image' => $this->getRandomImage(), // Randomly selected image URL
        ];
    }

    public function configure()
    {
        return $this->afterCreating(function (Post $post) {
            // Create comments for the post
            Comment::factory()
                ->count(3) // Adjust the count as needed
                ->create([
                    'post_id' => $post->id,
                    'user_id' => User::factory(), // Creates a new user for each comment
                ]);

            // Create likes for the post
            $likeCount = 5; // For example, create 5 likes
            for ($i = 0; $i < $likeCount; $i++) {
                Like::factory()->create([
                    'post_id' => $post->id,
                    'user_id' => User::factory(), // Creates a new user for each like
                ]);
            }
        });
    }

    /**
     * Get a random image URL from the collection.
     *
     * @return string
     */
    private function getRandomImage(): string
    {
        // Randomly select an image URL from the collection
        return self::$imageUrls[array_rand(self::$imageUrls)];
    }

    /**
     * Get a random post content from the predefined list.
     *
     * @return string
     */
    private function getRandomContent(): string
    {
        // Randomly select a post content from the predefined list
```

```
        return self::$postContent[array_rand(self::$postContent)];
    }
}
```

FILE: database/factories/ProductCategoryFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Product;
use App\Models\Category;
use App\Models\ProductCategory;
use Illuminate\Database\Eloquent\Factories\Factory;

class ProductCategoryFactory extends Factory
{
    // Define the associated model
    protected $model = ProductCategory::class;

    public function definition()
    {
        // Get random product and category IDs from existing records
        return [
            'products_id' => Product::factory(),  // Random product
            'categories_id' => Category::factory(),  // Random category
        ];
    }
}
```

FILE: database/factories/ProductCouponFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Product;
use App\Models\Coupon;
use App\Models\ProductCoupon;
use Illuminate\Database\Eloquent\Factories\Factory;

class ProductCouponFactory extends Factory
{
    protected $model = ProductCoupon::class;

    public function definition()
    {
        return [
            'products_id' => Product::factory(), // Assumes a Product factory exists
            'coupons_id' => Coupon::factory(),   // Assumes a Coupon factory exists
        ];
    }
}
```

FILE: database/factories/ProductFactory.php

```
<?php
```

```
namespace Database\Factories;

use App\Models\Product;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class ProductFactory extends Factory
{
    protected $model = Product::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        // List of real product names
        $productNames = [
            'Bose QuietComfort 35 II Wireless Headphones',
            'Samsung 65" QLED 4K Smart TV',
            'Fitbit Charge 5 Fitness and Health Tracker',
            'JBL Flip 5 Waterproof Bluetooth Speaker',
            'Wüsthof Classic 7-Piece Knife Set',
            'Fossil Men’s RFID-Blocking Leather Wallet',
            'Nike Air Zoom Pegasus 38 Running Shoes',
            'Breville Smart Kettle with Temperature Control',
            'Anker PowerCore 20000mAh Portable Charger',
            'Herman Miller Aeron Ergonomic Office Chair'
        ];

        // List of real product descriptions for e-commerce
        $descriptions = [
            'High-quality wireless headphones with noise cancellation, deep bass, and up to 20 hours of battery life.',
            'Ultra HD 4K Smart TV with built-in streaming apps, voice control, and a slim, modern design.',
            'Track your fitness goals, heart rate, steps, calories burned, and more with this stylish smartwatch.',
            'Water-resistant Bluetooth speaker with crystal-clear sound, deep bass, and 12 hours of continuous playtime.',
            'Professional-grade stainless steel kitchen knife set with ergonomic handles and razor-sharp blades for all your cooking
needs.',
            'Stylish and durable leather wallet with multiple card slots, a coin pocket, and a sleek, minimalist design.',
            'Comfortable, lightweight, and durable running shoes designed for both long-distance runs and casual wear.',
            'Cordless electric kettle with rapid boiling capabilities, temperature control, and automatic shut-off for safety and
convenience.',
            'Portable power bank with a high capacity to charge your devices multiple times, perfect for travel or emergencies.',
            'Ergonomic office chair with adjustable height, lumbar support, and breathable mesh fabric for all-day comfort.'
        ];

        // Randomly select a product name and description
        $name = $this->faker->randomElement($productNames);
        $description = $this->faker->randomElement($descriptions);

        // List of image URLs for products
        $imageUrls = [
            '/1.png',
            '/2.png',
            '/3.png',
            '/4.png',
            '/5.png',
            '/6.png',
            '/7.png',
            '/8.png',
        ];

        return [
            'name' => $name, // Real product name from the list
            'price' => $this->faker->randomFloat(2, 1, 1000), // Random price between 1 and 1000 with 2 decimals
```

```
            'description' => $description, // Real product description from the list
            'supplier_id' => User::factory(), // Randomly associate a user (supplier) using the User factory
            'isFeatured' => $this->faker->boolean(50), // Randomly set the isFeatured column to true (50% chance)
            'image' => $this->faker->randomElement($imageUrls), // Select a random image URL from the list
        ];
    }
}
```

FILE: database/factories/ProviderFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Provider;
use Illuminate\Database\Eloquent\Factories\Factory;

class ProviderFactory extends Factory
{
    protected $model = Provider::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'type' => $this->faker->randomElement(['Shipping', 'Storage', 'Logistics', 'Packaging']), // Random provider type
            'terms_of_service' => $this->faker->paragraph, // Random terms of service text
        ];
    }
}
```

FILE: database/factories/RatingFactory.php

```
<?php
namespace Database\Factories;

use App\Models\Rating;
use App\Models\Shipment;
use App\Models\Product;
use Illuminate\Database\Eloquent\Factories\Factory;

class RatingFactory extends Factory
{
    protected $model = Rating::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        // List of realistic feedback comments often found in e-commerce reviews
        $feedbacks = [
            'I am very satisfied with this product. It works as expected and is of good quality.',
            'Great value for the money! The product is exactly as described, and it arrived on time.',
            'Not bad, but the product could be improved in certain areas. It does the job, but I expected more.',
```

```
            'Disappointing. The quality did not meet my expectations, and it didn\'t work as I hoped.',
            'Terrible! This product broke within a few days of use. Would not recommend.',
            'Fantastic! Exceeded my expectations. Will definitely buy from this seller again.',
            'The product is decent but had some issues with the setup. Customer service was helpful though.',
            'Perfect fit for what I needed! Very easy to use and good build quality.',
            'Very poor quality. It broke after a week, and I couldn’t get a refund.',
            'The product is fine, but the shipping took longer than expected, which was frustrating.'
        ];

        return [
            'rating_value' => $this->faker->numberBetween(1, 5), // Random rating value between 1 and 5
            'feedback' => $this->faker->randomElement($feedbacks), // Randomly select a realistic feedback comment
            'date_created' => $this->faker->dateTimeBetween('-1 year', 'now'), // Random date within the last year
            'shipments_id' => Shipment::factory(), // Associate a shipment using the Shipment factory
            'products_id' => Product::factory(), // Associate a product using the Product factory
        ];
    }

    /**
     * Indicate that the rating is for a highly rated product.
     *
     * @return \Illuminate\Database\Eloquent\Factories\Factory
     */
    public function highlyRated(): static
    {
        return $this->state(fn (array $attributes) => [
            'rating_value' => 5, // Set rating to 5
            'feedback' => 'Excellent product! Highly recommended to everyone.',
        ]);
    }

    /**
     * Indicate that the rating is for a low-rated product.
     *
     * @return \Illuminate\Database\Eloquent\Factories\Factory
     */
    public function lowRated(): static
    {
        return $this->state(fn (array $attributes) => [
            'rating_value' => 1, // Set rating to 1
            'feedback' => 'Very poor quality. It broke after a few uses, not worth the price.',
        ]);
    }
}
```

FILE: database/factories/RoleFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Role;
use Illuminate\Database\Eloquent\Factories\Factory;

class RoleFactory extends Factory
{
    protected $model = Role::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
```

```
    {
        // List of roles with their corresponding descriptions
        $roles = [
            'Administration' => 'Responsible for overseeing all administrative functions and ensuring the organization runs smoothly.',
            'WarehouseManager' => 'Manages the daily operations of the warehouse, including inventory, shipping, and staff management.',
            'DeliveryDriver' => 'Ensures timely and accurate delivery of goods to customers, adhering to delivery schedules.',
            'Customer' => 'A client or individual who purchases or uses the services offered by the company.',
            'CustomerSupportStaff' => 'Provides assistance to customers by addressing inquiries, complaints, and service-related issues.',
            'FinanceManager' => 'Oversees financial operations, budgeting, and reporting to ensure the company’s financial health.',
            'ProductSaler' => 'Create product posts, sale products, delete products',
            'VehicleManager' => 'Manage Vehicle Maintenance',
            'DeliveryMan' => 'Deliver Orders',
        ];


        // Select a random role and its corresponding description
        $role_name = $this->faker->randomElement(array_keys($roles));
        $description = $roles[$role_name];

        return [
            'role_name' => $role_name,
            'description' => $description,
        ];
    }
}
```

FILE: database/factories/RolePermissionFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Role;
use App\Models\Permission;
use App\Models\RolePermission;
use Illuminate\Database\Eloquent\Factories\Factory;

class RolePermissionFactory extends Factory
{
    protected $model = RolePermission::class;

    public function definition()
    {
        return [
            'roles_id' => Role::factory(),         // Assuming a Role factory exists
            'permissions_id' => Permission::factory(), // Assuming a Permission factory exists
        ];
    }
}
```

FILE: database/factories/ShipmentFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Shipment;
use App\Models\Provider;
use App\Models\Order;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Str;
```

```
class ShipmentFactory extends Factory
{
    protected $model = Shipment::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'status' => $this->faker->randomElement(['Pending', 'Shipped', 'In Transit', 'Delivered', 'Returned']), // Random shipment
status
            'estimated_arrival' => $this->faker->dateTimeBetween('+1 week', '+1 month'), // Estimated arrival date (within the next month)
            'actual_arrival' => $this->faker->dateTimeBetween('+1 week', '+1 month'), // Actual arrival date (within the next month)
            'origin' => $this->faker->city, // Random origin city
            'destination' => $this->faker->city, // Random destination city
            'shipment_method' => $this->faker->randomElement(['Air', 'Sea', 'Land', 'Courier']), // Random shipment method
            'tracking_number' => $this->faker->regexify('[A-Z0-9]{10}'), // Random tracking number (e.g., ABC1234567)
            'last_updated' => $this->faker->dateTimeThisMonth(), // Last update date within the current month
            'total_amount' => $this->faker->randomFloat(2, 10, 500), // Total shipment cost (random between 10 and 500)
            'providers_id' => Provider::factory(), // Associate a provider using the Provider factory
            'orders_id' => Order::factory(), // Associate an order using the Order factory
        ];
    }

    /**
     * Indicate that the shipment is delivered.
     *
     * @return \Illuminate\Database\Eloquent\Factories\Factory
     */
    public function delivered(): static
    {
        return $this->state(fn (array $attributes) => [
            'status' => 'Delivered',
            'actual_arrival' => now(), // Set actual arrival to now
        ]);
    }

    /**
     * Indicate that the shipment is in transit.
     *
     * @return \Illuminate\Database\Eloquent\Factories\Factory
     */
    public function inTransit(): static
    {
        return $this->state(fn (array $attributes) => [
            'status' => 'In Transit',
            'actual_arrival' => null, // No actual arrival yet
        ]);
    }

    /**
     * Indicate that the shipment is pending.
     *
     * @return \Illuminate\Database\Eloquent\Factories\Factory
     */
    public function pending(): static
    {
        return $this->state(fn (array $attributes) => [
            'status' => 'Pending',
            'actual_arrival' => null, // No actual arrival yet
        ]);
    }
```

```
}
```

FILE: database/factories/UserAddressFactory.php

```
<?php
namespace Database\Factories;

use App\Models\UserAddress;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class UserAddressFactory extends Factory
{
    /**
     * The model that the factory is generating.
     */
    protected $model = UserAddress::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        // Predefined lists of real data for various address components
        $cities = ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix', 'Philadelphia'];
        $states = ['California', 'Texas', 'New York', 'Florida', 'Illinois', 'Pennsylvania'];
        $postalCodes = ['10001', '90001', '60601', '77001', '85001', '19102'];
        $countries = ['United States', 'Canada', 'United Kingdom', 'Australia', 'Germany', 'France'];

        // Randomly pick values from the predefined lists
        $city = $cities[array_rand($cities)];
        $state = $states[array_rand($states)];
        $postalCode = $postalCodes[array_rand($postalCodes)];
        $country = $countries[array_rand($countries)];

        // Randomly generate address line 1 and address line 2 (optional)
        $addressLine1 = $this->faker->buildingNumber() . ' ' . $this->faker->streetName();
        $addressLine2 = $this->faker->optional()->secondaryAddress();

        // Randomly decide whether the address is primary
        $isPrimary = $this->faker->boolean();

        // Get a random user to associate with the address
        $userId = User::inRandomOrder()->first()->id;

        return [
            'users_id' => $userId,           // Foreign key to the user
            'address_line1' => $addressLine1, // Address line 1
            'address_line2' => $addressLine2, // Address line 2 (optional)
            'city' => $city,                  // City
            'state' => $state,                // State
            'postal_code' => $postalCode,     // Postal code
            'country' => $country,            // Country
            'is_primary' => $isPrimary,       // Whether the address is primary for the user
        ];
    }
}
```

FILE: database/factories/UserConnectionFactory.php

```
<?php

namespace Database\Factories;

use App\Models\UserConnection;
use Illuminate\Database\Eloquent\Factories\Factory;
use App\Models\User;
class UserConnectionFactory extends Factory
{
    protected $model = UserConnection::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition()
    {
        return [
            'user_id' => User::factory(), // Create a new user for user_id
            'connection_id' => User::factory(), // Create a new user for connection_id
        ];
    }
}
```

FILE: database/factories/UserCouponFactory.php

```
<?php

namespace Database\Factories;

use App\Models\User;
use App\Models\Coupon;
use App\Models\UserCoupon;
use Illuminate\Database\Eloquent\Factories\Factory;

class UserCouponFactory extends Factory
{
    protected $model = UserCoupon::class;

    public function definition()
    {
        return [
            'users_id' => User::factory(),    // Assuming a User factory exists
            'coupons_id' => Coupon::factory(), // Assuming a Coupon factory exists
        ];
    }
}
```

FILE: database/factories/UserFactory.php

```
<?php

namespace Database\Factories;

use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Str;
```

```
/**
 * @extends \Illuminate\Database\Eloquent\Factories\Factory<\App\Models\User>
 */
class UserFactory extends Factory
{
    /**
     * The current password being used by the factory.
     */
    protected static ?string $password;

    // Create a list of image URLs
    private static $imageUrls = [
        'https://www.cnet.com/a/img/resize/11aa4eb14e5479a9569e7cf4d887f11ab0648512/hub/2020/05/18/5bac8cc1-4bd5-4496-a8c3-
66a6cd12d0cb/fb-avatar-2.jpg?auto=webp&width=768',
        'https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcThTDi9zqkRyyg3e9LJGPC5lTZqB72CMSlw8g&s',
        'https://api.duniagames.co.id/api/content/upload/file/8143860661599124172.jpg',
        'https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcR300hWAIiV7V5uxmHX5_Hvte-iABlC_tV9fw&s',
        'https://encrypted-tbn0.gstatic.com/images?
q=tbn:ANd9GcQSPmYCAnwPyxf8ySZQVO2moLKoDrDJBkKFRIROH6cjcIDEOJFLufZzjk9n805XvW9weLs&usqp=CAU',
    ];

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'first_name' => $this->faker->firstName(),
            'last_name' => $this->faker->lastName(),
            'phone_number' => '+65 8916 4359',
            'ip_address' => $this->faker->ipv4(),
            'email' => $this->faker->unique()->safeEmail(),
            'password' => static::$password ??= Hash::make('password'),
            'two_factor_enabled' => $this->faker->boolean(),
            'last_login' => $this->faker->dateTimeThisYear(),
            'language' => $this->faker->languageCode(),
            'is_admin' => $this->faker->boolean(),
            'last_password_change' => $this->faker->dateTimeThisYear(),
            'profile_picture' => $this->getRandomImage(),  // Random image URL
            'banner_img' => $this->getRandomImage(),  // Random image URL
            'headline' => $this->faker->sentence(),
            'about' => $this->faker->paragraph(),
        ];
    }

    /**
     * Get a random image URL from the collection.
     *
     * @return string
     */
    private function getRandomImage(): string
    {
        // Get a random key from the image URL collection and return the image URL
        return self::$imageUrls[array_rand(self::$imageUrls)];
    }

    /**
     * Indicate that the user is an admin.
     */
    public function admin(): static
    {
        return $this->state(fn (array $attributes) => [
            'is_admin' => true,
        ]);
```

```
    }

    /**
     * Indicate that the user has two-factor authentication enabled.
     */
    public function withTwoFactor(): static
    {
        return $this->state(fn (array $attributes) => [
            'two_factor_enabled' => true,
        ]);
    }

    /**
     * Add connections to the user.
     *
     * @param int $userCount
     * @return \Illuminate\Database\Eloquent\Factories\Factory
     */
    public function withConnections($userCount = 2)
    {
        return $this->afterCreating(function ($user) use ($userCount) {
            $connections = \App\Models\User::inRandomOrder()->take($userCount)->pluck('id')->toArray();
            $user->addConnection($connections); // Add connections
        });
    }
}
```

FILE: database/factories/UserRoleFactory.php

```
<?php

namespace Database\Factories;

use App\Models\User;
use App\Models\Role;
use App\Models\UserRole;
use Illuminate\Database\Eloquent\Factories\Factory;

class UserRoleFactory extends Factory
{
    protected $model = UserRole::class;

    public function definition()
    {
        return [
            'users_id' => User::factory(),  // Assuming a User factory exists
            'roles_id' => Role::factory(),  // Assuming a Role factory exists
        ];
    }
}
```

FILE: database/factories/VehicleFactory.php

```
<?php

namespace Database\Factories;

use App\Models\Vehicle;
use App\Models\User;
use App\Models\VehicleManagement;
use Illuminate\Database\Eloquent\Factories\Factory;
```

```
use Illuminate\Support\Str;

class VehicleFactory extends Factory
{
    protected $model = Vehicle::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'license_plate' => $this->faker->regexify('[A-Z]{3}-[0-9]{3}'), // Random license plate (e.g., ABC-123)
            'type' => $this->faker->randomElement(['Sedan', 'SUV', 'Truck', 'Van', 'Motorcycle']), // Random vehicle type
            'driver_id' => User::factory(), // Associate a user (driver)
            'capacity' => $this->faker->numberBetween(2, 20), // Random passenger capacity between 2 and 20
            'fuel_capacity' => $this->faker->numberBetween(30, 200), // Random fuel capacity between 30 and 200 liters
            'current_location' => $this->faker->address, // Random location
            'last_serviced' => $this->faker->dateTimeThisYear(), // Random last serviced date this year
            'status' => $this->faker->randomElement(['Active', 'Inactive', 'Under Maintenance']), // Random status
            'last_fuel_refill' => $this->faker->dateTimeThisYear(), // Random date of last fuel refill this year
            'last_location_update' => $this->faker->dateTimeThisYear(), // Random date for last location update
            'mileage' => $this->faker->numberBetween(1000, 200000), // Random mileage (between 1,000 and 200,000 km)
            'maintenance_logs' => $this->faker->text(100), // Random maintenance logs (brief text)
            'vehicle_management_id' => VehicleManagement::factory(), // Associate a vehicle management record

            // New fields
            'fuel_interval' => $this->faker->numberBetween(0, 1000), // Random fuel interval (e.g., between 0 and 1000 km)
            'fuel_type' => $this->faker->randomElement(['Petrol', 'Diesel', 'Electric', 'Hybrid']), // Random fuel type
            'vin' => $this->faker->bothify('??###??#####????'), // Random VIN (Vehicle Identification Number)

            // New fields for brand, model, and year of manufacture
            'brand' => $this->faker->company, // Random brand (could be any company name)
            'model' => $this->faker->word, // Random model (random word)
            'year_of_manufacture' => $this->faker->year(), // Random year of manufacture (current year or a random one)
        ];
    }

    /**
     * Indicate that the vehicle is under maintenance.
     *
     * @return \Illuminate\Database\Eloquent\Factories\Factory
     */
    public function underMaintenance(): static
    {
        return $this->state(fn (array $attributes) => [
            'status' => 'Under Maintenance',
            'last_serviced' => null, // No last serviced date
            'maintenance_logs' => 'Vehicle is under maintenance.',
            'fuel_interval' => 0, // Optionally set to 0 when under maintenance
            'fuel_type' => $this->faker->randomElement(['Petrol', 'Diesel', 'Electric', 'Hybrid']), // Random fuel type
            'vin' => $this->faker->bothify('??###??#####????'), // Random VIN

            // New fields for brand, model, and year of manufacture
            'brand' => $this->faker->company, // Random brand
            'model' => $this->faker->word, // Random model
            'year_of_manufacture' => $this->faker->year(), // Random year of manufacture
        ]);
    }

    /**
     * Indicate that the vehicle is active.
     *
     * @return \Illuminate\Database\Eloquent\Factories\Factory
```

```
     */
    public function active(): static
    {
        return $this->state(fn (array $attributes) => [
            'status' => 'Active',
            'last_serviced' => $this->faker->dateTimeThisYear(),
            'maintenance_logs' => 'Vehicle is in good condition.',
            'fuel_interval' => $this->faker->numberBetween(0, 1000), // Random fuel interval
            'fuel_type' => $this->faker->randomElement(['Petrol', 'Diesel', 'Electric', 'Hybrid']), // Random fuel type
            'vin' => $this->faker->bothify('??###??#####????'), // Random VIN

            // New fields for brand, model, and year of manufacture
            'brand' => $this->faker->company, // Random brand
            'model' => $this->faker->word, // Random model
            'year_of_manufacture' => $this->faker->year(), // Random year of manufacture
        ]);
    }
}
```

FILE: database/factories/VehicleManagementFactory.php

```
<?php

namespace Database\Factories;

use App\Models\VehicleManagement;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Str;

class VehicleManagementFactory extends Factory
{
    protected $model = VehicleManagement::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'fuel_consumption' => $this->faker->numberBetween(5, 20), // Random fuel consumption (L per 100km)
            'distance_traveled' => $this->faker->numberBetween(1000, 100000), // Random distance traveled in km
            'maintenance_status' => $this->faker->randomElement(['Pending', 'Completed', 'In Progress']), // Random maintenance status
            'last_maintenance_date' => $this->faker->dateTimeThisYear(), // Random date for last maintenance
            'maintenance_schedule' => $this->faker->dateTimeBetween('+1 week', '+1 year'), // Random next maintenance date
            'maintenance_cost' => $this->faker->numberBetween(5, 20), // Random fuel consumption (L per 100km)
            'users_id' => User::factory(), // Associate a user with the vehicle (create a new user if needed)
        ];
    }

    /**
     * Indicate that the vehicle's maintenance is completed.
     *
     * @return \Illuminate\Database\Eloquent\Factories\Factory
     */
    public function maintenanceCompleted(): static
    {
        return $this->state(fn (array $attributes) => [
            'maintenance_status' => 'Completed',
            'last_maintenance_date' => now(), // Set last maintenance date to now
            'maintenance_schedule' => $this->faker->dateTimeBetween('+1 week', '+1 year'), // Random next maintenance date
        ]);
```

```
    }

    /**
     * Indicate that the vehicle's maintenance is pending.
     *
     * @return \Illuminate\Database\Eloquent\Factories\Factory
     */
    public function maintenancePending(): static
    {
        return $this->state(fn (array $attributes) => [
            'maintenance_status' => 'Pending',
            'last_maintenance_date' => null, // No maintenance yet
            'maintenance_schedule' => $this->faker->dateTimeBetween('+1 week', '+1 month'), // Random future maintenance schedule
        ]);
    }

    /**
     * Indicate that the vehicle's maintenance is in progress.
     *
     * @return \Illuminate\Database\Eloquent\Factories\Factory
     */
    public function maintenanceInProgress(): static
    {
        return $this->state(fn (array $attributes) => [
            'maintenance_status' => 'In Progress',
            'last_maintenance_date' => $this->faker->dateTimeThisMonth(), // Last maintenance within this month
            'maintenance_schedule' => $this->faker->dateTimeBetween('+1 week', '+1 month'), // Random future maintenance schedule
        ]);
    }
}
```

FILE: database/factories/WarehouseFactory.php

```
<?php
namespace Database\Factories;

use App\Models\Warehouse;
use App\Models\User; // Import the User model to associate a user with the warehouse
use Illuminate\Database\Eloquent\Factories\Factory;

class WarehouseFactory extends Factory
{
    protected $model = Warehouse::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        // Predefined list of real warehouse names with unique addresses
        $warehouseAddresses = [
            'Amazon Fulfillment Center' => '2001 7th Ave, Seattle, WA, USA',
            'Walmart Distribution Center' => '8500 W Markham St, Little Rock, AR, USA',
            'FedEx Warehouse' => '123 Main St, Memphis, TN, USA',
            'UPS Supply Chain Solutions' => '1 UPS Way, Louisville, KY, USA',
            'Home Depot Distribution Hub' => '240 Peachtree St, Atlanta, GA, USA',
            'Target Warehouse' => '1000 Nicollet Mall, Minneapolis, MN, USA',
            'Costco Distribution Center' => '999 3rd Ave, Issaquah, WA, USA',
            'Kroger Supply Chain' => '1014 Vine St, Cincinnati, OH, USA',
            'Best Buy Warehouse' => '7601 Penn Ave S, Richfield, MN, USA',
            'Lidl Logistics Hub' => '1 Lidl Blvd, Arlington, VA, USA'
        ];
```

```
        // Predefined list of real countries and cities
        // Predefined list of real cities, states, and country codes (ISO3)
        $countriesISO3 = [
            'USA' => 'United States', 'CAN' => 'Canada', 'GBR' => 'United Kingdom',
            'DEU' => 'Germany', 'FRA' => 'France', 'AUS' => 'Australia', 'IND' => 'India',
            'BRA' => 'Brazil', 'CHN' => 'China', 'JPN' => 'Japan', 'MEX' => 'Mexico',
            'ITA' => 'Italy', 'ESP' => 'Spain', 'RUS' => 'Russia', 'ZAF' => 'South Africa',
            'ARG' => 'Argentina', 'KOR' => 'South Korea', 'SAU' => 'Saudi Arabia',
            'SWE' => 'Sweden', 'NLD' => 'Netherlands', 'BEL' => 'Belgium', 'CHE' => 'Switzerland',
            'DNK' => 'Denmark', 'NOR' => 'Norway', 'FIN' => 'Finland', 'POL' => 'Poland',
            'TUR' => 'Turkey', 'EGY' => 'Egypt', 'ARE' => 'United Arab Emirates', 'GRC' => 'Greece',
            'SGP' => 'Singapore', 'CHL' => 'Chile', 'PER' => 'Peru', 'KSA' => 'Saudi Arabia',
            'IDN' => 'Indonesia', 'PHL' => 'Philippines', 'MYS' => 'Malaysia', 'NZL' => 'New Zealand'
        ];

        $cities = [
            'USA' => ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix', 'San Francisco', 'Boston', 'Dallas', 'Seattle',
'Miami'],
            'CAN' => ['Toronto', 'Vancouver', 'Montreal', 'Ottawa', 'Calgary', 'Edmonton', 'Quebec City', 'Winnipeg', 'Hamilton',
'Kitchener'],
            'GBR' => ['London', 'Manchester', 'Birmingham', 'Glasgow', 'Liverpool', 'Edinburgh', 'Bristol', 'Sheffield', 'Leeds',
'Cardiff'],
            'DEU' => ['Berlin', 'Munich', 'Hamburg', 'Cologne', 'Frankfurt', 'Stuttgart', 'Dusseldorf', 'Dortmund', 'Leipzig', 'Essen'],
            'FRA' => ['Paris', 'Marseille', 'Lyon', 'Toulouse', 'Nice', 'Nantes', 'Montpellier', 'Strasbourg', 'Bordeaux', 'Lille'],
            'AUS' => ['Sydney', 'Melbourne', 'Brisbane', 'Perth', 'Adelaide', 'Gold Coast', 'Canberra', 'Hobart', 'Darwin', 'Newcastle'],
            'IND' => ['New Delhi', 'Mumbai', 'Kolkata', 'Bangalore', 'Hyderabad', 'Chennai', 'Ahmedabad', 'Pune', 'Jaipur', 'Lucknow'],
            'BRA' => ['São Paulo', 'Rio de Janeiro', 'Brasília', 'Salvador', 'Fortaleza', 'Belo Horizonte', 'Manaus', 'Curitiba',
'Recife', 'Porto Alegre'],
            'CHN' => ['Beijing', 'Shanghai', 'Guangzhou', 'Shenzhen', 'Chengdu', 'Hangzhou', 'Xi'an', 'Nanjing', 'Tianjin', 'Wuhan'],
            'JPN' => ['Tokyo', 'Osaka', 'Yokohama', 'Nagoya', 'Sapporo', 'Fukuoka', 'Kobe', 'Kyoto', 'Kawasaki', 'Saitama'],
            'MEX' => ['Mexico City', 'Guadalajara', 'Monterrey', 'Puebla', 'Cancún', 'Tijuana', 'Mérida', 'León', 'San Luis Potosí',
'Querétaro'],
            'ITA' => ['Rome', 'Milan', 'Naples', 'Turin', 'Palermo', 'Genoa', 'Bologna', 'Florence', 'Venice', 'Catania'],
            'ESP' => ['Madrid', 'Barcelona', 'Valencia', 'Sevilla', 'Zaragoza', 'Malaga', 'Murcia', 'Palma', 'Las Palmas', 'Bilbao'],
            'RUS' => ['Moscow', 'Saint Petersburg', 'Novosibirsk', 'Yekaterinburg', 'Nizhny Novgorod', 'Samara', 'Omsk', 'Kazan',
'Chelyabinsk', 'Rostov-on-Don'],
            'ZAF' => ['Johannesburg', 'Cape Town', 'Durban', 'Pretoria', 'Port Elizabeth', 'Bloemfontein', 'East London', 'Polokwane',
'Nelspruit', 'Kimberley'],
            'ARG' => ['Buenos Aires', 'Córdoba', 'Rosario', 'Mendoza', 'La Plata', 'San Miguel de Tucumán', 'Mar del Plata', 'Salta',
'Santa Fe', 'San Juan'],
            'KOR' => ['Seoul', 'Busan', 'Incheon', 'Daegu', 'Daejeon', 'Gwangju', 'Ulsan', 'Suwon', 'Changwon', 'Seongnam'],
            'SAU' => ['Riyadh', 'Jeddah', 'Mecca', 'Medina', 'Khobar', 'Dammam', 'Tabuk', 'Abha', 'Al Khobar', 'Najran'],
            'SWE' => ['Stockholm', 'Gothenburg', 'Malmö', 'Uppsala', 'Västerås', 'Örebro', 'Linköping', 'Helsingborg', 'Jönköping',
'Norrköping'],
            'NLD' => ['Amsterdam', 'Rotterdam', 'The Hague', 'Utrecht', 'Eindhoven', 'Groningen', 'Almere', 'Breda', 'Leiden',
'Hilversum'],
            'BEL' => ['Brussels', 'Antwerp', 'Ghent', 'Charleroi', 'Liège', 'Bruges', 'Leuven', 'Namur', 'Mons', 'Kortrijk'],
            'CHE' => ['Zurich', 'Geneva', 'Basel', 'Bern', 'Lucerne', 'Lausanne', 'St. Moritz', 'Zermatt', 'Interlaken', 'Montreux'],
            'DNK' => ['Copenhagen', 'Aarhus', 'Odense', 'Aalborg', 'Esbjerg', 'Randers', 'Kolding', 'Horsens', 'Vejle', 'Herning'],
            'NOR' => ['Oslo', 'Bergen', 'Stavanger', 'Drammen', 'Dundas', 'Trondheim', 'Bodø', 'Tromsø', 'Fredrikstad', 'Porsgrunn'],
            'FIN' => ['Helsinki', 'Espoo', 'Tampere', 'Vantaa', 'Oulu', 'Turku', 'Jyväskylä', 'Lahti', 'Kuopio', 'Pori'],
            'POL' => ['Warsaw', 'Kraków', 'Łódź', 'Wrocław', 'Poznań', 'Gdańsk', 'Szczecin', 'Bydgoszcz', 'Lublin', 'Katowice'],
            'TUR' => ['Istanbul', 'Ankara', 'Izmir', 'Bursa', 'Adana', 'Antalya', 'Konya', 'Gaziantep', 'Mersin', 'Diyarbakır'],
            'EGY' => ['Cairo', 'Alexandria', 'Giza', 'Shubra El Kheima', 'Port Said', 'Suez', 'Luxor', 'Aswan', 'Mansoura', 'Tanta'],
            'ARE' => ['Abu Dhabi', 'Dubai', 'Sharjah', 'Al Ain', 'Ajman', 'Umm Al-Quwain', 'Fujairah', 'Ras Al Khaimah'],
            'GRC' => ['Athens', 'Thessaloniki', 'Patras', 'Heraklion', 'Larissa', 'Volos', 'Ioannina', 'Chania', 'Rhodes', 'Kavala'],
            'SGP' => ['Singapore'],
            'CHL' => ['Santiago', 'Valparaíso', 'Concepción', 'La Serena', 'Antofagasta', 'Temuco', 'Iquique', 'Rancagua', 'Arica',
'Talca'],
            'PER' => ['Lima', 'Arequipa', 'Trujillo', 'Chiclayo', 'Piura', 'Cusco', 'Iquitos', 'Tacna', 'Huancayo', 'Chimbote'],
            'KSA' => ['Riyadh', 'Jeddah', 'Mecca', 'Medina', 'Khobar', 'Dammam', 'Tabuk', 'Abha', 'Al Khobar', 'Najran'],
            'IDN' => ['Jakarta', 'Surabaya', 'Bandung', 'Medan', 'Bekasi', 'Semarang', 'Tangerang', 'Makassar', 'Yogyakarta',
'Palembang'],
            'PHL' => ['Manila', 'Quezon City', 'Cebu City', 'Davao City', 'Makati', 'Cagayan de Oro', 'Taguig', 'Zamboanga City', 'Iloilo
City', 'Bacolod'],
```

```
            'MYS' => ['Kuala Lumpur', 'George Town', 'Johor Bahru', 'Ipoh', 'Kota Kinabalu', 'Melaka', 'Shah Alam', 'Alor Setar',
'Kuching', 'Miri'],
            'NZL' => ['Auckland', 'Wellington', 'Christchurch', 'Hamilton', 'Dunedin', 'Tauranga', 'Napier-Hastings', 'Palmerston North',
'Rotorua', 'Whangarei'],
        ];
        // Ensure that the warehouse names are unique by using array_keys to get the names
        $warehouseNames = array_keys($warehouseAddresses);

        // Randomly select a unique warehouse name and its corresponding unique address
        $warehouseName = array_shift($warehouseNames); // Get the first available name (shift ensures uniqueness)
        $location = $warehouseAddresses[$warehouseName]; // Get the corresponding address

        // Define realistic capacity and available space ranges
        $capacity = rand(1000, 10000);  // Warehouse capacity (1,000 to 10,000 units)
        $availableSpace = rand(0, $capacity);  // Available space within that range

        return [
            'warehouse_name' => $warehouseName, // Unique warehouse name
            'location' => $location, // Unique address for each warehouse name
            'capacity' => $capacity, // Random but realistic capacity
            'available_space' => $availableSpace, // Random available space
            'users_id' => User::factory(), // Associate a user with the warehouse (create a new user if needed)
        ];
    }
}
```

FILE: database/migrations/0001_01_01_000000_create_users_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        // Create the 'users' table
        Schema::create('users', function (Blueprint $table) {
            $table->id(); // Automatically creates 'Id' as an auto-incrementing integer
            $table->string('first_name', 100)->nullable()->default(null);
            $table->string('last_name', 100)->nullable()->default(null);
            $table->string('phone_number', 50)->nullable()->default(null);
            $table->string('ip_address', 500)->nullable()->default(null);
            $table->string('email', 255)->nullable()->default(null)->unique();
            $table->string('password', 255)->nullable()->default(null);
            $table->boolean('two_factor_enabled')->default(false);
            $table->timestamp('last_login')->nullable()->default(null);
            $table->string('language', 20)->nullable()->default(null);
            $table->boolean('is_admin')->default(false);
            $table->timestamps(); // CreatedAt and UpdatedAt
            $table->timestamp('last_password_change')->nullable()->default(null);
            $table->string('profile_picture',1000)->default('');
            $table->string('banner_img',1000)->default('');
            $table->string('headline')->default('Linkedin User');
            $table->text('about')->nullable();
            $table->string('two_factor_code')->nullable();
            $table->timestamp('two_factor_code_created_at')->nullable();
        });
```

```
        // Create 'password_reset_tokens' table
        Schema::create('password_reset_tokens', function (Blueprint $table) {
            $table->string('email')->primary(); // 'email' as the primary key
            $table->string('token');
            $table->timestamp('created_at')->nullable();
        });

        // Create 'sessions' table
        Schema::create('sessions', function (Blueprint $table) {
            $table->string('id')->primary(); // 'id' as the primary key for session table
            $table->foreignId('user_id')->nullable()->constrained()->index(); // Foreign key for user_id, referencing 'users' table
            $table->string('ip_address', 45)->nullable(); // To store IPv6 addresses
            $table->text('user_agent')->nullable(); // To store the user's browser info
            $table->longText('payload'); // To store session payload data
            $table->integer('last_activity')->index(); // To store the timestamp of the last activity
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users');
        Schema::dropIfExists('password_reset_tokens');
        Schema::dropIfExists('sessions');
    }
};
```

FILE: database/migrations/0001_01_01_000001_create_cache_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('cache', function (Blueprint $table) {
            $table->string('key')->primary();
            $table->mediumText('value');
            $table->integer('expiration');
        });

        Schema::create('cache_locks', function (Blueprint $table) {
            $table->string('key')->primary();
            $table->string('owner');
            $table->integer('expiration');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('cache');
        Schema::dropIfExists('cache_locks');
```

```
    }
};
```

FILE: database/migrations/0001_01_01_000002_create_jobs_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('jobs', function (Blueprint $table) {
            $table->id();
            $table->string('queue')->index();
            $table->longText('payload');
            $table->unsignedTinyInteger('attempts');
            $table->unsignedInteger('reserved_at')->nullable();
            $table->unsignedInteger('available_at');
            $table->unsignedInteger('created_at');
        });

        Schema::create('job_batches', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->string('name');
            $table->integer('total_jobs');
            $table->integer('pending_jobs');
            $table->integer('failed_jobs');
            $table->longText('failed_job_ids');
            $table->mediumText('options')->nullable();
            $table->integer('cancelled_at')->nullable();
            $table->integer('created_at');
            $table->integer('finished_at')->nullable();
        });

        Schema::create('failed_jobs', function (Blueprint $table) {
            $table->id();
            $table->string('uuid')->unique();
            $table->text('connection');
            $table->text('queue');
            $table->longText('payload');
            $table->longText('exception');
            $table->timestamp('failed_at')->useCurrent();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('jobs');
        Schema::dropIfExists('job_batches');
        Schema::dropIfExists('failed_jobs');
    }
};
```

FILE: database/migrations/2022_11_21_170127_create_applications_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('feedback_forms', function (Blueprint $table) {
            $table->id();
            $table->foreignIdFor(\App\Models\User::class, 'user_id');
            $table->foreignIdFor(\App\Models\Order::class, 'order_id')->nullable(); // Make order_id nullable
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::dropIfExists('feedback_forms');
    }
};
```

FILE: database/migrations/2022_11_21_170521_create_application_questions_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('feedback_form_questions', function (Blueprint $table) {
            $table->id();
            $table->string('type', 45);
            $table->string('question',2000);
            $table->longText('description')->nullable();
            $table->longText('data')->nullable();
            $table->foreignIdFor(\App\Models\FeedbackForm::class, 'feedback_form_id');
            $table->timestamps();
```

```
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::dropIfExists('feedback_form_questions');
    }
};
```

FILE: database/migrations/2022_11_21_170534_create_application_answers_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('feedback_form_answers', function (Blueprint $table) {
            $table->id();
            $table->foreignIdFor(\App\Models\FeedbackForm::class, 'feedback_form_id');
            $table->timestamp('start_date')->nullable();
            $table->timestamp('end_date')->nullable();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::dropIfExists('feedback_form_answers');
    }
};
```

FILE: database/migrations/2022_11_21_170552_create_application_question_answers_tab

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
```

```
return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('feedback_form_question_answers', function (Blueprint $table) {
            $table->id();
            $table->foreignIdFor(\App\Models\FeedbackFormQuestion::class, 'feedback_form_question_id');
            $table->foreignIdFor(\App\Models\FeedbackFormAnswer::class, 'feedback_form_answer_id');
            $table->text('answer');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::dropIfExists('feedback_form_question_answers');
    }
};
```

FILE: database/migrations/2024_01_16_114923_create_route_conditions_table.php

```
<?php
// database/migrations/xxxx_xx_xx_xxxxxx_create_route_conditions_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up()
    {
        Schema::create('route_conditions', function (Blueprint $table) {
            $table->id();  // Primary key
            $table->enum('weather', ['Clear', 'Rainy', 'Snowy', 'Foggy', 'Windy', 'Stormy'])->default('Clear');  // Weather conditions
            $table->enum('road_condition', ['Dry', 'Wet', 'Under Construction', 'Icy', 'Flooded'])->default('Dry');  // Road conditions
            $table->enum('traffic_condition', ['Light', 'Moderate', 'Heavy'])->default('Light');  // Traffic conditions
            $table->boolean('has_accident')->default(false);  // If there's an accident on the route
            $table->text('accident_description')->nullable();  // Detailed description of the accident, if any
            $table->text('road_closure_description')->nullable();  // Description of any road closure, if applicable
            $table->timestamp('reported_at')->useCurrent();  // Timestamp when the condition was reported
            $table->timestamps();  // Automatically manage created_at and updated_at

            // Optionally, you could add foreign key relationships if you want to link to other tables.
            // For example, if you wanted to associate route conditions with specific routes:
            // $table->unsignedBigInteger('route_id')->nullable();
            // $table->foreign('route_id')->references('id')->on('routes')->onDelete('set null');
        });
    }

    public function down()
    {
        Schema::dropIfExists('route_conditions');
    }
```

```
};
```

FILE: database/migrations/2024_04_13_093225_create_groups_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('groups', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->longText('description')->nullable();
            $table->foreignId('owner_id')->constrained('users')->onDelete('cascade');
            $table->timestamps();
        });

        Schema::create('group_users', function (Blueprint $table) {
            $table->id();
            $table->foreignId('group_id')->constrained('groups')->onDelete('cascade');
            $table->foreignId('user_id')->constrained('users')->onDelete('cascade');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('group_users');
        Schema::dropIfExists('groups');
    }
};
```

FILE: database/migrations/2024_04_13_093235_create_conversations_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('conversations', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id1')->constrained('users');
            $table->foreignId('user_id2')->constrained('users');
            $table->timestamps();
```

```
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('conversations');
    }
};
```

FILE: database/migrations/2024_04_13_093308_create_messages_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('messages', function (Blueprint $table) {
            $table->id();
            $table->longText('message')->nullable();
            $table->foreignId('sender_id')->constrained('users');
            $table->foreignId('receiver_id')->nullable()->constrained('users');
            $table->foreignId('group_id')->nullable()->constrained('groups');
            $table->foreignId('conversation_id')->nullable()->constrained('conversations');
            $table->timestamps();
        });

        Schema::table('groups', function (Blueprint $table) {
            $table->foreignId('last_message_id')->nullable()->constrained('messages');
        });

        Schema::table('conversations', function (Blueprint $table) {
            $table->foreignId('last_message_id')->nullable()->constrained('messages');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('messages');
    }
};
```

FILE: database/migrations/2024_04_13_093313_create_message_attachments_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
```

```
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('message_attachments', function (Blueprint $table) {
            $table->id();
            $table->foreignId('message_id')->constrained('messages');
            $table->string('name', 255); // test.png
            $table->string('path', 1024); //
            $table->string('mime', 255);  // image/png
            $table->integer('size');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('message_attachments');
    }
};
```

FILE: database/migrations/2024_10_31_160146_create_posts_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up()
    {
        Schema::create('posts', function (Blueprint $table) {
            $table->id();
            $table->foreignId('author_id')->constrained('users')->onDelete('cascade'); // Foreign key for author
            $table->text('content')->nullable();
            $table->string('image',1000)->nullable();
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('posts');
    }
};
```

FILE: database/migrations/2024_10_31_160320_create_notifications_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
```

```
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up()
    {
        Schema::create('notifications', function (Blueprint $table) {
            $table->id();
            $table->foreignId('recipient_id')->constrained('users')->onDelete('cascade'); // Foreign key for recipient
            $table->string('type'); // Changed from enum to string
            $table->foreignId('related_user')->nullable()->constrained('users')->onDelete('set null'); // Foreign key for related user
            $table->foreignId('related_post')->nullable()->constrained('posts')->onDelete('set null'); // Foreign key for related post
            $table->boolean('read')->default(false);
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('notifications');
    }
};
```

FILE: database/migrations/2024_10_31_160454_create_connection_requests_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up()
    {
        Schema::create('connection_requests', function (Blueprint $table) {
            $table->id();
            $table->foreignId('sender_id')->constrained('users')->onDelete('cascade'); // Foreign key for sender
            $table->foreignId('recipient_id')->constrained('users')->onDelete('cascade'); // Foreign key for recipient
            $table->enum('status', ['pending', 'accepted', 'rejected'])->default('pending'); // Correctly define enum column
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('connection_requests');
    }
};
```

FILE: database/migrations/2024_11_02_131647_create_comments_table.php

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
```

```
{
    public function up()
    {
        Schema::create('comments', function (Blueprint $table) {
            $table->id();
            $table->foreignId('post_id')->constrained()->onDelete('cascade'); // Foreign key to posts
            $table->foreignId('user_id')->constrained()->onDelete('cascade'); // Foreign key to users
            $table->text('content');
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('comments');
    }
};```

## FILE: database/migrations/2024_11_02_133312_create_likes_table.php
```php
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up()
    {
        Schema::create('likes', function (Blueprint $table) {
            $table->id();
            $table->foreignId('post_id')->constrained()->onDelete('cascade'); // Foreign key to posts
            $table->foreignId('user_id')->constrained()->onDelete('cascade'); // Foreign key to users
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('likes');
    }
};
```

FILE: database/migrations/2024_11_03_122501_create_user_connections_table.php

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up()
    {
        Schema::create('user_connections', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->foreignId('connection_id')->constrained('users')->onDelete('cascade');
            $table->timestamps();

            // Ensure that a user cannot connect to the same user more than once
            $table->unique(['user_id', 'connection_id']);
        });
    }
```
```

```
    public function down()
    {
        Schema::dropIfExists('user_connections');
    }
};```

## FILE: database/migrations/2024_12_19_071415_create_personal_access_tokens_table.php
```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('personal_access_tokens', function (Blueprint $table) {
            $table->id();
            $table->morphs('tokenable');
            $table->string('name');
            $table->string('token', 64)->unique();
            $table->text('abilities')->nullable();
            $table->timestamp('last_used_at')->nullable();
            $table->timestamp('expires_at')->nullable();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('personal_access_tokens');
    }
};
```

FILE: database/migrations/2024_12_21_062833_create_vehicle_management_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('vehicle_management', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->decimal('fuel_consumption', 18, 2)->nullable()->default(null);
            $table->decimal('distance_traveled', 18, 2)->nullable()->default(null);
```
```

```
            $table->string('maintenance_status', 100)->nullable()->default(null);
            $table->timestamp('last_maintenance_date')->nullable()->default(null);
            $table->timestamp('maintenance_schedule')->nullable()->default(null);
            $table->decimal('maintenance_cost', 18, 2)->nullable()->default(null);
            $table->foreignId('users_id')->constrained('users')->onDelete('restrict')->onUpdate('cascade'); // Foreign key for the user
            $table->timestamps(); // CreatedAt and UpdatedAt

            $table->index('users_id'); // Index for the users_id
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('vehicle_management');
    }
};
```

FILE: database/migrations/2024_12_21_063021_create_products_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->string('name', 100)->nullable()->default(null);
            $table->decimal('price', 18, 2)->nullable()->default(null);
            $table->string('description', 500)->nullable()->default(null);
            $table->foreignId('supplier_id')->nullable()->constrained('users')->nullOnDelete(); // Foreign key for the supplier
            $table->boolean('isFeatured')->default(false); // Add the isFeatured column, default to false
            $table->string('image',1000)->nullable(); // Add image column as a string (nullable to allow missing images)

            $table->timestamps(); // CreatedAt and UpdatedAt (DateAdded, DateUpdated)

            $table->index('supplier_id'); // Index for supplier_id
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('products');
    }
};
```

FILE: database/migrations/2024_12_21_063209_create_warehouses_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('warehouses', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->string('warehouse_name', 500)->nullable()->default(null);
            $table->string('location', 500)->nullable()->default(null);
            $table->decimal('capacity', 18, 2)->nullable()->default(null);
            $table->decimal('available_space', 18, 2)->nullable()->default(null);
            $table->foreignId('users_id')->constrained('users')->onDelete('restrict')->onUpdate('cascade'); // Foreign key for the user
            $table->timestamps(); // CreatedAt and UpdatedAt

            $table->index('users_id'); // Index for users_id
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('warehouses');
    }
};
```

FILE: database/migrations/2024_12_21_063343_create_inventory_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('inventory', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->integer('stock')->nullable()->default(null);
            $table->timestamp('last_updated')->nullable()->default(null);
            $table->decimal('weight_per_unit', 18, 2)->nullable()->default(null);
            $table->foreignId('products_id')->constrained('products')->onDelete('restrict')->onUpdate('cascade'); // Foreign key for the
product
            $table->foreignId('warehouses_id')->constrained('warehouses')->onDelete('restrict')->onUpdate('cascade'); // Foreign key for
the warehouse
            $table->timestamps(); // CreatedAt and UpdatedAt

            $table->index('products_id'); // Index for products_id
            $table->index('warehouses_id'); // Index for warehouses_id
        });
```

```
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('inventory');
    }
};
```

FILE: database/migrations/2024_12_21_063459_create_orders_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('orders', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->string('status', 30)->nullable()->default(null);
            $table->timestamp('order_date')->nullable()->default(null);
            $table->foreignId('user_id')->nullable()->constrained('users')->nullOnDelete(); // Foreign key for the user
            $table->string('tracking_number', 100)->nullable()->unique();
            $table->timestamps(); // CreatedAt and UpdatedAt (CreatedAt, UpdatedAt)
            $table->integer('total_amount')->nullable()->default(null);

            // Indexes
            $table->index('user_id'); // Index for user_id
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('orders');
    }
};
```

FILE: database/migrations/2024_12_21_063602_create_providers_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
```

```
     */
    public function up(): void
    {
        Schema::create('providers', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->string('type', 45)->nullable()->default(null); // Type of the provider
            $table->string('terms_of_service', 500)->nullable()->default(null); // Terms of service
            $table->timestamps(); // CreatedAt and UpdatedAt (CreatedAt, UpdatedAt)

        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('providers');
    }
};
```

FILE: database/migrations/2024_12_21_063734_create_payments_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('payments', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->decimal('total_amount', 18, 2)->nullable()->default(null); // Total amount of the payment
            $table->decimal('paid_amount', 18, 2)->nullable()->default(0.00); // Amount already paid
            $table->decimal('due_amount', 18, 2)->nullable()->default(null); // Remaining amount to be paid
            $table->string('payment_method', 50)->nullable()->default(null); // Method of payment
            $table->string('payment_status', 50)->nullable()->default(null); // Status of the payment (e.g., 'Completed', 'Pending')
            $table->timestamp('payment_date')->nullable()->default(null); // Date and time of payment
            $table->string('gateway', 100)->nullable()->default(null); // Payment gateway (e.g., PayPal, Stripe)
            $table->string('currency', 10)->nullable()->default(null); // Currency type (e.g., USD, EUR)
            $table->foreignId('order_id')->nullable()->constrained('orders')->nullOnDelete(); // Foreign key for the order
            $table->foreignId('providers_id')->constrained('providers')->onDelete('restrict')->onUpdate('cascade'); // Foreign key for the
provider
            $table->timestamps(); // CreatedAt and UpdatedAt

            // Indexes
            $table->index('order_id'); // Index for order_id
            $table->index('providers_id'); // Index for providers_id
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('payments');
    }
```

```
};
```

FILE: database/migrations/2024_12_21_063904_create_permissions_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('permissions', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->string('permission_name', 50)->nullable()->default(null); // Permission name
            $table->string('description', 500)->nullable()->default(null); // Description of the permission
            $table->timestamps(); // CreatedAt and UpdatedAt

        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('permissions');
    }
};
```

FILE: database/migrations/2024_12_21_064015_create_categories_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('categories', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->string('category_name', 100)->nullable()->default(null); // Category name
            $table->string('description', 500)->nullable()->default(null); // Description of the category
            $table->timestamps(); // CreatedAt and UpdatedAt

        });
    }

    /**
     * Reverse the migrations.
```

```
     */
    public function down(): void
    {
        Schema::dropIfExists('categories');
    }
};
```

FILE: database/migrations/2024_12_21_064128_create_shipments_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('shipments', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->string('status', 50)->nullable()->default(null);
            $table->dateTime('estimated_arrival')->nullable()->default(null);
            $table->dateTime('actual_arrival')->nullable()->default(null);
            $table->string('origin', 500)->nullable()->default(null);
            $table->string('destination', 500)->nullable()->default(null);
            $table->string('shipment_method', 50)->nullable()->default(null);
            $table->string('tracking_number', 100)->nullable()->unique();
            $table->dateTime('last_updated')->nullable()->default(null);
            $table->decimal('total_amount', 18, 2)->nullable()->default(null);
            $table->foreignId('providers_id')->constrained('providers'); // Foreign key to Providers table
            $table->foreignId('orders_id')->constrained('orders'); // Foreign key to Orders table


            $table->timestamps(); // CreatedAt and UpdatedAt
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('shipments');
    }
};
```

FILE: database/migrations/2024_12_21_064254_create_ratings_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
```

```
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('ratings', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->integer('rating_value')->nullable()->default(null); // Rating value (e.g., 1-5)
            $table->string('feedback', 1000)->nullable()->default('No feedback provided'); // Feedback text
            $table->dateTime('date_created')->nullable()->default(null); // Date of rating creation
            $table->foreignId('shipments_id')->constrained('shipments'); // Foreign key to Shipments table
            $table->foreignId('products_id')->constrained('products'); // Foreign key to Products table


            $table->timestamps(); // CreatedAt and UpdatedAt
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('ratings');
    }
};
```

FILE: database/migrations/2024_12_21_064420_create_roles_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('roles', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->string('role_name', 50)->nullable()->default(null); // Name of the role
            $table->string('description', 500)->nullable()->default(null); // Description of the role
            $table->timestamps(); // CreatedAt and UpdatedAt
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('roles');
    }
};
```

FILE: database/migrations/2024_12_21_064524_create_vehicles_table.php

```
<?php
```

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('vehicles', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->string('license_plate', 50)->nullable(); // License plate
            $table->string('type', 50)->nullable(); // Type of vehicle (e.g., truck, sedan)
            $table->foreignId('driver_id')->nullable()->constrained('users'); // Foreign key to Users table for driver
            $table->decimal('capacity', 18, 2)->nullable(); // Capacity of the vehicle
            $table->decimal('fuel_capacity', 18, 2)->nullable(); // Fuel capacity
            $table->string('current_location', 500)->nullable(); // Current location
            $table->dateTime('last_serviced')->nullable(); // Last serviced date
            $table->string('status', 100)->nullable(); // Status (e.g., active, maintenance)
            $table->dateTime('last_fuel_refill')->nullable(); // Last fuel refill date
            $table->dateTime('last_location_update')->nullable(); // Last location update
            $table->decimal('mileage', 18, 2)->nullable(); // Mileage of the vehicle
            $table->string('maintenance_logs', 500)->nullable(); // Maintenance logs
            $table->foreignId('vehicle_management_id')->constrained('vehicle_management'); // Foreign key to VehicleManagement table

            // New fields
            $table->integer('fuel_interval')->default(0); // Fuel interval (e.g., distance before next refill/service)
            $table->string('fuel_type', 50)->nullable(); // Fuel type (e.g., Petrol, Diesel, Electric, Hybrid)
            $table->string('vin', 50)->nullable(); // VIN (Vehicle Identification Number)

            // New fields for brand, model, and year of manufacture
            $table->string('brand', 100)->nullable(); // Vehicle brand
            $table->string('model', 100)->nullable(); // Vehicle model
            $table->integer('year_of_manufacture')->nullable(); // Year of manufacture

            $table->timestamps(); // CreatedAt and UpdatedAt
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('vehicles');
    }
};
```

FILE: database/migrations/2024_12_21_064745_create_invoices_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
```

```
    public function up(): void
    {
        Schema::create('invoices', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->string('customer_name', 45)->nullable(); // Customer name
            $table->dateTime('created_date')->nullable(); // Date when the invoice was created
            $table->string('payment_status', 45)->nullable(); // Payment status (e.g., 'paid', 'unpaid')
            $table->string('payment_method', 45)->nullable(); // Payment method (e.g., 'credit card', 'paypal')
            $table->string('description', 500)->nullable(); // Description of the invoice
            $table->decimal('shipping_cost', 18, 2)->nullable(); // Shipping cost
            $table->decimal('total_amount', 18, 2)->nullable(); // Total amount of the invoice
            $table->decimal('paid_amount', 18, 2)->nullable()->default(0); // Paid amount
            $table->decimal('due_amount', 18, 2)->nullable(); // Remaining due amount
            $table->string('currency', 10)->nullable(); // Currency for the invoice
            $table->decimal('discount', 18, 2)->nullable(); // Discount applied to the invoice
            $table->foreignId('payments_id')->constrained('payments'); // Foreign key to Payments table

            $table->timestamps(); // CreatedAt and UpdatedAt
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('invoices');
    }
};
```

FILE: database/migrations/2024_12_21_064905_create_products_categories_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('products_categories', function (Blueprint $table) {
            $table->unsignedBigInteger('products_id');
            $table->unsignedBigInteger('categories_id');
            $table->primary(['products_id', 'categories_id']); // Composite primary key

            // Foreign keys
            $table->foreign('products_id')
                  ->references('id')->on('products')
                  ->onDelete('cascade')
                  ->onUpdate('cascade');

            $table->foreign('categories_id')
                  ->references('id')->on('categories')
                  ->onDelete('cascade')
                  ->onUpdate('cascade');
        });
    }
```

```
    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('products_categories');
    }
};
```

FILE: database/migrations/2024_12_21_065141_create_users_roles_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('users_roles', function (Blueprint $table) {
            $table->unsignedBigInteger('roles_id');
            $table->unsignedBigInteger('users_id');
            $table->timestamps();

            $table->primary(['roles_id', 'users_id']); // Composite primary key

            // Foreign keys
            $table->foreign('roles_id')
                ->references('id')->on('roles')
                ->onDelete('cascade') // Cascade delete if the role is deleted
                ->onUpdate('cascade');

            $table->foreign('users_id')
                ->references('id')->on('users')
                ->onDelete('cascade') // Cascade delete if the user is deleted
                ->onUpdate('cascade');

            $table->index('users_id');
            $table->index('roles_id');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users_roles');
    }
};
```

FILE: database/migrations/2024_12_21_065255_create_roles_permissions_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('roles_permissions', function (Blueprint $table) {
            $table->unsignedBigInteger('permissions_id');
            $table->unsignedBigInteger('roles_id');
            $table->primary(['permissions_id', 'roles_id']); // Composite primary key

            // Foreign keys
            $table->foreign('permissions_id')
                ->references('id')->on('permissions')
                ->onDelete('cascade') // Cascade delete if permission is deleted
                ->onUpdate('cascade');

            $table->foreign('roles_id')
                ->references('id')->on('roles')
                ->onDelete('cascade') // Cascade delete if role is deleted
                ->onUpdate('cascade');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('roles_permissions');
    }
};
```

FILE: database/migrations/2024_12_21_065342_create_order_items_table.php

```php
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('order_items', function (Blueprint $table) {
            $table->id();  // Auto-incrementing primary key (id)

            // Define foreign key columns first
            $table->unsignedBigInteger('orders_id');  // Foreign key referencing orders table
            $table->unsignedBigInteger('products_id');  // Foreign key referencing products table

            // Define other columns
```

```
            $table->integer('quantity')->nullable();  // Quantity
            $table->integer('total_amount')->nullable();  // Total amount (as string)

            // Define composite primary key
            // Define foreign keys
            $table->foreign('orders_id')
                ->references('id')->on('orders')
                ->onDelete('no action')
                ->onUpdate('no action');

            $table->foreign('products_id')
                ->references('id')->on('products')
                ->onDelete('no action')
                ->onUpdate('no action');

            // Add indexes for performance
            $table->index('orders_id');
            $table->index('products_id');

            // Timestamp columns (if you want to track creation and updates)
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::dropIfExists('order_items');
    }
};
```

FILE: database/migrations/2024_12_21_065621_create_coupons_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('coupons', function (Blueprint $table) {
            $table->id(); // Id for the coupon
            $table->string('discount', 45)->nullable(); // Discount value (can be percentage or fixed)
            $table->dateTime('expiration_date')->nullable(); // Expiration date for the coupon
            $table->timestamps(); // Timestamps for created_at and updated_at

        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
```

```
        Schema::dropIfExists('coupons');
    }
};
```

FILE: database/migrations/2024_12_21_065731_create_users_coupons_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('users_coupons', function (Blueprint $table) {
            $table->unsignedBigInteger('users_id');
            $table->unsignedBigInteger('coupons_id');
            $table->timestamps();

            // Define the composite primary key
            $table->primary(['users_id', 'coupons_id']);

            // Foreign key references
            $table->foreign('users_id')
                ->references('id')->on('users')
                ->onDelete('cascade'); // Cascade on delete

            $table->foreign('coupons_id')
                ->references('id')->on('coupons')
                ->onDelete('cascade'); // Cascade on delete

            // Optional: Indexing the foreign keys
            $table->index('users_id');
            $table->index('coupons_id');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users_coupons');
    }
};
```

FILE: database/migrations/2024_12_21_065839_create_products_coupons_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
```

```
return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('products_coupons', function (Blueprint $table) {
            $table->unsignedBigInteger('products_id');
            $table->unsignedBigInteger('coupons_id');
            $table->timestamps();

            // Define the composite primary key
            $table->primary(['products_id', 'coupons_id']);

            // Foreign key references
            $table->foreign('products_id')
                ->references('id')->on('products')
                ->onDelete('cascade'); // Cascade on delete

            $table->foreign('coupons_id')
                ->references('id')->on('coupons')
                ->onDelete('cascade'); // Cascade on delete

            // Optional: Indexing the foreign keys
            $table->index('products_id');
            $table->index('coupons_id');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('products_coupons');
    }
};
```

FILE: database/migrations/2025_01_07_085605_create_location_histories_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('location_histories', function (Blueprint $table) {
            $table->id(); // Auto-incrementing primary key
            $table->foreignId('vehicle_id')->constrained('vehicles')->onDelete('cascade'); // Foreign key to vehicles table
            $table->string('location'); // Location (can store address or other descriptive text)
            $table->decimal('latitude', 10, 7); // Latitude for the location
            $table->decimal('longitude', 10, 7); // Longitude for the location
            $table->timestamp('timestamp')->nullable(); // Timestamp of when the location was logged

            $table->timestamps(); // CreatedAt and UpdatedAt
        });
```

```
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('location_histories');
    }
};
```

FILE: database/migrations/2025_01_09_114839_create_user_addresses_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up()
    {
        Schema::create('user_addresses', function (Blueprint $table) {
            $table->id();  // Automatically creates an 'id' column as primary key
            $table->unsignedBigInteger('users_id'); // Foreign key reference to 'users' table
            $table->string('address_line1');
            $table->string('address_line2')->nullable(); // Optional
            $table->string('city');
            $table->string('state');
            $table->string('postal_code');
            $table->string('country');
            $table->boolean('is_primary')->default(false); // Flag for primary address
            $table->timestamps(); // created_at and updated_at

            // Foreign key constraint
            $table->foreign('users_id')->references('id')->on('users')->onDelete('cascade');
        });
    }

    public function down()
    {
        Schema::dropIfExists('user_addresses');
    }
};
```

FILE: database/migrations/2025_01_11_122341_create_route_optimizations_table.php

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateRouteOptimizationsTable extends Migration
{
    public function up()
    {
        Schema::create('route_optimizations', function (Blueprint $table) {
            $table->id();
```

```
            $table->unsignedBigInteger('shipments_id'); // Link to shipments table
            $table->float('total_distance');
            $table->timestamps();

            // Foreign key constraint for shipment
            $table->foreign('shipments_id')->references('id')->on('shipments')->onDelete('cascade');
        });
    }

    public function down()
    {
        Schema::dropIfExists('route_optimizations');
    }
}
```

FILE: database/migrations/2025_01_11_122457_create_route_details_table.php

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateRouteDetailsTable extends Migration
{
    public function up()
    {
        Schema::create('route_details', function (Blueprint $table) {
            $table->id();
            $table->unsignedBigInteger('route_optimization_id')->nullable(); // Make it nullable first
            $table->unsignedBigInteger('route_condition_id')->nullable(); // Foreign key to route_conditions table
            $table->unsignedBigInteger('vehicle_id')->nullable(); // Foreign key to route_conditions table
            $table->string('route_name');
            $table->string('supplier_name')->nullable();
            $table->string('warehouse_name_1');
            $table->string('warehouse_name_2')->nullable();
            $table->string('start_location');
            $table->string('end_location');
            $table->dateTime('estimated_time');
            $table->decimal('start_latitude', 10, 7); // Latitude for the location
            $table->decimal('start_longitude', 10, 7); // Longitude for the location
            $table->decimal('end_latitude', 10, 7); // Latitude for the location
            $table->decimal('end_longitude', 10, 7); // Longitude for the location
            $table->float('distance');
            $table->timestamps();
            $table->foreign('vehicle_id')->references('id')->on('vehicles')->onDelete('set null');
            $table->foreign('route_condition_id')->references('id')->on('route_conditions')->onDelete('set null');
            $table->foreign('route_optimization_id')->references('id')->on('route_optimizations')->onDelete('set null');
        });
    }

    public function down()
    {
        Schema::dropIfExists('route_details');
    }
}
```

FILE: database/migrations/2025_02_21_085625_create_chatbots_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
```

```
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up()
    {
        Schema::create('chatbots', function (Blueprint $table) {
            $table->id();
            $table->string('user_id'); // User ID field
            $table->json('history'); // Store the chat history as JSON
            $table->timestamps(); // created_at, updated_at
        });
    }


    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('chatbots');
    }
};
```

FILE: database/migrations/2025_02_28_061922_create_follows_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up()
    {
        Schema::create('follows', function (Blueprint $table) {
            $table->id(); // Auto-increment primary key
            $table->foreignId('follower_id')->constrained('users')->onDelete('cascade'); // Foreign key to users table
            $table->foreignId('following_id')->constrained('users')->onDelete('cascade'); // Foreign key to users table
            $table->timestamps();

            // Unique constraint to ensure one follow relationship between two users
            $table->unique(['follower_id', 'following_id']);
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('follows');
    }
};
```

FILE: database/migrations/2025_02_28_062142_create_blocks_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up()
    {
        Schema::create('blocks', function (Blueprint $table) {
            $table->id(); // Auto-increment primary key
            $table->foreignId('blocker_id')->constrained('users')->onDelete('cascade'); // Foreign key to users table for blocker
            $table->foreignId('blocked_id')->constrained('users')->onDelete('cascade'); // Foreign key to users table for blocked
            $table->timestamps();

            // Unique constraint to ensure no duplicate block relationship between two users
            $table->unique(['blocker_id', 'blocked_id']);
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('blocks');
    }
};
```

FILE: database/migrations/2025_02_28_062340_create_streams_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up()
    {
        Schema::create('streams', function (Blueprint $table) {
            $table->id(); // Auto-increment primary key
            $table->string('title');
            $table->string('thumbnail')->nullable();
            $table->string('ingressId')->nullable()->unique();
            $table->string('serverUrl')->nullable();
            $table->string('streamKey')->nullable();
            $table->boolean('isLive')->default(false);
            $table->boolean('isChatEnabled')->default(true);
            $table->boolean('isChatDelayed')->default(false);
            $table->boolean('isChatFollowersOnly')->default(false);
            $table->foreignId('user_id')->constrained('users')->onDelete('cascade'); // Foreign key to users table
```

```
            $table->timestamps(); // Created_at and updated_at columns
        });
    }


    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('streams');
    }
};
```

FILE: database/migrations/2025_03_05_150850_create_stream_messages_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up()
    {
        Schema::create('stream_messages', function (Blueprint $table) {
            $table->id();
            $table->text('message');
            $table->foreignId('creator_id')->constrained('users');  // Creator of the message
            $table->foreignId('viewer_id')->constrained('users');   // Viewer of the message
            $table->foreignId('stream_id')->constrained('streams'); // Reference to the stream
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('stream_messages');
    }
};
```

FILE: database/migrations/2025_03_05_151520_create_stream_gifts_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
```

```
     * Run the migrations.
     */
    public function up()
    {
        Schema::create('stream_gifts', function (Blueprint $table) {
            $table->id();
            $table->foreignId('owner_id')->nullable()->constrained('users'); // Reference to users table (owner of the gift)
            $table->foreignId('stream_id')->nullable()->constrained('streams'); // Reference to the streams table
            $table->enum('gift_type', ['lion', 'flower', 'star', 'heart']); // Enum for gift types
            $table->decimal('price', 8, 2); // Adding price column
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('stream_gifts');
    }
};
```

FILE: database/seeders/DatabaseSeeder.php

```
<?php
namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Database\Factories\PermissionFactory;
use App\Models\{
    User,
    Role,
    Warehouse,
    Inventory,
    Product,
    Category,
    Coupon,
    ProductCategory,
    ProductCoupon,
    UserCoupon,
    UserRole,
    Provider,
    Rating,
    OrderItem,
    Order,
    LocationHistory,
    UserAddress,
    Permission,
    RolePermission,
    Vehicle,
    FeedbackForm,
    FeedbackFormQuestion,
    FeedbackFormAnswer,
    FeedbackFormQuestionAnswer,
    Post,
    Notification,
    ConnectionRequest,
    Comment,
    Like,
    UserConnection,
    Message,
    Conversation,
    Group,
```

```
    Chatbot,
    StreamGift,
};

use App\Enums\GiftTypeEnum;

use Carbon\Carbon;

use Faker\Factory as Faker;

class DatabaseSeeder extends Seeder
{
    /**
     * Run the database seeds.
     *
     * @return void
     */
    public function run()
    {
        // Create 100 users
        $users = User::factory(100)->create();

        // Loop through each value of the GiftTypeEnum and create a StreamGift
        foreach (GiftTypeEnum::cases() as $giftType) {
            // Create a new StreamGift for each gift type
            $streamGift = StreamGift::create([
                'gift_type' => $giftType->value, // Use the enum value (e.g., 'lion', 'flower', etc.)
                'price' => $giftType->getPrice(), // Dynamically set the price based on the gift type
            ]);
        }
        foreach ($users as $user) {
            // Create 5 chatbots for each user, passing the user_id
            Chatbot::factory(5)->create([
                'user_id' => $user->id
            ]);
        }

        // Create 5 groups
        $groups = Group::factory(5)->create();

        // Assign each user to a random group, ensuring at least one group for each user
        foreach ($users as $user) {
            $group = $groups->random();
            $user->groups()->attach($group->id);
        }

        // Create 10 conversations for each user with other random users
        foreach ($users as $user) {
            for ($i = 0; $i < 10; $i++) {
                // Ensure the other user in the conversation is not the same user
                $otherUser = $users->where('id', '!=', $user->id)->random();

                // Check if a conversation already exists between these two users
                $conversation = Conversation::where(function ($query) use ($user, $otherUser) {
                    $query->where('user_id1', $user->id)
                        ->where('user_id2', $otherUser->id);
                })->orWhere(function ($query) use ($user, $otherUser) {
                    $query->where('user_id1', $otherUser->id)
                        ->where('user_id2', $user->id);
                })->first();

                // If no conversation exists, create a new one
                if (!$conversation) {
                    $conversation = Conversation::create([
                        'user_id1' => $user->id,
                        'user_id2' => $otherUser->id,
```

```
                ]);
            }

            // Create 20 messages for each side of the conversation
            $lastMessage = null;
            for ($j = 0; $j < 20; $j++) {
                // User sends a message
                $message = Message::factory()->create([
                    'sender_id' => $user->id,
                    'receiver_id' => $otherUser->id,
                    'conversation_id' => $conversation->id,
                ]);

                // Update the conversation's last message id
                $conversation->update([
                    'last_message_id' => $message->id,
                ]);

                // Other user responds with a message
                $message = Message::factory()->create([
                    'sender_id' => $otherUser->id,
                    'receiver_id' => $user->id,
                    'conversation_id' => $conversation->id,
                ]);

                // Update the conversation's last message id
                $conversation->update([
                    'last_message_id' => $message->id,
                ]);
            }
        }
    }

    // Create group messages and ensure last_message_id is set for groups
    foreach ($groups as $group) {
        $usersInGroup = $group->users;

        foreach ($usersInGroup as $user) {
            // Create messages in groups, ensuring group_id is set
            for ($i = 0; $i < 10; $i++) {
                // Random message sender within the group
                $sender = $usersInGroup->random();
                $message = Message::factory()->create([
                    'sender_id' => $sender->id,
                    'group_id' => $group->id, // Make sure the group_id is set
                ]);

                // Update the group's last message id
                $group->update([
                    'last_message_id' => $message->id,
                ]);
            }
        }
    }



   foreach ($users as $user) {
        $role = Role::factory()->create();

        // Assign role to user
        UserRole::create([
            'users_id' => $user->id,
            'roles_id' => $role->id,
        ]);
```

```
        // Assign permissions suitable for the role
        $permissions = PermissionFactory::permissionsForRole($role->role_name);

        foreach ($permissions as $perm) {
            RolePermission::create([
                'roles_id' => $role->id,
                'permissions_id' => Permission::firstOrCreate(
                    ['permission_name' => $perm['permission_name']],
                    ['description' => $perm['description']]
                )->id,
            ]);
        }
    }

    // Seed Categories
    $categories = Category::factory(5)->create(); // Create 5 categories

    $productSaler = UserRole::whereHas('role', function ($query) {
        $query->where('role_name', 'ProductSaler');
    })->first();

    $products = Product::factory(20)->create([
        'supplier_id' => $productSaler->user->id,
    ]); // Create 20 products
    foreach ($products as $product) {

        // Associate each product with all categories
        foreach ($categories as $category) {
            ProductCategory::create([
                'products_id' => $product->id,
                'categories_id' => $category->id,
            ]);
        }

        // Create 5 ratings for each product
        Rating::factory(5)->create([
            'products_id' => $product->id,
        ]);
    }

    $faker = Faker::create();
    // Seed applications with associated questions and answers
    FeedbackForm::factory()
        ->count(10)
        ->create(['user_id' => $productSaler->user->id])
        ->each(function ($feedbackForm) use ($faker) {
            FeedbackFormQuestion::factory()
                ->count(10)
                ->create(['feedback_form_id' => $feedbackForm->id])
                ->each(function ($question) use ($feedbackForm, $faker) {
                    $answer = FeedbackFormAnswer::factory()->create(['feedback_form_id' => $feedbackForm->id]);
                    FeedbackFormQuestionAnswer::factory()->create([
                        'feedback_form_question_id' => $question->id,
                        'feedback_form_answer_id' => $answer->id,
                        'answer' => $faker->sentence,
                    ]);
                });
        });

    // Create posts for each user
    $posts = [];
    foreach ($users as $user) {
        $userPosts = Post::factory()->count(3)->create([
            'author_id' => $user->id,
        ]);
        $posts = array_merge($posts, $userPosts->toArray());
```

```
        }

        // Create notifications for each user
        foreach ($users as $user) {
            Notification::factory()->count(2)->create([
                'recipient_id' => $user->id,
                'type' => $faker->randomElement(['like', 'comment', 'connectionAccepted']),
                'related_user' => $users->random()->id,
                'related_post' => Post::factory()->create()->id,
            ]);
        }

        // Create comments for each post
        foreach ($posts as $post) {
            $commentCount = rand(1, 5);
            foreach (range(1, $commentCount) as $index) {
                Comment::factory()->create([
                    'post_id' => $post['id'],
                    'user_id' => $users->random()->id,
                    'content' => $faker->sentence,
                ]);
            }
        }

        // Create likes for some posts
        foreach ($posts as $post) {
            $likeCount = rand(1, 5);
            foreach (range(1, $likeCount) as $index) {
                Like::factory()->create([
                    'post_id' => $post['id'],
                    'user_id' => $users->random()->id,
                ]);
            }
        }
        // Create Coupons and associate with products and users
        $coupons = Coupon::factory(10)->create();
        foreach ($users as $user) {
            foreach ($coupons as $coupon) {
                foreach ($products as $product) {
                    // Check if the combination of product and coupon exists
                    $existingProductCoupon = ProductCoupon::where('products_id', $product->id)
                                                          ->where('coupons_id', $coupon->id)
                                                          ->first();

                    if (!$existingProductCoupon) {
                        // If the combination does not exist, create a new record
                        ProductCoupon::create([
                            'products_id' => $product->id,
                            'coupons_id' => $coupon->id,
                        ]);
                    }
                }

                // Check if the user-coupon relationship exists
                $existingUserCoupon = UserCoupon::where('users_id', $user->id)
                                                ->where('coupons_id', $coupon->id)
                                                ->first();

                if (!$existingUserCoupon) {
                    // If the combination does not exist, create a new record
                    UserCoupon::create([
                        'users_id' => $user->id,
                        'coupons_id' => $coupon->id,
                    ]);
                }
            }
```

```
        }




        // Seed Warehouses and associate warehouse managers
        $warehouses = Warehouse::factory(20)->create(); // Create 5 warehouses
        foreach ($warehouses as $warehouse) {
            // Get a warehouse manager role user (if exists)
            $warehouseManager = UserRole::whereHas('role', function ($query) {
                $query->where('role_name', 'WarehouseManager');
            })->first();

            if ($warehouseManager) {
                $warehouse->users_id = $warehouseManager->user->id;
                $warehouse->save();
            }
        }

        // Create Inventories for each Warehouse
        foreach ($warehouses as $warehouse) {
            $inventories = Inventory::factory(20)->create([
                'warehouses_id' => $warehouse->id,
            ]);

            // Associate each inventory with a unique product
            foreach ($inventories as $index => $inventory) {
                $inventory->products_id = $products[$index % count($products)]->id;
                $inventory->save();
            }
        }

        // Seed Providers
        $providers = Provider::factory(5)->create();

        // Seed Orders with OrderItems
        foreach (range(1, 20) as $i) {
            // Create an Order
            $order = Order::factory()->create();

            // Create 3 to 5 OrderItems per Order (linked to products)
            foreach (range(1, rand(3, 5)) as $j) {
                // Use sequential product selection for order items
                $product = $products[$j % count($products)];  // Ensure wrapping if there are fewer products than order items

                // Create the OrderItem
                OrderItem::factory()->create([
                    'orders_id' => $order->id,
                    'products_id' => $product->id,
                    'total_amount' => $product->price,  // Assuming price exists on product
                    'quantity' => rand(1, 10),
                ]);
            }
        }

        $allUsers = User::all();

        foreach ($allUsers as $user) {
            UserAddress::factory(5)->create(['users_id' => $user->id]);
        }

        $deliveryMan = UserRole::whereHas('role', function ($query) {
            $query->where('role_name', 'DeliveryMan');
        })->first();

        $vehicles = Vehicle::factory(1)->create(['driver_id' => $deliveryMan->user->id]);
```

```
    }
}
```

FILE: frontend/eslint.config.js

```
import js from "@eslint/js";
import globals from "globals";
import react from "eslint-plugin-react";
import reactHooks from "eslint-plugin-react-hooks";
import reactRefresh from "eslint-plugin-react-refresh";

export default [
    { ignores: ["dist"] },
    {
        files: ["**/*.{js,jsx}"],
        languageOptions: {
            ecmaVersion: 2020,
            globals: globals.browser,
            parserOptions: {
                ecmaVersion: "latest",
                ecmaFeatures: { jsx: true },
                sourceType: "module",
            },
        },
        settings: { react: { version: "18.3" } },
        plugins: {
            react,
            "react-hooks": reactHooks,
            "react-refresh": reactRefresh,
        },
        rules: {
            ...js.configs.recommended.rules,
            ...react.configs.recommended.rules,
            ...react.configs["jsx-runtime"].rules,
            ...reactHooks.configs.recommended.rules,
            "react/jsx-no-target-blank": "off",
            "react-refresh/only-export-components": ["warn", { allowConstantExport: true }],
            "react/prop-types": "off",
        },
    },
];
```

FILE: frontend/postcss.config.js

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}
```

FILE: frontend/src/App.jsx

```
import { Navigate, Route, Routes } from "react-router-dom";
import { useEffect } from "react";
import { Toaster } from "react-hot-toast";

import HomePage from "./pages/HomePage";
import SignUpPage from "./pages/SignUpPage";
import LoginPage from "./pages/LoginPage";
import AdminPage from "./pages/AdminPage";
import CategoryPage from "./pages/CategoryPage";
```

```
import Navbar from "./components/Navbar";
import LoadingSpinner from "./components/LoadingSpinner";
import CartPage from "./pages/CartPage";
import PurchaseSuccessPage from "./pages/PurchaseSuccessPage";
import PurchaseCancelPage from "./pages/PurchaseCancelPage";
import PaymentPage from "./pages/PaymentPage";
import { useUserStore } from "./stores/useUserStore";
import Wrapper from "./pages/Warehouse/Wrapper";
import Expenses from "./pages/Warehouse/Expenses";
import ProfileLayout from "./components/Profile/ProfileLayout";
import ProfilePage from "./pages/ProfilePage";
import EditProfilePage from "./pages/EditProfilePage";
import ChangePasswordPage from "./pages/ChangePasswordPage";
import ProductDetailPage from "./pages/ProductDetailPage.jsx";
import WarehouseInventories from "./pages/Warehouse/WarehouseInventories.jsx";
import Daily from "./pages/Warehouse/Daily.jsx";
import Geography from "./pages/Warehouse/Geography.jsx";
import CreateInventory from "./pages/Warehouse/CreateInventory.jsx";
import WarehouseOrders from "./pages/Warehouse/WarehouseOrders.jsx";
import VehicleListPage from "./pages/VehicleManagement/VehicleListPage.jsx";
import WrapperVehicle from "./pages/VehicleManagement/WrapperVehicle.jsx";
import './echo.js'
import UserAddressesPage from "./pages/UserAddressPage.jsx";
import WrapperMap from "./pages/Map/WrapperMap.jsx";
import { Elements } from '@stripe/react-stripe-js';
import { stripePromise } from './stripe'; // Make sure you have stripe.js configured
import LicensePlateCapturePage from "./pages/Warehouse/LicensePlateCapturePage.jsx";
import CreateFeedbackForm from "./pages/FeedbackForm/CreateFeedbackForm.jsx";
import SocialMediaPage from "./pages/SocialMedia/SocialMediaPage.jsx";
import NotificationsPage from "./pages/SocialMedia/NotificationsPage.jsx";
import NetworkPage from "./pages/SocialMedia/NetworkPage.jsx";
import ChatWrapper from "./pages/Chat/ChatWrapper.jsx";
import DashboardLayout from "./pages/ChatBot/DashboardLayout.jsx";
import DashboardPage from "./pages/ChatBot/DashboardPage.jsx";
import ChatPage from "./pages/ChatBot/ChatPage.jsx";
import LiveStreamWrapper from "./pages/LiveStream/LiveStreamWrapper.jsx";
import { useTwoFactorAuthenticationStore } from "./stores/useTwoFactorAuthenticationStore.js";
import TwoFactorAuthentication from "./pages/TwoFactorAuthentication.jsx";
import Wheel from "./pages/Wheel/Wheel.jsx";
import Editor from "./pages/Editor/Editor.jsx";

function App() {
  const { user, checkingAuth, checkAuth } = useUserStore();
  const { is2FAComplete } = useTwoFactorAuthenticationStore();
  // On initial load, check authentication status
  useEffect(() => {
    checkAuth();
  }, [checkAuth]);

  // If the authentication is still being checked, show checkingAuth spinner
  if (checkingAuth) {
    return <LoadingSpinner />;
  }

  const require2FA = (user && !is2FAComplete);

  console.log(require2FA);
  return (
    <div className="min-h-screen bg-gray-900 text-white relative overflow-hidden">
      {/* Background gradient */}
      <div className="absolute inset-0 overflow-hidden">
        <div className="absolute inset-0">
          <div className="absolute top-0 left-1/2 -translate-x-1/2 w-full h-full bg-[radial-
gradient(ellipse_at_top,rgba(16,185,129,0.3)_0%,rgba(10,80,60,0.2)_45%,rgba(0,0,0,0.1)_100%)]" />
        </div>
      </div>
```

```
      <div className="relative z-50 pt-20">
        <Navbar />
        <Routes>
          {/* Public Routes */}
          <Route path="/signup" element={!user ? <SignUpPage /> : <Navigate to="/2fa" />} />
          <Route path="/login" element={!user ? <LoginPage /> : <Navigate to="/2fa" />} />

          <Route path="/2fa" element={<TwoFactorAuthentication />} />

          {/* Home page route - If 2FA is not complete, redirect to /2fa */}
          <Route
            path="/"
            element={<HomePage />}
          />

          <Route path="/product/:id" element={<ProductDetailPage />} /> {/* Product details page
*/}
          <Route
            path="/secret-dashboard"
            element={user?.role === "admin" ? <AdminPage /> : <Navigate to="/" />}
          />
          <Route path="/category/:category" element={<CategoryPage />} />
          <Route path="/cart" element={user ? <CartPage /> : <Navigate to="/login" />} />
          <Route
            path="/purchase-success"
            element={user ? <PurchaseSuccessPage /> : <Navigate to="/login" />}
          />
          <Route
            path="/purchase-cancel"
            element={user ? <PurchaseCancelPage /> : <Navigate to="/login" />}
          />

          {/* Payment Route - Wrapped in Elements */}
          <Route
            path="/payment"
            element={user ? (
              <Elements stripe={stripePromise}>
                <PaymentPage />
              </Elements>
            ) : <Navigate to="/login" />}
          />


          <Route path="/social-media" element={user ? <SocialMediaPage /> : <Navigate to="/login"
/>} />
          <Route path="/notification" element={user ? <NotificationsPage /> : <Navigate to="/login"
/>} />
          <Route path="/network" element={user ? <NetworkPage /> : <Navigate to="/login" />} />

          <Route
            path="/map"
            element={user ? <WrapperMap /> : <Navigate to="/login" />}
          />

            <Route path="/warehouse" element={user ? <Wrapper/> : <Navigate to="/login" />}>
            <Route index element={<Expenses />} />
            <Route path="products" element={<WarehouseInventories />} />
            <Route path="linechart" element={<Daily />} />
            <Route path="geography" element={<Geography />} />
            <Route path="create-inventory" element={<CreateInventory />} />
            <Route path="orders" element={<WarehouseOrders />} />
            <Route path="license-plate" element={<LicensePlateCapturePage />} />
          </Route>

          <Route path="/vehicle" element={user ? <WrapperVehicle/> : <Navigate to="/login" />}>
```

```
            <Route index element={<VehicleListPage />} />
          </Route>

          <Route path="/profile" element={user ? <ProfileLayout/> : <Navigate to="/login"/> }>
            <Route index element={<ProfilePage />} /> {/* This will be the default route */}
            <Route path="edit" element={<EditProfilePage />} />
            <Route path="change-password" element={<ChangePasswordPage />} />
            <Route path="address" element={<UserAddressesPage />} />
          </Route>

          <Route path="/chatbot" element={user ? <DashboardLayout /> : <Navigate to="/login" />}>
            <Route index element={<DashboardPage />} />  {/* This handles the "/chatbot" path */}
            <Route path="chats/:id" element={<ChatPage />} />  {/* This handles
"/chatbot/chats/:id" */}
          </Route>

          <Route path="/feedback-forms/create" element={user ? <CreateFeedbackForm /> : <Navigate
to="/login" />} />

          <Route path="/chat" element={user ? <ChatWrapper/> : <Navigate to="/login" />} />

          <Route path="/wheel" element={user ? <Wheel/> : <Navigate to="/login" />} />

          <Route path="/editor" element={user ? <Editor/> : <Navigate to="/login" />} />


          <Route
            path="/live-stream"
            element={
              user ?
              <Elements stripe={stripePromise}>
                <LiveStreamWrapper/>
              </Elements> : <Navigate to="/login" />}
          />
        </Routes>
      </div>
      <Toaster />
    </div>
  );
}

export default App;
```

FILE: frontend/src/components/AnalyticsTab.jsx

```
import { motion } from "framer-motion";
import { useEffect, useState } from "react";
import axios from "../lib/axios";
import { Users, Package, ShoppingCart, DollarSign } from "lucide-react";
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Legend, ResponsiveContainer } from
"recharts";

const AnalyticsTab = () => {
    const [analyticsData, setAnalyticsData] = useState({
        users: 0,
        products: 0,
        totalSales: 0,
        totalRevenue: 0,
    });
    const [isLoading, setIsLoading] = useState(true);
    const [dailySalesData, setDailySalesData] = useState([]);

    useEffect(() => {
        const fetchAnalyticsData = async () => {
            try {
                const response = await axios.get("/analytics");
```

```
            setAnalyticsData(response.data.analyticsData);
            setDailySalesData(response.data.dailySalesData);
        } catch (error) {
            console.error("Error fetching analytics data:", error);
        } finally {
            setIsLoading(false);
        }
    };

    fetchAnalyticsData();
}, []);

if (isLoading) {
    return <div>Loading...</div>;
}

return (
    <div className='max-w-7xl mx-auto px-4 sm:px-6 lg:px-8'>
        <div className='grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6 mb-8'>
            <AnalyticsCard
                title='Total Users'
                value={analyticsData.users.toLocaleString()}
                icon={Users}
                color='from-emerald-500 to-teal-700'
            />
            <AnalyticsCard
                title='Total Products'
                value={analyticsData.products.toLocaleString()}
                icon={Package}
                color='from-emerald-500 to-green-700'
            />
            <AnalyticsCard
                title='Total Sales'
                value={analyticsData.totalSales.toLocaleString()}
                icon={ShoppingCart}
                color='from-emerald-500 to-cyan-700'
            />
            <AnalyticsCard
                title='Total Revenue'
                value={`$${analyticsData.totalRevenue.toLocaleString()}`}
                icon={DollarSign}
                color='from-emerald-500 to-lime-700'
            />
        </div>
        <motion.div
            className='bg-gray-800/60 rounded-lg p-6 shadow-lg'
            initial={{ opacity: 0, y: 20 }}
            animate={{ opacity: 1, y: 0 }}
            transition={{ duration: 0.5, delay: 0.25 }}
        >
            <ResponsiveContainer width='100%' height={400}>
                <LineChart data={dailySalesData}>
                    <CartesianGrid strokeDasharray='3 3' />
                    <XAxis dataKey='name' stroke='#D1D5DB' />
                    <YAxis yAxisId='left' stroke='#D1D5DB' />
                    <YAxis yAxisId='right' orientation='right' stroke='#D1D5DB' />
                    <Tooltip />
                    <Legend />
                    <Line
                        yAxisId='left'
                        type='monotone'
                        dataKey='sales'
                        stroke='#10B981'
                        activeDot={{ r: 8 }}
                        name='Sales'
                    />
```

```
                        <Line
                            yAxisId='right'
                            type='monotone'
                            dataKey='revenue'
                            stroke='#3B82F6'
                            activeDot={{ r: 8 }}
                            name='Revenue'
                        />
                    </LineChart>
                </ResponsiveContainer>
            </motion.div>
        </div>
    );
};
export default AnalyticsTab;

const AnalyticsCard = ({ title, value, icon: Icon, color }) => (
    <motion.div
        className={`bg-gray-800 rounded-lg p-6 shadow-lg overflow-hidden relative ${color}`}
        initial={{ opacity: 0, y: 20 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.5 }}
    >
        <div className='flex justify-between items-center'>
            <div className='z-10'>
                <p className='text-emerald-300 text-sm mb-1 font-semibold'>{title}</p>
                <h3 className='text-white text-3xl font-bold'>{value}</h3>
            </div>
        </div>
        <div className='absolute inset-0 bg-gradient-to-br from-emerald-600 to-emerald-900 opacity-
30' />
        <div className='absolute -bottom-4 -right-4 text-emerald-800 opacity-50'>
            <Icon className='h-32 w-32' />
        </div>
    </motion.div>
);
```

FILE: frontend/src/components/CartItem.jsx

```
import { Minus, Plus, Trash } from "lucide-react";
import { useCartStore } from "../stores/useCartStore";

const CartItem = ({ item }) => {
    const { removeFromCart, updateQuantity } = useCartStore();

    return (
        <div className='rounded-lg border p-4 shadow-sm border-gray-700 bg-gray-800 md:p-4'>
            <div className='flex flex-col space-y-2 md:flex-row md:space-x-4 md:space-y-0 md:items-
center md:justify-between'>
                {/* Image section */}
                <div className='shrink-0'>
                    <img className='h-16 md:h-24 rounded object-cover' src={item.product.image} />
                </div>

                {/* Quantity controls */}
                <div className='flex items-center gap-2'>
                    <button
                        className='inline-flex h-5 w-5 items-center justify-center rounded-md
border border-gray-600 bg-gray-700 hover:bg-gray-600 focus:outline-none focus:ring-2 focus:ring-
emerald-500'
                        onClick={() => updateQuantity(item.product.id, false)}
                    >
                        <Minus className='text-gray-300' />
                    </button>
                    <p className='text-base text-white'>{item.quantity}</p>
                    <button
```

```
                        className='inline-flex h-5 w-5 items-center justify-center rounded-md
border border-gray-600 bg-gray-700 hover:bg-gray-600 focus:outline-none focus:ring-2 focus:ring-
emerald-500'
                        onClick={() => updateQuantity(item.product.id, true)}
                    >
                        <Plus className='text-gray-300' />
                    </button>
                </div>

                {/* Product name and details */}
                <div className='text-sm text-white space-y-1'>
                    <p className='font-medium text-emerald-400'>{item.product.name}</p>
                    <p className='text-gray-400'>{item.product.description}</p>
                    <p className='text-gray-400'>Price: ${item.product.price}</p>
                    <p className='font-bold text-emerald-400'>Total: ${item.total_amount}</p>
                </div>

                {/* Remove from cart button */}
                <div className='flex items-center justify-center'>
                    <button
                        className='inline-flex items-center text-sm font-medium text-red-400
hover:text-red-300 hover:underline'
                        onClick={() => removeFromCart(item.product.id)}
                    >
                        <Trash />
                    </button>
                </div>
            </div>
        </div>
    );
};

export default CartItem;
```

FILE: frontend/src/components/CategoryItem.jsx

```
import { Link } from "react-router-dom";

const CategoryItem = ({ category }) => {
  return (
    <div className='relative overflow-hidden h-96 w-full rounded-lg group'>
      <Link to={"/category" + category.href}>
        <div className='w-full h-full cursor-pointer'>
          <div className='absolute inset-0 bg-gradient-to-b from-transparent to-gray-900 opacity-50
z-10' />
          <img
            src={category.imageUrl}
            alt={category.name}
            className='w-full h-full object-cover transition-transform duration-500 ease-out group-
hover:scale-110'
            loading='lazy'
          />
          <div className='absolute bottom-0 left-0 right-0 p-4 z-20'>
            <h3 className='text-white text-2xl font-bold mb-2'>{category.name}</h3>
            <p className='text-gray-200 text-sm mb-4'>{category.description}</p>
            <p className='text-gray-300 text-xs'>Explore {category.name}</p>
          </div>
        </div>
      </Link>
    </div>
  );
};

export default CategoryItem;
```

FILE: frontend/src/components/CreateProductForm.jsx

```
import { useState } from "react";
import { motion } from "framer-motion";
import { PlusCircle, Upload, Loader } from "lucide-react";
import { useProductStore } from "../stores/useProductStore";

const categories = ["jeans", "t-shirts", "shoes", "glasses", "jackets", "suits", "bags"];

const CreateProductForm = () => {
    const [newProduct, setNewProduct] = useState({
        name: "",
        description: "",
        price: "",
        category: "",
        image: "",
    });

    const { createProduct, loading } = useProductStore();

    const handleSubmit = async (e) => {
        e.preventDefault();
        try {
            await createProduct(newProduct);
            setNewProduct({ name: "", description: "", price: "", category: "", image: "" });
        } catch {
            console.log("error creating a product");
        }
    };

    const handleImageChange = (e) => {
        const file = e.target.files[0];
        if (file) {
            const reader = new FileReader();

            reader.onloadend = () => {
                setNewProduct({ ...newProduct, image: reader.result });
            };

            reader.readAsDataURL(file); // base64
        }
    };

    return (
        <motion.div
            className='bg-gray-800 shadow-lg rounded-lg p-8 mb-8 max-w-xl mx-auto'
            initial={{ opacity: 0, y: 20 }}
            animate={{ opacity: 1, y: 0 }}
            transition={{ duration: 0.8 }}
        >
            <h2 className='text-2xl font-semibold mb-6 text-emerald-300'>Create New Product</h2>

            <form onSubmit={handleSubmit} className='space-y-4'>
                <div>
                    <label htmlFor='name' className='block text-sm font-medium text-gray-300'>
                        Product Name
                    </label>
                    <input
                        type='text'
                        id='name'
                        name='name'
                        value={newProduct.name}
                        onChange={(e) => setNewProduct({ ...newProduct, name: e.target.value })}
                        className='mt-1 block w-full bg-gray-700 border border-gray-600 rounded-md
shadow-sm py-2
                         px-3 text-white focus:outline-none focus:ring-2
```

```
                        focus:ring-emerald-500 focus:border-emerald-500'
                        required
                    />
                </div>

                <div>
                    <label htmlFor='description' className='block text-sm font-medium text-gray-
300'>
                        Description
                    </label>
                    <textarea
                        id='description'
                        name='description'
                        value={newProduct.description}
                        onChange={(e) => setNewProduct({ ...newProduct, description: e.target.value
})}
                        rows='3'
                        className='mt-1 block w-full bg-gray-700 border border-gray-600 rounded-md
shadow-sm
                         py-2 px-3 text-white focus:outline-none focus:ring-2 focus:ring-emerald-
500
                         focus:border-emerald-500'
                        required
                    />
                </div>

                <div>
                    <label htmlFor='price' className='block text-sm font-medium text-gray-300'>
                        Price
                    </label>
                    <input
                        type='number'
                        id='price'
                        name='price'
                        value={newProduct.price}
                        onChange={(e) => setNewProduct({ ...newProduct, price: e.target.value })}
                        step='0.01'
                        className='mt-1 block w-full bg-gray-700 border border-gray-600 rounded-md
shadow-sm
                        py-2 px-3 text-white focus:outline-none focus:ring-2 focus:ring-emerald-500
                         focus:border-emerald-500'
                        required
                    />
                </div>

                <div>
                    <label htmlFor='category' className='block text-sm font-medium text-gray-300'>
                        Category
                    </label>
                    <select
                        id='category'
                        name='category'
                        value={newProduct.category}
                        onChange={(e) => setNewProduct({ ...newProduct, category: e.target.value
})}
                        className='mt-1 block w-full bg-gray-700 border border-gray-600 rounded-md
                         shadow-sm py-2 px-3 text-white focus:outline-none
                         focus:ring-2 focus:ring-emerald-500 focus:border-emerald-500'
                        required
                    >
                        <option value=''>Select a category</option>
                        {categories.map((category) => (
                            <option key={category} value={category}>
                                {category}
                            </option>
                        ))}
```

```
                    </select>
                </div>

                <div className='mt-1 flex items-center'>
                    <input type='file' id='image' className='sr-only' accept='image/*' onChange=
{handleImageChange} />
                    <label
                        htmlFor='image'
                        className='cursor-pointer bg-gray-700 py-2 px-3 border border-gray-600
rounded-md shadow-sm text-sm leading-4 font-medium text-gray-300 hover:bg-gray-600 focus:outline-
none focus:ring-2 focus:ring-offset-2 focus:ring-emerald-500'
                    >
                        <Upload className='h-5 w-5 inline-block mr-2' />
                        Upload Image
                    </label>
                    {newProduct.image && <span className='ml-3 text-sm text-gray-400'>Image
uploaded </span>}
                </div>

                <button
                    type='submit'
                    className='w-full flex justify-center py-2 px-4 border border-transparent
rounded-md
                    shadow-sm text-sm font-medium text-white bg-emerald-600 hover:bg-emerald-700
                    focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-emerald-500
disabled:opacity-50'
                    disabled={loading}
                >
                    {loading ? (
                        <>
                            <Loader className='mr-2 h-5 w-5 animate-spin' aria-hidden='true' />
                            Loading...
                        </>
                    ) : (
                        <>
                            <PlusCircle className='mr-2 h-5 w-5' />
                            Create Product
                        </>
                    )}
                </button>
            </form>
        </motion.div>
    );
};
export default CreateProductForm;
```

FILE: frontend/src/components/Editor/Badge.jsx

```
import React from "react";
import { cva } from "class-variance-authority";
import { cn } from "../../lib/utils";
const badgeVariants = cva(
  "inline-flex items-center rounded-full border px-2.5 py-0.5 text-xs font-semibold transition-
colors focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2",
  {
    variants: {
      variant: {
        default:
          "border-transparent bg-primary text-primary-foreground hover:bg-primary/80",
        secondary:
          "border-transparent bg-secondary text-secondary-foreground hover:bg-secondary/80",
        destructive:
          "border-transparent bg-destructive text-destructive-foreground hover:bg-destructive/80",
        outline: "text-foreground",
      },
    },
```

```
    defaultVariants: {
      variant: "default",
    },
  }
);

function Badge({ className, variant = "default", ...props }) {
  return (
    <div className={cn(badgeVariants({ variant }), className)} {...props} />
  );
}

export { Badge, badgeVariants };
```

FILE: frontend/src/components/Editor/Button.jsx

```
import React from "react";
import { cva } from "class-variance-authority";
import { cn } from "../../lib/utils";
import { Slot } from "@radix-ui/react-slot";
const buttonVariants = cva(
  "inline-flex items-center justify-center whitespace-nowrap rounded-md text-sm font-medium ring-
offset-background transition-colors focus-visible:outline-none focus-visible:ring-2 focus-
visible:ring-ring focus-visible:ring-offset-2 disabled:pointer-events-none disabled:opacity-50",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground hover:bg-primary/90",
        destructive:
          "bg-destructive text-destructive-foreground hover:bg-destructive/90",
        outline:
          "border border-input bg-background hover:bg-accent hover:text-accent-foreground",
        secondary:
          "bg-secondary text-secondary-foreground hover:bg-secondary/80",
        ghost: "hover:bg-accent hover:text-accent-foreground",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default: "h-10 px-4 py-2",
        sm: "h-9 rounded-md px-3",
        lg: "h-11 rounded-md px-8",
        icon: "h-10 w-10",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
);

function Button({ className, variant = "default", size = "default", asChild = false, ...props }) {
  const Comp = asChild ? Slot : "button";
  return (
    <Comp
      className={cn(buttonVariants({ variant, size, className }))}
      {...props}
    />
  );
}

Button.displayName = "Button";

export { Button, buttonVariants };
```

FILE: frontend/src/components/Editor/Card.jsx

```
import React from "react";
import { cn } from "../../lib/utils";
const Card = React.forwardRef((props, ref) => (
  <div
    ref={ref}
    className={cn("rounded-lg border bg-card text-card-foreground shadow-sm", props.className)}
    {...props}
  />
));
Card.displayName = "Card";

const CardHeader = React.forwardRef((props, ref) => (
  <div
    ref={ref}
    className={cn("flex flex-col space-y-1.5 p-6", props.className)}
    {...props}
  />
));
CardHeader.displayName = "CardHeader";

const CardTitle = React.forwardRef((props, ref) => (
  <h3
    ref={ref}
    className={cn("text-2xl font-semibold leading-none tracking-tight", props.className)}
    {...props}
  />
));
CardTitle.displayName = "CardTitle";

const CardDescription = React.forwardRef((props, ref) => (
  <p
    ref={ref}
    className={cn("text-sm text-muted-foreground", props.className)}
    {...props}
  />
));
CardDescription.displayName = "CardDescription";

const CardContent = React.forwardRef((props, ref) => (
  <div ref={ref} className={cn("p-6 pt-0", props.className)} {...props} />
));
CardContent.displayName = "CardContent";

const CardFooter = React.forwardRef((props, ref) => (
  <div
    ref={ref}
    className={cn("flex items-center p-6 pt-0", props.className)}
    {...props}
  />
));
CardFooter.displayName = "CardFooter";

export { Card, CardHeader, CardFooter, CardTitle, CardDescription, CardContent };
```

FILE: frontend/src/components/Editor/Checkbox.jsx

```
import React from "react";
import * as CheckboxPrimitive from "@radix-ui/react-checkbox";
import { Check } from "lucide-react";
import { cn } from "../../lib/utils";
const Checkbox = React.forwardRef(
  ({ className, ...props }, ref) => (
    <CheckboxPrimitive.Root
```

```
      ref={ref}
      className={cn(
        "peer h-4 w-4 shrink-0 rounded-sm border border-primary ring-offset-background focus-
visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2
disabled:cursor-not-allowed disabled:opacity-50 data-[state=checked]:bg-primary data-
[state=checked]:text-primary-foreground",
        className
      )}
      {...props}
    >
      <CheckboxPrimitive.Indicator
        className={cn("flex items-center justify-center text-current")}
      >
        <Check className="h-4 w-4" />
      </CheckboxPrimitive.Indicator>
    </CheckboxPrimitive.Root>
  )
);
Checkbox.displayName = CheckboxPrimitive.Root.displayName;

export { Checkbox };
```

FILE: frontend/src/components/Editor/Dialog.jsx

```
import React from "react";
import * as DialogPrimitive from "@radix-ui/react-dialog";
import { X } from "lucide-react";
import { cn } from "../../lib/utils";
const Dialog = DialogPrimitive.Root;

const DialogTrigger = DialogPrimitive.Trigger;

const DialogPortal = DialogPrimitive.Portal;

const DialogClose = DialogPrimitive.Close;

const DialogOverlay = React.forwardRef(
  ({ className, ...props }, ref) => (
    <DialogPrimitive.Overlay
      ref={ref}
      className={cn(
        "fixed inset-0 z-50 bg-black/80  data-[state=open]:animate-in data-[state=closed]:animate-
out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
        className
      )}
      {...props}
    />
  )
);
DialogOverlay.displayName = DialogPrimitive.Overlay.displayName;

const DialogContent = React.forwardRef(
  ({ className, children, ...props }, ref) => (
    <DialogPortal>
      <DialogOverlay />
      <DialogPrimitive.Content
        ref={ref}
        className={cn(
          "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-
[-50%] gap-4 border bg-background p-6 shadow-lg duration-200 data-[state=open]:animate-in data-
[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-
[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[state=closed]:slide-out-to-left-1/2
data-[state=closed]:slide-out-to-top-[48%] data-[state=open]:slide-in-from-left-1/2 data-
[state=open]:slide-in-from-top-[48%] sm:rounded-lg",
          className
        )}
```

```
        {...props}
      >
        {children}
        <DialogPrimitive.Close className="absolute right-4 top-4 rounded-sm opacity-70 ring-offset-
background transition-opacity hover:opacity-100 focus:outline-none focus:ring-2 focus:ring-ring
focus:ring-offset-2 disabled:pointer-events-none data-[state=open]:bg-accent data-
[state=open]:text-muted-foreground">
          <X className="h-4 w-4" />
          <span className="sr-only">Close</span>
        </DialogPrimitive.Close>
      </DialogPrimitive.Content>
    </DialogPortal>
  )
);
DialogContent.displayName = DialogPrimitive.Content.displayName;

const DialogHeader = ({ className, ...props }) => (
  <div
    className={cn("flex flex-col space-y-1.5 text-center sm:text-left", className)}
    {...props}
  />
);
DialogHeader.displayName = "DialogHeader";

const DialogFooter = ({ className, ...props }) => (
  <div
    className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)}
    {...props}
  />
);
DialogFooter.displayName = "DialogFooter";

const DialogTitle = React.forwardRef(
  ({ className, ...props }, ref) => (
    <DialogPrimitive.Title
      ref={ref}
      className={cn("text-lg font-semibold leading-none tracking-tight", className)}
      {...props}
    />
  )
);
DialogTitle.displayName = DialogPrimitive.Title.displayName;

const DialogDescription = React.forwardRef(
  ({ className, ...props }, ref) => (
    <DialogPrimitive.Description
      ref={ref}
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  )
);
DialogDescription.displayName = DialogPrimitive.Description.displayName;

export {
  Dialog,
  DialogPortal,
  DialogOverlay,
  DialogClose,
  DialogTrigger,
  DialogContent,
  DialogHeader,
  DialogFooter,
  DialogTitle,
  DialogDescription,
};
```

FILE: frontend/src/components/Editor/DropdownMenu.jsx

```
"use client";

import React from "react";
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu";
import { Check, ChevronRight, Circle } from "lucide-react";
import { cn } from "../../lib/utils";
const DropdownMenu = DropdownMenuPrimitive.Root;

const DropdownMenuTrigger = DropdownMenuPrimitive.Trigger;

const DropdownMenuGroup = DropdownMenuPrimitive.Group;

const DropdownMenuPortal = DropdownMenuPrimitive.Portal;

const DropdownMenuSub = DropdownMenuPrimitive.Sub;

const DropdownMenuRadioGroup = DropdownMenuPrimitive.RadioGroup;

const DropdownMenuSubTrigger = React.forwardRef(
  ({ className, inset, children, ...props }, ref) => (
    <DropdownMenuPrimitive.SubTrigger
      ref={ref}
      className={cn(
        "flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none
focus:bg-accent data-[state=open]:bg-accent",
        inset && "pl-8",
        className
      )}
      {...props}
    >
      {children}
      <ChevronRight className="ml-auto h-4 w-4" />
    </DropdownMenuPrimitive.SubTrigger>
  )
);
DropdownMenuSubTrigger.displayName = DropdownMenuPrimitive.SubTrigger.displayName;

const DropdownMenuSubContent = React.forwardRef(
  ({ className, ...props }, ref) => (
    <DropdownMenuPrimitive.SubContent
      ref={ref}
      className={cn(
        "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground
shadow-lg data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-
out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-
[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props}
    />
  )
);
DropdownMenuSubContent.displayName = DropdownMenuPrimitive.SubContent.displayName;

const DropdownMenuContent = React.forwardRef(
  ({ className, sideOffset = 4, ...props }, ref) => (
    <DropdownMenuPrimitive.Portal>
      <DropdownMenuPrimitive.Content
        ref={ref}
        sideOffset={sideOffset}
        className={cn(
          "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-
foreground shadow-md data-[state=open]:animate-in data-[state=closed]:animate-out data-
```

```
[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-
[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-
right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
          className
        )}
        {...props}
      />
    </DropdownMenuPrimitive.Portal>
  )
);
DropdownMenuContent.displayName = DropdownMenuPrimitive.Content.displayName;

const DropdownMenuItem = React.forwardRef(
  ({ className, inset, ...props }, ref) => (
    <DropdownMenuPrimitive.Item
      ref={ref}
      className={cn(
        "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm
outline-none transition-colors focus:bg-accent focus:text-accent-foreground data-
[disabled]:pointer-events-none data-[disabled]:opacity-50",
        inset && "pl-8",
        className
      )}
      {...props}
    />
  )
);
DropdownMenuItem.displayName = DropdownMenuPrimitive.Item.displayName;

const DropdownMenuCheckboxItem = React.forwardRef(
  ({ className, children, checked, ...props }, ref) => (
    <DropdownMenuPrimitive.CheckboxItem
      ref={ref}
      className={cn(
        "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm
outline-none transition-colors focus:bg-accent focus:text-accent-foreground data-
[disabled]:pointer-events-none data-[disabled]:opacity-50",
        className
      )}
      checked={checked}
      {...props}
    >
      <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
        <DropdownMenuPrimitive.ItemIndicator>
          <Check className="h-4 w-4" />
        </DropdownMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </DropdownMenuPrimitive.CheckboxItem>
  )
);
DropdownMenuCheckboxItem.displayName =
  DropdownMenuPrimitive.CheckboxItem.displayName;

const DropdownMenuRadioItem = React.forwardRef(
  ({ className, children, ...props }, ref) => (
    <DropdownMenuPrimitive.RadioItem
      ref={ref}
      className={cn(
        "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm
outline-none transition-colors focus:bg-accent focus:text-accent-foreground data-
[disabled]:pointer-events-none data-[disabled]:opacity-50",
        className
      )}
      {...props}
    >
```

```
      <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
        <DropdownMenuPrimitive.ItemIndicator>
          <Circle className="h-2 w-2 fill-current" />
        </DropdownMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </DropdownMenuPrimitive.RadioItem>
  )
);
DropdownMenuRadioItem.displayName = DropdownMenuPrimitive.RadioItem.displayName;

const DropdownMenuLabel = React.forwardRef(
  ({ className, inset, ...props }, ref) => (
    <DropdownMenuPrimitive.Label
      ref={ref}
      className={cn(
        "px-2 py-1.5 text-sm font-semibold",
        inset && "pl-8",
        className
      )}
      {...props}
    />
  )
);
DropdownMenuLabel.displayName = DropdownMenuPrimitive.Label.displayName;

const DropdownMenuSeparator = React.forwardRef(
  ({ className, ...props }, ref) => (
    <DropdownMenuPrimitive.Separator
      ref={ref}
      className={cn("-mx-1 my-1 h-px bg-muted", className)}
      {...props}
    />
  )
);
DropdownMenuSeparator.displayName = DropdownMenuPrimitive.Separator.displayName;

const DropdownMenuShortcut = ({
  className,
  ...props
}) => {
  return (
    <span
      className={cn("ml-auto text-xs tracking-widest opacity-60", className)}
      {...props}
    />
  );
};
DropdownMenuShortcut.displayName = "DropdownMenuShortcut";

export {
  DropdownMenu,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuCheckboxItem,
  DropdownMenuRadioItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuShortcut,
  DropdownMenuGroup,
  DropdownMenuPortal,
  DropdownMenuSub,
  DropdownMenuSubContent,
  DropdownMenuSubTrigger,
  DropdownMenuRadioGroup,
```

```
};
```

FILE: frontend/src/components/Editor/Form.jsx

```
import React from "react";
import { Slot } from "@radix-ui/react-slot";
import {
  Controller,
  FormProvider,
  useFormContext,
} from "react-hook-form";
import { cn } from "../../lib/utils";
import { Label } from "./Label";
const Form = FormProvider;

const FormFieldContext = React.createContext({});

const FormField = ({
  ...props
}) => {
  return (
    <FormFieldContext.Provider value={{ name: props.name }}>
      <Controller {...props} />
    </FormFieldContext.Provider>
  );
};

const useFormField = () => {
  const fieldContext = React.useContext(FormFieldContext);
  const itemContext = React.useContext(FormItemContext);
  const { getFieldState, formState } = useFormContext();

  const fieldState = getFieldState(fieldContext.name, formState);

  if (!fieldContext) {
    throw new Error("useFormField should be used within <FormField>");
  }

  const { id } = itemContext;

  return {
    id,
    name: fieldContext.name,
    formItemId: `${id}-form-item`,
    formDescriptionId: `${id}-form-item-description`,
    formMessageId: `${id}-form-item-message`,
    ...fieldState,
  };
};

const FormItemContext = React.createContext({});

const FormItem = React.forwardRef(({ className, ...props }, ref) => {
  const id = React.useId();

  return (
    <FormItemContext.Provider value={{ id }}>
      <div ref={ref} className={cn("space-y-2", className)} {...props} />
    </FormItemContext.Provider>
  );
});
FormItem.displayName = "FormItem";

const FormLabel = React.forwardRef(({ className, ...props }, ref) => {
  const { error, formItemId } = useFormField();
```

```
  return (
    <Label
      ref={ref}
      className={cn(error && "text-destructive", className)}
      htmlFor={formItemId}
      {...props}
    />
  );
});
FormLabel.displayName = "FormLabel";

const FormControl = React.forwardRef(({ ...props }, ref) => {
  const { error, formItemId, formDescriptionId, formMessageId } = useFormField();

  return (
    <Slot
      ref={ref}
      id={formItemId}
      aria-describedby={
        !error ? `${formDescriptionId}` : `${formDescriptionId} ${formMessageId}`
      }
      aria-invalid={!!error}
      {...props}
    />
  );
});
FormControl.displayName = "FormControl";

const FormDescription = React.forwardRef(({ className, ...props }, ref) => {
  const { formDescriptionId } = useFormField();

  return (
    <p
      ref={ref}
      id={formDescriptionId}
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  );
});
FormDescription.displayName = "FormDescription";

const FormMessage = React.forwardRef(({ className, children, ...props }, ref) => {
  const { error, formMessageId } = useFormField();
  const body = error ? String(error?.message) : children;

  if (!body) {
    return null;
  }

  return (
    <p
      ref={ref}
      id={formMessageId}
      className={cn("text-sm font-medium text-destructive", className)}
      {...props}
    >
      {body}
    </p>
  );
});
FormMessage.displayName = "FormMessage";

export {
  useFormField,
  Form,
```

```
  FormItem,
  FormLabel,
  FormControl,
  FormDescription,
  FormMessage,
  FormField,
};
```

FILE: frontend/src/components/Editor/ImageComparison.jsx

```
import {
  ReactCompareSlider,
  ReactCompareSliderImage,
} from "react-compare-slider"

export default function ImageComparison({ layers }) {
  if (layers.length === 0) {
    return <div>No layers selected for comparison</div>
  }

  if (layers.length === 1) {
    return (
      <div className="h-full ">
        <ReactCompareSliderImage
          src={layers[0].url || ""}
          srcSet={layers[0].url || ""}
          alt={layers[0].name || "Single image"}
          className=" rounded-lg object-contain"
        />
      </div>
    )
  }

  return (
    <ReactCompareSlider
      className=" "
      itemOne={
        <ReactCompareSliderImage
          src={layers[0].url || ""}
          srcSet={layers[0].url || ""}
          alt={layers[0].name || "Image one"}
        />
      }
      itemTwo={
        <ReactCompareSliderImage
          src={layers[1].url || ""}
          srcSet={layers[1].url || ""}
          alt={layers[1].name || "Image two"}
        />
      }
    />
  )
}
```

FILE: frontend/src/components/Editor/Input.jsx

```
import React from "react";
import { cn } from "../../lib/utils";
const Input = React.forwardRef(({ className, type, ...props }, ref) => {
  return (
    <input
      type={type}
      className={cn(
        "flex h-10 w-full rounded-md border border-input bg-background px-3 py-2 text-sm ring-
offset-background file:border-0 file:bg-transparent file:text-sm file:font-medium placeholder:text-
```

```
muted-foreground focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-
visible:ring-offset-2 disabled:cursor-not-allowed disabled:opacity-50",
        className
      )}
      ref={ref}
      {...props}
    />
  );
});
Input.displayName = "Input";

export { Input };
```

FILE: frontend/src/components/Editor/Label.jsx

```
import React from "react";
import * as LabelPrimitive from "@radix-ui/react-label";
import { cva } from "class-variance-authority";
import { cn } from "../../lib/utils";
const labelVariants = cva(
  "text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
);

const Label = React.forwardRef(({ className, ...props }, ref) => (
  <LabelPrimitive.Root
    ref={ref}
    className={cn(labelVariants(), className)}
    {...props}
  />
));

Label.displayName = LabelPrimitive.Root.displayName;

export { Label };
```

FILE: frontend/src/components/Editor/LayerImage.jsx

```
export default function LayerImage({ layer }) {
  if (layer.url) {
    return (
      <>
        <div className="w-12 h-12 flex items-center justify-center">
          <img
            className="w-full object-contain h-full rounded-sm"
            alt="layer"
            src={layer.format === "mp4" ? layer.poster || layer.url : layer.url}
            width={50}
            height={50}
          />
        </div>
        <div className="relative">
          <p className="text-xs">{`${layer.name?.slice(0, 15)}.${layer.format}`}</p>
        </div>
      </>
    );
  }

  return null; // Return null if no layer.url is available
}
```

FILE: frontend/src/components/Editor/Popover.jsx

```
import React from "react";
import * as PopoverPrimitive from "@radix-ui/react-popover";
```

```
import { cn } from "../../lib/utils";
const Popover = PopoverPrimitive.Root;

const PopoverTrigger = PopoverPrimitive.Trigger;

const PopoverContent = React.forwardRef(({ className, align = "start", side = "right", sideOffset =
4, ...props }, ref) => (
  <PopoverPrimitive.Portal>
    <PopoverPrimitive.Content
      ref={ref}
      align={align}
      side={side}
      sideOffset={sideOffset}
      className={cn(
        "z-50 w-72 rounded-md border bg-popover p-6 text-popover-foreground shadow-md outline-none
data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-
[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-
[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-
in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props}
    />
  </PopoverPrimitive.Portal>
));

PopoverContent.displayName = PopoverPrimitive.Content.displayName;

export { Popover, PopoverTrigger, PopoverContent };
```

FILE: frontend/src/components/Editor/RadioGroup.jsx

```
import React from "react";
import * as RadioGroupPrimitive from "@radix-ui/react-radio-group";
import { Circle } from "lucide-react";
import { cn } from "../../lib/utils";
const RadioGroup = React.forwardRef(({ className, ...props }, ref) => {
  return (
    <RadioGroupPrimitive.Root
      className={cn("grid gap-2", className)}
      {...props}
      ref={ref}
    />
  );
});
RadioGroup.displayName = RadioGroupPrimitive.Root.displayName;

const RadioGroupItem = React.forwardRef(({ className, ...props }, ref) => {
  return (
    <RadioGroupPrimitive.Item
      ref={ref}
      className={cn(
        "aspect-square h-4 w-4 rounded-full border border-primary text-primary ring-offset-
background focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-
offset-2 disabled:cursor-not-allowed disabled:opacity-50",
        className
      )}
      {...props}
    >
      <RadioGroupPrimitive.Indicator className="flex items-center justify-center">
        <Circle className="h-2.5 w-2.5 fill-current text-current" />
      </RadioGroupPrimitive.Indicator>
    </RadioGroupPrimitive.Item>
  );
});
RadioGroupItem.displayName = RadioGroupPrimitive.Item.displayName;
```

```
export { RadioGroup, RadioGroupItem };
```

FILE: frontend/src/components/Editor/Tooltip.jsx

```
import React from "react";
import * as TooltipPrimitive from "@radix-ui/react-tooltip";
import { cn } from "../../lib/utils";

const TooltipProvider = TooltipPrimitive.Provider;
const Tooltip = TooltipPrimitive.Root;
const TooltipTrigger = TooltipPrimitive.Trigger;

const TooltipContent = React.forwardRef(({ className, sideOffset = 4, ...props }, ref) => (
  <TooltipPrimitive.Content
    ref={ref}
    sideOffset={sideOffset}
    className={cn(
      "z-50 overflow-hidden rounded-md border bg-popover px-3 py-1.5 text-sm text-popover-
foreground shadow-md animate-in fade-in-0 zoom-in-95 data-[state=closed]:animate-out data-
[state=closed]:fade-out-0 data-[state=closed]:zoom-out-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-
[side=top]:slide-in-from-bottom-2",
      className
    )}
    {...props}
  />
));
TooltipContent.displayName = TooltipPrimitive.Content.displayName;

export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider };
```

FILE: frontend/src/components/FeaturedProducts.jsx

```
import { useEffect, useState } from "react";
import { ShoppingCart, ChevronLeft, ChevronRight } from "lucide-react";
import { useCartStore } from "../stores/useCartStore";
import { Link } from "react-router-dom"; // Import Link for navigation

const FeaturedProducts = ({ featuredProducts }) => {
  const [currentIndex, setCurrentIndex] = useState(0);
  const [itemsPerPage, setItemsPerPage] = useState(4);

  const { addToCart } = useCartStore();

  useEffect(() => {
    const handleResize = () => {
      if (window.innerWidth < 640) setItemsPerPage(1);
      else if (window.innerWidth < 1024) setItemsPerPage(2);
      else if (window.innerWidth < 1280) setItemsPerPage(3);
      else setItemsPerPage(4);
    };

    handleResize();
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  // Filter the featured products based on 'isFeatured' attribute from the Product model
  const filteredFeaturedProducts = featuredProducts.filter((product) => product.isFeatured);

  const nextSlide = () => {
    setCurrentIndex((prevIndex) => prevIndex + itemsPerPage);
  };
```

```
  const prevSlide = () => {
    setCurrentIndex((prevIndex) => prevIndex - itemsPerPage);
  };

  const isStartDisabled = currentIndex === 0;
  const isEndDisabled = currentIndex >= filteredFeaturedProducts.length - itemsPerPage;

  return (
    <div className="py-12">
      <div className="container mx-auto px-4">
        <h2 className="text-center text-5xl sm:text-6xl font-bold text-emerald-400 mb-4">Featured
Products</h2>
        <div className="relative">
          <div className="overflow-hidden">
            <div
              className="flex transition-transform duration-300 ease-in-out"
              style={{ transform: `translateX(-${currentIndex * (100 / itemsPerPage)}%)` }}
            >
              {/* Only display featured products */}
              {filteredFeaturedProducts?.map((product) => (
                <div key={product.id} className="w-full sm:w-1/2 lg:w-1/3 xl:w-1/4 flex-shrink-0
px-2">
                  <div className="bg-white bg-opacity-10 backdrop-blur-sm rounded-lg shadow-lg
overflow-hidden h-full transition-all duration-300 hover:shadow-xl border border-emerald-500/30">
                    <div className="overflow-hidden">
                      <Link to={`/product/${product.id}`}> {/* Link to product detail page */}
                        <img
                          src={product.image} // Ensure this field is properly populated
                          alt={product.name}
                          className="w-full h-48 object-cover transition-transform duration-300
ease-in-out hover:scale-110"
                        />
                      </Link>
                    </div>
                    <div className="p-4">
                      <Link to={`/product/${product.id}`}> {/* Link to product detail page */}
                        <h3 className="text-lg font-semibold mb-2 text-white">{product.name}</h3>
                      </Link>
                      <p className="text-emerald-300 font-medium mb-4">
                        ${(Number(product.price) || 0).toFixed(2)}
                      </p>
                      <button
                        onClick={() => addToCart(product.id)}
                        className="w-full bg-emerald-600 hover:bg-emerald-500 text-white font-
semibold py-2 px-4 rounded transition-colors duration-300
                                                flex items-center justify-center"
                      >
                        <ShoppingCart className="w-5 h-5 mr-2" />
                        Add to Cart
                      </button>
                    </div>
                  </div>
                </div>
              ))}
            </div>
          </div>
          {/* Left Slide Button */}
          <button
            onClick={prevSlide}
            disabled={isStartDisabled}
            className={`absolute top-1/2 -left-4 transform -translate-y-1/2 p-2 rounded-full
transition-colors duration-300 ${
              isStartDisabled ? "bg-gray-400 cursor-not-allowed" : "bg-emerald-600 hover:bg-
emerald-500"
            }`}
          >
```

```
            <ChevronLeft className="w-6 h-6" />
          </button>

          {/* Right Slide Button */}
          <button
            onClick={nextSlide}
            disabled={isEndDisabled}
            className={`absolute top-1/2 -right-4 transform -translate-y-1/2 p-2 rounded-full
transition-colors duration-300 ${
              isEndDisabled ? "bg-gray-400 cursor-not-allowed" : "bg-emerald-600 hover:bg-emerald-
500"
            }`}
          >
            <ChevronRight className="w-6 h-6" />
          </button>
        </div>
      </div>
    </div>
  );
};

export default FeaturedProducts;
```

FILE: frontend/src/components/FeedBackForm/FeedbackFormQuestions.jsx

```
import { PlusIcon } from "@heroicons/react/24/outline";
import { useEffect, useState } from "react";
import { v4 as uuidv4 } from "uuid";
import QuestionEditor from "./QuestionEditor";

export default function FeedbackFormQuestions({ questions, onQuestionsUpdate }) {
  const [myQuestions, setMyQuestions] = useState([...questions]);

  const addQuestion = (index) => {
    index = index !== undefined ? index : myQuestions.length;
    myQuestions.splice(index, 0, {
      id: uuidv4(),
      type: "text",
      question: "",
      description: "",
      data: {},
    });
    setMyQuestions([...myQuestions]);
    onQuestionsUpdate(myQuestions);
  };

  const questionChange = (question) => {
    if (!question) return;
    const newQuestions = myQuestions.map((q) => {
      if (q.id === question.id) {
        return { ...question };
      }
      return q;
    });
    setMyQuestions(newQuestions);
    onQuestionsUpdate(newQuestions);
  };

  const deleteQuestion = (question) => {
    const newQuestions = myQuestions.filter((q) => q.id !== question.id);

    setMyQuestions(newQuestions);
    onQuestionsUpdate(newQuestions);
  };

  useEffect(() => {
```

```
    setMyQuestions(questions);
  }, [questions]);

  return (
    <>
      <div className="flex justify-between text-emerald-400 border-2 border-white p-2">
        <h3 className="text-2xl font-bold">Feedback Form Questions</h3>
        <button
          type="button"
          className="flex items-center text-sm py-1 px-4 rounded-sm text-emerald-400 bg-none
border-2 border-white hover:bg-emerald-400 hover:text-white"
          onClick={() => addQuestion()}
        >
          <PlusIcon className="w-5 mr-2" />
          Add Question
        </button>
      </div>
      {myQuestions.length ? (
        myQuestions.map((q, ind) => (
          <QuestionEditor
            key={q.id}
            index={ind}
            question={q}
            questionChange={questionChange}
            addQuestion={addQuestion}
            deleteQuestion={deleteQuestion}
          />
        ))
      ) : (
        <div className="text-emerald-400 text-center py-4">
          You don't have any questions created for this feedback form.
        </div>
      )}
    </>
  );
}
```

FILE: frontend/src/components/FeedBackForm/PublicQuestionView.jsx

```
export default function PublicQuestionView({
  question,
  index,
  answerChanged,
}) {
  let selectedOptions = [];

  function onCheckboxChange(option, $event) {
    if ($event.target.checked) {
      selectedOptions.push(option.text);
    } else {
      selectedOptions = selectedOptions.filter(op => op !== option.text);
    }
    answerChanged(selectedOptions);
  }

  console.log(question);
  return (
    <>
      <fieldset className="mb-6 border-2 border-white text-emerald-400 p-6 rounded-lg">
        <div>
          <legend className="text-base font-medium text-emerald-400">
            {index + 1}. {question.question}
          </legend>
          <p className="text-sm text-emerald-400">{question.description}</p>
        </div>
```

```
        <div className="mt-4">
          {question.type === "select" && (
            <div>
              <select
                onChange={(ev) => answerChanged(ev.target.value)}
                className="mt-2 block w-full py-3 px-4 border-2 border-white text-emerald-400
rounded-md focus:outline-none focus:ring-emerald-500 focus:border-emerald-500 sm:text-sm"
              >
                <option value="">Please Select</option>
                {question.data.options.map((option) => (
                  <option key={option.uuid} value={option.text}>
                    {option.text}
                  </option>
                ))}
              </select>
            </div>
          )}
          {question.type === "radio" && (
            <div>
              {question.data.options.map((option) => (
                <div key={option.uuid} className="flex items-center">
                  <input
                    id={option.uuid}
                    name={"question" + question.id}
                    value={option.text}
                    onChange={(ev) => answerChanged(ev.target.value)}
                    type="radio"
                    className="focus:ring-emerald-500 h-4 w-4 text-emerald-600 border-2 border-
white rounded-full"
                  />
                  <label
                    htmlFor={option.uuid}
                    className="ml-3 block text-sm font-medium text-emerald-400"
                  >
                    {option.text}
                  </label>
                </div>
              ))}
            </div>
          )}
          {question.type === "checkbox" && (
            <div>
              {question.data.options.map((option) => (
                <div key={option.uuid} className="flex items-center">
                  <input
                    id={option.uuid}
                    onChange={(ev) => onCheckboxChange(option, ev)}
                    type="checkbox"
                    className="focus:ring-emerald-500 h-4 w-4 text-emerald-600 border-2 border-
white rounded"
                  />
                  <label
                    htmlFor={option.uuid}
                    className="ml-3 block text-sm font-medium text-emerald-400"
                  >
                    {option.text}
                  </label>
                </div>
              ))}
            </div>
          )}
          {question.type === "text" && (
            <div>
              <input
                type="text"
                onChange={(ev) => answerChanged(ev.target.value)}
```

```
                className="mt-2 focus:ring-emerald-500 focus:border-emerald-500 block w-full py-3
px-4 shadow-sm sm:text-sm border-2 border-white rounded-md text-emerald-400"
              />
            </div>
          )}
          {question.type === "textarea" && (
            <div>
              <textarea
                onChange={(ev) => answerChanged(ev.target.value)}
                className="mt-2 focus:ring-emerald-500 focus:border-emerald-500 block w-full py-3
px-4 shadow-sm sm:text-sm border-2 border-white rounded-md text-emerald-400"
              ></textarea>
            </div>
          )}
        </div>
      </fieldset>
      <hr className="mb-6 border-2 border-white rounded-full" />
    </>
  );
}
```

FILE: frontend/src/components/FeedBackForm/QuestionEditor.jsx

```
import { v4 as uuidv4 } from "uuid";
import { PlusIcon, TrashIcon } from "@heroicons/react/24/outline";
import { useEffect, useState } from "react";
import { useFeedbackFormStore } from "../../stores/useFeedbackFormStore";

export default function QuestionEditor({
  index = 0,
  question,
  addQuestion,
  deleteQuestion,
  questionChange,
}) {
  const { questionTypes } = useFeedbackFormStore();
  const [model, setModel] = useState({ ...question });

  useEffect(() => {
    questionChange(model);
  }, [model]);

  function upperCaseFirst(str) {
    return str.charAt(0).toUpperCase() + str.slice(1);
  }

  function shouldHaveOptions(type = null) {
    type = type || model.type;
    return ["select", "radio", "checkbox"].includes(type);
  }

  function onTypeChange(ev) {
    const newModel = {
      ...model,
      type: ev.target.value,
    };
    if (!shouldHaveOptions(model.type) && shouldHaveOptions(ev.target.value)) {
      if (!model.data.options) {
        newModel.data = {
          options: [{ uuid: uuidv4(), text: "" }],
        };
      }
    }
    setModel(newModel);
  }
```

```
  function addOption() {
    model.data.options.push({
      uuid: uuidv4(),
      text: "",
    });
    setModel({ ...model });
  }

  function deleteOption(op) {
    model.data.options = model.data.options.filter(option => option.uuid !== op.uuid);
    setModel({ ...model });
  }

  return (
    <>
      <div className="text-emerald-400 border-2 border-white p-3 mb-4">
        <div className="flex justify-between mb-3">
          <h4 className="text-emerald-400">
            {index + 1}. {model.question}
          </h4>
          <div className="flex items-center">
            <button
              type="button"
              className="flex items-center text-xs py-1 px-3 mr-2 rounded-sm text-emerald-400 bg-
none border-2 border-white hover:bg-emerald-400 hover:text-white"
              onClick={() => addQuestion(index + 1)}
            >
              <PlusIcon className="w-4" />
              Add
            </button>
            <button
              type="button"
              className="flex items-center text-xs py-1 px-3 rounded-sm border-2 border-white text-
red-500 hover:border-red-600 font-semibold"
              onClick={() => deleteQuestion(question)}
            >
              <TrashIcon className="w-4" />
              Delete
            </button>
          </div>
        </div>

        <div className="flex gap-3 justify-between mb-3">
          {/* Question Text */}
          <div className="flex-1">
            <label
              htmlFor="question"
              className="block text-sm font-medium text-emerald-400"
            >
              Question
            </label>
            <input
              type="text"
              name="question"
              id="question"
              value={model.question}
              onChange={(ev) =>
                setModel({ ...model, question: ev.target.value })
              }
              className="mt-1 block w-full rounded-md border-2 border-white text-emerald-400 bg-
none shadow-sm focus:border-indigo-500 focus:ring-indigo-500 sm:text-sm"
            />
          </div>

          {/* Question Type */}
          <div>
```

```
            <label
              htmlFor="questionType"
              className="block text-sm font-medium text-emerald-400 w-40"
            >
              Question Type
            </label>
            <select
              id="questionType"
              name="questionType"
              value={model.type}
              onChange={onTypeChange}
              className="mt-1 block w-full rounded-md border-2 border-white text-emerald-400 bg-
none py-2 px-3 shadow-sm focus:border-indigo-500 focus:outline-none focus:ring-indigo-500 sm:text-
sm"
            >
              {questionTypes.map((type) => (
                <option value={type} key={type}>
                  {upperCaseFirst(type)}
                </option>
              ))}
            </select>
          </div>
        </div>

        {/* Description */}
        <div className="mb-3">
          <label
            htmlFor="questionDescription"
            className="block text-sm font-medium text-emerald-400"
          >
            Description
          </label>
          <textarea
            name="questionDescription"
            id="questionDescription"
            value={model.description || ""}
            onChange={(ev) =>
              setModel({ ...model, description: ev.target.value })
            }
            className="mt-1 block w-full rounded-md border-2 border-white text-emerald-400 bg-none
shadow-sm focus:border-indigo-500 focus:ring-indigo-500 sm:text-sm"
          ></textarea>
        </div>

        {/* Options */}
        <div>
          {shouldHaveOptions() && (
            <div>
              <h4 className="text-sm font-semibold mb-1 flex justify-between items-center text-
emerald-400">
                Options
                <button
                  onClick={addOption}
                  type="button"
                  className="flex items-center text-xs py-1 px-2 rounded-sm text-emerald-400 bg-
none border-2 border-white hover:bg-emerald-400 hover:text-white"
                >
                  Add
                </button>
              </h4>

              {model.data.options.length === 0 && (
                <div className="text-xs text-gray-600 text-center py-3">
                  You don't have any options defined
                </div>
              )}
```

```
              {model.data.options.length > 0 && (
                <div>
                  {model.data.options.map((op, ind) => (
                    <div key={op.uuid} className="flex items-center mb-1">
                      <span className="w-6 text-sm text-emerald-400">{ind + 1}.</span>
                      <input
                        type="text"
                        value={op.text}
                        onInput={(ev) => {
                          op.text = ev.target.value;
                          setModel({ ...model });
                        }}
                        className="w-full rounded-sm py-1 px-2 text-xs border-2 border-white text-
emerald-400 bg-none focus:border-indigo-500"
                      />
                      <button
                        onClick={ev => deleteOption(op)}
                        type="button"
                        className="h-6 w-6 rounded-full flex items-center justify-center border-2
border-white hover:border-red-100"
                      >
                        <TrashIcon className="w-3 h-3 text-red-500" />
                      </button>
                    </div>
                  ))}
                </div>
              )}
            </div>
          )}
        </div>
      </div>
      <hr className="border-2 border-white" />
    </>
  );
}
```

FILE: frontend/src/components/GiftCouponCard.jsx

```
import { motion } from "framer-motion";
import { useEffect, useState } from "react";
import { useCartStore } from "../stores/useCartStore";

const GiftCouponCard = () => {
  const { userCoupons, isCouponApplied, applyCoupon, getMyCoupons } = useCartStore();
  const [selectedCoupon, setSelectedCoupon] = useState(null); // Track the selected coupon

  useEffect(() => {
    getMyCoupons(); // Fetch the list of coupons when the component mounts
  }, [getMyCoupons]);

  const handleApplyCoupon = (couponId) => {
    if (couponId) {
      console.log(couponId);
      applyCoupon(couponId); // Apply the selected coupon
      setSelectedCoupon(userCoupons.find(coupon => coupon.id === couponId)); // Set the selected
coupon
    }
  };

  return (
    <motion.div
      className="space-y-4 rounded-lg border border-gray-700 bg-gray-800 p-4 shadow-sm sm:p-6"
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.5, delay: 0.2 }}
    >
```

```
      <div className="space-y-4">
        <label htmlFor="voucher" className="mb-2 block text-sm font-medium text-gray-300">
          Select a Coupon:
        </label>

        <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">
          {userCoupons.map((coupon) => (
            <motion.div
              key={coupon.id}
              className={`cursor-pointer rounded-lg border p-4 text-center ${
                selectedCoupon?.id === coupon.id ? 'bg-emerald-600 border-emerald-500' : 'bg-gray-
700 border-gray-600'
              }`}
              onClick={() => handleApplyCoupon(coupon.id)}
              whileHover={{ scale: 1.05 }}
              whileTap={{ scale: 0.95 }}
            >
              <h4 className="text-lg font-medium text-white">{coupon.code}</h4>
              <p className="mt-2 text-sm text-gray-400">{coupon.discount}% off</p>
            </motion.div>
          ))}
        </div>
      </div>

      {/* Optionally display the applied coupon if one is applied */}
      {isCouponApplied && selectedCoupon && (
        <div className="mt-4">
          <h3 className="text-lg font-medium text-gray-300">Applied Coupon</h3>
          <p className="mt-2 text-sm text-gray-400">
            {selectedCoupon.code} - {selectedCoupon.discount}% off
          </p>
        </div>
      )}
    </motion.div>
  );
};

export default GiftCouponCard;
```

FILE: frontend/src/components/LiveStream/Navbar.jsx

```
import React from "react";
import { useAppDispatch, useAppSelector } from "../../pages/State/Redux";
import { setIsSidebarCollapsed } from "../../pages/State/State";
import { Menu } from "lucide-react";

const Navbar = () => {
  const dispatch = useAppDispatch();
  const isSidebarCollapsed = useAppSelector(
    (state) => state.global.isSidebarCollapsed
  );
  const toggleSidebar = () => {
    dispatch(setIsSidebarCollapsed(!isSidebarCollapsed));
  };

  return (
    <div className="flex justify-between items-center w-full mb-7">
      {/* LEFT SIDE */}
      <div className="flex justify-between items-center gap-5">
        <button
          className="px-3 py-3 bg-emerald-400 rounded-full hover:bg-emerald-200"
          onClick={toggleSidebar}
        >
          <Menu className="w-4 h-4" />
        </button>
      </div>
```

```
    </div>
  );
};

export default Navbar;
```

FILE: frontend/src/components/LiveStream/Sidebar.jsx

```
import React, { useEffect } from "react";
import { useAppDispatch, useAppSelector } from "../../pages/State/Redux";
import { setIsSidebarCollapsed } from "../../pages/State/State";
import { Menu } from "lucide-react";
import useLiveStreamStore from "../../stores/useLiveStreamStore";

const Sidebar = ({ onStreamSelect, onCreateStreamClick }) => {
  const dispatch = useAppDispatch();
  const isSidebarCollapsed = useAppSelector(
    (state) => state.global.isSidebarCollapsed
  );

  const { streams, getStreams } = useLiveStreamStore();

  useEffect(() => {
    getStreams();
  }, [getStreams]);

  const toggleSidebar = () => {
    dispatch(setIsSidebarCollapsed(!isSidebarCollapsed));
  };

  // Class names with transparent background and white border
  const sidebarClassNames = `fixed flex flex-col ${
    isSidebarCollapsed ? "w-0 md:w-16" : "w-72 md:w-64"
  } bg-transparent border-2 border-white transition-all duration-300 overflow-hidden h-full z-40`;

  const handleStreamClick = (stream) => {
    onStreamSelect(stream);
  };

  return (
    <div className={sidebarClassNames}>
      {/* TOP LOGO */}
      <div
        className={`flex gap-3 justify-between md:justify-normal items-center pt-8 ${
          isSidebarCollapsed ? "px-5" : "px-8"
        }`}
      >
        <button
          className="md:hidden px-3 py-3 bg-gray-100 rounded-full hover:bg-gray-900"
          onClick={toggleSidebar}
        >
          <Menu className="w-4 h-4" />
        </button>
      </div>

      {/* Create Stream Button */}
      <div className={`mt-6 ${isSidebarCollapsed ? "hidden" : "block"} px-4`}>
        <button
          className="w-full bg-emerald-400 text-white py-2 rounded-md"
          onClick={onCreateStreamClick}
        >
          Create Stream
        </button>
      </div>

      {/* Stream List Section */}
```

```
      <div className={`mt-6 ${isSidebarCollapsed ? "hidden" : "block"} overflow-y-auto`}>
        <h3 className="text-white font-bold text-lg mb-4 px-4">Active Streams</h3>
        {streams && streams.length > 0 ? (
          <ul className="space-y-3 px-4 max-h-[calc(100vh-120px)] overflow-y-auto">
            {streams.map((stream) => (
              <li
                key={stream.id}
                className="flex items-center text-white cursor-pointer hover:bg-gray-600 py-2 px-3
rounded-lg"
                onClick={() => handleStreamClick(stream)}
              >
                <img
                  src={stream.thumbnail}
                  alt={stream.title}
                  className="w-12 h-12 rounded-full object-cover"
                />
                <div className="ml-3">
                  <p className="font-semibold">{stream.title}</p>
                  <p className="text-sm text-gray-200">
                    {stream.user.first_name} {stream.user.last_name}
                  </p>
                </div>
              </li>
            ))}
          </ul>
        ) : (
          <div className="text-white text-sm text-center">No active streams</div>
        )}
      </div>
    </div>
  );
};

export default Sidebar;
```

FILE: frontend/src/components/LoadingSpinner.jsx

```
const LoadingSpinner = () => {
    return (
        <div className='flex items-center justify-center min-h-screen bg-gray-900'>
            <div className='relative'>
                <div className='w-20 h-20 border-emerald-200 border-2 rounded-full' />
                <div className='w-20 h-20 border-emerald-500 border-t-2 animate-spin rounded-full
absolute left-0 top-0' />
                <div className='sr-only'>Loading</div>
            </div>
        </div>
    );
};

export default LoadingSpinner;
```

FILE: frontend/src/components/Map/Navbar.jsx

```
import React from "react";
import { useAppDispatch, useAppSelector } from "../../pages/State/Redux";
import { setIsSidebarCollapsed } from "../../pages/State/State";
import { Menu } from "lucide-react";
const Navbar = () => {
  const dispatch = useAppDispatch();
  const isSidebarCollapsed = useAppSelector(
    (state) => state.global.isSidebarCollapsed
  );
  const toggleSidebar = () => {
    dispatch(setIsSidebarCollapsed(!isSidebarCollapsed));
```

```
  };

  return (
    <div className="flex justify-between items-center w-full mb-7">
      {/* LEFT SIDE */}
      <div className="flex justify-between items-center gap-5">
        <button
          className="px-3 py-3 bg-emerald-400 rounded-full hover:bg-emerald-200"
          onClick={toggleSidebar}
        >
          <Menu className="w-4 h-4" />
        </button>
      </div>
    </div>
  );
};

export default Navbar;
```

FILE: frontend/src/components/Map/Sidebar.jsx

```
import React from "react";
import { Menu } from "lucide-react"; // Assuming you're using Menu for the sidebar button

const Sidebar = ({ shipments, isLoading, errorMessage, setSelectedShipment }) => {
  return (
    <div className="flex flex-col transition-all duration-300 shadow-md z-40">
      {/* TOP LOGO */}
      <div className="flex gap-3 justify-between md:justify-normal items-center pt-8 px-8">
        <button
          className="md:hidden px-3 py-3 bg-gray-100 rounded-full hover:bg-gray-900"
          onClick={() => {}}
        >
          <Menu className="w-4 h-4" />
        </button>
      </div>

      {/* Shipments List (Scrollable part) */}
      <div className="mt-4 px-6 flex-1 overflow-y-auto"> {/* Make this part scrollable */}
        {/* Show loading indicator or error if available */}
        {isLoading ? (
          <p className="text-white">Loading shipments...</p>
        ) : errorMessage ? (
          <p className="text-red-500">{errorMessage}</p>
        ) : (
          <ul className="space-y-3 mt-4 text-sm text-gray-200">
            {shipments.length === 0 ? (
              <p className="text-white">No shipments available</p>
            ) : (
              shipments.map((shipment) => (
                <li
                  key={shipment.id}
                  className="hover:bg-emerald-700 p-2 rounded-md transition-all border-2 border-
white"
                  onClick={() => setSelectedShipment(shipment)} // Call the callback when clicked
                >
                  <div className="font-medium">Shipment {shipment.shipment.id}</div>
                  <div>Status: {shipment.shipment.status}</div>
                  <div>Origin: {shipment.shipment.origin}</div>
                  <div>Destination: {shipment.shipment.destination}</div>
                  <div>Estimated Arrival: {shipment.shipment.estimated_arrival}</div>
                  <div>Tracking Number: {shipment.shipment.tracking_number}</div>

                  {/* Route Optimizations */}
                  {shipment.routeOptimizations?.map((routeOptimization) => (
                    <div key={routeOptimization.routeOptimization.id}>
```

```
                        <h4 className="font-medium mt-2">Route Optimization</h4>
                        <div>Distance: {routeOptimization.routeOptimization.total_distance} km</div>

                        {/* Route Details */}
                        {routeOptimization.routeDetails?.map((routeDetail) => (
                          <div key={routeDetail.routeDetail.id} className="mt-2">
                            <h5>Route Detail: {routeDetail.routeDetail.route_name}</h5>
                            <div>Supplier: {routeDetail.routeDetail.supplier_name}</div>
                            <div>Warehouse 1: {routeDetail.routeDetail.warehouse_name_1}</div>
                            <div>Start Location: {routeDetail.routeDetail.start_location}</div>
                            <div>End Location: {routeDetail.routeDetail.end_location}</div>

                            {/* Route Condition */}
                            {routeDetail.routeCondition && (
                              <div>
                                <h6>Route Condition:</h6>
                                <p>Weather: {routeDetail.routeCondition.weather}</p>
                                <p>Road Condition: {routeDetail.routeCondition.road_condition}</p>
                                <p>Traffic Condition: {routeDetail.routeCondition.traffic_condition}
</p>
                                <p>Accident: {routeDetail.routeCondition.has_accident ? "Yes" : "No"}
</p>
                              </div>
                            )}
                          </div>
                        ))}
                      </div>
                    ))}
                  </li>
                ))
              )}
            </ul>
          )}
        </div>
      </div>
    );
};

export default Sidebar;
```

FILE: frontend/src/components/Navbar.jsx

```
import { useState } from "react";
import { Link } from "react-router-dom";
import {
  ShoppingCart,
  UserPlus,
  LogIn,
  LogOut,
  Lock,
  User,
} from "lucide-react";
import Select from "react-select";
import Flag from "react-world-flags";

import { useUserStore } from "../stores/useUserStore";
import { useCartStore } from "../stores/useCartStore";

/* =======================
   NAV CONFIG
======================= */

const NAV_ITEMS = [
  { key: "home", label: "Home", path: "/" },
  { key: "warehouse", label: "Warehouse", path: "/warehouse" },
  { key: "vehicle", label: "Vehicle", path: "/vehicle" },
```

```
  { key: "map", label: "Map", path: "/map" },
  { key: "feedback", label: "Create Feedback Form", path: "/feedback-forms/create" },
  { key: "social", label: "Social Media", path: "/social-media" },
  { key: "notification", label: "Notifications", path: "/notification" },
  { key: "network", label: "Network", path: "/network" },
  { key: "chat", label: "Chat", path: "/chat" },
  { key: "chatbot", label: "Chat Bot", path: "/chatbot" },
  { key: "livestream", label: "Live Stream", path: "/live-stream" },
  { key: "editor", label: "Editor", path: "/editor" },
  { key: "cart", label: "Cart", path: "/cart" },
];

const ROLE_NAV_ACCESS = {
  Administration: NAV_ITEMS.map((i) => i.key),
  WarehouseManager: ["home", "warehouse", "map", "notification"],
  VehicleManager: ["home", "vehicle", "map", "notification"],
  DeliveryDriver: ["home", "vehicle", "map", "notification"],
  DeliveryMan: ["home", "vehicle", "map", "notification"],
  Customer: [
    "home",
    "cart",
    "social",
    "notification",
    "network",
    "chat",
    "chatbot",
    "livestream",
    "editor",
  ],
  ProductSaler: [
    "home",
    "cart",
    "feedback",
    "social",
    "notification",
    "network",
    "chat",
    "chatbot",
    "livestream",
    "editor",
  ],
  CustomerSupportStaff: [
    "home",
    "social",
    "notification",
    "chat",
    "chatbot",
  ],
  FinanceManager: ["home", "cart", "notification", "map"],
  ShipmentManager: ["home", "vehicle", "map", "notification"],
};

/* =======================
   COMPONENT
======================= */

const Navbar = () => {
  const { user, userRoles, checkingAuth, logout, setLanguage } = useUserStore();
  const { cart } = useCartStore();
  const [selectedLanguage, setSelectedLanguage] = useState(null);

  if (checkingAuth) return null;
  const role = userRoles?.[0]?.role_name || 'Customer';
  console.log(role);
  const allowedKeys = ROLE_NAV_ACCESS[role] || ["home"];
```

```
  const visibleNavItems = NAV_ITEMS.filter((item) =>
    allowedKeys.includes(item.key)
  );

  const languageOptions = [
    { value: "en", label: "English", flag: "GB" },
    { value: "es", label: "Spanish", flag: "ES" },
    { value: "fr", label: "French", flag: "FR" },
    { value: "de", label: "German", flag: "DE" },
    { value: "ka", label: "Georgian", flag: "GE" },
  ];

  const handleLanguageChange = (option) => {
    setSelectedLanguage(option);
    setLanguage(option.value);
  };

  return (
    <header className="fixed top-0 left-0 w-full bg-gray-900 border-b border-emerald-800 z-50">
      <div className="container mx-auto px-4 py-3">
        <nav className="flex items-center gap-4 overflow-x-auto no-scrollbar whitespace-nowrap">

          {visibleNavItems.map((item) => (
            <Link
              key={item.key}
              to={item.path}
              className="text-gray-300 hover:text-emerald-400 transition"
            >
              {item.label}
            </Link>
          ))}

          {user && allowedKeys.includes("cart") && (
            <Link
              to="/cart"
              className="relative text-gray-300 hover:text-emerald-400"
            >
              <ShoppingCart size={20} />
              {cart.length > 0 && (
                <span className="absolute -top-2 -left-2 bg-emerald-500 text-white rounded-full px-
2 text-xs">
                  {cart.length}
                </span>
              )}
            </Link>
          )}

          {role === "Administration" && (
            <Link
              to="/secret-dashboard"
              className="bg-emerald-700 hover:bg-emerald-600 text-white px-3 py-1 rounded-md flex
items-center"
            >
              <Lock size={18} className="mr-1" />
              Dashboard
            </Link>
          )}

          {user ? (
            <>
              <Link
                to="/profile"
                className="flex items-center text-gray-300 hover:text-emerald-400"
              >
                <User size={20} className="mr-1" />
                Profile
```

```
              </Link>

              <button
                onClick={logout}
                className="bg-gray-700 hover:bg-gray-600 text-white px-3 py-1 rounded-md flex
items-center"
              >
                <LogOut size={18} />
                <span className="ml-1 hidden sm:inline">Log Out</span>
              </button>
            </>
          ) : (
            <>
              <Link
                to="/signup"
                className="bg-emerald-600 hover:bg-emerald-700 text-white px-3 py-1 rounded-md flex
items-center"
              >
                <UserPlus size={18} className="mr-1" />
                Sign Up
              </Link>

              <Link
                to="/login"
                className="bg-gray-700 hover:bg-gray-600 text-white px-3 py-1 rounded-md flex
items-center"
              >
                <LogIn size={18} className="mr-1" />
                Login
              </Link>
            </>
          )}

          <Select
            value={selectedLanguage}
            onChange={handleLanguageChange}
            options={languageOptions}
            placeholder="Language"
            getOptionLabel={(e) => (
              <div className="flex items-center">
                <Flag code={e.flag} className="w-5 h-5 mr-2" />
                {e.label}
              </div>
            )}
            className="w-36"
            menuPortalTarget={document.body}
            styles={{
              menuPortal: (base) => ({ ...base, zIndex: 9999 }),
            }}
          />
        </nav>
      </div>
    </header>
  );
};

export default Navbar;
```

FILE: frontend/src/components/OrderSummary.jsx

```
import { motion } from "framer-motion";
import { useCartStore } from "../stores/useCartStore";
import { Link, useNavigate } from "react-router-dom";
import { MoveRight } from "lucide-react";
import { useState, useEffect } from "react";
```

```
const OrderSummary = () => {
  const { cart, orderId, totalAmount, discountAmount, totalAfterDiscount, isCouponApplied, coupon,
applyCoupon } = useCartStore();

  const [isProcessing, setIsProcessing] = useState(false); // Track payment processing state
  const navigate = useNavigate(); // Initialize useNavigate hook

  // Format amounts to two decimal places
  const formattedTotal = totalAmount.toFixed(2);
  const formattedDiscountAmount = discountAmount.toFixed(2);
  const formattedTotalAfterDiscount = totalAfterDiscount.toFixed(2);

  // Handle payment (navigate to the payment page)
  const handlePayment = async () => {
    setIsProcessing(true); // Start processing
    // Simulating a delay for payment processing, then navigate to payment page
    setTimeout(() => {
      navigate("/payment"); // Redirect to the payment page
    }, 1500); // Simulate a 1.5s processing time
  };

  // Listen for cart changes and apply coupon if needed
  useEffect(() => {
    // Apply coupon automatically if it's available and not already applied
    if (orderId && cart.length > 0 && !isCouponApplied && coupon) {
      applyCoupon(coupon.id); // Apply coupon if not already applied
    }
  }, [cart, orderId, isCouponApplied, coupon, applyCoupon]);

  return (
    <motion.div
      className="space-y-4 rounded-lg border border-gray-700 bg-gray-800 p-4 shadow-sm sm:p-6"
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.5 }}
    >
      <p className="text-xl font-semibold text-emerald-400">Order summary</p>

      <div className="space-y-4">
        {/* Display Order Items */}
        <div className="space-y-2">
          {cart.map((item) => (
            <dl key={item.product.id} className="flex items-center justify-between gap-4">
              <dt className="text-base font-normal text-gray-300">{item.product.name}</dt>
              <dd className="text-base font-medium text-white">${item.total_amount.toFixed(2)}</dd>
            </dl>
          ))}

          <dl className="flex items-center justify-between gap-4 border-t border-gray-600 pt-2">
            <dt className="text-base font-bold text-white">Total</dt>
            <dd className="text-base font-bold text-emerald-400">${formattedTotal}</dd>
          </dl>

          {/* Conditionally display Discount Amount if Coupon is Applied */}
          {isCouponApplied && (
            <>
              <dl className="flex items-center justify-between gap-4">
                <dt className="text-base font-normal text-gray-300">Discount Applied</dt>
                <dd className="text-base font-medium text-emerald-400">-${formattedDiscountAmount}
</dd>
              </dl>
            </>
          )}

          {/* Display the final total after discount */}
          <dl className="flex items-center justify-between gap-4 border-t border-gray-600 pt-2">
```

```
            <dt className="text-base font-bold text-white">Total After Discount</dt>
            <dd className="text-base font-bold text-emerald-400">${formattedTotalAfterDiscount}
</dd>
          </dl>
        </div>

        {/* Proceed to Checkout Button */}
        <motion.button
          className="flex w-full items-center justify-center rounded-lg bg-emerald-600 px-5 py-2.5
text-sm font-medium text-white hover:bg-emerald-700 focus:outline-none focus:ring-4 focus:ring-
emerald-300"
          whileHover={{ scale: 1.05 }}
          whileTap={{ scale: 0.95 }}
          onClick={handlePayment}
          disabled={isProcessing} // Disable button while processing payment
        >
          {isProcessing ? "Processing..." : "Proceed to Checkout"}
        </motion.button>

        {/* Continue Shopping Link */}
        <div className="flex items-center justify-center gap-2">
          <span className="text-sm font-normal text-gray-400">or</span>
          <Link
            to="/"
            className="inline-flex items-center gap-2 text-sm font-medium text-emerald-400
underline hover:text-emerald-300 hover:no-underline"
          >
            Continue Shopping
            <MoveRight size={16} />
          </Link>
        </div>
      </div>
    </motion.div>
  );
};

export default OrderSummary;
```

FILE: frontend/src/components/PeopleAlsoBought.jsx

```
import { useEffect, useState } from "react";
import ProductCard from "./ProductCard";
import axios from "../lib/axios";
import toast from "react-hot-toast";
import LoadingSpinner from "./LoadingSpinner";

const PeopleAlsoBought = () => {
    const [recommendations, setRecommendations] = useState([]);
    const [isLoading, setIsLoading] = useState(true);

    useEffect(() => {
        const fetchRecommendations = async () => {
            try {
                const res = await axios.get("/products/recommendations");
                setRecommendations(res.data);
            } catch (error) {
                toast.error(error.response.data.message || "An error occurred while fetching
recommendations");
            } finally {
                setIsLoading(false);
            }
        };

        fetchRecommendations();
    }, []);
```

```
    if (isLoading) return <LoadingSpinner />;

    return (
        <div className='mt-8'>
            <h3 className='text-2xl font-semibold text-emerald-400'>People also bought</h3>
            <div className='mt-6 grid grid-cols-1 gap-4 sm:grid-cols-2 lg: grid-col-3'>
                {recommendations.map((product) => (
                    <ProductCard key={product._id} product={product} />
                ))}
            </div>
        </div>
    );
};
export default PeopleAlsoBought;
```

FILE: frontend/src/components/ProductCard.jsx

```
import { Link } from "react-router-dom";
import { ShoppingCart } from "lucide-react";
import { useCartStore } from "../stores/useCartStore";
import { useState, useEffect } from "react";

const ProductCard = ({ product, stock, weight_per_unit, categories, onUpdate }) => {
  const { addToCart } = useCartStore();

  const [newStock, setNewStock] = useState(stock);
  const [newWeight, setNewWeight] = useState(weight_per_unit);
  const [lastUpdated, setLastUpdated] = useState({ stock, weight_per_unit });

  const handleAddToCart = () => {
    addToCart(product.id);
  };

  const handleStockChange = (e) => {
    setNewStock(e.target.value);
  };

  const handleWeightChange = (e) => {
    setNewWeight(e.target.value);
  };

  // Prevent infinite loop by updating only when there is a real change in stock or weight
  useEffect(() => {
    // Only call onUpdate if the newStock or newWeight is different from the previous one
    if (
      newStock !== lastUpdated.stock ||
      newWeight !== lastUpdated.weight_per_unit
    ) {
      onUpdate(newStock, newWeight);
      setLastUpdated({ stock: newStock, weight_per_unit: newWeight });
    }
  }, [newStock, newWeight, lastUpdated, onUpdate]);

  // Check if the product is out of stock
  const isInWareHousePage = stock != null;

  return (
    <div className="flex w-full relative flex-col overflow-hidden rounded-lg border border-gray-700
shadow-lg">
      <div className="relative mx-3 mt-3 flex h-60 overflow-hidden rounded-xl">
        {/* Conditionally render the Link based on stock availability */}
        {!isInWareHousePage ? (
          <Link to={`/product/${product.id}`}>
            <img
              className="object-cover w-full cursor-pointer"
              src={product.image}
```

```
              alt="Product image"
            />
          </Link>
        ) : (
          <img
            className="object-cover w-full"
            src={product.image}
            alt="Product image"
          />
        )}
        <div className="absolute inset-0 bg-black bg-opacity-20" />
      </div>

      <div className="mt-4 px-5 pb-5">
        {/* Conditionally render the Link for product name */}
        {!isInWareHousePage ? (
          <Link to={`/product/${product.id}`}>
            <h5 className="text-xl font-semibold tracking-tight text-white cursor-pointer">
              {product.name}
            </h5>
          </Link>
        ) : (
          <h5 className="text-xl font-semibold tracking-tight text-white">
            {product.name}
          </h5>
        )}

        <div className="mt-2 mb-5 flex items-center justify-between">
          <p>
            <span className="text-3xl font-bold text-emerald-400">${product.price}</span>
          </p>
        </div>

        {/* Display stock if available */}
        {stock !== null && (
          <p className="text-sm text-gray-300">Stock: {stock}</p>
        )}

        {/* Display weight if available */}
        {weight_per_unit !== null && (
          <p className="text-sm text-gray-300">Weight: {weight_per_unit} kg</p>
        )}

        {/* Conditionally render the Add to Cart button if the product is in stock */}
        {!isInWareHousePage && (
          <button
            className="flex items-center justify-center rounded-lg bg-emerald-600 px-5 py-2.5 text-
center text-sm font-medium text-white hover:bg-emerald-700 focus:outline-none focus:ring-4
focus:ring-emerald-300"
            onClick={handleAddToCart}
          >
            <ShoppingCart size={22} className="mr-2" />
            Add to cart
          </button>
        )}

        {/* Editable fields for inventory update */}
        {isInWareHousePage && (
          <div className="mt-4">
            {/* Categories Section */}
            {categories && categories.length > 0 && (
              <div className="mb-4">
                <label className="text-sm text-gray-300">Categories:</label>
                <div className="flex flex-wrap space-x-2 mt-2">
                  {categories.map((category) => (
                    <span
```

```
                    key={category.id}
                    className="text-xs text-emerald-400 bg-gray-800 rounded-full px-3 py-1 mt-4"
                  >
                    {category.category_name}
                  </span>
                ))}
              </div>
            </div>
          )}

          {/* Editable fields for stock and weight */}
          <div className="mb-4">
            <label className="text-sm text-gray-300">Stock:</label>
            <input
              type="number"
              value={newStock}
              onChange={handleStockChange}
              className="w-full p-2 mt-2 bg-gray-800 text-white rounded-lg"
              min="0"
            />
          </div>
          <div className="mb-4">
            <label className="text-sm text-gray-300">Weight (kg):</label>
            <input
              type="number"
              value={newWeight}
              onChange={handleWeightChange}
              className="w-full p-2 mt-2 bg-gray-800 text-white rounded-lg"
              min="0"
              step="any"
            />
          </div>
        </div>
      )}
    </div>
  </div>
);
};

export default ProductCard;
```

FILE: frontend/src/components/ProductsList.jsx

```
import { motion } from "framer-motion";
import { Trash, Star } from "lucide-react";
import { useProductStore } from "../stores/useProductStore";

const ProductsList = () => {
    const { deleteProduct, toggleFeaturedProduct, products } = useProductStore();

    console.log("products", products);

    return (
        <motion.div
            className='bg-gray-800 shadow-lg rounded-lg overflow-hidden max-w-4xl mx-auto'
            initial={{ opacity: 0, y: 20 }}
            animate={{ opacity: 1, y: 0 }}
            transition={{ duration: 0.8 }}
        >
            <table className=' min-w-full divide-y divide-gray-700'>
                <thead className='bg-gray-700'>
                    <tr>
                        <th
                            scope='col'
                            className='px-6 py-3 text-left text-xs font-medium text-gray-300
uppercase tracking-wider'
```

```
                        >
                            Product
                        </th>
                        <th
                            scope='col'
                            className='px-6 py-3 text-left text-xs font-medium text-gray-300
uppercase tracking-wider'
                        >
                            Price
                        </th>
                        <th
                            scope='col'
                            className='px-6 py-3 text-left text-xs font-medium text-gray-300
uppercase tracking-wider'
                        >
                            Category
                        </th>

                        <th
                            scope='col'
                            className='px-6 py-3 text-left text-xs font-medium text-gray-300
uppercase tracking-wider'
                        >
                            Featured
                        </th>
                        <th
                            scope='col'
                            className='px-6 py-3 text-left text-xs font-medium text-gray-300
uppercase tracking-wider'
                        >
                            Actions
                        </th>
                    </tr>
                </thead>

                <tbody className='bg-gray-800 divide-y divide-gray-700'>
                    {products?.map((product) => (
                        <tr key={product._id} className='hover:bg-gray-700'>
                            <td className='px-6 py-4 whitespace-nowrap'>
                                <div className='flex items-center'>
                                    <div className='flex-shrink-0 h-10 w-10'>
                                        <img
                                            className='h-10 w-10 rounded-full object-cover'
                                            src={product.image}
                                            alt={product.name}
                                        />
                                    </div>
                                    <div className='ml-4'>
                                        <div className='text-sm font-medium text-white'>
{product.name}</div>
                                    </div>
                                </div>
                            </td>
                            <td className='px-6 py-4 whitespace-nowrap'>
                                <div className='text-sm text-gray-300'>${product.price.toFixed(2)}
</div>
                            </td>
                            <td className='px-6 py-4 whitespace-nowrap'>
                                <div className='text-sm text-gray-300'>{product.category}</div>
                            </td>
                            <td className='px-6 py-4 whitespace-nowrap'>
                                <button
                                    onClick={() => toggleFeaturedProduct(product._id)}
                                    className={`p-1 rounded-full ${
                                        product.isFeatured ? "bg-yellow-400 text-gray-900" : "bg-
gray-600 text-gray-300"
```

```
                                    } hover:bg-yellow-500 transition-colors duration-200`}
                                >
                                    <Star className='h-5 w-5' />
                                </button>
                            </td>
                            <td className='px-6 py-4 whitespace-nowrap text-sm font-medium'>
                                <button
                                    onClick={() => deleteProduct(product._id)}
                                    className='text-red-400 hover:text-red-300'
                                >
                                    <Trash className='h-5 w-5' />
                                </button>
                            </td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </motion.div>
    );
};
export default ProductsList;
```

FILE: frontend/src/components/Profile/ProfileLayout.jsx

```
import React, { useState, useEffect } from 'react';
import { Link, Outlet, useLocation } from 'react-router-dom';

const ProfileLayout = () => {
  const [isAddressPage, setIsAddressPage] = useState(false);
  const location = useLocation();  // Use the useLocation hook to track the URL

  // Update isAddressPage whenever the location changes
  useEffect(() => {
    if (location.pathname.includes("address")||
(!location.pathname.includes("edit")&&!location.pathname.includes("change"))) {
      setIsAddressPage(true);
    } else {
      setIsAddressPage(false);
    }
  }, [location]); // Dependency on location will trigger the effect on route change

  return (
    <div className={`flex ${isAddressPage ? 'h-full' : 'h-screen'} bg-gray-900`}>
      <div className="w-64 text-white p-6 shadow-lg border-r-2 border-emerald-500">
        <h2 className="text-3xl font-bold mb-8 text-center text-emerald-400">User Profile</h2>
        <ul className="space-y-4">
          <li>
            <Link
              to=""
              className="block text-lg hover:bg-emerald-600 hover:rounded-lg p-2 transition
duration-200"
            >
              View Profile
            </Link>
          </li>
          <li>
            <Link
              to="edit"
              className="block text-lg hover:bg-emerald-600 hover:rounded-lg p-2 transition
duration-200"
            >
              Edit Profile
            </Link>
          </li>
          <li>
            <Link
```

```
              to="change-password"
              className="block text-lg hover:bg-emerald-600 hover:rounded-lg p-2 transition
duration-200"
            >
              Change Password
            </Link>
          </li>
          <li>
            <Link
              to="address"
              className="block text-lg hover:bg-emerald-600 hover:rounded-lg p-2 transition
duration-200"
            >
              Address
            </Link>
          </li>
        </ul>
      </div>
      <div className="flex-1 p-8 bg-gray-900 text-white">
        <Outlet />
      </div>
    </div>
  );
};

export default ProfileLayout;
```

FILE: frontend/src/components/Vehicle/Navbar.jsx

```
import React from "react";
import { useAppDispatch, useAppSelector } from "../../pages/State/Redux";
import { setIsSidebarCollapsed } from "../../pages/State/State";
import { Menu } from "lucide-react";
const Navbar = () => {
  const dispatch = useAppDispatch();
  const isSidebarCollapsed = useAppSelector(
    (state) => state.global.isSidebarCollapsed
  );
  const toggleSidebar = () => {
    dispatch(setIsSidebarCollapsed(!isSidebarCollapsed));
  };

  return (
    <div className="flex justify-between items-center w-full mb-7">
      {/* LEFT SIDE */}
      <div className="flex justify-between items-center gap-5">
        <button
          className="px-3 py-3 bg-emerald-400 rounded-full hover:bg-emerald-200"
          onClick={toggleSidebar}
        >
          <Menu className="w-4 h-4" />
        </button>
      </div>
    </div>
  );
};

export default Navbar;
```

FILE: frontend/src/components/Vehicle/Sidebar.jsx

```
import { useAppDispatch, useAppSelector } from "../../pages/State/Redux";
import { setIsSidebarCollapsed } from "../../pages/State/State";
import {
  Car,
```

```
  Clipboard,
  Menu,
  LineChart,
  Map,
  FormInputIcon,
  ListOrdered
} from "lucide-react";
import { Link, useLocation } from "react-router-dom"; // Use React Router's Link and useLocation
import React from "react";

// SidebarLink Component
const SidebarLink = ({ href, icon: Icon, label, isCollapsed }) => {
  const location = useLocation(); // Use React Router's useLocation hook to get the current
pathname
  const isActive = location.pathname == href;

  return (
    <Link to={href}> {/* Use React Router's Link component */}
      <div
        className={`cursor-pointer flex items-center ${
          isCollapsed ? "justify-center py-4" : "justify-start px-8 py-4"
        }
        hover:text-gray-900 hover:bg-gray-900 gap-3 transition-colors ${
          isActive ? "bg-gray-900 text-white" : ""
        }
      }`}
      >
        <Icon className="w-6 h-6 !text-gray-700" />
        <span
          className={`${
            isCollapsed ? "hidden" : "block"
          } font-medium text-gray-700`}
        >
          {label}
        </span>
      </div>
    </Link>
  );
};

// Sidebar Component
const Sidebar = () => {
  const dispatch = useAppDispatch();
  const isSidebarCollapsed = useAppSelector(
    (state) => state.global.isSidebarCollapsed
  );

  const toggleSidebar = () => {
    dispatch(setIsSidebarCollapsed(!isSidebarCollapsed));
  };

  const sidebarClassNames = `fixed flex flex-col ${
    isSidebarCollapsed ? "w-0 md:w-16" : "w-72 md:w-64"
  } bg-emerald-400 transition-all duration-300 overflow-hidden h-full shadow-md z-40`;

  return (
    <div className={sidebarClassNames}>
      {/* TOP LOGO */}
      <div
        className={`flex gap-3 justify-between md:justify-normal items-center pt-8 ${
          isSidebarCollapsed ? "px-5" : "px-8"
        }`}
      >

        <button
          className="md:hidden px-3 py-3 bg-gray-100 rounded-full hover:bg-gray-900"
```

```
          onClick={toggleSidebar}
        >
          <Menu className="w-4 h-4" />
        </button>
      </div>

      {/* LINKS */}
      <div className="flex-grow mt-8">
        <SidebarLink
          href="/vehicle"
          icon={Car}
          label="Vehicles"
          isCollapsed={isSidebarCollapsed}
        />
      </div>

      {/* FOOTER */}
      <div className={`${isSidebarCollapsed ? "hidden" : "block"} mb-10`}>
        <p className="text-center text-xs text-gray-500">&copy; 2024 Edstock</p>
      </div>
    </div>
  );
};

export default Sidebar;
```

FILE: frontend/src/components/Warehouse/Navbar.jsx

```
import React from "react";
import { useAppDispatch, useAppSelector } from "../../pages/State/Redux";
import { setIsSidebarCollapsed } from "../../pages/State/State";
import { Menu } from "lucide-react";

const Navbar = () => {
  const dispatch = useAppDispatch();
  const isSidebarCollapsed = useAppSelector(
    (state) => state.global.isSidebarCollapsed
  );
  const toggleSidebar = () => {
    dispatch(setIsSidebarCollapsed(!isSidebarCollapsed));
  };

  return (
    <div className="flex justify-between items-center w-full mb-7">
      {/* LEFT SIDE */}
      <div className="flex justify-between items-center gap-5">
        <button
          className="px-3 py-3 bg-emerald-400 rounded-full hover:bg-emerald-200"
          onClick={toggleSidebar}
        >
          <Menu className="w-4 h-4" />
        </button>
      </div>
    </div>
  );
};

export default Navbar;
```

FILE: frontend/src/components/Warehouse/Sidebar.jsx

```
import { useAppDispatch, useAppSelector } from "../../pages/State/Redux";
import { setIsSidebarCollapsed } from "../../pages/State/State";
import {
  CircleDollarSign,
```

```
  Clipboard,
  Menu,
  LineChart,
  Map,
  FormInputIcon,
  ListOrdered,
  CameraIcon,
} from "lucide-react";
import { Link, useLocation } from "react-router-dom"; // Use React Router's Link and useLocation
import React from "react";

// SidebarLink Component
const SidebarLink = ({ href, icon: Icon, label, isCollapsed }) => {
  const location = useLocation(); // Use React Router's useLocation hook to get the current
pathname
  const isActive = location.pathname == href;

  return (
    <Link to={href}> {/* Use React Router's Link component */}
      <div
        className={`cursor-pointer flex items-center ${
          isCollapsed ? "justify-center py-4" : "justify-start px-8 py-4"
        }
        hover:text-gray-900 hover:bg-gray-900 gap-3 transition-colors ${
          isActive ? "bg-gray-900 text-white" : ""
        }
      }`}
      >
        <Icon className="w-6 h-6 !text-gray-700" />
        <span
          className={`${
            isCollapsed ? "hidden" : "block"
          } font-medium text-gray-700`}
        >
          {label}
        </span>
      </div>
    </Link>
  );
};

// Sidebar Component
const Sidebar = () => {
  const dispatch = useAppDispatch();
  const isSidebarCollapsed = useAppSelector(
    (state) => state.global.isSidebarCollapsed
  );

  const toggleSidebar = () => {
    dispatch(setIsSidebarCollapsed(!isSidebarCollapsed));
  };

  const sidebarClassNames = `fixed flex flex-col ${
    isSidebarCollapsed ? "w-0 md:w-16" : "w-72 md:w-64"
  } bg-emerald-400 transition-all duration-300 overflow-hidden h-full shadow-md z-40`;

  return (
    <div className={sidebarClassNames}>
      {/* TOP LOGO */}
      <div
        className={`flex gap-3 justify-between md:justify-normal items-center pt-8 ${
          isSidebarCollapsed ? "px-5" : "px-8"
        }`}
      >

        <button
```

```
          className="md:hidden px-3 py-3 bg-gray-100 rounded-full hover:bg-gray-900"
          onClick={toggleSidebar}
        >
          <Menu className="w-4 h-4" />
        </button>
      </div>

      {/* LINKS */}
      <div className="flex-grow mt-8">
        <SidebarLink
          href="/warehouse"
          icon={CircleDollarSign}
          label="Expenses"
          isCollapsed={isSidebarCollapsed}
        />
        <SidebarLink
          href="/warehouse/products"
          icon={Clipboard}
          label="Products"
          isCollapsed={isSidebarCollapsed}
        />
        <SidebarLink
          href="/warehouse/linechart"
          icon={LineChart}
          label="Line Chart"
          isCollapsed={isSidebarCollapsed}
        />
        <SidebarLink
          href="/warehouse/geography"
          icon={Map}
          label="Geography"
          isCollapsed={isSidebarCollapsed}
        />
        <SidebarLink
          href="/warehouse/create-inventory"
          icon={FormInputIcon}
          label="Create Inventory"
          isCollapsed={isSidebarCollapsed}
        />
        <SidebarLink
          href="/warehouse/orders"
          icon={ListOrdered}
          label="Orders"
          isCollapsed={isSidebarCollapsed}
        />

          <SidebarLink
          href="/warehouse/license-plate"
          icon={CameraIcon}
          label="License Plate"
          isCollapsed={isSidebarCollapsed}
        />
      </div>
    </div>
  );
};

export default Sidebar;
```

FILE: frontend/src/echo.js

```
import Echo from 'laravel-echo';
import Pusher from 'pusher-js';

// Debugging the environment variables
console.log('App Key:', import.meta.env.VITE_REVERB_APP_KEY);
```

```
console.log('WS Host:', import.meta.env.VITE_REVERB_HOST);
console.log('WS Port:', import.meta.env.VITE_REVERB_PORT);
console.log('WSS Port:', import.meta.env.VITE_REVERB_PORT ?? 443);
console.log('Force TLS:', (import.meta.env.VITE_REVERB_SCHEME ?? 'https') === 'https');

// Ensure Pusher is correctly initialized
window.Pusher = Pusher;

window.Echo = new Echo({
    broadcaster: 'reverb',  // Ensure you're using the correct broadcaster
    key: import.meta.env.VITE_REVERB_APP_KEY,  // This must be set in your .env file
    wsHost: import.meta.env.VITE_REVERB_HOST,  // Your WebSocket host
    wsPort: import.meta.env.VITE_REVERB_PORT ?? 80,  // WebSocket port
    wssPort: import.meta.env.VITE_REVERB_PORT ?? 443,  // Secure WebSocket port
    forceTLS: (import.meta.env.VITE_REVERB_SCHEME ?? 'https') === 'https',  // TLS/SSL support
    enabledTransports: ['ws', 'wss'],  // Enable WebSockets
});
```

FILE: frontend/src/EventBus.jsx

```
import React from "react";

export const EventBusContext = React.createContext();

export const EventBusProvider = ({ children }) => {
    const [events, setEvents] = React.useState({});

    const emit = (name, data) => {
        if (events[name]) {
            for (let cb of events[name]) {
                cb(data);
            }
        }
    };

    const on = (name, cb) => {
        if (!events[name]) {
            events[name] = [];
        }

        events[name].push(cb);

        return () => {
            events[name] = events[name].filter((callback) => callback !== cb);
        };
    };

    return (
        <EventBusContext.Provider value={{ emit, on }}>
            {children}
        </EventBusContext.Provider>
    );
};

export const useEventBus = () => {
    return React.useContext(EventBusContext);
};
```

FILE: frontend/src/helpers.js

```
import { clsx } from "clsx";
import { twMerge } from "tailwind-merge";
import stc from "string-to-color";

export function cn(...inputs) {
```

```
  return twMerge(clsx(inputs));
}

export const stringToColor = (str) => {
  return stc(str);
};

export const formatMessageDateLong = (date) => {
    const now = new Date();
    const inputDate = new Date(date);

    if (isToday(inputDate)) {
        return inputDate.toLocaleTimeString([], {
            hour: "2-digit",
            minute: "2-digit",
        });
    } else if (isYesterday(inputDate)) {
        return (
            "Yesterday " +
            inputDate.toLocaleTimeString([], {
                hour: "2-digit",
                minute: "2-digit",
            })
        );
    } else if (inputDate.getFullYear() === now.getFullYear()) {
        return inputDate.toLocaleDateString([], {
            day: "2-digit",
            month: "short",
        });
    } else {
        return inputDate.toLocaleDateString();
    }
};

export const formatMessageDateShort = (date) => {
    const now = new Date();
    const inputDate = new Date(date);

    if (isToday(inputDate)) {
        return inputDate.toLocaleTimeString([], {
            hour: "2-digit",
            minute: "2-digit",
        });
    } else if (isYesterday(inputDate)) {
        return "Yesterday";
    } else if (inputDate.getFullYear() === now.getFullYear()) {
        return inputDate.toLocaleDateString([], {
            day: "2-digit",
            month: "short",
        });
    } else {
        return inputDate.toLocaleDateString();
    }
};

export const isToday = (date) => {
    const today = new Date();
    return (
        date.getDate() === today.getDate() &&
        date.getMonth() === today.getMonth() &&
        date.getFullYear() === today.getFullYear()
    );
};

export const isYesterday = (date) => {
    const yesterday = new Date();
```

```
    yesterday.setDate(yesterday.getDate() - 1);
    return (
        date.getDate() === yesterday.getDate() &&
        date.getMonth() === yesterday.getMonth() &&
        date.getFullYear() === yesterday.getFullYear()
    );
};

export const isImage = (attachment) => {
    let mime = attachment.mime || attachment.type;
    mime = mime.split("/");
    return mime[0].toLowerCase() === "image";
};

export const isVideo = (attachment) => {
    let mime = attachment.mime || attachment.type;
    mime = mime.split("/");
    return mime[0].toLowerCase() === "video";
};

export const isAudio = (attachment) => {
    let mime = attachment.mime || attachment.type;
    mime = mime.split("/");
    return mime[0].toLowerCase() === "audio";
};

export const isPDF = (attachment) => {
    let mime = attachment.mime || attachment.type;
    return mime === "application/pdf";
};

export const isPreviewable = (attachment) => {
    return (
        isImage(attachment) ||
        isVideo(attachment) ||
        isAudio(attachment) ||
        isPDF(attachment)
    );
};

export const formatBytes = (bytes, decimals = 2) => {
    if (bytes === 0) return "0 Bytes";

    const k = 1024;
    const dm = decimals < 0 ? 0 : decimals;
    const sizes = ["Bytes", "KB", "MB", "GB"];

    let i = 0;
    let size = bytes;
    while (size >= k) {
        size /= k;
        i++;
    }

    return parseFloat(size.toFixed(dm)) + " " + sizes[i];
};
```

FILE: frontend/src/lib/axios.js

```
import axios from "axios";

const axiosInstance = axios.create({
    baseURL: "http://localhost:8000/api",
    withCredentials: true, // send cookies to the server
});
axiosInstance.interceptors.request.use((config) => {
```

```
    const token = localStorage.getItem("access_token"); // Retrieve the token
    if (token) {
        config.headers.Authorization = `Bearer ${token}`; // Set the token
    }
    return config;
});

export default axiosInstance;
```

FILE: frontend/src/lib/gemini.js

```
import {
  GoogleGenerativeAI,
  HarmBlockThreshold,
  HarmCategory,
} from "@google/generative-ai";

const safetySetting = [
  {
    category: HarmCategory.HARM_CATEGORY_HARASSMENT,
    threshold: HarmBlockThreshold.BLOCK_LOW_AND_ABOVE,
  },
  {
    category: HarmCategory.HARM_CATEGORY_HATE_SPEECH,
    threshold: HarmBlockThreshold.BLOCK_LOW_AND_ABOVE,
  },
];

const genAI = new GoogleGenerativeAI(import.meta.env.VITE_GEMINI_PUBLIC_KEY);

const model = genAI.getGenerativeModel({
  model: "gemini-3-flash-preview",
  safetySetting,
});

export default model;
```

FILE: frontend/src/lib/utils.js

```
import { clsx } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs) {
  return twMerge(clsx(inputs));
}
```

FILE: frontend/src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";
import { EventBusProvider } from "./EventBus.jsx";
import { BrowserRouter } from "react-router-dom";
import StoreProvider from "./pages/State/Redux.jsx";
import global from 'global';
global.global = global;

createRoot(document.getElementById("root")).render(
    <StrictMode>
        <BrowserRouter>
            <EventBusProvider>
                <StoreProvider>
                    <App />
```

```
                </StoreProvider>
            </EventBusProvider>
        </BrowserRouter>
    </StrictMode>
);
```

FILE: frontend/src/pages/AdminPage.jsx

```
import { BarChart, PlusCircle, ShoppingBasket } from "lucide-react";
import { useEffect, useState } from "react";
import { motion } from "framer-motion";

import AnalyticsTab from "../components/AnalyticsTab";
import CreateProductForm from "../components/CreateProductForm";
import ProductsList from "../components/ProductsList";
import { useProductStore } from "../stores/useProductStore";

const tabs = [
    { id: "create", label: "Create Product", icon: PlusCircle },
    { id: "products", label: "Products", icon: ShoppingBasket },
    { id: "analytics", label: "Analytics", icon: BarChart },
];

const AdminPage = () => {
    const [activeTab, setActiveTab] = useState("create");
    const { fetchAllProducts } = useProductStore();

    useEffect(() => {
        fetchAllProducts();
    }, [fetchAllProducts]);

    return (
        <div className='min-h-screen relative overflow-hidden'>
            <div className='relative z-10 container mx-auto px-4 py-16'>
                <motion.h1
                    className='text-4xl font-bold mb-8 text-emerald-400 text-center'
                    initial={{ opacity: 0, y: -20 }}
                    animate={{ opacity: 1, y: 0 }}
                    transition={{ duration: 0.8 }}
                >
                    Admin Dashboard
                </motion.h1>

                <div className='flex justify-center mb-8'>
                    {tabs.map((tab) => (
                        <button
                            key={tab.id}
                            onClick={() => setActiveTab(tab.id)}
                            className={`flex items-center px-4 py-2 mx-2 rounded-md transition-
colors duration-200 ${
                                activeTab === tab.id
                                    ? "bg-emerald-600 text-white"
                                    : "bg-gray-700 text-gray-300 hover:bg-gray-600"
                            }`}
                        >
                            <tab.icon className='mr-2 h-5 w-5' />
                            {tab.label}
                        </button>
                    ))}
                </div>
                {activeTab === "create" && <CreateProductForm />}
                {activeTab === "products" && <ProductsList />}
                {activeTab === "analytics" && <AnalyticsTab />}
            </div>
        </div>
    );
```

```
};
export default AdminPage;
```

FILE: frontend/src/pages/CartPage.jsx

```
import { Link } from "react-router-dom";
import { useCartStore } from "../stores/useCartStore";
import { motion } from "framer-motion";
import { ShoppingCart } from "lucide-react";
import CartItem from "../components/CartItem";
import PeopleAlsoBought from "../components/PeopleAlsoBought";
import OrderSummary from "../components/OrderSummary";
import GiftCouponCard from "../components/GiftCouponCard";

const CartPage = () => {
    const { cart } = useCartStore();

    return (
        <div className='py-8 md:py-16'>
            <div className='mx-auto max-w-screen-xl px-4 2xl:px-0'>
                <div className='mt-6 sm:mt-8 md:gap-6 lg:flex lg:items-start xl:gap-8'>
                    <motion.div
                        className='mx-auto w-full flex-none lg:max-w-2xl xl:max-w-4xl'
                        initial={{ opacity: 0, x: -20 }}
                        animate={{ opacity: 1, x: 0 }}
                        transition={{ duration: 0.5, delay: 0.2 }}
                    >
                        {cart.length === 0 ? (
                            <EmptyCartUI />
                        ) : (
                            <div className='space-y-6'>
                                {cart.map((item) => (
                                    <CartItem key={item._id} item={item} />
                                ))}
                            </div>
                        )}
                        {cart.length > 0 && <PeopleAlsoBought />}
                    </motion.div>

                    {cart.length > 0 && (
                        <motion.div
                            className='mx-auto mt-6 max-w-4xl flex-1 space-y-6 lg:mt-0 lg:w-full'
                            initial={{ opacity: 0, x: 20 }}
                            animate={{ opacity: 1, x: 0 }}
                            transition={{ duration: 0.5, delay: 0.4 }}
                        >
                            <OrderSummary />
                            <GiftCouponCard />
                        </motion.div>
                    )}
                </div>
            </div>
        </div>
    );
};
export default CartPage;

const EmptyCartUI = () => (
    <motion.div
        className='flex flex-col items-center justify-center space-y-4 py-16'
        initial={{ opacity: 0, y: 20 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.5 }}
    >
        <ShoppingCart className='h-24 w-24 text-gray-300' />
        <h3 className='text-2xl font-semibold '>Your cart is empty</h3>
```

```
        <p className='text-gray-400'>Looks like you {"haven't"} added anything to your cart yet.
</p>
        <Link
            className='mt-4 rounded-md bg-emerald-500 px-6 py-2 text-white transition-colors
hover:bg-emerald-600'
            to='/'
        >
            Start Shopping
        </Link>
    </motion.div>
);
```

FILE: frontend/src/pages/CategoryPage.jsx

```
import { useEffect } from "react";
import { useProductStore } from "../stores/useProductStore";
import { useParams } from "react-router-dom";
import { motion } from "framer-motion";
import ProductCard from "../components/ProductCard";

const CategoryPage = () => {
    const { fetchProductsByCategory, products } = useProductStore();

    const { category } = useParams();

    useEffect(() => {
        fetchProductsByCategory([category]);
    }, [fetchProductsByCategory, category]);

    console.log("products:", products);
    return (
        <div className='min-h-screen'>
            <div className='relative z-10 max-w-screen-xl mx-auto px-4 sm:px-6 lg:px-8 py-16'>
                <motion.h1
                    className='text-center text-4xl sm:text-5xl font-bold text-emerald-400 mb-8'
                    initial={{ opacity: 0, y: -20 }}
                    animate={{ opacity: 1, y: 0 }}
                    transition={{ duration: 0.8 }}
                >
                    {category.charAt(0).toUpperCase() + category.slice(1)}
                </motion.h1>

                <motion.div
                    className='grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-6
justify-items-center'
                    initial={{ opacity: 0, y: 20 }}
                    animate={{ opacity: 1, y: 0 }}
                    transition={{ duration: 0.8, delay: 0.2 }}
                >
                    {products?.length === 0 && (
                        <h2 className='text-3xl font-semibold text-gray-300 text-center col-span-
full'>
                            No products found
                        </h2>
                    )}

                    {products?.map((product) => (
                        <ProductCard key={product.id} product={product} />
                    ))}
                </motion.div>
            </div>
        </div>
    );
};
export default CategoryPage;
```

FILE: frontend/src/pages/ChangePasswordPage.jsx

```
import React, { useState } from 'react';
import { useUserStore } from '../stores/useUserStore';

const ChangePasswordPage = () => {
  const { changePassword } = useUserStore();
  const [passwordData, setPasswordData] = useState({
    currentPassword: '',
    newPassword: '',
    confirmPassword: '',
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setPasswordData((prevData) => ({ ...prevData, [name]: value }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const { currentPassword, newPassword, confirmPassword } = passwordData;
    changePassword(currentPassword, newPassword, confirmPassword);
  };

  return (
    <div className="w-full bg-transparent text-emerald-400 rounded-lg shadow-lg">
      <h2 className="text-3xl font-bold text-center mb-6">Change Password</h2>
      <form onSubmit={handleSubmit} className="space-y-5">
        <div>
          <label htmlFor="currentPassword" className="block text-sm font-medium text-gray-300">
            Current Password
          </label>
          <input
            id="currentPassword"
            name="currentPassword"
            type="password"
            className="w-full p-4 bg-transparent text-emerald-400 border-2 border-white rounded-lg
focus:outline-none focus:ring-2 focus:ring-emerald-500"
            value={passwordData.currentPassword}
            onChange={handleChange}
          />
        </div>
        <div>
          <label htmlFor="newPassword" className="block text-sm font-medium text-gray-300">
            New Password
          </label>
          <input
            id="newPassword"
            name="newPassword"
            type="password"
            className="w-full p-4 bg-transparent text-emerald-400 border-2 border-white rounded-lg
focus:outline-none focus:ring-2 focus:ring-emerald-500"
            value={passwordData.newPassword}
            onChange={handleChange}
          />
        </div>
        <div>
          <label htmlFor="confirmPassword" className="block text-sm font-medium text-gray-300">
            Confirm New Password
          </label>
          <input
            id="confirmPassword"
            name="confirmPassword"
            type="password"
            className="w-full p-4 bg-transparent text-emerald-400 border-2 border-white rounded-lg
focus:outline-none focus:ring-2 focus:ring-emerald-500"
```

```
            value={passwordData.confirmPassword}
            onChange={handleChange}
          />
        </div>
        <button
          type="submit"
          className="w-full bg-emerald-600 hover:bg-emerald-700 text-white font-semibold py-3
rounded-lg mt-4 transition duration-200"
        >
          Change Password
        </button>
      </form>
    </div>
  );
};

export default ChangePasswordPage;
```

FILE: frontend/src/pages/Chat/AttachmentPreview.jsx

```
import React from "react";
import { PaperClipIcon } from "@heroicons/react/24/solid";
import { formatBytes , isPDF, isPreviewable } from "../../helpers";
const AttachmentPreview = ({ file }) => {
    return (
        <div className="w-full flex items-center gap-2 py-2 px-3 rounded-md bg-slate-800">
            <div>
                {isPDF(file.file) && <img src="/img/pdf.png" className="w-8" />}
                {!isPreviewable(file.file) && (
                    <div className="flex justify-center items-center w-10 h-10 bg-gray-700
rounded">
                        <PaperClipIcon className="w-6" />
                    </div>
                )}
            </div>
            <div className="flex-1 text-gray-400 text-nowrap text-ellipsis overflow-hidden">
                <h3>{file.file.name}</h3>
                <p className="text-xs">{formatBytes(file.file.size)}</p>
            </div>
        </div>
    );
};

export default AttachmentPreview;
```

FILE: frontend/src/pages/Chat/AttachmentPreviewModal.jsx

```
import { Fragment, useEffect, useMemo, useState } from "react";
import { Transition, Dialog } from "@headlessui/react";
import {
    PaperClipIcon,
    XMarkIcon,
    ChevronLeftIcon,
    ChevronRightIcon,
} from "@heroicons/react/24/solid";
import { isAudio, isImage, isPDF, isPreviewable, isVideo } from "../../helpers";
export default function AttachmentPreviewModal({
    attachments,
    index,
    show = false,
    onClose = () => {},
}) {
    const [currentIndex, setCurrentIndex] = useState(0);

    const previewableAttachments = useMemo(() => {
```

```
        return attachments.filter((attachment) => isPreviewable(attachment));
    }, [attachments]);

    const attachment = useMemo(() => {
        return previewableAttachments[currentIndex];
    }, [attachments, currentIndex]);

    const close = () => {
        onClose();
    };

    const prev = () => {
        if (currentIndex === 0) {
            return;
        }
        setCurrentIndex(currentIndex - 1);
    };

    const next = () => {
        if (currentIndex === previewableAttachments.length - 1) {
            return;
        }
        setCurrentIndex(currentIndex + 1);
    };

    useEffect(() => {
        setCurrentIndex(index);
    }, [index]);

    return (
        <Transition show={show} as={Fragment} leave="duration-200">
            <Dialog
                as="div"
                id="modal"
                className="relative z-50"
                onClose={close}
            >
                <Transition.Child
                    as={Fragment}
                    enter="ease-out duration-300"
                    enterFrom="opacity-0"
                    enterTo="opacity-100"
                    leave="ease-in duration-200"
                    leaveFrom="opacity-100"
                    leaveTo="opacity-0"
                >
                    <div className="fixed inset-0 bg-black/25" />
                </Transition.Child>

                <div className="fixed inset-0 overflow-y-auto">
                    <div className="h-screen w-screen">
                        <Transition.Child
                            as={Fragment}
                            enter="ease-out duration-300"
                            enterFrom="opacity-0 translate-y-4 sm:translate-y-0 sm:scale-95"
                            enterTo="opacity-100 translate-y-0 sm:scale-100"
                            leave="ease-in duration-200"
                            leaveFrom="opacity-100 translate-y-0 sm:scale-100"
                            leaveTo="opacity-0 translate-y-4 sm:translate-y-0 sm:scale-95"
                        >
                            <Dialog.Panel className="flex flex-col w-full h-full transform
overflow-hidden bg-slate-800  text-left align-middle shadow-xl transition-all">
                                <button
                                    onClick={close}
                                    className="absolute right-3 top-3 w-10 h-10 rounded-full
hover:bg-black/10 transition flex items-center justify-center text-gray-100 z-40"
```

```
                                >
                                    <XMarkIcon className="w-6 h-6 " />
                                </button>
                                <div className="relative group h-full ">
                                    {currentIndex > 0 && (
                                        <div
                                            onClick={prev}
                                            className="absolute opacity-100 text-gray-100 cursor-
pointer flex items-center justify-center w-16 h-16 left-4 top-1/2 -translate-y-1/2 rounded-full bg-
black/50 z-30"
                                        >
                                            <ChevronLeftIcon className="w-12" />
                                        </div>
                                    )}
                                    {currentIndex <
                                        previewableAttachments.length - 1 && (
                                        <div
                                            onClick={next}
                                            className="absolute opacity-100 text-gray-100 cursor-
pointer flex items-center justify-center w-16 h-16 right-4 top-1/2 -translate-y-1/2 rounded-full
bg-black/50 z-30"
                                        >
                                            <ChevronRightIcon className="w-12" />
                                        </div>
                                    )}

                                    {attachment && (
                                        <div className="flex items-center justify-center w-full h-
full p-3">
                                            {isImage(attachment) && (
                                                <img
                                                    src={attachment.url}
                                                    className="max-w-full max-h-full"
                                                />
                                            )}
                                            {isVideo(attachment) && (
                                                <div className="flex items-center">
                                                    <video
                                                        src={attachment.url}
                                                        controls
                                                        autoPlay
                                                    ></video>
                                                </div>
                                            )}
                                            {isAudio(attachment) && (
                                                <div className="relative flex justify-center items-
center">
                                                    <audio
                                                        src={attachment.url}
                                                        controls
                                                        autoPlay
                                                    ></audio>
                                                </div>
                                            )}
                                            {isPDF(attachment) && (
                                                <iframe
                                                    src={attachment.url}
                                                    className="w-full h-full"
                                                ></iframe>
                                            )}
                                            {!isPreviewable(attachment) && (
                                                <div className="p-32 flex flex-col justify-center
items-center text-gray-100">
                                                    <PaperClipIcon className="w-10 h-10 mb-3" />

                                                    <small>
```

```
                                                                {attachment.name}
                                                            </small>
                                                        </div>
                                                    )}
                                                </div>
                                            )}
                                        </div>
                                    </Dialog.Panel>
                                </Transition.Child>
                            </div>
                        </div>
                    </Dialog>
                </Transition>
            );
}
```

FILE: frontend/src/pages/Chat/AudioRecorder.jsx

```
import { MicrophoneIcon, StopCircleIcon } from "@heroicons/react/24/solid";
import { useState } from "react";

const AudioRecorder = ({ fileReady }) => {
    const [mediaRecorder, setMediaRecorder] = useState(null);
    const [recording, setRecording] = useState(false);

    const onMicrophoneClick = async () => {
        if (recording) {
            setRecording(false);
            if (mediaRecorder) {
                mediaRecorder.stop();
                setMediaRecorder(null);
            }
            return;
        }
        setRecording(true);
        try {
            const stream = await navigator.mediaDevices.getUserMedia({
                audio: true,
            });
            const newMediaRecorder = new MediaRecorder(stream);
            const chunks = [];

            newMediaRecorder.addEventListener("dataavailable", (event) => {
                chunks.push(event.data);
            });

            newMediaRecorder.addEventListener("stop", () => {
                let audioBlob = new Blob(chunks, {
                    type: "audio/ogg; codecs=opus",
                });
                let audioFile = new File([audioBlob], "recorded_audio.ogg", {
                    type: "audio/ogg; codecs=opus",
                });

                const url = URL.createObjectURL(audioFile);

                fileReady(audioFile, url);
            });

            newMediaRecorder.start();
            setMediaRecorder(newMediaRecorder);
        } catch (error) {
            setRecording(false);
            console.error("Error accessing microphone:", error);
        }
    };
```

```
    return (
        <button
            onClick={onMicrophoneClick}
            className="p-1 text-gray-400 hover:text-gray-200"
        >
            {recording && <StopCircleIcon className="w-6 text-red-600" />}
            {!recording && <MicrophoneIcon className="w-6" />}
        </button>
    );
};

export default AudioRecorder;
```

FILE: frontend/src/pages/Chat/ChatMain.jsx

```
import { useEffect, useRef, useState } from "react";
import { ChatBubbleLeftRightIcon } from "@heroicons/react/24/outline";
import useChatStore from "../../stores/useChatStore";
import ConversationHeader from "./ConversationHeader";
import MessageItem from "./MessageItem";
import MessageInput from "./MessageInput";
import AttachmentPreviewModal from "./AttachmentPreviewModal";
import { useEventBus } from "../../EventBus";
import NewMessageNotification from "./NewMessageNotification";
export default function ChatMain({ selectedConversation, isUser }) {
  const { localMessages, noMoreMessages, scrollFromBottom, fetchConversationData, loadMoreMessages,
messageCreated, messageDeleted } = useChatStore();
  const { on } = useEventBus();

  const loadMoreIntersect = useRef(null);
  const messagesCtrRef = useRef(null);

  const [previewAttachment, setPreviewAttachment] = useState({ attachments: [], ind: 0 });
  const [showAttachmentPreview, setShowAttachmentPreview] = useState(false);

  useEffect(() => {
    if (selectedConversation) {
      fetchConversationData(selectedConversation.id, isUser); // Fetch messages when conversation
is selected
    }

    const offCreated = on("message.created", messageCreated);
    const offDeleted = on("message.deleted", messageDeleted);

    return () => {
      offCreated();
      offDeleted();
    };
  }, [selectedConversation, fetchConversationData, messageCreated, messageDeleted]);

  // Use Intersection Observer to load more messages when scrolled to top
  useEffect(() => {
    const observer = new IntersectionObserver(
      ([entry]) => {
        if (entry.isIntersecting && !noMoreMessages) {
          loadMoreMessages();
        }
      },
      { threshold: 1.0 }  // Trigger when the top of the list is fully visible
    );

    if (loadMoreIntersect.current) {
      observer.observe(loadMoreIntersect.current);
    }
```

```
    return () => {
      if (loadMoreIntersect.current) {
        observer.unobserve(loadMoreIntersect.current);
      }
    };
  }, [loadMoreMessages, noMoreMessages]);

  useEffect(() => {
    // Restore the scroll position based on scrollFromBottom after messages are loaded
    if (messagesCtrRef.current && scrollFromBottom !== null) {
      messagesCtrRef.current.scrollTop =
        messagesCtrRef.current.scrollHeight -
        messagesCtrRef.current.offsetHeight -
        scrollFromBottom;
    }
  }, [localMessages, scrollFromBottom]);

  const handleAttachmentClick = (attachments, ind) => {
    setPreviewAttachment({
      attachments,
      ind,
    });
    setShowAttachmentPreview(true);
  };

  return (
    <>
      <NewMessageNotification />
      <div className="flex flex-col h-full gap-8">
        {selectedConversation ? (
          <>
            <ConversationHeader selectedConversation={selectedConversation} />
            <div ref={messagesCtrRef} className="flex-1 overflow-y-auto p-5 messages-container flex
flex-col justify-between">
              {localMessages.length === 0 ? (
                <div className="flex justify-center items-center h-full">
                  <div className="text-lg text-slate-200">No messages found</div>
                </div>
              ) : (
                <div className="flex-1 flex flex-col">
                  <div ref={loadMoreIntersect}></div>
                  {localMessages.map((message) => (
                    <MessageItem
                      key={message.id}
                      message={message}
                      attachmentClick={handleAttachmentClick}
                    />
                  ))}
                </div>
              )}
            </div>
            <div className="mt-auto"> {/* This ensures that the input stays at the bottom */}
              <MessageInput conversation={selectedConversation} />
            </div>
          </>
        ) : (
          <div className="flex justify-center items-center text-center h-full opacity-35">
            <div className="text-2xl md:text-4xl p-16 text-slate-200">
              Please select a conversation to see messages
            </div>
            <ChatBubbleLeftRightIcon className="w-32 h-32 inline-block" />
          </div>
        )}
      </div>

      {/* Attachment Preview Modal */}
```

```
      <AttachmentPreviewModal
        attachments={previewAttachment.attachments}
        index={previewAttachment.ind}
        show={showAttachmentPreview}
        onClose={() => setShowAttachmentPreview(false)}
      />
    </>
  );
}
```

FILE: frontend/src/pages/Chat/ChatWrapper.jsx

```
import ConversationItem from './ConversationItem';
import GroupModal from './GroupModal';
import { useEventBus } from '../../EventBus';
import { PencilSquareIcon } from "@heroicons/react/24/solid";
import { useState, useEffect } from "react";
import useChatStore from '../../stores/useChatStore';
import { useUserStore } from '../../stores/useUserStore';
import ChatMain from './ChatMain';
const ChatWrapper = () => {
    const { selectedConversation, setSelectedConversation, fetchConversations, conversations,
isLoading } = useChatStore();
    const [localConversations, setLocalConversations] = useState([]);
    const [sortedConversations, setSortedConversations] = useState([]);
    const [onlineUsers, setOnlineUsers] = useState({});
    const [showGroupModal, setShowGroupModal] = useState(false);
    const { emit, on } = useEventBus();
    const { user } = useUserStore();

    useEffect(() => {
        fetchConversations(); // Fetch conversations on component mount
    }, [fetchConversations]);

    useEffect(() => {
        if (isLoading) return;
        console.log("Conversations updated:", conversations);
    }, [conversations]);

    useEffect(() => {
        if (!conversations || !user || isLoading) return;

        console.log(conversations);
        // Convert object to array if necessary
        Object.values(conversations).forEach((conversation) => {
            let channel = `message.group.${conversation.id}`;

            if (conversation.is_user) {
                channel = `message.user.${[parseInt(user.id), parseInt(conversation.id)]
                    .sort((a, b) => a - b)
                    .join("-")}`;
            }

            Echo.private(channel)
                .error((error) => {
                    console.error(`Error subscribing to ${channel}:`, error);
                })
                .listen("SocketMessage", (e) => {
                    const message = e.message;

                    emit("message.created", message);

                    if (message.sender_id === user.id) {
                        return;
                    }
```

```
                emit("newMessageNotification", {
                    user: message.sender,
                    group_id: message.group_id,
                    message: message.message ||
                        `Shared ${message.attachments.length === 1
                            ? "an attachment"
                            : `${message.attachments.length} attachments`}`,
                });
            });

        if (conversation.is_group) {
            Echo.private(`group.deleted.${conversation.id}`)
                .listen("GroupDeleted", (e) => {
                    emit("group.deleted", { id: e.id, name: e.name });
                })
                .error((e) => {
                    console.error(`Error subscribing to group.deleted.${conversation.id}:`, e);
                });
        }
    });

    return () => {
        Object.values(conversations).forEach((conversation) => {
            let channel = `message.group.${conversation.id}`;

            if (conversation.is_user) {
                channel = `message.user.${[parseInt(user.id), parseInt(conversation.id)]
                    .sort((a, b) => a - b)
                    .join("-")}`;
            }

            Echo.leave(channel);

            if (conversation.is_group) {
                Echo.leave(`group.deleted.${conversation.id}`);
            }
        });
    };
}, [conversations, user, emit]);

const handleConversationSelect = (conversation) => {
    setSelectedConversation(conversation);
};

const isUserOnline = (userId) => onlineUsers[userId];

const onSearch = (ev) => {
    const search = ev.target.value.toLowerCase();
    setLocalConversations(
        Object.values(conversations).filter((conversation) => {
            return conversation.name.toLowerCase().includes(search);
        })
    );
};

const messageCreated = (message) => {
    setLocalConversations((oldConversations) => {
        return oldConversations.map((u) => {
            if (
                message.receiver_id &&
                !u.is_group &&
                (u.id == message.sender_id || u.id == message.receiver_id)
            ) {
                u.last_message = message.message;
                u.last_message_date = message.created_at;
                return u;
```

```
                }
                if (
                    message.group_id &&
                    u.is_group &&
                    u.id == message.group_id
                ) {
                    u.last_message = message.message;
                    u.last_message_date = message.created_at;
                    return u;
                }
                return u;
            });
        });
    };

    const messageDeleted = ({ prevMessage }) => {
        if (!prevMessage) return;

        messageCreated(prevMessage);
    };

    useEffect(() => {
        const offCreated = on("message.created", messageCreated);
        const offDeleted = on("message.deleted", messageDeleted);
        const offModalShow = on("GroupModal.show", () => setShowGroupModal(true));
        const offGroupDelete = on("group.deleted", ({ id, name }) => {
            setLocalConversations((oldConversations) => oldConversations.filter((con) => con.id !==
id));
            emit("toast.show", `Group "${name}" was deleted`);
        });

        return () => {
            offCreated();
            offDeleted();
            offModalShow();
            offGroupDelete();
        };
    }, [on]);

    useEffect(() => {
        setSortedConversations(
            [...localConversations].sort((a, b) => {
                if (a.last_message_date && b.last_message_date) {
                    return b.last_message_date.localeCompare(a.last_message_date);
                } else if (a.last_message_date) {
                    return -1;
                } else if (b.last_message_date) {
                    return 1;
                }
                return 0;
            })
        );
    }, [localConversations]);

    useEffect(() => {
        setLocalConversations(Object.values(conversations));
    }, [conversations]);

    useEffect(() => {
        Echo.join("online")
            .here((users) => {
                const onlineUsersObj = Object.fromEntries(
                    users.map((user) => [user.id, user])
                );
                setOnlineUsers((prevOnlineUsers) => ({ ...prevOnlineUsers, ...onlineUsersObj }));
            })
```

```
            .joining((user) => {
                setOnlineUsers((prevOnlineUsers) => ({ ...prevOnlineUsers, [user.id]: user }));
            })
            .leaving((user) => {
                setOnlineUsers((prevOnlineUsers) => {
                    const updatedUsers = { ...prevOnlineUsers };
                    delete updatedUsers[user.id];
                    return updatedUsers;
                });
            })
            .error((error) => {
                console.error("error", error);
            });

        return () => {
            Echo.leave("online");
        };
    }, []);

    return (
        <>
            <div className="flex-1 w-full flex overflow-hidden h-screen">
                <div className={`transition-all w-full sm:w-[220px] md:w-[300px] bg-slate-800 flex
flex-col overflow-hidden ${
                    selectedConversation ? "-ml-[100%] sm:ml-0" : ""}`}>
                    <div className="flex items-center justify-between py-2 px-3 text-xl font-medium
text-gray-200">
                        My Conversations
                        <div className="tooltip tooltip-left" data-tip="Create new Group">
                            <button onClick={() => setShowGroupModal(true)} className="text-gray-
400 hover:text-gray-200">
                                <PencilSquareIcon className="w-4 h-4 inline-block ml-2" />
                            </button>
                        </div>
                    </div>
                    <div className="p-3">
                        <input
                            onKeyUp={onSearch}
                            placeholder="Filter users and groups"
                            className="w-full p-2 text-gray-700 rounded-md"
                        />
                    </div>
                    <div className="flex-1 overflow-auto">
                        {sortedConversations && sortedConversations.map((conversation) => (
                            <ConversationItem
                                key={`${
                                    conversation.is_group ? "group_" : "user_"
                                }${conversation.id}`}
                                conversation={conversation}
                                online={!!isUserOnline(conversation.id)}
                                selectedConversation={selectedConversation}
                                setSelectedConversation={handleConversationSelect}
                            />
                        ))}
                    </div>
                </div>
                <div className="flex-1 flex flex-col overflow-hidden">
                    <ChatMain selectedConversation={selectedConversation} isUser=
{selectedConversation?.is_user}/>
                </div>
            </div>
            <GroupModal show={showGroupModal} onClose={() => setShowGroupModal(false)} />
        </>
    );
};
```

```
export default ChatWrapper;
```

FILE: frontend/src/pages/Chat/ConversationHeader.jsx

```
import { Link } from "react-router-dom"; // Updated to react-router-dom
import {
    ArrowLeftIcon,
    PencilSquareIcon,
    TrashIcon,
} from "@heroicons/react/24/solid";
import UserAvatar from "./UserAvatar";
import GroupAvatar from "./GroupAvatar";
import GroupDescriptionPopover from "./GroupDescriptionPopover";
import GroupUsersPopover from "./GroupUsersPopover";
import { useEventBus } from "../../EventBus";
import { useUserStore } from "../../stores/useUserStore";
import useChatStore from "../../stores/useChatStore"; // Import useChatStore

const ConversationHeader = ({ selectedConversation }) => {
    const { user } = useUserStore();
    const { emit } = useEventBus();
    const { deleteGroup } = useChatStore(); // Get the deleteGroup function from the store

    const onDeleteGroup = () => {
        if (!window.confirm("Are you sure you want to delete this group?")) {
            return;
        }

        // Use deleteGroup from the store instead of making an API call directly
        deleteGroup(selectedConversation.id);
    };

    return (
        <>
            {selectedConversation && (
                <div className="p-3 flex justify-between items-center border-b border-slate-700">
                    <div className="flex items-center gap-3">
                        <Link
                            to="/" // Changed to 'to' for react-router-dom
                            className="inline-block sm:hidden"
                        >
                            <ArrowLeftIcon className="w-6" />
                        </Link>
                        {selectedConversation.is_user && (
                            <UserAvatar user={selectedConversation} />
                        )}
                        {selectedConversation.is_group && <GroupAvatar />}
                        <div>
                            <h3>{selectedConversation.name}</h3>
                            {selectedConversation.is_group && (
                                <p className="text-xs text-gray-500">
                                    {selectedConversation.users.length} members
                                </p>
                            )}
                        </div>
                    </div>
                    {selectedConversation.is_group && (
                        <div className="flex gap-3">
                            <GroupDescriptionPopover
                                description={selectedConversation.description}
                            />
                            <GroupUsersPopover
                                users={selectedConversation.users}
                            />
                            {selectedConversation.owner_id == user.id && (
                                <>
```

```
                                        <div
                                            className="tooltip tooltip-left"
                                            data-tip="Edit Group"
                                        >
                                            <button
                                                onClick={(ev) =>
                                                    emit(
                                                        "GroupModal.show",
                                                        selectedConversation
                                                    )
                                                }
                                                className="text-gray-400 hover:text-gray-200"
                                            >
                                                <PencilSquareIcon className="w-4" />
                                            </button>
                                        </div>
                                        <div
                                            className="tooltip tooltip-left"
                                            data-tip="Delete Group"
                                        >
                                            <button
                                                onClick={onDeleteGroup}
                                                className="text-gray-400 hover:text-gray-200"
                                            >
                                                <TrashIcon className="w-4" />
                                            </button>
                                        </div>
                                    </>
                                )}
                            </div>
                        )}
                    </div>
                )}
            </>
        );
};

export default ConversationHeader;
```

FILE: frontend/src/pages/Chat/ConversationItem.jsx

```
import UserAvatar from "./UserAvatar";
import GroupAvatar from "./GroupAvatar";
import { formatMessageDateShort } from "../../helpers";
import { useUserStore } from "../../stores/useUserStore";

const ConversationItem = ({
  conversation,
  selectedConversation = null,
  online = null,
  setSelectedConversation,
}) => {
  const { user } = useUserStore();

  let classes = "border-transparent";

  if (selectedConversation) {
    if (
      !selectedConversation.is_group &&
      !conversation.is_group &&
      selectedConversation.id === conversation.id
    ) {
      classes = "border-blue-500 bg-black/20";
    }
    if (
      selectedConversation.is_group &&
```

```
      conversation.is_group &&
      selectedConversation.id === conversation.id
    ) {
      classes = "border-blue-500 bg-black/20";
    }
  }

  return (
    <div
      onClick={() => setSelectedConversation(conversation)} // Fixed the function call
      className={
        "conversation-item flex items-center gap-2 p-2 text-gray-300 transition-all cursor-pointer
border-l-4 hover:bg-black/30 " +
        classes +
        (conversation.is_user ? " pr-2" : " pr-4")
      }
    >
      {/* User Avatar or Group Avatar */}
      {conversation.is_user ? (
        <UserAvatar user={conversation} online={online} />
      ) : (
        <GroupAvatar group={conversation} />
      )}

      <div
        className="flex-1 text-xs max-w-full overflow-hidden"
      >
        <div className="flex gap-1 justify-between items-center">
          <h3 className="text-sm font-semibold overflow-hidden text-nowrap text-ellipsis">
            {conversation.name}
          </h3>
          {conversation.last_message_date && (
            <span className="text-nowrap">
              {formatMessageDateShort(conversation.last_message_date)}
            </span>
          )}
        </div>
        {conversation.last_message && (
          <p className="text-xs text-nowrap overflow-hidden text-ellipsis">
            {conversation.last_message}
          </p>
        )}
      </div>

    </div>
  );
};

export default ConversationItem;
```

FILE: frontend/src/pages/Chat/CustomAudioPlayer.jsx

```
import { PauseCircleIcon, PlayCircleIcon } from "@heroicons/react/24/solid";
import React, { useRef, useState } from "react";

const CustomAudioPlayer = ({ file, showVolume = true }) => {
    const audioRef = useRef();
    const [isPlaying, setIsPlaying] = useState(false);
    const [volume, setVolume] = useState(1);
    const [duration, setDuration] = useState(0);
    const [currentTime, setCurrentTime] = useState(0);

    const togglePlayPause = () => {
        const audio = audioRef.current;
        if (isPlaying) {
            audio.pause();
```

```
        } else {
            console.log(audio, audio.duration);
            setDuration(audio.duration);
            audio.play();
        }
        setIsPlaying(!isPlaying);
    };

    const handleVolumeChange = (e) => {
        const volume = e.target.value;
        audioRef.current.volume = volume;
        setVolume(volume);
    };

    const handleTimeUpdate = (e) => {
        const audio = audioRef.current;
        setDuration(audio.duration);
        setCurrentTime(e.target.currentTime);
    };

    const handleLoadedMetadata = (e) => {
        setDuration(e.target.duration);
    };

    const handleSeekChange = (e) => {
        const time = e.target.value;
        audioRef.current.currentTime = time;
        setCurrentTime(time);
    };

    return (
        <div className="w-full flex items-center gap-2 py-2 px-3 rounded-md bg-slate-800">
            <audio
                ref={audioRef}
                src={file.url}
                controls
                onTimeUpdate={handleTimeUpdate}
                onLoadedMetadata={handleLoadedMetadata}
                className="hidden"
            />
            <button onClick={togglePlayPause}>
                {isPlaying && <PauseCircleIcon className="w-6 text-gray-400" />}
                {!isPlaying && <PlayCircleIcon className="w-6 text-gray-400" />}
            </button>
            {showVolume && (
                <input
                    type="range"
                    min="0"
                    max="1"
                    step="0.01"
                    value={volume}
                    onChange={handleVolumeChange}
                />
            )}
            <input
                type="range"
                className="flex-1"
                min="0"
                max={duration}
                step="0.01"
                value={currentTime}
                onChange={handleSeekChange}
            />
        </div>
    );
};
```

```
export default CustomAudioPlayer;
```

FILE: frontend/src/pages/Chat/GroupAvatar.jsx

```
import { UsersIcon } from "@heroicons/react/24/solid";

const GroupAvatar = ({}) => {
    return (
        <>
            <div className={`avatar placeholder`}>
                <div className={`bg-gray-400 text-gray-800 rounded-full w-8`}>
                    <span className="text-xl">
                        <UsersIcon className="w-4" />
                    </span>
                </div>
            </div>
        </>
    );
};

export default GroupAvatar;
```

FILE: frontend/src/pages/Chat/GroupDescriptionPopover.jsx

```
import { Popover, Transition } from "@headlessui/react";
import { ExclamationCircleIcon } from "@heroicons/react/24/solid";
import { Fragment } from "react";

export default function GroupDescriptionPopover({ description }) {
    return (
        <Popover className="relative">
            {({ open }) => (
                <>
                    <Popover.Button
                        className={`${
                            open ? "text-gray-200" : "text-gray-400"
                        } hover:text-gray-200`}
                    >
                        <ExclamationCircleIcon className="w-4" />
                    </Popover.Button>
                    <Transition
                        as={Fragment}
                        enter="transition ease-out duration-200"
                        enterFrom="opacity-0 translate-y-1"
                        enterTo="opacity-100 translate-y-0"
                        leave="transition ease-in duration-150"
                        leaveFrom="opacity-100 translate-y-0"
                        leaveTo="opacity-0 translate-y-1"
                    >
                        <Popover.Panel className="absolute right-0 z-10 mt-3 w-[300px] px-4 sm:px-
0">
                            <div className="overflow-hidden rounded-lg shadow-lg ring-1 ring-
black/5">
                                <div className="bg-gray-800 p-4">
                                    <h2 className="text-lg mb-3">
                                        Description
                                    </h2>
                                    {description && (
                                        <div className="text-xs">
                                            {description}
                                        </div>
                                    )}
                                    {!description && (
                                        <div className="text-xs text-gray-500 text-center py-4">
```

```
                                            No description is defined.
                                        </div>
                                    )}
                                </div>
                            </div>
                        </Popover.Panel>
                    </Transition>
                </>
            )}
        </Popover>
    );
}
```

FILE: frontend/src/pages/Chat/GroupModal.jsx

```
import { useState, useEffect } from "react";
import useChatStore from "../../stores/useChatStore";
import useSocialMediaStore from "../../stores/useSocialMediaStore";

const GroupModal = ({ show = false, onClose = () => {} }) => {
  const [group, setGroup] = useState({});
  const {
    recommendedUsers,
    isLoadingRecommendedUsers,
    fetchRecommendedUsers,
    isErrorRecommendedUsers,
  } = useSocialMediaStore();

  const [data, setData] = useState({
    id: "",
    name: "",
    description: "",
    user_ids: [],
  });
  const [errors, setErrors] = useState({});
  const [processing, setProcessing] = useState(false);

  const { createOrUpdateGroup } = useChatStore(); // Get the store method

  const handleSubmit = async (e) => {
    e.preventDefault();
    setProcessing(true);
    try {
      await createOrUpdateGroup(data, group.id); // Pass group data and group ID if it exists
      onClose(); // Close the modal after the group is created/updated
    } catch (err) {
      setErrors(err.response?.data?.errors || {});
    } finally {
      setProcessing(false);
    }
  };

  const closeModal = () => {
    setData({
      id: "",
      name: "",
      description: "",
      user_ids: [],
    });
    setErrors({});
    onClose();
  };

  useEffect(() => {
    if (show && group.id) {
      setData({
```

```
        name: group.name,
        description: group.description,
        user_ids: group.users.filter((u) => group.owner_id !== u.id).map((u) => u.id),
      });
      setGroup(group);
    }
  }, [show, group]);

  useEffect(() => {
    fetchRecommendedUsers(); // Fetch recommended users data
  }, [fetchRecommendedUsers]);

  // Handle loading states
  if (isLoadingRecommendedUsers) {
    return <div className="text-emerald-400">Loading...</div>;
  }

  // Handle error states
  if (isErrorRecommendedUsers) {
    return <div className="text-emerald-400">Error loading data. Please try again later.</div>;
  }
  if (!show) return null;

  return (
    <div
      className="fixed inset-0 flex items-center justify-center bg-black bg-opacity-70 z-50" //
Black background with opacity
      onClick={closeModal}
    >
      <div
        className="bg-black text-white border-2 border-white p-6 w-full sm:w-96 rounded-lg"
        onClick={(e) => e.stopPropagation()}
      >
        <form onSubmit={handleSubmit}>
          <h2 className="text-xl font-medium text-white">
            {group.id ? `Edit Group "${group.name}"` : "Create new Group"}
          </h2>

          <div className="mt-8">
            <label htmlFor="name" className="block text-sm font-medium text-white">
              Name
            </label>
            <input
              id="name"
              type="text"
              className="mt-1 block w-full border border-gray-300 rounded-md shadow-sm focus:ring-
indigo-500 focus:border-indigo-500"
              value={data.name}
              disabled={!!group.id}
              onChange={(e) => setData({ ...data, name: e.target.value })}
              required
            />
            {errors.name && <span className="text-sm text-red-500">{errors.name}</span>}
          </div>

          <div className="mt-4">
            <label htmlFor="description" className="block text-sm font-medium text-white">
              Description
            </label>
            <textarea
              id="description"
              rows="3"
              className="mt-1 block w-full border border-gray-300 rounded-md shadow-sm focus:ring-
indigo-500 focus:border-indigo-500"
              value={data.description || ""}
              onChange={(e) => setData({ ...data, description: e.target.value })}
```

```
            />
            {errors.description && <span className="text-sm text-red-500">{errors.description}
</span>}
          </div>

          <div className="mt-4">
            <label htmlFor="users" className="block text-sm font-medium text-white">
              Select Users
            </label>
            <select
              id="users"
              multiple
              value={data.user_ids}
              onChange={(e) => {
                const selectedOptions = Array.from(e.target.selectedOptions, option =>
option.value);
                setData({ ...data, user_ids: selectedOptions });
              }}
              className="mt-1 block w-full border border-gray-300 rounded-md shadow-sm focus:ring-
indigo-500 focus:border-indigo-500"
            >
              {recommendedUsers.map((user) => (
                <option key={user.id} value={user.id}>
                  {user.first_name + " " + user.last_name}
                </option>
              ))}
            </select>
            {errors.user_ids && <span className="text-sm text-red-500">{errors.user_ids}</span>}
          </div>

          <div className="mt-6 flex justify-end">
            <button
              type="button"
              onClick={closeModal}
              className="bg-gray-700 text-white rounded-md px-4 py-2 hover:bg-gray-600"
            >
              Cancel
            </button>
            <button
              type="submit"
              className="ml-3 bg-indigo-600 text-white rounded-md px-4 py-2 hover:bg-indigo-700"
              disabled={processing}
            >
              {group.id ? "Update" : "Create"}
            </button>
          </div>
        </form>
      </div>
    </div>
  );
};

export default GroupModal;
```

FILE: frontend/src/pages/Chat/GroupUsersPopover.jsx

```
import { Popover, Transition } from "@headlessui/react";
import { UsersIcon } from "@heroicons/react/24/solid";
import { Fragment } from "react";
import UserAvatar from "./UserAvatar";
import { Link } from "react-router-dom"; // Use react-router-dom Link instead of @inertiajs/react
Link

export default function GroupUsersPopover({ users = [] }) {
    return (
        <Popover className="relative">
```

```
            {({ open }) => (
                <>
                    <Popover.Button
                        className={`${
                            open ? "text-gray-200" : "text-gray-400"
                        } hover:text-gray-200`}
                    >
                        <UsersIcon className="w-4" />
                    </Popover.Button>
                    <Transition
                        as={Fragment}
                        enter="transition ease-out duration-200"
                        enterFrom="opacity-0 translate-y-1"
                        enterTo="opacity-100 translate-y-0"
                        leave="transition ease-in duration-150"
                        leaveFrom="opacity-100 translate-y-0"
                        leaveTo="opacity-0 translate-y-1"
                    >
                        <Popover.Panel className="absolute right-0 z-20 mt-3 w-[240px] px-4 sm:px-
0">
                            <div className="overflow-hidden rounded-lg shadow-lg ring-1 ring-
black/5">
                                <div className="bg-gray-800 py-2">
                                    {users.map((user) => (
                                        <Link
                                            to={`/chat/user/${user.id}`} // Using react-router-
dom's `to` for routing
                                            key={user.id}
                                            className="flex items-center gap-2 py-2 px-3 hover:bg-
black/30"
                                        >
                                            <UserAvatar user={user} />
                                            <div className="text-xs">{user.first_name + ' ' +
user.last_name}</div>
                                        </Link>
                                    ))}
                                </div>
                            </div>
                        </Popover.Panel>
                    </Transition>
                </>
            )}
        </Popover>
    );
}
```

FILE: frontend/src/pages/Chat/MessageAttachments.jsx

```
import {
    PaperClipIcon,
    ArrowDownTrayIcon,
    PlayCircleIcon,
} from "@heroicons/react/24/solid";
import { isAudio, isImage, isPDF, isPreviewable, isVideo } from "../../helpers";
const MessageAttachments = ({ attachments, attachmentClick }) => {
    return (
        <>
            {attachments.length > 0 && (
                <div className="mt-2 flex flex-wrap justify-end gap-1">
                    {attachments.map((attachment, ind) => (
                        <div
                            onClick={(ev) => attachmentClick(attachments, ind)}
                            key={attachment.id}
                            className={
                                `group flex flex-col items-center justify-center text-gray-500
relative cursor-pointer ` +
```

```
                                (isAudio(attachment)
                                    ? "w-84"
                                    : "w-32 aspect-square bg-blue-100")
                            }
                        >
                            {!isAudio(attachment) && (
                                <a
                                    onClick={(ev) => ev.stopPropagation()}
                                    download
                                    href={attachment.url}
                                    className="z-20 opacity-100 group-hover:opacity-100 transition-
all w-8 h-8 flex items-center justify-center text-gray-100 bg-gray-700 rounded absolute right-0
top-0 cursor-pointer hover:bg-gray-800"
                                >
                                    <ArrowDownTrayIcon className="w-4 h-4" />
                                </a>
                            )}

                            {isImage(attachment) && (
                                <img
                                    src={attachment.url}
                                    className="object-contain aspect-square"
                                />
                            )}
                            {isVideo(attachment) && (
                                <div className="relative flex justify-center items-center">
                                    <PlayCircleIcon className="z-20 absolute w-16 h-16 text-white
opacity-70" />

                                    <div className="absolute left-0 top-0 w-full h-full bg-black/50
z-10"></div>
                                    <video src={attachment.url}></video>
                                </div>
                            )}
                            {isAudio(attachment) && (
                                <div className="relative flex justify-center items-center">
                                    <audio
                                        src={attachment.url}
                                        controls
                                    ></audio>
                                </div>
                            )}
                            {isPDF(attachment) && (
                                <div className="relative flex justify-center items-center">
                                    <div className="absolute left-0 top-0 right-0 bottom-0 "></div>
                                    <iframe
                                        src={attachment.url}
                                        className="w-full h-full"
                                    ></iframe>
                                </div>
                            )}
                            {!isPreviewable(attachment) && (
                                <a
                                    onClick={(ev) => ev.stopPropagation()}
                                    download
                                    href={attachment.url}
                                    className="flex flex-col justify-center items-center"
                                >
                                    <PaperClipIcon className="w-10 h-10 mb-3" />

                                    <small className="text-center">
                                        {attachment.name}
                                    </small>
                                </a>
                            )}
                        </div>
```

```
                    ))}
                </div>
            )}
        </>
    );
};

export default MessageAttachments;
```

FILE: frontend/src/pages/Chat/MessageInput.jsx

```
import { useState } from "react";
import {
    PaperClipIcon,
    PhotoIcon,
    FaceSmileIcon,
    HandThumbUpIcon,
    PaperAirplaneIcon,
    XCircleIcon,
} from "@heroicons/react/24/solid";
import NewMessageInput from "./NewMessageInput";
import EmojiPicker from "emoji-picker-react";
import { Popover } from "@headlessui/react";
import { isAudio, isImage } from "../../helpers";
import AttachmentPreview from "./AttachmentPreview";
import CustomAudioPlayer from "./CustomAudioPlayer";
import AudioRecorder from "./AudioRecorder";
import useChatStore from "../../stores/useChatStore";
const MessageInput = ({ conversation = null }) => {
    const [newMessage, setNewMessage] = useState("");
    const [inputErrorMessage, setInputErrorMessage] = useState("");
    const [messageSending, setMessageSending] = useState(false);
    const [chosenFiles, setChosenFiles] = useState([]);
    const [uploadProgress, setUploadProgress] = useState(0);

    const { sendMessage } = useChatStore(); // Get sendMessage from the store

    const onFileChange = (ev) => {
        const files = ev.target.files;

        const updatedFiles = [...files].map((file) => {
            return {
                file: file,
                url: URL.createObjectURL(file),
            };
        });
        ev.target.value = null;

        setChosenFiles((prevFiles) => {
            return [...prevFiles, ...updatedFiles];
        });
    };

    const onSendClick = () => {
        if (messageSending) {
            return;
        }
        if (newMessage.trim() === "" && chosenFiles.length === 0) {
            setInputErrorMessage(
                "Please provide a message or upload attachments."
            );

            setTimeout(() => {
                setInputErrorMessage("");
            }, 3000);
            return;
```

```
        }

        setMessageSending(true);
        sendMessage(newMessage, chosenFiles, conversation)
            .then(() => {
                setNewMessage("");
                setMessageSending(false);
                setUploadProgress(0);
                setChosenFiles([]);
            })
            .catch((error) => {
                setMessageSending(false);
                setChosenFiles([]);
                setInputErrorMessage(
                    error?.response?.data?.message || "An error occurred while sending message"
                );
            });
    };

    const onLikeClick = () => {
        if (messageSending) {
            return;
        }
        const data = {
            message: "👍",
        };
        if (conversation.is_user) {
            data["receiver_id"] = conversation.id;
        } else if (conversation.is_group) {
            data["group_id"] = conversation.id;
        }

        axios.post("/message/store", data);
    };

    const recordedAudioReady = (file, url) => {
        setChosenFiles((prevFiles) => [...prevFiles, { file, url }]);
    };

    return (
        <div className="flex flex-wrap items-start border-t border-slate-700 py-3">
            <div className="order-2 flex-1 xs:flex-none xs:order-1 p-2">
                <button className="p-1 text-gray-400 hover:text-gray-300 relative overflow-hidden">
                    <PaperClipIcon className="w-6" />
                    <input
                        type="file"
                        multiple
                        onChange={onFileChange}
                        className="absolute left-0 top-0 right-0 bottom-0 z-20 opacity-0 cursor-
pointer"
                    />
                </button>
                <button className="p-1 text-gray-400 hover:text-gray-300 relative overflow-hidden">
                    <PhotoIcon className="w-6" />
                    <input
                        type="file"
                        multiple
                        accept="image/*"
                        onChange={onFileChange}
                        className="absolute left-0 top-0 right-0 bottom-0 z-20 opacity-0 cursor-
pointer"
                    />
                </button>
                <AudioRecorder fileReady={recordedAudioReady} />
            </div>
            <div className="order-1 px-3 xs:p-0 min-w-[220px] basis-full xs:basis-0 xs:order-2
```

```
flex-1 relative">
                <div className="flex ">
                    <NewMessageInput
                        value={newMessage}
                        onSend={onSendClick}
                        onChange={(ev) => setNewMessage(ev.target.value)}
                    />
                    <button
                        onClick={onSendClick}
                        disabled={messageSending}
                        className="btn btn-info rounded-l-none"
                    >
                        <PaperAirplaneIcon className="w-6" />
                        <span className="hidden sm:inline">Send</span>
                    </button>
                </div>{" "}
                {!!uploadProgress && (
                    <progress
                        className="progress progress-info w-full"
                        value={uploadProgress}
                        max="100"
                    ></progress>
                )}
                {inputErrorMessage && (
                    <p className="text-xs text-red-400">{inputErrorMessage}</p>
                )}
                <div className="flex flex-wrap gap-1 mt-2">
                    {chosenFiles.map((file) => (
                        <div
                            key={file.file.name}
                            className={
                                `relative flex justify-between cursor-pointer ` +
                                (!isImage(file.file) ? " w-[240px]" : "")
                            }
                        >
                            {isImage(file.file) && (
                                <img
                                    src={file.url}
                                    alt=""
                                    className="w-16 h-16 object-cover"
                                />
                            )}
                            {isAudio(file.file) && (
                                <CustomAudioPlayer
                                    file={file}
                                    showVolume={false}
                                />
                                // <audio src={file.url} controls></audio>
                            )}
                            {!isAudio(file.file) && !isImage(file.file) && (
                                <AttachmentPreview file={file} />
                            )}

                            <button
                                onClick={() =>
                                    setChosenFiles(
                                        chosenFiles.filter(
                                            (f) =>
                                                f.file.name !== file.file.name
                                        )
                                    )
                                }
                                className="absolute w-6 h-6 rounded-full bg-gray-800 -right-2 -top-
2 text-gray-300 hover:text-gray-100 z-10"
                            >
                                <XCircleIcon className="w-6" />
```

```
                            </button>
                        </div>
                    ))}
                </div>
            </div>
            <div className="order-3 xs:order-3 p-2 flex">
                <Popover className="relative">
                    <Popover.Button className="p-1 text-gray-400 hover:text-gray-300">
                        <FaceSmileIcon className="w-6 h-6" />
                    </Popover.Button>
                    <Popover.Panel className="absolute z-10 right-0 bottom-full">
                        <EmojiPicker
                            theme="dark"
                            onEmojiClick={(ev) =>
                                setNewMessage(newMessage + ev.emoji)
                            }
                        ></EmojiPicker>
                    </Popover.Panel>
                </Popover>
                <button
                    onClick={onLikeClick}
                    className="p-1 text-gray-400 hover:text-gray-300"
                >
                    <HandThumbUpIcon className="w-6 h-6" />
                </button>
            </div>
        </div>
    );
};

export default MessageInput;
```

FILE: frontend/src/pages/Chat/MessageItem.jsx

```
import ReactMarkdown from "react-markdown";
import React from "react";
import UserAvatar from "./UserAvatar";
import { formatMessageDateLong } from "../../helpers";
import MessageAttachments from "./MessageAttachments";
import MessageOptionsDropdown from "./MessageOptionsDropdown";
import { useUserStore } from "../../stores/useUserStore";

const MessageItem = ({ message, attachmentClick }) => {
  const { user } = useUserStore();

  return (
    <div
      className={`flex ${message.sender_id === user.id ? "justify-end" : "justify-start"} my-2`}
    >
      {/* User Avatar */}
      <div className="flex-shrink-0">
        <UserAvatar user={message.sender} />
      </div>

      {/* Chat Content */}
      <div className={`max-w-xs ${message.sender_id === user.id ? "ml-4" : "mr-4"}`}>
        {/* Chat Header */}
        <div className="flex items-center text-sm text-gray-500">
          {message.sender_id !== user.id && (
            <span>
              {message.sender.first_name} {message.sender.last_name}
            </span>
          )}
          <time className="ml-2 text-xs text-gray-400">
            {formatMessageDateLong(message.created_at)}
          </time>
```

```
        </div>

        {/* Chat Bubble */}
        <div
          className={`relative mt-1 p-3 rounded-lg ${
            message.sender_id === user.id
              ? "bg-blue-500 text-white"
              : "bg-gray-100 text-gray-800"
          }`}
        >
          {/* Message Options Dropdown for the Sender */}
          {message.sender_id === user.id && <MessageOptionsDropdown message={message} />}

          {/* Message Content */}
          <div className="prose max-w-none">
            <ReactMarkdown>{message.message}</ReactMarkdown>
          </div>

          {/* Message Attachments */}
          <MessageAttachments
            attachments={message.attachments}
            attachmentClick={attachmentClick}
          />
        </div>
      </div>
    </div>
  );
};

export default MessageItem;
```

FILE: frontend/src/pages/Chat/MessageOptionsDropdown.jsx

```
import { Menu, Transition } from "@headlessui/react";
import { Fragment } from "react";
import { EllipsisVerticalIcon, TrashIcon } from "@heroicons/react/24/solid";
import { useEventBus } from "../../EventBus";
import useChatStore from "../../stores/useChatStore";
export default function MessageOptionsDropdown({ message }) {
    const { emit } = useEventBus();
    const { deleteMessage } = useChatStore(); // Get the deleteMessage method from the store

    const onMessageDelete = () => {
        deleteMessage(message.id) // Call deleteMessage from the store
            .then(() => {
                emit("message.deleted", { message });
            })
            .catch((error) => {
                console.error("Error deleting message", error);
            });
    };

    return (
        <div className="absolute right-full text-gray-100 top-1/2 -translate-y-1/2 z-10">
            <Menu as="div" className="relative inline-block text-left">
                <div>
                    <Menu.Button className="flex justify-center items-center w-8 h-8 rounded-full
hover:bg-black/40">
                        <EllipsisVerticalIcon className="h-5 w-5" />
                    </Menu.Button>
                </div>
                <Transition
                    as={Fragment}
                    enter="transition ease-out duration-100"
                    enterFrom="transform opacity-0 scale-95"
                    enterTo="transform opacity-100 scale-100"
```

```
                    leave="transition ease-in duration-75"
                    leaveFrom="transform opacity-100 scale-100"
                    leaveTo="transform opacity-0 scale-95"
                >
                    <Menu.Items className="absolute left-0 mt-2 w-48 rounded-md bg-gray-800 shadow-
lg z-[100]">
                        <div className="px-1 py-1 ">
                            <Menu.Item>
                                {({ active }) => (
                                    <button
                                        onClick={onMessageDelete}
                                        className={`${
                                            active
                                                ? "bg-black/30 text-white"
                                                : "text-gray-100"
                                        } group flex w-full items-center rounded-md px-2 py-2 text-
sm`}
                                    >
                                        <TrashIcon className="w-4 h-4 mr-2" />
                                        Delete
                                    </button>
                                )}
                            </Menu.Item>
                        </div>
                    </Menu.Items>
                </Transition>
            </Menu>
        </div>
    );
}
```

FILE: frontend/src/pages/Chat/NewMessageInput.jsx

```
import React, { useEffect, useRef } from "react";

const NewMessageInput = ({ value, onChange, onSend }) => {
    const input = useRef();

    const onInputKeyDown = (ev) => {
        if (ev.key === "Enter" && !ev.shiftKey) {
            ev.preventDefault();
            onSend();
        }
    };

    const onChangeEvent = (ev) => {
        setTimeout(() => {
            adjustHeight();
        }, 10);
        onChange(ev);
    };

    const adjustHeight = () => {
        setTimeout(() => {
            input.current.style.height = "auto";
            input.current.style.height = input.current.scrollHeight + 1 + "px";
        }, 100);
    };

    useEffect(() => {
        adjustHeight();
    }, [value]);

    return (
        <textarea
            ref={input}
```

```
            value={value}
            rows="1"
            placeholder="Type a message"
            onKeyDown={onInputKeyDown}
            onChange={(ev) => onChangeEvent(ev)}
            className="input input-bordered w-full rounded-r-none resize-none overflow-y-auto max-
h-40 text-black"
        ></textarea>
    );
};

export default NewMessageInput;
```

FILE: frontend/src/pages/Chat/NewMessageNotification.jsx

```
import { useEventBus } from "../../EventBus";
import { useEffect, useState } from "react";
import { v4 as uuidv4 } from "uuid";
import UserAvatar from "./UserAvatar";
import { Link } from "react-router-dom"; // Use react-router-dom's Link instead of @inertiajs/react
Link

export default function NewMessageNotification({}) {
    const [toasts, setToasts] = useState([]);
    const { on } = useEventBus();

    useEffect(() => {
        on("newMessageNotification", ({ message, user, group_id }) => {
            const uuid = uuidv4();

            setToasts((oldToasts) => [
                ...oldToasts,
                { message, uuid, user, group_id },
            ]);

            setTimeout(() => {
                setToasts((oldToasts) =>
                    oldToasts.filter((toast) => toast.uuid !== uuid)
                );
            }, 5000);
        });
    }, [on]);

    return (
        <div className="toast toast-top toast-center min-w-[300px]">
            {toasts.map((toast) => (
                <Link
                    key={toast.uuid}
                    to={
                        toast.group_id
                            ? `/chat/group/${toast.group_id}` // Using react-router-dom's `to` prop
                            : `/chat/user/${toast.user.id}`   // Using react-router-dom's `to` prop
                    }
                    className="alert alert-success py-3 px-4 text-gray-100 rounded-md w-full flex
items-center gap-2"
                >
                    <UserAvatar user={toast.user} />
                    <span className="text-nowrap text-ellipsis overflow-hidden">
                        {toast.message}
                    </span>
                </Link>
            ))}
        </div>
    );
}
```

FILE: frontend/src/pages/Chat/Toast.jsx

```
import { useEventBus } from "@/EventBus";
import { useEffect, useState } from "react";
import { v4 as uuidv4 } from "uuid";

export default function Toast({}) {
    const [toasts, setToasts] = useState([]);
    const { on } = useEventBus();

    useEffect(() => {
        on("toast.show", (message) => {
            const uuid = uuidv4();

            setToasts((oldToasts) => [...oldToasts, { message, uuid }]);

            setTimeout(() => {
                setToasts((oldToasts) =>
                    oldToasts.filter((toast) => toast.uuid !== uuid)
                );
            }, 5000);
        });
    }, [on]);

    return (
        <div className="toast min-w-[300px] w-full xs:w-auto">
            {toasts.map((toast, index) => (
                <div
                    key={toast.uuid}
                    className="alert alert-success py-3 px-4 text-gray-100 rounded-md "
                >
                    <span className="w-full text-nowrap overflow-ellipsis overflow-hidden">
                        {toast.message}
                    </span>
                </div>
            ))}
        </div>
    );
}
```

FILE: frontend/src/pages/Chat/UserAvatar.jsx

```
const UserAvatar = ({ user, online = null, profile = false }) => {
    let onlineClass =
        online === true ? "online" : online === false ? "offline" : "";

    const sizeClass = profile ? "w-40" : "w-8";

    return (
        <>
            {user.profile_picture && (
                <div className={`chat-image avatar ${onlineClass}`}>
                    <div className={`rounded-full ${sizeClass}`}>
                        <img src={user.profile_picture} />
                    </div>
                </div>
            )}
            {!user.profile_picture && (
                <div className={`chat-image avatar placeholder ${onlineClass}`}>
                    <div
                        className={`bg-gray-400 text-gray-800 rounded-full ${sizeClass}`}
                    >
                        <span className="text-xl">
                            {`${user.first_name} ${user.last_name}`}
                        </span>
```

```
                    </div>
                </div>
            )}
        </>
    );
};

export default UserAvatar;
```

FILE: frontend/src/pages/Chat/UserPicker.jsx

```
import { Fragment, useState } from "react";
import { Combobox, Transition } from "@headlessui/react";
import { CheckIcon, ChevronUpDownIcon } from "@heroicons/react/20/solid";

export default function UserPicker({ value, options, onSelect }) {
    const [selected, setSelected] = useState(value);
    const [query, setQuery] = useState("");

    const filteredPeople =
        query === ""
            ? options
            : options.filter((person) =>
                  person.name
                      .toLowerCase()
                      .replace(/\s+/g, "")
                      .includes(query.toLowerCase().replace(/\s+/g, ""))
              );

    const onSelected = (persons) => {
        setSelected(persons);
        onSelect(persons);
    };

    return (
        <>
            <Combobox value={selected} onChange={onSelected} multiple>
                <div className="relative mt-1">
                    <div className="relative w-full cursor-default overflow-hidden rounded-lg text-
left shadow-md focus:outline-none focus-visible:ring-2 focus-visible:ring-white/75 focus-
visible:ring-offset-2 focus-visible:ring-offset-teal-300 sm:text-sm">
                        <Combobox.Input
                            className="border-gray-300 dark:border-gray-700 dark:bg-gray-900
dark:text-gray-300 focus:border-indigo-500 dark:focus:border-indigo-600 focus:ring-indigo-500
dark:focus:ring-indigo-600 rounded-md shadow-sm mt-1 block w-full"
                            displayValue={(persons) =>
                                persons.length
                                    ? `${persons.length} users selected`
                                    : ""
                            }
                            placeholder="Select users..."
                            onChange={(event) => setQuery(event.target.value)}
                        />
                        <Combobox.Button className="absolute inset-y-0 right-0 flex items-center
pr-2">
                            <ChevronUpDownIcon
                                className="h-5 w-5 text-gray-400"
                                aria-hidden="true"
                            />
                        </Combobox.Button>
                    </div>
                    <Transition
                        as={Fragment}
                        leave="transition ease-in duration-100"
                        leaveFrom="opacity-100"
                        leaveTo="opacity-0"
```

```
                        afterLeave={() => setQuery("")}
                    >
                        <Combobox.Options className="absolute mt-1 max-h-60 w-full overflow-auto
rounded-md bg-gray-900 py-1 text-base shadow-lg ring-1 ring-black/5 focus:outline-none sm:text-sm">
                            {filteredPeople.length === 0 && query !== "" ? (
                                <div className="relative cursor-default select-none px-4 py-2 text-
gray-700">
                                    Nothing found.
                                </div>
                            ) : (
                                filteredPeople.map((person) => (
                                    <Combobox.Option
                                        key={person.id}
                                        className={({ active }) =>
                                            `relative cursor-default select-none py-2 pl-10 pr-4 ${
                                                active
                                                    ? "bg-teal-600 text-white"
                                                    : "bg-gray-900 text-gray-100"
                                            }`
                                        }
                                        value={person}
                                    >
                                        {({ selected, active }) => (
                                            <>
                                                <span
                                                    className={`block truncate ${
                                                        selected
                                                            ? "font-medium"
                                                            : "font-normal"
                                                    }`}
                                                >
                                                    {person.name}
                                                </span>
                                                {selected ? (
                                                    <span
                                                        className={`absolute inset-y-0 left-0 flex
items-center pl-3 text-white`}
                                                    >
                                                        <CheckIcon
                                                            className="h-5 w-5"
                                                            aria-hidden="true"
                                                        />
                                                    </span>
                                                ) : null}
                                            </>
                                        )}
                                    </Combobox.Option>
                                ))
                            )}
                        </Combobox.Options>
                    </Transition>
                </div>
            </Combobox>
            {selected && (
                <div className="flex flex-wrap gap-2 mt-3">
                    {selected.map((person) => (
                        <div
                            key={person.id}
                            className="badge badge-primary gap-2"
                        >
                            {person.name}
                        </div>
                    ))}
                </div>
            )}
        </>
```

```
    );
}
```

FILE: frontend/src/pages/ChatBot/ChatList.jsx

```
import { Link } from 'react-router-dom';
import { useEffect } from 'react';
import useChatBotStore from '../../stores/useChatBotStore';

const ChatList = () => {
  const { isLoading, error, chats, fetchUserChats } = useChatBotStore();

  useEffect(() => {
    fetchUserChats();
  }, [fetchUserChats]);

  return (
    <div className="pt-12 flex flex-col h-full">
      <Link to="/chatbot" className="p-2 rounded-md hover:bg-gray-700">
        Create a new Chat
      </Link>

      <hr className="border-none h-[2px] bg-gray-300 opacity-10 rounded-md my-5" />

      <span className="font-semibold text-xs mb-2">RECENT CHATS</span>

      <div className="flex flex-col overflow-y-auto">
        {isLoading ? (
          'Loading...'
        ) : error ? (
          'Something went wrong!'
        ) : (
          chats?.map((chat) => {
            const lastMessage =
              chat.history?.[chat.history.length - 1]?.parts?.[0]?.text || '';

            return (
              <Link
                to={`/chatbot/chats/${chat.id}`}
                key={chat.id}
                className="p-2 rounded-md hover:bg-gray-700 text-white truncate"
              >
                {lastMessage.length > 50
                  ? lastMessage.slice(0, 50) + '...'
                  : lastMessage}
              </Link>
            );
          })
        )}
      </div>
    </div>
  );
};

export default ChatList;
```

FILE: frontend/src/pages/ChatBot/ChatPage.jsx

```
import { useEffect } from "react";
import { useLocation } from "react-router-dom";
import Markdown from "react-markdown";
import NewPrompt from "./NewPrompt";
import useChatBotStore from "../../stores/useChatBotStore";

const ChatPage = () => {
```

```
  const path = useLocation().pathname;
  const chatId = path.split("/").pop();

  // Get the necessary state and actions from your Zustand store
  const { chat, isLoading, isError, errorMessage, fetchChat } = useChatBotStore();

  useEffect(() => {
    // Fetch the chat data when the component mounts
    fetchChat(chatId);
  }, [chatId, fetchChat]);

  return (
    <div className="flex flex-col items-center h-screen pt-6">
      <div className="flex-1 w-full overflow-y-auto p-5 pb-36">
        <div className="max-w-2xl mx-auto flex flex-col gap-5">
          {isLoading ? (
            <div className="text-center text-lg text-gray-500">Loading...</div>
          ) : isError ? (
            <div className="text-center text-lg text-red-500">{errorMessage}</div>
          ) : (
            chat?.history?.map((message, i) => (
              <div key={i} className="flex justify-start gap-3">
                <div
                  className={`${
                    message.role === "user"
                      ? "bg-[#2c2937] text-white self-end max-w-[80%] rounded-[20px] ml-auto"
                      : "bg-[#ffffff] text-black self-start max-w-[80%] rounded-[20px] mr-auto"
                  } p-4`}
                >
                  <Markdown>{message.parts[0].text}</Markdown>
                </div>
              </div>
            ))
          )}
        </div>
      </div>

      {/* New Prompt Section */}
      {chat && <NewPrompt data={chat} />}
    </div>
  );
};

export default ChatPage;
```

FILE: frontend/src/pages/ChatBot/DashboardLayout.jsx

```
import { Outlet } from "react-router-dom";
import ChatList from "./ChatList";

const DashboardLayout = () => {
  return (
    <div className="h-screen flex flex-col lg:flex-row">
      {/* Sidebar (Chat List) */}
      <div className="w-full lg:w-1/4 p-5">
        <ChatList />
      </div>

      {/* Main content area */}
      <div className="w-full lg:w-3/4 p-5">
        <Outlet />
      </div>
    </div>
  );
};
```

```
export default DashboardLayout;
```

FILE: frontend/src/pages/ChatBot/DashboardPage.jsx

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import useChatBotStore from "../../stores/useChatBotStore";
const DashboardPage = () => {
  const [text, setText] = useState("");
  const navigate = useNavigate();

  // Get the necessary state and actions from your Zustand store
  const { createChat, isLoading } = useChatBotStore();

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!text) return;

    createChat(text, navigate); // Call the createChat action from the store
  };

  return (
    <div className="flex flex-col items-center h-full">
      <div className="mt-auto w-1/2 bg-[#2c2937] rounded-3xl flex">
        <form onSubmit={handleSubmit} className="w-full flex items-center justify-between gap-5 p-
4">
          <input
            type="text"
            name="text"
            placeholder="Ask me anything..."
            value={text}
            onChange={(e) => setText(e.target.value)} // Handle text change
            className="flex-1 p-5 bg-transparent border-none outline-none text-[#ecececc]"
          />
          <button
            type="submit"
            className="bg-[#605e68] rounded-full p-3 flex items-center justify-center mr-5 cursor-
pointer"
          >
            {isLoading ? (
              <span>Loading...</span>
            ) : (
              <img src="/arrow.png" alt="" className="w-4 h-4" />
            )}
          </button>
        </form>
      </div>
    </div>
  );
};

export default DashboardPage;
```

FILE: frontend/src/pages/ChatBot/NewPrompt.jsx

```
import { useEffect, useRef, useState } from "react";
import model from "../../lib/gemini";
import { useChatBotStore } from "../../stores/useChatBotStore";

const NewPrompt = ({ data }) => {
  const [question, setQuestion] = useState("");
  const [answer, setAnswer] = useState("");
  const [isSending, setIsSending] = useState(false); // track sending status

  // Zustand store
```

```
  const { updateChat, fetchChatbotContext, chatbotContext } = useChatBotStore();

  // Scroll to bottom ref
  const endRef = useRef(null);
  const formRef = useRef(null);

  // Scroll on new messages
  useEffect(() => {
    endRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [data, question, answer]);

  // Fetch context on mount
  useEffect(() => {
    fetchChatbotContext();
  }, [fetchChatbotContext]);

  // Format initial chat history
  const formattedHistory =
    data?.history?.length > 0
      ? data.history.map(({ role, parts }) => ({
          role,
          parts: [{ text: parts[0].text }],
        }))
      : [{ role: "user", parts: [{ text: "Hello, how can I assist you?" }] }];

  // Initialize chat
  const chat = model.startChat({
    history: formattedHistory,
    generationConfig: {
      // maxOutputTokens: 100,
    },
  });

  // Flag to run initial message only once
  const hasRun = useRef(false);
  useEffect(() => {
    if (!hasRun.current && data?.history?.length === 1) {
      add(data.history[0].parts[0].text, true);
    }
    hasRun.current = true;
  }, []);

  // Main send function
  const add = async (text, isInitial) => {
    if (!isInitial) setQuestion(text);

    // Don't send if no text
    if (!text || text.trim() === "") return;

    // Check context validity
    const hasContext =
      chatbotContext && typeof chatbotContext === "string" && chatbotContext.trim() !== "";

    // Optionally, prevent sending if context is required
    // if (!hasContext) {
    //   console.warn("Context not ready yet!");
    //   return;
    // }

    setIsSending(true);

    try {
      // Build message
      let combinedMessage = text;
      if (hasContext) {
        combinedMessage = `
```

```
Here is the context data for you:
${chatbotContext}

Now, based on this context, please answer the following question:

${text}

Respond in a professional way and do not sound robotic.
`;
      }

      // Stream response
      const result = await chat.sendMessageStream([combinedMessage]);

      let accumulatedText = "";
      for await (const chunk of result.stream) {
        const chunkText = chunk.text();
        accumulatedText += chunkText;
        setAnswer(accumulatedText);
      }

      // Update chat store
      await updateChat(data.id, text, accumulatedText);
    } catch (err) {
      console.error("Gemini Error:", err);
    } finally {
      setIsSending(false);
    }
  };

  // Handle form submit
  const handleSubmit = async (e) => {
    e.preventDefault();
    const text = e.target.text.value;
    if (!text || text.trim() === "") return;
    add(text, false);
    e.target.reset();
  };

  return (
    <>
      {/* Messages */}
      <div className="messages flex flex-col gap-3 overflow-y-auto p-5">
        {question && (
          <div className="message user bg-[#3a3a3a] text-white p-3 rounded-lg self-end max-w-
[70%]">
            {question}
          </div>
        )}
        {answer && (
          <div className="message ai bg-[#505050] text-white p-3 rounded-lg self-start max-w-
[70%]">
            {answer}
          </div>
        )}
        <div className="endChat" ref={endRef}></div>
      </div>

      {/* Input form */}
      <form
        className="newForm flex items-center gap-5 px-5 py-2 absolute bottom-0 bg-[#2c2937]
rounded-2xl w-1/2"
        onSubmit={handleSubmit}
        ref={formRef}
      >
        <input
```

```
          type="text"
          name="text"
          placeholder={chatbotContext ? "Ask anything..." : "Loading context..."}
          className="flex-1 p-5 bg-transparent border-none outline-none text-[#ececec]"
          disabled={isSending || !chatbotContext} // disable if sending or context not ready
        />
        <button
          className={`w-12 h-12 rounded-full flex items-center justify-center cursor-pointer ${
            isSending || !chatbotContext ? "bg-gray-600 cursor-not-allowed" : "bg-[#605e68]"
          }`}
          disabled={isSending || !chatbotContext}
        >
          <img src="/arrow.png" alt="Send" className="w-4 h-4" />
        </button>
      </form>
    </>
  );
};

export default NewPrompt;
```

FILE: frontend/src/pages/Editor/ActiveImage.jsx

```
import React from 'react';
import { cn } from "../../lib/utils";
import { motion } from "framer-motion";
import ImageComparison from "../../components/Editor/ImageComparison";
import { useEditorStore } from "../../stores/useEditorStore";

export default function ActiveImage() {
  const { getState } = useEditorStore; // 🔥 getState for imperative state reads
  const state = getState();

  const {
    removingBackgroundGenerating,
    replacingBackgroundGenerating,
    extractingImageGenerating,
    genFillingGenerating,
    genRemovingGenerating,
    recoloringImageGenerating,
    croppingVideoGenerating,
    transcribingGenerating,
    uploadingImageGenerating,
    uploadingVideoGenerating,
    activeLayer,
    layerComparisonMode,
    comparedLayers,
    layers,
  } = state;

  console.log(activeLayer);

  const anyGenerating =
    removingBackgroundGenerating ||
    replacingBackgroundGenerating ||
    extractingImageGenerating ||
    genFillingGenerating ||
    genRemovingGenerating ||
    recoloringImageGenerating ||
    croppingVideoGenerating ||
    transcribingGenerating ||
    uploadingImageGenerating ||
    uploadingVideoGenerating;

  if (!activeLayer?.url && comparedLayers.length === 0) return null;
```

```
  const renderLayer = (layer) => (
    <div className="relative w-full h-full flex items-center justify-center">
      {layer.resourceType === "image" && (
        <img
          alt={layer.name || "Image"}
          src={layer.url || ""}
          className={cn(
            "rounded-lg object-contain",
            anyGenerating && "animate-pulse"
          )}
          style={{ width: "100%", height: "auto" }}
        />
      )}
      {layer.resourceType === "video" && (
        <video
          width={layer.width}
          height={layer.height}
          controls
          className="rounded-lg object-contain max-w-full max-h-full"
          src={layer.transcriptionURL || layer.url}
        />
      )}
    </div>
  );

  if (layerComparisonMode && comparedLayers.length > 0) {
    const comparisonLayers = comparedLayers
      .map((id) => layers.find((l) => l.id === id))
      .filter(Boolean);

    return (
      <motion.div
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        exit={{ opacity: 0 }}
        className="w-full relative h-screen p-24 bg-secondary flex flex-col items-center justify-
center"
      >
        <ImageComparison layers={comparisonLayers} />
      </motion.div>
    );
  }

  return (
    <div className="w-full relative h-screen p-24 bg-secondary flex flex-col items-center justify-
center">
      {renderLayer(activeLayer)}
    </div>
  );
}
```

FILE: frontend/src/pages/Editor/BgRemove.jsx

```
import { Button } from "../../components/Editor/Button";
import { Popover, PopoverContent, PopoverTrigger } from "../../components/Editor/Popover";
import { Image } from "lucide-react";
import { toast } from "sonner";
import { useEditorStore } from "../../stores/useEditorStore";

export default function BgRemove() {
  const { getState } = useEditorStore; // ✅ get direct state access functions
  const state = getState();
  console.log(state.activeLayer);
  return (
```

```
<Popover>
  <PopoverTrigger
    disabled={!getState().activeLayer?.url} // ✅ access latest activeLayer
    asChild
  >
    <Button variant="outline" className="py-8">
      <span className="flex gap-1 items-center justify-center flex-col text-xs font-medium">
        BG Removal
        <Image size={18} />
      </span>
    </Button>
  </PopoverTrigger>

  <PopoverContent className="w-full">
    <div className="grid gap-4">
      <div className="space-y-2">
        <h4 className="font-medium leading-none">Background Removal</h4>
        <p className="text-sm max-w-xs text-muted-foreground">
          Remove the background of an image with one simple click.
        </p>
      </div>
    </div>

    <Button
      disabled={() => {
        const state = getState();
        return (
          !state.activeLayer?.url ||
          !state.activeTag ||
          !state.activeColor ||
          state.removingBackgroundGenerating
        );
      }}
      className="w-full mt-4"
      onClick={async () => {
        const stateBefore = getState();
        const { activeLayer, removeBackground } = stateBefore;

        await removeBackground(
          activeLayer.url,
          activeLayer.format,
          activeLayer.id
        );

        const stateAfter = getState();
        const { removeBackgroundError, removingBackgroundGenerating } = stateAfter;
        const latestActiveLayer = stateAfter.activeLayer;

        if (removeBackgroundError) {
          toast.error(removeBackgroundError);
          return;
        }

        if (!removeBackgroundError && !removingBackgroundGenerating) {
          const newLayerId = crypto.randomUUID();
          stateAfter.addLayer({
            id: newLayerId,
            name: "bg-removed-" + latestActiveLayer.name,
            format: "png",
            height: latestActiveLayer.height,
            width: latestActiveLayer.width,
            url: latestActiveLayer.url,
            publicId: latestActiveLayer.publicId,
            resourceType: "image",
          });
          stateAfter.setActiveLayer(newLayerId);
```

```
            }
          }}
        >
          {getState().removingBackgroundGenerating ? "Removing..." : "Remove Background"}
        </Button>
      </PopoverContent>
    </Popover>
  );
}
```

FILE: frontend/src/pages/Editor/BgReplace.jsx

```
import { Button } from "../../components/Editor/Button";
import { Popover, PopoverContent, PopoverTrigger } from "../../components/Editor/Popover";
import { Input } from "../../components/Editor/Input";
import { Label } from "../../components/Editor/Label";
import { ImageOff } from "lucide-react";
import { useState } from "react";
import { toast } from "sonner";
import { useEditorStore } from "../../stores/useEditorStore";

export default function AIBackgroundReplace() {
  const { getState } = useEditorStore; // ✅ only getState for all state reads
  const [prompt, setPrompt] = useState("");
  const state = getState();
  console.log(state.activeLayer);
  return (
    <Popover>
      <PopoverTrigger
        disabled={!getState().activeLayer?.url} // ✅ latest activeLayer
        asChild
      >
        <Button variant="outline" className="py-8">
          <span className="flex gap-1 items-center justify-center flex-col text-xs font-medium">
            AI BG Replace
            <ImageOff size={18} />
          </span>
        </Button>
      </PopoverTrigger>

      <PopoverContent className="w-full">
        <div className="grid gap-4">
          <div className="space-y-2">
            <h4 className="font-medium leading-none">Generative Background Replace</h4>
            <p className="text-sm text-muted-foreground">
              Replace the background of your image with AI-generated content.
            </p>
          </div>
          <div class="grid gap-2">
            <div class="grid grid-cols-3 items-center gap-4">
              <Label htmlFor="prompt">Prompt (optional)</Label>
              <Input
                id="prompt"
                value={prompt}
                onChange={(e) => setPrompt(e.target.value)}
                placeholder="Describe the new background"
                className="col-span-2 h-8"
              />
            </div>
          </div>
        </div>

        <Button
          disabled={() => {
            const state = getState();
```

```
            return !state.activeLayer?.url || state.replacingBackgroundGenerating;
          }}
          className="w-full mt-4"
          onClick={async () => {
            const stateBefore = getState();
            const { activeLayer, replaceBackground } = stateBefore;

            if (!activeLayer?.url) {
              toast.error("No active layer to replace background!");
              return;
            }

            await replaceBackground(activeLayer.url, prompt, activeLayer.id);

            const stateAfter = getState();
            const {
              activeLayer: latestActiveLayer,
              replaceBackgroundError,
              replacingBackgroundGenerating,
              addLayer,
              setActiveLayer,
            } = stateAfter;

            if (replaceBackgroundError) {
              toast.error(replaceBackgroundError);
            } else if (!replacingBackgroundGenerating) {
              const newLayerId = crypto.randomUUID();
              addLayer({
                id: newLayerId,
                name: "bg-replaced-" + latestActiveLayer.name,
                format: latestActiveLayer.format,
                height: latestActiveLayer.height,
                width: latestActiveLayer.width,
                url: latestActiveLayer.url,
                publicId: latestActiveLayer.publicId,
                resourceType: "image",
              });
              setActiveLayer(newLayerId);
              toast.success("Background replaced successfully!");
            }
          }}
        >
          {getState().replacingBackgroundGenerating
            ? "Replacing..."
            : "Replace Background"}
        </Button>
      </PopoverContent>
    </Popover>
  );
}
```

FILE: frontend/src/pages/Editor/Editor.jsx

```
import UploadForm from "./UploadForm"
import ActiveImage from "./ActiveImage"
import Layers from "./Layers"
import ImageTools from "./ImageTools"
import VideoTools from "./VideoTools"
import Loading from "./Loading"
import { useEditorStore } from "../../stores/useEditorStore"
export default function Editor() {
  const activeLayer = useEditorStore((state) => state.activeLayer)
  console.log(activeLayer);
  return (
    <div className="flex h-full ">
      <div className="py-6 px-4  min-w-48 ">
```

```
        <div className="flex flex-col gap-4 ">
          {activeLayer.resourceType === "video" ? <VideoTools /> : null}
          {activeLayer.resourceType === "image" ? <ImageTools /> : null}
        </div>
      </div>
      <Loading />
      <ActiveImage />
      <UploadForm />
      <Layers />
    </div>
  )
}
```

FILE: frontend/src/pages/Editor/ExtractPart.jsx

```
import React, { useState } from "react";
import { Button } from "../../components/Editor/Button";
import { Popover, PopoverContent, PopoverTrigger } from "../../components/Editor/Popover";
import { Input } from "../../components/Editor/Input";
import { Label } from "../../components/Editor/Label";
import { Scissors } from "lucide-react";
import { Checkbox } from "../../components/Editor/Checkbox";
import { RadioGroup, RadioGroupItem } from "../../components/Editor/RadioGroup";
import { useEditorStore } from "../../stores/useEditorStore";
import { toast } from "sonner"; // Don't forget toast!

export default function ExtractPart() {
  const { getState } = useEditorStore; // ✅ use getState
  const extractImage = useEditorStore((state) => state.extractImage); // keep method access
reactive

  const [prompts, setPrompts] = useState([""]);
  const [multiple, setMultiple] = useState(false);
  const [mode, setMode] = useState("default");
  const [invert, setInvert] = useState(false);

  const addPrompt = () => setPrompts([...prompts, ""]);
  const updatePrompt = (index, value) => {
    const newPrompts = [...prompts];
    newPrompts[index] = value;
    setPrompts(newPrompts);
  };

  // pull state values manually on render (non-reactive)
  const state = getState();
  console.log(state.activeLayer);
  const isDisabled =
    !state.activeLayer?.url ||
    state.extractingImageGenerating ||
    prompts.every((p) => p.trim() === "");

  return (
    <Popover>
      <PopoverTrigger disabled={!state.activeLayer?.url} asChild>
        <Button variant="outline" className="py-8">
          <span className="flex gap-1 items-center justify-center flex-col text-xs font-medium">
            AI Extract
            <Scissors size={18} />
          </span>
        </Button>
      </PopoverTrigger>

      <PopoverContent className="w-full">
        <div className="grid gap-4">
          <div className="space-y-2">
```

```
        <h4 className="font-medium leading-none">AI Extract</h4>
        <p className="text-sm text-muted-foreground">
          Extract specific areas or objects from your image using AI.
        </p>
      </div>

      <div className="grid gap-2">
        {prompts.map((prompt, index) => (
          <div key={index} className="grid grid-cols-3 items-center gap-4">
            <Label htmlFor={`prompt-${index}`}>Prompt {index + 1}</Label>
            <Input
              id={`prompt-${index}`}
              value={prompt}
              onChange={(e) => updatePrompt(index, e.target.value)}
              placeholder="Describe what to extract"
              className="col-span-2 h-8"
            />
          </div>
        ))}
        <Button onClick={addPrompt} size="sm">
          Add Prompt
        </Button>

        <div className="flex items-center space-x-2">
          <Checkbox
            id="multiple"
            checked={multiple}
            onCheckedChange={(checked) => setMultiple(checked)}
          />
          <Label htmlFor="multiple">Extract multiple objects</Label>
        </div>

        <RadioGroup value={mode} onValueChange={setMode}>
          <div className="flex items-center space-x-2">
            <RadioGroupItem value="default" id="mode-default" />
            <Label htmlFor="mode-default">Default (transparent background)</Label>
          </div>
          <div className="flex items-center space-x-2">
            <RadioGroupItem value="mask" id="mode-mask" />
            <Label htmlFor="mode-mask">Mask</Label>
          </div>
        </RadioGroup>

        <div className="flex items-center space-x-2">
          <Checkbox
            id="invert"
            checked={invert}
            onCheckedChange={(checked) => setInvert(checked)}
          />
          <Label htmlFor="invert">Invert (keep background)</Label>
        </div>
      </div>
    </div>

    <Button
      disabled={isDisabled}
      className="w-full mt-4"
      onClick={async () => {
        const stateBefore = getState();
        const { activeLayer } = stateBefore;

        if (!activeLayer?.url) {
          toast.error("No active layer to extract from!");
          return;
        }
```

```
            await extractImage(
              activeLayer.url,
              prompts.filter((p) => p.trim() !== ""),
              multiple,
              mode,
              invert,
              activeLayer.format,
              activeLayer.id
            );

            const stateAfter = getState();
            const {
              activeLayer: latestActiveLayer,
              extractingImageGenerating,
              extractImageError,
              addLayer,
              setActiveLayer,
            } = stateAfter;

            if (latestActiveLayer?.url && !extractingImageGenerating) {
              const newLayerId = crypto.randomUUID();
              addLayer({
                id: newLayerId,
                name: "extracted-" + latestActiveLayer.name,
                format: ".png",
                height: latestActiveLayer.height,
                width: latestActiveLayer.width,
                url: latestActiveLayer.url,
                publicId: latestActiveLayer.publicId,
                resourceType: "image",
              });
              setActiveLayer(newLayerId);
              toast.success("Image extracted successfully!");
            } else {
              toast.error(extractImageError ?? "Failed to extract image.");
            }
          }}
        >
          {getState().extractingImageGenerating ? "Extracting..." : "Extract"}
        </Button>
      </PopoverContent>
    </Popover>
  );
}
```

FILE: frontend/src/pages/Editor/GenerativeFill.jsx

```
import React, { useMemo, useState } from "react";
import { Button } from "../../components/Editor/Button";
import { Popover, PopoverContent, PopoverTrigger } from "../../components/Editor/Popover";
import { Input } from "../../components/Editor/Input";
import { Label } from "../../components/Editor/Label";
import { Crop } from "lucide-react";
import { AnimatePresence, motion } from "framer-motion";
import { useEditorStore } from "../../stores/useEditorStore";

const PREVIEW_SIZE = 250;
const EXPANSION_THRESHOLD = 250; // px

export default function GenerativeFill() {
  const { getState } = useEditorStore; // ✅ getState for dynamic access

  const {
    addLayer,
    genFill,
```

```
    genFillingGenerating,
    setActiveLayer,
  } = useEditorStore((state) => ({
    addLayer: state.addLayer,
    genFill: state.genFill,
    genFillingGenerating: state.genFillingGenerating,
    setActiveLayer: state.setActiveLayer,
    genFillError: state.genFillError,
  }));

  const [height, setHeight] = useState(0);
  const [width, setWidth] = useState(0);

  const activeLayer = getState().activeLayer; // ✅ always access latest activeLayer
  console.log(activeLayer);
  const previewStyle = useMemo(() => {
    if (!activeLayer?.width || !activeLayer?.height) return {};

    const newWidth = activeLayer.width + width;
    const newHeight = activeLayer.height + height;

    const scale = Math.min(PREVIEW_SIZE / newWidth, PREVIEW_SIZE / newHeight);

    return {
      width: `${newWidth * scale}px`,
      height: `${newHeight * scale}px`,
      backgroundImage: `url(${activeLayer.url})`,
      backgroundSize: `${activeLayer.width * scale}px ${activeLayer.height * scale}px`,
      backgroundPosition: "center",
      backgroundRepeat: "no-repeat",
      position: "relative",
    };
  }, [activeLayer, width, height]);

  const previewOverlayStyle = useMemo(() => {
    if (!activeLayer?.width || !activeLayer?.height) return {};

    const scale = Math.min(
      PREVIEW_SIZE / (activeLayer.width + width),
      PREVIEW_SIZE / (activeLayer.height + height)
    );

    const leftWidth = width > 0 ? `${(width / 2) * scale}px` : "0";
    const rightWidth = width > 0 ? `${(width / 2) * scale}px` : "0";
    const topHeight = height > 0 ? `${(height / 2) * scale}px` : "0";
    const bottomHeight = height > 0 ? `${(height / 2) * scale}px` : "0";

    return {
      position: "absolute",
      top: "0",
      left: "0",
      right: "0",
      bottom: "0",
      boxShadow: `inset ${leftWidth} ${topHeight} 0 rgba(48, 119, 255, 1),
                  inset -${rightWidth} ${topHeight} 0 rgba(48, 119, 255, 1),
                  inset ${leftWidth} -${bottomHeight} 0 rgba(48, 119, 255, 1),
                  inset -${rightWidth} -${bottomHeight} 0 rgba(48, 119, 255,1)`,
    };
  }, [activeLayer, width, height]);

  const handleGenFill = async () => {
    const currentActiveLayer = getState().activeLayer; // ✅ dynamic fetch
    if (!currentActiveLayer) return;

    await genFill({
      width: (width + currentActiveLayer.width).toString(),
```

```
      height: (height + currentActiveLayer.height).toString(),
      aspect: "1:1",
      activeImage: currentActiveLayer.url,
      id: currentActiveLayer.id,
    });

    const latestState = getState(); // ✅ access latest state after async
    if (!latestState.genFillError && !latestState.genFillingGenerating) {
      const newLayerId = crypto.randomUUID();
      addLayer({
        id: newLayerId,
        name: "generative-fill",
        format: currentActiveLayer.format,
        height: height + currentActiveLayer.height,
        width: width + currentActiveLayer.width,
        url: latestState.activeLayer.url, // use latest state url
        publicId: currentActiveLayer.publicId,
        resourceType: "image",
      });
      setActiveLayer(newLayerId);
    } else {
      // handle error
      console.error("Gen fill failed: ", latestState.genFillError);
    }
  };

  const ExpansionIndicator = ({ value, axis }) => {
    const isVisible = Math.abs(value) >= EXPANSION_THRESHOLD;
    const position =
      axis === "x"
        ? { top: "50%", [value > 0 ? "right" : "left"]: 0, transform: "translateY(-50%)" }
        : { left: "50%", [value > 0 ? "bottom" : "top"]: 0, transform: "translateX(-50%)" };

    return (
      <AnimatePresence>
        {isVisible && (
          <motion.div
            initial={{ opacity: 0, scale: 0.5 }}
            animate={{ opacity: 1, scale: 1 }}
            exit={{ opacity: 0, scale: 0.5 }}
            className="absolute bg-primary text-white px-2 py-1 rounded-md text-xs font-bold"
            style={position}
          >
            {Math.abs(value)}px
          </motion.div>
        )}
      </AnimatePresence>
    );
  };

  return (
    <Popover>
      <PopoverTrigger disabled={!activeLayer?.url} asChild>
        <Button variant="outline" className="py-8">
          <span className="flex gap-1 items-center justify-center flex-col text-xs font-medium">
            Generative Fill
            <Crop size={18} />
          </span>
        </Button>
      </PopoverTrigger>
      <PopoverContent className="w-full">
        <div className="flex flex-col h-full">
          <div className="space-y-2">
            <h4 className="font-medium text-center py-2 leading-none">Generative Fill</h4>
            {activeLayer.width && activeLayer.height ? (
              <div className="flex justify-evenly">
```

```
            <div className="flex flex-col items-center">
              <span className="text-xs">Current Size:</span>
              <p className="text-sm text-primary font-bold">
                {activeLayer.width}x{activeLayer.height}
              </p>
            </div>
            <div className="flex flex-col items-center">
              <span className="text-xs">New Size:</span>
              <p className="text-sm text-primary font-bold">
                {activeLayer.width + width}x{activeLayer.height + height}
              </p>
            </div>
          </div>
        ) : null}
      </div>
      <div className="flex gap-2 items-center justify-center">
        <div className="text-center">
          <Label htmlFor="maxWidth">Modify Width</Label>
          <Input
            name="width"
            type="range"
            max={activeLayer.width}
            value={width}
            onChange={(e) => setWidth(parseInt(e.target.value))}
            className="h-8"
          />
        </div>
        <div className="text-center">
          <Label htmlFor="maxHeight">Modify Height</Label>
          <Input
            name="height"
            type="range"
            min={-activeLayer.height + 100}
            max={activeLayer.height}
            value={height}
            step={2}
            onChange={(e) => setHeight(parseInt(e.target.value))}
            className="h-8"
          />
        </div>
      </div>
      <div
        className="preview-container flex-grow"
        style={{
          width: `${PREVIEW_SIZE}px`,
          height: `${PREVIEW_SIZE}px`,
          display: "flex",
          justifyContent: "center",
          alignItems: "center",
          overflow: "hidden",
          margin: "auto",
        }}
      >
        <div style={previewStyle}>
          <div className="animate-pulsate" style={previewOverlayStyle}></div>
          <ExpansionIndicator value={width} axis="x" />
          <ExpansionIndicator value={height} axis="y" />
        </div>
      </div>
      <Button
        className="w-full mt-4"
        disabled={!activeLayer.url || (!width && !height) || genFillingGenerating}
        onClick={handleGenFill}
      >
        {genFillingGenerating ? "Generating..." : "Generative Fill 🎨"}
      </Button>
```

```
        </div>
      </PopoverContent>
    </Popover>
  );
}
```

FILE: frontend/src/pages/Editor/GenRemove.jsx

```
import { Button } from "../../components/Editor/Button"
import { Badge } from "../../components/Editor/Badge"
import { Popover, PopoverContent, PopoverTrigger } from "../../components/Editor/Popover"
import { Input } from "../../components/Editor/Input"
import { Label } from "../../components/Editor/Label"
import { cn } from "../../lib/utils"
import { Eraser } from "lucide-react"
import { useEditorStore } from "../../stores/useEditorStore" // Import store

export default function GenRemove() {
  const { getState, setState } = useEditorStore // Access getState and setState
  const state = getState();
  console.log(state.activeLayer);
  const handleClickTag = (tag) => {
    setState({ activeTag: tag })
  }

  const handleInputChange = (e) => {
    setState({ activeTag: e.target.value })
  }

  const handleGenRemove = async () => {
    const {
      activeLayer,
      activeTag,
      genRemove,
      genRemoveError,
      genRemovingGenerating,
      addLayer,
      setActiveLayer
    } = getState()

    await genRemove(activeLayer.url, activeTag, activeLayer.id)

    if (!genRemoveError && !genRemovingGenerating) {
      const newLayerId = crypto.randomUUID()
      addLayer({
        id: newLayerId,
        url: activeLayer,
        format: activeLayer.format,
        height: activeLayer.height,
        width: activeLayer.width,
        name: activeLayer.name,
        publicId: activeLayer.publicId,
        resourceType: "image",
      })
      setActiveLayer(newLayerId)
    }
  }

  const { tags, activeTag, activeColor, activeLayer, genRemovingGenerating } = getState()

  return (
    <Popover>
      <PopoverTrigger disabled={!activeLayer?.url} asChild>
        <Button variant="outline" className="p-8">
          <span className="flex gap-1 items-center justify-center flex-col text-xs font-medium">
            Content Aware <Eraser size={20} />
```

```
          </span>
        </Button>
      </PopoverTrigger>
      <PopoverContent className="w-full">
        <div className="grid gap-4">
          <div className="space-y-2">
            <h4 className="font-medium leading-none">Smart AI Remove</h4>
            <p className="text-sm text-muted-foreground">
              Generative Remove any part of the image
            </p>
          </div>
          <div className="grid gap-2">
            <h3 className="text-xs">Suggested selections</h3>
            <div className="flex gap-2">
              {tags.length === 0 && (
                <p className="text-xs text-muted-foreground">No tags available</p>
              )}
              {tags.map((tag) => (
                <Badge
                  key={tag}
                  onClick={() => handleClickTag(tag)}
                  className={cn(
                    "px-2 py-1 rounded text-xs",
                    activeTag === tag && "bg-primary text-white"
                  )}
                >
                  {tag}
                </Badge>
              ))}
            </div>
            <div className="grid grid-cols-3 items-center gap-4">
              <Label htmlFor="width">Selection</Label>
              <Input
                className="col-span-2 h-8"
                value={activeTag}
                name="tag"
                onChange={handleInputChange}
              />
            </div>
          </div>
        </div>
        <Button
          className="w-full mt-4"
          disabled={
            !activeTag || !activeColor || !activeLayer.url || genRemovingGenerating
          }
          onClick={handleGenRemove}
        >
          Magic Remove 🎨
        </Button>
      </PopoverContent>
    </Popover>
  )
}
```

FILE: frontend/src/pages/Editor/ImageTools.jsx

```
import AIRecolor from "./Recolor"
import GenerativeFill from "./GenerativeFill"
import GenRemove from "./GenRemove"
import BgRemove from "./BgRemove"
import AIBackgroundReplace from "./BgReplace"
import ExtractPart from "./ExtractPart"

export default function ImageTools() {
```

```
  return (
    <>
      <GenerativeFill />
      <AIRecolor />
      <GenRemove />
      <AIBackgroundReplace />
      <ExtractPart />
      <BgRemove />
    </>
  )
}
```

FILE: frontend/src/pages/Editor/LayerInfo.jsx

```
import { Dialog, DialogContent, DialogTrigger } from "../../components/Editor/Dialog"
import { Button } from "../../components/Editor/Button"
import { Ellipsis, Trash } from "lucide-react"
import { useEditorStore } from "../../stores/useEditorStore"

export default function LayerInfo({
  layer,
  layerIndex,
}) {
  const { layers, setActiveLayer, removeLayer } = useEditorStore((state) => ({
    layers: state.layers,
    setActiveLayer: state.setActiveLayer,
    removeLayer: state.removeLayer,
  }));


  return (
    <Dialog>
      <DialogTrigger asChild>
        <Button variant="outline">
          <Ellipsis size={18} />
        </Button>
      </DialogTrigger>
      <DialogContent className="text-xs">
        <h3 className="text-lg font-medium text-center mb-2">
          Layer {layer.id}
        </h3>
        <div className="py-4 space-y-0.5">
          <p>
            <span className="font-bold">Filename:</span> {layer.name}
          </p>
          <p>
            <span className="font-bold">Format:</span> {layer.format}
          </p>
          <p>
            <span className="font-bold"> Size:</span> {layer.width}X
            {layer.height}
          </p>
        </div>
        <Button
          onClick={(e) => {
            e.stopPropagation()
            setActiveLayer(layerIndex === 0 ? layers[1].id : layers[0].id)
            removeLayer(layer.id)
          }}
          variant={"destructive"}
          className="flex items-center gap-2 w-full"
        >
          <span> Delete Layer</span>
          <Trash size={14} />
        </Button>
      </DialogContent>
```

```
    </Dialog>
  )
}
```

FILE: frontend/src/pages/Editor/Layers.jsx

```
import React, { useMemo } from "react";
import { AnimatePresence, motion } from "framer-motion";
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from
"../../components/Editor/Card";
import { Button } from "../../components/Editor/Button";
import { ArrowRight, Images, Layers2 } from "lucide-react";
import LayerImage from "../../components/Editor/LayerImage";
import { cn } from "../../lib/utils";
import LayerInfo from "./LayerInfo";
import { useEditorStore } from "../../stores/useEditorStore";

export default function Layers() {
  const {
    layers,
    activeLayer,
    setActiveLayer,
    addLayer,
    removingBackgroundGenerating,
    replacingBackgroundGenerating,
    extractingImageGenerating,
    genFillingGenerating,
    genRemovingGenerating,
    recoloringImageGenerating,
    croppingVideoGenerating,
    transcribingGenerating,
    uploadingImageGenerating,
    uploadingVideoGenerating,
    layerComparisonMode,
    setLayerComparisonMode,
    comparedLayers,
    toggleComparedLayer,
    setComparedLayers,
  } = useEditorStore((state) => ({
    layers: state.layers,
    activeLayer: state.activeLayer,
    setActiveLayer: state.setActiveLayer,
    addLayer: state.addLayer,
    removingBackgroundGenerating: state.removingBackgroundGenerating,
    replacingBackgroundGenerating: state.replacingBackgroundGenerating,
    extractingImageGenerating: state.extractingImageGenerating,
    genFillingGenerating: state.genFillingGenerating,
    genRemovingGenerating: state.genRemovingGenerating,
    recoloringImageGenerating: state.recoloringImageGenerating,
    croppingVideoGenerating: state.croppingVideoGenerating,
    transcribingGenerating: state.transcribingGenerating,
    uploadingImageGenerating: state.uploadingImageGenerating,
    uploadingVideoGenerating: state.uploadingVideoGenerating,
    layerComparisonMode: state.layerComparisonMode,
    setLayerComparisonMode: state.setLayerComparisonMode,
    comparedLayers: state.comparedLayers,
    toggleComparedLayer: state.toggleComparedLayer,
    setComparedLayers: state.setComparedLayers,
  }));

  const generating =
    removingBackgroundGenerating ||
    replacingBackgroundGenerating ||
    extractingImageGenerating ||
    genFillingGenerating ||
    genRemovingGenerating ||
```

```
    recoloringImageGenerating ||
    croppingVideoGenerating ||
    transcribingGenerating ||
    uploadingImageGenerating ||
    uploadingVideoGenerating;

  const MCard = useMemo(() => motion(Card), []);
  const MButton = useMemo(() => motion(Button), []);

  const getLayerName = useMemo(
    () => (id) => {
      const layer = layers.find((l) => l.id === id);
      return layer ? layer.url : "Nothing here";
    },
    [layers]
  );

  const visibleLayers = useMemo(
    () =>
      layerComparisonMode
        ? layers.filter((layer) => layer.url && layer.resourceType === "image")
        : layers,
    [layerComparisonMode, layers]
  );

  return (
    <MCard
      layout
      className="basis-[320px] shrink-0  scrollbar-thin scrollbar-track-secondary overflow-y-scroll
scrollbar-thumb-primary scrollbar-thumb-rounded-full scrollbar-track-rounded-full overflow-x-hidden
relative flex flex-col shadow-2xl"
    >
      <CardHeader className="sticky top-0 z-50 px-4 py-6  min-h-28 bg-card shadow-sm">
        {layerComparisonMode ? (
          <div>
            <CardTitle className="text-sm pb-2">Comparing...</CardTitle>
            <CardDescription className="flex gap-2 items-center">
              <img
                alt="compare"
                width={32}
                height={32}
                src={getLayerName(comparedLayers[0])}
              />
              {comparedLayers.length > 0 && <ArrowRight />}
              {comparedLayers.length > 1 ? (
                <img
                  alt="compare"
                  width={32}
                  height={32}
                  src={getLayerName(comparedLayers[1])}
                />
              ) : (
                "Nothing here"
              )}
            </CardDescription>
          </div>
        ) : (
          <div className="flex flex-col gap-1 ">
            <CardTitle className="text-sm ">
              {activeLayer.name || "Layers"}
            </CardTitle>
            {activeLayer.width && activeLayer.height ? (
              <CardDescription className="text-xs">
                {activeLayer.width}X{activeLayer.height}
              </CardDescription>
            ) : null}
```

```
        </div>
      )}
    </CardHeader>
    <motion.div className="flex-1 flex flex-col ">
      <AnimatePresence>
        {visibleLayers.map((layer, index) => (
          <motion.div
            animate={{ scale: 1, opacity: 1 }}
            initial={{ scale: 0, opacity: 0 }}
            exit={{ scale: 0, opacity: 0 }}
            layout
            className={cn(
              "cursor-pointer ease-in-out hover:bg-secondary border border-transparent",
              {
                "border-primary": layerComparisonMode
                  ? comparedLayers.includes(layer.id)
                  : activeLayer.id === layer.id,
                "animate-pulse": generating,
              }
            )}
            key={layer.id}
            onClick={() => {
              if (generating) return;
              if (layerComparisonMode) {
                toggleComparedLayer(layer.id);
              } else {
                console.log(layer.id);
                setActiveLayer(layer.id);
              }
            }}
          >
            <div className="relative p-4 flex items-center">
              <div className="flex gap-2 items-center h-8 w-full justify-between">
                {!layer.url ? (
                  <p className="text-xs font-medium justify-self-end ">
                    New layer
                  </p>
                ) : null}
                <LayerImage layer={layer} />
                {layers.length !== 1 && (
                  <LayerInfo layer={layer} layerIndex={index} />
                )}
              </div>
            </div>
          </motion.div>
        ))}
      </AnimatePresence>
    </motion.div>
    <CardContent className="sticky bottom-0 bg-card flex gap-2  shrink-0">
      <MButton
        layout
        onClick={() => {
          addLayer({
            id: crypto.randomUUID(),
            url: "",
            height: 0,
            width: 0,
            publicId: "",
            name: "",
            format: "",
          });
        }}
        variant="outline"
        className="w-full flex gap-2"
      >
        <span className="text-xs">Create Layer</span>
```

```
          <Layers2 className="text-secondary-foreground" size={18} />
        </MButton>
        <MButton
          disabled={!activeLayer.url || activeLayer.resourceType === "video"}
          layout
          onClick={() => {
            if (layerComparisonMode) {
              setLayerComparisonMode(!layerComparisonMode);
            } else {
              setComparedLayers([activeLayer.id]);
            }
          }}
          variant={layerComparisonMode ? "destructive" : "outline"}
          className="w-full flex gap-2"
        >
          <motion.span className={cn("text-xs font-bold")}>
            {layerComparisonMode ? "Stop Comparing" : "Compare"}
          </motion.span>
          {!layerComparisonMode && (
            <Images className="text-secondary-foreground" size={18} />
          )}
        </MButton>
      </CardContent>
    </MCard>
  );
}
```

FILE: frontend/src/pages/Editor/Loading.jsx

```
import { Dialog, DialogDescription, DialogHeader, DialogContent, DialogTitle } from
"../../components/Editor/Dialog"
import loadingAnimation from "../../../src/animations/loading.json"
import Lottie from "lottie-react"
import { useEditorStore } from "../../stores/useEditorStore"

export default function Loading() {
  const {
    removingBackgroundGenerating,
    replacingBackgroundGenerating,
    extractingImageGenerating,
    genFillingGenerating,
    genRemovingGenerating,
    recoloringImageGenerating,
    croppingVideoGenerating,
    transcribingGenerating,
    uploadingImageGenerating,
    uploadingVideoGenerating,
    activeLayer,
  } = useEditorStore((state) => ({
    removingBackgroundGenerating: state.removingBackgroundGenerating,
    replacingBackgroundGenerating: state.replacingBackgroundGenerating,
    extractingImageGenerating: state.extractingImageGenerating,
    genFillingGenerating: state.genFillingGenerating,
    genRemovingGenerating: state.genRemovingGenerating,
    recoloringImageGenerating: state.recoloringImageGenerating,
    croppingVideoGenerating: state.croppingVideoGenerating,
    transcribingGenerating: state.transcribingGenerating,
    uploadingImageGenerating: state.uploadingImageGenerating,
    uploadingVideoGenerating: state.uploadingVideoGenerating,
    activeLayer: state.activeLayer,
  }));

  const generating =
    removingBackgroundGenerating ||
    replacingBackgroundGenerating ||
    extractingImageGenerating ||
```

```
    genFillingGenerating ||
    genRemovingGenerating ||
    recoloringImageGenerating ||
    croppingVideoGenerating ||
    transcribingGenerating ||
    uploadingImageGenerating ||
    uploadingVideoGenerating;

  return (
    <Dialog open={generating}>
      <DialogContent className="sm:max-w-[425px] flex flex-col items-center">
        <DialogHeader>
          <DialogTitle>Editing {activeLayer.name}</DialogTitle>
          <DialogDescription>
            Please note that this operation might take up to a couple of seconds.
          </DialogDescription>
        </DialogHeader>
        <Lottie className="w-36" animationData={loadingAnimation} />
      </DialogContent>
    </Dialog>
  );
}
```

FILE: frontend/src/pages/Editor/Recolor.jsx

```
import { Button } from "../../components/Editor/Button"
import { Badge } from "../../components/Editor/Badge"
import { Popover, PopoverContent, PopoverTrigger } from "../../components/Editor/Popover"
import { Input } from "../../components/Editor/Input"
import { Label } from "../../components/Editor/Label"
import { cn } from "../../lib/utils"
import { Paintbrush } from "lucide-react"
import { useEditorStore } from "../../stores/useEditorStore"
import { toast } from "sonner"

export default function AIRecolor() {
  const { getState, setState } = useEditorStore

  const handleClickTag = (tag) => {
    setState({ activeTag: tag })
  }

  const handleInputTagChange = (e) => {
    setState({ activeTag: e.target.value })
  }

  const handleClickColor = (color) => {
    setState({ activeColor: color })
  }

  const handleInputColorChange = (e) => {
    setState({ activeColor: e.target.value })
  }
  const state = getState();
  console.log(state.activeLayer);
  const handleRecolor = async () => {
    const {
      activeLayer,
      activeTag,
      activeColor,
      recolorImage,
      addLayer,
      setActiveLayer,
      recolorImageError,
      recoloringImageGenerating,
    } = getState()
```

```
    if (!activeLayer?.url || !activeTag || !activeColor) return

    await recolorImage(
      activeLayer.url,
      "prompt_" + activeTag,
      "to-color_" + activeColor,
      activeLayer.id
    )

    if (!recolorImageError && !recoloringImageGenerating) {
      const newLayerId = crypto.randomUUID()
      addLayer({
        id: newLayerId,
        name: "recolored" + activeLayer.name,
        format: activeLayer.format,
        height: activeLayer.height,
        width: activeLayer.width,
        url: getState().res?.data?.success, // assuming result is stored in state
        publicId: activeLayer.publicId,
        resourceType: "image",
      })
      setActiveLayer(newLayerId)
    } else {
      toast.error(recolorImageError)
    }
  }

  const { tags, activeTag, activeColor, activeLayer, recoloringImageGenerating } = getState()

  return (
    <Popover>
      <PopoverTrigger disabled={!activeLayer?.url} asChild>
        <Button variant="outline" className="py-8">
          <span className="flex gap-1 items-center justify-center flex-col text-xs font-medium">
            AI Recolor
            <Paintbrush size={18} />
          </span>
        </Button>
      </PopoverTrigger>
      <PopoverContent className="w-full">
        <div className="grid gap-4">
          <div className="space-y-2">
            <h4 className="font-medium leading-none">Generative Recolor</h4>
            <p className="text-sm text-muted-foreground">
              Recolor any part of your image with generative recolor.
            </p>
          </div>
          <div className="grid gap-2">
            <h3 className="text-xs">Suggested selections</h3>
            <div className="flex gap-2">
              {tags.length === 0 ? (
                <p className="text-xs text-muted-foreground">No tags available</p>
              ) : (
                tags.map((tag) => (
                  <Badge
                    key={tag}
                    onClick={() => handleClickTag(tag)}
                    className={cn(
                      "px-2 py-1 rounded text-xs",
                      activeTag === tag && "bg-primary text-white"
                    )}
                  >
                    {tag}
                  </Badge>
                ))
```

```
            )}
          </div>
          <div className="grid grid-cols-3 items-center gap-4">
            <Label htmlFor="width">Selection</Label>
            <Input
              className="col-span-2 h-8"
              value={activeTag}
              name="tag"
              onChange={handleInputTagChange}
            />
          </div>
          <h3 className="text-xs">Suggested colors</h3>
          <div className="flex gap-2">
            <div
              className="w-4 h-4 bg-blue-500 rounded-sm cursor-pointer"
              onClick={() => handleClickColor("blue")}
            ></div>
            <div
              className="w-4 h-4 bg-red-500 rounded-sm cursor-pointer"
              onClick={() => handleClickColor("red")}
            ></div>
            <div
              className="w-4 h-4 bg-green-500 rounded-sm cursor-pointer"
              onClick={() => handleClickColor("green")}
            ></div>
            <div
              className="w-4 h-4 bg-yellow-500 rounded-sm cursor-pointer"
              onClick={() => handleClickColor("yellow")}
            ></div>
          </div>
          <div className="grid grid-cols-3 items-center gap-4">
            <Label htmlFor="maxWidth">Color</Label>
            <Input
              name="color"
              value={activeColor}
              onChange={handleInputColorChange}
              className="col-span-2 h-8"
            />
          </div>
        </div>
      </div>
      <Button
        disabled={
          !activeLayer?.url || !activeTag || !activeColor || recoloringImageGenerating
        }
        className="w-full mt-4"
        onClick={handleRecolor}
      >
        {recoloringImageGenerating ? "Recoloring..." : "Recolor"}
      </Button>
    </PopoverContent>
  </Popover>
)
}
```

FILE: frontend/src/pages/Editor/Resizeable.jsx

```
import React from "react"
import { motion, useMotionValue, useTransform } from "framer-motion"

export default function Resizable() {
  const x = useMotionValue(0)
  const y = useMotionValue(0)
  const handleDragEnd = (e) => {}

  return (
```

```
    <motion.div className="border-4 relative w-full h-full ">
      {/* Right overlay */}
      <motion.div
        className=" top-0 h-full bg-black opacity-50 flex items-center justify-center text-white
text-lg"
        style={{
          width: useTransform(x, (latest) => Math.max(0, latest)),
        }}
      >
        {useTransform(x, (latest) =>
          latest > 0 ? `+${Math.round(latest)}px` : ""
        )}
      </motion.div>
      <motion.div
        className=" top-0 h-full bg-black opacity-50 flex items-center justify-center text-white
text-lg"
        style={{
          width: useTransform(x, (latest) => Math.max(0, -latest)),
        }}
      >
        {useTransform(x, (latest) =>
          latest > 0 ? `+${Math.round(latest)}px` : ""
        )}
      </motion.div>
      {/* Right resize handle */}
      <motion.div
        className="absolute right-0 top-0 w-2 h-full cursor-ew-resize bg-orange-600"
        drag="x"
        dragConstraints={{ left: 0 }}
        dragElastic={0}
        onDragEnd={handleDragEnd}
        style={{ x }}
      />

      {/* Top resize handle */}
      <motion.div
        className="absolute left-0 top-0 w-full h-2 z-50 cursor-ns-resize bg-orange-600"
        drag="y"
        dragConstraints={{ bottom: 0 }}
        dragElastic={0}
        onDragEnd={handleDragEnd}
        style={{ y }}
      />
      <motion.div
        className="absolute w-full left-0 bg-black opacity-50 flex items-center justify-center
text-white text-lg"
        style={{
          top: useTransform(y, (latest) => Math.min(0, latest)),
          height: useTransform(y, (latest) => Math.max(0, -latest)),
        }}
      >
        {useTransform(y, (latest) =>
          latest < 0 ? `+${Math.round(-latest)}px` : ""
        )}
      </motion.div>
      <motion.div
        className="absolute w-full left-0 bg-black opacity-50 flex items-center justify-center
text-white text-lg"
        style={{
          bottom: useTransform(y, (latest) => Math.min(0, latest)),
          height: useTransform(y, (latest) => Math.max(0, -latest)),
        }}
      >
        {useTransform(y, (latest) =>
          latest < 0 ? `+${Math.round(-latest)}px` : ""
        )}
```

```
      </motion.div>

      {/* Border around the original image */}
      <div className="absolute inset-0  border border-primary pointer-events-none" />
    </motion.div>
  )
}
```

FILE: frontend/src/pages/Editor/SmartCrop.jsx

```
import React, { useState } from "react"
import { useEditorStore } from "../../stores/useEditorStore"
import { Button } from "../../components/Editor/Button"
import { Popover, PopoverContent, PopoverTrigger } from "../../components/Editor/Popover"
import { Crop, Square } from "lucide-react"
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from
"../../components/Editor/Card"
import TikTok from "./Tiktok"
import Youtube from "./Youtube"
import { cn } from "../../lib/utils"
import { toast } from "sonner"

export default function SmartCrop() {
  const { getState, setState } = useEditorStore()

  const [aspectRatio, setAspectRatio] = useState("16:9")
  const [height, setHeight] = useState(0)
  const [width, setWidth] = useState(0)
  const state = getState;
  console.log(state.activeLayer);
  const handleGenCrop = async () => {
    const {
      activeLayer,
      cropVideo,
      addLayer,
      setActiveLayer,
      cropVideoError,
      croppingVideoGenerating,
    } = getState()

    if (!activeLayer?.url || croppingVideoGenerating) return

    try {
      await cropVideo(activeLayer.url, aspectRatio, height, activeLayer.id)

      if (!cropVideoError && !croppingVideoGenerating) {
        const newLayerId = crypto.randomUUID()
        const thumbnailUrl = activeLayer?.url.replace(/\.[^/.]+$/, ".jpg")

        addLayer({
          id: newLayerId,
          name: "cropped " + activeLayer.name,
          format: activeLayer.format,
          height: height + activeLayer.height,
          width: width + activeLayer.width,
          url: activeLayer?.url,
          publicId: activeLayer.publicId,
          resourceType: "video",
          poster: thumbnailUrl,
        })

        toast.success("Video cropped successfully!")
        setActiveLayer(newLayerId)
      } else {
        toast.error("Failed to crop video")
      }
```

```
    } catch (e) {
      toast.error(cropVideoError)
      console.error("Error:", e)
    }
  }

  return (
    <Popover>
      <PopoverTrigger disabled={!getState().activeLayer?.url} asChild>
        <Button variant="outline" className="py-8">
          <span className="flex gap-1 items-center flex-col text-xs font-medium">
            Smart Crop
            <Crop size={18} />
          </span>
        </Button>
      </PopoverTrigger>
      <PopoverContent className="w-full">
        <div className="flex flex-col h-full">
          <div className="space-y-2 pb-4">
            <h3 className="font-medium text-center py-2 leading-none">
              Smart Recrop
            </h3>
          </div>
          <h4 className="text-md font-medium pb-2">Format</h4>
          <div className={"flex gap-4 items-center justify-center pb-2"}>
            <Card
              className={cn(
                aspectRatio === "16:9" ? " border-primary" : "",
                "p-4 w-36 cursor-pointer"
              )}
              onClick={() => setAspectRatio("16:9")}
            >
              <CardHeader className="text-center p-0">
                <CardTitle className="text-md">Youtube</CardTitle>
                <CardDescription>
                  <p className="text-sm font-bold">16:9</p>
                </CardDescription>
              </CardHeader>
              <CardContent className="flex items-center justify-center p-0 pt-2">
                <Youtube />
              </CardContent>
            </Card>
            <Card
              className={cn(
                aspectRatio === "9:16" ? " border-primary" : "",
                "p-4 w-36 cursor-pointer"
              )}
              onClick={() => setAspectRatio("9:16")}
            >
              <CardHeader className="p-0 text-center">
                <CardTitle className="text-md">Tiktok</CardTitle>
                <CardDescription>
                  <p className="text-sm font-bold">9:16</p>
                </CardDescription>
              </CardHeader>
              <CardContent className="flex items-center justify-center p-0 pt-2">
                <TikTok />
              </CardContent>
            </Card>
            <Card
              className={cn(
                aspectRatio === "1:1" ? " border-primary" : "",
                "p-4 w-36 cursor-pointer"
              )}
              onClick={() => setAspectRatio("1:1")}
            >
```

```
              <CardHeader className="p-0 text-center">
                <CardTitle className="text-md">Square</CardTitle>
                <CardDescription>
                  <p className="text-sm font-bold">1:1</p>
                </CardDescription>
              </CardHeader>
              <CardContent className="flex items-center justify-center p-0 pt-2">
                <Square className="w-10 h-10" />
              </CardContent>
            </Card>
          </div>

          <Button
            onClick={handleGenCrop}
            className="w-full mt-4"
            variant={"outline"}
            disabled={!getState().activeLayer?.url || getState().croppingVideoGenerating}
          >
            {getState().croppingVideoGenerating ? "Cropping..." : "Smart Crop 🎨"}
          </Button>
        </div>
      </PopoverContent>
    </Popover>
  )
}
```

FILE: frontend/src/pages/Editor/Tiktok.jsx

```
export default function TikTok() {
  return (
    <svg
      className="w-10 h-10"
      height="2500"
      width="2179"
      xmlns="http://www.w3.org/2000/svg"
      viewBox="-0.32296740998066475 -3.1283528999801873 42.68446958125966 42.128352899980186"
    >
      <g fill="none">
        <path
          d="M14 15.599v-1.486A13.1 13.1 0 0 0 12.337 14C5.535 14 0 19.18 0 25.547 0 29.452 2.086
32.91 5.267 35c-2.13-2.132-3.315-4.942-3.313-7.861 0-6.276 5.377-11.394 12.046-11.54"
          fill="#00f2ea"
        />
        <path
          d="M14.327 32c2.876 0 5.221-2.273 5.328-5.107l.01-25.292h4.65A8.72 8.72 0 0 1 24.164 0h-
6.35l-.011 25.293c-.106 2.832-2.453 5.105-5.328 5.105a5.329 5.329 0 0 1-2.476-.61A5.34 5.34 0 0 0
14.327 32m18.672-21.814V8.78a8.818 8.818 0 0 1-4.81-1.421A8.85 8.85 0 0 0 33 10.186"
          fill="#00f2ea"
        />
        <path
          d="M28 7.718A8.63 8.63 0 0 1 25.832 2h-1.697A8.735 8.735 0 0 0 28 7.718M12.325 20.065c-
2.94.004-5.322 2.361-5.325 5.27A5.267 5.267 0 0 0 9.854 30a5.2 5.2 0 0 1-1.008-3.073c.003-2.91
2.385-5.268 5.325-5.271.55 0 1.075.09 1.572.244v-6.4a11.72 11.72 0 0 0-1.572-.114c-.092
0-.183.006-.274.007v4.916a5.286 5.286 0 0 0-1.572-.244"
          fill="#ff004f"
        />
        <path
          d="M32.153 11v4.884a15.15 15.15 0 0 1-8.813-2.811V25.84c0 6.377-5.23 11.565-11.658
11.565-2.485 0-4.789-.778-6.682-2.097A11.67 11.67 0 0 0 13.528 39c6.429 0 11.659-5.188 11.659-
11.564V14.668A15.15 15.15 0 0 0 34 17.478v-6.283A8.87 8.87 0 0 1 32.153 11"
          fill="#ff004f"
        />
        <path
          d="M23.979 25.42V12.632A15.741 15.741 0 0 0 33 15.448v-4.89a9.083 9.083 0 0 1-4.912-
2.82C26.016 6.431 24.586 4.358 24.132 2h-4.747l-.01 25.215c-.11 2.824-2.505 5.09-5.44 5.09-
```

```
1.754-.002-3.398-.822-4.42-2.204-1.794-.913-2.919-2.716-2.92-4.682.003-2.92 2.44-5.285 5.45-
5.289.56 0 1.098.09 1.608.245v-4.933C7.202 15.589 2 20.722 2 27.016c0 3.045 1.219 5.816 3.205
7.885A12.115 12.115 0 0 0 12.045 37c6.58 0 11.934-5.195 11.934-11.58"
          fill="#fff"
        />
      </g>
    </svg>
  )
}
```

FILE: frontend/src/pages/Editor/Transcribe.jsx

```
import { Button } from "../../components/Editor/Button";
import { useEditorStore } from "../../stores/useEditorStore";
import { toast } from "sonner";
import { Captions } from "lucide-react";

export default function VideoTranscription() {
  const { getState, setState } = useEditorStore();
  const state = getState;
  console.log(state.activeLayer);
  const handleTranscribe = async () => {
    const {
      activeLayer,
      initiateTranscription,
      updateLayer,
      setActiveLayer,
      initiateTranscriptionError,
    } = getState();

    if (!activeLayer.publicId || activeLayer.resourceType !== "video") {
      toast.error("Please select a video layer first");
      return;
    }

    try {
      await initiateTranscription(activeLayer.publicId, activeLayer.id);

      if (activeLayer.subtitledVideoUrl) {
        // If the URL is available (set in the store after success), update layer
        updateLayer({
          ...activeLayer,
          transcriptionURL: activeLayer.subtitledVideoUrl,
        });
        setActiveLayer(activeLayer.id);
      }
    } catch (error) {
      toast.error(initiateTranscriptionError || "An error occurred during transcription.");
    }
  };

  const { activeLayer, transcribingGenerating, initiateTranscriptionError } = getState();

  return (
    <div className="flex items-center">
      {!activeLayer.transcriptionURL && (
        <Button
          className="py-8 w-full"
          onClick={handleTranscribe}
          disabled={transcribingGenerating || activeLayer.resourceType !== "video"}
          variant="outline"
        >
          <span className="flex gap-1 items-center justify-center flex-col text-xs font-medium">
            {transcribingGenerating ? "Transcribing..." : "Transcribe"}
            <Captions size={18} />
          </span>
```

```
        </Button>
      )}

      {activeLayer.transcriptionURL && (
        <Button className="py-8 w-full" variant="outline" asChild>
          <a
            href={activeLayer.transcriptionURL}
            target="_blank"
            rel="noopener noreferrer"
          >
            <span className="flex gap-1 items-center justify-center flex-col text-xs font-medium">
              View Transcription
              <Captions size={18} />
            </span>
          </a>
        </Button>
      )}

      {initiateTranscriptionError && (
        <p className="text-red-500 text-xs mt-2">{initiateTranscriptionError}</p>
      )}
    </div>
  );
}
```

FILE: frontend/src/pages/Editor/UploadForm.jsx

```
import { Card, CardContent } from "../../components/Editor/Card";
import { cn } from "../../lib/utils";
import { RadioGroup, RadioGroupItem } from "../../components/Editor/RadioGroup";
import { Label } from "../../components/Editor/Label";
import { ImageIcon, VideoIcon } from "lucide-react";
import { useState } from "react";
import UploadImage from "./UploadImage";
import UploadVideo from "./UploadVideo";
import { useEditorStore } from "../../stores/useEditorStore";

export default function UploadForm() {
  const { getState } = useEditorStore; // getState for imperative reads
  const [selectedType, setSelectedType] = useState("image");

  const latestState = getState();
  const activeLayer = latestState.activeLayer;
  const layerComparisonMode = latestState.layerComparisonMode;

  console.log(activeLayer);

  if (!activeLayer.url && !layerComparisonMode) {
    return (
      <div className="w-full p-24 flex flex-col justify-center h-full">
        {selectedType === "image" ? <UploadImage /> : null}
        {selectedType === "video" ? <UploadVideo /> : null}

        <RadioGroup
          defaultValue="image"
          onValueChange={(e) => setSelectedType(e)}
          className="flex items-center justify-center gap-8 py-8"
        >
          <Card
            onClick={() => setSelectedType("image")}
            className={cn(
              "flex flex-col items-center justify-center py-4 px-6 gap-4 cursor-pointer",
              selectedType === "image" ? "border-primary" : null
            )}
          >
            <CardContent className="flex items-center space-x-2 p-0">
```

```
              <RadioGroupItem value="image" id="image-mode" hidden />
              <Label
                className={selectedType === "image" ? "text-primary" : ""}
                htmlFor="image-mode"
              >
                Image Mode
              </Label>
            </CardContent>
            <ImageIcon
              className={selectedType === "image" ? "text-primary" : ""}
              size={36}
            />
          </Card>

          <Card
            onClick={() => setSelectedType("video")}
            className={cn(
              "flex flex-col items-center justify-center p-4 gap-4 cursor-pointer",
              selectedType === "video" ? "border-primary" : null
            )}
          >
            <CardContent className="flex items-center space-x-2 p-0">
              <RadioGroupItem value="video" id="video-mode" hidden />
              <Label
                className={selectedType === "video" ? "text-primary" : ""}
                htmlFor="video-mode"
              >
                Video Mode
              </Label>
            </CardContent>
            <VideoIcon
              className={selectedType === "video" ? "text-primary" : ""}
              size={36}
            />
          </Card>
        </RadioGroup>
      </div>
    );
  }

  return null;
}
```

FILE: frontend/src/pages/Editor/UploadImage.jsx

```
import { useDropzone } from "react-dropzone";
import Lottie from "lottie-react";
import { Card, CardContent } from "../../components/Editor/Card";
import { cn } from "../../lib/utils";
import imageAnimation from "../../../src/animations/image-upload.json";
import { toast } from "sonner";
import { useEditorStore } from "../../stores/useEditorStore";

export default function UploadImage() {
  const { getState, setState } = useEditorStore;

  const { uploadingImageGenerating, activeLayer, uploadImage, uploadImageError, setTags,
updateLayer, setActiveLayer } = getState();

  const { getRootProps, getInputProps, isDragActive } = useDropzone({
    maxFiles: 1,
    accept: {
      "image/png": [".png"],
      "image/jpg": [".jpg"],
      "image/webp": [".webp"],
      "image/jpeg": [".jpeg"],
```

```
    },
    onDrop: async (acceptedFiles, fileRejections) => {
      if (acceptedFiles.length) {
        // Generate Object URL for preview
        const objectUrl = URL.createObjectURL(acceptedFiles[0]);

        // Update the layer with temporary uploading state
        updateLayer({
          id: activeLayer.id,
          url: objectUrl,
          width: 0,
          height: 0,
          name: "uploading",
          publicId: "",
          format: "",
          resourceType: "image",
        });

        // Set the active layer directly
        setActiveLayer(activeLayer.id);

        // Upload image using the Zustand store function
        await uploadImage(acceptedFiles[0], activeLayer.id);

        const latestActiveLayer = getState().activeLayer;

        if (!uploadImageError && !uploadingImageGenerating) {
          // Update the layer after successful upload
          updateLayer({
            id: latestActiveLayer.id,
            url: latestActiveLayer.url,
            width: latestActiveLayer.width,
            height: latestActiveLayer.height,
            name: latestActiveLayer.original_filename,
            publicId: latestActiveLayer.public_id,
            format: latestActiveLayer.format,
            resourceType: latestActiveLayer.resource_type,
          });

          // Set tags from upload response
          setTags(latestActiveLayer.tags);
          setActiveLayer(latestActiveLayer.id);
        } else {
          toast.error(uploadImageError || "Failed to upload the image");
        }
      }

      if (fileRejections.length) {
        toast.error(fileRejections[0].errors[0].message);
      }
    },
  });

  const latestActiveLayer = getState().activeLayer;

  if (!latestActiveLayer.url) {
    return (
      <Card
        {...getRootProps()}
        className={cn(
          "hover:cursor-pointer hover:bg-secondary hover:border-primary transition-all ease-in-
out",
          `${isDragActive ? "animate-pulse border-primary bg-secondary" : ""}`
        )}
      >
        <CardContent className="flex flex-col h-full items-center justify-center px-2 py-24 text-
```

```
xs">
          <input {...getInputProps()} />
          <div className="flex items-center flex-col justify-center gap-4">
            <Lottie className="h-48" animationData={imageAnimation} />
            <p className="text-muted-foreground text-2xl">
              {isDragActive ? "Drop your image here!" : "Start by uploading an image"}
            </p>
            <p className="text-muted-foreground">
              Supported Formats .jpeg .jpg .png .webp
            </p>
          </div>
        </CardContent>
      </Card>
    );
  }

  return null; // Optional: handle what to render when activeLayer.url exists
}
```

FILE: frontend/src/pages/Editor/UploadVideo.jsx

```
import { useDropzone } from "react-dropzone";
import Lottie from "lottie-react";
import { Card, CardContent } from "../../components/Editor/Card";
import { cn } from "../../lib/utils";
import videoAnimation from "../../../src/animations/video-upload.json";
import { toast } from "sonner";
import { useEditorStore } from "../../stores/useEditorStore";

export default function UploadVideo() {
  const { getState } = useEditorStore;
  const state = getState();
  console.log(state.activeLayer);
  const { getRootProps, getInputProps, isDragActive } = useDropzone({
    maxFiles: 1,
    accept: { "video/mp4": [".mp4", ".MP4"] },
    onDrop: async (acceptedFiles, fileRejections) => {
      const {
        updateLayer,
        setActiveLayer,
        uploadVideo,
        setTags,
      } = getState();

      const activeLayer = getState().activeLayer;

      if (acceptedFiles.length) {
        const objectUrl = URL.createObjectURL(acceptedFiles[0]);

        // Temporary preview update
        updateLayer({
          id: activeLayer.id,
          url: objectUrl,
          width: 0,
          height: 0,
          name: "uploading",
          publicId: "",
          format: "",
          poster: "",
          resourceType: "video",
        });

        setActiveLayer(activeLayer.id);

        // Upload video
        await uploadVideo(acceptedFiles[0], activeLayer.id);
```

```
        const latestState = getState();
        const latestActiveLayer = latestState.activeLayer;

        if (!latestState.uploadVideoError && !latestState.uploadingVideoGenerating) {
          const thumbnailUrl = latestActiveLayer.url.replace(/\.[^/.]+$/, ".jpg");

          updateLayer({
            id: latestActiveLayer.id,
            url: latestActiveLayer.url,
            width: latestActiveLayer.width,
            height: latestActiveLayer.height,
            name: latestActiveLayer.original_filename,
            publicId: latestActiveLayer.public_id,
            format: latestActiveLayer.format,
            poster: thumbnailUrl,
            resourceType: latestActiveLayer.resource_type,
          });

          setTags(latestActiveLayer.tags);
          setActiveLayer(latestActiveLayer.id);
        } else {
          toast.error(latestState.uploadVideoError || "Upload failed");
        }
      }

      if (fileRejections.length) {
        toast.error(fileRejections[0].errors[0].message);
      }
    },
  });

  const latestActiveLayer = getState().activeLayer;

  if (!latestActiveLayer.url) {
    return (
      <Card
        {...getRootProps()}
        className={cn(
          "hover:cursor-pointer hover:bg-secondary hover:border-primary transition-all ease-in-
out",
          `${isDragActive ? "animate-pulse border-primary bg-secondary" : ""}`
        )}
      >
        <CardContent className="flex flex-col h-full items-center justify-center px-2 py-24 text-
xs">
          <input {...getInputProps()} />
          <div className="flex items-center flex-col justify-center gap-4">
            <Lottie className="h-48" animationData={videoAnimation} />
            <p className="text-muted-foreground text-2xl">
              {isDragActive ? "Drop your video here!" : "Start by uploading a video"}
            </p>
            <p className="text-muted-foreground">Supported Format: .mp4</p>
          </div>
        </CardContent>
      </Card>
    );
  }

  return null; // Or render something else if video already uploaded
}
```

FILE: frontend/src/pages/Editor/VideoTools.jsx

```
import VideoTranscription from "./Transcribe"
import SmartCrop from "./SmartCrop"
```

```
import { useEditorStore } from "../../stores/useEditorStore"

export default function VideoTools() {
  const activeLayer = useEditorStore((state) => state.activeLayer)
  if (activeLayer.resourceType === "video")
    return (
      <>
        <VideoTranscription />
        <SmartCrop />
      </>
    )
}
```

FILE: frontend/src/pages/Editor/Youtube.jsx

```
export default function Youtube() {
  return (
    <svg
      className="w-10 h-10"
      height="800px"
      width="800px"
      version="1.1"
      id="Layer_1"
      viewBox="0 0 461.001 461.001"
      fill="#FF0000"
    >
      <g>
        <path
          d="M365.257,67.393H95.744C42.866,67.393,0,110.259,0,163.137v134.728
        c0,52.878,42.866,95.744,95.744,95.744h269.513c52.878,0,95.744-42.866,95.744-95.744V163.137
        C461.001,110.259,418.135,67.393,365.257,67.393z M300.506,237.056l-126.06,60.123c-
3.359,1.602-7.239-0.847-7.239-4.568V168.607
        c0-3.774,3.982-6.22,7.348-
4.514l126.06,63.881C304.363,229.873,304.298,235.248,300.506,237.056z"
        />
      </g>
    </svg>
  )
}
```

FILE: frontend/src/pages/EditProfilePage.jsx

```
import React, { useState, useEffect } from 'react';
import { useUserStore } from '../stores/useUserStore';

const EditProfilePage = () => {
  const { user, updateProfile, loading } = useUserStore();
  const [formData, setFormData] = useState({
    first_name: '',
    last_name: '',
    phone_number: '',
    language: '',
    email: '',
  });

  useEffect(() => {
    if (user) {
      setFormData({
        first_name: user.first_name,
        last_name: user.last_name,
        phone_number: user.phone_number,
        language: user.language,
        email: user.email,
      });
    }
```

```
  }, [user]);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prevData) => ({ ...prevData, [name]: value }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    updateProfile(formData);
  };

  if (loading) return <div className="text-center text-gray-500">Loading...</div>;

  return (
    <div className="w-full bg-transparent text-emerald-400 rounded-lg shadow-lg">
      <h2 className="text-3xl font-bold text-center mb-6">Edit Profile</h2>
      <form onSubmit={handleSubmit} className="space-y-5">
        {Object.keys(formData).map((key) => (
          <div key={key}>
            <label htmlFor={key} className="block text-sm font-medium text-gray-300">
              {key.replace('_', ' ').replace(/\b\w/g, (char) => char.toUpperCase())}
            </label>
            <input
              id={key}
              name={key}
              type={key === 'email' ? 'email' : 'text'}
              className="w-full p-4 bg-transparent text-emerald-400 border-2 border-white rounded-
lg focus:outline-none focus:ring-2 focus:ring-emerald-500"
              value={formData[key]}
              onChange={handleChange}
              required
            />
          </div>
        ))}
        <button
          type="submit"
          className="w-full bg-emerald-600 hover:bg-emerald-700 text-white font-semibold py-3
rounded-lg transition duration-200"
        >
          Save Changes
        </button>
      </form>
    </div>
  );
};

export default EditProfilePage;
```

FILE: frontend/src/pages/FeedbackForm/CreateFeedbackForm.jsx

```
import { useState } from "react";
import { useFeedbackFormStore } from "../../stores/useFeedbackFormStore.js";
import FeedbackFormQuestions from "../../components/FeedBackForm/FeedbackFormQuestions.jsx";
import { v4 as uuidv4 } from "uuid";

export default function CreateFeedbackForm() {
  const { createFeedbackForm, isLoadingForm, errorMessage } = useFeedbackFormStore();  // Using
Zustand for state management
  const [feedbackForm, setFeedbackForm] = useState({
    questions: [],
  });
  const [error, setError] = useState("");

  const onSubmit = async (ev) => {
    ev.preventDefault();
```

```
    const payload = { ...feedbackForm };

    try {
      await createFeedbackForm(payload);  // Use the Zustand store to create the feedback form
      setFeedbackForm({ questions: [] });  // Reset the form after successful creation
    } catch (err) {
      setError(errorMessage || "An error occurred while creating the feedback form.");
    }
  };

  const onQuestionsUpdate = (questions) => {
    setFeedbackForm({
      ...feedbackForm,
      questions,
    });
  };

  const addQuestion = () => {
    feedbackForm.questions.push({
      id: uuidv4(),
      type: "text",
      question: "",
      description: "",
      data: {},
    });
    setFeedbackForm({ ...feedbackForm });
  };

  return (
    <>
      {isLoadingForm && <div className="text-center text-lg text-emerald-400">Submitting...</div>}
      <form action="#" method="POST" onSubmit={onSubmit}>
        <div className="shadow sm:overflow-hidden sm:rounded-md border-2 border-white">
          <div className="space-y-6 px-4 py-5 sm:p-6">
            {error && (
              <div className="bg-none text-emerald-400 py-3 px-3 border-2 border-white">{error}
</div>
            )}
            {errorMessage && (
              <div className="bg-none text-emerald-400 py-3 px-3 border-2 border-white">
{errorMessage}</div>
            )}

            <button
              type="button"
              onClick={addQuestion}
              className="text-emerald-400 border-2 border-white py-2 px-4 rounded"
            >
              Add question
            </button>
            <FeedbackFormQuestions
              questions={feedbackForm.questions}
              onQuestionsUpdate={onQuestionsUpdate}
            />
          </div>
          <div className="bg-none px-4 py-3 text-right sm:px-6">
            <button
              type="submit"
              className="inline-flex justify-center py-2 px-4 border-2 border-white text-sm font-
medium rounded-md text-emerald-400 hover:bg-transparent focus:outline-none focus:ring-2 focus:ring-
emerald-400 focus:ring-offset-2"
            >
              Save
            </button>
          </div>
        </div>
```

```
      </form>
    </>
  );
}
```

FILE: frontend/src/pages/FeedbackForm/FeedbackFormPublicView.jsx

```
import { useState, useEffect } from "react";
import PublicQuestionView from "../../components/FeedBackForm/PublicQuestionView";
import { useFeedbackFormStore } from "../../stores/useFeedbackFormStore";

export default function FeedbackFormPublicView({ currentFeedbackForm, continueIndex }) {
  const { submitFeedbackForm, isLoadingForm, isError, errorMessage } = useFeedbackFormStore();
  const [answers, setAnswers] = useState({});
  const [feedbackFormFinished, setFeedbackFormFinished] = useState(false);
  console.log(currentFeedbackForm.id);
  function handleAnswerChange(questionId, value) {
    setAnswers((prev) => ({ ...prev, [questionId]: value }));
  }

  const handleSubmit = async (ev) => {
    ev.preventDefault();
    const success = await submitFeedbackForm(currentFeedbackForm.id, answers);
    if (success) {
      setFeedbackFormFinished(true);
      continueIndex((prev) => prev+1)
    }
  };

  if (isLoadingForm) return <div className="flex justify-center text-lg text-emerald-
400">Loading...</div>;
  if (isError) return <div className="text-center text-lg text-red-500">{errorMessage}</div>;

  return (
    <div className="container mx-auto p-4">
      <form onSubmit={handleSubmit} className="bg-transparent border-2 border-white text-emerald-
400 shadow-md rounded-lg p-6">
        {feedbackFormFinished && (
          <div className="py-4 px-4 text-emerald-400">
            Thank you for submitting the feedback form!
          </div>
        )}

        {!feedbackFormFinished && currentFeedbackForm && (
          <>
            <h2 className="text-lg font-semibold mb-4 text-emerald-400">Questions</h2>
            {currentFeedbackForm.questions.map((question, index) => (
              <PublicQuestionView
                key={question.id}
                question={question}
                index={index}
                answerChanged={(val) => handleAnswerChange(question.id, val)}
              />
            ))}
            <button
              type="submit"
              className="mt-4 inline-flex justify-center py-2 px-4 border-2 border-white text-
emerald-400 font-medium rounded-md hover:bg-transparent focus:outline-none focus:ring-2 focus:ring-
offset-2 focus:ring-emerald-500 transition duration-200"
            >
              Submit
            </button>
          </>
        )}
      </form>
    </div>
```

```
  );
}
```

FILE: frontend/src/pages/HomePage.jsx

```
import { useEffect } from "react";
import CategoryItem from "../components/CategoryItem";
import { useProductStore } from "../stores/useProductStore";
import FeaturedProducts from "../components/FeaturedProducts";

const categories = [
  {
    href: "/culpa",
    name: "Repudiandae",
    imageUrl: "/jeans.jpg",
    description: "Comfortable and stylish jeans for every occasion."
  },
  {
    href: "/t-shirts",
    name: "T-Shirts",
    imageUrl: "/tshirts.jpg",
    description: "Soft and breathable t-shirts for casual wear."
  },
  {
    href: "/shoes",
    name: "Shoes",
    imageUrl: "/shoes.jpg",
    description: "Stylish shoes to complement any outfit."
  },
  {
    href: "/glasses",
    name: "Glasses",
    imageUrl: "/glasses.png",
    description: "Fashionable eyewear for a perfect look."
  },
  {
    href: "/jackets",
    name: "Jackets",
    imageUrl: "/jackets.jpg",
    description: "Keep warm with our stylish jacket collection."
  },
  {
    href: "/suits",
    name: "Suits",
    imageUrl: "/suits.jpg",
    description: "Elegantly tailored suits for formal occasions."
  },
  {
    href: "/bags",
    name: "Bags",
    imageUrl: "/bags.jpg",
    description: "Chic and durable bags to carry your essentials."
  },
];

const HomePage = () => {
  const { fetchFeaturedProducts, products, isLoading } = useProductStore();

  useEffect(() => {
    fetchFeaturedProducts();
  }, [fetchFeaturedProducts]);

  return (
    <div className='relative min-h-screen text-white overflow-hidden'>
      <div className='relative z-10 max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-16'>
      {!isLoading && products.length > 0 && <FeaturedProducts featuredProducts={products} />}
```

```
        <h1 className='text-center text-5xl sm:text-6xl font-bold text-emerald-400 mb-4'>
          Explore Our Categories
        </h1>
        <p className='text-center text-xl text-gray-300 mb-12'>
          Discover the latest fashion trends and products
        </p>

        <div className='grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4'>
          {categories.map((category) => (
            <CategoryItem category={category} key={category.name} />
          ))}
        </div>
      </div>
    </div>
  );
};

export default HomePage;
```

FILE: frontend/src/pages/LiveStream/AboutCard.jsx

```
import { useState } from "react";
import { VerifiedMark } from "./VerifiedMark";
import { BioModal } from "./BioModal";

export const AboutCard = ({
  hostName,
  hostIdentity,
  viewerIdentity,
  headline,
  about,
  followedByCount,
}) => {
  const [isModalOpen, setIsModalOpen] = useState(false); // Manage modal visibility state

  const hostAsViewer = `Host-${hostIdentity}`;
  const isHost = viewerIdentity === hostAsViewer;

  const followedByLabel = followedByCount === 1 ? "follower" : "followers";

  const handleModalClose = () => {
    setIsModalOpen(false); // Close the modal when cancel is clicked
  };

  return (
    <div className="px-4">
      <div className="group rounded-xl p-6 lg:p-10 flex flex-col gap-y-3  bg-transparent">
        <div className="flex items-center justify-between">
          <div className="flex items-center gap-x-2 font-semibold text-lg lg:text-2xl text-emerald-
400">
            About {hostName}
            <VerifiedMark className="text-emerald-400" /> {/* Make icon emerald-400 */}
          </div>
          {/* Show the BioModal if it's the host and modal is open */}
          {isHost && (
            <button
              onClick={() => setIsModalOpen(true)} // Open the modal when the user clicks edit
              className="text-sm text-emerald-400 hover:underline border-2 border-white px-2 py-1
rounded-md"
            >
              Edit Profile
            </button>
          )}
        </div>
        <div className="text-sm text-emerald-400">
```

```
          <span className="font-semibold">{followedByCount}</span> {followedByLabel}
        </div>
        <p className="text-sm text-emerald-400">{about || "This user has not set a bio yet."}</p>
      </div>

      {/* Conditionally render BioModal when isModalOpen is true */}
      {isModalOpen && (
        <BioModal
          initialHeadline={headline}
          initialAbout={about}
          onClose={handleModalClose} // Pass the close function to BioModal
        />
      )}
    </div>
  );
};
```

FILE: frontend/src/pages/LiveStream/Actions.jsx

```
import { toast } from "sonner";
import { Heart } from "lucide-react";
import { useTransition } from "react";
import { cn } from "../../helpers";
import { useUserStore } from "../../stores/useUserStore";
import useLiveStreamStore from "../../stores/useLiveStreamStore";

export const Actions = ({
  hostIdentity,
  isFollowing,
  isHost,
}) => {
  const [isPending, startTransition] = useTransition();
  const { followUser, unfollowUser } = useLiveStreamStore(); // Fetch the store's actions
  const { user } = useUserStore(); // Get user data from the user store
  const userId = user.id;

  const handleFollow = async () => {
    startTransition(() => {
      try {
        const data = followUser(hostIdentity);
        // Update toast to show first_name and last_name
        toast.success(`You are now following ${data.following.first_name}
${data.following.last_name}`);
      } catch (error) {
        toast.error("Something went wrong");
      }
    });
  };

  const handleUnfollow = async () => {
    startTransition(() => {
      try {
        const data = unfollowUser(hostIdentity);
        // Update toast to show first_name and last_name
        toast.success(`You have unfollowed ${data.following.first_name}
${data.following.last_name}`);
      } catch (error) {
        toast.error("Something went wrong");
      }
    });
  };

  const toggleFollow = () => {
    if (!userId) {
      return alert("You need to sign in first"); // Replaced Next.js router with a simple alert
    }
```

```
    if (isHost) return;

    if (isFollowing) {
      handleUnfollow();
    } else {
      handleFollow();
    }
  };

  return (
    <button
      disabled={isPending || isHost}
      onClick={toggleFollow}
      className={`w-full max-w-full px-4 py-2 border-2 border-white text-emerald-400 bg-transparent
rounded-md cursor-pointer flex items-center justify-center gap-2 ${
        isPending || isHost ? "opacity-50 cursor-not-allowed" : ""
      }`}
    >
      <Heart
        className={cn("h-4 w-4", isFollowing ? "fill-emerald-400" : "fill-none")}
      />
      {isFollowing ? "Unfollow" : "Follow"}
    </button>
  );
};

export const ActionsSkeleton = () => {
  return (
    <div
      className="h-10 w-32 bg-gray-300 rounded-md"
      style={{ height: "40px", width: "100%", maxWidth: "100px" }}
    />
  );
};
```

FILE: frontend/src/pages/LiveStream/BioModal.jsx

```
import { useState, useTransition, useRef, useEffect } from "react";
import useLiveStreamStore from "../../stores/useLiveStreamStore";

export const BioModal = ({ initialHeadline, initialAbout, onClose }) => {
  const closeRef = useRef(null);
  const [isPending, startTransition] = useTransition();
  const [headline, setHeadline] = useState(initialHeadline || "");
  const [about, setAbout] = useState(initialAbout || "");

  // Access the Zustand store's functions and state
  const { updateUserHeadlineAndAbout, isProcessingUpdate, isErrorUpdate, errorMessageUpdate } =
useLiveStreamStore();

  // Handle form submission
  const onSubmit = (e) => {
    e.preventDefault();

    // If update is already processing, prevent submitting
    if (isProcessingUpdate) return;

    startTransition(() => {
      updateUserHeadlineAndAbout(headline, about);
    });
  };

  // Show error messages if there is an error
  useEffect(() => {
    if (isErrorUpdate) {
```

```
      // You can use any alert or UI component here to display the error
      alert(errorMessageUpdate || "Something went wrong while updating.");
    }
  }, [isErrorUpdate, errorMessageUpdate]);

  // Close the modal when Cancel is clicked
  const handleCancel = () => {
    if (onClose) {
      onClose(); // Call the onClose function passed from parent component
    }
  };

  return (
    <div className="fixed inset-0 bg-black bg-opacity-50 flex justify-center items-center z-50">
      <div className="bg-white p-6 rounded-lg w-full max-w-md">
        <h2 className="text-2xl font-semibold mb-4">Edit user profile</h2>
        <form onSubmit={onSubmit} className="space-y-4">
          <div>
            <label htmlFor="headline" className="block text-sm font-medium text-gray-
700">Headline</label>
            <input
              id="headline"
              type="text"
              value={headline}
              onChange={(e) => setHeadline(e.target.value)}
              className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm
focus:outline-none focus:ring-indigo-500 focus:border-indigo-500 sm:text-sm"
              disabled={isPending || isProcessingUpdate}
              placeholder="Enter your headline"
            />
          </div>

          <div>
            <label htmlFor="about" className="block text-sm font-medium text-gray-
700">About</label>
            <textarea
              id="about"
              value={about}
              onChange={(e) => setAbout(e.target.value)}
              className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm
focus:outline-none focus:ring-indigo-500 focus:border-indigo-500 sm:text-sm resize-none"
              rows="4"
              disabled={isPending || isProcessingUpdate}
              placeholder="Tell us something about you"
            />
          </div>

          <div className="flex justify-between mt-4">
            <button
              ref={closeRef}
              type="button"
              onClick={handleCancel} // Close modal on Cancel
              className="bg-gray-300 text-gray-700 hover:bg-gray-400 py-2 px-4 rounded-md"
            >
              Cancel
            </button>
            <button
              type="submit"
              disabled={isPending || isProcessingUpdate}
              className={`${
                isPending || isProcessingUpdate ? "bg-gray-500" : "bg-blue-600 hover:bg-blue-700"
              } text-white py-2 px-4 rounded-md`}
            >
              {isPending || isProcessingUpdate ? "Saving..." : "Save"}
            </button>
          </div>
```

```
        </form>
      </div>
    </div>
  );
};
```

FILE: frontend/src/pages/LiveStream/ChatCommunity.jsx

```
import { useMemo, useState, useEffect } from "react";
import { useParticipants } from "@livekit/components-react"; // Using useParticipants hook
import { User } from "lucide-react"; // Example icon import

const CommunityItem = ({ participantName, participantIdentity }) => {
  return (
    <div className="flex items-center p-2 border-b border-white text-emerald-400">
      <User className="h-5 w-5 text-emerald-400" /> {/* Apply emerald color to icon */}
      <p className="ml-2">{participantName}</p>
    </div>
  );
};

export const ChatCommunity = ({
  hostName,
  viewerName,
  isHidden,
}) => {
  const [value, setValue] = useState("");
  const [debouncedValue, setDebouncedValue] = useState("");

  // Debouncing logic: Update the debounced value after a delay of 500ms
  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, 500);

    return () => {
      clearTimeout(handler); // Cleanup the previous timeout on value change
    };
  }, [value]);

  const participants = useParticipants();

  const onChange = (newValue) => {
    setValue(newValue);
  };

  const filteredParticipants = useMemo(() => {
    const deduped = participants.reduce((acc, participant) => {
      const hostAsViewer = `Host-${participant.identity}`;
      if (!acc.some((p) => p.identity === hostAsViewer)) {
        acc.push(participant);
      }
      return acc;
    }, []);

    return deduped.filter((participant) => {
      return participant.name
        ?.toLowerCase()
        .includes(debouncedValue.toLowerCase());
    });
  }, [participants, debouncedValue]);

  if (isHidden) {
    return (
      <div className="flex flex-1 items-center justify-center bg-transparent border-2 border-white
p-4 text-emerald-400">
```

```
        <p className="text-sm text-muted-foreground">Community is disabled</p>
      </div>
    );
  }

  return (
    <div className="p-4 bg-transparent border-2 border-white text-emerald-400">
      <input
        type="text"
        value={value}
        onChange={(e) => onChange(e.target.value)}
        placeholder="Search community"
        className="border-2 border-white p-2 rounded-md w-full text-emerald-400 bg-transparent
focus:ring-emerald-400 focus:border-emerald-400"
      />
      <div className="gap-y-2 mt-4 overflow-y-auto max-h-60">
        {filteredParticipants.length === 0 && (
          <p className="text-center text-sm text-muted-foreground p-2">No results</p>
        )}
        {filteredParticipants.map((participant) => (
          <CommunityItem
            key={participant.identity}
            hostName={hostName}
            viewerName={viewerName}
            participantName={participant.name}
            participantIdentity={participant.identity}
          />
        ))}
      </div>
    </div>
  );
};
```

FILE: frontend/src/pages/LiveStream/CommunityItem.jsx

```
import { useTransition } from "react";
import { MinusCircle } from "lucide-react";
import useLiveStreamStore from "../../stores/useLiveStreamStore";
import { cn, stringToColor } from "../../helpers";

const CommunityItem = ({
  hostName,
  viewerName,
  participantIdentity,
  participantName,
}) => {
  const [isPending, startTransition] = useTransition();

  // Access the blockUser method and the block-related states from the store
  const {
    blockUser,
    isProcessingBlock,
    isErrorBlock,
    errorMessageBlock,
  } = useLiveStreamStore((state) => ({
    blockUser: state.blockUser,
    isProcessingBlock: state.isProcessingBlock,
    isErrorBlock: state.isErrorBlock,
    errorMessageBlock: state.errorMessageBlock,
  }));

  const color = stringToColor(participantName || "");
  const isSelf = participantName === viewerName;
  const isHost = viewerName === hostName;

  const handleBlock = () => {
```

```
    if (!participantName || isSelf || !isHost) return;

    startTransition(() => {
      blockUser(participantIdentity);
    });
  };

  return (
    <div
      className={cn(
        "group flex items-center justify-between w-full p-2 rounded-md text-sm border-2 border-
white bg-transparent hover:bg-white/5",
        isPending || isProcessingBlock ? "opacity-50 pointer-events-none" : ""
      )}
    >
      {/* Participant Name */}
      <p className="text-emerald-400" style={{ color: color }}>
        {participantName}
      </p>

      {/* Block Button */}
      {isHost && !isSelf && !isProcessingBlock && !isErrorBlock && (
        <div
          title="Block"
          onClick={handleBlock}
          className="h-auto w-auto p-1 opacity-0 group-hover:opacity-100 transition cursor-pointer"
        >
          <MinusCircle className="h-4 w-4 text-emerald-400" />
        </div>
      )}

      {/* Display an error message if blocking fails */}
      {isErrorBlock && errorMessageBlock && (
        <p className="text-xs text-red-500">{errorMessageBlock}</p>
      )}

      {/* Show loading state */}
      {isProcessingBlock && (
        <p className="text-xs text-muted-foreground">Blocking...</p>
      )}
    </div>
  );
};

export default CommunityItem;
```

FILE: frontend/src/pages/LiveStream/CopyButton.jsx

```
import { useState } from "react";
import { CheckCheck, Copy } from "lucide-react";
import { toast } from "react-hot-toast";

const CopyButton = ({ value }) => {
  const [isCopied, setIsCopied] = useState(false);

  const onCopy = () => {
    if (!value) return;

    setIsCopied(true);
    navigator.clipboard.writeText(value);
    setTimeout(() => {
      setIsCopied(false);
    }, 1000);
    toast.success("Copied");
  };
```

```
  const Icon = isCopied ? CheckCheck : Copy;

  return (
    <button
      onClick={onCopy}
      disabled={!value || isCopied}
      className="flex items-center justify-center p-2 bg-transparent border border-gray-300
rounded-md hover:bg-gray-100 disabled:opacity-50"
    >
      <Icon className="h-4 w-4" />
    </button>
  );
};

export default CopyButton;
```

FILE: frontend/src/pages/LiveStream/FullscreenControl.jsx

```
import { Maximize, Minimize } from "lucide-react";

const FullscreenControl = ({ isFullscreen, onToggle }) => {
  const Icon = isFullscreen ? Minimize : Maximize;

  const label = isFullscreen ? "Exit fullscreen" : "Enter fullscreen";

  return (
    <div className="flex items-center justify-center gap-4">
      <button
        onClick={onToggle}
        className="text-emerald-400 p-1.5 border-2 border-white rounded-lg bg-transparent hover:bg-
white/10"
        aria-label={label} // Adding aria-label for accessibility
      >
        <Icon className="h-5 w-5 text-emerald-400" />
      </button>
    </div>
  );
};

export default FullscreenControl;
```

FILE: frontend/src/pages/LiveStream/GiftSelector.jsx

```
import React, { useState, useEffect } from 'react';
import { useStripe, useElements, CardElement } from '@stripe/react-stripe-js';
import { useGiftStore } from '../../stores/useGiftStore';

const GiftSelector = () => {
  const {
    isPaymentProcessing,
    stripeClientSecret,
    createStripePaymentIntent,
  } = useGiftStore(); // Accessing your cart store

  const [price, setPrice] = useState(0);
  const [isPaid, setIsPaid] = useState(false);

  // Define a default set of images with associated prices
  const images = [
    { src: '/hat.png', alt: 'Hat', price: 2 },
    { src: '/rose.png', alt: 'Rose', price: 1 },
    { src: '/cowboy.png', alt: 'Cowboy', price: 3 },
    { src: '/donut.png', alt: 'Donut', price: 4 },
  ];
```

```
  const stripe = useStripe();
  const elements = useElements();

  // Handle image selection and price update
  const handleImageClick = (image) => {
    setPrice(image.price);
  };

  // Handle payment form submission
  const handlePaymentSubmit = async (e) => {
    e.preventDefault();

    if (!stripe || !elements) {
      console.error('Stripe has not loaded yet');
      return;
    }

    // Create payment intent on the backend and get client secret
    await createStripePaymentIntent('USD', price);

    // Confirm the payment with Stripe
    const cardElement = elements.getElement(CardElement);

    if (stripeClientSecret) {
      const { error, paymentIntent } = await stripe.confirmCardPayment(stripeClientSecret, {
        payment_method: {
          card: cardElement,
          billing_details: {
            address: {
              postal_code: '118560', // Include the postal code here
            },
          },
        },
      });

      if (error) {
        console.error(error.message);
        alert(error.message);
        return;
      }

      if (paymentIntent.status === 'succeeded') {
        setIsPaid(true);
      }
    }
  };

  useEffect(() => {
    if (isPaid) {
      console.log('Payment was successful');
      // You can redirect to a success page or show a success message
    }
  }, [isPaid]);

  return (
    <div >
      <div className="container mx-auto max-w-md  text-white rounded-lg shadow-md">
        <div className="grid grid-cols-1 gap-6">
          {/* Left Section: Select Gift */}
          <div className="p-4 bg-gray-900 rounded-md shadow-md">
            <div className="flex justify-center gap-4 mb-4">
              {images.map((image) => (
                <div
                  key={image.alt}
                  onClick={() => handleImageClick(image)}
                  className="cursor-pointer transform hover:scale-110 transition-all text-center"
```

```
                >
                  <img
                    src={image.src}
                    alt={image.alt}
                    className="w-12 h-12 rounded-md shadow-md object-cover mx-auto"
                  />
                  {/* Displaying price below the image */}
                  <p className="mt-2 text-sm text-emerald-400">${image.price}</p>
                </div>
              ))}
            </div>

            {price > 0 && (
              <div className="mt-2 text-center">
                <p className="text-sm">Selected Price: ${price}</p>
              </div>
            )}
          </div>

          {/* Right Section: Payment Form */}
          <div className="bg-gray-900 p-4 rounded-md shadow-md">

            <form onSubmit={handlePaymentSubmit} className="space-y-4">
              {/* Card Element for Stripe */}
              {price > 0 && (
                <div>
                  <CardElement
                    options={{
                      style: {
                        base: {
                          color: '#fff',
                          fontSize: '14px',
                          fontFamily: 'sans-serif',
                          '::placeholder': {
                            color: '#aaa',
                          },
                        },
                      },
                    }}
                  />
                </div>
              )}

              {/* Payment Button */}
              <button
                type="submit"
                disabled={isPaymentProcessing}
                className="w-full py-2 bg-emerald-600 text-white font-semibold rounded-lg hover:bg-
emerald-700 focus:outline-none focus:ring-4 focus:ring-emerald-300 disabled:bg-gray-600 transition-
all"
              >
                {isPaymentProcessing ? 'Processing...' : 'Confirm Payment'}
              </button>
            </form>
          </div>
        </div>
      </div>
    </div>
  );
};

export default GiftSelector;
```

FILE: frontend/src/pages/LiveStream/Header.jsx

```
import { UserIcon } from "lucide-react";
```

```
import { useParticipants, useRemoteParticipant } from "@livekit/components-react";

import { VerifiedMark } from "./VerifiedMark";
import { UserAvatar, UserAvatarSkeleton } from "./UserAvatar";
import { Actions, ActionsSkeleton } from "./Actions";

const Header = ({
  imageUrl,
  hostName,
  hostIdentity,
  viewerIdentity,
  isFollowing = false,
  name,
}) => {
  const participants = useParticipants();
  const participant = useRemoteParticipant(hostIdentity);

  const isLive = !!participant;
  const participantCount = participants.length - 1;

  const hostAsViewer = `Host-${hostIdentity}`;
  const isHost = viewerIdentity === hostAsViewer;

  return (
    <div className="flex flex-col lg:flex-row gap-y-4 lg:gap-y-0 items-start justify-between px-4
bg-transparent border-2 border-white">
      <div className="flex items-center gap-x-3">
        <UserAvatar
          imageUrl={imageUrl}
          username={hostName}
          size="lg"
          isLive={isLive}
          showBadge
        />
        <div className="space-y-1">
          <div className="flex items-center gap-x-2">
            <h2 className="text-lg font-semibold text-emerald-400">{hostName}</h2>
            <VerifiedMark />
          </div>
          <p className="text-sm font-semibold text-emerald-400">{name}</p>
          {isLive ? (
            <div className="font-semibold flex gap-x-1 items-center text-xs text-emerald-400">
              <UserIcon className="h-4 w-4 text-emerald-400" />
              <p>
                {participantCount}{" "}
                {participantCount === 1 ? "viewer" : "viewers"}
              </p>
            </div>
          ) : (
            <p className="font-semibold text-xs text-emerald-400">
              Offline
            </p>
          )}
        </div>
      </div>
      <Actions
        isFollowing={isFollowing}
        hostIdentity={hostIdentity}
        isHost={isHost}
      />
    </div>
  );
};

const HeaderSkeleton = () => {
  return (
```

```
    <div className="flex flex-col lg:flex-row gap-y-4 lg:gap-y-0 items-start justify-between px-4
bg-transparent border-2 border-white">
      <div className="flex items-center gap-x-2">
        <UserAvatarSkeleton size="lg" />
        <div className="space-y-2">
          <UserAvatarSkeleton size="md" />
          <UserAvatarSkeleton size="sm" />
        </div>
      </div>
      <ActionsSkeleton />
    </div>
  );
};

export { Header, HeaderSkeleton };
```

FILE: frontend/src/pages/LiveStream/Hint.jsx

```
import { useState } from "react";

const Hint = ({ label, children, asChild, side, align }) => {
  const [isVisible, setIsVisible] = useState(false);

  // Tailwind classes for tooltip positioning and styling
  const tooltipClasses = `
    absolute z-10 bg-transparent text-emerald-400 text-sm p-2 rounded-md
    border-2 border-white shadow-lg transition-opacity opacity-0
    ${isVisible ? 'opacity-100' : ''}
    ${side === 'top' ? 'bottom-full mb-2' : ''}
    ${side === 'bottom' ? 'top-full mt-2' : ''}
    ${align === 'left' ? 'left-0' : ''}
    ${align === 'right' ? 'right-0' : ''}
    ${align === 'center' ? 'left-1/2 transform -translate-x-1/2' : ''}
  `;

  return (
    <div
      className="relative inline-block"
      onMouseEnter={() => setIsVisible(true)}
      onMouseLeave={() => setIsVisible(false)}
    >
      {children}
      {isVisible && (
        <div className={tooltipClasses}>
          <p className="font-semibold">{label}</p>
        </div>
      )}
    </div>
  );
};

export default Hint;
```

FILE: frontend/src/pages/LiveStream/InfoCard.jsx

```
import { Pencil } from "lucide-react";
import InfoModal from "./InfoModal";

const InfoCard = ({ name, thumbnailUrl, hostIdentity, viewerIdentity }) => {
  const hostAsViewer = `Host-${hostIdentity}`;
  const isHost = viewerIdentity === hostAsViewer;

  if (!isHost) return null;

  return (
```

```
    <div className="px-4">
      <div className="rounded-xl bg-transparent border-2 border-white">
        <div className="flex items-center gap-x-2.5 p-4">
          <div className="rounded-md bg-transparent p-2 h-auto w-auto border-2 border-white">
            <Pencil className="h-5 w-5 text-emerald-400" />
          </div>
          <div>
            <h2 className="text-sm lg:text-lg font-semibold text-emerald-400 capitalize">
              Edit your stream info
            </h2>
            <p className="text-emerald-400 text-xs lg:text-sm">
              Maximize your visibility
            </p>
          </div>
          <InfoModal initialName={name} initialThumbnailUrl={thumbnailUrl} />
        </div>

        {/* Replaced Separator with a simple horizontal line */}
        <div className="border-t-2 border-white my-4"></div>

        <div className="p-4 lg:p-6 space-y-4">
          <div>
            <h3 className="text-sm text-emerald-400 mb-2">Name</h3>
            <p className="text-sm font-semibold text-emerald-400">{name}</p>
          </div>
          <div>
            <h3 className="text-sm text-emerald-400 mb-2">Thumbnail</h3>
            {thumbnailUrl && (
              <div className="relative aspect-video rounded-md overflow-hidden w-[200px] border-2
border-white">
                <img
                  src={thumbnailUrl}
                  alt={name}
                  className="object-cover w-full h-full"
                />
              </div>
            )}
          </div>
        </div>
      </div>
    </div>
  );
};

export default InfoCard;
```

FILE: frontend/src/pages/LiveStream/InfoModal.jsx

```
import { useState, useTransition, useRef } from "react";
import { Trash } from "lucide-react";
import Hint from "./Hint";
import useLiveStreamStore from "../../stores/useLiveStreamStore";

const InfoModal = ({ initialName, initialThumbnailUrl }) => {
  const closeRef = useRef(null);
  const [isPending, startTransition] = useTransition();

  // Access stream state from the store
  const { streamData, isProcessingStream, isErrorStream, errorMessageStream, updateStream } =
useLiveStreamStore();

  const [name, setName] = useState(initialName);
  const [thumbnailUrl, setThumbnailUrl] = useState(initialThumbnailUrl);
  const [newThumbnail, setNewThumbnail] = useState(null); // State for new thumbnail file

  const onRemove = () => {
```

```
    startTransition(() => {
      updateStream({ thumbnail: null });
      closeRef?.current?.click();
    });
  };

  const onSubmit = (e) => {
    e.preventDefault();

    startTransition(() => {
      updateStream({ title: name });
      closeRef?.current?.click();
    });
  };

  const onChange = (e) => {
    setName(e.target.value);
  };

  const onThumbnailChange = (e) => {
    const file = e.target.files[0];
    if (file) {
      setNewThumbnail(URL.createObjectURL(file)); // Preview the image locally before upload
    }
  };

  const onUploadThumbnail = () => {
    if (newThumbnail) {
      // Assuming you have an API endpoint to upload the thumbnail
      startTransition(() => {
        updateStream({ thumbnail: newThumbnail });
        closeRef?.current?.click();
      });
    }
  };

  // If there is an error, show it
  const renderError = () => {
    if (isErrorStream) {
      return (
        <div className="text-red-500 mt-4">
          <strong>Error:</strong> {errorMessageStream}
        </div>
      );
    }
    return null;
  };

  return (
    <div className="dialog bg-transparent">
      <button
        type="button"
        className="link-button text-emerald-400"
        onClick={() => {}}
      >
        Edit
      </button>
      <div className="dialog-content bg-transparent border-2 border-white">
        <div className="dialog-header">
          <h2 className="text-emerald-400">Edit stream info</h2>
        </div>
        <form onSubmit={onSubmit} className="space-y-14">
          <div className="space-y-2">
            <label htmlFor="name" className="text-emerald-400">Name</label>
            <input
              id="name"
```

```
              disabled={isProcessingStream || isPending} // Disable input if processing or
transition is pending
              placeholder="Stream name"
              onChange={onChange}
              value={name}
              className="input text-emerald-400 border-2 border-white"
            />
          </div>

          {/* Display current stream name from streamData */}
          {streamData && streamData.title && !name && (
            <div className="text-sm text-gray-500">
              Current Name: {streamData.title}
            </div>
          )}

          <div className="space-y-2">
            <label className="text-emerald-400">Thumbnail</label>
            <div className="relative aspect-video rounded-xl overflow-hidden border-2 border-
white">
              <div className="absolute top-2 right-2 z-[10]">
                <Hint label="Remove thumbnail" asChild side="left">
                  <button
                    type="button"
                    disabled={isProcessingStream || isPending} // Disable button if processing or
transition is pending
                    onClick={onRemove}
                    className="button-remove-thumbnail text-emerald-400"
                  >
                    <Trash className="h-4 w-4 text-emerald-400" />
                  </button>
                </Hint>
              </div>
              <img
                alt="Thumbnail"
                src={thumbnailUrl}
                className="object-cover w-full h-full"
              />
            </div>
          </div>

          {/* File upload input for a new thumbnail */}
          <div className="space-y-2">
            <label htmlFor="thumbnail" className="text-emerald-400">Upload New Thumbnail</label>
            <input
              type="file"
              accept="image/*"
              disabled={isProcessingStream || isPending} // Disable input if processing or
transition is pending
              onChange={onThumbnailChange}
              id="thumbnail"
              className="input-file text-emerald-400 border-2 border-white"
            />
            {newThumbnail && (
              <div className="relative aspect-video rounded-xl overflow-hidden border-2 border-
white mt-2">
                <img
                  alt="New Thumbnail Preview"
                  src={newThumbnail}
                  className="object-cover w-full h-full"
                />
              </div>
            )}
          </div>

          <div className="flex justify-between">
```

```
            <button ref={closeRef} type="button" className="ghost-button text-emerald-400">
              Cancel
            </button>
            <button
              disabled={isProcessingStream || isPending || !newThumbnail} // Disable if processing,
pending, or no new thumbnail
              type="button"
              onClick={onUploadThumbnail}
              className="primary-button text-emerald-400 border-2 border-white"
            >
              {isPending ? "Uploading..." : "Upload Thumbnail"} {/* Show uploading state */}
            </button>
            <button disabled={isProcessingStream || isPending} type="submit" className="primary-
button text-emerald-400 border-2 border-white">
              {isPending ? "Saving..." : "Save"} {/* Show saving state */}
            </button>
          </div>
        </form>

        {/* Display error message if exists */}
        {renderError()}
      </div>
    </div>
  );
};

export default InfoModal;
```

FILE: frontend/src/pages/LiveStream/KeyCard.jsx

```
import { useState } from "react";
import CopyButton from "./CopyButton";
const KeyCard = ({ value }) => {
  const [show, setShow] = useState(false);

  return (
    <div className="rounded-xl bg-muted p-6">
      <div className="flex items-start gap-x-10">
        <p className="font-semibold shrink-0">Stream Key</p>
        <div className="space-y-2 w-full">
          <div className="w-full flex items-center gap-x-2">
            <input
              value={value || ""}
              type={show ? "text" : "password"}
              disabled
              placeholder="Stream key"
              className="w-full p-2 bg-gray-100 text-gray-700 rounded-md border border-gray-300"
            />
            <CopyButton value={value || ""} />
          </div>
          <button
            onClick={() => setShow(!show)}
            className="text-sm text-blue-500 hover:text-blue-700 mt-2"
          >
            {show ? "Hide" : "Show"}
          </button>
        </div>
      </div>
    </div>
  );
};

export default KeyCard;
```

FILE: frontend/src/pages/LiveStream/LiveBadge.jsx

```
import { cn } from "../../helpers";

const LiveBadge = ({ className }) => {
  return (
    <div
      className={cn(
        "bg-transparent text-emerald-400 text-center p-0.5 px-1.5 rounded-md uppercase text-[10px]
border-2 border-white font-semibold tracking-wide",
        className
      )}
    >
      Live
    </div>
  );
};

export default LiveBadge;
```

FILE: frontend/src/pages/LiveStream/LiveStreamWrapper.jsx

```
import React, { useState } from "react";
import Navbar from "../../components/LiveStream/Navbar";
import Sidebar from "../../components/LiveStream/Sidebar";
import { useAppSelector } from "../State/Redux";
import { StreamPlayer } from "./StreamPlayer";
import StoreProvider from "../State/Redux";
import StreamCreationForm from "./StreamCreationForm";

const DashboardLayout = () => {
  const [selectedStream, setSelectedStream] = useState(null);
  const [isCreateStreamVisible, setIsCreateStreamVisible] = useState(false); // State to control
the visibility of the Stream Creation form

  const isSidebarCollapsed = useAppSelector(
    (state) => state.global.isSidebarCollapsed
  );

  const handleStreamSelect = (stream) => {
    setSelectedStream(stream);
  };

  const handleCreateStreamClick = () => {
    setIsCreateStreamVisible(true); // Show the stream creation form when the button is clicked
  };

  const handleCloseCreateStreamClick = () => {
    setIsCreateStreamVisible(false); // Close the stream creation form
  };

  return (
    <div className="flex w-full h-screen text-emerald-400 bg-transparent border-2 border-white
relative">
      {/* Sidebar */}
      <Sidebar onStreamSelect={handleStreamSelect} onCreateStreamClick={handleCreateStreamClick} />
      <main
        className={`flex flex-col w-full h-full py-7 px-9 ${
          isSidebarCollapsed ? "md:pl-24" : "md:pl-72"
        } text-emerald-400 bg-transparent border-2 border-white`}
      >
        <Navbar />

        {/* Separate condition for StreamCreationForm */}
        {isCreateStreamVisible && (
          <StreamCreationForm />
        )}
```

```
        {/* Separate condition for StreamPlayer */}
        {selectedStream && !isCreateStreamVisible && (
          <StreamPlayer
            user={selectedStream.user}
            stream={selectedStream}
            isFollowing={true}
            onCloseCreateStream={handleCloseCreateStreamClick}
          />
        )}

        {/* Fallback when no stream is selected and form is not visible */}
        {!selectedStream && !isCreateStreamVisible && (
          <div className="text-emerald-400 text-center">Select a stream to watch</div>
        )}
      </main>
    </div>
  );
};

const LiveStreamWrapper = ({ children }) => {
  return (
    <StoreProvider>
      <DashboardLayout>{children}</DashboardLayout>
    </StoreProvider>
  );
};

export default LiveStreamWrapper;
```

FILE: frontend/src/pages/LiveStream/LiveVideo.jsx

```
import { useRef, useState, useEffect } from "react";
import { Track } from "livekit-client";
import { useTracks } from "@livekit/components-react";

import { VolumeControl } from "./VolumeControl";
import FullscreenControl from "./FullscreenControl";

const LiveVideo = ({ participant }) => {
  const videoRef = useRef(null);
  const wrapperRef = useRef(null);

  const [isFullscreen, setIsFullscreen] = useState(false);
  const [volume, setVolume] = useState(0);

  const onVolumeChange = (value) => {
    setVolume(+value);
    if (videoRef?.current) {
      videoRef.current.muted = value === 0;
      videoRef.current.volume = +value * 0.01;
    }
  };

  const toggleMute = () => {
    const isMuted = volume === 0;

    setVolume(isMuted ? 50 : 0);

    if (videoRef?.current) {
      videoRef.current.muted = !isMuted;
      videoRef.current.volume = isMuted ? 0.5 : 0;
    }
  };

  useEffect(() => {
    onVolumeChange(0);
```

```
  }, []);

  const toggleFullscreen = () => {
    if (isFullscreen) {
      document.exitFullscreen();
    } else if (wrapperRef?.current) {
      wrapperRef.current.requestFullscreen();
    }
  };

  const handleFullscreenChange = () => {
    const isCurrentlyFullscreen = document.fullscreenElement !== null;
    setIsFullscreen(isCurrentlyFullscreen);
  };

  useEffect(() => {
    // Adding the fullscreen change event listener
    document.addEventListener("fullscreenchange", handleFullscreenChange);

    // Cleanup the event listener when the component unmounts
    return () => {
      document.removeEventListener("fullscreenchange", handleFullscreenChange);
    };
  }, []);

  useTracks([Track.Source.Camera, Track.Source.Microphone])
    .filter((track) => track.participant.identity === participant.identity)
    .forEach((track) => {
      if (videoRef.current) {
        track.publication.track?.attach(videoRef.current);
      }
    });

  return (
    <div
      ref={wrapperRef}
      className="relative h-full flex bg-transparent border-2 border-white"
    >
      <video ref={videoRef} width="100%" />
      <div className="absolute top-0 h-full w-full opacity-0 hover:opacity-100 hover:transition-
all">
        <div className="absolute bottom-0 flex h-14 w-full items-center justify-between bg-
gradient-to-r from-transparent px-4">
          <VolumeControl
            onChange={onVolumeChange}
            value={volume}
            onToggle={toggleMute}
            className="text-emerald-400 border-2 border-white"
          />
          <FullscreenControl
            isFullscreen={isFullscreen}
            onToggle={toggleFullscreen}
            className="text-emerald-400 border-2 border-white"
          />
        </div>
      </div>
    </div>
  );
};

export default LiveVideo;
```

FILE: frontend/src/pages/LiveStream/LoadingVideo.jsx

```
import { Loader } from "lucide-react";
```

```
const LoadingVideo = ({ label }) => {
  return (
    <div className="h-full flex flex-col space-y-4 justify-center items-center bg-transparent
border-2 border-white">
      <Loader className="h-10 w-10 text-emerald-400 animate-spin" />
      <p className="text-emerald-400 capitalize">{label}</p>
    </div>
  );
};

export default LoadingVideo;
```

FILE: frontend/src/pages/LiveStream/OfflineVideo.jsx

```
import { WifiOff } from "lucide-react";

const OfflineVideo = ({ username }) => {
  return (
    <div className="h-full flex flex-col space-y-4 justify-center items-center bg-transparent
border-2 border-white">
      <WifiOff className="h-10 w-10 text-emerald-400" />
      <p className="text-emerald-400">{username} is offline</p>
    </div>
  );
};

export default OfflineVideo;
```

FILE: frontend/src/pages/LiveStream/StreamCreationForm.jsx

```
import React, { useState } from 'react';
import useLiveStreamStore from '../../stores/useLiveStreamStore';

const StreamCreationForm = () => {
  const [title, setTitle] = useState('');
  const [thumbnail, setThumbnail] = useState(null); // Thumbnail is now a file object
  const [isChatEnabled, setIsChatEnabled] = useState(true);
  const [isChatFollowersOnly, setIsChatFollowersOnly] = useState(false);
  const [isChatDelayed, setIsChatDelayed] = useState(false);

  // Access the `createStream` function and state from Zustand store
  const { createStream, isProcessingStream, errorMessageStream } = useLiveStreamStore();

  const handleThumbnailChange = (e) => {
    const file = e.target.files[0];
    if (file) {
      setThumbnail(file);
    }
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    try {
      // Disable submit while processing
      const formData = new FormData();
      formData.append('title', title);
      formData.append('thumbnail', thumbnail); // Add the file to FormData
      formData.append('isChatEnabled', isChatEnabled);
      formData.append('isChatFollowersOnly', isChatFollowersOnly);
      formData.append('isChatDelayed', isChatDelayed);

      // Call the createStream function with the FormData
      await createStream(formData);
    } catch (err) {
```

```
      console.error("Error during stream creation:", err);
    }
  };

  return (
    <div className="w-full mx-auto mt-10 p-6 bg-transparent shadow-lg rounded-lg border-2 border-
white">
      <h2 className="text-3xl font-semibold text-emerald-400 mb-6 text-center">Create Your Live
Stream</h2>

      <form onSubmit={handleSubmit} className="space-y-6">
        {/* Stream Title */}
        <div className="mb-6">
          <label htmlFor="title" className="block text-sm font-medium text-emerald-400 mb-2">
            Stream Title
          </label>
          <input
            id="title"
            type="text"
            value={title}
            onChange={(e) => setTitle(e.target.value)}
            className="w-full p-4 border-2 border-white rounded-md focus:ring-2 focus:ring-emerald-
400 transition duration-300 bg-transparent text-emerald-400"
            required
          />
        </div>

        {/* Thumbnail Upload */}
        <div className="mb-6">
          <label htmlFor="thumbnail" className="block text-sm font-medium text-emerald-400 mb-2">
            Thumbnail (Upload Image)
          </label>
          <input
            id="thumbnail"
            type="file"
            accept="image/*" // Ensures only image files are accepted
            onChange={handleThumbnailChange}
            className="w-full p-4 border-2 border-white rounded-md focus:ring-2 focus:ring-emerald-
400 transition duration-300 bg-transparent text-emerald-400"
            required
          />
        </div>

        {/* Chat Settings */}
        <div className="space-y-4">
          <div className="flex items-center">
            <input
              type="checkbox"
              id="isChatEnabled"
              checked={isChatEnabled}
              onChange={() => setIsChatEnabled(!isChatEnabled)}
              className="h-5 w-5 text-emerald-400 border-white rounded-md"
            />
            <label htmlFor="isChatEnabled" className="text-sm text-emerald-400 ml-3">
              Enable Chat
            </label>
          </div>

          <div className="flex items-center">
            <input
              type="checkbox"
              id="isChatFollowersOnly"
              checked={isChatFollowersOnly}
              onChange={() => setIsChatFollowersOnly(!isChatFollowersOnly)}
              className="h-5 w-5 text-emerald-400 border-white rounded-md"
            />
```

```
            <label htmlFor="isChatFollowersOnly" className="text-sm text-emerald-400 ml-3">
              Followers Only Chat
            </label>
          </div>

          <div className="flex items-center">
            <input
              type="checkbox"
              id="isChatDelayed"
              checked={isChatDelayed}
              onChange={() => setIsChatDelayed(!isChatDelayed)}
              className="h-5 w-5 text-emerald-400 border-white rounded-md"
            />
            <label htmlFor="isChatDelayed" className="text-sm text-emerald-400 ml-3">
              Delayed Chat
            </label>
          </div>
        </div>

        {/* Submit Button */}
        <div className="mt-6">
          <button
            type="submit"
            className="w-full py-3 bg-emerald-400 text-white text-lg font-semibold rounded-md
hover:bg-emerald-500 transition duration-300"
            disabled={isProcessingStream} // Disable the button while processing
          >
            {isProcessingStream ? 'Creating Stream...' : 'Create Stream'}
          </button>
        </div>
      </form>

      {/* Error Message */}
      {errorMessageStream && (
        <div className="mt-6 p-4 bg-red-100 text-red-700 rounded-md shadow-md">
          <strong>Error:</strong> {errorMessageStream}
        </div>
      )}
    </div>
  );
};

export default StreamCreationForm;
```

FILE: frontend/src/pages/LiveStream/StreamMessages.jsx

```
import React, { useState, useEffect } from "react";
import { FaceSmileIcon, PaperAirplaneIcon, ClockIcon } from "@heroicons/react/24/solid";
import EmojiPicker from "emoji-picker-react"; // Import the emoji picker
import { Popover } from "@headlessui/react";
import useStreamMessageStore from "../../stores/useStreamMessageStore";
import GiftSelector from "./GiftSelector";

// Accept creatorId, viewerId, and streamId as props
const StreamMessages = ({ creatorId, viewerId, streamId }) => {
  const [message, setMessage] = useState("");

  // Access store state and actions from Zustand
  const {
    listenForNewMessage,
    messages,
    isProcessingMessage,
    isErrorMessage,
    errorMessageMessage,
    isSuccessMessage,
    sendMessage,
```

```
    getMessagesByCreator,
    reset,
  } = useStreamMessageStore();


  useEffect(() => {
    listenForNewMessage(streamId);
  }, [streamId]);

  useEffect(() => {
    if (creatorId) {
      // Call the getMessagesByCreator function to fetch messages
      getMessagesByCreator(creatorId);
    }
  }, [creatorId, getMessagesByCreator]);

  // Handler for sending the message
  const handleSendMessage = async (e) => {
    e.preventDefault();

    // Ensure the message is provided
    if (!message) {
      return;
    }

    // Send the message with creatorId, viewerId, and streamId as URL parameters
    await sendMessage(message, creatorId, viewerId, streamId);
  };

  // Effect to reset store on component unmount
  useEffect(() => {
    return () => {
      reset();
    };
  }, [reset]);

  // Handler to add emoji to the message input
  const onEmojiClick = (emojiData) => {
    // Append the emoji to the current message
    setMessage((prevMessage) => prevMessage + emojiData.emoji);
  };

  return (
    <div className="max-w-4xl mx-auto p-6 rounded-lg shadow-lg border-2 border-white flex flex-col"
style={{ height: 690 }}>
      {/* Displaying error if any */}
      {isErrorMessage && console.log(errorMessageMessage)}


      <div className="flex-1 overflow-auto space-y-4">
        {messages.length > 0 ? (
          <ul className="space-y-2">
            {messages.map((msg) => (
              <li key={msg.id} className="bg-transparent text-emerald-400 p-4 border-2 border-white
rounded-md shadow-sm">
                <strong>{msg.viewer.last_name + " " + msg.viewer.first_name}</strong>:
{msg.message}
              </li>
            ))}
          </ul>
        ) : (
          <p className="text-emerald-400">No messages found.</p>
        )}
      </div>

      {/* Display success message */}
```

```
      {isSuccessMessage && console.log(isSuccessMessage)}


      {/* Send message form */}
      <form onSubmit={handleSendMessage} className="space-y-4 mt-6">
        <div>
          <textarea
            id="message"
            value={message}
            onChange={(e) => setMessage(e.target.value)}
            rows="3"
            required
            className="w-full p-2 border-2 border-white text-emerald-400 bg-transparent rounded-md
focus:outline-none focus:ring-2 focus:ring-emerald-400"
          />
        </div>

        <div className="flex items-center space-x-2">
          {/* Send Button */}
          {/* Emoji Picker */}
          <Popover className="relative">
                <Popover.Button className="p-1 text-emerald-400 hover:text-emerald-500">
                <FaceSmileIcon className="w-6 h-6" />
                </Popover.Button>
                <Popover.Panel className="absolute z-10 right-0 bottom-full">
                <EmojiPicker
                    theme="dark"
                    onEmojiClick={onEmojiClick} // Add emoji to message when clicked
                />
                </Popover.Panel>
            </Popover>

            <button
            type="submit"
            disabled={isProcessingMessage}
            className={`py-2 px-4 rounded-md font-semibold text-white ${isProcessingMessage ? "bg-
gray-400 cursor-not-allowed" : "bg-emerald-400 hover:bg-emerald-500 focus:ring-2 focus:ring-
emerald-400"}`}
            >
            {/* Conditional rendering based on isProcessingMessage */}
            {isProcessingMessage ? (
                <ClockIcon className="w-5 h-5 animate-spin" />
            ) : (
                <PaperAirplaneIcon className="w-5 h-5" />
            )}
            </button>

        </div>
      </form>
      <GiftSelector/>
    </div>
  );
};

export default StreamMessages;
```

FILE: frontend/src/pages/LiveStream/StreamPlayer.jsx

```
import { LiveKitRoom } from "@livekit/components-react";
import { useViewerTokenStore } from "../../stores/useViewerTokenStore";
import InfoCard from "./InfoCard";
import { AboutCard } from "./AboutCard";
import { Video, VideoSkeleton } from "./Video";
import { Header, HeaderSkeleton } from "./Header";
import { useEffect, useState } from "react";
import { toast } from "react-hot-toast";
```

```
import useLiveStreamStore from "../../stores/useLiveStreamStore";
import KeyCard from "./KeyCard";
import UrlCard from "./UrlCard";
import StreamMessages from "./StreamMessages";
import { useUserStore } from "../../stores/useUserStore";

const StreamPlayer = ({ user, stream, isFollowing, onCloseCreateStream }) => {
  onCloseCreateStream();
  const { user: authenticatedUser } = useUserStore();
  const [loading, setLoading] = useState(true);
  const { token, name, identity, fetchViewerToken } = useViewerTokenStore((state) => ({
    token: state.token,
    name: state.name,
    identity: state.identity,
    fetchViewerToken: state.fetchViewerToken,
  }));

  const { isStoppingStream, stopStream } = useLiveStreamStore((state) => ({
    isStoppingStream: state.isStoppingStream,
    stopStream: state.stopStream,
  }));

  useEffect(() => {
    if (user?.id) {
      fetchViewerToken(user.id);
      setLoading(true);
    }
  }, [user?.id, fetchViewerToken]);

  useEffect(() => {
    if (token) {
      setLoading(false);
    }
  }, [token]);

  if (loading || !user || !stream || !token || !name || !identity) {
    return <StreamPlayerSkeleton />;
  }

  const fullName = `${user.first_name || ""} ${user.last_name || ""}`.trim();

  const handleStopStream = () => {
    toast.success("Stream is being stopped...");
    stopStream(stream.id);
  };

  return (
    <>
      <LiveKitRoom
        token={token}
        serverUrl={import.meta.env.VITE_PUBLIC_LIVEKIT_WS_URL}
        className="h-full flex flex-col lg:flex-row gap-6 p-4"
      >
        {/* Left section: Stream and Stream Details */}
        <div className="flex-1 space-y-4 lg:overflow-y-auto hidden-scrollbar pb-10">
          <Video hostName={fullName} hostIdentity={user.id} />
          <KeyCard value={stream.streamKey} />
          <UrlCard value={stream.serverUrl} />
          <button
            onClick={handleStopStream}
            disabled={isStoppingStream}
            className="bg-red-500 text-white px-4 py-2 rounded-lg mt-4"
          >
            {isStoppingStream ? "Stopping..." : "Stop Stream"}
          </button>
          <Header
```

```
            hostName={fullName}
            hostIdentity={user.id}
            viewerIdentity={identity}
            imageUrl={user.profile_picture}
            isFollowing={isFollowing}
            name={stream.title || "Untitled Stream"}
          />
          <InfoCard
            hostIdentity={user.id}
            viewerIdentity={identity}
            name={stream.title || "Untitled Stream"}
            thumbnailUrl={stream.thumbnail || ""}
          />
          <AboutCard
            hostName={fullName}
            hostIdentity={user.id}
            viewerIdentity={identity}
            headline={user.headline}
            about={user.about}
            followedByCount={user._count?.followedBy || 0}
          />
        </div>

        {/* Right section: StreamMessages */}
        <div className="w-full lg:w-1/3 rounded-lg shadow-md h-screen">
          <StreamMessages
            creatorId={user.id}
            viewerId={authenticatedUser.id}
            streamId={stream.id}
          />
        </div>
      </LiveKitRoom>
    </>
  );
};

const StreamPlayerSkeleton = () => {
  return (
    <div className="h-full flex flex-col lg:flex-row gap-6 p-4">
      {/* Left section: Skeleton for Stream and Stream Details */}
      <div className="flex-1 space-y-4 lg:overflow-y-auto hidden-scrollbar pb-10">
        <VideoSkeleton />
        <HeaderSkeleton />
      </div>

      {/* Right section: Skeleton for StreamMessages */}
      <div className="w-full lg:w-1/3 bg-gray-200 rounded-lg shadow-md h-screen">
        <div className="h-full w-full bg-gray-300 rounded-md" />
      </div>
    </div>
  );
};

export { StreamPlayer, StreamPlayerSkeleton };
```

FILE: frontend/src/pages/LiveStream/Thumbnail.jsx

```
import { useState } from "react";
import LiveBadge from "./LiveBadge";
import { UserAvatar } from "./UserAvatar";

const Thumbnail = ({ src, fallback, isLive, username }) => {
  const [isImageLoaded, setIsImageLoaded] = useState(false);

  let content;
```

```
  const handleImageLoad = () => {
    setIsImageLoaded(true);
  };

  if (!src) {
    content = (
      <div className="bg-transparent flex flex-col items-center justify-center gap-y-4 h-full w-
full transition-transform group-hover:translate-x-2 group-hover:-translate-y-2 rounded-md border-2
border-white">
        <UserAvatar
          size="lg"
          showBadge
          username={username}
          imageUrl={fallback}
          isLive={isLive}
        />
      </div>
    );
  } else {
    content = (
      <img
        src={src}
        alt="Thumbnail"
        onLoad={handleImageLoad}
        className={`object-cover transition-transform group-hover:translate-x-2 group-hover:-
translate-y-2 rounded-md ${isImageLoaded ? "opacity-100" : "opacity-0"}`}
      />
    );
  }

  return (
    <div className="group aspect-video relative rounded-md cursor-pointer border-2 border-white">
      <div className="rounded-md absolute inset-0 bg-transparent opacity-0 group-hover:opacity-100
transition-opacity flex items-center justify-center" />
      {content}
      {isLive && src && (
        <div className="absolute top-2 left-2 group-hover:translate-x-2 group-hover:-translate-y-2
transition-transform">
          <LiveBadge className="text-emerald-400 border-2 border-white" />
        </div>
      )}
    </div>
  );
};

export { Thumbnail };
```

FILE: frontend/src/pages/LiveStream/UrlCard.jsx

```
import CopyButton from "./CopyButton";
const UrlCard = ({ value }) => {
  return (
    <div className="rounded-xl bg-muted p-6">
      <div className="flex items-center gap-x-10">
        <p className="font-semibold shrink-0">Server URL</p>
        <div className="space-y-2 w-full">
          <div className="w-full flex items-center gap-x-2">
            <input
              value={value || ""}
              disabled
              placeholder="Server URL"
              className="w-full p-2 bg-gray-100 text-gray-700 rounded-md border border-gray-300"
            />
            <CopyButton value={value || ""} />
          </div>
        </div>
```

```
      </div>
    </div>
  );
};

export default UrlCard;
```

FILE: frontend/src/pages/LiveStream/UserAvatar.jsx

```
import { cva } from "class-variance-authority";
import { cn } from "../../helpers";
import LiveBadge from "./LiveBadge";

const avatarSizes = cva("", {
  variants: {
    size: {
      default: "h-8 w-8",
      lg: "h-14 w-14",
    },
  },
  defaultVariants: {
    size: "default",
  },
});

const UserAvatar = ({
  username,
  imageUrl,
  isLive,
  showBadge,
  size,
}) => {
  const canShowBadge = showBadge && isLive;

  return (
    <div className="relative">
      <div
        className={cn(
          isLive && "ring-2 ring-emerald-400 border-2 border-white", // Border color is emerald and
white
          avatarSizes({ size }),
          "rounded-full overflow-hidden bg-transparent" // Set background to transparent
        )}
      >
        {imageUrl ? (
          <img
            src={imageUrl}
            alt={`${username}'s avatar`}
            className="object-cover w-full h-full"
          />
        ) : (
          <span className="flex items-center justify-center w-full h-full text-emerald-400 bg-
transparent">
            {username[0]}
            {username[username.length - 1]}
          </span>
        )}
      </div>
      {canShowBadge && (
        <div className="absolute -bottom-3 left-1/2 transform -translate-x-1/2 text-emerald-400">
{/* Icon color set to emerald-400 */}
          <LiveBadge />
        </div>
      )}
    </div>
  );
```

```
};

const UserAvatarSkeleton = ({ size }) => {
  return (
    <div
      className={cn(
        "bg-transparent animate-pulse border-2 border-white rounded-full", // Background
transparent, white border
        avatarSizes({ size })
      )}
    />
  );
};

export { UserAvatar, UserAvatarSkeleton };
```

FILE: frontend/src/pages/LiveStream/VerifiedMark.jsx

```
import { Check } from "lucide-react";

export const VerifiedMark = () => {
  return (
    <div className="p-0.5 flex items-center justify-center h-4 w-4 rounded-full border-2 border-
white bg-transparent">
      <Check className="h-[10px] w-[10px] text-emerald-400 stroke-[4px]" />
    </div>
  );
};
```

FILE: frontend/src/pages/LiveStream/Video.jsx

```
import { ConnectionState, Track } from "livekit-client";
import {
  useConnectionState,
  useRemoteParticipant,
  useTracks,
} from "@livekit/components-react";

import OfflineVideo from "./OfflineVideo";
import LoadingVideo from "./LoadingVideo";
import LiveVideo from "./LiveVideo";

export const Video = ({ hostName, hostIdentity }) => {
  const connectionState = useConnectionState();
  const participant = useRemoteParticipant(hostIdentity);
  const tracks = useTracks([
    Track.Source.Camera,
    Track.Source.Microphone,
  ]).filter((track) => track.participant.identity === hostIdentity);

  let content;

  if (!participant && connectionState === ConnectionState.Connected) {
    content = <OfflineVideo username={hostName} />;
  } else if (!participant || tracks.length === 0) {
    content = <LoadingVideo label={connectionState} />;
  } else {
    content = <LiveVideo participant={participant} />;
  }

  return (
    <div className="aspect-video border-2 border-white group relative bg-transparent">
      {content}
    </div>
  );
```

```
};

export const VideoSkeleton = () => {
  return (
    <div className="aspect-video border-2 border-white bg-transparent relative">
      <div className="absolute inset-0 bg-gray-200 animate-pulse rounded-md" />
    </div>
  );
};
```

FILE: frontend/src/pages/LiveStream/VolumeControl.jsx

```
import { Volume1, Volume2, VolumeX } from "lucide-react";
import Hint from "./Hint";

export const VolumeControl = ({ onToggle, onChange, value }) => {
  const isMuted = value === 0;
  const isAboveHalf = value > 50;

  let Icon = Volume1;

  if (isMuted) {
    Icon = VolumeX;
  } else if (isAboveHalf) {
    Icon = Volume2;
  }

  const label = isMuted ? "Unmute" : "Mute";

  const handleChange = (event) => {
    onChange(+event.target.value); // `+` is used to convert the value to a number
  };

  return (
    <div className="flex items-center gap-2">
      <Hint label={label} asChild>
        <button
          onClick={onToggle}
          className="text-emerald-400 hover:bg-transparent p-1.5 rounded-lg border-2 border-white"
        >
          <Icon className="h-6 w-6 text-emerald-400" />
        </button>
      </Hint>

      <input
        type="range"
        min="0"
        max="100"
        step="1"
        value={value}
        onChange={handleChange}
        className="w-[8rem] cursor-pointer appearance-none bg-transparent border-2 border-white
rounded-full h-2 focus:outline-none focus:ring-2 focus:ring-emerald-400"
      />
    </div>
  );
};
```

FILE: frontend/src/pages/LoginPage.jsx

```
import { useState } from "react";
import { motion } from "framer-motion";
import { Link } from "react-router-dom";
import { LogIn, Mail, Lock, ArrowRight, Loader } from "lucide-react";
import { useUserStore } from "../stores/useUserStore";
```

```
const LoginPage = () => {
    const [email, setEmail] = useState("");
    const [password, setPassword] = useState("");

    const { login, loading } = useUserStore();

    const handleSubmit = (e) => {
        e.preventDefault();
        console.log(email, password);
        login(email, password);
    };

    return (
        <div className='flex flex-col justify-center py-12 sm:px-6 lg:px-8'>
            <motion.div
                className='sm:mx-auto sm:w-full sm:max-w-md'
                initial={{ opacity: 0, y: -20 }}
                animate={{ opacity: 1, y: 0 }}
                transition={{ duration: 0.8 }}
            >
                <h2 className='mt-6 text-center text-3xl font-extrabold text-emerald-400'>Log in
with your account</h2>
            </motion.div>

            <motion.div
                className='mt-8 sm:mx-auto sm:w-full sm:max-w-md'
                initial={{ opacity: 0, y: 20 }}
                animate={{ opacity: 1, y: 0 }}
                transition={{ duration: 0.8, delay: 0.2 }}
            >
                <div className='bg-gray-800 py-8 px-4 shadow sm:rounded-lg sm:px-10'>
                    <form onSubmit={handleSubmit} className='space-y-6'>
                        <div>
                            <label htmlFor='email' className='block text-sm font-medium text-gray-
300'>
                                Email address
                            </label>
                            <div className='mt-1 relative rounded-md shadow-sm'>
                                <div className='absolute inset-y-0 left-0 pl-3 flex items-center
pointer-events-none'>
                                    <Mail className='h-5 w-5 text-gray-400' aria-hidden='true' />
                                </div>
                                <input
                                    id='email'
                                    type='email'
                                    required
                                    value={email}
                                    onChange={(e) => setEmail(e.target.value)}
                                    className=' block w-full px-3 py-2 pl-10 bg-gray-700 border
border-gray-600
                                    rounded-md shadow-sm
                                     placeholder-gray-400 focus:outline-none focus:ring-emerald-500
                                     focus:border-emerald-500 sm:text-sm'
                                    placeholder='you@example.com'
                                />
                            </div>
                        </div>

                        <div>
                            <label htmlFor='password' className='block text-sm font-medium text-
gray-300'>
                                Password
                            </label>
                            <div className='mt-1 relative rounded-md shadow-sm'>
                                <div className='absolute inset-y-0 left-0 pl-3 flex items-center
```

```
pointer-events-none'>
                                    <Lock className='h-5 w-5 text-gray-400' aria-hidden='true' />
                                </div>
                                <input
                                    id='password'
                                    type='password'
                                    required
                                    value={password}
                                    onChange={(e) => setPassword(e.target.value)}
                                    className=' block w-full px-3 py-2 pl-10 bg-gray-700 border
border-gray-600
                                    rounded-md shadow-sm placeholder-gray-400 focus:outline-none
focus:ring-emerald-500 focus:border-emerald-500 sm:text-sm'
                                    placeholder='••••••••'
                                />
                            </div>
                        </div>

                        <button
                            type='submit'
                            className='w-full flex justify-center py-2 px-4 border border-
transparent
                            rounded-md shadow-sm text-sm font-medium text-white bg-emerald-600
                             hover:bg-emerald-700 focus:outline-none focus:ring-2 focus:ring-
offset-2
                              focus:ring-emerald-500 transition duration-150 ease-in-out
disabled:opacity-50'
                            disabled={loading}
                        >
                            {loading ? (
                                <>
                                    <Loader className='mr-2 h-5 w-5 animate-spin' aria-
hidden='true' />
                                    Loading...
                                </>
                            ) : (
                                <>
                                    <LogIn className='mr-2 h-5 w-5' aria-hidden='true' />
                                    Login
                                </>
                            )}
                        </button>
                    </form>

                    <p className='mt-8 text-center text-sm text-gray-400'>
                        Not a member?{" "}
                        <Link to='/signup' className='font-medium text-emerald-400 hover:text-
emerald-300'>
                            Sign up now <ArrowRight className='inline h-4 w-4' />
                        </Link>
                    </p>
                </div>
            </motion.div>
        </div>
    );
};
export default LoginPage;
```

FILE: frontend/src/pages/Map/MapBoxMap.jsx

```
import React, { useEffect, useState, useRef } from "react";
import { Map } from "react-map-gl";
import "mapbox-gl/dist/mapbox-gl.css";
import MapBoxRoute from "./MapBoxRoute";
import Markers from "./Markers";
```

```
const MAPBOX_DRIVING_ENDPOINT =
  "https://api.mapbox.com/directions/v5/mapbox/driving/";

const MapboxMap = ({ shipment, onSelectedRoute }) => {
  const mapRef = useRef();
  const mapContainerRef = useRef();  // Ref to the parent container for resize detection
  const [routesData, setRoutesData] = useState([]);
  const [routeColors, setRouteColors] = useState({});

  const generateRandomColor = () => {
    const letters = "0123456789ABCDEF";
    let color = "#";
    for (let i = 0; i < 6; i++) {
      color += letters[Math.floor(Math.random() * 16)];
    }
    return color;
  };

  useEffect(() => {
    if (shipment && shipment.routeOptimizations) {
      const routePromises = shipment.routeOptimizations.flatMap((routeOptimization) => {
        return routeOptimization.routeDetails.map((routeDetail) => {
          const { start_latitude, start_longitude, end_latitude, end_longitude, id } =
routeDetail.routeDetail;

          const routeRequestUrl =
`${MAPBOX_DRIVING_ENDPOINT}${start_longitude},${start_latitude};${end_longitude},${end_latitude}?
geometries=geojson&access_token=${import.meta.env.VITE_MAPBOX_API_KEY}`;

          return fetch(routeRequestUrl)
            .then((response) => response.json())
            .then((data) => {
              if (data.routes && data.routes.length > 0) {
                setRouteColors((prevColors) => {
                  if (!prevColors[id]) {
                    return {
                      ...prevColors,
                      [id]: generateRandomColor(),
                    };
                  }
                  return prevColors;
                });

                return {
                  routeId: id,
                  coordinates: data.routes[0].geometry.coordinates,
                };
              }
              return null;
            })
            .catch((error) => {
              console.error("Error fetching route:", error);
              return null;
            });
        });
      });

      Promise.all(routePromises).then((routeDetails) => {
        const validRouteDetails = routeDetails.filter((route) => route !== null);
        setRoutesData(validRouteDetails);
      });

      const firstRoute = shipment.routeOptimizations[0].routeDetails[0].routeDetail;
      const { start_latitude, start_longitude } = firstRoute;
      mapRef.current?.flyTo({
        center: [start_longitude, start_latitude],
```

```
        duration: 2500,
      });
    }
  }, [shipment]);

  const getRouteColor = (routeId) => {
    return routeColors[routeId] || "#1e90ff";
  };

  // Handle window resize or container resize
  useEffect(() => {
    const handleResize = () => {
      if (mapRef.current) {
        mapRef.current.resize(); // Resize the map when the container size changes
      }
    };

    // Listen for resize events on the window and the parent container
    window.addEventListener("resize", handleResize);
    if (mapContainerRef.current) {
      new ResizeObserver(handleResize).observe(mapContainerRef.current); // Observe parent
container resize
    }

    return () => {
      window.removeEventListener("resize", handleResize);
      if (mapContainerRef.current) {
        // Cleanup observer on unmount
        const resizeObserver = new ResizeObserver(handleResize);
        resizeObserver.disconnect();
      }
    };
  }, []);

  return (
    <div className="w-full" style={{ height: "100vh" }} ref={mapContainerRef}> {/* Parent container
ref for resize detection */}
      <h2 className="text-[20px] font-semibold">Map</h2>
      <div style={{ height: "100%" }} className="rounded-lg">
        {shipment ? (
          <Map
            ref={mapRef}
            mapboxAccessToken={import.meta.env.VITE_MAPBOX_API_KEY}
            initialViewState={{
              longitude:
shipment.routeOptimizations[0]?.routeDetails[0]?.routeDetail?.start_longitude || -74.0060,
              latitude:
shipment.routeOptimizations[0]?.routeDetails[0]?.routeDetail?.start_latitude || 40.7128,
              zoom: 5,
            }}
            style={{ width: "100%", height: "100%", borderRadius: "10px" }} // Ensure the map takes
full height of its parent container
            mapStyle="mapbox://styles/mapbox/streets-v9"
          >
            <Markers shipment={shipment} onSelectedRoute={onSelectedRoute} />
            {routesData.map((route) => (
              <MapBoxRoute
                key={route.routeId}
                coordinates={route.coordinates}
                routeColor={getRouteColor(route.routeId)}
              />
            ))}
          </Map>
        ) : null}
      </div>
    </div>
```

```
  );
};

export default MapboxMap;
```

FILE: frontend/src/pages/Map/MapBoxRoute.jsx

```
import React from "react";
import { Layer, Source } from "react-map-gl";

const MapBoxRoute = ({ coordinates, routeColor }) => {
  return (
    <Source type="geojson" data={{ type: "Feature", geometry: { type: "LineString", coordinates }
}}>
      <Layer
        type="line"
        layout={{ "line-join": "round", "line-cap": "square" }}
        paint={{
          "line-color": routeColor,
          "line-width": 4,
        }}
      />
    </Source>
  );
};

export default MapBoxRoute;
```

FILE: frontend/src/pages/Map/Markers.jsx

```
import React from "react";
import { Marker } from "react-map-gl";
import { FaMapMarkerAlt } from "react-icons/fa";  // Importing a marker icon from react-icons

const Markers = ({ shipment, onSelectedRoute }) => {
  const handleMarkerClick = (routeDetail) => {
    // Call the onSelectedRoute with the clicked routeDetail
    onSelectedRoute(routeDetail);
  };
  return (
    <div>
      {/* Iterate over route details and display start/end markers */}
      {shipment.routeOptimizations.map((routeOptimization) =>
        routeOptimization.routeDetails.map((routeDetail) => (
          <div key={routeDetail.routeDetail.id}>
            {/* Start Location Marker with Icon from react-icons */}
            {  console.log(routeDetail.routeDetail)
            }
            <Marker
              longitude={routeDetail.routeDetail.start_longitude}
              latitude={routeDetail.routeDetail.start_latitude}
              anchor="bottom"
              onClick={() => handleMarkerClick(routeDetail.routeDetail)} // Handle start marker
click
            >
              <FaMapMarkerAlt size={30} color="green" /> {/* Start marker */}
            </Marker>

            {/* End Location Marker with Icon from react-icons */}
            <Marker
              longitude={routeDetail.routeDetail.end_longitude}
              latitude={routeDetail.routeDetail.end_latitude}
              anchor="bottom"
              onClick={() => handleMarkerClick(routeDetail.routeDetail)} // Handle end marker click
            >
```

```
              <FaMapMarkerAlt size={30} color="red" /> {/* End marker */}
            </Marker>
          </div>
        ))
      )}
    </div>
  );
};

export default Markers;
```

FILE: frontend/src/pages/Map/RouteDetailPage.jsx

```
import React from 'react';
import useDeliveryStore from '../../stores/useDeliveryStore';

const RouteDetailPage = ({ routeDetail, vehicle }) => {
    const { handleSaveVehicleId, isLoading, isError, errorMessage } = useDeliveryStore();

    // Check if routeDetail or vehicle is null or undefined
    if (!routeDetail) {
        return <div className="text-center py-8 text-lg text-gray-500">Loading route details...
</div>;
    }

    if (!vehicle) {
        return <div className="text-center py-8 text-lg text-gray-500">Loading vehicle
information...</div>;
    }

    const assignedVehicleId = vehicle.id;

    const handleSave = () => {
        handleSaveVehicleId(routeDetail.id, assignedVehicleId);
    };

    return (
        <div className="mt-10 max-w-6xl mx-auto p-6 bg-transparent rounded-lg shadow-xl border-2
border-gray-300">
            <h1 className="text-4xl font-bold text-center mb-6 text-emerald-400">Route Detail</h1>

            <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
                {/* Route Information */}
                <div className="bg-transparent p-6 rounded-lg shadow-md border-2 border-gray-300
space-y-4">
                    <h2 className="text-2xl font-semibold text-emerald-400">Route Information</h2>
                    <div className="space-y-2">
                        <p className='text-white'><strong className="text-emerald-400">Route Name:
</strong> {routeDetail.route_name || 'N/A'}</p>
                        <p className='text-white'><strong className="text-emerald-400">Supplier
Name:</strong> {routeDetail.supplier_name || 'N/A'}</p>
                        <p className='text-white'><strong className="text-emerald-400">Warehouse 1:
</strong> {routeDetail.warehouse_name_1 || 'N/A'}</p>
                        <p className='text-white'><strong className="text-emerald-400">Warehouse 2:
</strong> {routeDetail.warehouse_name_2 || 'N/A'}</p>
                        <p className='text-white'><strong className="text-emerald-400">Start
Location:</strong> {routeDetail.start_location || 'N/A'}</p>
                        <p className='text-white'><strong className="text-emerald-400">End
Location:</strong> {routeDetail.end_location || 'N/A'}</p>
                        <p className='text-white'><strong className="text-emerald-400">Estimated
Time:</strong> {routeDetail.estimated_time || 'N/A'}</p>
                        <p className='text-white'><strong className="text-emerald-400">Distance:
</strong> {routeDetail.distance ? `${routeDetail.distance} km` : 'N/A'}</p>
                        <p className='text-white'><strong className="text-emerald-400">Route
Condition:</strong> {routeDetail.route_condition?.name || 'N/A'}</p>
                    </div>
```

```
                </div>

                {/* Assigned Vehicle Information */}
                <div className="bg-transparent p-6 rounded-lg shadow-md border-2 border-gray-300
space-y-4">
                    <h2 className="text-2xl font-semibold text-emerald-400">Assigned Vehicle</h2>
                    <div className="flex items-center space-x-4">
                        <div className="flex-shrink-0">
                            <img
                                src={vehicle.image_url || "/path/to/default-image.jpg"}
                                alt={vehicle.name}
                                className="w-20 h-20 object-cover rounded-md shadow-lg border-2
border-gray-300"
                            />
                        </div>
                        <div>
                            <p className='text-white'><strong className="text-emerald-400">Vehicle
Name:</strong> {vehicle.name || 'N/A'}</p>
                            <p className='text-white'><strong className="text-emerald-400">Model:
</strong> {vehicle.model || 'N/A'}</p>
                            <p className='text-white'><strong className="text-emerald-400">License
Plate:</strong> {vehicle.license_plate || 'N/A'}</p>
                        </div>
                    </div>
                </div>
            </div>

            {/* Save Button */}
            <div className="mt-6 text-center">
                <button
                    onClick={handleSave}
                    disabled={isLoading}
                    className={`border-2 border-white px-6 py-3 text-lg font-semibold rounded-md
${isLoading ? 'bg-gray-500' : 'bg-emtext-emerald-400'} text-white hover:bg-emtext-emerald-400
focus:outline-none focus:ring-2 focus:ring-emerald-400 focus:ring-opacity-50 transition duration-
300`}
                >
                    {isLoading ? 'Saving...' : 'Pick This Route'}
                </button>
            </div>

            {/* Error Message */}
            {isError && (
                <div className="mt-6 text-center text-red-600 text-lg font-medium">
                    <p className='text-white'>{errorMessage || 'Something went wrong. Please try
again.'}</p>
                </div>
            )}
        </div>
    );
};

export default RouteDetailPage;
```

FILE: frontend/src/pages/Map/WrapperMap.jsx

```
import React, { useEffect, useState } from "react";
import Navbar from "../../components/Map/Navbar";
import Sidebar from "../../components/Map/Sidebar";
import { useAppSelector } from "../State/Redux";
import useDeliveryStore from "../../stores/useDeliveryStore";
import MapboxMap from "./MapBoxMap";
import RouteDetailPage from "./RouteDetailPage";

const DashboardLayout = () => {
  const isSidebarCollapsed = useAppSelector(
```

```
    (state) => state.global.isSidebarCollapsed
  );
  const isDarkMode = useAppSelector((state) => state.global.isDarkMode);

  // Use the custom hook to fetch shipments and vehicle
  const { shipments, isLoading, errorMessage, fetchShipmentsWithRouteDetails,
getVehicleForAuthenticatedUser, vehicle } = useDeliveryStore();
  const [selectedShipment, setSelectedShipment] = useState(null);
  const [route, setRoute] = useState(null);

  const onSelectedRoute = (route) => {
    setRoute(route);
  }

  // Callback to set the selected shipment
  const handleSetSelectedShipment = (shipment) => {
    setSelectedShipment(shipment);
  };

  useEffect(() => {
    // Fetch shipments when the component mounts
    if (shipments.length === 0) {
      fetchShipmentsWithRouteDetails();
    }

    // Fetch the vehicle for the authenticated user
    getVehicleForAuthenticatedUser();

    // Handle dark mode toggling
    if (isDarkMode) {
      document.documentElement.classList.add("dark");
    } else {
      document.documentElement.classList.remove("dark");
    }
  }, [isDarkMode, shipments.length, fetchShipmentsWithRouteDetails,
getVehicleForAuthenticatedUser]);

  return (
    <div
      className={`${
        isDarkMode ? "dark" : "light"
      } flex bg-gray-900 text-gray-900 w-full h-full`}
    >
      {/* Pass the shipments data to the Sidebar */}
      {!isSidebarCollapsed && (
        <Sidebar
          shipments={shipments}
          isLoading={isLoading}
          errorMessage={errorMessage}
          setSelectedShipment={handleSetSelectedShipment}
        />
      )}

      <main
        className={"flex flex-col w-full h-full py-7 px-9 bg-gray-900"}
      >
        <Navbar />
        <div className="flex flex-col">
          <MapboxMap shipment={selectedShipment} onSelectedRoute={onSelectedRoute} />
          {/* Pass the route and vehicle to the RouteDetailPage */}
          <RouteDetailPage routeDetail={route} vehicle={vehicle} />
        </div>
      </main>
    </div>
  );
};
```

```
const WrapperMap = ({ children }) => {
  return (
    <DashboardLayout>{children}</DashboardLayout>
  );
};

export default WrapperMap;
```

FILE: frontend/src/pages/MapBoxMap.jsx

```
import React, { useEffect, useState, useRef } from "react";
import { Map } from "react-map-gl";
import "mapbox-gl/dist/mapbox-gl.css";
import MapBoxRoute from "./MapBoxRoute";
import Markers from "./Markers";

const MAPBOX_DRIVING_ENDPOINT =
  "https://api.mapbox.com/directions/v5/mapbox/driving/";

const MapboxMap = ({ routeDetails }) => {
  const mapRef = useRef();
  const mapContainerRef = useRef();  // Ref to the parent container for resize detection
  const [routesData, setRoutesData] = useState([]);
  const [routeColors, setRouteColors] = useState({});

  const generateRandomColor = () => {
    const letters = "0123456789ABCDEF";
    let color = "#";
    for (let i = 0; i < 6; i++) {
      color += letters[Math.floor(Math.random() * 16)];
    }
    return color;
  };
  console.log(routeDetails)
  useEffect(() => {
    if (routeDetails && routeDetails.length > 0) {
      const routePromises = routeDetails.flatMap((routeDetail) => {
        const { start_latitude, start_longitude, end_latitude, end_longitude, id } = routeDetail;

        // Only continue if we have coordinates for both start and end
        if (start_latitude && start_longitude && end_latitude && end_longitude) {
          const routeRequestUrl =
`${MAPBOX_DRIVING_ENDPOINT}${start_longitude},${start_latitude};${end_longitude},${end_latitude}?
geometries=geojson&access_token=${import.meta.env.VITE_MAPBOX_API_KEY}`;

          return fetch(routeRequestUrl)
            .then((response) => response.json())
            .then((data) => {
              if (data.routes && data.routes.length > 0) {
                // Assign a unique random color for each route
                setRouteColors((prevColors) => {
                  if (!prevColors[id]) {
                    return {
                      ...prevColors,
                      [id]: generateRandomColor(),
                    };
                  }
                  return prevColors;
                });

                return {
                  routeId: id,
                  coordinates: data.routes[0].geometry.coordinates,
                };
              }
```

```
            return null;
          })
          .catch((error) => {
            console.error("Error fetching route:", error);
            return null;
          });
      }
      return null; // Skip this route if coordinates are missing
    });

    Promise.all(routePromises).then((routeDetails) => {
      const validRouteDetails = routeDetails.filter((route) => route !== null);
      setRoutesData(validRouteDetails); // Only set valid routes
    });

    const firstRoute = routeDetails[0];
    const { start_latitude, start_longitude } = firstRoute;
    mapRef.current?.flyTo({
      center: [start_longitude, start_latitude],
      duration: 2500,
    });
  }
}, [routeDetails]);

const getRouteColor = (routeId) => {
  return routeColors[routeId] || "#1e90ff"; // Default color if not set
};

// Handle window resize or container resize
useEffect(() => {
  const handleResize = () => {
    if (mapRef.current) {
      mapRef.current.resize(); // Resize the map when the container size changes
    }
  };

  // Listen for resize events on the window and the parent container
  window.addEventListener("resize", handleResize);
  if (mapContainerRef.current) {
    new ResizeObserver(handleResize).observe(mapContainerRef.current); // Observe parent
container resize
  }

  return () => {
    window.removeEventListener("resize", handleResize);
    if (mapContainerRef.current) {
      // Cleanup observer on unmount
      const resizeObserver = new ResizeObserver(handleResize);
      resizeObserver.disconnect();
    }
  };
}, []);

return (
  <div className="w-full" style={{ height: "100vh" }} ref={mapContainerRef}> {/* Parent container
ref for resize detection */}
    <h2 className="text-[20px] font-semibold">Map</h2>
    <div style={{ height: "100%" }} className="rounded-lg">
      {routeDetails && routeDetails.length > 0 ? (
        <Map
          ref={mapRef}
          mapboxAccessToken={import.meta.env.VITE_MAPBOX_API_KEY}
          initialViewState={{
            longitude: routeDetails[0]?.start_longitude || -74.0060,
            latitude: routeDetails[0]?.start_latitude || 40.7128,
            zoom: 5,
```

```
            }}
            style={{ width: "100%", height: "100%", borderRadius: "10px" }} // Ensure the map takes
full height of its parent container
            mapStyle="mapbox://styles/mapbox/streets-v9"
          >
            <Markers routeDetails={routeDetails} />
            {routesData.map((route) => (
              <MapBoxRoute
                key={route.routeId}
                coordinates={route.coordinates}
                routeColor={getRouteColor(route.routeId)}
              />
            ))}
          </Map>
        ) : null}
      </div>
    </div>
  );
};

export default MapboxMap;
```

FILE: frontend/src/pages/MapBoxRoute.jsx

```
import React from "react";
import { Layer, Source } from "react-map-gl";

const MapBoxRoute = ({ coordinates, routeColor }) => {
  return (
    <Source type="geojson" data={{ type: "Feature", geometry: { type: "LineString", coordinates }
}}>
      <Layer
        type="line"
        layout={{ "line-join": "round", "line-cap": "square" }}
        paint={{
          "line-color": routeColor,
          "line-width": 4,
        }}
      />
    </Source>
  );
};

export default MapBoxRoute;
```

FILE: frontend/src/pages/Markers.jsx

```
import React from "react";
import { Marker } from "react-map-gl";
import { FaMapMarkerAlt } from "react-icons/fa"; // Importing a marker icon from react-icons

const Markers = ({ routeDetails }) => {
  return (
    <div>
      {/* Iterate over routeDetails and display start/end markers */}
      {routeDetails.map((routeDetail) => (
        <div key={routeDetail.id}> {/* Use the route ID as the key */}
          {/* Start Location Marker with Icon from react-icons */}
          {routeDetail.start_latitude && routeDetail.start_longitude && (
            <Marker
              longitude={parseFloat(routeDetail.start_longitude)}  // Ensure longitude is a float
              latitude={parseFloat(routeDetail.start_latitude)}   // Ensure latitude is a float
              anchor="bottom"
            >
              <FaMapMarkerAlt size={30} color="green" /> {/* Start marker */}
```

```
            </Marker>
          )}

          {/* End Location Marker with Icon from react-icons */}
          {routeDetail.end_latitude && routeDetail.end_longitude && (
            <Marker
              longitude={parseFloat(routeDetail.end_longitude)}  // Ensure longitude is a float
              latitude={parseFloat(routeDetail.end_latitude)}   // Ensure latitude is a float
              anchor="bottom"
            >
              <FaMapMarkerAlt size={30} color="red" /> {/* End marker */}
            </Marker>
          )}
        </div>
      ))}
    </div>
  );
};

export default Markers;
```

FILE: frontend/src/pages/PaymentPage.jsx

```
import React, { useState, useEffect } from 'react';
import { useCartStore } from '../stores/useCartStore';
import { useNavigate } from 'react-router-dom';
import { useUserStore } from '../stores/useUserStore';
import { useStripe, useElements, CardElement } from '@stripe/react-stripe-js';
import { loadStripe } from '@stripe/stripe-js';
// Load Stripe with your public key
const stripePromise =
loadStripe('pk_test_51PMjcCIw69kb65LMm6zd49cWEi1zj4wFnwVbF9mxurJg1JlgoH0S7tOPdJglr0YmyejIYYfDTHhVTFC

const PaymentPage = () => {
  const {
    cart,
    totalAmount,
    discountAmount,
    totalAfterDiscount,
    processPayment,
    isPaymentProcessing,
    paymentMessage,
    createStripePaymentIntent, // Ensure this is available in your store
    stripeClientSecret,
    setUserLocation,
  } = useCartStore();

  const { userAddresses, getUserAddresses } = useUserStore();
  useEffect(() => {
    // Fetch addresses when the component mounts
    getUserAddresses();
  }, [getUserAddresses]);

  const [paymentMethod, setPaymentMethod] = useState('credit_card');
  const [paymentGateway, setPaymentGateway] = useState('stripe');
  const [currency, setCurrency] = useState('USD');
  const navigate = useNavigate();
  const [isPaid, setIsPaid] = useState(false);
  const [selectedAddress, setSelectedAddress] = useState(userAddresses[0] || null);

  // Stripe Elements hooks
  const stripe = useStripe();
  const elements = useElements();

  // Handle payment form submission
  const handlePaymentSubmit = async (e) => {
```

```
    e.preventDefault();
    setIsPaid(false);

    if (!stripe || !elements) {
      console.error('Stripe has not loaded yet');
      return;
    }

    // Get the client secret from your store (ensure it's already created)
    await createStripePaymentIntent(currency, totalAfterDiscount); // This will set the client
secret in your store

    // Make sure the postal code is provided
    const postalCode = selectedAddress ? selectedAddress.postal_code : '';

    // Confirm payment using Stripe
    const cardElement = elements.getElement(CardElement);
    if(stripeClientSecret){

      const { error, paymentIntent } = await stripe.confirmCardPayment(stripeClientSecret, {
        payment_method: {
          card: cardElement,
          billing_details: {
            address: {
              postal_code: postalCode, // Include the postal code here
            },
          },
        },
      });

      if (error) {
        console.error(error.message);
        alert(error.message);
        return;
      }

      console.log(paymentIntent.status);

      // If payment was successful, call your store's `processPayment` method
      if (paymentIntent.status === 'succeeded') {
        setIsPaid(true);
        processPayment(paymentMethod, paymentGateway, currency);

      }
    }
  };

  useEffect(() => {
    if (!selectedAddress) return;
    const userLocation = `${selectedAddress.address_line1}, ${selectedAddress.city},
${selectedAddress.state}, ${selectedAddress.country}, ${selectedAddress.postal_code}`;
    setUserLocation(userLocation);
  }, [selectedAddress]);

  useEffect(() => {
    console.log(paymentMessage)
    if (paymentMessage === 'ok' && isPaid) {
      navigate('/purchase-success');
    }
  }, [paymentMessage, isPaid, navigate]);

  return (
    <div className="w-full h-full p-8">
      <div className="container mx-auto max-w-7xl p-6 bg-gray-800 text-white rounded-xl shadow-lg">
        <div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
          {/* Left Section: Order Details */}
```

```
          <div className="space-y-6">
            <div className="p-6 bg-gray-900 rounded-md shadow-md">
              <p className="text-xl font-semibold">Total Amount: ${totalAmount.toFixed(2)}</p>
              {discountAmount > 0 && (
                <>
                  <p className="text-lg">Discount Applied: -${discountAmount.toFixed(2)}</p>
                  <p className="text-lg font-semibold">Total After Discount:
${totalAfterDiscount.toFixed(2)}</p>
                </>
              )}
            </div>

            <div className="mt-6">
              <h3 className="text-2xl font-semibold mb-4">Order Details</h3>
              <div className="space-y-4">
                {cart.map((item) => (
                  <div key={item.products_id} className="flex justify-between items-center bg-gray-
800 p-3 rounded-md shadow-lg hover:bg-gray-700 transition-all">
                    <div className="flex items-center">
                      <img
                        src={item.product.image}
                        alt={item.product.name}
                        className="w-16 h-16 object-cover mr-4 rounded-md"
                      />
                      <span className="text-lg">{item.product.name}</span>
                    </div>
                    <div className="text-lg font-medium">
                      {item.quantity} x ${item.total_amount.toFixed(2)}
                    </div>
                  </div>
                ))}
              </div>
            </div>
          </div>

          {/* Right Section: Payment Form */}
          <div className="bg-gray-900 p-6 rounded-md shadow-lg">
            <h3 className="text-2xl font-semibold mb-4">Payment Information</h3>

            <form onSubmit={handlePaymentSubmit} className="space-y-6">
              {/* Payment Method */}
              <div>
                <label htmlFor="payment_method" className="block text-lg font-medium mb-2">Payment
Method</label>
                <select
                  id="payment_method"
                  value={paymentMethod}
                  onChange={(e) => setPaymentMethod(e.target.value)}
                  className="w-full px-4 py-3 bg-gray-700 text-white rounded-lg focus:outline-none
focus:ring-2 focus:ring-emerald-500"
                >
                  <option value="credit_card">Credit Card</option>
                  <option value="paypal">Cash</option>
                </select>
              </div>

              {/* Payment Gateway */}
              <div>
                <label htmlFor="payment_gateway" className="block text-lg font-medium mb-2">Payment
Gateway</label>
                <select
                  id="payment_gateway"
                  value={paymentGateway}
                  onChange={(e) => setPaymentGateway(e.target.value)}
                  className="w-full px-4 py-3 bg-gray-700 text-white rounded-lg focus:outline-none
focus:ring-2 focus:ring-emerald-500"
```

```
                >
                  <option value="stripe">Stripe</option>
                  <option value="paypal">PayPal</option>
                  <option value="razorpay">Razorpay</option>
                  <option value="zalopay">Zalopay</option>
                </select>
              </div>

              {/* Currency Selection */}
              <div>
                <label htmlFor="currency" className="block text-lg font-medium mb-
2">Currency</label>
                <select
                  id="currency"
                  value={currency}
                  onChange={(e) => setCurrency(e.target.value)}
                  className="w-full px-4 py-3 bg-gray-700 text-white rounded-lg focus:outline-none
focus:ring-2 focus:ring-emerald-500"
                >
                  <option value="USD">USD</option>
                  <option value="EUR">EUR</option>
                  <option value="INR">INR</option>
                </select>
              </div>

              {/* Card Element */}
              {paymentGateway === 'stripe' && (
                <div>
                  <label className="block text-lg font-medium mb-2">Credit Card Information</label>
                  <CardElement
                    options={{
                      style: {
                        base: {
                          color: '#fff',
                          fontSize: '16px',
                          fontFamily: 'sans-serif',
                          '::placeholder': {
                            color: '#aaa',
                          },
                        },
                      },
                    }}
                  />
                </div>
              )}

              {/* Payment Button */}
              <button
                type="submit"
                disabled={isPaymentProcessing}
                className="w-full py-3 bg-emerald-600 text-white font-semibold rounded-lg hover:bg-
emerald-700 focus:outline-none focus:ring-4 focus:ring-emerald-300 disabled:bg-gray-600 transition-
all"
              >
                {isPaymentProcessing ? 'Processing...' : 'Confirm Payment'}
              </button>
            </form>

            {/* Delivery Address Selection */}
            <div className="mt-6">
              <h4 className="text-xl font-semibold">Select Delivery Address</h4>
              <select
                value={selectedAddress ? `${selectedAddress.address_line1},
${selectedAddress.city}, ${selectedAddress.state}` : ''}
                onChange={(e) => {
                  const selected = userAddresses.find(address =>
```

```
                    `${address.address_line1}, ${address.city}, ${address.state}` ===
e.target.value
                  );
                  setSelectedAddress(selected);
                }}
                className="w-full px-4 py-3 bg-gray-700 text-white rounded-lg focus:outline-none
focus:ring-2 focus:ring-teal-500 mt-2"
              >
                {userAddresses.map((address) => (
                  <option key={address.id} value={`${address.address_line1}, ${address.city},
${address.state}`}>
                    {address.address_line1}, {address.city}, {address.state}
                  </option>
                ))}
              </select>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
};

export default PaymentPage;
```

FILE: frontend/src/pages/ProductDetailPage.jsx

```
import { useEffect, useState } from "react";
import { useParams } from "react-router-dom";
import { useCartStore } from "../stores/useCartStore";
import { useProductStore } from "../stores/useProductStore";
import useRatingStore from "../stores/useRatingStore"; // Import the rating store
import { FaStar } from "react-icons/fa"; // For Star Rating Icon
import ProductCard from "../components/ProductCard";
const ProductDetailPage = () => {
  const { id } = useParams(); // Getting the product ID from the URL
  const { addToCart } = useCartStore();
  const { currentProduct, loading, fetchProductById, fetchProductsByCategory, products } =
useProductStore();

  // Access rating store
  const { ratings, ratingLoading, fetchRatings, submitRating, isSubmitting } = useRatingStore();

  const [feedback, setFeedback] = useState("");
  const [ratingValue, setRatingValue] = useState(0);
  const [relatedProducts, setRelatedProducts] = useState([]); // State for related products

  useEffect(() => {
    fetchProductById(id); // Fetch the product details when the component mounts
    fetchRatings(id); // Fetch ratings for the product
  }, [id, fetchProductById, fetchRatings]);

  useEffect(() => {
    if (currentProduct?.categories) {
      // Fetch products by categories after the product details are fetched
      const categories = currentProduct.categories.map(category => category.category_name);
      console.log(categories)
      fetchProductsByCategory(categories);
    }
  }, [currentProduct, fetchProductsByCategory]);

  // Assuming fetchProductsByCategory updates the `relatedProducts` state
  useEffect(() => {
    console.log(products)
    if (products.length > 0) {
      setRelatedProducts(products); // Update related products when fetched
```

```
    }
  }, [products]);

  const handleAddToCart = () => {
    addToCart(currentProduct.id);
  };

  const handleFeedbackSubmit = async (e) => {
    e.preventDefault();
    if (ratingValue === 0) {
      alert("Please provide a rating!");
      return;
    }

    await submitRating(id, ratingValue, feedback, 1); // Assuming shipmentId is 1 for now
    setFeedback(""); // Reset feedback after submission
    setRatingValue(0); // Reset rating
  };

  const renderStars = (value) => {
    const stars = [];
    for (let i = 1; i <= 5; i++) {
      stars.push(
        <FaStar
          key={i}
          className={`cursor-pointer ${i <= value ? "text-yellow-500" : "text-gray-300"}`}
          onClick={() => setRatingValue(i)}
        />
      );
    }
    return stars;
  };

  if (loading || ratingLoading) {
    return (
      <div className="flex justify-center items-center min-h-screen bg-gradient-to-r from-green-500
to-teal-500">
        <p className="text-white text-xl">Loading...</p>
      </div>
    );
  }

  if (!currentProduct) {
    return (
      <div className="flex justify-center items-center min-h-screen bg-gradient-to-r from-green-500
to-teal-500">
        <p className="text-white text-xl">Product not found</p>
      </div>
    );
  }

  return (
    <div className="container mx-auto px-4 py-12 rounded-xl shadow-xl">
      <div className="flex flex-col lg:flex-row items-center lg:items-start space-y-6 lg:space-y-0
lg:space-x-12">
        {/* Product Image */}
        <div className="flex-shrink-0 w-full sm:w-64 lg:w-96 rounded-lg overflow-hidden shadow-lg
hover:scale-105 transform transition-all">
          <img
            className="object-cover w-full h-64 lg:h-80 rounded-lg"
            src={currentProduct.image}
            alt={currentProduct.name}
          />
        </div>

        {/* Product Information */}
```

```
        <div className="flex-grow text-center lg:text-left">
          <h1 className="text-3xl font-extrabold text-emerald-400 mb-4">{currentProduct.name}</h1>
          <p className="text-xl text-emerald-400 mb-4">${currentProduct.price}</p>
          <p className="text-lg text-white mb-6">{currentProduct.description}</p>

          {/* Categories */}
          <div className="mb-6">
            <strong className="text-lg text-emerald-400">Categories:</strong>
            {currentProduct.categories && currentProduct.categories.length > 0 ? (
              <div className="flex space-x-4 mt-2">
                {currentProduct.categories.map((category) => (
                  <span key={category.id} className="text-lg text-white px-3 py-1 bg-emerald-600
rounded-full shadow-md hover:bg-emerald-700 cursor-pointer">
                    {category.category_name}
                  </span>
                ))}
              </div>
            ) : (
              <p className="text-gray-500">No categories available</p>
            )}
          </div>

          {/* Supplier */}
          <div className="mb-6">
            <strong className="text-lg text-emerald-400">Supplier:</strong>
            <p className="text-lg text-white">
              {(currentProduct.supplier?.first_name + " " + currentProduct.supplier?.last_name) ||
"Unknown Supplier"}
            </p>
          </div>

          {/* Add to Cart Button */}
          <button
            className="w-full sm:w-auto bg-emerald-600 hover:bg-emerald-700 text-white font-bold
py-3 px-8 rounded-lg shadow-md transform transition-all hover:scale-105"
            onClick={handleAddToCart}
          >
            Add to Cart
          </button>
        </div>
      </div>

      {/* Ratings and Feedback */}
      <div className="mt-12">
        <h2 className="text-2xl font-bold text-white mb-6">Ratings & Feedback</h2>

        {/* Display Ratings */}
        {ratings.length > 0 ? (
          <div className="space-y-4">
            {ratings.map((rating) => (
              <div key={rating.id} className="bg-gray-700 p-4 rounded-lg shadow-md">
                <div className="flex items-center space-x-2">
                  <div className="flex">
                    {renderStars(rating.rating_value)}
                  </div>
                  <span className="text-sm text-gray-400">- {new
Date(rating.date_created).toLocaleDateString()}</span>
                </div>
                {rating.feedback && <p className="text-white mt-2">{rating.feedback}</p>}
              </div>
            ))}
          </div>
        ) : (
          <p className="text-gray-500">No ratings yet for this product</p>
        )}
```

```
        {/* Add Feedback Form */}
        <div className="mt-8">
          <form onSubmit={handleFeedbackSubmit} className="space-y-4">
            <div>
              <label className="text-white font-semibold">Your Rating:</label>
              <div className="flex space-x-2 mt-2">
                {renderStars(ratingValue)}
              </div>
            </div>

            <div>
              <textarea
                value={feedback}
                onChange={(e) => setFeedback(e.target.value)}
                placeholder="Your feedback..."
                className="w-full px-4 py-2 rounded-lg bg-gray-700 text-white mt-4"
                rows="4"
              />
            </div>

            <button
              type="submit"
              className="w-full sm:w-auto bg-emerald-600 hover:bg-emerald-700 text-white font-bold
py-3 px-8 rounded-lg shadow-md transform transition-all hover:scale-105"
              disabled={isSubmitting}
            >
              {isSubmitting ? "Submitting..." : "Submit Feedback"}
            </button>
          </form>
        </div>
      </div>

      {/* Related Products */}
      <div className="mt-12">
        <h2 className="text-2xl font-bold text-white mb-6">Related Products</h2>
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-8">
          {relatedProducts.length > 0 ? (
            relatedProducts.map((product) => (
              <ProductCard key={product.id} product={product} />
            ))
          ) : (
            <p className="text-gray-500">No related products found.</p>
          )}
        </div>
      </div>
    </div>
  );
};

export default ProductDetailPage;
```

FILE: frontend/src/pages/ProfilePage.jsx

```
import React, { useState } from 'react';
import { useUserStore } from '../stores/useUserStore';

const ProfilePage = () => {
  const {
    user,
    loading,
    userOrders,
    userInvoices,
    userRoles,
    userPermissions
  } = useUserStore();
```

```
  const [searchQuery, setSearchQuery] = useState('');
  const [currentPage, setCurrentPage] = useState(1);
  const itemsPerPage = 2;
  const [sortField, setSortField] = useState('orderDate');
  const [sortOrder, setSortOrder] = useState('asc');

  if (loading) {
    return <div className="text-center text-gray-500">Loading...</div>;
  }

  if (!user) {
    return (
      <div className="text-center text-emerald-400">
        Please log in to view your profile.
      </div>
    );
  }

  /* ========================
     SEARCH
  ========================= */
  const filteredOrders = userOrders?.filter(order => (
    order.id.toString().includes(searchQuery) ||
    order.status?.toLowerCase().includes(searchQuery.toLowerCase()) ||
    order.order_date?.includes(searchQuery)
  )) || [];

  /* ========================
     SORT
  ========================= */
  const sortedOrders = [...filteredOrders].sort((a, b) => {
    const aValue = sortField === 'orderDate'
      ? new Date(a.order_date)
      : a[sortField];

    const bValue = sortField === 'orderDate'
      ? new Date(b.order_date)
      : b[sortField];

    return sortOrder === 'asc'
      ? aValue - bValue
      : bValue - aValue;
  });

  /* ========================
     PAGINATION
  ========================= */
  const totalPages = Math.ceil(sortedOrders.length / itemsPerPage);
  const displayedOrders = sortedOrders.slice(
    (currentPage - 1) * itemsPerPage,
    currentPage * itemsPerPage
  );

  return (
    <div className="w-full bg-gray-900 text-emerald-400 p-6">
      <h2 className="text-4xl font-bold text-center mb-8">Profile</h2>

      <div className="space-y-8">

        {/* ========================
           USER INFO
        ========================= */}
        <div className="bg-gray-800 p-6 rounded-lg shadow-md">
          <h3 className="text-2xl font-semibold mb-4">User Information</h3>

          <div><span className="font-semibold">Name:</span> {user.first_name} {user.last_name}
```

```
</div>
          <div><span className="font-semibold">Email:</span> {user.email}</div>
          <div><span className="font-semibold">Phone:</span> {user.phone_number}</div>
          <div><span className="font-semibold">Language:</span> {user.language}</div>

          {/* ==========================
             ROLES
          ========================== */}
          <div className="mt-6">
            <span className="font-semibold text-lg">Roles:</span>
            {userRoles?.length > 0 ? (
              <div className="flex flex-col gap-2 mt-2">
                {userRoles.map((role, index) => (
                  <div
                    key={index}
                    className="bg-emerald-600 text-white px-3 py-2 rounded-lg text-sm flex flex-
col"
                  >
                    <span className="font-semibold">{role.role_name}</span>
                    <span className="text-gray-200 text-xs">{role.description}</span>
                  </div>
                ))}
              </div>
            ) : (
              <div className="text-gray-500 mt-2">No roles assigned</div>
            )}
          </div>

          {/* ==========================
   PERMISSIONS
========================== */}
<div className="mt-6">
  <span className="font-semibold text-lg">Permissions:</span>

  {userRoles && userRoles.length > 0 ? (
    <div className="flex flex-col gap-2 mt-2">
      {userRoles.map((role, roleIndex) =>
        role.permissions?.length > 0 ? (
          role.permissions.map((permission, permIndex) => (
            <div
              key={`${roleIndex}-${permIndex}`}
              className="border border-emerald-500 px-3 py-2 rounded-lg text-sm flex flex-col text-
white"
            >
              <span className="font-semibold">{permission.permission_name}</span>
              <span className="text-gray-200 text-xs">{permission.description}</span>
            </div>
          ))
        ) : null
      )}
    </div>
  ) : (
    <div className="text-gray-500 mt-2">No permissions assigned</div>
  )}
</div>

        </div>

        {/* ==========================
           SEARCH ORDERS
        ========================== */}
        <div className="bg-gray-800 p-6 rounded-lg shadow-md">
          <h3 className="text-2xl font-semibold mb-4">Search Orders</h3>
          <input
            type="text"
            placeholder="Search by Order ID, Status, or Date"
```

```
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            className="p-2 w-full bg-transparent border-2 border-emerald-400 text-emerald-400
rounded-lg focus:outline-none focus:border-emerald-500"
          />
        </div>

        {/* ========================
           ORDERS
        ========================== */}
        <div className="space-y-4">
          <h3 className="text-2xl font-semibold mb-4">Orders</h3>

          {displayedOrders.length > 0 ? (
            displayedOrders.map((order) => (
              <div key={order.id} className="bg-gray-800 p-6 rounded-lg shadow-md">
                <div><span className="font-semibold">Order ID:</span> {order.id}</div>
                <div><span className="font-semibold">Status:</span> {order.status}</div>
                <div><span className="font-semibold">Order Date:</span> {order.order_date}</div>
                <div><span className="font-semibold">Total Amount:</span> ${order.total_amount}
</div>
                <div><span className="font-semibold">Tracking Number:</span>
{order.tracking_number}</div>
              </div>
            ))
          ) : (
            <div>No orders found</div>
          )}
        </div>

        {/* ========================
           PAGINATION
        ========================== */}
        <div className="flex justify-between mt-4">
          <button
            onClick={() => setCurrentPage(prev => Math.max(prev - 1, 1))}
            disabled={currentPage === 1}
            className="bg-transparent border-2 border-white text-emerald-400 p-2 rounded-lg
hover:bg-emerald-400 hover:text-gray-900 transition"
          >
            Previous
          </button>

          <span>Page {currentPage} of {totalPages || 1}</span>

          <button
            onClick={() => setCurrentPage(prev => Math.min(prev + 1, totalPages))}
            disabled={currentPage === totalPages}
            className="bg-transparent border-2 border-white text-emerald-400 p-2 rounded-lg
hover:bg-emerald-400 hover:text-gray-900 transition"
          >
            Next
          </button>
        </div>

        {/* ========================
           INVOICES
        ========================== */}
        <div className="mt-8">
          <h3 className="text-2xl font-semibold mb-4">Invoices</h3>

          {userInvoices?.length > 0 ? (
            userInvoices.map((invoice) => (
              <div key={invoice.id} className="bg-gray-800 p-6 rounded-lg shadow-md mb-4">
                <div><span className="font-semibold">Invoice ID:</span> {invoice.id}</div>
                <div><span className="font-semibold">Total Amount:</span> ${invoice.total_amount}
```

```
</div>
                <div><span className="font-semibold">Payment Status:</span>
{invoice.payment_status}</div>
              </div>
            ))
          ) : (
            <div>No invoices found</div>
          )}
        </div>

      </div>
    </div>
  );
};

export default ProfilePage;
```

FILE: frontend/src/pages/PurchaseCancelPage.jsx

```
import { XCircle, ArrowLeft } from "lucide-react";
import { motion } from "framer-motion";
import { Link } from "react-router-dom";

const PurchaseCancelPage = () => {
    return (
        <div className='min-h-screen flex items-center justify-center px-4'>
            <motion.div
                initial={{ opacity: 0, y: 20 }}
                animate={{ opacity: 1, y: 0 }}
                transition={{ duration: 0.5 }}
                className='max-w-md w-full bg-gray-800 rounded-lg shadow-xl overflow-hidden
relative z-10'
            >
                <div className='p-6 sm:p-8'>
                    <div className='flex justify-center'>
                        <XCircle className='text-red-500 w-16 h-16 mb-4' />
                    </div>
                    <h1 className='text-2xl sm:text-3xl font-bold text-center text-red-500 mb-
2'>Purchase Cancelled</h1>
                    <p className='text-gray-300 text-center mb-6'>
                        Your order has been cancelled. No charges have been made.
                    </p>
                    <div className='bg-gray-700 rounded-lg p-4 mb-6'>
                        <p className='text-sm text-gray-400 text-center'>
                            If you encountered any issues during the checkout process, please
don&apos;t hesitate to
                            contact our support team.
                        </p>
                    </div>
                    <div className='space-y-4'>
                        <Link
                            to={"/"}
                            className='w-full bg-gray-700 hover:bg-gray-600 text-gray-300 font-bold
py-2 px-4 rounded-lg transition duration-300 flex items-center justify-center'
                        >
                            <ArrowLeft className='mr-2' size={18} />
                            Return to Shop
                        </Link>
                    </div>
                </div>
            </motion.div>
        </div>
    );
};

export default PurchaseCancelPage;
```

FILE: frontend/src/pages/PurchaseSuccessPage.jsx

```
import { useEffect, useState } from "react";
import { Link, useParams } from "react-router-dom";
import { useCartStore } from "../stores/useCartStore";
import { useOrderStore } from "../stores/useOrderStore";
import { useFeedbackFormStore } from "../stores/useFeedbackFormStore";
import Confetti from "react-confetti";
import FeedbackFormPublicView from "./FeedbackForm/FeedbackFormPublicView";
import { Receipt,NotebookPenIcon,Package,Truck,Home,Calendar,Factory,CheckCircle, XCircleIcon }
from "lucide-react";
import MapboxMap from "./MapBoxMap";
import useVehicleStore from "../stores/useVehicleStore";
import useMailStore from "../stores/useMailStore";
const PurchaseSuccessPage = () => {
  const [isProcessing, setIsProcessing] = useState(true);
  const [currentFormIndex, setCurrentFormIndex] = useState(0); // Track which form is displayed
  const [isDeliveryPreparing, setIsDeliveryPreparing] = useState(false);
  const [isMailSent, setIsMailSent] = useState(false);
  const { cart, orderId, clearCart, routeDetails, totalDistance, totalAmount, discountAmount,
totalAfterDiscount, prepareDelivery, userLocation, resetRouteDetails } = useCartStore();
  const { getFeedbackFormsForOrder, isLoadingForm, feedbackForms } = useFeedbackFormStore();
  const { orderStatus, getOrderStatusById, listenForOrderStatusUpdates, isLoading, errorMessage } =
useOrderStore(state => ({
    orderStatus: state.orderStatus,
    getOrderStatusById: state.getOrderStatusById,
    listenForOrderStatusUpdates: state.listenForOrderStatusUpdates,
    isLoading: state.isLoading,
    errorMessage: state.errorMessage
  }));
  const { sendMail, isProcessingMail, isErrorMail, errorMessageMail, isSuccessMail } =
useMailStore();


  const {routeDetailsWithCoordinates, getRouteDetailsByOrderId } = useVehicleStore();

  const { orderIdFromURL } = useParams();

  const statuses = ['Route Optimization Created', 'Paid', 'Pending', 'Confirmed', 'Packed',
'Delivery Maintenance Checked', 'On Delivery', 'Delivered', 'Canceled'];

  const orderData = {
    cart: cart,
    orderId: orderId,
    routeDetails: routeDetails,
    totalAmount: totalAmount,
    discountAmount: discountAmount,
    totalAfterDiscount: totalAfterDiscount,
    orderStatus: orderStatus,
    totalDistance: totalDistance, // Replace with actual value
  };
  useEffect(() => {
    // Prevent sending mail again if it's already been sent or there's an error
    if (
      orderId != null &&
      cart != null &&
      routeDetails != null &&
      totalAmount != null &&
      discountAmount != null &&
      totalAfterDiscount != null &&
      orderStatus != null &&
      totalDistance != null &&
      !isMailSent && errorMessageMail === "" && !isErrorMail
    ) {
      sendMail(orderData); // Call the sendMail function only if all values are non-null
      setIsMailSent(true); // Mark the mail as sent
```

```
    }
  }, [orderData, isMailSent, errorMessageMail, isErrorMail]); // Adding isMailSent and
errorMessageMail to dependencies ensures the effect only runs once when these change

  useEffect(() => {
    if (!userLocation) return;
    resetRouteDetails();
    console.log(userLocation);
    // Fetch route details when the component mounts or orderId changes
    setIsDeliveryPreparing(true);
    prepareDelivery(userLocation);
    setIsDeliveryPreparing(false);
  }, [userLocation]);

  useEffect(() => {
      if (!orderId || !routeDetails) return;
      // Fetch route details when the component mounts or orderId changes
      getRouteDetailsByOrderId(orderId);
    }, [orderId, routeDetails, getRouteDetailsByOrderId]);

  const getStatusClass = (status) => {
    const index = statuses.indexOf(status);
    const currentIndex = statuses.indexOf(orderStatus);
    return currentIndex >= index
      ? "bg-emerald-400 text-white"
      : "bg-gray-500 text-gray-300";
  };

  const getStatusIcon = (status) => {
    switch (status) {
      case 'Paid':
        return <Receipt size={24} />;
      case 'Pending':
        return <NotebookPenIcon size={24} />;
      case 'Confirmed':
        return <CheckCircle size={24} />;
      case 'Packed':
        return <Package size={24} />;
      case 'On Delivery':
        return <Truck size={24} />;
      case 'Delivered':
        return <Home size={24} />;
      case 'Delivery Scheduled':
        return <Calendar size={24} />;
      case 'Delivery Maintenance Checked':
        return <Factory size={24} />;
      case 'Canceled':
        return <XCircleIcon size={24} />;
      default:
        return <Factory size={24} />;
    }
  };

  useEffect(() => {
    const id = orderId || orderIdFromURL;

    if (id) {
      const fetchOrderStatus = async () => {
        try {
          await getOrderStatusById(id); // Fetch status using store
          listenForOrderStatusUpdates(id); // Start listening for real-time updates
        } catch (err) {
          console.error('Error fetching order status:', err);
        }
      };
```

```
      fetchOrderStatus();
    }

    // Simulate a delay or processing state if needed
    setTimeout(() => {
      setIsProcessing(false); // Once processing is done, hide loading state
    }, 2000);

    // Fetch the feedback forms
    const fetchFeedbackForms = async () => {
      try {
        await getFeedbackFormsForOrder(id);
      } catch (err) {
        console.error('Error fetching feedback forms:', err);
      }
    };

    if (id) {
      fetchFeedbackForms(); // Fetch the feedback forms for this order
    }
  }, [orderId, orderIdFromURL, getOrderStatusById, listenForOrderStatusUpdates]); // Re-run if
orderId or orderIdFromURL changes

  if (isProcessing || isLoading || isLoadingForm || isDeliveryPreparing) return <div>Processing...
</div>;

  if (errorMessage) return <div>{`Error: ${errorMessage}`}</div>;

  if (isProcessingMail) {
    return <div>Processing...</div>;
  }

  if (isErrorMail) {
    return <div>{`Error: ${errorMessageMail}`}</div>;
  }


  console.log(routeDetailsWithCoordinates);
  const handleContinueForm = () => {
    if (currentFormIndex < feedbackForms.length - 1) {
      setCurrentFormIndex(currentFormIndex + 1); // Skip to the next form
    } else {
      setCurrentFormIndex(feedbackForms.length); // End the form process
    }
  };

  const handleSkipAll = () => {
    setCurrentFormIndex(feedbackForms.length); // Skip to the end of all forms
  };

  const handleBackToForm = () => {
    setCurrentFormIndex(0); // Skip to the end of all forms
  };

  return (
    <div className="h-full flex items-center justify-center">
      <Confetti
        width={window.innerWidth}
        height={window.innerHeight}
        gravity={0.1}
        style={{ zIndex: 99 }}
        numberOfPieces={700}
        recycle={false}
      />
      <div className="w-full bg-gray-800 rounded-xl shadow-xl overflow-hidden relative z-10">
        <div className="p-8 sm:p-10">
```

```
          <div className="flex justify-center">
            <CheckCircle className="text-emerald-400 w-20 h-20 mb-6 animate-bounce" />
          </div>

          <h1 className="text-4xl sm:text-5xl font-extrabold text-center text-emerald-400 mb-
4">Purchase Successful!</h1>
          <p className="text-gray-300 text-center mb-4">Thank you for your order. We're processing
it now.</p>
          <p className="text-emerald-400 text-center text-lg mb-8">Check your email for order
details and updates.</p>

          {/* Loop through feedback forms */}
          {console.log(feedbackForms)}
          {feedbackForms && (
            <div className="mt-6">
              <h4 className="text-xl font-semibold mb-4">Feedback Forms</h4>
              <div className="bg-gray-700 p-6 rounded-lg shadow-lg">
                {/* Display the current feedback form */}
                {currentFormIndex <= feedbackForms.length - 1 && (<FeedbackFormPublicView
currentFeedbackForm={feedbackForms[currentFormIndex]} continueIndex={setCurrentFormIndex}/>)}

                {/* Skip button */}

                {currentFormIndex <= feedbackForms.length - 1 && (<div className="mt-4">
                  <button
                    onClick={handleContinueForm}
                    className="w-full bg-gray-600 hover:bg-gray-500 text-white font-bold py-3 px-6
rounded-lg transition duration-300 flex items-center justify-center"
                  >
                    Continue
                  </button>
                </div>)}

                {/* Skip All Button */}
                {currentFormIndex <= feedbackForms.length - 1 && (<div className="mt-4">
                  <button
                    onClick={handleSkipAll}
                    className="w-full bg-emerald-400 hover:bg-emerald-600 text-white font-bold py-3
px-6 rounded-lg transition duration-300 flex items-center justify-center"
                  >
                    Skip All Forms
                  </button>
                </div>)}

                {currentFormIndex == feedbackForms.length && (<div className="mt-4">
                  <button
                    onClick={handleBackToForm}
                    className="w-full bg-emerald-400 hover:bg-emerald-600 text-white font-bold py-3
px-6 rounded-lg transition duration-300 flex items-center justify-center"
                  >
                    Back To Forms
                  </button>
                </div>)}


              </div>
            </div>
          )}

          {/* Route Details */}
          {routeDetails.length > 0 && routeDetails  && (
            <div className="mt-6">
              <h4 className="text-xl font-semibold">Expected Delivery Routes</h4>
              <ul className="space-y-4 text-white mt-4">
                {routeDetails.map((route, index) => (
                  <li key={index} className="bg-gray-700 p-3 rounded-md">
```

```
                    <p><strong>Route Name:</strong> {route.route_name}</p>
                    {route.supplier_name && (<p><strong>Supplier:</strong> {route.supplier_name}
</p>)}
                    <p><strong>Warehouse:</strong> {route.warehouse_name_1}</p>
                    {route.warehouse_name_2 && (<p><strong>Warehouse:</strong>
{route.warehouse_name_2}</p>)}
                    <p><strong>Start Location</strong> {route.start_location}</p>
                    <p><strong>Destination Location</strong> {route.end_location}</p>
                    <p><strong>Estimated Time</strong> {route.estimated_time}</p>
                    <p><strong>Distance:</strong> {route.distance} km</p>
                  </li>
                ))}
              </ul>
              <div className="mt-4">
                <p><strong>Total Distance:</strong> {totalDistance} km</p>
              </div>
            </div>
          )}

          {/* Order Timeline */}
          <div className="flex items-center justify-center space-x-8 mb-8 flex-wrap">
            {statuses.map((status, index) => (
              <div key={status} className="flex items-center justify-center space-x-2 mt-8">
                <div
                  className={`w-12 h-12 rounded-full flex items-center justify-center transition-
all duration-300
                    ${getStatusClass(status)} shadow-lg transform hover:scale-110`}
                >
                  {getStatusIcon(status)}
                </div>

                {/* Line between status circles - only show if orderStatus has reached or passed
the current status */}
                {index < statuses.length - 1 && statuses.indexOf(orderStatus) >= index && (
                  <div className="w-16 h-1 bg-emerald-400"></div>
                )}

                {index < statuses.length - 1 && statuses.indexOf(orderStatus) >= index &&
(orderStatus == 'Canceled') && (
                  <div className="w-16 h-1 bg-red-600"></div>
                )}

                {(index >= statuses.length - 1 || statuses.indexOf(orderStatus) < index) && (
                  <div className="w-16 h-1 bg-white"></div>
                )}

                {/* Status Text */}
                <span className="text-sm text-center text-gray-200">
                  {status}
                </span>
              </div>
            ))}
          </div>
          <div className="p-6 bg-gray-700 rounded-md shadow-md my-4">
              <p className="text-xl font-semibold">Total Amount: ${totalAmount.toFixed(2)}</p>
              {discountAmount > 0 && (
                <>
                  <p className="text-lg">Discount Applied: -${discountAmount.toFixed(2)}</p>
                  <p className="text-lg font-semibold">Total After Discount:
${totalAfterDiscount.toFixed(2)}</p>
                </>
              )}
          </div>
          {/* Order Details */}
          <div className="bg-gray-700 rounded-lg p-6 mb-8 shadow-lg">
            <h2 className="text-2xl font-semibold text-emerald-400 mb-4">Order Details</h2>
```

```
            <div className="space-y-6">
              {cart.map((item) => (
                <div
                  key={item.products_id}
                  className="flex justify-between items-center bg-gray-800 p-4 rounded-md shadow-
md"
                >
                  <div className="flex items-center">
                    <img
                      src={item.product.image}
                      alt={item.product.name}
                      className="w-20 h-20 object-cover mr-6 rounded-md shadow-md"
                    />
                    <span className="text-xl text-gray-200">{item.product.name}</span>
                  </div>
                  <div className="text-xl text-gray-300">
                    {item.quantity} x ${item.total_amount.toFixed(2)}
                  </div>
                </div>
              ))}
            </div>
          </div>
          {routeDetailsWithCoordinates?.length > 0 && <MapboxMap routeDetails=
{routeDetailsWithCoordinates} />}
          {/* Action Buttons */}
          <div className="space-y-6">
            <button
              onClick={() => clearCart()}
              className="w-full bg-emerald-600 hover:bg-emerald-700 text-white font-bold py-3 px-6
rounded-lg transition duration-300 flex items-center justify-center transform hover:scale-105"
            >
              Thanks for trusting us!
            </button>

            <Link
              to="/"
              className="w-full bg-gray-700 hover:bg-gray-600 text-emerald-400 font-bold py-3 px-6
rounded-lg transition duration-300 flex items-center justify-center transform hover:scale-105"
            >
              Continue Shopping
            </Link>
          </div>
        </div>
      </div>
    </div>
  );
};

export default PurchaseSuccessPage;
```

FILE: frontend/src/pages/SignUpPage.jsx

```
import { useState } from "react";
import { Link } from "react-router-dom";
import { UserPlus, Mail, Lock, User, ArrowRight, Loader, Phone } from "lucide-react";
import { motion } from "framer-motion";
import { useUserStore } from "../stores/useUserStore";

const SignUpPage = () => {
  const [formData, setFormData] = useState({
    first_name: "",
    last_name: "",
    phone_number: "",
    email: "",
    password: "",
    confirmPassword: "",
```

```
    language: "en",  // Default language
    two_factor_enabled: false,
  });

  const { signup, loading } = useUserStore();

  const handleSubmit = (e) => {
    e.preventDefault();

    // Basic password confirmation validation
    if (formData.password !== formData.confirmPassword) {
      alert("Passwords do not match!");
      return;
    }

    // Submit the form data to the signup function
    signup(formData);
  };

  return (
    <div className="flex flex-col justify-center py-12 sm:px-6 lg:px-8">
      <motion.div
        className="sm:mx-auto sm:w-full sm:max-w-md"
        initial={{ opacity: 0, y: -20 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.8 }}
      >
        <h2 className="mt-6 text-center text-3xl font-extrabold text-emerald-400">
          Create your account
        </h2>
      </motion.div>

      <motion.div
        className="mt-8 sm:mx-auto sm:w-full sm:max-w-md"
        initial={{ opacity: 0, y: 20 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.8, delay: 0.2 }}
      >
        <div className="bg-gray-800 py-8 px-4 shadow sm:rounded-lg sm:px-10">
          <form onSubmit={handleSubmit} className="space-y-6">
            {/* Full Name Fields */}
            <div className="flex space-x-4">
              <div className="w-1/2">
                <label htmlFor="first_name" className="block text-sm font-medium text-gray-300">
                  First name
                </label>
                <input
                  id="first_name"
                  type="text"
                  required
                  value={formData.first_name}
                  onChange={(e) => setFormData({ ...formData, first_name: e.target.value })}
                  className="block w-full px-3 py-2 bg-gray-700 border border-gray-600 rounded-md
shadow-sm placeholder-gray-400 focus:outline-none focus:ring-emerald-500 focus:border-emerald-500
sm:text-sm"
                  placeholder="John"
                />
              </div>
              <div className="w-1/2">
                <label htmlFor="last_name" className="block text-sm font-medium text-gray-300">
                  Last name
                </label>
                <input
                  id="last_name"
                  type="text"
                  required
```

```
                  value={formData.last_name}
                  onChange={(e) => setFormData({ ...formData, last_name: e.target.value })}
                  className="block w-full px-3 py-2 bg-gray-700 border border-gray-600 rounded-md
shadow-sm placeholder-gray-400 focus:outline-none focus:ring-emerald-500 focus:border-emerald-500
sm:text-sm"
                  placeholder="Doe"
                />
              </div>
            </div>

            {/* Phone Number Field */}
            <div>
              <label htmlFor="phone_number" className="block text-sm font-medium text-gray-300">
                Phone number
              </label>
              <div className="mt-1 relative rounded-md shadow-sm">
                <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-
none">
                  <Phone className="h-5 w-5 text-gray-400" aria-hidden="true" />
                </div>
                <input
                  id="phone_number"
                  type="tel"
                  required
                  value={formData.phone_number}
                  onChange={(e) => setFormData({ ...formData, phone_number: e.target.value })}
                  className="block w-full px-3 py-2 pl-10 bg-gray-700 border border-gray-600
rounded-md shadow-sm placeholder-gray-400 focus:outline-none focus:ring-emerald-500 focus:border-
emerald-500 sm:text-sm"
                  placeholder="123-456-7890"
                />
              </div>
            </div>

            {/* Email Field */}
            <div>
              <label htmlFor="email" className="block text-sm font-medium text-gray-300">
                Email address
              </label>
              <div className="mt-1 relative rounded-md shadow-sm">
                <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-
none">
                  <Mail className="h-5 w-5 text-gray-400" aria-hidden="true" />
                </div>
                <input
                  id="email"
                  type="email"
                  required
                  value={formData.email}
                  onChange={(e) => setFormData({ ...formData, email: e.target.value })}
                  className="block w-full px-3 py-2 pl-10 bg-gray-700 border border-gray-600
rounded-md shadow-sm placeholder-gray-400 focus:outline-none focus:ring-emerald-500 focus:border-
emerald-500 sm:text-sm"
                  placeholder="you@example.com"
                />
              </div>
            </div>

            {/* Password Field */}
            <div>
              <label htmlFor="password" className="block text-sm font-medium text-gray-300">
                Password
              </label>
              <div className="mt-1 relative rounded-md shadow-sm">
                <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-
none">
```

```
                <Lock className="h-5 w-5 text-gray-400" aria-hidden="true" />
              </div>
              <input
                id="password"
                type="password"
                required
                value={formData.password}
                onChange={(e) => setFormData({ ...formData, password: e.target.value })}
                className="block w-full px-3 py-2 pl-10 bg-gray-700 border border-gray-600
rounded-md shadow-sm placeholder-gray-400 focus:outline-none focus:ring-emerald-500 focus:border-
emerald-500 sm:text-sm"
                placeholder="••••••••"
              />
            </div>
          </div>

          {/* Confirm Password Field */}
          <div>
            <label htmlFor="confirmPassword" className="block text-sm font-medium text-gray-300">
              Confirm Password
            </label>
            <div className="mt-1 relative rounded-md shadow-sm">
              <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-
none">
                <Lock className="h-5 w-5 text-gray-400" aria-hidden="true" />
              </div>
              <input
                id="confirmPassword"
                type="password"
                required
                value={formData.confirmPassword}
                onChange={(e) => setFormData({ ...formData, confirmPassword: e.target.value })}
                className="block w-full px-3 py-2 pl-10 bg-gray-700 border border-gray-600
rounded-md shadow-sm placeholder-gray-400 focus:outline-none focus:ring-emerald-500 focus:border-
emerald-500 sm:text-sm"
                placeholder="••••••••"
              />
            </div>
          </div>

          {/* Language Selection */}
          <div>
            <label htmlFor="language" className="block text-sm font-medium text-gray-300">
              Preferred Language
            </label>
            <select
              id="language"
              value={formData.language}
              onChange={(e) => setFormData({ ...formData, language: e.target.value })}
              className="block w-full px-3 py-2 bg-gray-700 border border-gray-600 rounded-md
shadow-sm text-gray-400 focus:outline-none focus:ring-emerald-500 focus:border-emerald-500 sm:text-
sm"
            >
              <option value="en">English</option>
              <option value="es">Spanish</option>
              <option value="fr">French</option>
              {/* Add more languages as needed */}
            </select>
          </div>

          {/* Submit Button */}
          <button
            type="submit"
            className="w-full flex justify-center py-2 px-4 border border-transparent rounded-md
shadow-sm text-sm font-medium text-white bg-emerald-600 hover:bg-emerald-700 focus:outline-none
focus:ring-2 focus:ring-offset-2 focus:ring-emerald-500 transition duration-150 ease-in-out
```

```
disabled:opacity-50"
              disabled={loading}
            >
              {loading ? (
                <>
                  <Loader className="mr-2 h-5 w-5 animate-spin" aria-hidden="true" />
                  Loading...
                </>
              ) : (
                <>
                  <UserPlus className="mr-2 h-5 w-5" aria-hidden="true" />
                  Sign up
                </>
              )}
            </button>
          </form>

          <p className="mt-8 text-center text-sm text-gray-400">
            Already have an account?{" "}
            <Link to="/login" className="font-medium text-emerald-400 hover:text-emerald-300">
              Login here <ArrowRight className="inline h-4 w-4" />
            </Link>
          </p>
        </div>
      </motion.div>
    </div>
  );
};

export default SignUpPage;
```

FILE: frontend/src/pages/SocialMedia/FriendRequest.jsx

```
import useSocialMediaStore from "../../stores/useSocialMediaStore";

const FriendRequest = ({ request }) => {
  const { acceptConnectionRequest, rejectConnectionRequest } = useSocialMediaStore();

  const handleAccept = () => {
    acceptConnectionRequest(request.id);
  };

  const handleReject = () => {
    rejectConnectionRequest(request.id);
  };

  return (
    <div className="border-2 border-white rounded-lg p-4 flex items-center justify-between
transition-all hover:shadow-md text-emerald-400">
      <div className="flex items-center gap-4">
        <div>
          <div className="font-semibold text-lg">
            {`${request.sender.first_name} ${request.sender.last_name}`}
          </div>
          <p className="text-emerald-400">{request.sender.headline}</p>
        </div>
      </div>

      <div className="space-x-2">
        <button
          className="border-2 border-white text-emerald-400 px-4 py-2 rounded-md hover:bg-emerald-
400 hover:text-white transition-colors"
          onClick={handleAccept}
        >
          Accept
        </button>
```

```
        <button
          className="border-2 border-white text-emerald-400 px-4 py-2 rounded-md hover:bg-emerald-
400 hover:text-white transition-colors"
          onClick={handleReject}
        >
          Reject
        </button>
      </div>
    </div>
  );
};

export default FriendRequest;
```

FILE: frontend/src/pages/SocialMedia/NetworkPage.jsx

```
import { useEffect } from "react";
import { UserPlus } from "lucide-react";
import FriendRequest from "./FriendRequest";
import UserCard from "./UserCard";
import useSocialMediaStore from "../../stores/useSocialMediaStore";

const NetworkPage = () => {
  const { connectionRequests, connections, fetchConnectionRequests, fetchConnections } =
useSocialMediaStore();

  useEffect(() => {
    fetchConnectionRequests();
    fetchConnections();
  }, [fetchConnectionRequests, fetchConnections]);

  return (
    <div className="w-screen min-h-screen flex justify-center">
      <div className="w-full max-w-screen-xl px-6">
        <div className="border-2 border-white rounded-lg p-6 mb-6 text-emerald-400">
          <h1 className="text-2xl font-bold mb-6">My Network</h1>

          {connectionRequests?.length > 0 ? (
            <div className="mb-8">
              <h2 className="text-xl font-semibold mb-2">Connection Request</h2>
              <div className="space-y-4">
                {connectionRequests.map((request) => (
                  <FriendRequest key={request.id} request={request} />
                ))}
              </div>
            </div>
          ) : (
            <div className="border-2 border-white rounded-lg p-6 text-center mb-6">
              <UserPlus size={48} className="mx-auto text-emerald-400 mb-4" />
              <h3 className="text-xl font-semibold mb-2 text-emerald-400">No Connection
Requests</h3>
              <p className="text-emerald-400">
                You don&apos;t have any pending connection requests at the moment.
              </p>
              <p className="text-emerald-400 mt-2">
                Explore suggested connections below to expand your network!
              </p>
            </div>
          )}

          {connections?.length > 0 && (
            <div className="mb-8">
              <h2 className="text-xl font-semibold mb-4 text-emerald-400">My Connections</h2>
              <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
                {connections.map((connection) => (
                  <UserCard key={connection.id} user={connection} isConnection={true} />
```

```
                ))}
              </div>
            </div>
          )}
        </div>
      </div>
    </div>
  );
};

export default NetworkPage;
```

FILE: frontend/src/pages/SocialMedia/NotificationsPage.jsx

```
import { useEffect } from "react";
import { ExternalLink, Eye, MessageSquare, ThumbsUp, Trash2, UserPlus } from "lucide-react";
import { formatDistanceToNow } from "date-fns";
import useSocialMediaStore from "../../stores/useSocialMediaStore";

const NotificationsPage = () => {
  const { notifications, isLoadingNotifications, fetchNotifications, markAsRead, deleteNotification
} = useSocialMediaStore();

  useEffect(() => {
    fetchNotifications();
  }, [fetchNotifications]);

  const renderNotificationIcon = (type) => {
    switch (type) {
      case "like":
        return <ThumbsUp className="text-emerald-400" />;
      case "comment":
        return <MessageSquare className="text-emerald-400" />;
      case "connectionAccepted":
        return <UserPlus className="text-emerald-400" />;
      default:
        return null;
    }
  };

  const renderNotificationContent = (notification) => {
    switch (notification.type) {
      case "like":
        return (
          <span>
            <strong>{notification.related_user.first_name + " " +
notification.related_user.last_name}</strong> liked your post
          </span>
        );
      case "comment":
        return (
          <span>
            {notification.related_user.first_name + " " + notification.related_user.last_name}
commented on your post
          </span>
        );
      case "connectionAccepted":
        return (
          <span>
            {notification.related_user.first_name + " " + notification.related_user.last_name}
accepted your connection request
          </span>
        );
      default:
        return null;
    }
```

```
  };

  const renderRelatedPost = (relatedPost) => {
    if (!relatedPost) return null;

    return (
      <>
        {relatedPost.image && (
          <img src={relatedPost.image} alt="Post preview" className="w-10 h-10 object-cover
rounded" />
        )}
        <div className="flex-1 overflow-hidden">
          <p className="text-sm text-emerald-400 truncate">{relatedPost.content}</p>
        </div>
        <ExternalLink size={14} className="text-emerald-400" />
      </>
    );
  };

  return (
    <div className="w-screen min-h-screen flex justify-center">
      <div className="w-full max-w-screen-xl px-6">
        <div className="border-2 border-white rounded-lg p-6 text-emerald-400">
          <h1 className="text-2xl font-bold mb-6">Notifications</h1>

          {isLoadingNotifications ? (
            <p>Loading notifications...</p>
          ) : notifications && notifications.length > 0 ? (
            <ul>
              {notifications.map((notification) => (
                <li
                  key={notification.id}
                  className={`bg-transparent border-2 border-white rounded-lg p-4 my-4 transition-
all hover:shadow-md ${
                    !notification.read ? "border-emerald-400" : "border-gray-200"
                  }`}
                >
                  <div className="flex items-start justify-between">
                    <div className="flex items-center space-x-4">
                      <img
                        src={notification.related_user.profile_picture || "/avatar.png"}
                        className="w-12 h-12 rounded-full object-cover"
                      />

                      <div>
                        <div className="flex items-center gap-2">
                          <div className="p-1 bg-gray-100 rounded-full">
                            {renderNotificationIcon(notification.type)}
                          </div>
                          <p className="text-sm text-emerald-400">
{renderNotificationContent(notification)}</p>
                        </div>
                        <p className="text-xs text-emerald-400 mt-1">
                          {formatDistanceToNow(new Date(notification.created_at), {
                            addSuffix: true,
                          })}
                        </p>
                        {renderRelatedPost(notification.relatedPost)}
                      </div>
                    </div>

                    <div className="flex gap-2">
                      {!notification.read && (
                        <button
                          onClick={() => markAsRead(notification.id)}
                          className="p-1 bg-blue-100 text-blue-600 rounded hover:bg-blue-200
```

```
transition-colors"
                          aria-label="Mark as read"
                        >
                          <Eye size={16} />
                        </button>
                      )}

                      <button
                        onClick={() => deleteNotification(notification.id)}
                        className="p-1 bg-red-100 text-red-600 rounded hover:bg-red-200 transition-
colors"
                        aria-label="Delete notification"
                      >
                        <Trash2 size={16} />
                      </button>
                    </div>
                  </div>
                </li>
              ))}
            </ul>
          ) : (
            <p>No notifications at the moment.</p>
          )}
        </div>
      </div>
    </div>
  );
};

export default NotificationsPage;
```

FILE: frontend/src/pages/SocialMedia/Post.jsx

```
import { useState } from "react";
import { Loader, MessageCircle, Send, Share2, ThumbsUp, Trash2 } from "lucide-react";
import { formatDistanceToNow } from "date-fns";
import PostAction from "./PostAction";
import useSocialMediaStore from "../../stores/useSocialMediaStore";
import { useUserStore } from "../../stores/useUserStore";

const Post = ({ post }) => {
  const { user } = useUserStore();
  const { likePost, createComment, deletePost } = useSocialMediaStore();

  const [showComments, setShowComments] = useState(false);
  const [newComment, setNewComment] = useState("");
  const [comments, setComments] = useState(post.comments || []);
  const isOwner = user?.id === post.author.id;
  const isLiked = post.likes.includes(user?.id);

  const handleDeletePost = () => {
    if (!window.confirm("Are you sure you want to delete this post?")) return;
    deletePost(post.id);
  };

  const handleLikePost = () => {
    if (!isLiked) {
      likePost(post.id, user.id);
    }
  };

  const handleAddComment = async (e) => {
    e.preventDefault();
    if (newComment.trim()) {
      await createComment(post.id, newComment, user);
      setNewComment("");
```

```
      setComments([
        ...comments,
        {
          content: newComment,
          user: {
            id: user.id,
            first_name: user.first_name, // Keep first_name separate
            last_name: user.last_name,   // Keep last_name separate
            profile_picture: user.profile_picture,
            name: `${user.first_name} ${user.last_name}`, // Concatenate name field
          },
          created_at: new Date(),
        },
      ]);
    }
  };

  return (
    <div className="border-2 border-white rounded-lg mb-4 text-emerald-400">
      <div className="p-4">
        <div className="flex items-center justify-between mb-4">
          <div className="flex items-center">
            <div>
              <h3 className="font-semibold text-emerald-400">{`${post.author.first_name}
${post.author.last_name}`}</h3>
              <p className="text-xs text-emerald-400">
                {isNaN(new Date(post.created_at).getTime())
                  ? "Error: Invalid date"
                  : formatDistanceToNow(new Date(post.created_at), { addSuffix: true })}
              </p>
            </div>
          </div>
          {isOwner && (
            <button onClick={handleDeletePost} className="text-red-500 hover:text-red-700">
              <Trash2 size={18} />
            </button>
          )}
        </div>
        {console.log(post.image)}
        <p className="mb-4 text-emerald-400">{post.content}</p>
        {post.image && <img  src={post.image} alt="Post content" className="rounded-lg w-full mb-4"
/>}

        <div className="flex justify-between text-emerald-400">
          <PostAction
            icon={<ThumbsUp size={18} className={isLiked ? "text-blue-500 fill-blue-300" : ""} />}
            text={`Like (${post.likes.length})`}
            onClick={handleLikePost}
          />
          <PostAction
            icon={<MessageCircle size={18} />}
            text={`Comment (${comments.length})`}
            onClick={() => setShowComments(!showComments)}
          />
          <PostAction icon={<Share2 size={18} />} text="Share" />
        </div>
      </div>

      {showComments && (
        <div className="px-4 pb-4">
          <div className="mb-4 max-h-60 overflow-y-auto">
            {comments.map((comment) => (
              <div key={comment.id} className="mb-2 bg-transparent p-2 rounded flex items-start">
                <div className="flex-grow">
                  <div className="flex items-center mb-1">
                    <span className="font-semibold text-emerald-400 mr-2">
```

```
                      {`${comment.user.first_name} ${comment.user.last_name}`} {/* Concatenate
first_name and last_name */}
                    </span>
                    <span className="text-xs text-emerald-400">
                      {isNaN(new Date(comment.created_at).getTime())
                        ? "Error: Invalid date"
                        : formatDistanceToNow(new Date(comment.created_at), { addSuffix: true })}
                    </span>
                  </div>
                  <p className="text-emerald-400">{comment.content}</p>
                </div>
              </div>
            ))}
          </div>

          <form onSubmit={handleAddComment} className="flex items-center">
            <input
              type="text"
              value={newComment}
              onChange={(e) => setNewComment(e.target.value)}
              placeholder="Add a comment..."
              className="flex-grow p-2 rounded-l-full bg-transparent focus:outline-none focus:ring-
2 focus:ring-primary text-emerald-400"
            />
            <button
              type="submit"
              className="border-2 border-white text-emerald-400 p-2 rounded-r-full hover:bg-
emerald-400 hover:text-white transition duration-300"
              disabled={false}
            >
              {false ? <Loader size={18} className="animate-spin" /> : <Send size={18} />}
            </button>
          </form>
        </div>
      )}
    </div>
  );
};

export default Post;
```

FILE: frontend/src/pages/SocialMedia/PostAction.jsx

```
export default function PostAction({ icon, text, onClick }) {
    return (
        <button className='flex items-center' onClick={onClick}>
            <span className='mr-1'>{icon}</span>
            <span className='hidden sm:inline'>{text}</span>
        </button>
    );
}
```

FILE: frontend/src/pages/SocialMedia/PostCreation.jsx

```
import { useState } from "react";
import { Image, Loader } from "lucide-react";
import useSocialMediaStore from "../../stores/useSocialMediaStore";

const PostCreation = ({ user }) => {
  const [content, setContent] = useState("");
  const [image, setImage] = useState(null);
  const [imagePreview, setImagePreview] = useState(null);

  // Access the createPost method from the store
  const { createPost, isLoadingPost, isErrorPost, errorMessagePost } = useSocialMediaStore((state)
```

```
=> state);

  const handlePostCreation = async () => {
    try {
      const postData = new FormData();
      postData.append("content", content);
      if (image) {
        postData.append("image", image);
      }

      // Call the createPost action from the store
      await createPost(postData);
      resetForm();
    } catch (error) {
      console.error("Error in handlePostCreation:", error);
    }
  };

  const resetForm = () => {
    setContent("");
    setImage(null);
    setImagePreview(null);
  };

  const handleImageChange = (e) => {
    const file = e.target.files[0];
    setImage(file);
    if (file) {
      readFileAsDataURL(file).then(setImagePreview);
    } else {
      setImagePreview(null);
    }
  };

  const readFileAsDataURL = (file) => {
    return new Promise((resolve, reject) => {
      const reader = new FileReader();
      reader.onloadend = () => resolve(reader.result);
      reader.onerror = reject;
      reader.readAsDataURL(file);
    });
  };

  return (
    <div className="border-2 border-white rounded-lg p-4 text-emerald-400">
      <div className="flex space-x-3 items-center">
        <div>{`${user.first_name} ${user.last_name}`}</div>
        <textarea
          placeholder="What's on your mind?"
          className="w-full p-3 rounded-lg bg-transparent hover:bg-transparent focus:bg-transparent
focus:outline-none resize-none transition-colors duration-200 min-h-[100px] text-emerald-400"
          value={content}
          onChange={(e) => setContent(e.target.value)}
        />
      </div>

      {imagePreview && (
        <div className="mt-4">
          <img src={imagePreview} alt="Selected" className="w-full h-auto rounded-lg" />
        </div>
      )}

      {isErrorPost && errorMessagePost && (
        <div className="text-red-500 mt-2">{errorMessagePost}</div>
      )}
```

```
      <div className="flex justify-between items-center mt-4">
        <div className="flex space-x-4">
          <label className="flex items-center text-emerald-400 hover:text-emerald-500 transition-
colors duration-200 cursor-pointer">
            <Image size={20} className="mr-2" />
            <span>Photo</span>
            <input
              type="file"
              accept="image/*"
              className="hidden"
              onChange={handleImageChange}
            />
          </label>
        </div>

        <button
          className="bg-primary text-white rounded-lg px-4 py-2 hover:bg-primary-dark transition-
colors duration-200"
          onClick={handlePostCreation}
          disabled={isLoadingPost}
        >
          {isLoadingPost ? <Loader className="size-5 animate-spin" /> : "Share"}
        </button>
      </div>
    </div>
  );
};

export default PostCreation;
```

FILE: frontend/src/pages/SocialMedia/RecommendedUser.jsx

```
import { useEffect } from "react";
import useSocialMediaStore from "../../stores/useSocialMediaStore";
import { Check, Clock, UserCheck, UserPlus, X } from "lucide-react";

const RecommendedUser = ({ user }) => {
  const {
    connectionStatuses,
    isLoadingConnectionStatus,
    fetchConnectionStatus,
    sendConnectionRequest,
    acceptConnectionRequestForRecommendedUser,
    rejectConnectionRequestForRecommendedUser
  } = useSocialMediaStore();

  // Fetch connection status when the component mounts
  useEffect(() => {
    // Fetch connection status for this user only when the user ID changes
    fetchConnectionStatus(user.id);
  }, [user.id, fetchConnectionStatus]);

  const renderButton = () => {
    // Use get() to ensure `connectionStatuses` exists and get the status
    const status = connectionStatuses.find(conn => conn.userId === user.id); // Find the status for
this user
    console.log(status);
    if (isLoadingConnectionStatus) {
      return (
        <button className="px-3 py-1 rounded-full text-sm border-2 border-white text-emerald-400"
disabled>
          Loading...
        </button>
      );
    }
```

```
    if (!status) {
      return (
        <button
          className="px-3 py-1 rounded-full text-sm border-2 border-white text-emerald-400
hover:bg-emerald-400 hover:text-white transition-colors duration-200"
          onClick={() => sendConnectionRequest(user.id)}
        >
          <UserPlus size={16} className="mr-1 text-emerald-400" />
          Connect
        </button>
      );
    }

    switch (status.status.status) {
      case "pending":
        return (
          <button className="px-3 py-1 rounded-full text-sm border-2 border-white text-emerald-400
flex items-center" disabled>
            <Clock size={16} className="mr-1 text-emerald-400" />
            Pending
          </button>
        );
      case "received":
        return (
          <div className="flex gap-2">
            <button
              onClick={() => acceptConnectionRequestForRecommendedUser(status.status.requestId)}
              className="rounded-full p-1 flex items-center justify-center border-2 border-white
text-emerald-400 hover:bg-emerald-400 hover:text-white transition-colors"
            >
              <Check size={16} />
            </button>
            <button
              onClick={() => rejectConnectionRequestForRecommendedUser(status.status.requestId)}
              className="rounded-full p-1 flex items-center justify-center border-2 border-white
text-emerald-400 hover:bg-red-500 hover:text-white transition-colors"
            >
              <X size={16} />
            </button>
          </div>
        );
      case "connected":
        return (
          <button className="px-3 py-1 rounded-full text-sm border-2 border-white text-emerald-400
flex items-center" disabled>
            <UserCheck size={16} className="mr-1 text-emerald-400" />
            Connected
          </button>
        );
      default:
        return (
          <button
            className="px-3 py-1 rounded-full text-sm border-2 border-white text-emerald-400
hover:bg-emerald-400 hover:text-white transition-colors duration-200"
            onClick={() => sendConnectionRequest(user.id)}
          >
            <UserPlus size={16} className="mr-1 text-emerald-400" />
            Connect
          </button>
        );
    }
  };

  return (
    <div className="flex items-center justify-between mb-4 border-2 border-white rounded-lg p-4
text-emerald-400">
```

```
      <div className="flex items-center">
        <img src={user.profile_picture || "/avatar.png"} className="w-12 h-12 rounded-full mr-3" />
        <div className="text-start">
          <h3 className="font-semibold text-sm">{user.first_name + " " + user.last_name}</h3>
          <p className="text-xs">{user.headline}</p>
        </div>
      </div>
      <div className="ml-4">
        {renderButton()}
      </div>
    </div>
  );
};

export default RecommendedUser;
```

FILE: frontend/src/pages/SocialMedia/SocialMediaPage.jsx

```
import { useEffect, useState } from "react";
import useSocialMediaStore from "../../stores/useSocialMediaStore";
import PostCreation from "./PostCreation";
import Post from "./Post";
import { Users } from "lucide-react";
import RecommendedUser from "./RecommendedUser";
import { useUserStore } from "../../stores/useUserStore";

const SocialMediaPage = () => {
  const { user } = useUserStore();
  const {
    recommendedUsers,
    posts,
    isLoadingPosts,
    isLoadingRecommendedUsers,
    fetchPosts,
    fetchRecommendedUsers,
    isErrorPosts,
    isErrorRecommendedUsers,
  } = useSocialMediaStore();

  const [searchQuery, setSearchQuery] = useState("");
  const [filteredUsers, setFilteredUsers] = useState(recommendedUsers);

  useEffect(() => {
    fetchRecommendedUsers(); // Fetch recommended users data
    fetchPosts(); // Fetch posts
  }, [fetchRecommendedUsers, fetchPosts]);

  // Filter the users when search query changes
  useEffect(() => {
    if (searchQuery) {
      const filtered = recommendedUsers.filter((user) => {
        const fullName = `${user.first_name} ${user.last_name}`.toLowerCase();
        return fullName.includes(searchQuery.toLowerCase());
      });
      setFilteredUsers(filtered);
    } else {
      setFilteredUsers(recommendedUsers);
    }
  }, [searchQuery, recommendedUsers]);

  // Handle loading states
  if (isLoadingRecommendedUsers || isLoadingPosts) {
    return <div className="text-emerald-400">Loading...</div>;
  }

  // Handle error states
```

```
  if (isErrorRecommendedUsers || isErrorPosts) {
    return <div className="text-emerald-400">Error loading data. Please try again later.</div>;
  }

  return (
    <div className="grid grid-cols-1 lg:grid-cols-4 gap-6 h-full p-4">
      <div className="col-span-1 lg:col-span-2 order-first lg:order-none border-2 border-white
rounded-lg text-emerald-400">
        <PostCreation user={user} />

        <div
          style={{
            maxHeight: '470px',
            overflowY: 'auto',
            scrollbarWidth: 'none',
            msOverflowStyle: 'none',
          }}
          className="posts-container mt-4"
        >
          {posts?.map((post) => (
            <Post key={post.id} post={post} />
          ))}
          <style jsx>{`
            .posts-container::-webkit-scrollbar {
              display: none;
            }
          `}</style>
        </div>

        {posts?.length === 0 && (
          <div className="border-2 border-white rounded-lg shadow p-8 text-emerald-400 text-
center">
            <div className="mb-6">
              <Users size={64} className="mx-auto text-emerald-400" />
            </div>
            <h2 className="text-2xl font-bold mb-4 text-emerald-400">No Posts Yet</h2>
          </div>
        )}
      </div>

      {recommendedUsers?.length > 0 && (
        <div className="col-span-1 lg:col-span-2 hidden lg:block ">
          <div className="border-2 border-white rounded-lg p-4 text-emerald-400">
            <h2 className="font-semibold mb-4 text-emerald-400">People you may know</h2>

            {/* Search Bar */}
            <input
              type="text"
              placeholder="Search by name..."
              className="w-full p-2 mb-4 border-2 border-white rounded-lg text-emerald-400"
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
            />

            <div
              style={{
                maxHeight: '610px',
                overflowY: 'auto',
                scrollbarWidth: 'none',
                msOverflowStyle: 'none',
              }}
            >
              {filteredUsers.map((user) => (
                <RecommendedUser key={user.id} user={user} />
              ))}
              <style jsx>{`
```

```
                div::-webkit-scrollbar {
                  display: none;
                }
              `}</style>
            </div>
          </div>
        </div>
      )}
    </div>
  );
};

export default SocialMediaPage;
```

FILE: frontend/src/pages/SocialMedia/UserCard.jsx

```
function UserCard({ user, isConnection }) {
    console.log(user);
    return (
        <div className='border-2 border-white rounded-lg p-4 flex flex-col items-center transition-
all hover:shadow-md hover:bg-emerald-400'>
            <img
                src={user.connection.profile_picture || "/avatar.png"}
                className='w-24 h-24 rounded-full object-cover mb-4'
            />
            <h3 className='font-semibold text-lg text-emerald-400 text-center'>
                {user.connection.first_name + " " + user.connection.last_name}
            </h3>
            <p className='text-emerald-400 text-center'>{user.connection.headline}</p>
            <p className='text-sm text-emerald-400 mt-2'>
                {user.connection.connections?.length} connections
            </p>
            <button className='mt-4 border-2 border-white text-emerald-400 px-4 py-2 rounded-md
hover:bg-emerald-400 hover:text-white transition-colors w-full'>
                {isConnection ? "Connected" : "Connect"}
            </button>
        </div>
    );
}

export default UserCard;
```

FILE: frontend/src/pages/State/Redux.jsx

```
import { useRef } from "react";
import { combineReducers, configureStore } from "@reduxjs/toolkit";
import { useDispatch, useSelector, Provider } from "react-redux";
import globalReducer from "./State"; // Assuming this is your custom reducer

import {
  persistStore,
  persistReducer,
  FLUSH,
  REHYDRATE,
  PAUSE,
  PERSIST,
  PURGE,
  REGISTER,
} from "redux-persist";
import { PersistGate } from "redux-persist/integration/react";
import createWebStorage from "redux-persist/lib/storage/createWebStorage";

/* REDUX PERSISTENCE */
const createNoopStorage = () => {
  return {
```

```
    getItem(_key) {
      return Promise.resolve(null);
    },
    setItem(_key, value) {
      return Promise.resolve(value);
    },
    removeItem(_key) {
      return Promise.resolve();
    },
  };
};

const storage =
  typeof window === "undefined"
    ? createNoopStorage()
    : createWebStorage("local");

const persistConfig = {
  key: "root",
  storage,
  whitelist: ["global"], // Add global reducer to whitelist if it's needed for persistence
};

const rootReducer = combineReducers({
  global: globalReducer,
});

const persistedReducer = persistReducer(persistConfig, rootReducer);

/* REDUX STORE */
export const makeStore = () => {
  return configureStore({
    reducer: persistedReducer,
    middleware: (getDefaultMiddleware) =>
      getDefaultMiddleware({
        serializableCheck: {
          ignoredActions: [FLUSH, REHYDRATE, PAUSE, PERSIST, PURGE, REGISTER],
        },
      }), // Removed api.middleware
  });
};

/* PROVIDER */
export const useAppDispatch = () => useDispatch();
export const useAppSelector = useSelector;

/* STORE PROVIDER COMPONENT */
export default function StoreProvider({ children }) {
  const storeRef = useRef();
  if (!storeRef.current) {
    storeRef.current = makeStore();
  }
  const persistor = persistStore(storeRef.current);

  return (
    <Provider store={storeRef.current}>
      <PersistGate loading={null} persistor={persistor}>
        {children}
      </PersistGate>
    </Provider>
  );
}
```

FILE: frontend/src/pages/State/State.jsx

```
import { createSlice } from "@reduxjs/toolkit";
```

```
const initialState = {
  isSidebarCollapsed: false,
};

export const globalSlice = createSlice({
  name: "global",
  initialState,
  reducers: {
    setIsSidebarCollapsed: (state, action) => {
      state.isSidebarCollapsed = action.payload;
    },
  },
});

export const { setIsSidebarCollapsed } = globalSlice.actions;

export default globalSlice.reducer;
```

FILE: frontend/src/pages/TwoFactorAuthentication.jsx

```
import { useState } from 'react';
import { useTwoFactorAuthenticationStore } from '../stores/useTwoFactorAuthenticationStore';
import { useUserStore } from '../stores/useUserStore';
import { useNavigate } from 'react-router-dom';  // Import useNavigate

const TwoFactorAuthentication = () => {
  const { sendTwoFactorCode, verifyTwoFactorCode, sendLoading, verifyLoading, sendError,
verifyError, is2FAComplete } = useTwoFactorAuthenticationStore();
  const { user } = useUserStore();

  const [code, setCode] = useState(''); // Store entered 2FA code for verification
  const [isCodeSent, setIsCodeSent] = useState(false); // Track if the code has been sent

  const navigate = useNavigate();  // Initialize useNavigate to redirect

  const handleSendCode = async () => {
    if (!user.phone_number) {
      alert('Phone number is not provided.');
      return;
    }

    // Call the function to send 2FA code
    await sendTwoFactorCode(user.phone_number);
    setIsCodeSent(true); // Set flag to show the verification input after the code is sent
  };

  const handleVerifyCode = () => {
    if (!code) {
      alert('Please enter the 2FA code.');
      return;
    }
    verifyTwoFactorCode(code);
  };

  // Redirect to the home page after 2FA is verified
  if (is2FAComplete) {
    navigate('/');  // Redirect to home page
  }

  return (
    <div className="max-w-xl mx-auto p-6 bg-transparent border-2 border-white rounded-lg">
      <h2 className="text-2xl font-semibold text-center mb-6 text-emerald-400">Two-Factor
Authentication</h2>

      {!isCodeSent ? (
```

```
        <div className="flex flex-col items-center space-y-4">
          <p className="text-lg font-medium text-emerald-400">Phone Number: <span className="font-
bold">{user.phone_number}</span></p>
          <button
            onClick={handleSendCode}
            disabled={sendLoading}
            className={`px-6 py-2 rounded-md text-emerald-400 font-semibold focus:outline-none
focus:ring-2 focus:ring-offset-2 ${sendLoading ? 'bg-transparent' : 'bg-transparent hover:bg-
emerald-400'}`}
          >
            {sendLoading ? 'Sending...' : 'Send 2FA Code'}
          </button>
        </div>
      ) : (
        <div className="flex flex-col items-center space-y-4">
          <input
            type="text"
            value={code}
            onChange={(e) => setCode(e.target.value)}
            placeholder="Enter the 2FA code"
            disabled={verifyLoading}
            className="w-full p-3 rounded-md border-2 border-white text-emerald-400 focus:outline-
none focus:ring-2 focus:ring-emerald-400"
          />
          <button
            onClick={handleVerifyCode}
            disabled={verifyLoading}
            className={`px-6 py-2 rounded-md text-emerald-400 font-semibold focus:outline-none
focus:ring-2 focus:ring-offset-2 ${verifyLoading ? 'bg-transparent' : 'bg-transparent hover:bg-
emerald-400'}`}
          >
            {verifyLoading ? 'Verifying...' : 'Verify Code'}
          </button>
        </div>
      )}

      {/* Error messages */}
      {sendError && <p className="mt-4 text-red-600 text-center">{sendError}</p>}
      {verifyError && <p className="mt-4 text-red-600 text-center">{verifyError}</p>}
    </div>
  );
};

export default TwoFactorAuthentication;
```

FILE: frontend/src/pages/UserAddressPage.jsx

```
import React, { useEffect, useState } from "react";
import { useUserStore } from "../stores/useUserStore";

const UserAddressesPage = () => {
  const { userAddresses, getUserAddresses, storeUserAddress, updateUserAddress } =
useUserStore(state => state);
  const [isAdding, setIsAdding] = useState(false);
  const [selectedAddress, setSelectedAddress] = useState(null);
  const [addressForm, setAddressForm] = useState({
    address_line1: "",
    address_line2: "",
    city: "",
    state: "",
    postal_code: "",
    country: "",
    is_primary: false,
  });

  useEffect(() => {
```

```
    // Fetch addresses when the component mounts
    getUserAddresses();
  }, [getUserAddresses]);

  // Handle form input changes
  const handleInputChange = (e) => {
    const { name, value, type, checked } = e.target;
    setAddressForm(prev => ({
      ...prev,
      [name]: type === "checkbox" ? checked : value,
    }));
  };

  // Handle the form submission for creating a new address
  const handleSubmit = async (e) => {
    e.preventDefault();
    if (isAdding) {
      await storeUserAddress(addressForm);
    } else {
      await updateUserAddress(selectedAddress.id, addressForm);
    }
    setAddressForm({
      address_line1: "",
      address_line2: "",
      city: "",
      state: "",
      postal_code: "",
      country: "",
      is_primary: false,
    });
    setSelectedAddress(null);
    setIsAdding(true);
  };

  // Handle selecting an address for editing
  const handleEdit = (address) => {
    setSelectedAddress(address);
    setAddressForm({
      address_line1: address.address_line1,
      address_line2: address.address_line2 || "",
      city: address.city,
      state: address.state,
      postal_code: address.postal_code,
      country: address.country,
      is_primary: address.is_primary,
    });
    setIsAdding(false);
  };

  return (
    <div className="mx-auto p-4 h-full bg-transparent text-emerald-400">
      <h2 className="text-3xl font-semibold mb-6">Your Addresses</h2>

      <div className="mb-6">
        <button
          onClick={() => setIsAdding(true)}
          className="bg-transparent border-2 border-white text-emerald-400 px-4 py-2 rounded-md
hover:bg-gray-800 hover:text-emerald-400"
        >
          Add New Address
        </button>
      </div>

      {/* Address Form */}
      <div className="bg-transparent border-2 border-white p-6 rounded-lg shadow-lg">
        <h3 className="text-xl font-semibold mb-4">{isAdding ? "Add Address" : "Edit Address"}</h3>
```

```
        <form onSubmit={handleSubmit}>
          <div className="space-y-4">
            <div>
              <label htmlFor="address_line1" className="block text-sm font-medium text-gray-
700">Address Line 1</label>
              <input
                type="text"
                id="address_line1"
                name="address_line1"
                value={addressForm.address_line1}
                onChange={handleInputChange}
                className="mt-1 block w-full px-4 py-2 border-2 border-white bg-transparent text-
emerald-400 rounded-md shadow-sm focus:outline-none focus:ring-2 focus:ring-blue-500"
                required
              />
            </div>

            <div>
              <label htmlFor="address_line2" className="block text-sm font-medium text-gray-
700">Address Line 2</label>
              <input
                type="text"
                id="address_line2"
                name="address_line2"
                value={addressForm.address_line2}
                onChange={handleInputChange}
                className="mt-1 block w-full px-4 py-2 border-2 border-white bg-transparent text-
emerald-400 rounded-md shadow-sm focus:outline-none focus:ring-2 focus:ring-blue-500"
              />
            </div>

            <div className="grid grid-cols-2 gap-4">
              <div>
                <label htmlFor="city" className="block text-sm font-medium text-gray-
700">City</label>
                <input
                  type="text"
                  id="city"
                  name="city"
                  value={addressForm.city}
                  onChange={handleInputChange}
                  className="mt-1 block w-full px-4 py-2 border-2 border-white bg-transparent text-
emerald-400 rounded-md shadow-sm focus:outline-none focus:ring-2 focus:ring-blue-500"
                  required
                />
              </div>

              <div>
                <label htmlFor="state" className="block text-sm font-medium text-gray-
700">State</label>
                <input
                  type="text"
                  id="state"
                  name="state"
                  value={addressForm.state}
                  onChange={handleInputChange}
                  className="mt-1 block w-full px-4 py-2 border-2 border-white bg-transparent text-
emerald-400 rounded-md shadow-sm focus:outline-none focus:ring-2 focus:ring-blue-500"
                  required
                />
              </div>
            </div>

            <div className="grid grid-cols-2 gap-4">
              <div>
```

```
                <label htmlFor="postal_code" className="block text-sm font-medium text-gray-
700">Postal Code</label>
                <input
                  type="text"
                  id="postal_code"
                  name="postal_code"
                  value={addressForm.postal_code}
                  onChange={handleInputChange}
                  className="mt-1 block w-full px-4 py-2 border-2 border-white bg-transparent text-
emerald-400 rounded-md shadow-sm focus:outline-none focus:ring-2 focus:ring-blue-500"
                  required
                />
              </div>

              <div>
                <label htmlFor="country" className="block text-sm font-medium text-gray-
700">Country</label>
                <input
                  type="text"
                  id="country"
                  name="country"
                  value={addressForm.country}
                  onChange={handleInputChange}
                  className="mt-1 block w-full px-4 py-2 border-2 border-white bg-transparent text-
emerald-400 rounded-md shadow-sm focus:outline-none focus:ring-2 focus:ring-blue-500"
                  required
                />
              </div>
            </div>

            <div>
              <label htmlFor="is_primary" className="inline-flex items-center">
                <input
                  type="checkbox"
                  id="is_primary"
                  name="is_primary"
                  checked={addressForm.is_primary}
                  onChange={handleInputChange}
                  className="h-4 w-4 text-blue-600 border-gray-300 rounded"
                />
                <span className="ml-2 text-sm text-gray-700">Set as primary address</span>
              </label>
            </div>

            <div className="mt-4">
              <button
                type="submit"
                className="bg-transparent border-2 border-white text-emerald-400 px-6 py-2 rounded-
md hover:bg-gray-800 hover:text-emerald-400"
              >
                {isAdding ? "Add Address" : "Update Address"}
              </button>
            </div>
          </div>
        </form>
      </div>

      {/* Address List */}
      <div className="mt-8 h-full">
        <h3 className="text-xl font-semibold mb-4">Your Saved Addresses</h3>
        <ul className="space-y-4">
          {userAddresses.map((address) => (
            <li key={address.id} className="bg-transparent border-2 border-white p-4 rounded-lg
shadow-md">
              <div>
                <p>{address.address_line1}, {address.address_line2 && `${address.address_line2},`}
```

```
</p>
                <p>{address.city}, {address.state} {address.postal_code}</p>
                <p>{address.country}</p>
                <p>{address.is_primary ? "Primary Address" : "Secondary Address"}</p>
              </div>
              <button
                onClick={() => handleEdit(address)}
                className="mt-2 text-emerald-400 border-2 border-white p-2 rounded-lg
hover:underline"
              >
                Edit
              </button>
            </li>
          ))}
        </ul>
      </div>
    </div>
  );
};

export default UserAddressesPage;
```

FILE: frontend/src/pages/VehicleManagement/VehicleListPage.jsx

```
import React, { useEffect, useState } from "react";
import useVehicleStore from "../../stores/useVehicleStore";
import { IoIosArrowDown, IoIosArrowUp } from "react-icons/io"; // For collapsible arrow

const VehicleListPage = () => {
  const { vehicles, isLoading, isError, errorMessage, fetchVehicles, updateVehicle } =
useVehicleStore();
  const [search, setSearch] = useState("");
  const [sortedVehicles, setSortedVehicles] = useState(vehicles || []);
  const [expandedVehicleId, setExpandedVehicleId] = useState(null);
  const [currentPage, setCurrentPage] = useState(1);
  const [itemsPerPage, setItemsPerPage] = useState(10);

  // For sorting
  const [sortField, setSortField] = useState("");
  const [sortOrder, setSortOrder] = useState("asc"); // "asc" or "desc"

  // For editing vehicle
  const [editVehicle, setEditVehicle] = useState(null);
  const [updatedVehicleData, setUpdatedVehicleData] = useState({
    license_plate: "",
    type: "",
    status: "",
    driver_id: "",
    capacity: "",
    fuel_capacity: "",
    current_location: "",
    last_serviced: "",
    last_fuel_refill: "",
    last_location_update: "",
    vehicle_management_id: "",
    fuel_interval: "",
    fuel_type: "",
    vin: "",
    brand: "",
    model: "",
    year_of_manufacture: "",
    mileage: "",
    vehicle_management: {
      maintenance_status: "",
      last_maintenance_date: "",
      maintenance_schedule: "",
```

```
      maintenance_cost: "",
      fuel_consumption: "",
      distance_traveled: "",
    },
  });

  useEffect(() => {
    fetchVehicles();
  }, [fetchVehicles]);

  useEffect(() => {
    setSortedVehicles(vehicles || []);
  }, [vehicles]);

  const handleSort = (field) => {
    if (sortField === field) {
      setSortOrder(sortOrder === "asc" ? "desc" : "asc");
    } else {
      setSortField(field);
      setSortOrder("asc");
    }
  };

  // Sorting vehicles
  const sortedAndFilteredVehicles = [...vehicles].sort((a, b) => {
    if (!sortField) return 0;

    const aValue = a[sortField];
    const bValue = b[sortField];

    if (aValue > bValue) return sortOrder === "asc" ? 1 : -1;
    if (aValue < bValue) return sortOrder === "asc" ? -1 : 1;
    return 0;
  });

  // Filtering vehicles by search
  const filteredVehicles = sortedAndFilteredVehicles.filter((vehicle) => {
    return (
      Object.keys(vehicle).some((key) => {
        if (typeof vehicle[key] === "string") {
          return vehicle[key].toLowerCase().includes(search.toLowerCase());
        }
        return false;
      }) ||
      Object.keys(vehicle.vehicle_management || {}).some((key) => {
        if (typeof vehicle.vehicle_management[key] === "string") {
          return vehicle.vehicle_management[key].toLowerCase().includes(search.toLowerCase());
        }
        return false;
      })
    );
  });

  const indexOfLastVehicle = currentPage * itemsPerPage;
  const indexOfFirstVehicle = indexOfLastVehicle - itemsPerPage;
  const currentVehicles = filteredVehicles.slice(indexOfFirstVehicle, indexOfLastVehicle);
  const totalPages = Math.ceil(filteredVehicles.length / itemsPerPage);

  const handlePageChange = (pageNumber) => {
    setCurrentPage(pageNumber);
  };

  const handleEditChange = (e) => {
    const { name, value } = e.target;

    // If it's a vehicle management field, update that section
```

```
    if (name.startsWith("vehicle_management.")) {
      const field = name.split(".")[1];
      setUpdatedVehicleData((prevData) => ({
        ...prevData,
        vehicle_management: {
          ...prevData.vehicle_management,
          [field]: value,
        },
      }));
    } else {
      // Update the top-level fields
      setUpdatedVehicleData((prevData) => ({
        ...prevData,
        [name]: value,
      }));
    }
  };

  const handleUpdateSubmit = async (e) => {
    e.preventDefault();
    if (!editVehicle) return;

    await updateVehicle(editVehicle.id, updatedVehicleData);
    setEditVehicle(null);
  };

  const handleEdit = (vehicle) => {
    setEditVehicle(vehicle);

    const vehicleManagement = vehicle.vehicle_management || {};

    setUpdatedVehicleData({
      license_plate: vehicle.license_plate || "",
      type: vehicle.type || "",
      status: vehicle.status || "",
      driver_id: vehicle.driver_id || "",
      capacity: vehicle.capacity || "",
      fuel_capacity: vehicle.fuel_capacity || "",
      current_location: vehicle.current_location || "",
      last_serviced: vehicle.last_serviced || "",
      last_fuel_refill: vehicle.last_fuel_refill || "",
      last_location_update: vehicle.last_location_update || "",
      vehicle_management_id: vehicle.vehicle_management_id || "",
      fuel_interval: vehicle.fuel_interval || "",
      fuel_type: vehicle.fuel_type || "",
      vin: vehicle.vin || "",
      brand: vehicle.brand || "",
      model: vehicle.model || "",
      year_of_manufacture: vehicle.year_of_manufacture || "",
      mileage: vehicle.mileage || "",
      vehicle_management: {
        maintenance_status: vehicleManagement.maintenance_status || "",
        last_maintenance_date: vehicleManagement.last_maintenance_date || "",
        maintenance_schedule: vehicleManagement.maintenance_schedule || "",
        maintenance_cost: vehicleManagement.maintenance_cost || "",
        fuel_consumption: vehicleManagement.fuel_consumption || "",
        distance_traveled: vehicleManagement.distance_traveled || "",
      },
    });
  };

  if (isLoading) {
    return (
      <div className="flex justify-center items-center h-screen">
        <div className="animate-spin w-12 h-12 border-t-4 border-blue-600 rounded-full"></div>
      </div>
```

```
    );
  }

  if (isError) {
    return (
      <div className="flex justify-center items-center h-screen">
        <div className="text-xl font-bold text-red-600">{errorMessage}</div>
      </div>
    );
  }

  if (!vehicles || filteredVehicles.length === 0) {
    return (
      <div className="flex justify-center items-center h-screen">
        <div className="text-xl font-bold text-emerald-400">No vehicles found.</div>
      </div>
    );
  }

  return (
    <div className="w-full mx-auto px-4 sm:px-6 lg:px-8 py-6 rounded-lg shadow-md">
      {/* Search bar */}
      <div className="mb-6">
        <input
          type="text"
          placeholder="Search vehicles by any field (brand, model, vin, etc.)"
          className="p-4 rounded-lg border border-gray-300 bg-gray-900 w-full text-emerald-400
shadow focus:outline-none focus:ring-2 focus:ring-emerald-400"
          value={search}
          onChange={(e) => setSearch(e.target.value)}
        />
      </div>

      {/* Sort buttons */}
      <div className="flex space-x-4 mb-6">
        {["year_of_manufacture", "mileage", "brand", "model", "license_plate", "vin"].map((field)
=> (
          <button
            key={field}
            onClick={() => handleSort(field)}
            className={`text-sm font-semibold ${sortField === field ? "text-emerald-900" : "text-
emerald-400"} hover:text-emerald-900 transition duration-200`}
          >
            Sort by {field.replace(/_/g, " ")}
            {sortField === field && (sortOrder === "asc" ? " ↑" : " ↓")}
          </button>
        ))}
      </div>

      <div className="space-y-6">
        {currentVehicles.map((vehicle) => (
          <div key={vehicle.id} className="bg-gray-900 p-6 rounded-lg shadow-lg border border-gray-
200">
            <h3 className="text-xl font-semibold text-emerald-400 flex justify-between items-
center">
              {vehicle.brand} {vehicle.model} ({vehicle.year_of_manufacture})
              <button
                onClick={() => setExpandedVehicleId(expandedVehicleId === vehicle.id ? null :
vehicle.id)}
                className="text-emerald-400 hover:text-emerald-400"
              >
                {expandedVehicleId === vehicle.id ? <IoIosArrowDown /> : <IoIosArrowUp />}
              </button>
            </h3>

            {/* Editable or Display Mode */}
```

```
            {editVehicle?.id === vehicle.id ? (
              <div className="mt-4 space-y-4 bg-gray-800 p-6 rounded-lg border border-gray-700
shadow-md">
                <h4 className="text-lg text-emerald-400 font-bold">Edit Vehicle Details</h4>
                <form onSubmit={handleUpdateSubmit}>
                  {/* Vehicle Details Editing */}
                  {Object.keys(updatedVehicleData).map((key) => {
                    if (key !== "vehicle_management") {
                      return (
                        <div key={key} className="space-y-2">
                          <label className="text-emerald-400 font-semibold">{key.replace(/_/g, "
").toUpperCase()}:</label>
                          <input
                            type="text"
                            name={key}
                            value={updatedVehicleData[key]}
                            onChange={handleEditChange}
                            className="w-full p-3 border border-gray-500 rounded-lg bg-gray-900
text-white shadow-sm focus:outline-none focus:ring-2 focus:ring-emerald-400"
                          />
                        </div>
                      );
                    }
                  })}

                  {/* Vehicle Management Editing */}
                  <div className="mt-4 space-y-4">
                    <h4 className="text-lg text-emerald-400 font-bold">Edit Vehicle Management</h4>
                    {Object.keys(updatedVehicleData.vehicle_management).map((key) => (
                      <div key={key} className="space-y-2">
                        <label className="text-emerald-400 font-semibold">{key.replace(/_/g, "
").toUpperCase()}:</label>
                        <input
                          type="text"
                          name={`vehicle_management.${key}`}  // Name includes the
vehicle_management prefix
                          value={updatedVehicleData.vehicle_management[key]}  // Use the
vehicle_management value
                          onChange={handleEditChange}
                          className="w-full p-3 border border-gray-500 rounded-lg bg-gray-900 text-
white shadow-sm focus:outline-none focus:ring-2 focus:ring-emerald-400"
                        />
                      </div>
                    ))}
                  </div>

                  {/* Save Changes Button */}
                  <div className="mt-6">
                    <button
                      type="submit"
                      className="w-full py-3 bg-emerald-500 text-white rounded-lg hover:bg-emerald-
600 transition duration-200"
                    >
                      Save Changes
                    </button>
                  </div>
                </form>
              </div>
            ) : (
              <div className="space-y-4">
                {/* Display Vehicle Information */}
                {Object.keys(vehicle).map((key) => (
                  key !== "vehicle_management" && (
                    <div key={key} className="space-y-2">
                      <label className="text-emerald-400 font-semibold">{key.replace(/_/g, "
").toUpperCase()}:</label>
```

```
                      <p className="text-white">{vehicle[key]}</p>
                    </div>
                  )
                ))}

                {/* Vehicle Management Information */}
                {expandedVehicleId === vehicle.id && vehicle.vehicle_management && (
                  <div className="mt-4 space-y-4 p-4 border-2 border-white rounded-xl">
                    <h4 className="text-lg text-emerald-400 font-bold">Vehicle Management</h4>
                    {Object.keys(vehicle.vehicle_management).map((key) => (
                      <div key={key} className="space-y-2">
                        <label className="text-emerald-400 font-semibold">{key.replace(/_/g, "
").toUpperCase()}:</label>
                        <p className="text-white">{vehicle.vehicle_management[key]}</p>
                      </div>
                    ))}
                  </div>
                )}

                {/* Edit Button */}
                <div className="mt-4">
                  <button
                    onClick={() => handleEdit(vehicle)}
                    className="text-white bg-emerald-400 py-2 px-4 rounded-lg"
                  >
                    Edit Vehicle
                  </button>
                </div>
              </div>
            )}
          </div>
        ))}
      </div>

      {/* Pagination */}
      <div className="flex justify-center mt-6">
        <button
          disabled={currentPage === 1}
          onClick={() => handlePageChange(currentPage - 1)}
          className="px-4 py-2 text-sm font-semibold bg-emerald-400 text-white rounded-md
disabled:bg-gray-600"
        >
          Prev
        </button>
        <div className="px-4 py-2 text-sm font-semibold text-emerald-400">
          Page {currentPage} of {totalPages}
        </div>
        <button
          disabled={currentPage === totalPages}
          onClick={() => handlePageChange(currentPage + 1)}
          className="px-4 py-2 text-sm font-semibold bg-emerald-400 text-white rounded-md
disabled:bg-gray-600"
        >
          Next
        </button>
      </div>
    </div>
  );
};

export default VehicleListPage;
```

FILE: frontend/src/pages/VehicleManagement/WrapperVehicle.jsx

```
import React, { useEffect } from "react";
```

```
import Navbar from "../../components/Vehicle/Navbar";
import Sidebar from "../../components/Vehicle/Sidebar";
import { useAppSelector } from "../State/Redux";
import { Outlet } from "react-router-dom"; // Import Outlet for nested routes

const DashboardLayout = () => {
  const isSidebarCollapsed = useAppSelector(
    (state) => state.global.isSidebarCollapsed
  );
  const isDarkMode = useAppSelector((state) => state.global.isDarkMode);

  useEffect(() => {
    if (isDarkMode) {
      document.documentElement.classList.add("dark");
    } else {
      document.documentElement.classList.remove("dark");
    }
  }, [isDarkMode]); // Add dependency array to avoid unnecessary re-renders

  return (
    <div
      className={`${
        isDarkMode ? "dark" : "light"
      } flex bg-gray-900 text-gray-900 w-full h-full`}
    >
      <Sidebar />
      <main
        className={`flex flex-col w-full h-full py-7 px-9 bg-gray-900 ${
          isSidebarCollapsed ? "md:pl-24" : "md:pl-72"
        }`}
      >
        <Navbar />
        {/* Render child components inside Outlet */}
        <Outlet />
      </main>
    </div>
  );
};

const WrapperVehicle = ({ children }) => {
  return (
      <DashboardLayout>{children}</DashboardLayout>
  );
};

export default WrapperVehicle;
```

FILE: frontend/src/pages/Warehouse/CreateInventory.jsx

```
import React from 'react';
import useInventoryStore from '../../stores/useInventoryStore';

const CreateInventory = () => {
    const {
        productDetails,
        setProductDetails,
        stock,
        weightPerUnit,
        setInventoryDetails,
        createInventory,
        isSubmitting,
    } = useInventoryStore();

    const handleProductChange = (e) => {
        const { name, value, type, checked } = e.target;
        setProductDetails({
```

```
            ...productDetails,
            [name]: type === 'checkbox' ? checked : value
        });
    };

    const handleInventoryChange = (e) => {
        const { name, value } = e.target;
        setInventoryDetails({ [name]: value });
    };

    const handleSubmit = (e) => {
        e.preventDefault();
        createInventory();
    };

    return (
        <div className="w-full p-8 shadow-xl rounded-lg">
            <h1 className="text-3xl font-extrabold text-center text-emerald-400 mb-8">Create
Product and Inventory</h1>
            <form onSubmit={handleSubmit} className="space-y-8">

                {/* Product Details Section */}
                <div className="bg-emerald-400 p-6 rounded-lg shadow-sm">
                    <h3 className="text-xl font-semibold text-gray-800 mb-6">Product Details</h3>
                    <div className="space-y-6">
                        <div className="flex flex-col">
                            <label className="text-sm font-medium text-gray-700"
htmlFor="name">Product Name</label>
                            <input
                                id="name"
                                type="text"
                                name="name"
                                value={productDetails.name}
                                onChange={handleProductChange}
                                className="mt-2 p-4 border border-gray-300 rounded-lg shadow-sm
focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 transition-all"
                                required
                            />
                        </div>
                        <div className="flex flex-col">
                            <label className="text-sm font-medium text-gray-700"
htmlFor="description">Description</label>
                            <input
                                id="description"
                                type="text"
                                name="description"
                                value={productDetails.description}
                                onChange={handleProductChange}
                                className="mt-2 p-4 border border-gray-300 rounded-lg shadow-sm
focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 transition-all"
                                required
                            />
                        </div>
                        <div className="flex flex-col">
                            <label className="text-sm font-medium text-gray-700"
htmlFor="price">Price</label>
                            <input
                                id="price"
                                type="number"
                                name="price"
                                value={productDetails.price}
                                onChange={handleProductChange}
                                className="mt-2 p-4 border border-gray-300 rounded-lg shadow-sm
focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 transition-all"
                                required
                            />
```

```
                        </div>
                        <div className="flex flex-col">
                            <label className="text-sm font-medium text-gray-700"
htmlFor="image">Product Image</label>
                            <input
                                id="image"
                                type="file"
                                name="image"
                                onChange={(e) => setProductDetails({ ...productDetails, image:
e.target.files[0] })}
                                className="mt-2 p-2 text-sm border border-gray-300 rounded-lg"
                            />
                        </div>
                        <div className="flex flex-col">
                            <label className="text-sm font-medium text-gray-700"
htmlFor="category">Category</label>
                            <input
                                id="category"
                                type="text"
                                name="category"
                                value={productDetails.category}
                                onChange={handleProductChange}
                                className="mt-2 p-4 border border-gray-300 rounded-lg shadow-sm
focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 transition-all"
                            />
                        </div>
                        <div className="flex items-center">
                            <input
                                id="isFeatured"
                                type="checkbox"
                                name="isFeatured"
                                checked={productDetails.isFeatured}
                                onChange={handleProductChange}
                                className="mr-2 w-5 h-5 text-indigo-600 focus:ring-indigo-500"
                            />
                            <label htmlFor="isFeatured" className="text-sm font-medium text-gray-
700">Is Featured</label>
                        </div>
                    </div>
                </div>

                {/* Inventory Details Section */}
                <div className="bg-emerald-400 p-6 rounded-lg shadow-sm">
                    <h3 className="text-xl font-semibold text-gray-800 mb-6">Inventory Details</h3>
                    <div className="space-y-6">
                        <div className="flex flex-col">
                            <label className="text-sm font-medium text-gray-700"
htmlFor="stock">Stock</label>
                            <input
                                id="stock"
                                type="number"
                                name="stock"
                                value={stock}
                                onChange={handleInventoryChange}
                                className="mt-2 p-4 border border-gray-300 rounded-lg shadow-sm
focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 transition-all"
                                required
                            />
                        </div>
                        <div className="flex flex-col">
                            <label className="text-sm font-medium text-gray-700"
htmlFor="weightPerUnit">Weight Per Unit</label>
                            <input
                                id="weightPerUnit"
                                type="number"
                                name="weightPerUnit"
```

```
                                value={weightPerUnit}
                                onChange={handleInventoryChange}
                                className="mt-2 p-4 border border-gray-300 rounded-lg shadow-sm
focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 transition-all"
                                required
                            />
                        </div>
                    </div>
                </div>

                {/* Submit Button */}
                <div className="flex justify-center">
                    <button
                        type="submit"
                        disabled={isSubmitting}
                        className={`px-8 py-3 mt-6 rounded-lg text-white font-semibold text-lg
${isSubmitting ? 'bg-gray-400' : 'bg-emerald-400 hover:bg-emerald-200'} transition-all
focus:outline-none focus:ring-2 focus:ring-indigo-500 focus:ring-offset-2`}
                    >
                        {isSubmitting ? 'Submitting...' : 'Create Product and Inventory'}
                    </button>
                </div>
            </form>
        </div>
    );
};

export default CreateInventory;
```

FILE: frontend/src/pages/Warehouse/Daily.jsx

```
import React, { useState, useEffect, useMemo } from "react";
import { Box } from "@mui/material";
import { ResponsiveLine } from "@nivo/line";
import DatePicker from "react-datepicker";
import "react-datepicker/dist/react-datepicker.css";
import useLineChartStore from "../../stores/useLineChartStore";

const Daily = () => {
  const [startDate, setStartDate] = useState(new Date("2024-05-01"));
  const [endDate, setEndDate] = useState(new Date("2025-05-15"));

  const {
    data,
    loading,
    isError,
    errorMessage,
    fetchLineChartData,
  } = useLineChartStore();

  useEffect(() => {
    if (startDate && endDate) {
      fetchLineChartData(startDate, endDate);
    }
  }, [startDate, endDate, fetchLineChartData]);

  /**
   * Format + Sort Data (Ascending by Date)
   */
  const formattedData = useMemo(() => {
    if (!Array.isArray(data) || data.length === 0) return [];

    // 1️⃣ Keep only valid date records
    const safeData = data.filter(
      (item) =>
        item?.date &&
```

```
        !isNaN(new Date(item.date))
    );

    // 2️⃣ Sort ascending (oldest → newest)
    const sortedData = [...safeData].sort(
      (a, b) => new Date(a.date) - new Date(b.date)
    );

    // 3️⃣ Map into chart structure
    return [
      {
        id: "Shipment",
        color: "yellow",
        data: sortedData
          .filter((item) => item.total_shipment_amount != null)
          .map((item) => ({
            x: new Date(item.date),
            y: Number(item.total_shipment_amount) || 0,
          })),
      },
      {
        id: "Vehicle Maintenance",
        color: "green",
        data: sortedData
          .filter((item) => item.total_maintenance_cost != null)
          .map((item) => ({
            x: new Date(item.date),
            y: Number(item.total_maintenance_cost) || 0,
          })),
      },
      {
        id: "Product Expenses",
        color: "red",
        data: sortedData
          .filter((item) => item.total_product_expenses != null)
          .map((item) => ({
            x: new Date(item.date),
            y: Number(item.total_product_expenses) || 0,
          })),
      },
    ];
  }, [data]);

  return (
    <Box m="1.5rem 2.5rem">
      <Box height="75vh">

        {/* Date Pickers */}
        <Box display="flex" justifyContent="flex-end" gap={2} mb={3}>
          <DatePicker
            selected={startDate}
            onChange={(date) => setStartDate(date)}
            selectsStart
            startDate={startDate}
            endDate={endDate}
            className="bg-emerald-400 px-2 py-1 rounded"
          />

          <DatePicker
            selected={endDate}
            onChange={(date) => setEndDate(date)}
            selectsEnd
            startDate={startDate}
            endDate={endDate}
            minDate={startDate}
            className="bg-emerald-400 px-2 py-1 rounded"
```

```
        />
      </Box>

      {loading ? (
        <>Loading...</>
      ) : isError ? (
        <Box color="red">{errorMessage}</Box>
      ) : formattedData.length > 0 &&
        formattedData.some((serie) => serie.data.length > 0) ? (
        <ResponsiveLine
          data={formattedData}
          margin={{ top: 50, right: 50, bottom: 70, left: 60 }}

          /* Time Scale (Required for real dates) */
          xScale={{
            type: "time",
            format: "native",
            precision: "day",
          }}

          yScale={{
            type: "linear",
            min: "auto",
            max: "auto",
            stacked: false,
            reverse: false,
          }}

          xFormat="time:%Y-%m-%d"
          yFormat=" >-.2f"
          curve="catmullRom"

          axisBottom={{
            format: "%b %d",
            tickValues: "every 1 month",
            tickRotation: 45,
            legend: "Date",
            legendOffset: 50,
            legendPosition: "middle",
          }}

          axisLeft={{
            legend: "Total",
            legendOffset: -50,
            legendPosition: "middle",
          }}

          enableGridX={false}
          enableGridY={true}

          pointSize={6}
          pointColor={{ theme: "background" }}
          pointBorderWidth={2}
          pointBorderColor={{ from: "serieColor" }}
          useMesh={true}

          colors={{ datum: "color" }}

          theme={{
            axis: {
              domain: { line: { stroke: "#9e9e9e" } },
              ticks: {
                line: { stroke: "#9e9e9e" },
                text: { fill: "#9e9e9e" },
              },
              legend: {
```

```
                text: { fill: "#9e9e9e" },
              },
            },
            legends: {
              text: { fill: "#9e9e9e" },
            },
            tooltip: {
              container: {
                color: "#1976d2",
              },
            },
          }}

          legends={[
            {
              anchor: "top-left",
              direction: "column",
              translateX: 50,
              itemWidth: 120,
              itemHeight: 20,
              symbolSize: 12,
              symbolShape: "circle",
              effects: [
                {
                  on: "hover",
                  style: {
                    itemBackground: "rgba(0,0,0,0.03)",
                    itemOpacity: 1,
                  },
                },
              ],
            },
          ]}
        />
      ) : (
        <>No Data Available</>
      )}
    </Box>
  </Box>
);
};

export default Daily;
```

FILE: frontend/src/pages/Warehouse/Expenses.jsx

```
import { useEffect, useMemo } from "react";
import { useExpensesStore } from "../../stores/useExpensesStore";
import {
  Cell,
  Legend,
  Pie,
  PieChart,
  ResponsiveContainer,
  Tooltip,
} from "recharts";

const Expenses = () => {
  // Get states and actions from the store
  const {
    expenses,
    isLoading,
    isError,
    fetchExpenses,
    startDate,
    endDate,
```

```
    setStartDate,
    setEndDate,
  } = useExpensesStore();

  // Fetch expenses whenever the startDate or endDate changes
  useEffect(() => {
    if (startDate && endDate) {
      fetchExpenses(); // Fetch expenses whenever the filters are set
    }
  }, [startDate, endDate, fetchExpenses]);

  const aggregatedData = useMemo(() => {
    // The expenses are now in the correct format, an array with name and amount
    return expenses.map((item) => ({
      name: item.name,
      amount: item.amount,
    }));
  }, [expenses]);

  const handleStartDateChange = (e) => {
    setStartDate(e.target.value);
  };

  const handleEndDateChange = (e) => {
    setEndDate(e.target.value);
  };

  if (isLoading) {
    return (
      <div className="flex justify-center items-center h-screen">
        <div className="loader"></div> {/* You can customize this loader with Tailwind or use an
external loader */}
      </div>
    );
  }

  if (isError) {
    return (
      <div className="text-red-500 text-center mt-4">
        <p>Error fetching data</p>
      </div>
    );
  }

  return (
    <div className="bg-gradient-to-r from-teal-500 via-green-500 to-emerald-500 p-8 rounded-2xl
shadow-lg">
      {/* Filters for date range */}
      <div className="mb-6 space-y-4 bg-gray-900 p-6 rounded-xl shadow-lg">
        <h2 className="text-xl font-semibold text-white">Filter Expenses</h2>
        <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
          {/* Start Date */}
          <div>
            <label htmlFor="startDate" className="block text-sm font-medium text-white">
              Start Date
            </label>
            <input
              type="date"
              id="startDate"
              value={startDate}
              onChange={handleStartDateChange}
              className="mt-1 w-full px-4 py-2 bg-gray-100 border border-gray-300 rounded-md
focus:outline-none focus:ring-2 focus:ring-teal-500"
            />
          </div>
```

```
          {/* End Date */}
          <div>
            <label htmlFor="endDate" className="block text-sm font-medium text-white">
              End Date
            </label>
            <input
              type="date"
              id="endDate"
              value={endDate}
              onChange={handleEndDateChange}
              className="mt-1 w-full px-4 py-2 bg-gray-100 border border-gray-300 rounded-md
focus:outline-none focus:ring-2 focus:ring-teal-500"
            />
          </div>
        </div>
      </div>

      {/* Pie Chart */}
      <div className="my-6">
        <h2 className="text-2xl font-semibold text-gray-700 text-center mb-4">Expense
Distribution</h2>
        <ResponsiveContainer width="100%" height={400}>
          <PieChart>
            <Pie
              data={aggregatedData}
              cx="50%"
              cy="50%"
              label
              outerRadius={150}
              fill="#8884d8"
              dataKey="amount"
            >
              {aggregatedData.map((entry, index) => (
                <Cell
                  key={`cell-${index}`}
                  fill={`#${Math.floor(Math.random() * 16777215).toString(16)}`}
                />
              ))}
            </Pie>
            <Tooltip />
            <Legend iconSize={10} layout="horizontal" verticalAlign="bottom" />
          </PieChart>
        </ResponsiveContainer>
      </div>
    </div>
  );
};

export default Expenses;
```

FILE: frontend/src/pages/Warehouse/Geography.jsx

```
import React, { useEffect } from "react";
import { Box } from "@mui/material";
import { useGeographyStore } from "../../stores/useGeographyStore";
import { ResponsiveChoropleth } from "@nivo/geo";
import { geoData } from "../../stores/geoData";
const Geography = () => {
  const { geographyData, isLoading, isError, errorMessage, fetchGeography } = useGeographyStore();

  // Fetch geography data on component mount
  useEffect(() => {
    fetchGeography();
  }, [fetchGeography]);

  // If loading, show a loading message
```

```
  if (isLoading) {
    return (
      <Box m="1.5rem 2.5rem">
        <Box mt="40px" height="75vh" border="1px solid #ccc" borderRadius="4px">
          <p>Loading...</p>
        </Box>
      </Box>
    );
  }

  // If there's an error, show an error message
  if (isError) {
    return (
      <Box m="1.5rem 2.5rem">
        <Box mt="40px" height="75vh" border="1px solid #ccc" borderRadius="4px">
          <p style={{ color: "red" }}>{errorMessage}</p>
        </Box>
      </Box>
    );
  }

  return (
    <Box m="1.5rem 2.5rem">
      <Box
        mt="40px"
        height="75vh"
        border="1px solid #ccc"
        borderRadius="4px"
      >
        {geographyData.length > 0 ? (
          <ResponsiveChoropleth
            data={geographyData}
            theme={{
              axis: {
                domain: {
                  line: {
                    stroke: "#ccc", // Light grey line color
                  },
                },
                legend: {
                  text: {
                    fill: "#ccc", // Light grey text color
                  },
                },
                ticks: {
                  line: {
                    stroke: "#ccc",
                    strokeWidth: 1,
                  },
                  text: {
                    fill: "#ccc", // Light grey text color
                  },
                },
              },
              legends: {
                text: {
                  fill: "#ccc", // Light grey text color
                },
              },
              tooltip: {
                container: {
                  color: "#333", // Dark color for tooltip text
                },
              },
            }}
            features={geoData.features}
```

```
            margin={{ top: 0, right: 0, bottom: 0, left: -50 }}
            domain={[0, 60]}
            unknownColor="#666666"
            label="properties.name"
            valueFormat=".2s"
            projectionScale={80}
            projectionTranslation={[0.45, 0.5]}
            projectionRotation={[0, 0, 0]}
            borderWidth={1.3}
            borderColor="#ffffff"
            legends={[
              {
                anchor: "bottom-right",
                direction: "column",
                justify: true,
                translateX: 0,
                translateY: -125,
                itemsSpacing: 0,
                itemWidth: 94,
                itemHeight: 18,
                itemDirection: "left-to-right",
                itemTextColor: "#ccc", // Light grey text for legend items
                itemOpacity: 0.85,
                symbolSize: 18,
                effects: [
                  {
                    on: "hover",
                    style: {
                      itemTextColor: "#fff", // White text on hover
                      itemOpacity: 1,
                    },
                  },
                ],
              },
            ]}
          />
        ) : (
          <p>No data available</p>
        )}
      </Box>
    </Box>
  );
};

export default Geography;
```

FILE: frontend/src/pages/Warehouse/LicensePlateCapturePage.jsx

```
import React, { useState, useRef, useEffect } from 'react';
import Webcam from 'react-webcam';
import useLicensePlateStore from '../../stores/useLicensePlateStore';

const LicensePlateCapturePage = () => {
  const [boundingBoxes, setBoundingBoxes] = useState([]);
  const webcamRef = useRef(null);
  const canvasRef = useRef(null);  // Add a reference to the canvas

  const { licensePlates, isLoading, processLicensePlate } = useLicensePlateStore(state => ({
    licensePlates: state.licensePlates,
    isLoading: state.isLoading,
    processLicensePlate: state.processLicensePlate,
  }));
  console.log(licensePlates)

  // Function to capture a single image and process it
  const handleCapture = () => {
```

```
    const imageSrc = webcamRef.current.getScreenshot();
    processLicensePlate(imageSrc);  // Send the image to the backend for license plate detection
  };

  // UseEffect to update bounding boxes once the license plates are processed
  useEffect(() => {
    if (licensePlates.length > 0) {
      const boxes = licensePlates.map(plate => plate.bounding_box);
      setBoundingBoxes(boxes);  // Update the bounding boxes after processing
    }
  }, [licensePlates]);
// Function to draw bounding boxes on the canvas (overlay on webcam)
const drawBoundingBoxes = (ctx) => {
  if (boundingBoxes.length > 0) {
    boundingBoxes.forEach(bounding_box => {
      // Ensure bounding_box is an object with x, y, w, h properties
      if (bounding_box && typeof bounding_box === 'object' && bounding_box.hasOwnProperty('x') &&
bounding_box.hasOwnProperty('y') && bounding_box.hasOwnProperty('w') &&
bounding_box.hasOwnProperty('h')) {
        const { x, y, w, h } = bounding_box;

        // Check if the coordinates and dimensions are valid numbers
        if (typeof x === 'number' && typeof y === 'number' && typeof w === 'number' && typeof h ===
'number') {
          const x1 = x;
          const y1 = y;
          const x2 = x + w;  // Calculate the bottom-right x coordinate
          const y2 = y + h;  // Calculate the bottom-right y coordinate

          ctx.strokeStyle = 'red'; // Color of the bounding box
          ctx.lineWidth = 2;
          ctx.strokeRect(x1, y1, x2 - x1, y2 - y1); // Draw the bounding box
        }
      } else {
        console.error('Invalid bounding box data:', bounding_box);
      }
    });
  }
};


  // Function to update canvas with webcam stream and bounding boxes
  const drawCanvas = () => {
    const video = webcamRef.current.video;
    const canvas = canvasRef.current;
    const ctx = canvas.getContext('2d');

    // Draw the video frame onto the canvas
    ctx.drawImage(video, 0, 0, canvas.width, canvas.height);

    // Draw the bounding boxes on top of the video
    drawBoundingBoxes(ctx);
  };

  // UseEffect to keep updating the canvas every frame
  useEffect(() => {
    const interval = setInterval(() => {
      if (canvasRef.current && webcamRef.current) {
        drawCanvas();
      }
    }, 100); // Redraw every 100ms (10 FPS)

    return () => clearInterval(interval); // Cleanup on unmount
  }, [boundingBoxes]);

  return (
```

```
    <div className="w-full flex justify-center items-center mt-4">

      {/* Webcam container */}
      <div className="relative" style={{ width: '60%', height: 'auto' }}>
        {/* Webcam component */}
        <Webcam
          audio={false}
          screenshotFormat="image/jpeg"
          width="100%"
          height="100%" // Maintain aspect ratio with the container
          ref={webcamRef}
          videoConstraints={{
            facingMode: 'environment',
          }}
        />

        {/* Canvas to overlay the bounding boxes on webcam feed */}
        <canvas
          ref={canvasRef}
          style={{
            position: 'absolute',
            top: 0,
            left: 0,
            zIndex: 1,
            width: '100%',
            height: '100%',
          }}
        />
      </div>

      {/* Capture button */}
      <button
        onClick={handleCapture}
        disabled={isLoading}
        className="mt-4 p-2 bg-blue-500 text-white rounded"
      >
        {isLoading ? 'Processing...' : 'Capture License Plate'}
      </button>

      {/* Show loading text */}
      {isLoading && <p className="mt-2 text-white">Processing...</p>}

      {/* Detected license plates */}
      <div className="mt-4 text-white">
        {licensePlates.length > 0 ? (
          <div>
            <h2>Detected License Plates:</h2>
            <ul>
              {licensePlates.map((plate, index) => (
                <li key={index}>License Plate: {plate.license_plate}</li>
              ))}
            </ul>
          </div>
        ) : (
          <p>No license plates detected</p>
        )}
      </div>
    </div>
  );
};

export default LicensePlateCapturePage;
```

FILE: frontend/src/pages/Warehouse/WarehouseInventories.jsx

```
import { useEffect, useState } from "react";
```

```
import { useProductStore } from "../../stores/useProductStore";
import { motion } from "framer-motion";
import ProductCard from "../../components/ProductCard";
import { Search } from "lucide-react";
import { ClipLoader } from "react-spinners"; // Optional spinner library, you can choose another
one
import useInventoryStore from "../../stores/useInventoryStore";

const WarehouseInventories = () => {
  const { inventories, getInventoriesForWarehouse, updateInventoriesForWarehouse, loading } =
useProductStore();
  const { getTotalInventoryWeightForUserWarehouse, warehouse, loadingWarehouseData } =
useInventoryStore(); // Ensure loadingWarehouseData is available if needed
  const [inventoryUpdates, setInventoryUpdates] = useState([]);

  // State for search filters
  const [productName, setProductName] = useState("");
  const [category, setCategory] = useState("");

  // Called once when component mounts
  useEffect(() => {
    // Fetch the warehouse data (total weight and capacity)
    getTotalInventoryWeightForUserWarehouse();

    // Perform an initial search when the page is first loaded
    getInventoriesForWarehouse(productName, category);
  }, []); // Empty dependency array ensures this runs only on initial load

  // Function to trigger the search
  const triggerSearch = () => {
    getInventoriesForWarehouse(productName, category);
  };

  const handleInventoryUpdate = (inventoryId, stock, weight) => {
    setInventoryUpdates((prev) => [
      ...prev,
      { inventory_id: inventoryId, stock, weight_per_unit: weight },
    ]);
  };

  const handleUpdate = async () => {
    if (inventoryUpdates.length > 0) {
      await updateInventoriesForWarehouse(inventoryUpdates);
      setInventoryUpdates([]); // Reset after update
    }
  };

  // Handle changes to search fields
  const handleProductNameChange = (e) => {
    setProductName(e.target.value);
  };

  const handleCategoryChange = (e) => {
    setCategory(e.target.value);
  };

  console.log("inventories:", inventories);

  return (
    <div className="min-h-screen">
      <div className="relative z-10 max-w-screen-xl mx-auto px-4 sm:px-6 lg:px-8 py-16">
      <div className="p-6 rounded-lg shadow-xl border border-gray-200 max-w-4xl mx-auto mb-5">
            {/* Warehouse Overview Title */}
            <h2 className="text-2xl font-semibold text-emerald-500 text-center mb-6">
                Warehouse Overview
            </h2>
```

```
            <div className="flex justify-between items-center gap-6 text-center">
                {/* Total Inventory Weight */}
                <div className="flex-1 min-w-[240px]">
                <div className="text-lg font-semibold text-emerald-500 mb-2">Total Inventory
Weight</div>
                <div className="text-3xl text-green-600 font-bold">
                    {warehouse?.totalWeight || 0} kg
                </div>
                </div>

                {/* Vertical Divider */}
                <div className="w-[1px] h-16 bg-gray-300" />

                {/* Warehouse Capacity */}
                <div className="flex-1 min-w-[240px]">
                <div className="text-lg font-semibold text-emerald-500 mb-2">Warehouse
Capacity</div>
                <div className="text-3xl text-orange-600 font-bold">
                    {warehouse?.capacity || 0} kg
                </div>
                </div>

                {/* Vertical Divider */}
                <div className="w-[1px] h-16 bg-gray-300" />

                {/* Remaining Capacity */}
                <div className="flex-1 min-w-[240px]">
                <div className="text-lg font-semibold text-emerald-500 mb-2">Remaining
Capacity</div>
                <div className="text-3xl text-blue-600 font-bold">
                    {(warehouse?.capacity - warehouse?.totalWeight) || 0} kg
                </div>
                </div>
            </div>
        </div>


        {/* Search Fields */}
        <div className="flex space-x-4 mb-8">
          {/* Product Name Search */}
          <div className="flex-1 relative">
            <input
              type="text"
              value={productName}
              onChange={handleProductNameChange}
              placeholder="Search by product name"
              className="p-2 w-full border border-gray-300 rounded-md focus:ring-indigo-500 pl-10"
// Added padding for the icon
            />
            {/* Search Icon */}
            <Search
              className="absolute left-3 top-1/2 transform -translate-y-1/2 h-5 w-5 text-emerald-
400 cursor-pointer"
              onClick={triggerSearch} // Trigger search on icon click
            />
          </div>

          {/* Category Search */}
          <div className="flex-1 relative">
            <input
              type="text"
              value={category}
              onChange={handleCategoryChange}
              placeholder="Search by category"
              className="p-2 w-full border border-gray-300 rounded-md focus:ring-indigo-500 pl-10"
```

```
// Added padding for the icon
            />
            {/* Search Icon */}
            <Search
              className="absolute left-3 top-1/2 transform -translate-y-1/2 h-5 w-5 text-emerald-
400 cursor-pointer"
              onClick={triggerSearch} // Trigger search on icon click
            />
          </div>
        </div>

        {/* Loading Spinner */}
        {loading ? (
          <div className="flex justify-center items-center py-8">
            <ClipLoader size={50} color="#10B981" />
          </div>
        ) : (
          <motion.div
            className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-6 justify-
items-center"
            initial={{ opacity: 0, y: 20 }}
            animate={{ opacity: 1, y: 0 }}
            transition={{ duration: 0.8, delay: 0.2 }}
          >
            {inventories?.length === 0 && (
              <h2 className="text-3xl font-semibold text-gray-300 text-center col-span-full">
                No products found
              </h2>
            )}

            {inventories?.map((inventory) => (
              <ProductCard
                key={inventory.product.id}
                product={inventory.product}
                stock={inventory.stock}
                weight_per_unit={inventory.weight_per_unit}
                categories={inventory.categories}
                onUpdate={(stock, weight) => handleInventoryUpdate(inventory.id, stock, weight)}
              />
            ))}
          </motion.div>
        )}

        <div className="mt-4 text-center">
          <button
            onClick={handleUpdate}
            className="px-4 py-2 bg-emerald-400 text-white rounded-md"
          >
            Save Inventories
          </button>
        </div>
      </div>
    </div>
  );
};

export default WarehouseInventories;
```

FILE: frontend/src/pages/Warehouse/WarehouseOrders.jsx

```
import React, { useEffect } from "react";
import useOrderStore from "../../stores/useOrderStore";
import { toast } from "react-hot-toast";

const WarehouseOrders = () => {
  const { orders, isLoading, isError, errorMessage, fetchOrders, updateOrderStatus } =
```

```
useOrderStore(state => ({
    orders: state.orders,
    isLoading: state.isLoading,
    isError: state.isError,
    errorMessage: state.errorMessage,
    fetchOrders: state.fetchOrders,
    updateOrderStatus: state.updateOrderStatus,
  }));

  useEffect(() => {
    fetchOrders();
  }, [fetchOrders]);

  // Handle order status update
  const handleStatusChange = (orderId, newStatus) => {
    updateOrderStatus(orderId, newStatus);
  };

  if (isLoading) {
    return (
      <div className="flex justify-center items-center h-screen bg-gray-50">
        <span className="text-2xl text-gray-600">Loading...</span>
      </div>
    );
  }

  if (isError) {
    return (
      <div className="flex justify-center items-center h-screen bg-gray-50">
        <span className="text-xl text-red-500">{errorMessage}</span>
      </div>
    );
  }

  return (
    <div className="container mx-auto mt-10 px-6">
      <h1 className="text-3xl font-semibold text-emerald-400 mb-6">Warehouse Orders</h1>

      {/* Table Container */}
      <div className="overflow-x-auto bg-white rounded-lg shadow-lg">
        <table className="min-w-full bg-white border-collapse">
          <thead>
            <tr className="bg-emerald-400 border-b">
              <th className="py-3 px-6 text-left text-sm font-medium text-gray-600">Order ID</th>
              <th className="py-3 px-6 text-left text-sm font-medium text-gray-600">Order Date</th>
              <th className="py-3 px-6 text-left text-sm font-medium text-gray-600">Total
Amount</th>
              <th className="py-3 px-6 text-left text-sm font-medium text-gray-600">Status</th>
              <th className="py-3 px-6 text-left text-sm font-medium text-gray-600">Update
Status</th>
            </tr>
          </thead>
          <tbody>
            {orders.map((order) => (
              <tr key={order.id} className="hover:bg-emerald-200 border-b">
                <td className="py-3 px-6 text-sm text-gray-800">{order.id}</td>
                <td className="py-3 px-6 text-sm text-gray-800">{order.order_date}</td>
                <td className="py-3 px-6 text-sm text-gray-800">${order.total_amount}</td>
                <td className="py-3 px-6 text-sm text-gray-800">{order.status}</td>

                {/* Dropdown for updating status */}
                <td className="py-3 px-6 text-sm">
                  <select
                    value={order.status}
                    onChange={(e) => handleStatusChange(order.id, e.target.value)}
                    className="border border-gray-300 rounded-md p-2 text-gray-800 focus:ring-2
```

```
focus:ring-indigo-500 focus:outline-none w-full"
                  >
                    <option value="Pending">Pending</option>
                    <option value="Confirmed">Confirmed</option>
                    <option value="Packed">Packed</option>
                  </select>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
};

export default WarehouseOrders;
```

FILE: frontend/src/pages/Warehouse/Wrapper.jsx

```
import React, { useEffect } from "react";
import Navbar from "../../components/Warehouse/Navbar";
import Sidebar from "../../components/Warehouse/Sidebar";
import { useAppSelector } from "../State/Redux";
import { Outlet } from "react-router-dom"; // Import Outlet for nested routes

const DashboardLayout = () => {
  const isSidebarCollapsed = useAppSelector(
    (state) => state.global.isSidebarCollapsed
  );
  const isDarkMode = useAppSelector((state) => state.global.isDarkMode);

  useEffect(() => {
    if (isDarkMode) {
      document.documentElement.classList.add("dark");
    } else {
      document.documentElement.classList.remove("dark");
    }
  }, [isDarkMode]); // Add dependency array to avoid unnecessary re-renders

  return (
    <div
      className={`${
        isDarkMode ? "dark" : "light"
      } flex bg-gray-900 text-gray-900 w-full h-full`}
    >
      <Sidebar />
      <main
        className={`flex flex-col w-full h-full py-7 px-9 bg-gray-900 ${
          isSidebarCollapsed ? "md:pl-24" : "md:pl-72"
        }`}
      >
        <Navbar />
        {/* Render child components inside Outlet */}
        <Outlet />
      </main>
    </div>
  );
};

const Wrapper = ({ children }) => {
  return (
      <DashboardLayout>{children}</DashboardLayout>
  );
};
```

```
export default Wrapper;
```

FILE: frontend/src/pages/Wheel/Coupon.jsx

```
import React, { useEffect } from "react";
import useCouponStore from "../../stores/useCouponStore";

const CouponComponent = ({ couponData, isDataChanged }) => {
  const {
    createCoupon,
    getUserCoupons,
    userCoupons,
    isProcessingCoupon,
    isErrorCoupon,
    errorMessageCoupon,
    isSuccessCoupon,
    isProcessingGetCoupons,
    isErrorGetCoupons,
    errorMessageGetCoupons,
    isSuccessGetCoupons,
  } = useCouponStore();

  // Fetch user's coupons when the component mounts
  useEffect(() => {
    getUserCoupons();
  }, [getUserCoupons]);

  // Auto-create a new coupon whenever couponData changes and isDataChanged is true
  useEffect(() => {
    if (couponData && isDataChanged) {
      createCoupon(couponData);
    }
  }, [couponData, isDataChanged, createCoupon]); // Dependency on couponData and isDataChanged

  return (
    <div className="max-w-4xl mx-auto p-6 bg-transparent shadow-lg rounded-lg border-2 border-white
h-96 overflow-y-auto w-96">
      <h1 className="text-2xl font-semibold text-center text-emerald-400 mb-6">My Coupons</h1>

      {isProcessingCoupon && (
        <div className="flex justify-center mb-6">
          <p className="text-lg text-emerald-400">Creating Coupon</p>
        </div>
      )}
      {/* Coupon Creation Success or Error Message */}
      {isSuccessCoupon && (
        <div className="flex justify-center mb-6">
          <p className="text-lg text-emerald-400">Coupon created successfully!</p>
        </div>
      )}
      {isErrorCoupon && (
        <div className="flex justify-center mb-6">
          <p className="text-lg text-red-500">Error: {errorMessageCoupon}</p>
        </div>
      )}

      {/* Fetching Coupons State */}
      {isProcessingGetCoupons ? (
        <div className="flex justify-center">
          <p className="text-lg text-emerald-400">Loading your coupons...</p>
        </div>
      ) : isErrorGetCoupons ? (
        <div className="flex justify-center">
          <p className="text-lg text-red-500">Error: {errorMessageGetCoupons}</p>
        </div>
      ) : isSuccessGetCoupons && userCoupons.length === 0 ? (
```

```
        <div className="flex justify-center">
          <p className="text-lg text-emerald-400">You have no coupons yet.</p>
        </div>
      ) : (
        <ul className="space-y-4">
          {userCoupons.map((coupon) => (
            <li key={coupon.id} className="p-4 bg-gray-100 rounded-lg border border-gray-300 flex
justify-between items-center">
              <span className="text-xl text-emerald-400">{coupon.discount}% Off</span>
            </li>
          ))}
        </ul>
      )}
    </div>
  );
};

export default CouponComponent;
```

FILE: frontend/src/pages/Wheel/Wheel.jsx

```
import React, { useEffect, useState, useRef } from 'react';
import CouponComponent from './Coupon';
import Confetti from "react-confetti";

const Wheel = () => {
  // Define the default values inside the component
  const segments = ['Coupon 10%', 'Coupon 20%', 'Coupon 30%', 'Coupon 40%', 'Coupon 50%'];
  const segColors = ['#FF5733', '#33FF57', '#3357FF', '#FF33A1', '#FFD700'];
  const primaryColor = 'black';
  const contrastColor = '#34D399';
  const buttonText = 'Spin';
  const isOnlyOnce = false; // Allow multiple spins
  const size = window.innerWidth / 5;  // Adjusted to make the wheel smaller
  const upDuration = 100; // Reduced for shorter spin
  const downDuration = 800; // Reduced for shorter spin
  const fontFamily = 'proxima-nova';
  const fontSize = '1em';
  const outlineWidth = 10;

  const randomString = () => {
    const chars = '0123456789ABCDEFGHIJKLMNOPQRSTUVWXTZabcdefghiklmnopqrstuvwxyz'.split('');
    const length = 8;
    let str = '';
    for (let i = 0; i < length; i++) {
      str += chars[Math.floor(Math.random() * chars.length)];
    }
    return str;
  };

  const canvasId = useRef(`canvas-${randomString()}`);
  const wheelId = useRef(`wheel-${randomString()}`);
  const dimension = (size + 20) * 2;
  let currentSegment = '';
  let isStarted = false;
  const [isFinished, setFinished] = useState(false);
  const [result, setResult] = useState('');
  const [isSpinning, setIsSpinning] = useState(false); // Track if the wheel is spinning
  let timerHandle = 0;
  const timerDelay = segments.length;
  let angleCurrent = 0;
  let angleDelta = 0;
  let canvasContext = null;
  let maxSpeed = Math.PI / segments.length;
  const upTime = segments.length * upDuration;
  const downTime = segments.length * downDuration;
```

```
  let spinStart = 0;
  let frames = 0;
  const centerX = size + 20;
  const centerY = size + 20;

  useEffect(() => {
    wheelInit();
    setTimeout(() => {
      window.scrollTo(0, 1);
    }, 0);
  }, []);

  const wheelInit = () => {
    initCanvas();
    wheelDraw();
  };

  const initCanvas = () => {
    let canvas = document.getElementById(canvasId.current);
    if (navigator.userAgent.indexOf('MSIE') !== -1) {
      canvas = document.createElement('canvas');
      canvas.setAttribute('width', `${dimension}`);
      canvas.setAttribute('height', `${dimension}`);
      canvas.setAttribute('id', canvasId.current);
      document.getElementById(wheelId.current).appendChild(canvas);
    }
    canvas.addEventListener('click', spin, false);
    canvasContext = canvas.getContext('2d');
  };

  const spin = () => {
    if (isSpinning) return; // Prevent spinning while already spinning
    setIsSpinning(true); // Set the spinning state
    setFinished(false);  // Reset finished state when spinning again
    setResult('');  // Clear the previous result
    // Reset spin variables before starting
    angleCurrent = 0;
    angleDelta = 0;
    frames = 0;

    if (timerHandle !== 0) {
      clearInterval(timerHandle); // Clear the previous interval
    }

    // Randomize the winning segment each time the wheel spins
    const randomIndex = Math.floor(Math.random() * segments.length);
    currentSegment = segments[randomIndex];

    // Start the new spin
    spinStart = new Date().getTime();
    maxSpeed = Math.PI / segments.length;
    timerHandle = window.setInterval(onTimerTick, timerDelay);
  };

  const onTimerTick = () => {
    frames++;
    draw();
    const duration = new Date().getTime() - spinStart;
    let progress = 0;
    let finished = false;
    if (duration < upTime) {
      progress = duration / upTime;
      angleDelta = maxSpeed * Math.sin((progress * Math.PI) / 2);
    } else {
      progress = duration / downTime;
      angleDelta = maxSpeed * Math.sin((progress * Math.PI) / 2 + Math.PI / 2);
```

```
      if (progress >= 1) finished = true;
    }

    angleCurrent += angleDelta;
    while (angleCurrent >= Math.PI * 2) angleCurrent -= Math.PI * 2;
    if (finished) {
      setFinished(true);
      setResult(currentSegment);  // Set the winning segment
      clearInterval(timerHandle); // Stop the timer when the spin finishes
      timerHandle = 0;
      angleDelta = 0;
      setIsSpinning(false); // Reset spinning state
    }
  };

  const wheelDraw = () => {
    clear();
    drawWheel();
    drawNeedle();
  };

  const draw = () => {
    clear();
    drawWheel();
    drawNeedle();
  };

  const drawSegment = (key, lastAngle, angle) => {
    if (!canvasContext) {
      return false;
    }
    const ctx = canvasContext;
    const value = segments[key];
    ctx.save();
    ctx.beginPath();
    ctx.moveTo(centerX, centerY);
    ctx.arc(centerX, centerY, size, lastAngle, angle, false);
    ctx.lineTo(centerX, centerY);
    ctx.closePath();
    ctx.fillStyle = segColors[key % segColors.length];
    ctx.fill();
    ctx.stroke();
    ctx.save();
    ctx.translate(centerX, centerY);
    ctx.rotate((lastAngle + angle) / 2);
    ctx.fillStyle = 'black';
    ctx.font = `bold ${fontSize} ${fontFamily}`;
    ctx.fillText(value.substring(0, 21), size / 2 + 20, 0);
    ctx.restore();
  };

  const drawWheel = () => {
    if (!canvasContext) {
      return false;
    }
    const ctx = canvasContext;
    let lastAngle = angleCurrent;
    const len = segments.length;
    const PI2 = Math.PI * 2;
    ctx.lineWidth = 1;
    ctx.strokeStyle = primaryColor;
    ctx.textBaseline = 'middle';
    ctx.textAlign = 'center';
    ctx.font = '1em ' + fontFamily;
    for (let i = 1; i <= len; i++) {
      const angle = PI2 * (i / len) + angleCurrent;
```

```
      drawSegment(i - 1, lastAngle, angle);
      lastAngle = angle;
    }

    // Draw center circle
    ctx.beginPath();
    ctx.arc(centerX, centerY, 50, 0, PI2, false);
    ctx.closePath();
    ctx.fillStyle = primaryColor;
    ctx.lineWidth = 10;
    ctx.strokeStyle = contrastColor;
    ctx.fill();
    ctx.font = 'bold 1em ' + fontFamily;
    ctx.fillStyle = contrastColor;
    ctx.textAlign = 'center';
    ctx.fillText(buttonText, centerX, centerY + 3);
    ctx.stroke();

    // Draw outer circle
    ctx.beginPath();
    ctx.arc(centerX, centerY, size, 0, PI2, false);
    ctx.closePath();
    ctx.lineWidth = outlineWidth;
    ctx.strokeStyle = primaryColor;
    ctx.stroke();
  };

  const drawNeedle = () => {
    if (!canvasContext) {
      return false;
    }
    const ctx = canvasContext;
    ctx.lineWidth = 1;
    ctx.strokeStyle = contrastColor;
    ctx.fillStyle = contrastColor;
    ctx.beginPath();
    ctx.moveTo(centerX + 20, centerY - 50);
    ctx.lineTo(centerX - 20, centerY - 50);
    ctx.lineTo(centerX, centerY - 70);
    ctx.closePath();
    ctx.fill();
    const change = angleCurrent + Math.PI / 2;
    let i = segments.length - Math.floor((change / (Math.PI * 2)) * segments.length) - 1;
    if (i < 0) i = i + segments.length;
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillStyle = primaryColor;
    ctx.font = 'bold 1.5em ' + fontFamily;
    currentSegment = segments[i];
    isStarted && ctx.fillText(currentSegment, centerX + 10, centerY + size + 50);
  };

  const clear = () => {
    if (!canvasContext) {
      return false;
    }
    const ctx = canvasContext;
    ctx.clearRect(0, 0, dimension, dimension);
  };
    // Use regex to extract the discount number
  const match = result.match(/\d+/); // This matches one or more digits
  const discountNumber = match ? parseInt(match[0], 10) : null;
  const now = new Date();
  const expirationDate = new Date(now.setDate(now.getDate() - 60)).toISOString().split('T')[0]; //
Get date 60 days ago in 'YYYY-MM-DD' format
  return (
```

```
    <>
    {isFinished && (<Confetti
    width={window.innerWidth}
    height={window.innerHeight}
    gravity={0.1}
    style={{ zIndex: 99 }}
    numberOfPieces={700}
    recycle={false}
    />)}
    <div style={{ display: 'flex', justifyContent: 'center', alignItems: 'center', gap: '20px' }}>
        {/* Wheel on the left */}
        <div id={wheelId.current}>
            <canvas
            id={canvasId.current}
            width={dimension}
            height={dimension}
            style={{
                pointerEvents: isSpinning && !isOnlyOnce ? 'none' : 'auto', // Prevent clicks while
spinning
            }}
            />
        </div>

        {/* Coupon component on the right */}
        <div>
            <CouponComponent isDataChanged={isFinished} couponData={{ discount: discountNumber,
expiration_date: expirationDate }} />
        </div>
    </div>
    </>

  );
};

export default Wheel;
```

FILE: frontend/src/stores/geoData.js

```
export const geoData = {
  type: "FeatureCollection",
  features: [
    {
      type: "Feature",
      properties: {
        name: "Afghanistan",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [61.210817, 35.650072],
            [62.230651, 35.270664],
            [62.984662, 35.404041],
            [63.193538, 35.857166],
            [63.982896, 36.007957],
            [64.546479, 36.312073],
            [64.746105, 37.111818],
            [65.588948, 37.305217],
            [65.745631, 37.661164],
            [66.217385, 37.39379],
            [66.518607, 37.362784],
            [67.075782, 37.356144],
            [67.83, 37.144994],
            [68.135562, 37.023115],
            [68.859446, 37.344336],
            [69.196273, 37.151144],
```

```
          [69.518785, 37.608997],
          [70.116578, 37.588223],
          [70.270574, 37.735165],
          [70.376304, 38.138396],
          [70.806821, 38.486282],
          [71.348131, 38.258905],
          [71.239404, 37.953265],
          [71.541918, 37.905774],
          [71.448693, 37.065645],
          [71.844638, 36.738171],
          [72.193041, 36.948288],
          [72.63689, 37.047558],
          [73.260056, 37.495257],
          [73.948696, 37.421566],
          [74.980002, 37.41999],
          [75.158028, 37.133031],
          [74.575893, 37.020841],
          [74.067552, 36.836176],
          [72.920025, 36.720007],
          [71.846292, 36.509942],
          [71.262348, 36.074388],
          [71.498768, 35.650563],
          [71.613076, 35.153203],
          [71.115019, 34.733126],
          [71.156773, 34.348911],
          [70.881803, 33.988856],
          [69.930543, 34.02012],
          [70.323594, 33.358533],
          [69.687147, 33.105499],
          [69.262522, 32.501944],
          [69.317764, 31.901412],
          [68.926677, 31.620189],
          [68.556932, 31.71331],
          [67.792689, 31.58293],
          [67.683394, 31.303154],
          [66.938891, 31.304911],
          [66.381458, 30.738899],
          [66.346473, 29.887943],
          [65.046862, 29.472181],
          [64.350419, 29.560031],
          [64.148002, 29.340819],
          [63.550261, 29.468331],
          [62.549857, 29.318572],
          [60.874248, 29.829239],
          [61.781222, 30.73585],
          [61.699314, 31.379506],
          [60.941945, 31.548075],
          [60.863655, 32.18292],
          [60.536078, 32.981269],
          [60.9637, 33.528832],
          [60.52843, 33.676446],
          [60.803193, 34.404102],
          [61.210817, 35.650072],
        ],
      ],
    },
    id: "AFG",
  },
  {
    type: "Feature",
    properties: {
      name: "Angola",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
```

```
[
  [
    [16.326528, -5.87747],
    [16.57318, -6.622645],
    [16.860191, -7.222298],
    [17.089996, -7.545689],
    [17.47297, -8.068551],
    [18.134222, -7.987678],
    [18.464176, -7.847014],
    [19.016752, -7.988246],
    [19.166613, -7.738184],
    [19.417502, -7.155429],
    [20.037723, -7.116361],
    [20.091622, -6.94309],
    [20.601823, -6.939318],
    [20.514748, -7.299606],
    [21.728111, -7.290872],
    [21.746456, -7.920085],
    [21.949131, -8.305901],
    [21.801801, -8.908707],
    [21.875182, -9.523708],
    [22.208753, -9.894796],
    [22.155268, -11.084801],
    [22.402798, -10.993075],
    [22.837345, -11.017622],
    [23.456791, -10.867863],
    [23.912215, -10.926826],
    [24.017894, -11.237298],
    [23.904154, -11.722282],
    [24.079905, -12.191297],
    [23.930922, -12.565848],
    [24.016137, -12.911046],
    [21.933886, -12.898437],
    [21.887843, -16.08031],
    [22.562478, -16.898451],
    [23.215048, -17.523116],
    [21.377176, -17.930636],
    [18.956187, -17.789095],
    [18.263309, -17.309951],
    [14.209707, -17.353101],
    [14.058501, -17.423381],
    [13.462362, -16.971212],
    [12.814081, -16.941343],
    [12.215461, -17.111668],
    [11.734199, -17.301889],
    [11.640096, -16.673142],
    [11.778537, -15.793816],
    [12.123581, -14.878316],
    [12.175619, -14.449144],
    [12.500095, -13.5477],
    [12.738479, -13.137906],
    [13.312914, -12.48363],
    [13.633721, -12.038645],
    [13.738728, -11.297863],
    [13.686379, -10.731076],
    [13.387328, -10.373578],
    [13.120988, -9.766897],
    [12.87537, -9.166934],
    [12.929061, -8.959091],
    [13.236433, -8.562629],
    [12.93304, -7.596539],
    [12.728298, -6.927122],
    [12.227347, -6.294448],
    [12.322432, -6.100092],
    [12.735171, -5.965682],
    [13.024869, -5.984389],
```

```
            [13.375597, -5.864241],
            [16.326528, -5.87747],
          ],
        ],
        [
          [
            [12.436688, -5.684304],
            [12.182337, -5.789931],
            [11.914963, -5.037987],
            [12.318608, -4.60623],
            [12.62076, -4.438023],
            [12.995517, -4.781103],
            [12.631612, -4.991271],
            [12.468004, -5.248362],
            [12.436688, -5.684304],
          ],
        ],
      ],
    },
    id: "AGO",
  },
  {
    type: "Feature",
    properties: {
      name: "Albania",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [20.590247, 41.855404],
          [20.463175, 41.515089],
          [20.605182, 41.086226],
          [21.02004, 40.842727],
          [20.99999, 40.580004],
          [20.674997, 40.435],
          [20.615, 40.110007],
          [20.150016, 39.624998],
          [19.98, 39.694993],
          [19.960002, 39.915006],
          [19.406082, 40.250773],
          [19.319059, 40.72723],
          [19.40355, 41.409566],
          [19.540027, 41.719986],
          [19.371769, 41.877548],
          [19.304486, 42.195745],
          [19.738051, 42.688247],
          [19.801613, 42.500093],
          [20.0707, 42.58863],
          [20.283755, 42.32026],
          [20.52295, 42.21787],
          [20.590247, 41.855404],
        ],
      ],
    },
    id: "ALB",
  },
  {
    type: "Feature",
    properties: {
      name: "United Arab Emirates",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
```

```
          [51.579519, 24.245497],
          [51.757441, 24.294073],
          [51.794389, 24.019826],
          [52.577081, 24.177439],
          [53.404007, 24.151317],
          [54.008001, 24.121758],
          [54.693024, 24.797892],
          [55.439025, 25.439145],
          [56.070821, 26.055464],
          [56.261042, 25.714606],
          [56.396847, 24.924732],
          [55.886233, 24.920831],
          [55.804119, 24.269604],
          [55.981214, 24.130543],
          [55.528632, 23.933604],
          [55.525841, 23.524869],
          [55.234489, 23.110993],
          [55.208341, 22.70833],
          [55.006803, 22.496948],
          [52.000733, 23.001154],
          [51.617708, 24.014219],
          [51.579519, 24.245497],
        ],
      ],
    },
    id: "ARE",
  },
  {
    type: "Feature",
    properties: {
      name: "Argentina",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [-65.5, -55.2],
            [-66.45, -55.25],
            [-66.95992, -54.89681],
            [-67.56244, -54.87001],
            [-68.63335, -54.8695],
            [-68.63401, -52.63637],
            [-68.25, -53.1],
            [-67.75, -53.85],
            [-66.45, -54.45],
            [-65.05, -54.7],
            [-65.5, -55.2],
          ],
        ],
        [
          [
            [-64.964892, -22.075862],
            [-64.377021, -22.798091],
            [-63.986838, -21.993644],
            [-62.846468, -22.034985],
            [-62.685057, -22.249029],
            [-60.846565, -23.880713],
            [-60.028966, -24.032796],
            [-58.807128, -24.771459],
            [-57.777217, -25.16234],
            [-57.63366, -25.603657],
            [-58.618174, -27.123719],
            [-57.60976, -27.395899],
            [-56.486702, -27.548499],
            [-55.695846, -27.387837],
```

```
            [-54.788795, -26.621786],
            [-54.625291, -25.739255],
            [-54.13005, -25.547639],
            [-53.628349, -26.124865],
            [-53.648735, -26.923473],
            [-54.490725, -27.474757],
            [-55.162286, -27.881915],
            [-56.2909, -28.852761],
            [-57.625133, -30.216295],
            [-57.874937, -31.016556],
            [-58.14244, -32.044504],
            [-58.132648, -33.040567],
            [-58.349611, -33.263189],
            [-58.427074, -33.909454],
            [-58.495442, -34.43149],
            [-57.22583, -35.288027],
            [-57.362359, -35.97739],
            [-56.737487, -36.413126],
            [-56.788285, -36.901572],
            [-57.749157, -38.183871],
            [-59.231857, -38.72022],
            [-61.237445, -38.928425],
            [-62.335957, -38.827707],
            [-62.125763, -39.424105],
            [-62.330531, -40.172586],
            [-62.145994, -40.676897],
            [-62.745803, -41.028761],
            [-63.770495, -41.166789],
            [-64.73209, -40.802677],
            [-65.118035, -41.064315],
            [-64.978561, -42.058001],
            [-64.303408, -42.359016],
            [-63.755948, -42.043687],
            [-63.458059, -42.563138],
            [-64.378804, -42.873558],
            [-65.181804, -43.495381],
            [-65.328823, -44.501366],
            [-65.565269, -45.036786],
            [-66.509966, -45.039628],
            [-67.293794, -45.551896],
            [-67.580546, -46.301773],
            [-66.597066, -47.033925],
            [-65.641027, -47.236135],
            [-65.985088, -48.133289],
            [-67.166179, -48.697337],
            [-67.816088, -49.869669],
            [-68.728745, -50.264218],
            [-69.138539, -50.73251],
            [-68.815561, -51.771104],
            [-68.149995, -52.349983],
            [-68.571545, -52.299444],
            [-69.498362, -52.142761],
            [-71.914804, -52.009022],
            [-72.329404, -51.425956],
            [-72.309974, -50.67701],
            [-72.975747, -50.74145],
            [-73.328051, -50.378785],
            [-73.415436, -49.318436],
            [-72.648247, -48.878618],
            [-72.331161, -48.244238],
            [-72.447355, -47.738533],
            [-71.917258, -46.884838],
            [-71.552009, -45.560733],
            [-71.659316, -44.973689],
            [-71.222779, -44.784243],
            [-71.329801, -44.407522],
```

```
            [-71.793623, -44.207172],
            [-71.464056, -43.787611],
            [-71.915424, -43.408565],
            [-72.148898, -42.254888],
            [-71.746804, -42.051386],
            [-71.915734, -40.832339],
            [-71.680761, -39.808164],
            [-71.413517, -38.916022],
            [-70.814664, -38.552995],
            [-71.118625, -37.576827],
            [-71.121881, -36.658124],
            [-70.364769, -36.005089],
            [-70.388049, -35.169688],
            [-69.817309, -34.193571],
            [-69.814777, -33.273886],
            [-70.074399, -33.09121],
            [-70.535069, -31.36501],
            [-69.919008, -30.336339],
            [-70.01355, -29.367923],
            [-69.65613, -28.459141],
            [-69.001235, -27.521214],
            [-68.295542, -26.89934],
            [-68.5948, -26.506909],
            [-68.386001, -26.185016],
            [-68.417653, -24.518555],
            [-67.328443, -24.025303],
            [-66.985234, -22.986349],
            [-67.106674, -22.735925],
            [-66.273339, -21.83231],
            [-64.964892, -22.075862],
          ],
        ],
      ],
    },
    id: "ARG",
  },
  {
    type: "Feature",
    properties: {
      name: "Armenia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [43.582746, 41.092143],
          [44.97248, 41.248129],
          [45.179496, 40.985354],
          [45.560351, 40.81229],
          [45.359175, 40.561504],
          [45.891907, 40.218476],
          [45.610012, 39.899994],
          [46.034534, 39.628021],
          [46.483499, 39.464155],
          [46.50572, 38.770605],
          [46.143623, 38.741201],
          [45.735379, 39.319719],
          [45.739978, 39.473999],
          [45.298145, 39.471751],
          [45.001987, 39.740004],
          [44.79399, 39.713003],
          [44.400009, 40.005],
          [43.656436, 40.253564],
          [43.752658, 40.740201],
          [43.582746, 41.092143],
        ],
```

```
        ],
      },
      id: "ARM",
    },
    {
      type: "Feature",
      properties: {
        name: "Antarctica",
      },
      geometry: {
        type: "MultiPolygon",
        coordinates: [
          [
            [
              [-59.572095, -80.040179],
              [-59.865849, -80.549657],
              [-60.159656, -81.000327],
              [-62.255393, -80.863178],
              [-64.488125, -80.921934],
              [-65.741666, -80.588827],
              [-65.741666, -80.549657],
              [-66.290031, -80.255773],
              [-64.037688, -80.294944],
              [-61.883246, -80.39287],
              [-61.138976, -79.981371],
              [-60.610119, -79.628679],
              [-59.572095, -80.040179],
            ],
          ],
          [
            [
              [-159.208184, -79.497059],
              [-161.127601, -79.634209],
              [-162.439847, -79.281465],
              [-163.027408, -78.928774],
              [-163.066604, -78.869966],
              [-163.712896, -78.595667],
              [-163.712896, -78.595667],
              [-163.105801, -78.223338],
              [-161.245113, -78.380176],
              [-160.246208, -78.693645],
              [-159.482405, -79.046338],
              [-159.208184, -79.497059],
            ],
          ],
          [
            [
              [-45.154758, -78.04707],
              [-43.920828, -78.478103],
              [-43.48995, -79.08556],
              [-43.372438, -79.516645],
              [-43.333267, -80.026123],
              [-44.880537, -80.339644],
              [-46.506174, -80.594357],
              [-48.386421, -80.829485],
              [-50.482107, -81.025442],
              [-52.851988, -80.966685],
              [-54.164259, -80.633528],
              [-53.987991, -80.222028],
              [-51.853134, -79.94773],
              [-50.991326, -79.614623],
              [-50.364595, -79.183487],
              [-49.914131, -78.811209],
              [-49.306959, -78.458569],
              [-48.660616, -78.047018],
              [-48.660616, -78.047019],
```

```
        [-48.151396, -78.04707],
        [-46.662857, -77.831476],
        [-45.154758, -78.04707],
      ],
    ],
    [
      [
        [-121.211511, -73.50099],
        [-119.918851, -73.657725],
        [-118.724143, -73.481353],
        [-119.292119, -73.834097],
        [-120.232217, -74.08881],
        [-121.62283, -74.010468],
        [-122.621735, -73.657778],
        [-122.621735, -73.657777],
        [-122.406245, -73.324619],
        [-121.211511, -73.50099],
      ],
    ],
    [
      [
        [-125.559566, -73.481353],
        [-124.031882, -73.873268],
        [-124.619469, -73.834097],
        [-125.912181, -73.736118],
        [-127.28313, -73.461769],
        [-127.28313, -73.461768],
        [-126.558472, -73.246226],
        [-125.559566, -73.481353],
      ],
    ],
    [
      [
        [-98.98155, -71.933334],
        [-97.884743, -72.070535],
        [-96.787937, -71.952971],
        [-96.20035, -72.521205],
        [-96.983765, -72.442864],
        [-98.198083, -72.482035],
        [-99.432013, -72.442864],
        [-100.783455, -72.50162],
        [-101.801868, -72.305663],
        [-102.330725, -71.894164],
        [-102.330725, -71.894164],
        [-101.703967, -71.717792],
        [-100.430919, -71.854993],
        [-98.98155, -71.933334],
      ],
    ],
    [
      [
        [-68.451346, -70.955823],
        [-68.333834, -71.406493],
        [-68.510128, -71.798407],
        [-68.784297, -72.170736],
        [-69.959471, -72.307885],
        [-71.075889, -72.503842],
        [-72.388134, -72.484257],
        [-71.8985, -72.092343],
        [-73.073622, -72.229492],
        [-74.19004, -72.366693],
        [-74.953895, -72.072757],
        [-75.012625, -71.661258],
        [-73.915819, -71.269345],
        [-73.915819, -71.269344],
        [-73.230331, -71.15178],
```

```
        [-72.074717, -71.190951],
        [-71.780962, -70.681473],
        [-71.72218, -70.309196],
        [-71.741791, -69.505782],
        [-71.173815, -69.035475],
        [-70.253252, -68.87874],
        [-69.724447, -69.251017],
        [-69.489422, -69.623346],
        [-69.058518, -70.074016],
        [-68.725541, -70.505153],
        [-68.451346, -70.955823],
      ],
    ],
    [
      [
        [-58.614143, -64.152467],
        [-59.045073, -64.36801],
        [-59.789342, -64.211223],
        [-60.611928, -64.309202],
        [-61.297416, -64.54433],
        [-62.0221, -64.799094],
        [-62.51176, -65.09303],
        [-62.648858, -65.484942],
        [-62.590128, -65.857219],
        [-62.120079, -66.190326],
        [-62.805567, -66.425505],
        [-63.74569, -66.503847],
        [-64.294106, -66.837004],
        [-64.881693, -67.150474],
        [-65.508425, -67.58161],
        [-65.665082, -67.953887],
        [-65.312545, -68.365335],
        [-64.783715, -68.678908],
        [-63.961103, -68.913984],
        [-63.1973, -69.227556],
        [-62.785955, -69.619419],
        [-62.570516, -69.991747],
        [-62.276736, -70.383661],
        [-61.806661, -70.716768],
        [-61.512906, -71.089045],
        [-61.375809, -72.010074],
        [-61.081977, -72.382351],
        [-61.003661, -72.774265],
        [-60.690269, -73.166179],
        [-60.827367, -73.695242],
        [-61.375809, -74.106742],
        [-61.96337, -74.439848],
        [-63.295201, -74.576997],
        [-63.74569, -74.92974],
        [-64.352836, -75.262847],
        [-65.860987, -75.635124],
        [-67.192818, -75.79191],
        [-68.446282, -76.007452],
        [-69.797724, -76.222995],
        [-70.600724, -76.634494],
        [-72.206776, -76.673665],
        [-73.969536, -76.634494],
        [-75.555977, -76.712887],
        [-77.24037, -76.712887],
        [-76.926979, -77.104802],
        [-75.399294, -77.28107],
        [-74.282876, -77.55542],
        [-73.656119, -77.908112],
        [-74.772536, -78.221633],
        [-76.4961, -78.123654],
        [-77.925858, -78.378419],
```

```
[-77.984666, -78.789918],
[-78.023785, -79.181833],
[-76.848637, -79.514939],
[-76.633224, -79.887216],
[-75.360097, -80.259545],
[-73.244852, -80.416331],
[-71.442946, -80.69063],
[-70.013163, -81.004151],
[-68.191646, -81.317672],
[-65.704279, -81.474458],
[-63.25603, -81.748757],
[-61.552026, -82.042692],
[-59.691416, -82.37585],
[-58.712121, -82.846106],
[-58.222487, -83.218434],
[-57.008117, -82.865691],
[-55.362894, -82.571755],
[-53.619771, -82.258235],
[-51.543644, -82.003521],
[-49.76135, -81.729171],
[-47.273931, -81.709586],
[-44.825708, -81.846735],
[-42.808363, -82.081915],
[-42.16202, -81.65083],
[-40.771433, -81.356894],
[-38.244818, -81.337309],
[-36.26667, -81.121715],
[-34.386397, -80.906172],
[-32.310296, -80.769023],
[-30.097098, -80.592651],
[-28.549802, -80.337938],
[-29.254901, -79.985195],
[-29.685805, -79.632503],
[-29.685805, -79.260226],
[-31.624808, -79.299397],
[-33.681324, -79.456132],
[-35.639912, -79.456132],
[-35.914107, -79.083855],
[-35.77701, -78.339248],
[-35.326546, -78.123654],
[-33.896763, -77.888526],
[-32.212369, -77.65345],
[-30.998051, -77.359515],
[-29.783732, -77.065579],
[-28.882779, -76.673665],
[-27.511752, -76.497345],
[-26.160336, -76.360144],
[-25.474822, -76.281803],
[-23.927552, -76.24258],
[-22.458598, -76.105431],
[-21.224694, -75.909474],
[-20.010375, -75.674346],
[-18.913543, -75.439218],
[-17.522982, -75.125698],
[-16.641589, -74.79254],
[-15.701491, -74.498604],
[-15.40771, -74.106742],
[-16.46532, -73.871614],
[-16.112784, -73.460114],
[-15.446855, -73.146542],
[-14.408805, -72.950585],
[-13.311973, -72.715457],
[-12.293508, -72.401936],
[-11.510067, -72.010074],
[-11.020433, -71.539767],
[-10.295774, -71.265416],
```

```
[-9.101015, -71.324224],
[-8.611381, -71.65733],
[-7.416622, -71.696501],
[-7.377451, -71.324224],
[-6.868232, -70.93231],
[-5.790985, -71.030289],
[-5.536375, -71.402617],
[-4.341667, -71.461373],
[-3.048981, -71.285053],
[-1.795492, -71.167438],
[-0.659489, -71.226246],
[-0.228637, -71.637745],
[0.868195, -71.304639],
[1.886686, -71.128267],
[3.022638, -70.991118],
[4.139055, -70.853917],
[5.157546, -70.618789],
[6.273912, -70.462055],
[7.13572, -70.246512],
[7.742866, -69.893769],
[8.48711, -70.148534],
[9.525135, -70.011333],
[10.249845, -70.48164],
[10.817821, -70.834332],
[11.953824, -70.638375],
[12.404287, -70.246512],
[13.422778, -69.972162],
[14.734998, -70.030918],
[15.126757, -70.403247],
[15.949342, -70.030918],
[17.026589, -69.913354],
[18.201711, -69.874183],
[19.259373, -69.893769],
[20.375739, -70.011333],
[21.452985, -70.07014],
[21.923034, -70.403247],
[22.569403, -70.697182],
[23.666184, -70.520811],
[24.841357, -70.48164],
[25.977309, -70.48164],
[27.093726, -70.462055],
[28.09258, -70.324854],
[29.150242, -70.20729],
[30.031583, -69.93294],
[30.971733, -69.75662],
[31.990172, -69.658641],
[32.754053, -69.384291],
[33.302443, -68.835642],
[33.870419, -68.502588],
[34.908495, -68.659271],
[35.300202, -69.012014],
[36.16201, -69.247142],
[37.200035, -69.168748],
[37.905108, -69.52144],
[38.649404, -69.776205],
[39.667894, -69.541077],
[40.020431, -69.109941],
[40.921358, -68.933621],
[41.959434, -68.600514],
[42.938702, -68.463313],
[44.113876, -68.267408],
[44.897291, -68.051866],
[45.719928, -67.816738],
[46.503343, -67.601196],
[47.44344, -67.718759],
[48.344419, -67.366068],
```

```
            [48.990736, -67.091718],
            [49.930885, -67.111303],
            [50.753471, -66.876175],
            [50.949325, -66.523484],
            [51.791547, -66.249133],
            [52.614133, -66.053176],
            [53.613038, -65.89639],
            [54.53355, -65.818049],
            [55.414943, -65.876805],
            [56.355041, -65.974783],
            [57.158093, -66.249133],
            [57.255968, -66.680218],
            [58.137361, -67.013324],
            [58.744508, -67.287675],
            [59.939318, -67.405239],
            [60.605221, -67.679589],
            [61.427806, -67.953887],
            [62.387489, -68.012695],
            [63.19049, -67.816738],
            [64.052349, -67.405239],
            [64.992447, -67.620729],
            [65.971715, -67.738345],
            [66.911864, -67.855909],
            [67.891133, -67.934302],
            [68.890038, -67.934302],
            [69.712624, -68.972791],
            [69.673453, -69.227556],
            [69.555941, -69.678226],
            [68.596258, -69.93294],
            [67.81274, -70.305268],
            [67.949889, -70.697182],
            [69.066307, -70.677545],
            [68.929157, -71.069459],
            [68.419989, -71.441788],
            [67.949889, -71.853287],
            [68.71377, -72.166808],
            [69.869307, -72.264787],
            [71.024895, -72.088415],
            [71.573285, -71.696501],
            [71.906288, -71.324224],
            [72.454627, -71.010703],
            [73.08141, -70.716768],
            [73.33602, -70.364024],
            [73.864877, -69.874183],
            [74.491557, -69.776205],
            [75.62756, -69.737034],
            [76.626465, -69.619419],
            [77.644904, -69.462684],
            [78.134539, -69.07077],
            [78.428371, -68.698441],
            [79.113859, -68.326216],
            [80.093127, -68.071503],
            [80.93535, -67.875546],
            [81.483792, -67.542388],
            [82.051767, -67.366068],
            [82.776426, -67.209282],
            [83.775331, -67.30726],
            [84.676206, -67.209282],
            [85.655527, -67.091718],
            [86.752359, -67.150474],
            [87.477017, -66.876175],
            [87.986289, -66.209911],
            [88.358411, -66.484261],
            [88.828408, -66.954568],
            [89.67063, -67.150474],
            [90.630365, -67.228867],
```

```
[91.5901, -67.111303],
[92.608539, -67.189696],
[93.548637, -67.209282],
[94.17542, -67.111303],
[95.017591, -67.170111],
[95.781472, -67.385653],
[96.682399, -67.248504],
[97.759646, -67.248504],
[98.68021, -67.111303],
[99.718182, -67.248504],
[100.384188, -66.915346],
[100.893356, -66.58224],
[101.578896, -66.30789],
[102.832411, -65.563284],
[103.478676, -65.700485],
[104.242557, -65.974783],
[104.90846, -66.327527],
[106.181561, -66.934931],
[107.160881, -66.954568],
[108.081393, -66.954568],
[109.15864, -66.837004],
[110.235835, -66.699804],
[111.058472, -66.425505],
[111.74396, -66.13157],
[112.860378, -66.092347],
[113.604673, -65.876805],
[114.388088, -66.072762],
[114.897308, -66.386283],
[115.602381, -66.699804],
[116.699161, -66.660633],
[117.384701, -66.915346],
[118.57946, -67.170111],
[119.832924, -67.268089],
[120.871, -67.189696],
[121.654415, -66.876175],
[122.320369, -66.562654],
[123.221296, -66.484261],
[124.122274, -66.621462],
[125.160247, -66.719389],
[126.100396, -66.562654],
[127.001427, -66.562654],
[127.882768, -66.660633],
[128.80328, -66.758611],
[129.704259, -66.58224],
[130.781454, -66.425505],
[131.799945, -66.386283],
[132.935896, -66.386283],
[133.85646, -66.288304],
[134.757387, -66.209963],
[135.031582, -65.72007],
[135.070753, -65.308571],
[135.697485, -65.582869],
[135.873805, -66.033591],
[136.206705, -66.44509],
[136.618049, -66.778197],
[137.460271, -66.954568],
[138.596223, -66.895761],
[139.908442, -66.876175],
[140.809421, -66.817367],
[142.121692, -66.817367],
[143.061842, -66.797782],
[144.374061, -66.837004],
[145.490427, -66.915346],
[146.195552, -67.228867],
[145.999699, -67.601196],
[146.646067, -67.895131],
```

```
            [147.723263, -68.130259],
            [148.839629, -68.385024],
            [150.132314, -68.561292],
            [151.483705, -68.71813],
            [152.502247, -68.874813],
            [153.638199, -68.894502],
            [154.284567, -68.561292],
            [155.165857, -68.835642],
            [155.92979, -69.149215],
            [156.811132, -69.384291],
            [158.025528, -69.482269],
            [159.181013, -69.599833],
            [159.670699, -69.991747],
            [160.80665, -70.226875],
            [161.570479, -70.579618],
            [162.686897, -70.736353],
            [163.842434, -70.716768],
            [164.919681, -70.775524],
            [166.11444, -70.755938],
            [167.309095, -70.834332],
            [168.425616, -70.971481],
            [169.463589, -71.20666],
            [170.501665, -71.402617],
            [171.20679, -71.696501],
            [171.089227, -72.088415],
            [170.560422, -72.441159],
            [170.109958, -72.891829],
            [169.75737, -73.24452],
            [169.287321, -73.65602],
            [167.975101, -73.812806],
            [167.387489, -74.165498],
            [166.094803, -74.38104],
            [165.644391, -74.772954],
            [164.958851, -75.145283],
            [164.234193, -75.458804],
            [163.822797, -75.870303],
            [163.568239, -76.24258],
            [163.47026, -76.693302],
            [163.489897, -77.065579],
            [164.057873, -77.457442],
            [164.273363, -77.82977],
            [164.743464, -78.182514],
            [166.604126, -78.319611],
            [166.995781, -78.750748],
            [165.193876, -78.907483],
            [163.666217, -79.123025],
            [161.766385, -79.162248],
            [160.924162, -79.730482],
            [160.747894, -80.200737],
            [160.316964, -80.573066],
            [159.788211, -80.945395],
            [161.120016, -81.278501],
            [161.629287, -81.690001],
            [162.490992, -82.062278],
            [163.705336, -82.395435],
            [165.095949, -82.708956],
            [166.604126, -83.022477],
            [168.895665, -83.335998],
            [169.404782, -83.825891],
            [172.283934, -84.041433],
            [172.477049, -84.117914],
            [173.224083, -84.41371],
            [175.985672, -84.158997],
            [178.277212, -84.472518],
            [180, -84.71338],
            [-179.942499, -84.721443],
```

```
[-179.058677, -84.139412],
[-177.256772, -84.452933],
[-177.140807, -84.417941],
[-176.084673, -84.099259],
[-175.947235, -84.110449],
[-175.829882, -84.117914],
[-174.382503, -84.534323],
[-173.116559, -84.117914],
[-172.889106, -84.061019],
[-169.951223, -83.884647],
[-168.999989, -84.117914],
[-168.530199, -84.23739],
[-167.022099, -84.570497],
[-164.182144, -84.82521],
[-161.929775, -85.138731],
[-158.07138, -85.37391],
[-155.192253, -85.09956],
[-150.942099, -85.295517],
[-148.533073, -85.609038],
[-145.888918, -85.315102],
[-143.107718, -85.040752],
[-142.892279, -84.570497],
[-146.829068, -84.531274],
[-150.060732, -84.296146],
[-150.902928, -83.904232],
[-153.586201, -83.68869],
[-153.409907, -83.23802],
[-153.037759, -82.82652],
[-152.665637, -82.454192],
[-152.861517, -82.042692],
[-154.526299, -81.768394],
[-155.29018, -81.41565],
[-156.83745, -81.102129],
[-154.408787, -81.160937],
[-152.097662, -81.004151],
[-150.648293, -81.337309],
[-148.865998, -81.043373],
[-147.22075, -80.671045],
[-146.417749, -80.337938],
[-146.770286, -79.926439],
[-148.062947, -79.652089],
[-149.531901, -79.358205],
[-151.588416, -79.299397],
[-153.390322, -79.162248],
[-155.329376, -79.064269],
[-155.975668, -78.69194],
[-157.268302, -78.378419],
[-158.051768, -78.025676],
[-158.365134, -76.889207],
[-157.875474, -76.987238],
[-156.974573, -77.300759],
[-155.329376, -77.202728],
[-153.742832, -77.065579],
[-152.920247, -77.496664],
[-151.33378, -77.398737],
[-150.00195, -77.183143],
[-148.748486, -76.908845],
[-147.612483, -76.575738],
[-146.104409, -76.47776],
[-146.143528, -76.105431],
[-146.496091, -75.733154],
[-146.20231, -75.380411],
[-144.909624, -75.204039],
[-144.322037, -75.537197],
[-142.794353, -75.34124],
[-141.638764, -75.086475],
```

```
[-140.209007, -75.06689],
[-138.85759, -74.968911],
[-137.5062, -74.733783],
[-136.428901, -74.518241],
[-135.214583, -74.302699],
[-134.431194, -74.361455],
[-133.745654, -74.439848],
[-132.257168, -74.302699],
[-130.925311, -74.479019],
[-129.554284, -74.459433],
[-128.242038, -74.322284],
[-126.890622, -74.420263],
[-125.402082, -74.518241],
[-124.011496, -74.479019],
[-122.562152, -74.498604],
[-121.073613, -74.518241],
[-119.70256, -74.479019],
[-118.684145, -74.185083],
[-117.469801, -74.028348],
[-116.216312, -74.243891],
[-115.021552, -74.067519],
[-113.944331, -73.714828],
[-113.297988, -74.028348],
[-112.945452, -74.38104],
[-112.299083, -74.714198],
[-111.261059, -74.420263],
[-110.066325, -74.79254],
[-108.714909, -74.910103],
[-107.559346, -75.184454],
[-106.149148, -75.125698],
[-104.876074, -74.949326],
[-103.367949, -74.988497],
[-102.016507, -75.125698],
[-100.645531, -75.302018],
[-100.1167, -74.870933],
[-100.763043, -74.537826],
[-101.252703, -74.185083],
[-102.545337, -74.106742],
[-103.113313, -73.734413],
[-103.328752, -73.362084],
[-103.681289, -72.61753],
[-102.917485, -72.754679],
[-101.60524, -72.813436],
[-100.312528, -72.754679],
[-99.13738, -72.911414],
[-98.118889, -73.20535],
[-97.688037, -73.558041],
[-96.336595, -73.616849],
[-95.043961, -73.4797],
[-93.672907, -73.283743],
[-92.439003, -73.166179],
[-91.420564, -73.401307],
[-90.088733, -73.322914],
[-89.226951, -72.558722],
[-88.423951, -73.009393],
[-87.268337, -73.185764],
[-86.014822, -73.087786],
[-85.192236, -73.4797],
[-83.879991, -73.518871],
[-82.665646, -73.636434],
[-81.470913, -73.851977],
[-80.687447, -73.4797],
[-80.295791, -73.126956],
[-79.296886, -73.518871],
[-77.925858, -73.420892],
[-76.907367, -73.636434],
```

```
            [-76.221879, -73.969541],
            [-74.890049, -73.871614],
            [-73.852024, -73.65602],
            [-72.833533, -73.401307],
            [-71.619215, -73.264157],
            [-70.209042, -73.146542],
            [-68.935916, -73.009393],
            [-67.956622, -72.79385],
            [-67.369061, -72.480329],
            [-67.134036, -72.049244],
            [-67.251548, -71.637745],
            [-67.56494, -71.245831],
            [-67.917477, -70.853917],
            [-68.230843, -70.462055],
            [-68.485452, -70.109311],
            [-68.544209, -69.717397],
            [-68.446282, -69.325535],
            [-67.976233, -68.953206],
            [-67.5845, -68.541707],
            [-67.427843, -68.149844],
            [-67.62367, -67.718759],
            [-67.741183, -67.326845],
            [-67.251548, -66.876175],
            [-66.703184, -66.58224],
            [-66.056815, -66.209963],
            [-65.371327, -65.89639],
            [-64.568276, -65.602506],
            [-64.176542, -65.171423],
            [-63.628152, -64.897073],
            [-63.001394, -64.642308],
            [-62.041686, -64.583552],
            [-61.414928, -64.270031],
            [-60.709855, -64.074074],
            [-59.887269, -63.95651],
            [-59.162585, -63.701745],
            [-58.594557, -63.388224],
            [-57.811143, -63.27066],
            [-57.223582, -63.525425],
            [-57.59573, -63.858532],
            [-58.614143, -64.152467],
          ],
        ],
      ],
    },
    id: "ATA",
  },
  {
    type: "Feature",
    properties: {
      name: "French Southern and Antarctic Lands",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [68.935, -48.625],
          [69.58, -48.94],
          [70.525, -49.065],
          [70.56, -49.255],
          [70.28, -49.71],
          [68.745, -49.775],
          [68.72, -49.2425],
          [68.8675, -48.83],
          [68.935, -48.625],
        ],
      ],
```

```
      },
      id: "ATF",
    },
    {
      type: "Feature",
      properties: {
        name: "Australia",
      },
      geometry: {
        type: "MultiPolygon",
        coordinates: [
          [
            [
              [145.397978, -40.792549],
              [146.364121, -41.137695],
              [146.908584, -41.000546],
              [147.689259, -40.808258],
              [148.289068, -40.875438],
              [148.359865, -42.062445],
              [148.017301, -42.407024],
              [147.914052, -43.211522],
              [147.564564, -42.937689],
              [146.870343, -43.634597],
              [146.663327, -43.580854],
              [146.048378, -43.549745],
              [145.43193, -42.693776],
              [145.29509, -42.03361],
              [144.718071, -41.162552],
              [144.743755, -40.703975],
              [145.397978, -40.792549],
            ],
          ],
          [
            [
              [143.561811, -13.763656],
              [143.922099, -14.548311],
              [144.563714, -14.171176],
              [144.894908, -14.594458],
              [145.374724, -14.984976],
              [145.271991, -15.428205],
              [145.48526, -16.285672],
              [145.637033, -16.784918],
              [145.888904, -16.906926],
              [146.160309, -17.761655],
              [146.063674, -18.280073],
              [146.387478, -18.958274],
              [147.471082, -19.480723],
              [148.177602, -19.955939],
              [148.848414, -20.39121],
              [148.717465, -20.633469],
              [149.28942, -21.260511],
              [149.678337, -22.342512],
              [150.077382, -22.122784],
              [150.482939, -22.556142],
              [150.727265, -22.402405],
              [150.899554, -23.462237],
              [151.609175, -24.076256],
              [152.07354, -24.457887],
              [152.855197, -25.267501],
              [153.136162, -26.071173],
              [153.161949, -26.641319],
              [153.092909, -27.2603],
              [153.569469, -28.110067],
              [153.512108, -28.995077],
              [153.339095, -29.458202],
              [153.069241, -30.35024],
```

```
[153.089602, -30.923642],
[152.891578, -31.640446],
[152.450002, -32.550003],
[151.709117, -33.041342],
[151.343972, -33.816023],
[151.010555, -34.31036],
[150.714139, -35.17346],
[150.32822, -35.671879],
[150.075212, -36.420206],
[149.946124, -37.109052],
[149.997284, -37.425261],
[149.423882, -37.772681],
[148.304622, -37.809061],
[147.381733, -38.219217],
[146.922123, -38.606532],
[146.317922, -39.035757],
[145.489652, -38.593768],
[144.876976, -38.417448],
[145.032212, -37.896188],
[144.485682, -38.085324],
[143.609974, -38.809465],
[142.745427, -38.538268],
[142.17833, -38.380034],
[141.606582, -38.308514],
[140.638579, -38.019333],
[139.992158, -37.402936],
[139.806588, -36.643603],
[139.574148, -36.138362],
[139.082808, -35.732754],
[138.120748, -35.612296],
[138.449462, -35.127261],
[138.207564, -34.384723],
[137.71917, -35.076825],
[136.829406, -35.260535],
[137.352371, -34.707339],
[137.503886, -34.130268],
[137.890116, -33.640479],
[137.810328, -32.900007],
[136.996837, -33.752771],
[136.372069, -34.094766],
[135.989043, -34.890118],
[135.208213, -34.47867],
[135.239218, -33.947953],
[134.613417, -33.222778],
[134.085904, -32.848072],
[134.273903, -32.617234],
[132.990777, -32.011224],
[132.288081, -31.982647],
[131.326331, -31.495803],
[129.535794, -31.590423],
[128.240938, -31.948489],
[127.102867, -32.282267],
[126.148714, -32.215966],
[125.088623, -32.728751],
[124.221648, -32.959487],
[124.028947, -33.483847],
[123.659667, -33.890179],
[122.811036, -33.914467],
[122.183064, -34.003402],
[121.299191, -33.821036],
[120.580268, -33.930177],
[119.893695, -33.976065],
[119.298899, -34.509366],
[119.007341, -34.464149],
[118.505718, -34.746819],
[118.024972, -35.064733],
```

```
[117.295507, -35.025459],
[116.625109, -35.025097],
[115.564347, -34.386428],
[115.026809, -34.196517],
[115.048616, -33.623425],
[115.545123, -33.487258],
[115.714674, -33.259572],
[115.679379, -32.900369],
[115.801645, -32.205062],
[115.689611, -31.612437],
[115.160909, -30.601594],
[114.997043, -30.030725],
[115.040038, -29.461095],
[114.641974, -28.810231],
[114.616498, -28.516399],
[114.173579, -28.118077],
[114.048884, -27.334765],
[113.477498, -26.543134],
[113.338953, -26.116545],
[113.778358, -26.549025],
[113.440962, -25.621278],
[113.936901, -25.911235],
[114.232852, -26.298446],
[114.216161, -25.786281],
[113.721255, -24.998939],
[113.625344, -24.683971],
[113.393523, -24.384764],
[113.502044, -23.80635],
[113.706993, -23.560215],
[113.843418, -23.059987],
[113.736552, -22.475475],
[114.149756, -21.755881],
[114.225307, -22.517488],
[114.647762, -21.82952],
[115.460167, -21.495173],
[115.947373, -21.068688],
[116.711615, -20.701682],
[117.166316, -20.623599],
[117.441545, -20.746899],
[118.229559, -20.374208],
[118.836085, -20.263311],
[118.987807, -20.044203],
[119.252494, -19.952942],
[119.805225, -19.976506],
[120.85622, -19.683708],
[121.399856, -19.239756],
[121.655138, -18.705318],
[122.241665, -18.197649],
[122.286624, -17.798603],
[122.312772, -17.254967],
[123.012574, -16.4052],
[123.433789, -17.268558],
[123.859345, -17.069035],
[123.503242, -16.596506],
[123.817073, -16.111316],
[124.258287, -16.327944],
[124.379726, -15.56706],
[124.926153, -15.0751],
[125.167275, -14.680396],
[125.670087, -14.51007],
[125.685796, -14.230656],
[126.125149, -14.347341],
[126.142823, -14.095987],
[126.582589, -13.952791],
[127.065867, -13.817968],
[127.804633, -14.276906],
```

```
            [128.35969, -14.86917],
            [128.985543, -14.875991],
            [129.621473, -14.969784],
            [129.4096, -14.42067],
            [129.888641, -13.618703],
            [130.339466, -13.357376],
            [130.183506, -13.10752],
            [130.617795, -12.536392],
            [131.223495, -12.183649],
            [131.735091, -12.302453],
            [132.575298, -12.114041],
            [132.557212, -11.603012],
            [131.824698, -11.273782],
            [132.357224, -11.128519],
            [133.019561, -11.376411],
            [133.550846, -11.786515],
            [134.393068, -12.042365],
            [134.678632, -11.941183],
            [135.298491, -12.248606],
            [135.882693, -11.962267],
            [136.258381, -12.049342],
            [136.492475, -11.857209],
            [136.95162, -12.351959],
            [136.685125, -12.887223],
            [136.305407, -13.29123],
            [135.961758, -13.324509],
            [136.077617, -13.724278],
            [135.783836, -14.223989],
            [135.428664, -14.715432],
            [135.500184, -14.997741],
            [136.295175, -15.550265],
            [137.06536, -15.870762],
            [137.580471, -16.215082],
            [138.303217, -16.807604],
            [138.585164, -16.806622],
            [139.108543, -17.062679],
            [139.260575, -17.371601],
            [140.215245, -17.710805],
            [140.875463, -17.369069],
            [141.07111, -16.832047],
            [141.274095, -16.38887],
            [141.398222, -15.840532],
            [141.702183, -15.044921],
            [141.56338, -14.561333],
            [141.63552, -14.270395],
            [141.519869, -13.698078],
            [141.65092, -12.944688],
            [141.842691, -12.741548],
            [141.68699, -12.407614],
            [141.928629, -11.877466],
            [142.118488, -11.328042],
            [142.143706, -11.042737],
            [142.51526, -10.668186],
            [142.79731, -11.157355],
            [142.866763, -11.784707],
            [143.115947, -11.90563],
            [143.158632, -12.325656],
            [143.522124, -12.834358],
            [143.597158, -13.400422],
            [143.561811, -13.763656],
          ],
        ],
      ],
    },
    id: "AUS",
  },
```

```
    {
      type: "Feature",
      properties: {
        name: "Austria",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [16.979667, 48.123497],
            [16.903754, 47.714866],
            [16.340584, 47.712902],
            [16.534268, 47.496171],
            [16.202298, 46.852386],
            [16.011664, 46.683611],
            [15.137092, 46.658703],
            [14.632472, 46.431817],
            [13.806475, 46.509306],
            [12.376485, 46.767559],
            [12.153088, 47.115393],
            [11.164828, 46.941579],
            [11.048556, 46.751359],
            [10.442701, 46.893546],
            [9.932448, 46.920728],
            [9.47997, 47.10281],
            [9.632932, 47.347601],
            [9.594226, 47.525058],
            [9.896068, 47.580197],
            [10.402084, 47.302488],
            [10.544504, 47.566399],
            [11.426414, 47.523766],
            [12.141357, 47.703083],
            [12.62076, 47.672388],
            [12.932627, 47.467646],
            [13.025851, 47.637584],
            [12.884103, 48.289146],
            [13.243357, 48.416115],
            [13.595946, 48.877172],
            [14.338898, 48.555305],
            [14.901447, 48.964402],
            [15.253416, 49.039074],
            [16.029647, 48.733899],
            [16.499283, 48.785808],
            [16.960288, 48.596982],
            [16.879983, 48.470013],
            [16.979667, 48.123497],
          ],
        ],
      },
      id: "AUT",
    },
    {
      type: "Feature",
      properties: {
        name: "Azerbaijan",
      },
      geometry: {
        type: "MultiPolygon",
        coordinates: [
          [
            [
              [45.001987, 39.740004],
              [45.298145, 39.471751],
              [45.739978, 39.473999],
              [45.735379, 39.319719],
              [46.143623, 38.741201],
```

```
            [45.457722, 38.874139],
            [44.952688, 39.335765],
            [44.79399, 39.713003],
            [45.001987, 39.740004],
          ],
        ],
        [
          [
            [47.373315, 41.219732],
            [47.815666, 41.151416],
            [47.987283, 41.405819],
            [48.584353, 41.80887],
            [49.110264, 41.282287],
            [49.618915, 40.572924],
            [50.08483, 40.526157],
            [50.392821, 40.256561],
            [49.569202, 40.176101],
            [49.395259, 39.399482],
            [49.223228, 39.049219],
            [48.856532, 38.815486],
            [48.883249, 38.320245],
            [48.634375, 38.270378],
            [48.010744, 38.794015],
            [48.355529, 39.288765],
            [48.060095, 39.582235],
            [47.685079, 39.508364],
            [46.50572, 38.770605],
            [46.483499, 39.464155],
            [46.034534, 39.628021],
            [45.610012, 39.899994],
            [45.891907, 40.218476],
            [45.359175, 40.561504],
            [45.560351, 40.81229],
            [45.179496, 40.985354],
            [44.97248, 41.248129],
            [45.217426, 41.411452],
            [45.962601, 41.123873],
            [46.501637, 41.064445],
            [46.637908, 41.181673],
            [46.145432, 41.722802],
            [46.404951, 41.860675],
            [46.686071, 41.827137],
            [47.373315, 41.219732],
          ],
        ],
      ],
    },
    id: "AZE",
  },
  {
    type: "Feature",
    properties: {
      name: "Burundi",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [29.339998, -4.499983],
          [29.276384, -3.293907],
          [29.024926, -2.839258],
          [29.632176, -2.917858],
          [29.938359, -2.348487],
          [30.469696, -2.413858],
          [30.527677, -2.807632],
          [30.743013, -3.034285],
```

```
          [30.752263, -3.35933],
          [30.50556, -3.568567],
          [30.116333, -4.090138],
          [29.753512, -4.452389],
          [29.339998, -4.499983],
        ],
      ],
    },
    id: "BDI",
  },
  {
    type: "Feature",
    properties: {
      name: "Belgium",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [3.314971, 51.345781],
          [4.047071, 51.267259],
          [4.973991, 51.475024],
          [5.606976, 51.037298],
          [6.156658, 50.803721],
          [6.043073, 50.128052],
          [5.782417, 50.090328],
          [5.674052, 49.529484],
          [4.799222, 49.985373],
          [4.286023, 49.907497],
          [3.588184, 50.378992],
          [3.123252, 50.780363],
          [2.658422, 50.796848],
          [2.513573, 51.148506],
          [3.314971, 51.345781],
        ],
      ],
    },
    id: "BEL",
  },
  {
    type: "Feature",
    properties: {
      name: "Benin",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [2.691702, 6.258817],
          [1.865241, 6.142158],
          [1.618951, 6.832038],
          [1.664478, 9.12859],
          [1.463043, 9.334624],
          [1.425061, 9.825395],
          [1.077795, 10.175607],
          [0.772336, 10.470808],
          [0.899563, 10.997339],
          [1.24347, 11.110511],
          [1.447178, 11.547719],
          [1.935986, 11.64115],
          [2.154474, 11.94015],
          [2.490164, 12.233052],
          [2.848643, 12.235636],
          [3.61118, 11.660167],
          [3.572216, 11.327939],
          [3.797112, 10.734746],
```

```
          [3.60007, 10.332186],
          [3.705438, 10.06321],
          [3.220352, 9.444153],
          [2.912308, 9.137608],
          [2.723793, 8.506845],
          [2.749063, 7.870734],
          [2.691702, 6.258817],
        ],
      ],
    },
    id: "BEN",
  },
  {
    type: "Feature",
    properties: {
      name: "Burkina Faso",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-2.827496, 9.642461],
          [-3.511899, 9.900326],
          [-3.980449, 9.862344],
          [-4.330247, 9.610835],
          [-4.779884, 9.821985],
          [-4.954653, 10.152714],
          [-5.404342, 10.370737],
          [-5.470565, 10.95127],
          [-5.197843, 11.375146],
          [-5.220942, 11.713859],
          [-4.427166, 12.542646],
          [-4.280405, 13.228444],
          [-4.006391, 13.472485],
          [-3.522803, 13.337662],
          [-3.103707, 13.541267],
          [-2.967694, 13.79815],
          [-2.191825, 14.246418],
          [-2.001035, 14.559008],
          [-1.066363, 14.973815],
          [-0.515854, 15.116158],
          [-0.266257, 14.924309],
          [0.374892, 14.928908],
          [0.295646, 14.444235],
          [0.429928, 13.988733],
          [0.993046, 13.33575],
          [1.024103, 12.851826],
          [2.177108, 12.625018],
          [2.154474, 11.94015],
          [1.935986, 11.64115],
          [1.447178, 11.547719],
          [1.24347, 11.110511],
          [0.899563, 10.997339],
          [0.023803, 11.018682],
          [-0.438702, 11.098341],
          [-0.761576, 10.93693],
          [-1.203358, 11.009819],
          [-2.940409, 10.96269],
          [-2.963896, 10.395335],
          [-2.827496, 9.642461],
        ],
      ],
    },
    id: "BFA",
  },
  {
```

```
      type: "Feature",
      properties: {
        name: "Bangladesh",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [92.672721, 22.041239],
            [92.652257, 21.324048],
            [92.303234, 21.475485],
            [92.368554, 20.670883],
            [92.082886, 21.192195],
            [92.025215, 21.70157],
            [91.834891, 22.182936],
            [91.417087, 22.765019],
            [90.496006, 22.805017],
            [90.586957, 22.392794],
            [90.272971, 21.836368],
            [89.847467, 22.039146],
            [89.70205, 21.857116],
            [89.418863, 21.966179],
            [89.031961, 22.055708],
            [88.876312, 22.879146],
            [88.52977, 23.631142],
            [88.69994, 24.233715],
            [88.084422, 24.501657],
            [88.306373, 24.866079],
            [88.931554, 25.238692],
            [88.209789, 25.768066],
            [88.563049, 26.446526],
            [89.355094, 26.014407],
            [89.832481, 25.965082],
            [89.920693, 25.26975],
            [90.872211, 25.132601],
            [91.799596, 25.147432],
            [92.376202, 24.976693],
            [91.915093, 24.130414],
            [91.46773, 24.072639],
            [91.158963, 23.503527],
            [91.706475, 22.985264],
            [91.869928, 23.624346],
            [92.146035, 23.627499],
            [92.672721, 22.041239],
          ],
        ],
      },
      id: "BGD",
    },
    {
      type: "Feature",
      properties: {
        name: "Bulgaria",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [22.65715, 44.234923],
            [22.944832, 43.823785],
            [23.332302, 43.897011],
            [24.100679, 43.741051],
            [25.569272, 43.688445],
            [26.065159, 43.943494],
            [27.2424, 44.175986],
            [27.970107, 43.812468],
```

```
          [28.558081, 43.707462],
          [28.039095, 43.293172],
          [27.673898, 42.577892],
          [27.99672, 42.007359],
          [27.135739, 42.141485],
          [26.117042, 41.826905],
          [26.106138, 41.328899],
          [25.197201, 41.234486],
          [24.492645, 41.583896],
          [23.692074, 41.309081],
          [22.952377, 41.337994],
          [22.881374, 41.999297],
          [22.380526, 42.32026],
          [22.545012, 42.461362],
          [22.436595, 42.580321],
          [22.604801, 42.898519],
          [22.986019, 43.211161],
          [22.500157, 43.642814],
          [22.410446, 44.008063],
          [22.65715, 44.234923],
        ],
      ],
    },
    id: "BGR",
  },
  {
    type: "Feature",
    properties: {
      name: "The Bahamas",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [-77.53466, 23.75975],
            [-77.78, 23.71],
            [-78.03405, 24.28615],
            [-78.40848, 24.57564],
            [-78.19087, 25.2103],
            [-77.89, 25.17],
            [-77.54, 24.34],
            [-77.53466, 23.75975],
          ],
        ],
        [
          [
            [-77.82, 26.58],
            [-78.91, 26.42],
            [-78.98, 26.79],
            [-78.51, 26.87],
            [-77.85, 26.84],
            [-77.82, 26.58],
          ],
        ],
        [
          [
            [-77, 26.59],
            [-77.17255, 25.87918],
            [-77.35641, 26.00735],
            [-77.34, 26.53],
            [-77.78802, 26.92516],
            [-77.79, 27.04],
            [-77, 26.59],
          ],
        ],
```

```
        ],
      },
      id: "BHS",
    },
    {
      type: "Feature",
      properties: {
        name: "Bosnia and Herzegovina",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [19.005486, 44.860234],
            [19.36803, 44.863],
            [19.11761, 44.42307],
            [19.59976, 44.03847],
            [19.454, 43.5681],
            [19.21852, 43.52384],
            [19.03165, 43.43253],
            [18.70648, 43.20011],
            [18.56, 42.65],
            [17.674922, 43.028563],
            [17.297373, 43.446341],
            [16.916156, 43.667722],
            [16.456443, 44.04124],
            [16.23966, 44.351143],
            [15.750026, 44.818712],
            [15.959367, 45.233777],
            [16.318157, 45.004127],
            [16.534939, 45.211608],
            [17.002146, 45.233777],
            [17.861783, 45.06774],
            [18.553214, 45.08159],
            [19.005486, 44.860234],
          ],
        ],
      },
      id: "BIH",
    },
    {
      type: "Feature",
      properties: {
        name: "Belarus",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [23.484128, 53.912498],
            [24.450684, 53.905702],
            [25.536354, 54.282423],
            [25.768433, 54.846963],
            [26.588279, 55.167176],
            [26.494331, 55.615107],
            [27.10246, 55.783314],
            [28.176709, 56.16913],
            [29.229513, 55.918344],
            [29.371572, 55.670091],
            [29.896294, 55.789463],
            [30.873909, 55.550976],
            [30.971836, 55.081548],
            [30.757534, 54.811771],
            [31.384472, 54.157056],
            [31.791424, 53.974639],
            [31.731273, 53.794029],
```

```
          [32.405599, 53.618045],
          [32.693643, 53.351421],
          [32.304519, 53.132726],
          [31.497644, 53.167427],
          [31.305201, 53.073996],
          [31.540018, 52.742052],
          [31.785998, 52.101678],
          [30.927549, 52.042353],
          [30.619454, 51.822806],
          [30.555117, 51.319503],
          [30.157364, 51.416138],
          [29.254938, 51.368234],
          [28.992835, 51.602044],
          [28.617613, 51.427714],
          [28.241615, 51.572227],
          [27.454066, 51.592303],
          [26.337959, 51.832289],
          [25.327788, 51.910656],
          [24.553106, 51.888461],
          [24.005078, 51.617444],
          [23.527071, 51.578454],
          [23.508002, 52.023647],
          [23.199494, 52.486977],
          [23.799199, 52.691099],
          [23.804935, 53.089731],
          [23.527536, 53.470122],
          [23.484128, 53.912498],
        ],
      ],
    },
    id: "BLR",
  },
  {
    type: "Feature",
    properties: {
      name: "Belize",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-89.14308, 17.808319],
          [-89.150909, 17.955468],
          [-89.029857, 18.001511],
          [-88.848344, 17.883198],
          [-88.490123, 18.486831],
          [-88.300031, 18.499982],
          [-88.296336, 18.353273],
          [-88.106813, 18.348674],
          [-88.123479, 18.076675],
          [-88.285355, 17.644143],
          [-88.197867, 17.489475],
          [-88.302641, 17.131694],
          [-88.239518, 17.036066],
          [-88.355428, 16.530774],
          [-88.551825, 16.265467],
          [-88.732434, 16.233635],
          [-88.930613, 15.887273],
          [-89.229122, 15.886938],
          [-89.150806, 17.015577],
          [-89.14308, 17.808319],
        ],
      ],
    },
    id: "BLZ",
  },
```

```
{
  type: "Feature",
  properties: {
    name: "Bolivia",
  },
  geometry: {
    type: "Polygon",
    coordinates: [
      [
        [-62.846468, -22.034985],
        [-63.986838, -21.993644],
        [-64.377021, -22.798091],
        [-64.964892, -22.075862],
        [-66.273339, -21.83231],
        [-67.106674, -22.735925],
        [-67.82818, -22.872919],
        [-68.219913, -21.494347],
        [-68.757167, -20.372658],
        [-68.442225, -19.405068],
        [-68.966818, -18.981683],
        [-69.100247, -18.260125],
        [-69.590424, -17.580012],
        [-68.959635, -16.500698],
        [-69.389764, -15.660129],
        [-69.160347, -15.323974],
        [-69.339535, -14.953195],
        [-68.948887, -14.453639],
        [-68.929224, -13.602684],
        [-68.88008, -12.899729],
        [-68.66508, -12.5613],
        [-69.529678, -10.951734],
        [-68.786158, -11.03638],
        [-68.271254, -11.014521],
        [-68.048192, -10.712059],
        [-67.173801, -10.306812],
        [-66.646908, -9.931331],
        [-65.338435, -9.761988],
        [-65.444837, -10.511451],
        [-65.321899, -10.895872],
        [-65.402281, -11.56627],
        [-64.316353, -12.461978],
        [-63.196499, -12.627033],
        [-62.80306, -13.000653],
        [-62.127081, -13.198781],
        [-61.713204, -13.489202],
        [-61.084121, -13.479384],
        [-60.503304, -13.775955],
        [-60.459198, -14.354007],
        [-60.264326, -14.645979],
        [-60.251149, -15.077219],
        [-60.542966, -15.09391],
        [-60.15839, -16.258284],
        [-58.24122, -16.299573],
        [-58.388058, -16.877109],
        [-58.280804, -17.27171],
        [-57.734558, -17.552468],
        [-57.498371, -18.174188],
        [-57.676009, -18.96184],
        [-57.949997, -19.400004],
        [-57.853802, -19.969995],
        [-58.166392, -20.176701],
        [-58.183471, -19.868399],
        [-59.115042, -19.356906],
        [-60.043565, -19.342747],
        [-61.786326, -19.633737],
        [-62.265961, -20.513735],
```

```
          [-62.291179, -21.051635],
          [-62.685057, -22.249029],
          [-62.846468, -22.034985],
        ],
      ],
    },
    id: "BOL",
  },
  {
    type: "Feature",
    properties: {
      name: "Brazil",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-57.625133, -30.216295],
          [-56.2909, -28.852761],
          [-55.162286, -27.881915],
          [-54.490725, -27.474757],
          [-53.648735, -26.923473],
          [-53.628349, -26.124865],
          [-54.13005, -25.547639],
          [-54.625291, -25.739255],
          [-54.428946, -25.162185],
          [-54.293476, -24.5708],
          [-54.29296, -24.021014],
          [-54.652834, -23.839578],
          [-55.027902, -24.001274],
          [-55.400747, -23.956935],
          [-55.517639, -23.571998],
          [-55.610683, -22.655619],
          [-55.797958, -22.35693],
          [-56.473317, -22.0863],
          [-56.88151, -22.282154],
          [-57.937156, -22.090176],
          [-57.870674, -20.732688],
          [-58.166392, -20.176701],
          [-57.853802, -19.969995],
          [-57.949997, -19.400004],
          [-57.676009, -18.96184],
          [-57.498371, -18.174188],
          [-57.734558, -17.552468],
          [-58.280804, -17.27171],
          [-58.388058, -16.877109],
          [-58.24122, -16.299573],
          [-60.15839, -16.258284],
          [-60.542966, -15.09391],
          [-60.251149, -15.077219],
          [-60.264326, -14.645979],
          [-60.459198, -14.354007],
          [-60.503304, -13.775955],
          [-61.084121, -13.479384],
          [-61.713204, -13.489202],
          [-62.127081, -13.198781],
          [-62.80306, -13.000653],
          [-63.196499, -12.627033],
          [-64.316353, -12.461978],
          [-65.402281, -11.56627],
          [-65.321899, -10.895872],
          [-65.444837, -10.511451],
          [-65.338435, -9.761988],
          [-66.646908, -9.931331],
          [-67.173801, -10.306812],
          [-68.048192, -10.712059],
```

```
[-68.271254, -11.014521],
[-68.786158, -11.03638],
[-69.529678, -10.951734],
[-70.093752, -11.123972],
[-70.548686, -11.009147],
[-70.481894, -9.490118],
[-71.302412, -10.079436],
[-72.184891, -10.053598],
[-72.563033, -9.520194],
[-73.226713, -9.462213],
[-73.015383, -9.032833],
[-73.571059, -8.424447],
[-73.987235, -7.52383],
[-73.723401, -7.340999],
[-73.724487, -6.918595],
[-73.120027, -6.629931],
[-73.219711, -6.089189],
[-72.964507, -5.741251],
[-72.891928, -5.274561],
[-71.748406, -4.593983],
[-70.928843, -4.401591],
[-70.794769, -4.251265],
[-69.893635, -4.298187],
[-69.444102, -1.556287],
[-69.420486, -1.122619],
[-69.577065, -0.549992],
[-70.020656, -0.185156],
[-70.015566, 0.541414],
[-69.452396, 0.706159],
[-69.252434, 0.602651],
[-69.218638, 0.985677],
[-69.804597, 1.089081],
[-69.816973, 1.714805],
[-67.868565, 1.692455],
[-67.53781, 2.037163],
[-67.259998, 1.719999],
[-67.065048, 1.130112],
[-66.876326, 1.253361],
[-66.325765, 0.724452],
[-65.548267, 0.789254],
[-65.354713, 1.095282],
[-64.611012, 1.328731],
[-64.199306, 1.492855],
[-64.083085, 1.916369],
[-63.368788, 2.2009],
[-63.422867, 2.411068],
[-64.269999, 2.497006],
[-64.408828, 3.126786],
[-64.368494, 3.79721],
[-64.816064, 4.056445],
[-64.628659, 4.148481],
[-63.888343, 4.02053],
[-63.093198, 3.770571],
[-62.804533, 4.006965],
[-62.08543, 4.162124],
[-60.966893, 4.536468],
[-60.601179, 4.918098],
[-60.733574, 5.200277],
[-60.213683, 5.244486],
[-59.980959, 5.014061],
[-60.111002, 4.574967],
[-59.767406, 4.423503],
[-59.53804, 3.958803],
[-59.815413, 3.606499],
[-59.974525, 2.755233],
[-59.718546, 2.24963],
```

```
        [-59.646044, 1.786894],
        [-59.030862, 1.317698],
        [-58.540013, 1.268088],
        [-58.429477, 1.463942],
        [-58.11345, 1.507195],
        [-57.660971, 1.682585],
        [-57.335823, 1.948538],
        [-56.782704, 1.863711],
        [-56.539386, 1.899523],
        [-55.995698, 1.817667],
        [-55.9056, 2.021996],
        [-56.073342, 2.220795],
        [-55.973322, 2.510364],
        [-55.569755, 2.421506],
        [-55.097587, 2.523748],
        [-54.524754, 2.311849],
        [-54.088063, 2.105557],
        [-53.778521, 2.376703],
        [-53.554839, 2.334897],
        [-53.418465, 2.053389],
        [-52.939657, 2.124858],
        [-52.556425, 2.504705],
        [-52.249338, 3.241094],
        [-51.657797, 4.156232],
        [-51.317146, 4.203491],
        [-51.069771, 3.650398],
        [-50.508875, 1.901564],
        [-49.974076, 1.736483],
        [-49.947101, 1.04619],
        [-50.699251, 0.222984],
        [-50.388211, -0.078445],
        [-48.620567, -0.235489],
        [-48.584497, -1.237805],
        [-47.824956, -0.581618],
        [-46.566584, -0.941028],
        [-44.905703, -1.55174],
        [-44.417619, -2.13775],
        [-44.581589, -2.691308],
        [-43.418791, -2.38311],
        [-41.472657, -2.912018],
        [-39.978665, -2.873054],
        [-38.500383, -3.700652],
        [-37.223252, -4.820946],
        [-36.452937, -5.109404],
        [-35.597796, -5.149504],
        [-35.235389, -5.464937],
        [-34.89603, -6.738193],
        [-34.729993, -7.343221],
        [-35.128212, -8.996401],
        [-35.636967, -9.649282],
        [-37.046519, -11.040721],
        [-37.683612, -12.171195],
        [-38.423877, -13.038119],
        [-38.673887, -13.057652],
        [-38.953276, -13.79337],
        [-38.882298, -15.667054],
        [-39.161092, -17.208407],
        [-39.267339, -17.867746],
        [-39.583521, -18.262296],
        [-39.760823, -19.599113],
        [-40.774741, -20.904512],
        [-40.944756, -21.937317],
        [-41.754164, -22.370676],
        [-41.988284, -22.97007],
        [-43.074704, -22.967693],
        [-44.647812, -23.351959],
```

```
          [-45.352136, -23.796842],
          [-46.472093, -24.088969],
          [-47.648972, -24.885199],
          [-48.495458, -25.877025],
          [-48.641005, -26.623698],
          [-48.474736, -27.175912],
          [-48.66152, -28.186135],
          [-48.888457, -28.674115],
          [-49.587329, -29.224469],
          [-50.696874, -30.984465],
          [-51.576226, -31.777698],
          [-52.256081, -32.24537],
          [-52.7121, -33.196578],
          [-53.373662, -33.768378],
          [-53.650544, -33.202004],
          [-53.209589, -32.727666],
          [-53.787952, -32.047243],
          [-54.572452, -31.494511],
          [-55.60151, -30.853879],
          [-55.973245, -30.883076],
          [-56.976026, -30.109686],
          [-57.625133, -30.216295],
        ],
      ],
    },
    id: "BRA",
  },
  {
    type: "Feature",
    properties: {
      name: "Brunei",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [114.204017, 4.525874],
          [114.599961, 4.900011],
          [115.45071, 5.44773],
          [115.4057, 4.955228],
          [115.347461, 4.316636],
          [114.869557, 4.348314],
          [114.659596, 4.007637],
          [114.204017, 4.525874],
        ],
      ],
    },
    id: "BRN",
  },
  {
    type: "Feature",
    properties: {
      name: "Bhutan",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [91.696657, 27.771742],
          [92.103712, 27.452614],
          [92.033484, 26.83831],
          [91.217513, 26.808648],
          [90.373275, 26.875724],
          [89.744528, 26.719403],
          [88.835643, 27.098966],
          [88.814248, 27.299316],
```

```
          [89.47581, 28.042759],
          [90.015829, 28.296439],
          [90.730514, 28.064954],
          [91.258854, 28.040614],
          [91.696657, 27.771742],
        ],
      ],
    },
    id: "BTN",
  },
  {
    type: "Feature",
    properties: {
      name: "Botswana",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [25.649163, -18.536026],
          [25.850391, -18.714413],
          [26.164791, -19.293086],
          [27.296505, -20.39152],
          [27.724747, -20.499059],
          [27.727228, -20.851802],
          [28.02137, -21.485975],
          [28.794656, -21.639454],
          [29.432188, -22.091313],
          [28.017236, -22.827754],
          [27.11941, -23.574323],
          [26.786407, -24.240691],
          [26.485753, -24.616327],
          [25.941652, -24.696373],
          [25.765849, -25.174845],
          [25.664666, -25.486816],
          [25.025171, -25.71967],
          [24.211267, -25.670216],
          [23.73357, -25.390129],
          [23.312097, -25.26869],
          [22.824271, -25.500459],
          [22.579532, -25.979448],
          [22.105969, -26.280256],
          [21.605896, -26.726534],
          [20.889609, -26.828543],
          [20.66647, -26.477453],
          [20.758609, -25.868136],
          [20.165726, -24.917962],
          [19.895768, -24.76779],
          [19.895458, -21.849157],
          [20.881134, -21.814327],
          [20.910641, -18.252219],
          [21.65504, -18.219146],
          [23.196858, -17.869038],
          [23.579006, -18.281261],
          [24.217365, -17.889347],
          [24.520705, -17.887125],
          [25.084443, -17.661816],
          [25.264226, -17.73654],
          [25.649163, -18.536026],
        ],
      ],
    },
    id: "BWA",
  },
  {
    type: "Feature",
```

```
      properties: {
        name: "Central African Republic",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [15.27946, 7.421925],
            [16.106232, 7.497088],
            [16.290562, 7.754307],
            [16.456185, 7.734774],
            [16.705988, 7.508328],
            [17.96493, 7.890914],
            [18.389555, 8.281304],
            [18.911022, 8.630895],
            [18.81201, 8.982915],
            [19.094008, 9.074847],
            [20.059685, 9.012706],
            [21.000868, 9.475985],
            [21.723822, 10.567056],
            [22.231129, 10.971889],
            [22.864165, 11.142395],
            [22.977544, 10.714463],
            [23.554304, 10.089255],
            [23.55725, 9.681218],
            [23.394779, 9.265068],
            [23.459013, 8.954286],
            [23.805813, 8.666319],
            [24.567369, 8.229188],
            [25.114932, 7.825104],
            [25.124131, 7.500085],
            [25.796648, 6.979316],
            [26.213418, 6.546603],
            [26.465909, 5.946717],
            [27.213409, 5.550953],
            [27.374226, 5.233944],
            [27.044065, 5.127853],
            [26.402761, 5.150875],
            [25.650455, 5.256088],
            [25.278798, 5.170408],
            [25.128833, 4.927245],
            [24.805029, 4.897247],
            [24.410531, 5.108784],
            [23.297214, 4.609693],
            [22.84148, 4.710126],
            [22.704124, 4.633051],
            [22.405124, 4.02916],
            [21.659123, 4.224342],
            [20.927591, 4.322786],
            [20.290679, 4.691678],
            [19.467784, 5.031528],
            [18.932312, 4.709506],
            [18.542982, 4.201785],
            [18.453065, 3.504386],
            [17.8099, 3.560196],
            [17.133042, 3.728197],
            [16.537058, 3.198255],
            [16.012852, 2.26764],
            [15.907381, 2.557389],
            [15.862732, 3.013537],
            [15.405396, 3.335301],
            [15.03622, 3.851367],
            [14.950953, 4.210389],
            [14.478372, 4.732605],
            [14.558936, 5.030598],
            [14.459407, 5.451761],
```

```
          [14.53656, 6.226959],
          [14.776545, 6.408498],
          [15.27946, 7.421925],
        ],
      ],
    },
    id: "CAF",
  },
  {
    type: "Feature",
    properties: {
      name: "Canada",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [-63.6645, 46.55001],
            [-62.9393, 46.41587],
            [-62.01208, 46.44314],
            [-62.50391, 46.03339],
            [-62.87433, 45.96818],
            [-64.1428, 46.39265],
            [-64.39261, 46.72747],
            [-64.01486, 47.03601],
            [-63.6645, 46.55001],
          ],
        ],
        [
          [
            [-61.806305, 49.10506],
            [-62.29318, 49.08717],
            [-63.58926, 49.40069],
            [-64.51912, 49.87304],
            [-64.17322, 49.95718],
            [-62.85829, 49.70641],
            [-61.835585, 49.28855],
            [-61.806305, 49.10506],
          ],
        ],
        [
          [
            [-123.510002, 48.510011],
            [-124.012891, 48.370846],
            [-125.655013, 48.825005],
            [-125.954994, 49.179996],
            [-126.850004, 49.53],
            [-127.029993, 49.814996],
            [-128.059336, 49.994959],
            [-128.444584, 50.539138],
            [-128.358414, 50.770648],
            [-127.308581, 50.552574],
            [-126.695001, 50.400903],
            [-125.755007, 50.295018],
            [-125.415002, 49.950001],
            [-124.920768, 49.475275],
            [-123.922509, 49.062484],
            [-123.510002, 48.510011],
          ],
        ],
        [
          [
            [-56.134036, 50.68701],
            [-56.795882, 49.812309],
            [-56.143105, 50.150117],
```

```
        [-55.471492, 49.935815],
        [-55.822401, 49.587129],
        [-54.935143, 49.313011],
        [-54.473775, 49.556691],
        [-53.476549, 49.249139],
        [-53.786014, 48.516781],
        [-53.086134, 48.687804],
        [-52.958648, 48.157164],
        [-52.648099, 47.535548],
        [-53.069158, 46.655499],
        [-53.521456, 46.618292],
        [-54.178936, 46.807066],
        [-53.961869, 47.625207],
        [-54.240482, 47.752279],
        [-55.400773, 46.884994],
        [-55.997481, 46.91972],
        [-55.291219, 47.389562],
        [-56.250799, 47.632545],
        [-57.325229, 47.572807],
        [-59.266015, 47.603348],
        [-59.419494, 47.899454],
        [-58.796586, 48.251525],
        [-59.231625, 48.523188],
        [-58.391805, 49.125581],
        [-57.35869, 50.718274],
        [-56.73865, 51.287438],
        [-55.870977, 51.632094],
        [-55.406974, 51.588273],
        [-55.600218, 51.317075],
        [-56.134036, 50.68701],
      ],
    ],
    [
      [
        [-132.710008, 54.040009],
        [-132.710009, 54.040009],
        [-132.710008, 54.040009],
        [-132.710008, 54.040009],
        [-131.74999, 54.120004],
        [-132.04948, 52.984621],
        [-131.179043, 52.180433],
        [-131.57783, 52.182371],
        [-132.180428, 52.639707],
        [-132.549992, 53.100015],
        [-133.054611, 53.411469],
        [-133.239664, 53.85108],
        [-133.180004, 54.169975],
        [-132.710008, 54.040009],
      ],
    ],
    [
      [
        [-79.26582, 62.158675],
        [-79.65752, 61.63308],
        [-80.09956, 61.7181],
        [-80.36215, 62.01649],
        [-80.315395, 62.085565],
        [-79.92939, 62.3856],
        [-79.52002, 62.36371],
        [-79.26582, 62.158675],
      ],
    ],
    [
      [
        [-81.89825, 62.7108],
        [-83.06857, 62.15922],
```

```
        [-83.77462, 62.18231],
        [-83.99367, 62.4528],
        [-83.25048, 62.91409],
        [-81.87699, 62.90458],
        [-81.89825, 62.7108],
      ],
    ],
    [
      [
        [-85.161308, 65.657285],
        [-84.975764, 65.217518],
        [-84.464012, 65.371772],
        [-83.882626, 65.109618],
        [-82.787577, 64.766693],
        [-81.642014, 64.455136],
        [-81.55344, 63.979609],
        [-80.817361, 64.057486],
        [-80.103451, 63.725981],
        [-80.99102, 63.411246],
        [-82.547178, 63.651722],
        [-83.108798, 64.101876],
        [-84.100417, 63.569712],
        [-85.523405, 63.052379],
        [-85.866769, 63.637253],
        [-87.221983, 63.541238],
        [-86.35276, 64.035833],
        [-86.224886, 64.822917],
        [-85.883848, 65.738778],
        [-85.161308, 65.657285],
      ],
    ],
    [
      [
        [-75.86588, 67.14886],
        [-76.98687, 67.09873],
        [-77.2364, 67.58809],
        [-76.81166, 68.14856],
        [-75.89521, 68.28721],
        [-75.1145, 68.01036],
        [-75.10333, 67.58202],
        [-75.21597, 67.44425],
        [-75.86588, 67.14886],
      ],
    ],
    [
      [
        [-95.647681, 69.10769],
        [-96.269521, 68.75704],
        [-97.617401, 69.06003],
        [-98.431801, 68.9507],
        [-99.797401, 69.40003],
        [-98.917401, 69.71003],
        [-98.218261, 70.14354],
        [-97.157401, 69.86003],
        [-96.557401, 69.68003],
        [-96.257401, 69.49003],
        [-95.647681, 69.10769],
      ],
    ],
    [
      [
        [-90.5471, 69.49766],
        [-90.55151, 68.47499],
        [-89.21515, 69.25873],
        [-88.01966, 68.61508],
        [-88.31749, 67.87338],
```

```
            [-87.35017, 67.19872],
            [-86.30607, 67.92146],
            [-85.57664, 68.78456],
            [-85.52197, 69.88211],
            [-84.10081, 69.80539],
            [-82.62258, 69.65826],
            [-81.28043, 69.16202],
            [-81.2202, 68.66567],
            [-81.96436, 68.13253],
            [-81.25928, 67.59716],
            [-81.38653, 67.11078],
            [-83.34456, 66.41154],
            [-84.73542, 66.2573],
            [-85.76943, 66.55833],
            [-86.0676, 66.05625],
            [-87.03143, 65.21297],
            [-87.32324, 64.77563],
            [-88.48296, 64.09897],
            [-89.91444, 64.03273],
            [-90.70398, 63.61017],
            [-90.77004, 62.96021],
            [-91.93342, 62.83508],
            [-93.15698, 62.02469],
            [-94.24153, 60.89865],
            [-94.62931, 60.11021],
            [-94.6846, 58.94882],
            [-93.21502, 58.78212],
            [-92.76462, 57.84571],
            [-92.29703, 57.08709],
            [-90.89769, 57.28468],
            [-89.03953, 56.85172],
            [-88.03978, 56.47162],
            [-87.32421, 55.99914],
            [-86.07121, 55.72383],
            [-85.01181, 55.3026],
            [-83.36055, 55.24489],
            [-82.27285, 55.14832],
            [-82.4362, 54.28227],
            [-82.12502, 53.27703],
            [-81.40075, 52.15788],
            [-79.91289, 51.20842],
            [-79.14301, 51.53393],
            [-78.60191, 52.56208],
            [-79.12421, 54.14145],
            [-79.82958, 54.66772],
            [-78.22874, 55.13645],
            [-77.0956, 55.83741],
            [-76.54137, 56.53423],
            [-76.62319, 57.20263],
            [-77.30226, 58.05209],
            [-78.51688, 58.80458],
            [-77.33676, 59.85261],
            [-77.77272, 60.75788],
            [-78.10687, 62.31964],
            [-77.41067, 62.55053],
            [-75.69621, 62.2784],
            [-74.6682, 62.18111],
            [-73.83988, 62.4438],
            [-72.90853, 62.10507],
            [-71.67708, 61.52535],
            [-71.37369, 61.13717],
            [-69.59042, 61.06141],
            [-69.62033, 60.22125],
            [-69.2879, 58.95736],
            [-68.37455, 58.80106],
            [-67.64976, 58.21206],
```

```
            [-66.20178, 58.76731],
            [-65.24517, 59.87071],
            [-64.58352, 60.33558],
            [-63.80475, 59.4426],
            [-62.50236, 58.16708],
            [-61.39655, 56.96745],
            [-61.79866, 56.33945],
            [-60.46853, 55.77548],
            [-59.56962, 55.20407],
            [-57.97508, 54.94549],
            [-57.3332, 54.6265],
            [-56.93689, 53.78032],
            [-56.15811, 53.64749],
            [-55.75632, 53.27036],
            [-55.68338, 52.14664],
            [-56.40916, 51.7707],
            [-57.12691, 51.41972],
            [-58.77482, 51.0643],
            [-60.03309, 50.24277],
            [-61.72366, 50.08046],
            [-63.86251, 50.29099],
            [-65.36331, 50.2982],
            [-66.39905, 50.22897],
            [-67.23631, 49.51156],
            [-68.51114, 49.06836],
            [-69.95362, 47.74488],
            [-71.10458, 46.82171],
            [-70.25522, 46.98606],
            [-68.65, 48.3],
            [-66.55243, 49.1331],
            [-65.05626, 49.23278],
            [-64.17099, 48.74248],
            [-65.11545, 48.07085],
            [-64.79854, 46.99297],
            [-64.47219, 46.23849],
            [-63.17329, 45.73902],
            [-61.52072, 45.88377],
            [-60.51815, 47.00793],
            [-60.4486, 46.28264],
            [-59.80287, 45.9204],
            [-61.03988, 45.26525],
            [-63.25471, 44.67014],
            [-64.24656, 44.26553],
            [-65.36406, 43.54523],
            [-66.1234, 43.61867],
            [-66.16173, 44.46512],
            [-64.42549, 45.29204],
            [-66.02605, 45.25931],
            [-67.13741, 45.13753],
            [-67.79134, 45.70281],
            [-67.79046, 47.06636],
            [-68.23444, 47.35486],
            [-68.905, 47.185],
            [-69.237216, 47.447781],
            [-69.99997, 46.69307],
            [-70.305, 45.915],
            [-70.66, 45.46],
            [-71.08482, 45.30524],
            [-71.405, 45.255],
            [-71.50506, 45.0082],
            [-73.34783, 45.00738],
            [-74.867, 45.00048],
            [-75.31821, 44.81645],
            [-76.375, 44.09631],
            [-76.5, 44.018459],
            [-76.820034, 43.628784],
```

```
            [-77.737885, 43.629056],
            [-78.72028, 43.625089],
            [-79.171674, 43.466339],
            [-79.01, 43.27],
            [-78.92, 42.965],
            [-78.939362, 42.863611],
            [-80.247448, 42.3662],
            [-81.277747, 42.209026],
            [-82.439278, 41.675105],
            [-82.690089, 41.675105],
            [-83.02981, 41.832796],
            [-83.142, 41.975681],
            [-83.12, 42.08],
            [-82.9, 42.43],
            [-82.43, 42.98],
            [-82.137642, 43.571088],
            [-82.337763, 44.44],
            [-82.550925, 45.347517],
            [-83.592851, 45.816894],
            [-83.469551, 45.994686],
            [-83.616131, 46.116927],
            [-83.890765, 46.116927],
            [-84.091851, 46.275419],
            [-84.14212, 46.512226],
            [-84.3367, 46.40877],
            [-84.6049, 46.4396],
            [-84.543749, 46.538684],
            [-84.779238, 46.637102],
            [-84.87608, 46.900083],
            [-85.652363, 47.220219],
            [-86.461991, 47.553338],
            [-87.439793, 47.94],
            [-88.378114, 48.302918],
            [-89.272917, 48.019808],
            [-89.6, 48.01],
            [-90.83, 48.27],
            [-91.64, 48.14],
            [-92.61, 48.45],
            [-93.63087, 48.60926],
            [-94.32914, 48.67074],
            [-94.64, 48.84],
            [-94.81758, 49.38905],
            [-95.15609, 49.38425],
            [-95.15907, 49],
            [-97.22872, 49.0007],
            [-100.65, 49],
            [-104.04826, 48.99986],
            [-107.05, 49],
            [-110.05, 49],
            [-113, 49],
            [-116.04818, 49],
            [-117.03121, 49],
            [-120, 49],
            [-122.84, 49],
            [-122.97421, 49.002538],
            [-124.91024, 49.98456],
            [-125.62461, 50.41656],
            [-127.43561, 50.83061],
            [-127.99276, 51.71583],
            [-127.85032, 52.32961],
            [-129.12979, 52.75538],
            [-129.30523, 53.56159],
            [-130.51497, 54.28757],
            [-130.53611, 54.80278],
            [-129.98, 55.285],
            [-130.00778, 55.91583],
```

```
            [-131.70781, 56.55212],
            [-132.73042, 57.69289],
            [-133.35556, 58.41028],
            [-134.27111, 58.86111],
            [-134.945, 59.27056],
            [-135.47583, 59.78778],
            [-136.47972, 59.46389],
            [-137.4525, 58.905],
            [-138.34089, 59.56211],
            [-139.039, 60],
            [-140.013, 60.27682],
            [-140.99778, 60.30639],
            [-140.9925, 66.00003],
            [-140.986, 69.712],
            [-139.12052, 69.47102],
            [-137.54636, 68.99002],
            [-136.50358, 68.89804],
            [-135.62576, 69.31512],
            [-134.41464, 69.62743],
            [-132.92925, 69.50534],
            [-131.43136, 69.94451],
            [-129.79471, 70.19369],
            [-129.10773, 69.77927],
            [-128.36156, 70.01286],
            [-128.13817, 70.48384],
            [-127.44712, 70.37721],
            [-125.75632, 69.48058],
            [-124.42483, 70.1584],
            [-124.28968, 69.39969],
            [-123.06108, 69.56372],
            [-122.6835, 69.85553],
            [-121.47226, 69.79778],
            [-119.94288, 69.37786],
            [-117.60268, 69.01128],
            [-116.22643, 68.84151],
            [-115.2469, 68.90591],
            [-113.89794, 68.3989],
            [-115.30489, 67.90261],
            [-113.49727, 67.68815],
            [-110.798, 67.80612],
            [-109.94619, 67.98104],
            [-108.8802, 67.38144],
            [-107.79239, 67.88736],
            [-108.81299, 68.31164],
            [-108.16721, 68.65392],
            [-106.95, 68.7],
            [-106.15, 68.8],
            [-105.34282, 68.56122],
            [-104.33791, 68.018],
            [-103.22115, 68.09775],
            [-101.45433, 67.64689],
            [-99.90195, 67.80566],
            [-98.4432, 67.78165],
            [-98.5586, 68.40394],
            [-97.66948, 68.57864],
            [-96.11991, 68.23939],
            [-96.12588, 67.29338],
            [-95.48943, 68.0907],
            [-94.685, 68.06383],
            [-94.23282, 69.06903],
            [-95.30408, 69.68571],
            [-96.47131, 70.08976],
            [-96.39115, 71.19482],
            [-95.2088, 71.92053],
            [-93.88997, 71.76015],
            [-92.87818, 71.31869],
```

```
            [-91.51964, 70.19129],
            [-92.40692, 69.69997],
            [-90.5471, 69.49766],
          ],
        ],
        [
          [
            [-114.16717, 73.12145],
            [-114.66634, 72.65277],
            [-112.44102, 72.9554],
            [-111.05039, 72.4504],
            [-109.92035, 72.96113],
            [-109.00654, 72.63335],
            [-108.18835, 71.65089],
            [-107.68599, 72.06548],
            [-108.39639, 73.08953],
            [-107.51645, 73.23598],
            [-106.52259, 73.07601],
            [-105.40246, 72.67259],
            [-104.77484, 71.6984],
            [-104.46476, 70.99297],
            [-102.78537, 70.49776],
            [-100.98078, 70.02432],
            [-101.08929, 69.58447],
            [-102.73116, 69.50402],
            [-102.09329, 69.11962],
            [-102.43024, 68.75282],
            [-104.24, 68.91],
            [-105.96, 69.18],
            [-107.12254, 69.11922],
            [-109, 68.78],
            [-111.534149, 68.630059],
            [-113.3132, 68.53554],
            [-113.85496, 69.00744],
            [-115.22, 69.28],
            [-116.10794, 69.16821],
            [-117.34, 69.96],
            [-116.67473, 70.06655],
            [-115.13112, 70.2373],
            [-113.72141, 70.19237],
            [-112.4161, 70.36638],
            [-114.35, 70.6],
            [-116.48684, 70.52045],
            [-117.9048, 70.54056],
            [-118.43238, 70.9092],
            [-116.11311, 71.30918],
            [-117.65568, 71.2952],
            [-119.40199, 71.55859],
            [-118.56267, 72.30785],
            [-117.86642, 72.70594],
            [-115.18909, 73.31459],
            [-114.16717, 73.12145],
          ],
        ],
        [
          [
            [-104.5, 73.42],
            [-105.38, 72.76],
            [-106.94, 73.46],
            [-106.6, 73.6],
            [-105.26, 73.64],
            [-104.5, 73.42],
          ],
        ],
        [
          [
```

```
        [-76.34, 73.102685],
        [-76.251404, 72.826385],
        [-77.314438, 72.855545],
        [-78.39167, 72.876656],
        [-79.486252, 72.742203],
        [-79.775833, 72.802902],
        [-80.876099, 73.333183],
        [-80.833885, 73.693184],
        [-80.353058, 73.75972],
        [-78.064438, 73.651932],
        [-76.34, 73.102685],
      ],
    ],
    [
      [
        [-86.562179, 73.157447],
        [-85.774371, 72.534126],
        [-84.850112, 73.340278],
        [-82.31559, 73.750951],
        [-80.600088, 72.716544],
        [-80.748942, 72.061907],
        [-78.770639, 72.352173],
        [-77.824624, 72.749617],
        [-75.605845, 72.243678],
        [-74.228616, 71.767144],
        [-74.099141, 71.33084],
        [-72.242226, 71.556925],
        [-71.200015, 70.920013],
        [-68.786054, 70.525024],
        [-67.91497, 70.121948],
        [-66.969033, 69.186087],
        [-68.805123, 68.720198],
        [-66.449866, 68.067163],
        [-64.862314, 67.847539],
        [-63.424934, 66.928473],
        [-61.851981, 66.862121],
        [-62.163177, 66.160251],
        [-63.918444, 64.998669],
        [-65.14886, 65.426033],
        [-66.721219, 66.388041],
        [-68.015016, 66.262726],
        [-68.141287, 65.689789],
        [-67.089646, 65.108455],
        [-65.73208, 64.648406],
        [-65.320168, 64.382737],
        [-64.669406, 63.392927],
        [-65.013804, 62.674185],
        [-66.275045, 62.945099],
        [-68.783186, 63.74567],
        [-67.369681, 62.883966],
        [-66.328297, 62.280075],
        [-66.165568, 61.930897],
        [-68.877367, 62.330149],
        [-71.023437, 62.910708],
        [-72.235379, 63.397836],
        [-71.886278, 63.679989],
        [-73.378306, 64.193963],
        [-74.834419, 64.679076],
        [-74.818503, 64.389093],
        [-77.70998, 64.229542],
        [-78.555949, 64.572906],
        [-77.897281, 65.309192],
        [-76.018274, 65.326969],
        [-73.959795, 65.454765],
        [-74.293883, 65.811771],
        [-73.944912, 66.310578],
```

```
        [-72.651167, 67.284576],
        [-72.92606, 67.726926],
        [-73.311618, 68.069437],
        [-74.843307, 68.554627],
        [-76.869101, 68.894736],
        [-76.228649, 69.147769],
        [-77.28737, 69.76954],
        [-78.168634, 69.826488],
        [-78.957242, 70.16688],
        [-79.492455, 69.871808],
        [-81.305471, 69.743185],
        [-84.944706, 69.966634],
        [-87.060003, 70.260001],
        [-88.681713, 70.410741],
        [-89.51342, 70.762038],
        [-88.467721, 71.218186],
        [-89.888151, 71.222552],
        [-90.20516, 72.235074],
        [-89.436577, 73.129464],
        [-88.408242, 73.537889],
        [-85.826151, 73.803816],
        [-86.562179, 73.157447],
      ],
    ],
    [
      [
        [-100.35642, 73.84389],
        [-99.16387, 73.63339],
        [-97.38, 73.76],
        [-97.12, 73.47],
        [-98.05359, 72.99052],
        [-96.54, 72.56],
        [-96.72, 71.66],
        [-98.35966, 71.27285],
        [-99.32286, 71.35639],
        [-100.01482, 71.73827],
        [-102.5, 72.51],
        [-102.48, 72.83],
        [-100.43836, 72.70588],
        [-101.54, 73.36],
        [-100.35642, 73.84389],
      ],
    ],
    [
      [
        [-93.196296, 72.771992],
        [-94.269047, 72.024596],
        [-95.409856, 72.061881],
        [-96.033745, 72.940277],
        [-96.018268, 73.43743],
        [-95.495793, 73.862417],
        [-94.503658, 74.134907],
        [-92.420012, 74.100025],
        [-90.509793, 73.856732],
        [-92.003965, 72.966244],
        [-93.196296, 72.771992],
      ],
    ],
    [
      [
        [-120.46, 71.383602],
        [-123.09219, 70.90164],
        [-123.62, 71.34],
        [-125.928949, 71.868688],
        [-125.5, 72.292261],
        [-124.80729, 73.02256],
```

```
        [-123.94, 73.68],
        [-124.91775, 74.29275],
        [-121.53788, 74.44893],
        [-120.10978, 74.24135],
        [-117.55564, 74.18577],
        [-116.58442, 73.89607],
        [-115.51081, 73.47519],
        [-116.76794, 73.22292],
        [-119.22, 72.52],
        [-120.46, 71.82],
        [-120.46, 71.383602],
      ],
    ],
    [
      [
        [-93.612756, 74.979997],
        [-94.156909, 74.592347],
        [-95.608681, 74.666864],
        [-96.820932, 74.927623],
        [-96.288587, 75.377828],
        [-94.85082, 75.647218],
        [-93.977747, 75.29649],
        [-93.612756, 74.979997],
      ],
    ],
    [
      [
        [-98.5, 76.72],
        [-97.735585, 76.25656],
        [-97.704415, 75.74344],
        [-98.16, 75],
        [-99.80874, 74.89744],
        [-100.88366, 75.05736],
        [-100.86292, 75.64075],
        [-102.50209, 75.5638],
        [-102.56552, 76.3366],
        [-101.48973, 76.30537],
        [-99.98349, 76.64634],
        [-98.57699, 76.58859],
        [-98.5, 76.72],
      ],
    ],
    [
      [
        [-108.21141, 76.20168],
        [-107.81943, 75.84552],
        [-106.92893, 76.01282],
        [-105.881, 75.9694],
        [-105.70498, 75.47951],
        [-106.31347, 75.00527],
        [-109.7, 74.85],
        [-112.22307, 74.41696],
        [-113.74381, 74.39427],
        [-113.87135, 74.72029],
        [-111.79421, 75.1625],
        [-116.31221, 75.04343],
        [-117.7104, 75.2222],
        [-116.34602, 76.19903],
        [-115.40487, 76.47887],
        [-112.59056, 76.14134],
        [-110.81422, 75.54919],
        [-109.0671, 75.47321],
        [-110.49726, 76.42982],
        [-109.5811, 76.79417],
        [-108.54859, 76.67832],
        [-108.21141, 76.20168],
```

```
      ],
    ],
    [
      [
        [-94.684086, 77.097878],
        [-93.573921, 76.776296],
        [-91.605023, 76.778518],
        [-90.741846, 76.449597],
        [-90.969661, 76.074013],
        [-89.822238, 75.847774],
        [-89.187083, 75.610166],
        [-87.838276, 75.566189],
        [-86.379192, 75.482421],
        [-84.789625, 75.699204],
        [-82.753445, 75.784315],
        [-81.128531, 75.713983],
        [-80.057511, 75.336849],
        [-79.833933, 74.923127],
        [-80.457771, 74.657304],
        [-81.948843, 74.442459],
        [-83.228894, 74.564028],
        [-86.097452, 74.410032],
        [-88.15035, 74.392307],
        [-89.764722, 74.515555],
        [-92.422441, 74.837758],
        [-92.768285, 75.38682],
        [-92.889906, 75.882655],
        [-93.893824, 76.319244],
        [-95.962457, 76.441381],
        [-97.121379, 76.751078],
        [-96.745123, 77.161389],
        [-94.684086, 77.097878],
      ],
    ],
    [
      [
        [-116.198587, 77.645287],
        [-116.335813, 76.876962],
        [-117.106051, 76.530032],
        [-118.040412, 76.481172],
        [-119.899318, 76.053213],
        [-121.499995, 75.900019],
        [-122.854924, 76.116543],
        [-122.854925, 76.116543],
        [-121.157535, 76.864508],
        [-119.103939, 77.51222],
        [-117.570131, 77.498319],
        [-116.198587, 77.645287],
      ],
    ],
    [
      [
        [-93.840003, 77.519997],
        [-94.295608, 77.491343],
        [-96.169654, 77.555111],
        [-96.436304, 77.834629],
        [-94.422577, 77.820005],
        [-93.720656, 77.634331],
        [-93.840003, 77.519997],
      ],
    ],
    [
      [
        [-110.186938, 77.697015],
        [-112.051191, 77.409229],
        [-113.534279, 77.732207],
```

```
            [-112.724587, 78.05105],
            [-111.264443, 78.152956],
            [-109.854452, 77.996325],
            [-110.186938, 77.697015],
          ],
        ],
        [
          [
            [-109.663146, 78.601973],
            [-110.881314, 78.40692],
            [-112.542091, 78.407902],
            [-112.525891, 78.550555],
            [-111.50001, 78.849994],
            [-110.963661, 78.804441],
            [-109.663146, 78.601973],
          ],
        ],
        [
          [
            [-95.830295, 78.056941],
            [-97.309843, 77.850597],
            [-98.124289, 78.082857],
            [-98.552868, 78.458105],
            [-98.631984, 78.87193],
            [-97.337231, 78.831984],
            [-96.754399, 78.765813],
            [-95.559278, 78.418315],
            [-95.830295, 78.056941],
          ],
        ],
        [
          [
            [-100.060192, 78.324754],
            [-99.670939, 77.907545],
            [-101.30394, 78.018985],
            [-102.949809, 78.343229],
            [-105.176133, 78.380332],
            [-104.210429, 78.67742],
            [-105.41958, 78.918336],
            [-105.492289, 79.301594],
            [-103.529282, 79.165349],
            [-100.825158, 78.800462],
            [-100.060192, 78.324754],
          ],
        ],
        [
          [
            [-87.02, 79.66],
            [-85.81435, 79.3369],
            [-87.18756, 79.0393],
            [-89.03535, 78.28723],
            [-90.80436, 78.21533],
            [-92.87669, 78.34333],
            [-93.95116, 78.75099],
            [-93.93574, 79.11373],
            [-93.14524, 79.3801],
            [-94.974, 79.37248],
            [-96.07614, 79.70502],
            [-96.70972, 80.15777],
            [-96.01644, 80.60233],
            [-95.32345, 80.90729],
            [-94.29843, 80.97727],
            [-94.73542, 81.20646],
            [-92.40984, 81.25739],
            [-91.13289, 80.72345],
            [-89.45, 80.509322],
```

```
            [-87.81, 80.32],
            [-87.02, 79.66],
          ],
        ],
        [
          [
            [-68.5, 83.106322],
            [-65.82735, 83.02801],
            [-63.68, 82.9],
            [-61.85, 82.6286],
            [-61.89388, 82.36165],
            [-64.334, 81.92775],
            [-66.75342, 81.72527],
            [-67.65755, 81.50141],
            [-65.48031, 81.50657],
            [-67.84, 80.9],
            [-69.4697, 80.61683],
            [-71.18, 79.8],
            [-73.2428, 79.63415],
            [-73.88, 79.430162],
            [-76.90773, 79.32309],
            [-75.52924, 79.19766],
            [-76.22046, 79.01907],
            [-75.39345, 78.52581],
            [-76.34354, 78.18296],
            [-77.88851, 77.89991],
            [-78.36269, 77.50859],
            [-79.75951, 77.20968],
            [-79.61965, 76.98336],
            [-77.91089, 77.022045],
            [-77.88911, 76.777955],
            [-80.56125, 76.17812],
            [-83.17439, 76.45403],
            [-86.11184, 76.29901],
            [-87.6, 76.42],
            [-89.49068, 76.47239],
            [-89.6161, 76.95213],
            [-87.76739, 77.17833],
            [-88.26, 77.9],
            [-87.65, 77.970222],
            [-84.97634, 77.53873],
            [-86.34, 78.18],
            [-87.96192, 78.37181],
            [-87.15198, 78.75867],
            [-85.37868, 78.9969],
            [-85.09495, 79.34543],
            [-86.50734, 79.73624],
            [-86.93179, 80.25145],
            [-84.19844, 80.20836],
            [-83.408696, 80.1],
            [-81.84823, 80.46442],
            [-84.1, 80.58],
            [-87.59895, 80.51627],
            [-89.36663, 80.85569],
            [-90.2, 81.26],
            [-91.36786, 81.5531],
            [-91.58702, 81.89429],
            [-90.1, 82.085],
            [-88.93227, 82.11751],
            [-86.97024, 82.27961],
            [-85.5, 82.652273],
            [-84.260005, 82.6],
            [-83.18, 82.32],
            [-82.42, 82.86],
            [-81.1, 83.02],
            [-79.30664, 83.13056],
```

```
            [-76.25, 83.172059],
            [-75.71878, 83.06404],
            [-72.83153, 83.23324],
            [-70.665765, 83.169781],
            [-68.5, 83.106322],
          ],
        ],
      ],
    },
    id: "CAN",
  },
  {
    type: "Feature",
    properties: {
      name: "Switzerland",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [9.594226, 47.525058],
          [9.632932, 47.347601],
          [9.47997, 47.10281],
          [9.932448, 46.920728],
          [10.442701, 46.893546],
          [10.363378, 46.483571],
          [9.922837, 46.314899],
          [9.182882, 46.440215],
          [8.966306, 46.036932],
          [8.489952, 46.005151],
          [8.31663, 46.163642],
          [7.755992, 45.82449],
          [7.273851, 45.776948],
          [6.843593, 45.991147],
          [6.5001, 46.429673],
          [6.022609, 46.27299],
          [6.037389, 46.725779],
          [6.768714, 47.287708],
          [6.736571, 47.541801],
          [7.192202, 47.449766],
          [7.466759, 47.620582],
          [8.317301, 47.61358],
          [8.522612, 47.830828],
          [9.594226, 47.525058],
        ],
      ],
    },
    id: "CHE",
  },
  {
    type: "Feature",
    properties: {
      name: "Chile",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [-68.63401, -52.63637],
            [-68.63335, -54.8695],
            [-67.56244, -54.87001],
            [-66.95992, -54.89681],
            [-67.29103, -55.30124],
            [-68.14863, -55.61183],
            [-68.639991, -55.580018],
```

```
        [-69.2321, -55.49906],
        [-69.95809, -55.19843],
        [-71.00568, -55.05383],
        [-72.2639, -54.49514],
        [-73.2852, -53.95752],
        [-74.66253, -52.83749],
        [-73.8381, -53.04743],
        [-72.43418, -53.7154],
        [-71.10773, -54.07433],
        [-70.59178, -53.61583],
        [-70.26748, -52.93123],
        [-69.34565, -52.5183],
        [-68.63401, -52.63637],
      ],
    ],
    [
      [
        [-68.219913, -21.494347],
        [-67.82818, -22.872919],
        [-67.106674, -22.735925],
        [-66.985234, -22.986349],
        [-67.328443, -24.025303],
        [-68.417653, -24.518555],
        [-68.386001, -26.185016],
        [-68.5948, -26.506909],
        [-68.295542, -26.89934],
        [-69.001235, -27.521214],
        [-69.65613, -28.459141],
        [-70.01355, -29.367923],
        [-69.919008, -30.336339],
        [-70.535069, -31.36501],
        [-70.074399, -33.09121],
        [-69.814777, -33.273886],
        [-69.817309, -34.193571],
        [-70.388049, -35.169688],
        [-70.364769, -36.005089],
        [-71.121881, -36.658124],
        [-71.118625, -37.576827],
        [-70.814664, -38.552995],
        [-71.413517, -38.916022],
        [-71.680761, -39.808164],
        [-71.915734, -40.832339],
        [-71.746804, -42.051386],
        [-72.148898, -42.254888],
        [-71.915424, -43.408565],
        [-71.464056, -43.787611],
        [-71.793623, -44.207172],
        [-71.329801, -44.407522],
        [-71.222779, -44.784243],
        [-71.659316, -44.973689],
        [-71.552009, -45.560733],
        [-71.917258, -46.884838],
        [-72.447355, -47.738533],
        [-72.331161, -48.244238],
        [-72.648247, -48.878618],
        [-73.415436, -49.318436],
        [-73.328051, -50.378785],
        [-72.975747, -50.74145],
        [-72.309974, -50.67701],
        [-72.329404, -51.425956],
        [-71.914804, -52.009022],
        [-69.498362, -52.142761],
        [-68.571545, -52.299444],
        [-69.461284, -52.291951],
        [-69.94278, -52.537931],
        [-70.845102, -52.899201],
```

```
            [-71.006332, -53.833252],
            [-71.429795, -53.856455],
            [-72.557943, -53.53141],
            [-73.702757, -52.835069],
            [-73.702757, -52.83507],
            [-74.946763, -52.262754],
            [-75.260026, -51.629355],
            [-74.976632, -51.043396],
            [-75.479754, -50.378372],
            [-75.608015, -48.673773],
            [-75.18277, -47.711919],
            [-74.126581, -46.939253],
            [-75.644395, -46.647643],
            [-74.692154, -45.763976],
            [-74.351709, -44.103044],
            [-73.240356, -44.454961],
            [-72.717804, -42.383356],
            [-73.3889, -42.117532],
            [-73.701336, -43.365776],
            [-74.331943, -43.224958],
            [-74.017957, -41.794813],
            [-73.677099, -39.942213],
            [-73.217593, -39.258689],
            [-73.505559, -38.282883],
            [-73.588061, -37.156285],
            [-73.166717, -37.12378],
            [-72.553137, -35.50884],
            [-71.861732, -33.909093],
            [-71.43845, -32.418899],
            [-71.668721, -30.920645],
            [-71.370083, -30.095682],
            [-71.489894, -28.861442],
            [-70.905124, -27.64038],
            [-70.724954, -25.705924],
            [-70.403966, -23.628997],
            [-70.091246, -21.393319],
            [-70.16442, -19.756468],
            [-70.372572, -18.347975],
            [-69.858444, -18.092694],
            [-69.590424, -17.580012],
            [-69.100247, -18.260125],
            [-68.966818, -18.981683],
            [-68.442225, -19.405068],
            [-68.757167, -20.372658],
            [-68.219913, -21.494347],
          ],
        ],
      ],
    },
    id: "CHL",
  },
  {
    type: "Feature",
    properties: {
      name: "China",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [110.339188, 18.678395],
            [109.47521, 18.197701],
            [108.655208, 18.507682],
            [108.626217, 19.367888],
            [109.119056, 19.821039],
```

```
            [110.211599, 20.101254],
            [110.786551, 20.077534],
            [111.010051, 19.69593],
            [110.570647, 19.255879],
            [110.339188, 18.678395],
          ],
        ],
        [
          [
            [127.657407, 49.76027],
            [129.397818, 49.4406],
            [130.582293, 48.729687],
            [130.987282, 47.790132],
            [132.506672, 47.78897],
            [133.373596, 48.183442],
            [135.026311, 48.47823],
            [134.500814, 47.57844],
            [134.112362, 47.212467],
            [133.769644, 46.116927],
            [133.097127, 45.144066],
            [131.883454, 45.321162],
            [131.025212, 44.967953],
            [131.288555, 44.11152],
            [131.144688, 42.92999],
            [130.633866, 42.903015],
            [130.640016, 42.395009],
            [129.994267, 42.985387],
            [129.596669, 42.424982],
            [128.052215, 41.994285],
            [128.208433, 41.466772],
            [127.343783, 41.503152],
            [126.869083, 41.816569],
            [126.182045, 41.107336],
            [125.079942, 40.569824],
            [124.265625, 39.928493],
            [122.86757, 39.637788],
            [122.131388, 39.170452],
            [121.054554, 38.897471],
            [121.585995, 39.360854],
            [121.376757, 39.750261],
            [122.168595, 40.422443],
            [121.640359, 40.94639],
            [120.768629, 40.593388],
            [119.639602, 39.898056],
            [119.023464, 39.252333],
            [118.042749, 39.204274],
            [117.532702, 38.737636],
            [118.059699, 38.061476],
            [118.87815, 37.897325],
            [118.911636, 37.448464],
            [119.702802, 37.156389],
            [120.823457, 37.870428],
            [121.711259, 37.481123],
            [122.357937, 37.454484],
            [122.519995, 36.930614],
            [121.104164, 36.651329],
            [120.637009, 36.11144],
            [119.664562, 35.609791],
            [119.151208, 34.909859],
            [120.227525, 34.360332],
            [120.620369, 33.376723],
            [121.229014, 32.460319],
            [121.908146, 31.692174],
            [121.891919, 30.949352],
            [121.264257, 30.676267],
            [121.503519, 30.142915],
```

```
[122.092114, 29.83252],
[121.938428, 29.018022],
[121.684439, 28.225513],
[121.125661, 28.135673],
[120.395473, 27.053207],
[119.585497, 25.740781],
[118.656871, 24.547391],
[117.281606, 23.624501],
[115.890735, 22.782873],
[114.763827, 22.668074],
[114.152547, 22.22376],
[113.80678, 22.54834],
[113.241078, 22.051367],
[111.843592, 21.550494],
[110.785466, 21.397144],
[110.444039, 20.341033],
[109.889861, 20.282457],
[109.627655, 21.008227],
[109.864488, 21.395051],
[108.522813, 21.715212],
[108.05018, 21.55238],
[107.04342, 21.811899],
[106.567273, 22.218205],
[106.725403, 22.794268],
[105.811247, 22.976892],
[105.329209, 23.352063],
[104.476858, 22.81915],
[103.504515, 22.703757],
[102.706992, 22.708795],
[102.170436, 22.464753],
[101.652018, 22.318199],
[101.80312, 21.174367],
[101.270026, 21.201652],
[101.180005, 21.436573],
[101.150033, 21.849984],
[100.416538, 21.558839],
[99.983489, 21.742937],
[99.240899, 22.118314],
[99.531992, 22.949039],
[98.898749, 23.142722],
[98.660262, 24.063286],
[97.60472, 23.897405],
[97.724609, 25.083637],
[98.671838, 25.918703],
[98.712094, 26.743536],
[98.68269, 27.508812],
[98.246231, 27.747221],
[97.911988, 28.335945],
[97.327114, 28.261583],
[96.248833, 28.411031],
[96.586591, 28.83098],
[96.117679, 29.452802],
[95.404802, 29.031717],
[94.56599, 29.277438],
[93.413348, 28.640629],
[92.503119, 27.896876],
[91.696657, 27.771742],
[91.258854, 28.040614],
[90.730514, 28.064954],
[90.015829, 28.296439],
[89.47581, 28.042759],
[88.814248, 27.299316],
[88.730326, 28.086865],
[88.120441, 27.876542],
[86.954517, 27.974262],
[85.82332, 28.203576],
```

```
[85.011638, 28.642774],
[84.23458, 28.839894],
[83.898993, 29.320226],
[83.337115, 29.463732],
[82.327513, 30.115268],
[81.525804, 30.422717],
[81.111256, 30.183481],
[79.721367, 30.882715],
[78.738894, 31.515906],
[78.458446, 32.618164],
[79.176129, 32.48378],
[79.208892, 32.994395],
[78.811086, 33.506198],
[78.912269, 34.321936],
[77.837451, 35.49401],
[76.192848, 35.898403],
[75.896897, 36.666806],
[75.158028, 37.133031],
[74.980002, 37.41999],
[74.829986, 37.990007],
[74.864816, 38.378846],
[74.257514, 38.606507],
[73.928852, 38.505815],
[73.675379, 39.431237],
[73.960013, 39.660008],
[73.822244, 39.893973],
[74.776862, 40.366425],
[75.467828, 40.562072],
[76.526368, 40.427946],
[76.904484, 41.066486],
[78.187197, 41.185316],
[78.543661, 41.582243],
[80.11943, 42.123941],
[80.25999, 42.349999],
[80.18015, 42.920068],
[80.866206, 43.180362],
[79.966106, 44.917517],
[81.947071, 45.317027],
[82.458926, 45.53965],
[83.180484, 47.330031],
[85.16429, 47.000956],
[85.720484, 47.452969],
[85.768233, 48.455751],
[86.598776, 48.549182],
[87.35997, 49.214981],
[87.751264, 49.297198],
[88.013832, 48.599463],
[88.854298, 48.069082],
[90.280826, 47.693549],
[90.970809, 46.888146],
[90.585768, 45.719716],
[90.94554, 45.286073],
[92.133891, 45.115076],
[93.480734, 44.975472],
[94.688929, 44.352332],
[95.306875, 44.241331],
[95.762455, 43.319449],
[96.349396, 42.725635],
[97.451757, 42.74889],
[99.515817, 42.524691],
[100.845866, 42.663804],
[101.83304, 42.514873],
[103.312278, 41.907468],
[104.522282, 41.908347],
[104.964994, 41.59741],
[106.129316, 42.134328],
```

```
            [107.744773, 42.481516],
            [109.243596, 42.519446],
            [110.412103, 42.871234],
            [111.129682, 43.406834],
            [111.829588, 43.743118],
            [111.667737, 44.073176],
            [111.348377, 44.457442],
            [111.873306, 45.102079],
            [112.436062, 45.011646],
            [113.463907, 44.808893],
            [114.460332, 45.339817],
            [115.985096, 45.727235],
            [116.717868, 46.388202],
            [117.421701, 46.672733],
            [118.874326, 46.805412],
            [119.66327, 46.69268],
            [119.772824, 47.048059],
            [118.866574, 47.74706],
            [118.064143, 48.06673],
            [117.295507, 47.697709],
            [116.308953, 47.85341],
            [115.742837, 47.726545],
            [115.485282, 48.135383],
            [116.191802, 49.134598],
            [116.678801, 49.888531],
            [117.879244, 49.510983],
            [119.288461, 50.142883],
            [119.279366, 50.582908],
            [120.18205, 51.643566],
            [120.738191, 51.964115],
            [120.725789, 52.516226],
            [120.177089, 52.753886],
            [121.003085, 53.251401],
            [122.245748, 53.431726],
            [123.571507, 53.458804],
            [125.068211, 53.161045],
            [125.946349, 52.792799],
            [126.564399, 51.784255],
            [126.939157, 51.353894],
            [127.287456, 50.739797],
            [127.657407, 49.76027],
          ],
        ],
      ],
    },
    id: "CHN",
  },
  {
    type: "Feature",
    properties: {
      name: "Ivory Coast",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-2.856125, 4.994476],
          [-3.311084, 4.984296],
          [-4.00882, 5.179813],
          [-4.649917, 5.168264],
          [-5.834496, 4.993701],
          [-6.528769, 4.705088],
          [-7.518941, 4.338288],
          [-7.712159, 4.364566],
          [-7.635368, 5.188159],
          [-7.539715, 5.313345],
```

```
          [-7.570153, 5.707352],
          [-7.993693, 6.12619],
          [-8.311348, 6.193033],
          [-8.60288, 6.467564],
          [-8.385452, 6.911801],
          [-8.485446, 7.395208],
          [-8.439298, 7.686043],
          [-8.280703, 7.68718],
          [-8.221792, 8.123329],
          [-8.299049, 8.316444],
          [-8.203499, 8.455453],
          [-7.8321, 8.575704],
          [-8.079114, 9.376224],
          [-8.309616, 9.789532],
          [-8.229337, 10.12902],
          [-8.029944, 10.206535],
          [-7.89959, 10.297382],
          [-7.622759, 10.147236],
          [-6.850507, 10.138994],
          [-6.666461, 10.430811],
          [-6.493965, 10.411303],
          [-6.205223, 10.524061],
          [-6.050452, 10.096361],
          [-5.816926, 10.222555],
          [-5.404342, 10.370737],
          [-4.954653, 10.152714],
          [-4.779884, 9.821985],
          [-4.330247, 9.610835],
          [-3.980449, 9.862344],
          [-3.511899, 9.900326],
          [-2.827496, 9.642461],
          [-2.56219, 8.219628],
          [-2.983585, 7.379705],
          [-3.24437, 6.250472],
          [-2.810701, 5.389051],
          [-2.856125, 4.994476],
        ],
      ],
    },
    id: "CIV",
  },
  {
    type: "Feature",
    properties: {
      name: "Cameroon",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [13.075822, 2.267097],
          [12.951334, 2.321616],
          [12.35938, 2.192812],
          [11.751665, 2.326758],
          [11.276449, 2.261051],
          [9.649158, 2.283866],
          [9.795196, 3.073404],
          [9.404367, 3.734527],
          [8.948116, 3.904129],
          [8.744924, 4.352215],
          [8.488816, 4.495617],
          [8.500288, 4.771983],
          [8.757533, 5.479666],
          [9.233163, 6.444491],
          [9.522706, 6.453482],
          [10.118277, 7.03877],
```

```
          [10.497375, 7.055358],
          [11.058788, 6.644427],
          [11.745774, 6.981383],
          [11.839309, 7.397042],
          [12.063946, 7.799808],
          [12.218872, 8.305824],
          [12.753672, 8.717763],
          [12.955468, 9.417772],
          [13.1676, 9.640626],
          [13.308676, 10.160362],
          [13.57295, 10.798566],
          [14.415379, 11.572369],
          [14.468192, 11.904752],
          [14.577178, 12.085361],
          [14.181336, 12.483657],
          [14.213531, 12.802035],
          [14.495787, 12.859396],
          [14.893386, 12.219048],
          [14.960152, 11.555574],
          [14.923565, 10.891325],
          [15.467873, 9.982337],
          [14.909354, 9.992129],
          [14.627201, 9.920919],
          [14.171466, 10.021378],
          [13.954218, 9.549495],
          [14.544467, 8.965861],
          [14.979996, 8.796104],
          [15.120866, 8.38215],
          [15.436092, 7.692812],
          [15.27946, 7.421925],
          [14.776545, 6.408498],
          [14.53656, 6.226959],
          [14.459407, 5.451761],
          [14.558936, 5.030598],
          [14.478372, 4.732605],
          [14.950953, 4.210389],
          [15.03622, 3.851367],
          [15.405396, 3.335301],
          [15.862732, 3.013537],
          [15.907381, 2.557389],
          [16.012852, 2.26764],
          [15.940919, 1.727673],
          [15.146342, 1.964015],
          [14.337813, 2.227875],
          [13.075822, 2.267097],
        ],
      ],
    },
    id: "CMR",
  },
  {
    type: "Feature",
    properties: {
      name: "Democratic Republic of the Congo",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [30.83386, 3.509166],
          [30.773347, 2.339883],
          [31.174149, 2.204465],
          [30.85267, 1.849396],
          [30.468508, 1.583805],
          [30.086154, 1.062313],
          [29.875779, 0.59738],
```

```
        [29.819503, -0.20531],
        [29.587838, -0.587406],
        [29.579466, -1.341313],
        [29.291887, -1.620056],
        [29.254835, -2.21511],
        [29.117479, -2.292211],
        [29.024926, -2.839258],
        [29.276384, -3.293907],
        [29.339998, -4.499983],
        [29.519987, -5.419979],
        [29.419993, -5.939999],
        [29.620032, -6.520015],
        [30.199997, -7.079981],
        [30.740015, -8.340007],
        [30.346086, -8.238257],
        [29.002912, -8.407032],
        [28.734867, -8.526559],
        [28.449871, -9.164918],
        [28.673682, -9.605925],
        [28.49607, -10.789884],
        [28.372253, -11.793647],
        [28.642417, -11.971569],
        [29.341548, -12.360744],
        [29.616001, -12.178895],
        [29.699614, -13.257227],
        [28.934286, -13.248958],
        [28.523562, -12.698604],
        [28.155109, -12.272481],
        [27.388799, -12.132747],
        [27.16442, -11.608748],
        [26.553088, -11.92444],
        [25.75231, -11.784965],
        [25.418118, -11.330936],
        [24.78317, -11.238694],
        [24.314516, -11.262826],
        [24.257155, -10.951993],
        [23.912215, -10.926826],
        [23.456791, -10.867863],
        [22.837345, -11.017622],
        [22.402798, -10.993075],
        [22.155268, -11.084801],
        [22.208753, -9.894796],
        [21.875182, -9.523708],
        [21.801801, -8.908707],
        [21.949131, -8.305901],
        [21.746456, -7.920085],
        [21.728111, -7.290872],
        [20.514748, -7.299606],
        [20.601823, -6.939318],
        [20.091622, -6.94309],
        [20.037723, -7.116361],
        [19.417502, -7.155429],
        [19.166613, -7.738184],
        [19.016752, -7.988246],
        [18.464176, -7.847014],
        [18.134222, -7.987678],
        [17.47297, -8.068551],
        [17.089996, -7.545689],
        [16.860191, -7.222298],
        [16.57318, -6.622645],
        [16.326528, -5.87747],
        [13.375597, -5.864241],
        [13.024869, -5.984389],
        [12.735171, -5.965682],
        [12.322432, -6.100092],
        [12.182337, -5.789931],
```

```
          [12.436688, -5.684304],
          [12.468004, -5.248362],
          [12.631612, -4.991271],
          [12.995517, -4.781103],
          [13.25824, -4.882957],
          [13.600235, -4.500138],
          [14.144956, -4.510009],
          [14.209035, -4.793092],
          [14.582604, -4.970239],
          [15.170992, -4.343507],
          [15.75354, -3.855165],
          [16.00629, -3.535133],
          [15.972803, -2.712392],
          [16.407092, -1.740927],
          [16.865307, -1.225816],
          [17.523716, -0.74383],
          [17.638645, -0.424832],
          [17.663553, -0.058084],
          [17.82654, 0.288923],
          [17.774192, 0.855659],
          [17.898835, 1.741832],
          [18.094276, 2.365722],
          [18.393792, 2.900443],
          [18.453065, 3.504386],
          [18.542982, 4.201785],
          [18.932312, 4.709506],
          [19.467784, 5.031528],
          [20.290679, 4.691678],
          [20.927591, 4.322786],
          [21.659123, 4.224342],
          [22.405124, 4.02916],
          [22.704124, 4.633051],
          [22.84148, 4.710126],
          [23.297214, 4.609693],
          [24.410531, 5.108784],
          [24.805029, 4.897247],
          [25.128833, 4.927245],
          [25.278798, 5.170408],
          [25.650455, 5.256088],
          [26.402761, 5.150875],
          [27.044065, 5.127853],
          [27.374226, 5.233944],
          [27.979977, 4.408413],
          [28.428994, 4.287155],
          [28.696678, 4.455077],
          [29.159078, 4.389267],
          [29.715995, 4.600805],
          [29.9535, 4.173699],
          [30.83386, 3.509166],
        ],
      ],
    },
    id: "COD",
  },
  {
    type: "Feature",
    properties: {
      name: "Republic of the Congo",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [12.995517, -4.781103],
          [12.62076, -4.438023],
          [12.318608, -4.60623],
```

```
          [11.914963, -5.037987],
          [11.093773, -3.978827],
          [11.855122, -3.426871],
          [11.478039, -2.765619],
          [11.820964, -2.514161],
          [12.495703, -2.391688],
          [12.575284, -1.948511],
          [13.109619, -2.42874],
          [13.992407, -2.470805],
          [14.29921, -1.998276],
          [14.425456, -1.333407],
          [14.316418, -0.552627],
          [13.843321, 0.038758],
          [14.276266, 1.19693],
          [14.026669, 1.395677],
          [13.282631, 1.314184],
          [13.003114, 1.830896],
          [13.075822, 2.267097],
          [14.337813, 2.227875],
          [15.146342, 1.964015],
          [15.940919, 1.727673],
          [16.012852, 2.26764],
          [16.537058, 3.198255],
          [17.133042, 3.728197],
          [17.8099, 3.560196],
          [18.453065, 3.504386],
          [18.393792, 2.900443],
          [18.094276, 2.365722],
          [17.898835, 1.741832],
          [17.774192, 0.855659],
          [17.82654, 0.288923],
          [17.663553, -0.058084],
          [17.638645, -0.424832],
          [17.523716, -0.74383],
          [16.865307, -1.225816],
          [16.407092, -1.740927],
          [15.972803, -2.712392],
          [16.00629, -3.535133],
          [15.75354, -3.855165],
          [15.170992, -4.343507],
          [14.582604, -4.970239],
          [14.209035, -4.793092],
          [14.144956, -4.510009],
          [13.600235, -4.500138],
          [13.25824, -4.882957],
          [12.995517, -4.781103],
        ],
      ],
    },
    id: "COG",
  },
  {
    type: "Feature",
    properties: {
      name: "Colombia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-75.373223, -0.152032],
          [-75.801466, 0.084801],
          [-76.292314, 0.416047],
          [-76.57638, 0.256936],
          [-77.424984, 0.395687],
          [-77.668613, 0.825893],
```

```
        [-77.855061, 0.809925],
        [-78.855259, 1.380924],
        [-78.990935, 1.69137],
        [-78.617831, 1.766404],
        [-78.662118, 2.267355],
        [-78.42761, 2.629556],
        [-77.931543, 2.696606],
        [-77.510431, 3.325017],
        [-77.12769, 3.849636],
        [-77.496272, 4.087606],
        [-77.307601, 4.667984],
        [-77.533221, 5.582812],
        [-77.318815, 5.845354],
        [-77.476661, 6.691116],
        [-77.881571, 7.223771],
        [-77.753414, 7.70984],
        [-77.431108, 7.638061],
        [-77.242566, 7.935278],
        [-77.474723, 8.524286],
        [-77.353361, 8.670505],
        [-76.836674, 8.638749],
        [-76.086384, 9.336821],
        [-75.6746, 9.443248],
        [-75.664704, 9.774003],
        [-75.480426, 10.61899],
        [-74.906895, 11.083045],
        [-74.276753, 11.102036],
        [-74.197223, 11.310473],
        [-73.414764, 11.227015],
        [-72.627835, 11.731972],
        [-72.238195, 11.95555],
        [-71.75409, 12.437303],
        [-71.399822, 12.376041],
        [-71.137461, 12.112982],
        [-71.331584, 11.776284],
        [-71.973922, 11.608672],
        [-72.227575, 11.108702],
        [-72.614658, 10.821975],
        [-72.905286, 10.450344],
        [-73.027604, 9.73677],
        [-73.304952, 9.152],
        [-72.78873, 9.085027],
        [-72.660495, 8.625288],
        [-72.439862, 8.405275],
        [-72.360901, 8.002638],
        [-72.479679, 7.632506],
        [-72.444487, 7.423785],
        [-72.198352, 7.340431],
        [-71.960176, 6.991615],
        [-70.674234, 7.087785],
        [-70.093313, 6.960376],
        [-69.38948, 6.099861],
        [-68.985319, 6.206805],
        [-68.265052, 6.153268],
        [-67.695087, 6.267318],
        [-67.34144, 6.095468],
        [-67.521532, 5.55687],
        [-67.744697, 5.221129],
        [-67.823012, 4.503937],
        [-67.621836, 3.839482],
        [-67.337564, 3.542342],
        [-67.303173, 3.318454],
        [-67.809938, 2.820655],
        [-67.447092, 2.600281],
        [-67.181294, 2.250638],
        [-66.876326, 1.253361],
```

```
          [-67.065048, 1.130112],
          [-67.259998, 1.719999],
          [-67.53781, 2.037163],
          [-67.868565, 1.692455],
          [-69.816973, 1.714805],
          [-69.804597, 1.089081],
          [-69.218638, 0.985677],
          [-69.252434, 0.602651],
          [-69.452396, 0.706159],
          [-70.015566, 0.541414],
          [-70.020656, -0.185156],
          [-69.577065, -0.549992],
          [-69.420486, -1.122619],
          [-69.444102, -1.556287],
          [-69.893635, -4.298187],
          [-70.394044, -3.766591],
          [-70.692682, -3.742872],
          [-70.047709, -2.725156],
          [-70.813476, -2.256865],
          [-71.413646, -2.342802],
          [-71.774761, -2.16979],
          [-72.325787, -2.434218],
          [-73.070392, -2.308954],
          [-73.659504, -1.260491],
          [-74.122395, -1.002833],
          [-74.441601, -0.53082],
          [-75.106625, -0.057205],
          [-75.373223, -0.152032],
        ],
      ],
    },
    id: "COL",
  },
  {
    type: "Feature",
    properties: {
      name: "Costa Rica",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-82.965783, 8.225028],
          [-83.508437, 8.446927],
          [-83.711474, 8.656836],
          [-83.596313, 8.830443],
          [-83.632642, 9.051386],
          [-83.909886, 9.290803],
          [-84.303402, 9.487354],
          [-84.647644, 9.615537],
          [-84.713351, 9.908052],
          [-84.97566, 10.086723],
          [-84.911375, 9.795992],
          [-85.110923, 9.55704],
          [-85.339488, 9.834542],
          [-85.660787, 9.933347],
          [-85.797445, 10.134886],
          [-85.791709, 10.439337],
          [-85.659314, 10.754331],
          [-85.941725, 10.895278],
          [-85.71254, 11.088445],
          [-85.561852, 11.217119],
          [-84.903003, 10.952303],
          [-84.673069, 11.082657],
          [-84.355931, 10.999226],
          [-84.190179, 10.79345],
```

```
          [-83.895054, 10.726839],
          [-83.655612, 10.938764],
          [-83.40232, 10.395438],
          [-83.015677, 9.992982],
          [-82.546196, 9.566135],
          [-82.932891, 9.476812],
          [-82.927155, 9.07433],
          [-82.719183, 8.925709],
          [-82.868657, 8.807266],
          [-82.829771, 8.626295],
          [-82.913176, 8.423517],
          [-82.965783, 8.225028],
        ],
      ],
    },
    id: "CRI",
  },
  {
    type: "Feature",
    properties: {
      name: "Cuba",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-82.268151, 23.188611],
          [-81.404457, 23.117271],
          [-80.618769, 23.10598],
          [-79.679524, 22.765303],
          [-79.281486, 22.399202],
          [-78.347434, 22.512166],
          [-77.993296, 22.277194],
          [-77.146422, 21.657851],
          [-76.523825, 21.20682],
          [-76.19462, 21.220565],
          [-75.598222, 21.016624],
          [-75.67106, 20.735091],
          [-74.933896, 20.693905],
          [-74.178025, 20.284628],
          [-74.296648, 20.050379],
          [-74.961595, 19.923435],
          [-75.63468, 19.873774],
          [-76.323656, 19.952891],
          [-77.755481, 19.855481],
          [-77.085108, 20.413354],
          [-77.492655, 20.673105],
          [-78.137292, 20.739949],
          [-78.482827, 21.028613],
          [-78.719867, 21.598114],
          [-79.285, 21.559175],
          [-80.217475, 21.827324],
          [-80.517535, 22.037079],
          [-81.820943, 22.192057],
          [-82.169992, 22.387109],
          [-81.795002, 22.636965],
          [-82.775898, 22.68815],
          [-83.494459, 22.168518],
          [-83.9088, 22.154565],
          [-84.052151, 21.910575],
          [-84.54703, 21.801228],
          [-84.974911, 21.896028],
          [-84.447062, 22.20495],
          [-84.230357, 22.565755],
          [-83.77824, 22.788118],
          [-83.267548, 22.983042],
```

```
          [-82.510436, 23.078747],
          [-82.268151, 23.188611],
        ],
      ],
    },
    id: "CUB",
  },
  {
    type: "Feature",
    properties: {
      name: "Northern Cyprus",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [32.73178, 35.140026],
          [32.802474, 35.145504],
          [32.946961, 35.386703],
          [33.667227, 35.373216],
          [34.576474, 35.671596],
          [33.900804, 35.245756],
          [33.973617, 35.058506],
          [33.86644, 35.093595],
          [33.675392, 35.017863],
          [33.525685, 35.038688],
          [33.475817, 35.000345],
          [33.455922, 35.101424],
          [33.383833, 35.162712],
          [33.190977, 35.173125],
          [32.919572, 35.087833],
          [32.73178, 35.140026],
        ],
      ],
    },
    id: "-99",
  },
  {
    type: "Feature",
    properties: {
      name: "Cyprus",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [33.973617, 35.058506],
          [34.004881, 34.978098],
          [32.979827, 34.571869],
          [32.490296, 34.701655],
          [32.256667, 35.103232],
          [32.73178, 35.140026],
          [32.919572, 35.087833],
          [33.190977, 35.173125],
          [33.383833, 35.162712],
          [33.455922, 35.101424],
          [33.475817, 35.000345],
          [33.525685, 35.038688],
          [33.675392, 35.017863],
          [33.86644, 35.093595],
          [33.973617, 35.058506],
        ],
      ],
    },
    id: "CYP",
  },
```

```
    {
      type: "Feature",
      properties: {
        name: "Czech Republic",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [16.960288, 48.596982],
            [16.499283, 48.785808],
            [16.029647, 48.733899],
            [15.253416, 49.039074],
            [14.901447, 48.964402],
            [14.338898, 48.555305],
            [13.595946, 48.877172],
            [13.031329, 49.307068],
            [12.521024, 49.547415],
            [12.415191, 49.969121],
            [12.240111, 50.266338],
            [12.966837, 50.484076],
            [13.338132, 50.733234],
            [14.056228, 50.926918],
            [14.307013, 51.117268],
            [14.570718, 51.002339],
            [15.016996, 51.106674],
            [15.490972, 50.78473],
            [16.238627, 50.697733],
            [16.176253, 50.422607],
            [16.719476, 50.215747],
            [16.868769, 50.473974],
            [17.554567, 50.362146],
            [17.649445, 50.049038],
            [18.392914, 49.988629],
            [18.853144, 49.49623],
            [18.554971, 49.495015],
            [18.399994, 49.315001],
            [18.170498, 49.271515],
            [18.104973, 49.043983],
            [17.913512, 48.996493],
            [17.886485, 48.903475],
            [17.545007, 48.800019],
            [17.101985, 48.816969],
            [16.960288, 48.596982],
          ],
        ],
      },
      id: "CZE",
    },
    {
      type: "Feature",
      properties: {
        name: "Germany",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [9.921906, 54.983104],
            [9.93958, 54.596642],
            [10.950112, 54.363607],
            [10.939467, 54.008693],
            [11.956252, 54.196486],
            [12.51844, 54.470371],
            [13.647467, 54.075511],
            [14.119686, 53.757029],
```

```
          [14.353315, 53.248171],
          [14.074521, 52.981263],
          [14.4376, 52.62485],
          [14.685026, 52.089947],
          [14.607098, 51.745188],
          [15.016996, 51.106674],
          [14.570718, 51.002339],
          [14.307013, 51.117268],
          [14.056228, 50.926918],
          [13.338132, 50.733234],
          [12.966837, 50.484076],
          [12.240111, 50.266338],
          [12.415191, 49.969121],
          [12.521024, 49.547415],
          [13.031329, 49.307068],
          [13.595946, 48.877172],
          [13.243357, 48.416115],
          [12.884103, 48.289146],
          [13.025851, 47.637584],
          [12.932627, 47.467646],
          [12.62076, 47.672388],
          [12.141357, 47.703083],
          [11.426414, 47.523766],
          [10.544504, 47.566399],
          [10.402084, 47.302488],
          [9.896068, 47.580197],
          [9.594226, 47.525058],
          [8.522612, 47.830828],
          [8.317301, 47.61358],
          [7.466759, 47.620582],
          [7.593676, 48.333019],
          [8.099279, 49.017784],
          [6.65823, 49.201958],
          [6.18632, 49.463803],
          [6.242751, 49.902226],
          [6.043073, 50.128052],
          [6.156658, 50.803721],
          [5.988658, 51.851616],
          [6.589397, 51.852029],
          [6.84287, 52.22844],
          [7.092053, 53.144043],
          [6.90514, 53.482162],
          [7.100425, 53.693932],
          [7.936239, 53.748296],
          [8.121706, 53.527792],
          [8.800734, 54.020786],
          [8.572118, 54.395646],
          [8.526229, 54.962744],
          [9.282049, 54.830865],
          [9.921906, 54.983104],
        ],
      ],
    },
    id: "DEU",
  },
  {
    type: "Feature",
    properties: {
      name: "Djibouti",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [43.081226, 12.699639],
          [43.317852, 12.390148],
```

```
          [43.286381, 11.974928],
          [42.715874, 11.735641],
          [43.145305, 11.46204],
          [42.776852, 10.926879],
          [42.55493, 11.10511],
          [42.31414, 11.0342],
          [41.75557, 11.05091],
          [41.73959, 11.35511],
          [41.66176, 11.6312],
          [42, 12.1],
          [42.35156, 12.54223],
          [42.779642, 12.455416],
          [43.081226, 12.699639],
        ],
      ],
    },
    id: "DJI",
  },
  {
    type: "Feature",
    properties: {
      name: "Denmark",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [12.690006, 55.609991],
            [12.089991, 54.800015],
            [11.043543, 55.364864],
            [10.903914, 55.779955],
            [12.370904, 56.111407],
            [12.690006, 55.609991],
          ],
        ],
        [
          [
            [10.912182, 56.458621],
            [10.667804, 56.081383],
            [10.369993, 56.190007],
            [9.649985, 55.469999],
            [9.921906, 54.983104],
            [9.282049, 54.830865],
            [8.526229, 54.962744],
            [8.120311, 55.517723],
            [8.089977, 56.540012],
            [8.256582, 56.809969],
            [8.543438, 57.110003],
            [9.424469, 57.172066],
            [9.775559, 57.447941],
            [10.580006, 57.730017],
            [10.546106, 57.215733],
            [10.25, 56.890016],
            [10.369993, 56.609982],
            [10.912182, 56.458621],
          ],
        ],
      ],
    },
    id: "DNK",
  },
  {
    type: "Feature",
    properties: {
      name: "Dominican Republic",
```

```
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-71.712361, 19.714456],
            [-71.587304, 19.884911],
            [-70.806706, 19.880286],
            [-70.214365, 19.622885],
            [-69.950815, 19.648],
            [-69.76925, 19.293267],
            [-69.222126, 19.313214],
            [-69.254346, 19.015196],
            [-68.809412, 18.979074],
            [-68.317943, 18.612198],
            [-68.689316, 18.205142],
            [-69.164946, 18.422648],
            [-69.623988, 18.380713],
            [-69.952934, 18.428307],
            [-70.133233, 18.245915],
            [-70.517137, 18.184291],
            [-70.669298, 18.426886],
            [-70.99995, 18.283329],
            [-71.40021, 17.598564],
            [-71.657662, 17.757573],
            [-71.708305, 18.044997],
            [-71.687738, 18.31666],
            [-71.945112, 18.6169],
            [-71.701303, 18.785417],
            [-71.624873, 19.169838],
            [-71.712361, 19.714456],
          ],
        ],
      },
      id: "DOM",
    },
    {
      type: "Feature",
      properties: {
        name: "Algeria",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [11.999506, 23.471668],
            [8.572893, 21.565661],
            [5.677566, 19.601207],
            [4.267419, 19.155265],
            [3.158133, 19.057364],
            [3.146661, 19.693579],
            [2.683588, 19.85623],
            [2.060991, 20.142233],
            [1.823228, 20.610809],
            [-1.550055, 22.792666],
            [-4.923337, 24.974574],
            [-8.6844, 27.395744],
            [-8.665124, 27.589479],
            [-8.66559, 27.656426],
            [-8.674116, 28.841289],
            [-7.059228, 29.579228],
            [-6.060632, 29.7317],
            [-5.242129, 30.000443],
            [-4.859646, 30.501188],
            [-3.690441, 30.896952],
            [-3.647498, 31.637294],
```

```
          [-3.06898, 31.724498],
          [-2.616605, 32.094346],
          [-1.307899, 32.262889],
          [-1.124551, 32.651522],
          [-1.388049, 32.864015],
          [-1.733455, 33.919713],
          [-1.792986, 34.527919],
          [-2.169914, 35.168396],
          [-1.208603, 35.714849],
          [-0.127454, 35.888662],
          [0.503877, 36.301273],
          [1.466919, 36.605647],
          [3.161699, 36.783905],
          [4.815758, 36.865037],
          [5.32012, 36.716519],
          [6.26182, 37.110655],
          [7.330385, 37.118381],
          [7.737078, 36.885708],
          [8.420964, 36.946427],
          [8.217824, 36.433177],
          [8.376368, 35.479876],
          [8.140981, 34.655146],
          [7.524482, 34.097376],
          [7.612642, 33.344115],
          [8.430473, 32.748337],
          [8.439103, 32.506285],
          [9.055603, 32.102692],
          [9.48214, 30.307556],
          [9.805634, 29.424638],
          [9.859998, 28.95999],
          [9.683885, 28.144174],
          [9.756128, 27.688259],
          [9.629056, 27.140953],
          [9.716286, 26.512206],
          [9.319411, 26.094325],
          [9.910693, 25.365455],
          [9.948261, 24.936954],
          [10.303847, 24.379313],
          [10.771364, 24.562532],
          [11.560669, 24.097909],
          [11.999506, 23.471668],
        ],
      ],
    },
    id: "DZA",
  },
  {
    type: "Feature",
    properties: {
      name: "Ecuador",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-80.302561, -3.404856],
          [-79.770293, -2.657512],
          [-79.986559, -2.220794],
          [-80.368784, -2.685159],
          [-80.967765, -2.246943],
          [-80.764806, -1.965048],
          [-80.933659, -1.057455],
          [-80.58337, -0.906663],
          [-80.399325, -0.283703],
          [-80.020898, 0.36034],
          [-80.09061, 0.768429],
```

```
          [-79.542762, 0.982938],
          [-78.855259, 1.380924],
          [-77.855061, 0.809925],
          [-77.668613, 0.825893],
          [-77.424984, 0.395687],
          [-76.57638, 0.256936],
          [-76.292314, 0.416047],
          [-75.801466, 0.084801],
          [-75.373223, -0.152032],
          [-75.233723, -0.911417],
          [-75.544996, -1.56161],
          [-76.635394, -2.608678],
          [-77.837905, -3.003021],
          [-78.450684, -3.873097],
          [-78.639897, -4.547784],
          [-79.205289, -4.959129],
          [-79.624979, -4.454198],
          [-80.028908, -4.346091],
          [-80.442242, -4.425724],
          [-80.469295, -4.059287],
          [-80.184015, -3.821162],
          [-80.302561, -3.404856],
        ],
      ],
    },
    id: "ECU",
  },
  {
    type: "Feature",
    properties: {
      name: "Egypt",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [34.9226, 29.50133],
          [34.64174, 29.09942],
          [34.42655, 28.34399],
          [34.15451, 27.8233],
          [33.92136, 27.6487],
          [33.58811, 27.97136],
          [33.13676, 28.41765],
          [32.42323, 29.85108],
          [32.32046, 29.76043],
          [32.73482, 28.70523],
          [33.34876, 27.69989],
          [34.10455, 26.14227],
          [34.47387, 25.59856],
          [34.79507, 25.03375],
          [35.69241, 23.92671],
          [35.49372, 23.75237],
          [35.52598, 23.10244],
          [36.69069, 22.20485],
          [36.86623, 22],
          [32.9, 22],
          [29.02, 22],
          [25, 22],
          [25, 25.6825],
          [25, 29.238655],
          [24.70007, 30.04419],
          [24.95762, 30.6616],
          [24.80287, 31.08929],
          [25.16482, 31.56915],
          [26.49533, 31.58568],
          [27.45762, 31.32126],
```

```
          [28.45048, 31.02577],
          [28.91353, 30.87005],
          [29.68342, 31.18686],
          [30.09503, 31.4734],
          [30.97693, 31.55586],
          [31.68796, 31.4296],
          [31.96041, 30.9336],
          [32.19247, 31.26034],
          [32.99392, 31.02407],
          [33.7734, 30.96746],
          [34.26544, 31.21936],
          [34.9226, 29.50133],
        ],
      ],
    },
    id: "EGY",
  },
  {
    type: "Feature",
    properties: {
      name: "Eritrea",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [42.35156, 12.54223],
          [42.00975, 12.86582],
          [41.59856, 13.45209],
          [41.155194, 13.77332],
          [40.8966, 14.11864],
          [40.026219, 14.519579],
          [39.34061, 14.53155],
          [39.0994, 14.74064],
          [38.51295, 14.50547],
          [37.90607, 14.95943],
          [37.59377, 14.2131],
          [36.42951, 14.42211],
          [36.323189, 14.822481],
          [36.75386, 16.291874],
          [36.85253, 16.95655],
          [37.16747, 17.26314],
          [37.904, 17.42754],
          [38.41009, 17.998307],
          [38.990623, 16.840626],
          [39.26611, 15.922723],
          [39.814294, 15.435647],
          [41.179275, 14.49108],
          [41.734952, 13.921037],
          [42.276831, 13.343992],
          [42.589576, 13.000421],
          [43.081226, 12.699639],
          [42.779642, 12.455416],
          [42.35156, 12.54223],
        ],
      ],
    },
    id: "ERI",
  },
  {
    type: "Feature",
    properties: {
      name: "Spain",
    },
    geometry: {
      type: "Polygon",
```

```
      coordinates: [
        [
          [-9.034818, 41.880571],
          [-8.984433, 42.592775],
          [-9.392884, 43.026625],
          [-7.97819, 43.748338],
          [-6.754492, 43.567909],
          [-5.411886, 43.57424],
          [-4.347843, 43.403449],
          [-3.517532, 43.455901],
          [-1.901351, 43.422802],
          [-1.502771, 43.034014],
          [0.338047, 42.579546],
          [0.701591, 42.795734],
          [1.826793, 42.343385],
          [2.985999, 42.473015],
          [3.039484, 41.89212],
          [2.091842, 41.226089],
          [0.810525, 41.014732],
          [0.721331, 40.678318],
          [0.106692, 40.123934],
          [-0.278711, 39.309978],
          [0.111291, 38.738514],
          [-0.467124, 38.292366],
          [-0.683389, 37.642354],
          [-1.438382, 37.443064],
          [-2.146453, 36.674144],
          [-3.415781, 36.6589],
          [-4.368901, 36.677839],
          [-4.995219, 36.324708],
          [-5.37716, 35.94685],
          [-5.866432, 36.029817],
          [-6.236694, 36.367677],
          [-6.520191, 36.942913],
          [-7.453726, 37.097788],
          [-7.537105, 37.428904],
          [-7.166508, 37.803894],
          [-7.029281, 38.075764],
          [-7.374092, 38.373059],
          [-7.098037, 39.030073],
          [-7.498632, 39.629571],
          [-7.066592, 39.711892],
          [-7.026413, 40.184524],
          [-6.86402, 40.330872],
          [-6.851127, 41.111083],
          [-6.389088, 41.381815],
          [-6.668606, 41.883387],
          [-7.251309, 41.918346],
          [-7.422513, 41.792075],
          [-8.013175, 41.790886],
          [-8.263857, 42.280469],
          [-8.671946, 42.134689],
          [-9.034818, 41.880571],
        ],
      ],
    },
    id: "ESP",
  },
  {
    type: "Feature",
    properties: {
      name: "Estonia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
```

```
        [
          [24.312863, 57.793424],
          [24.428928, 58.383413],
          [24.061198, 58.257375],
          [23.42656, 58.612753],
          [23.339795, 59.18724],
          [24.604214, 59.465854],
          [25.864189, 59.61109],
          [26.949136, 59.445803],
          [27.981114, 59.475388],
          [28.131699, 59.300825],
          [27.420166, 58.724581],
          [27.716686, 57.791899],
          [27.288185, 57.474528],
          [26.463532, 57.476389],
          [25.60281, 57.847529],
          [25.164594, 57.970157],
          [24.312863, 57.793424],
        ],
      ],
    },
    id: "EST",
  },
  {
    type: "Feature",
    properties: {
      name: "Ethiopia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [37.90607, 14.95943],
          [38.51295, 14.50547],
          [39.0994, 14.74064],
          [39.34061, 14.53155],
          [40.02625, 14.51959],
          [40.8966, 14.11864],
          [41.1552, 13.77333],
          [41.59856, 13.45209],
          [42.00975, 12.86582],
          [42.35156, 12.54223],
          [42, 12.1],
          [41.66176, 11.6312],
          [41.73959, 11.35511],
          [41.75557, 11.05091],
          [42.31414, 11.0342],
          [42.55493, 11.10511],
          [42.776852, 10.926879],
          [42.55876, 10.57258],
          [42.92812, 10.02194],
          [43.29699, 9.54048],
          [43.67875, 9.18358],
          [46.94834, 7.99688],
          [47.78942, 8.003],
          [44.9636, 5.00162],
          [43.66087, 4.95755],
          [42.76967, 4.25259],
          [42.12861, 4.23413],
          [41.855083, 3.918912],
          [41.1718, 3.91909],
          [40.76848, 4.25702],
          [39.85494, 3.83879],
          [39.559384, 3.42206],
          [38.89251, 3.50074],
          [38.67114, 3.61607],
```

```
          [38.43697, 3.58851],
          [38.120915, 3.598605],
          [36.855093, 4.447864],
          [36.159079, 4.447864],
          [35.817448, 4.776966],
          [35.817448, 5.338232],
          [35.298007, 5.506],
          [34.70702, 6.59422],
          [34.25032, 6.82607],
          [34.0751, 7.22595],
          [33.56829, 7.71334],
          [32.95418, 7.78497],
          [33.2948, 8.35458],
          [33.8255, 8.37916],
          [33.97498, 8.68456],
          [33.96162, 9.58358],
          [34.25745, 10.63009],
          [34.73115, 10.91017],
          [34.83163, 11.31896],
          [35.26049, 12.08286],
          [35.86363, 12.57828],
          [36.27022, 13.56333],
          [36.42951, 14.42211],
          [37.59377, 14.2131],
          [37.90607, 14.95943],
        ],
      ],
    },
    id: "ETH",
  },
  {
    type: "Feature",
    properties: {
      name: "Finland",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [28.59193, 69.064777],
          [28.445944, 68.364613],
          [29.977426, 67.698297],
          [29.054589, 66.944286],
          [30.21765, 65.80598],
          [29.54443, 64.948672],
          [30.444685, 64.204453],
          [30.035872, 63.552814],
          [31.516092, 62.867687],
          [31.139991, 62.357693],
          [30.211107, 61.780028],
          [28.069998, 60.503517],
          [26.255173, 60.423961],
          [24.496624, 60.057316],
          [22.869695, 59.846373],
          [22.290764, 60.391921],
          [21.322244, 60.72017],
          [21.544866, 61.705329],
          [21.059211, 62.607393],
          [21.536029, 63.189735],
          [22.442744, 63.81781],
          [24.730512, 64.902344],
          [25.398068, 65.111427],
          [25.294043, 65.534346],
          [23.903379, 66.006927],
          [23.56588, 66.396051],
          [23.539473, 67.936009],
```

```
          [21.978535, 68.616846],
          [20.645593, 69.106247],
          [21.244936, 69.370443],
          [22.356238, 68.841741],
          [23.66205, 68.891247],
          [24.735679, 68.649557],
          [25.689213, 69.092114],
          [26.179622, 69.825299],
          [27.732292, 70.164193],
          [29.015573, 69.766491],
          [28.59193, 69.064777],
        ],
      ],
    },
    id: "FIN",
  },
  {
    type: "Feature",
    properties: {
      name: "Fiji",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [178.3736, -17.33992],
            [178.71806, -17.62846],
            [178.55271, -18.15059],
            [177.93266, -18.28799],
            [177.38146, -18.16432],
            [177.28504, -17.72465],
            [177.67087, -17.38114],
            [178.12557, -17.50481],
            [178.3736, -17.33992],
          ],
        ],
        [
          [
            [179.364143, -16.801354],
            [178.725059, -17.012042],
            [178.596839, -16.63915],
            [179.096609, -16.433984],
            [179.413509, -16.379054],
            [180, -16.067133],
            [180, -16.555217],
            [179.364143, -16.801354],
          ],
        ],
        [
          [
            [-179.917369, -16.501783],
            [-180, -16.555217],
            [-180, -16.067133],
            [-179.79332, -16.020882],
            [-179.917369, -16.501783],
          ],
        ],
      ],
    },
    id: "FJI",
  },
  {
    type: "Feature",
    properties: {
      name: "Falkland Islands",
```

```
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-61.2, -51.85],
            [-60, -51.25],
            [-59.15, -51.5],
            [-58.55, -51.1],
            [-57.75, -51.55],
            [-58.05, -51.9],
            [-59.4, -52.2],
            [-59.85, -51.85],
            [-60.7, -52.3],
            [-61.2, -51.85],
          ],
        ],
      },
      id: "FLK",
    },
    {
      type: "Feature",
      properties: {
        name: "France",
      },
      geometry: {
        type: "MultiPolygon",
        coordinates: [
          [
            [
              [-52.556425, 2.504705],
              [-52.939657, 2.124858],
              [-53.418465, 2.053389],
              [-53.554839, 2.334897],
              [-53.778521, 2.376703],
              [-54.088063, 2.105557],
              [-54.524754, 2.311849],
              [-54.27123, 2.738748],
              [-54.184284, 3.194172],
              [-54.011504, 3.62257],
              [-54.399542, 4.212611],
              [-54.478633, 4.896756],
              [-53.958045, 5.756548],
              [-53.618453, 5.646529],
              [-52.882141, 5.409851],
              [-51.823343, 4.565768],
              [-51.657797, 4.156232],
              [-52.249338, 3.241094],
              [-52.556425, 2.504705],
            ],
          ],
          [
            [
              [9.560016, 42.152492],
              [9.229752, 41.380007],
              [8.775723, 41.583612],
              [8.544213, 42.256517],
              [8.746009, 42.628122],
              [9.390001, 43.009985],
              [9.560016, 42.152492],
            ],
          ],
          [
            [
              [3.588184, 50.378992],
              [4.286023, 49.907497],
```

```
            [4.799222, 49.985373],
            [5.674052, 49.529484],
            [5.897759, 49.442667],
            [6.18632, 49.463803],
            [6.65823, 49.201958],
            [8.099279, 49.017784],
            [7.593676, 48.333019],
            [7.466759, 47.620582],
            [7.192202, 47.449766],
            [6.736571, 47.541801],
            [6.768714, 47.287708],
            [6.037389, 46.725779],
            [6.022609, 46.27299],
            [6.5001, 46.429673],
            [6.843593, 45.991147],
            [6.802355, 45.70858],
            [7.096652, 45.333099],
            [6.749955, 45.028518],
            [7.007562, 44.254767],
            [7.549596, 44.127901],
            [7.435185, 43.693845],
            [6.529245, 43.128892],
            [4.556963, 43.399651],
            [3.100411, 43.075201],
            [2.985999, 42.473015],
            [1.826793, 42.343385],
            [0.701591, 42.795734],
            [0.338047, 42.579546],
            [-1.502771, 43.034014],
            [-1.901351, 43.422802],
            [-1.384225, 44.02261],
            [-1.193798, 46.014918],
            [-2.225724, 47.064363],
            [-2.963276, 47.570327],
            [-4.491555, 47.954954],
            [-4.59235, 48.68416],
            [-3.295814, 48.901692],
            [-1.616511, 48.644421],
            [-1.933494, 49.776342],
            [-0.989469, 49.347376],
            [1.338761, 50.127173],
            [1.639001, 50.946606],
            [2.513573, 51.148506],
            [2.658422, 50.796848],
            [3.123252, 50.780363],
            [3.588184, 50.378992],
          ],
        ],
      ],
    },
    id: "FRA",
  },
  {
    type: "Feature",
    properties: {
      name: "Gabon",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [11.093773, -3.978827],
          [10.066135, -2.969483],
          [9.405245, -2.144313],
          [8.797996, -1.111301],
          [8.830087, -0.779074],
```

```
          [9.04842, -0.459351],
          [9.291351, 0.268666],
          [9.492889, 1.01012],
          [9.830284, 1.067894],
          [11.285079, 1.057662],
          [11.276449, 2.261051],
          [11.751665, 2.326758],
          [12.35938, 2.192812],
          [12.951334, 2.321616],
          [13.075822, 2.267097],
          [13.003114, 1.830896],
          [13.282631, 1.314184],
          [14.026669, 1.395677],
          [14.276266, 1.19693],
          [13.843321, 0.038758],
          [14.316418, -0.552627],
          [14.425456, -1.333407],
          [14.29921, -1.998276],
          [13.992407, -2.470805],
          [13.109619, -2.42874],
          [12.575284, -1.948511],
          [12.495703, -2.391688],
          [11.820964, -2.514161],
          [11.478039, -2.765619],
          [11.855122, -3.426871],
          [11.093773, -3.978827],
        ],
      ],
    },
    id: "GAB",
  },
  {
    type: "Feature",
    properties: {
      name: "UK",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [-5.661949, 54.554603],
            [-6.197885, 53.867565],
            [-6.95373, 54.073702],
            [-7.572168, 54.059956],
            [-7.366031, 54.595841],
            [-7.572168, 55.131622],
            [-6.733847, 55.17286],
            [-5.661949, 54.554603],
          ],
        ],
        [
          [
            [-3.005005, 58.635],
            [-4.073828, 57.553025],
            [-3.055002, 57.690019],
            [-1.959281, 57.6848],
            [-2.219988, 56.870017],
            [-3.119003, 55.973793],
            [-2.085009, 55.909998],
            [-2.005676, 55.804903],
            [-1.114991, 54.624986],
            [-0.430485, 54.464376],
            [0.184981, 53.325014],
            [0.469977, 52.929999],
            [1.681531, 52.73952],
```

```
            [1.559988, 52.099998],
            [1.050562, 51.806761],
            [1.449865, 51.289428],
            [0.550334, 50.765739],
            [-0.787517, 50.774989],
            [-2.489998, 50.500019],
            [-2.956274, 50.69688],
            [-3.617448, 50.228356],
            [-4.542508, 50.341837],
            [-5.245023, 49.96],
            [-5.776567, 50.159678],
            [-4.30999, 51.210001],
            [-3.414851, 51.426009],
            [-3.422719, 51.426848],
            [-4.984367, 51.593466],
            [-5.267296, 51.9914],
            [-4.222347, 52.301356],
            [-4.770013, 52.840005],
            [-4.579999, 53.495004],
            [-3.093831, 53.404547],
            [-3.09208, 53.404441],
            [-2.945009, 53.985],
            [-3.614701, 54.600937],
            [-3.630005, 54.615013],
            [-4.844169, 54.790971],
            [-5.082527, 55.061601],
            [-4.719112, 55.508473],
            [-5.047981, 55.783986],
            [-5.586398, 55.311146],
            [-5.644999, 56.275015],
            [-6.149981, 56.78501],
            [-5.786825, 57.818848],
            [-5.009999, 58.630013],
            [-4.211495, 58.550845],
            [-3.005005, 58.635],
          ],
        ],
      ],
    },
    id: "GBR",
  },
  {
    type: "Feature",
    properties: {
      name: "Georgia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [41.554084, 41.535656],
          [41.703171, 41.962943],
          [41.45347, 42.645123],
          [40.875469, 43.013628],
          [40.321394, 43.128634],
          [39.955009, 43.434998],
          [40.076965, 43.553104],
          [40.922185, 43.382159],
          [42.394395, 43.220308],
          [43.756017, 42.740828],
          [43.9312, 42.554974],
          [44.537623, 42.711993],
          [45.470279, 42.502781],
          [45.77641, 42.092444],
          [46.404951, 41.860675],
          [46.145432, 41.722802],
```

```
          [46.637908, 41.181673],
          [46.501637, 41.064445],
          [45.962601, 41.123873],
          [45.217426, 41.411452],
          [44.97248, 41.248129],
          [43.582746, 41.092143],
          [42.619549, 41.583173],
          [41.554084, 41.535656],
        ],
      ],
    },
    id: "GEO",
  },
  {
    type: "Feature",
    properties: {
      name: "Ghana",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [1.060122, 5.928837],
          [-0.507638, 5.343473],
          [-1.063625, 5.000548],
          [-1.964707, 4.710462],
          [-2.856125, 4.994476],
          [-2.810701, 5.389051],
          [-3.24437, 6.250472],
          [-2.983585, 7.379705],
          [-2.56219, 8.219628],
          [-2.827496, 9.642461],
          [-2.963896, 10.395335],
          [-2.940409, 10.96269],
          [-1.203358, 11.009819],
          [-0.761576, 10.93693],
          [-0.438702, 11.098341],
          [0.023803, 11.018682],
          [-0.049785, 10.706918],
          [0.36758, 10.191213],
          [0.365901, 9.465004],
          [0.461192, 8.677223],
          [0.712029, 8.312465],
          [0.490957, 7.411744],
          [0.570384, 6.914359],
          [0.836931, 6.279979],
          [1.060122, 5.928837],
        ],
      ],
    },
    id: "GHA",
  },
  {
    type: "Feature",
    properties: {
      name: "Guinea",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-8.439298, 7.686043],
          [-8.722124, 7.711674],
          [-8.926065, 7.309037],
          [-9.208786, 7.313921],
          [-9.403348, 7.526905],
```

```
      [-9.33728, 7.928534],
      [-9.755342, 8.541055],
      [-10.016567, 8.428504],
      [-10.230094, 8.406206],
      [-10.505477, 8.348896],
      [-10.494315, 8.715541],
      [-10.65477, 8.977178],
      [-10.622395, 9.26791],
      [-10.839152, 9.688246],
      [-11.117481, 10.045873],
      [-11.917277, 10.046984],
      [-12.150338, 9.858572],
      [-12.425929, 9.835834],
      [-12.596719, 9.620188],
      [-12.711958, 9.342712],
      [-13.24655, 8.903049],
      [-13.685154, 9.494744],
      [-14.074045, 9.886167],
      [-14.330076, 10.01572],
      [-14.579699, 10.214467],
      [-14.693232, 10.656301],
      [-14.839554, 10.876572],
      [-15.130311, 11.040412],
      [-14.685687, 11.527824],
      [-14.382192, 11.509272],
      [-14.121406, 11.677117],
      [-13.9008, 11.678719],
      [-13.743161, 11.811269],
      [-13.828272, 12.142644],
      [-13.718744, 12.247186],
      [-13.700476, 12.586183],
      [-13.217818, 12.575874],
      [-12.499051, 12.33209],
      [-12.278599, 12.35444],
      [-12.203565, 12.465648],
      [-11.658301, 12.386583],
      [-11.513943, 12.442988],
      [-11.456169, 12.076834],
      [-11.297574, 12.077971],
      [-11.036556, 12.211245],
      [-10.87083, 12.177887],
      [-10.593224, 11.923975],
      [-10.165214, 11.844084],
      [-9.890993, 12.060479],
      [-9.567912, 12.194243],
      [-9.327616, 12.334286],
      [-9.127474, 12.30806],
      [-8.905265, 12.088358],
      [-8.786099, 11.812561],
      [-8.376305, 11.393646],
      [-8.581305, 11.136246],
      [-8.620321, 10.810891],
      [-8.407311, 10.909257],
      [-8.282357, 10.792597],
      [-8.335377, 10.494812],
      [-8.029944, 10.206535],
      [-8.229337, 10.12902],
      [-8.309616, 9.789532],
      [-8.079114, 9.376224],
      [-7.8321, 8.575704],
      [-8.203499, 8.455453],
      [-8.299049, 8.316444],
      [-8.221792, 8.123329],
      [-8.280703, 7.68718],
      [-8.439298, 7.686043],
    ],
```

```
        ],
      },
      id: "GIN",
    },
    {
      type: "Feature",
      properties: {
        name: "Gambia",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-16.841525, 13.151394],
            [-16.713729, 13.594959],
            [-15.624596, 13.623587],
            [-15.39877, 13.860369],
            [-15.081735, 13.876492],
            [-14.687031, 13.630357],
            [-14.376714, 13.62568],
            [-14.046992, 13.794068],
            [-13.844963, 13.505042],
            [-14.277702, 13.280585],
            [-14.712197, 13.298207],
            [-15.141163, 13.509512],
            [-15.511813, 13.27857],
            [-15.691001, 13.270353],
            [-15.931296, 13.130284],
            [-16.841525, 13.151394],
          ],
        ],
      },
      id: "GMB",
    },
    {
      type: "Feature",
      properties: {
        name: "Guinea Bissau",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-15.130311, 11.040412],
            [-15.66418, 11.458474],
            [-16.085214, 11.524594],
            [-16.314787, 11.806515],
            [-16.308947, 11.958702],
            [-16.613838, 12.170911],
            [-16.677452, 12.384852],
            [-16.147717, 12.547762],
            [-15.816574, 12.515567],
            [-15.548477, 12.62817],
            [-13.700476, 12.586183],
            [-13.718744, 12.247186],
            [-13.828272, 12.142644],
            [-13.743161, 11.811269],
            [-13.9008, 11.678719],
            [-14.121406, 11.677117],
            [-14.382192, 11.509272],
            [-14.685687, 11.527824],
            [-15.130311, 11.040412],
          ],
        ],
      },
      id: "GNB",
```

```
    },
    {
      type: "Feature",
      properties: {
        name: "Equatorial Guinea",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [9.492889, 1.01012],
            [9.305613, 1.160911],
            [9.649158, 2.283866],
            [11.276449, 2.261051],
            [11.285079, 1.057662],
            [9.830284, 1.067894],
            [9.492889, 1.01012],
          ],
        ],
      },
      id: "GNQ",
    },
    {
      type: "Feature",
      properties: {
        name: "Greece",
      },
      geometry: {
        type: "MultiPolygon",
        coordinates: [
          [
            [
              [23.69998, 35.705004],
              [24.246665, 35.368022],
              [25.025015, 35.424996],
              [25.769208, 35.354018],
              [25.745023, 35.179998],
              [26.290003, 35.29999],
              [26.164998, 35.004995],
              [24.724982, 34.919988],
              [24.735007, 35.084991],
              [23.514978, 35.279992],
              [23.69998, 35.705004],
            ],
          ],
          [
            [
              [26.604196, 41.562115],
              [26.294602, 40.936261],
              [26.056942, 40.824123],
              [25.447677, 40.852545],
              [24.925848, 40.947062],
              [23.714811, 40.687129],
              [24.407999, 40.124993],
              [23.899968, 39.962006],
              [23.342999, 39.960998],
              [22.813988, 40.476005],
              [22.626299, 40.256561],
              [22.849748, 39.659311],
              [23.350027, 39.190011],
              [22.973099, 38.970903],
              [23.530016, 38.510001],
              [24.025025, 38.219993],
              [24.040011, 37.655015],
              [23.115003, 37.920011],
              [23.409972, 37.409991],
```

```
            [22.774972, 37.30501],
            [23.154225, 36.422506],
            [22.490028, 36.41],
            [21.670026, 36.844986],
            [21.295011, 37.644989],
            [21.120034, 38.310323],
            [20.730032, 38.769985],
            [20.217712, 39.340235],
            [20.150016, 39.624998],
            [20.615, 40.110007],
            [20.674997, 40.435],
            [20.99999, 40.580004],
            [21.02004, 40.842727],
            [21.674161, 40.931275],
            [22.055378, 41.149866],
            [22.597308, 41.130487],
            [22.76177, 41.3048],
            [22.952377, 41.337994],
            [23.692074, 41.309081],
            [24.492645, 41.583896],
            [25.197201, 41.234486],
            [26.106138, 41.328899],
            [26.117042, 41.826905],
            [26.604196, 41.562115],
          ],
        ],
      ],
    },
    id: "GRC",
  },
  {
    type: "Feature",
    properties: {
      name: "Greenland",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-46.76379, 82.62796],
          [-43.40644, 83.22516],
          [-39.89753, 83.18018],
          [-38.62214, 83.54905],
          [-35.08787, 83.64513],
          [-27.10046, 83.51966],
          [-20.84539, 82.72669],
          [-22.69182, 82.34165],
          [-26.51753, 82.29765],
          [-31.9, 82.2],
          [-31.39646, 82.02154],
          [-27.85666, 82.13178],
          [-24.84448, 81.78697],
          [-22.90328, 82.09317],
          [-22.07175, 81.73449],
          [-23.16961, 81.15271],
          [-20.62363, 81.52462],
          [-15.76818, 81.91245],
          [-12.77018, 81.71885],
          [-12.20855, 81.29154],
          [-16.28533, 80.58004],
          [-16.85, 80.35],
          [-20.04624, 80.17708],
          [-17.73035, 80.12912],
          [-18.9, 79.4],
          [-19.70499, 78.75128],
          [-19.67353, 77.63859],
```

```
[-18.47285, 76.98565],
[-20.03503, 76.94434],
[-21.67944, 76.62795],
[-19.83407, 76.09808],
[-19.59896, 75.24838],
[-20.66818, 75.15585],
[-19.37281, 74.29561],
[-21.59422, 74.22382],
[-20.43454, 73.81713],
[-20.76234, 73.46436],
[-22.17221, 73.30955],
[-23.56593, 73.30663],
[-22.31311, 72.62928],
[-22.29954, 72.18409],
[-24.27834, 72.59788],
[-24.79296, 72.3302],
[-23.44296, 72.08016],
[-22.13281, 71.46898],
[-21.75356, 70.66369],
[-23.53603, 70.471],
[-24.30702, 70.85649],
[-25.54341, 71.43094],
[-25.20135, 70.75226],
[-26.36276, 70.22646],
[-23.72742, 70.18401],
[-22.34902, 70.12946],
[-25.02927, 69.2588],
[-27.74737, 68.47046],
[-30.67371, 68.12503],
[-31.77665, 68.12078],
[-32.81105, 67.73547],
[-34.20196, 66.67974],
[-36.35284, 65.9789],
[-37.04378, 65.93768],
[-38.37505, 65.69213],
[-39.81222, 65.45848],
[-40.66899, 64.83997],
[-40.68281, 64.13902],
[-41.1887, 63.48246],
[-42.81938, 62.68233],
[-42.41666, 61.90093],
[-42.86619, 61.07404],
[-43.3784, 60.09772],
[-44.7875, 60.03676],
[-46.26364, 60.85328],
[-48.26294, 60.85843],
[-49.23308, 61.40681],
[-49.90039, 62.38336],
[-51.63325, 63.62691],
[-52.14014, 64.27842],
[-52.27659, 65.1767],
[-53.66166, 66.09957],
[-53.30161, 66.8365],
[-53.96911, 67.18899],
[-52.9804, 68.35759],
[-51.47536, 68.72958],
[-51.08041, 69.14781],
[-50.87122, 69.9291],
[-52.013585, 69.574925],
[-52.55792, 69.42616],
[-53.45629, 69.283625],
[-54.68336, 69.61003],
[-54.75001, 70.28932],
[-54.35884, 70.821315],
[-53.431315, 70.835755],
[-51.39014, 70.56978],
```

```
          [-53.10937, 71.20485],
          [-54.00422, 71.54719],
          [-55, 71.406537],
          [-55.83468, 71.65444],
          [-54.71819, 72.58625],
          [-55.32634, 72.95861],
          [-56.12003, 73.64977],
          [-57.32363, 74.71026],
          [-58.59679, 75.09861],
          [-58.58516, 75.51727],
          [-61.26861, 76.10238],
          [-63.39165, 76.1752],
          [-66.06427, 76.13486],
          [-68.50438, 76.06141],
          [-69.66485, 76.37975],
          [-71.40257, 77.00857],
          [-68.77671, 77.32312],
          [-66.76397, 77.37595],
          [-71.04293, 77.63595],
          [-73.297, 78.04419],
          [-73.15938, 78.43271],
          [-69.37345, 78.91388],
          [-65.7107, 79.39436],
          [-65.3239, 79.75814],
          [-68.02298, 80.11721],
          [-67.15129, 80.51582],
          [-63.68925, 81.21396],
          [-62.23444, 81.3211],
          [-62.65116, 81.77042],
          [-60.28249, 82.03363],
          [-57.20744, 82.19074],
          [-54.13442, 82.19962],
          [-53.04328, 81.88833],
          [-50.39061, 82.43883],
          [-48.00386, 82.06481],
          [-46.59984, 81.985945],
          [-44.523, 81.6607],
          [-46.9007, 82.19979],
          [-46.76379, 82.62796],
        ],
      ],
    },
    id: "GRL",
  },
  {
    type: "Feature",
    properties: {
      name: "Guatemala",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-90.095555, 13.735338],
          [-90.608624, 13.909771],
          [-91.23241, 13.927832],
          [-91.689747, 14.126218],
          [-92.22775, 14.538829],
          [-92.20323, 14.830103],
          [-92.087216, 15.064585],
          [-92.229249, 15.251447],
          [-91.74796, 16.066565],
          [-90.464473, 16.069562],
          [-90.438867, 16.41011],
          [-90.600847, 16.470778],
          [-90.711822, 16.687483],
```

```
          [-91.08167, 16.918477],
          [-91.453921, 17.252177],
          [-91.002269, 17.254658],
          [-91.00152, 17.817595],
          [-90.067934, 17.819326],
          [-89.14308, 17.808319],
          [-89.150806, 17.015577],
          [-89.229122, 15.886938],
          [-88.930613, 15.887273],
          [-88.604586, 15.70638],
          [-88.518364, 15.855389],
          [-88.225023, 15.727722],
          [-88.68068, 15.346247],
          [-89.154811, 15.066419],
          [-89.22522, 14.874286],
          [-89.145535, 14.678019],
          [-89.353326, 14.424133],
          [-89.587343, 14.362586],
          [-89.534219, 14.244816],
          [-89.721934, 14.134228],
          [-90.064678, 13.88197],
          [-90.095555, 13.735338],
        ],
      ],
    },
    id: "GTM",
  },
  {
    type: "Feature",
    properties: {
      name: "Guyana",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-59.758285, 8.367035],
          [-59.101684, 7.999202],
          [-58.482962, 7.347691],
          [-58.454876, 6.832787],
          [-58.078103, 6.809094],
          [-57.542219, 6.321268],
          [-57.147436, 5.97315],
          [-57.307246, 5.073567],
          [-57.914289, 4.812626],
          [-57.86021, 4.576801],
          [-58.044694, 4.060864],
          [-57.601569, 3.334655],
          [-57.281433, 3.333492],
          [-57.150098, 2.768927],
          [-56.539386, 1.899523],
          [-56.782704, 1.863711],
          [-57.335823, 1.948538],
          [-57.660971, 1.682585],
          [-58.11345, 1.507195],
          [-58.429477, 1.463942],
          [-58.540013, 1.268088],
          [-59.030862, 1.317698],
          [-59.646044, 1.786894],
          [-59.718546, 2.24963],
          [-59.974525, 2.755233],
          [-59.815413, 3.606499],
          [-59.53804, 3.958803],
          [-59.767406, 4.423503],
          [-60.111002, 4.574967],
          [-59.980959, 5.014061],
```

```
          [-60.213683, 5.244486],
          [-60.733574, 5.200277],
          [-61.410303, 5.959068],
          [-61.139415, 6.234297],
          [-61.159336, 6.696077],
          [-60.543999, 6.856584],
          [-60.295668, 7.043911],
          [-60.637973, 7.415],
          [-60.550588, 7.779603],
          [-59.758285, 8.367035],
        ],
      ],
    },
    id: "GUY",
  },
  {
    type: "Feature",
    properties: {
      name: "Honduras",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-87.316654, 12.984686],
          [-87.489409, 13.297535],
          [-87.793111, 13.38448],
          [-87.723503, 13.78505],
          [-87.859515, 13.893312],
          [-88.065343, 13.964626],
          [-88.503998, 13.845486],
          [-88.541231, 13.980155],
          [-88.843073, 14.140507],
          [-89.058512, 14.340029],
          [-89.353326, 14.424133],
          [-89.145535, 14.678019],
          [-89.22522, 14.874286],
          [-89.154811, 15.066419],
          [-88.68068, 15.346247],
          [-88.225023, 15.727722],
          [-88.121153, 15.688655],
          [-87.901813, 15.864458],
          [-87.61568, 15.878799],
          [-87.522921, 15.797279],
          [-87.367762, 15.84694],
          [-86.903191, 15.756713],
          [-86.440946, 15.782835],
          [-86.119234, 15.893449],
          [-86.001954, 16.005406],
          [-85.683317, 15.953652],
          [-85.444004, 15.885749],
          [-85.182444, 15.909158],
          [-84.983722, 15.995923],
          [-84.52698, 15.857224],
          [-84.368256, 15.835158],
          [-84.063055, 15.648244],
          [-83.773977, 15.424072],
          [-83.410381, 15.270903],
          [-83.147219, 14.995829],
          [-83.489989, 15.016267],
          [-83.628585, 14.880074],
          [-83.975721, 14.749436],
          [-84.228342, 14.748764],
          [-84.449336, 14.621614],
          [-84.649582, 14.666805],
          [-84.820037, 14.819587],
```

```
          [-84.924501, 14.790493],
          [-85.052787, 14.551541],
          [-85.148751, 14.560197],
          [-85.165365, 14.35437],
          [-85.514413, 14.079012],
          [-85.698665, 13.960078],
          [-85.801295, 13.836055],
          [-86.096264, 14.038187],
          [-86.312142, 13.771356],
          [-86.520708, 13.778487],
          [-86.755087, 13.754845],
          [-86.733822, 13.263093],
          [-86.880557, 13.254204],
          [-87.005769, 13.025794],
          [-87.316654, 12.984686],
        ],
      ],
    },
    id: "HND",
  },
  {
    type: "Feature",
    properties: {
      name: "Croatia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [18.829838, 45.908878],
          [19.072769, 45.521511],
          [19.390476, 45.236516],
          [19.005486, 44.860234],
          [18.553214, 45.08159],
          [17.861783, 45.06774],
          [17.002146, 45.233777],
          [16.534939, 45.211608],
          [16.318157, 45.004127],
          [15.959367, 45.233777],
          [15.750026, 44.818712],
          [16.23966, 44.351143],
          [16.456443, 44.04124],
          [16.916156, 43.667722],
          [17.297373, 43.446341],
          [17.674922, 43.028563],
          [18.56, 42.65],
          [18.450016, 42.479991],
          [17.50997, 42.849995],
          [16.930006, 43.209998],
          [16.015385, 43.507215],
          [15.174454, 44.243191],
          [15.37625, 44.317915],
          [14.920309, 44.738484],
          [14.901602, 45.07606],
          [14.258748, 45.233777],
          [13.952255, 44.802124],
          [13.656976, 45.136935],
          [13.679403, 45.484149],
          [13.71506, 45.500324],
          [14.411968, 45.466166],
          [14.595109, 45.634941],
          [14.935244, 45.471695],
          [15.327675, 45.452316],
          [15.323954, 45.731783],
          [15.67153, 45.834154],
          [15.768733, 46.238108],
```

```
          [16.564808, 46.503751],
          [16.882515, 46.380632],
          [17.630066, 45.951769],
          [18.456062, 45.759481],
          [18.829838, 45.908878],
        ],
      ],
    },
    id: "HRV",
  },
  {
    type: "Feature",
    properties: {
      name: "Haiti",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-73.189791, 19.915684],
          [-72.579673, 19.871501],
          [-71.712361, 19.714456],
          [-71.624873, 19.169838],
          [-71.701303, 18.785417],
          [-71.945112, 18.6169],
          [-71.687738, 18.31666],
          [-71.708305, 18.044997],
          [-72.372476, 18.214961],
          [-72.844411, 18.145611],
          [-73.454555, 18.217906],
          [-73.922433, 18.030993],
          [-74.458034, 18.34255],
          [-74.369925, 18.664908],
          [-73.449542, 18.526053],
          [-72.694937, 18.445799],
          [-72.334882, 18.668422],
          [-72.79165, 19.101625],
          [-72.784105, 19.483591],
          [-73.415022, 19.639551],
          [-73.189791, 19.915684],
        ],
      ],
    },
    id: "HTI",
  },
  {
    type: "Feature",
    properties: {
      name: "Hungary",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [16.202298, 46.852386],
          [16.534268, 47.496171],
          [16.340584, 47.712902],
          [16.903754, 47.714866],
          [16.979667, 48.123497],
          [17.488473, 47.867466],
          [17.857133, 47.758429],
          [18.696513, 47.880954],
          [18.777025, 48.081768],
          [19.174365, 48.111379],
          [19.661364, 48.266615],
          [19.769471, 48.202691],
```

```
          [20.239054, 48.327567],
          [20.473562, 48.56285],
          [20.801294, 48.623854],
          [21.872236, 48.319971],
          [22.085608, 48.422264],
          [22.64082, 48.15024],
          [22.710531, 47.882194],
          [22.099768, 47.672439],
          [21.626515, 46.994238],
          [21.021952, 46.316088],
          [20.220192, 46.127469],
          [19.596045, 46.17173],
          [18.829838, 45.908878],
          [18.456062, 45.759481],
          [17.630066, 45.951769],
          [16.882515, 46.380632],
          [16.564808, 46.503751],
          [16.370505, 46.841327],
          [16.202298, 46.852386],
        ],
      ],
    },
    id: "HUN",
  },
  {
    type: "Feature",
    properties: {
      name: "Indonesia",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [120.715609, -10.239581],
            [120.295014, -10.25865],
            [118.967808, -9.557969],
            [119.90031, -9.36134],
            [120.425756, -9.665921],
            [120.775502, -9.969675],
            [120.715609, -10.239581],
          ],
        ],
        [
          [
            [124.43595, -10.140001],
            [123.579982, -10.359987],
            [123.459989, -10.239995],
            [123.550009, -9.900016],
            [123.980009, -9.290027],
            [124.968682, -8.89279],
            [125.07002, -9.089987],
            [125.08852, -9.393173],
            [124.43595, -10.140001],
          ],
        ],
        [
          [
            [117.900018, -8.095681],
            [118.260616, -8.362383],
            [118.87846, -8.280683],
            [119.126507, -8.705825],
            [117.970402, -8.906639],
            [117.277731, -9.040895],
            [116.740141, -9.032937],
            [117.083737, -8.457158],
```

```
        [117.632024, -8.449303],
        [117.900018, -8.095681],
      ],
    ],
    [
      [
        [122.903537, -8.094234],
        [122.756983, -8.649808],
        [121.254491, -8.933666],
        [119.924391, -8.810418],
        [119.920929, -8.444859],
        [120.715092, -8.236965],
        [121.341669, -8.53674],
        [122.007365, -8.46062],
        [122.903537, -8.094234],
      ],
    ],
    [
      [
        [108.623479, -6.777674],
        [110.539227, -6.877358],
        [110.759576, -6.465186],
        [112.614811, -6.946036],
        [112.978768, -7.594213],
        [114.478935, -7.776528],
        [115.705527, -8.370807],
        [114.564511, -8.751817],
        [113.464734, -8.348947],
        [112.559672, -8.376181],
        [111.522061, -8.302129],
        [110.58615, -8.122605],
        [109.427667, -7.740664],
        [108.693655, -7.6416],
        [108.277763, -7.766657],
        [106.454102, -7.3549],
        [106.280624, -6.9249],
        [105.365486, -6.851416],
        [106.051646, -5.895919],
        [107.265009, -5.954985],
        [108.072091, -6.345762],
        [108.486846, -6.421985],
        [108.623479, -6.777674],
      ],
    ],
    [
      [
        [134.724624, -6.214401],
        [134.210134, -6.895238],
        [134.112776, -6.142467],
        [134.290336, -5.783058],
        [134.499625, -5.445042],
        [134.727002, -5.737582],
        [134.724624, -6.214401],
      ],
    ],
    [
      [
        [127.249215, -3.459065],
        [126.874923, -3.790983],
        [126.183802, -3.607376],
        [125.989034, -3.177273],
        [127.000651, -3.129318],
        [127.249215, -3.459065],
      ],
    ],
    [
```

```
      [
        [130.471344, -3.093764],
        [130.834836, -3.858472],
        [129.990547, -3.446301],
        [129.155249, -3.362637],
        [128.590684, -3.428679],
        [127.898891, -3.393436],
        [128.135879, -2.84365],
        [129.370998, -2.802154],
        [130.471344, -3.093764],
      ],
    ],
    [
      [
        [134.143368, -1.151867],
        [134.422627, -2.769185],
        [135.457603, -3.367753],
        [136.293314, -2.307042],
        [137.440738, -1.703513],
        [138.329727, -1.702686],
        [139.184921, -2.051296],
        [139.926684, -2.409052],
        [141.00021, -2.600151],
        [141.017057, -5.859022],
        [141.033852, -9.117893],
        [140.143415, -8.297168],
        [139.127767, -8.096043],
        [138.881477, -8.380935],
        [137.614474, -8.411683],
        [138.039099, -7.597882],
        [138.668621, -7.320225],
        [138.407914, -6.232849],
        [137.92784, -5.393366],
        [135.98925, -4.546544],
        [135.164598, -4.462931],
        [133.66288, -3.538853],
        [133.367705, -4.024819],
        [132.983956, -4.112979],
        [132.756941, -3.746283],
        [132.753789, -3.311787],
        [131.989804, -2.820551],
        [133.066845, -2.460418],
        [133.780031, -2.479848],
        [133.696212, -2.214542],
        [132.232373, -2.212526],
        [131.836222, -1.617162],
        [130.94284, -1.432522],
        [130.519558, -0.93772],
        [131.867538, -0.695461],
        [132.380116, -0.369538],
        [133.985548, -0.78021],
        [134.143368, -1.151867],
      ],
    ],
    [
      [
        [125.240501, 1.419836],
        [124.437035, 0.427881],
        [123.685505, 0.235593],
        [122.723083, 0.431137],
        [121.056725, 0.381217],
        [120.183083, 0.237247],
        [120.04087, -0.519658],
        [120.935905, -1.408906],
        [121.475821, -0.955962],
        [123.340565, -0.615673],
```

```
      [123.258399, -1.076213],
      [122.822715, -0.930951],
      [122.38853, -1.516858],
      [121.508274, -1.904483],
      [122.454572, -3.186058],
      [122.271896, -3.5295],
      [123.170963, -4.683693],
      [123.162333, -5.340604],
      [122.628515, -5.634591],
      [122.236394, -5.282933],
      [122.719569, -4.464172],
      [121.738234, -4.851331],
      [121.489463, -4.574553],
      [121.619171, -4.188478],
      [120.898182, -3.602105],
      [120.972389, -2.627643],
      [120.305453, -2.931604],
      [120.390047, -4.097579],
      [120.430717, -5.528241],
      [119.796543, -5.6734],
      [119.366906, -5.379878],
      [119.653606, -4.459417],
      [119.498835, -3.494412],
      [119.078344, -3.487022],
      [118.767769, -2.801999],
      [119.180974, -2.147104],
      [119.323394, -1.353147],
      [119.825999, 0.154254],
      [120.035702, 0.566477],
      [120.885779, 1.309223],
      [121.666817, 1.013944],
      [122.927567, 0.875192],
      [124.077522, 0.917102],
      [125.065989, 1.643259],
      [125.240501, 1.419836],
    ],
  ],
  [
    [
      [128.688249, 1.132386],
      [128.635952, 0.258486],
      [128.12017, 0.356413],
      [127.968034, -0.252077],
      [128.379999, -0.780004],
      [128.100016, -0.899996],
      [127.696475, -0.266598],
      [127.39949, 1.011722],
      [127.600512, 1.810691],
      [127.932378, 2.174596],
      [128.004156, 1.628531],
      [128.594559, 1.540811],
      [128.688249, 1.132386],
    ],
  ],
  [
    [
      [117.875627, 1.827641],
      [118.996747, 0.902219],
      [117.811858, 0.784242],
      [117.478339, 0.102475],
      [117.521644, -0.803723],
      [116.560048, -1.487661],
      [116.533797, -2.483517],
      [116.148084, -4.012726],
      [116.000858, -3.657037],
      [114.864803, -4.106984],
```

```
        [114.468652, -3.495704],
        [113.755672, -3.43917],
        [113.256994, -3.118776],
        [112.068126, -3.478392],
        [111.703291, -2.994442],
        [111.04824, -3.049426],
        [110.223846, -2.934032],
        [110.070936, -1.592874],
        [109.571948, -1.314907],
        [109.091874, -0.459507],
        [108.952658, 0.415375],
        [109.069136, 1.341934],
        [109.66326, 2.006467],
        [109.830227, 1.338136],
        [110.514061, 0.773131],
        [111.159138, 0.976478],
        [111.797548, 0.904441],
        [112.380252, 1.410121],
        [112.859809, 1.49779],
        [113.80585, 1.217549],
        [114.621355, 1.430688],
        [115.134037, 2.821482],
        [115.519078, 3.169238],
        [115.865517, 4.306559],
        [117.015214, 4.306094],
        [117.882035, 4.137551],
        [117.313232, 3.234428],
        [118.04833, 2.28769],
        [117.875627, 1.827641],
      ],
    ],
    [
      [
        [105.817655, -5.852356],
        [104.710384, -5.873285],
        [103.868213, -5.037315],
        [102.584261, -4.220259],
        [102.156173, -3.614146],
        [101.399113, -2.799777],
        [100.902503, -2.050262],
        [100.141981, -0.650348],
        [99.26374, 0.183142],
        [98.970011, 1.042882],
        [98.601351, 1.823507],
        [97.699598, 2.453184],
        [97.176942, 3.308791],
        [96.424017, 3.86886],
        [95.380876, 4.970782],
        [95.293026, 5.479821],
        [95.936863, 5.439513],
        [97.484882, 5.246321],
        [98.369169, 4.26837],
        [99.142559, 3.59035],
        [99.693998, 3.174329],
        [100.641434, 2.099381],
        [101.658012, 2.083697],
        [102.498271, 1.3987],
        [103.07684, 0.561361],
        [103.838396, 0.104542],
        [103.437645, -0.711946],
        [104.010789, -1.059212],
        [104.369991, -1.084843],
        [104.53949, -1.782372],
        [104.887893, -2.340425],
        [105.622111, -2.428844],
        [106.108593, -3.061777],
```

```
            [105.857446, -4.305525],
            [105.817655, -5.852356],
          ],
        ],
      ],
    },
    id: "IDN",
  },
  {
    type: "Feature",
    properties: {
      name: "India",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [77.837451, 35.49401],
          [78.912269, 34.321936],
          [78.811086, 33.506198],
          [79.208892, 32.994395],
          [79.176129, 32.48378],
          [78.458446, 32.618164],
          [78.738894, 31.515906],
          [79.721367, 30.882715],
          [81.111256, 30.183481],
          [80.476721, 29.729865],
          [80.088425, 28.79447],
          [81.057203, 28.416095],
          [81.999987, 27.925479],
          [83.304249, 27.364506],
          [84.675018, 27.234901],
          [85.251779, 26.726198],
          [86.024393, 26.630985],
          [87.227472, 26.397898],
          [88.060238, 26.414615],
          [88.174804, 26.810405],
          [88.043133, 27.445819],
          [88.120441, 27.876542],
          [88.730326, 28.086865],
          [88.814248, 27.299316],
          [88.835643, 27.098966],
          [89.744528, 26.719403],
          [90.373275, 26.875724],
          [91.217513, 26.808648],
          [92.033484, 26.83831],
          [92.103712, 27.452614],
          [91.696657, 27.771742],
          [92.503119, 27.896876],
          [93.413348, 28.640629],
          [94.56599, 29.277438],
          [95.404802, 29.031717],
          [96.117679, 29.452802],
          [96.586591, 28.83098],
          [96.248833, 28.411031],
          [97.327114, 28.261583],
          [97.402561, 27.882536],
          [97.051989, 27.699059],
          [97.133999, 27.083774],
          [96.419366, 27.264589],
          [95.124768, 26.573572],
          [95.155153, 26.001307],
          [94.603249, 25.162495],
          [94.552658, 24.675238],
          [94.106742, 23.850741],
          [93.325188, 24.078556],
```

```
            [93.286327, 23.043658],
            [93.060294, 22.703111],
            [93.166128, 22.27846],
            [92.672721, 22.041239],
            [92.146035, 23.627499],
            [91.869928, 23.624346],
            [91.706475, 22.985264],
            [91.158963, 23.503527],
            [91.46773, 24.072639],
            [91.915093, 24.130414],
            [92.376202, 24.976693],
            [91.799596, 25.147432],
            [90.872211, 25.132601],
            [89.920693, 25.26975],
            [89.832481, 25.965082],
            [89.355094, 26.014407],
            [88.563049, 26.446526],
            [88.209789, 25.768066],
            [88.931554, 25.238692],
            [88.306373, 24.866079],
            [88.084422, 24.501657],
            [88.69994, 24.233715],
            [88.52977, 23.631142],
            [88.876312, 22.879146],
            [89.031961, 22.055708],
            [88.888766, 21.690588],
            [88.208497, 21.703172],
            [86.975704, 21.495562],
            [87.033169, 20.743308],
            [86.499351, 20.151638],
            [85.060266, 19.478579],
            [83.941006, 18.30201],
            [83.189217, 17.671221],
            [82.192792, 17.016636],
            [82.191242, 16.556664],
            [81.692719, 16.310219],
            [80.791999, 15.951972],
            [80.324896, 15.899185],
            [80.025069, 15.136415],
            [80.233274, 13.835771],
            [80.286294, 13.006261],
            [79.862547, 12.056215],
            [79.857999, 10.357275],
            [79.340512, 10.308854],
            [78.885345, 9.546136],
            [79.18972, 9.216544],
            [78.277941, 8.933047],
            [77.941165, 8.252959],
            [77.539898, 7.965535],
            [76.592979, 8.899276],
            [76.130061, 10.29963],
            [75.746467, 11.308251],
            [75.396101, 11.781245],
            [74.864816, 12.741936],
            [74.616717, 13.992583],
            [74.443859, 14.617222],
            [73.534199, 15.990652],
            [73.119909, 17.92857],
            [72.820909, 19.208234],
            [72.824475, 20.419503],
            [72.630533, 21.356009],
            [71.175273, 20.757441],
            [70.470459, 20.877331],
            [69.16413, 22.089298],
            [69.644928, 22.450775],
            [69.349597, 22.84318],
```

```
          [68.176645, 23.691965],
          [68.842599, 24.359134],
          [71.04324, 24.356524],
          [70.844699, 25.215102],
          [70.282873, 25.722229],
          [70.168927, 26.491872],
          [69.514393, 26.940966],
          [70.616496, 27.989196],
          [71.777666, 27.91318],
          [72.823752, 28.961592],
          [73.450638, 29.976413],
          [74.42138, 30.979815],
          [74.405929, 31.692639],
          [75.258642, 32.271105],
          [74.451559, 32.7649],
          [74.104294, 33.441473],
          [73.749948, 34.317699],
          [74.240203, 34.748887],
          [75.757061, 34.504923],
          [76.871722, 34.653544],
          [77.837451, 35.49401],
        ],
      ],
    },
    id: "IND",
  },
  {
    type: "Feature",
    properties: {
      name: "Ireland",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-6.197885, 53.867565],
          [-6.032985, 53.153164],
          [-6.788857, 52.260118],
          [-8.561617, 51.669301],
          [-9.977086, 51.820455],
          [-9.166283, 52.864629],
          [-9.688525, 53.881363],
          [-8.327987, 54.664519],
          [-7.572168, 55.131622],
          [-7.366031, 54.595841],
          [-7.572168, 54.059956],
          [-6.95373, 54.073702],
          [-6.197885, 53.867565],
        ],
      ],
    },
    id: "IRL",
  },
  {
    type: "Feature",
    properties: {
      name: "Iran",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [53.921598, 37.198918],
          [54.800304, 37.392421],
          [55.511578, 37.964117],
          [56.180375, 37.935127],
```

```
            [56.619366, 38.121394],
            [57.330434, 38.029229],
            [58.436154, 37.522309],
            [59.234762, 37.412988],
            [60.377638, 36.527383],
            [61.123071, 36.491597],
            [61.210817, 35.650072],
            [60.803193, 34.404102],
            [60.52843, 33.676446],
            [60.9637, 33.528832],
            [60.536078, 32.981269],
            [60.863655, 32.18292],
            [60.941945, 31.548075],
            [61.699314, 31.379506],
            [61.781222, 30.73585],
            [60.874248, 29.829239],
            [61.369309, 29.303276],
            [61.771868, 28.699334],
            [62.72783, 28.259645],
            [62.755426, 27.378923],
            [63.233898, 27.217047],
            [63.316632, 26.756532],
            [61.874187, 26.239975],
            [61.497363, 25.078237],
            [59.616134, 25.380157],
            [58.525761, 25.609962],
            [57.397251, 25.739902],
            [56.970766, 26.966106],
            [56.492139, 27.143305],
            [55.72371, 26.964633],
            [54.71509, 26.480658],
            [53.493097, 26.812369],
            [52.483598, 27.580849],
            [51.520763, 27.86569],
            [50.852948, 28.814521],
            [50.115009, 30.147773],
            [49.57685, 29.985715],
            [48.941333, 30.31709],
            [48.567971, 29.926778],
            [48.014568, 30.452457],
            [48.004698, 30.985137],
            [47.685286, 30.984853],
            [47.849204, 31.709176],
            [47.334661, 32.469155],
            [46.109362, 33.017287],
            [45.416691, 33.967798],
            [45.64846, 34.748138],
            [46.151788, 35.093259],
            [46.07634, 35.677383],
            [45.420618, 35.977546],
            [44.77267, 37.17045],
            [44.225756, 37.971584],
            [44.421403, 38.281281],
            [44.109225, 39.428136],
            [44.79399, 39.713003],
            [44.952688, 39.335765],
            [45.457722, 38.874139],
            [46.143623, 38.741201],
            [46.50572, 38.770605],
            [47.685079, 39.508364],
            [48.060095, 39.582235],
            [48.355529, 39.288765],
            [48.010744, 38.794015],
            [48.634375, 38.270378],
            [48.883249, 38.320245],
            [49.199612, 37.582874],
```

```
          [50.147771, 37.374567],
          [50.842354, 36.872814],
          [52.264025, 36.700422],
          [53.82579, 36.965031],
          [53.921598, 37.198918],
        ],
      ],
    },
    id: "IRN",
  },
  {
    type: "Feature",
    properties: {
      name: "Iraq",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [45.420618, 35.977546],
          [46.07634, 35.677383],
          [46.151788, 35.093259],
          [45.64846, 34.748138],
          [45.416691, 33.967798],
          [46.109362, 33.017287],
          [47.334661, 32.469155],
          [47.849204, 31.709176],
          [47.685286, 30.984853],
          [48.004698, 30.985137],
          [48.014568, 30.452457],
          [48.567971, 29.926778],
          [47.974519, 29.975819],
          [47.302622, 30.05907],
          [46.568713, 29.099025],
          [44.709499, 29.178891],
          [41.889981, 31.190009],
          [40.399994, 31.889992],
          [39.195468, 32.161009],
          [38.792341, 33.378686],
          [41.006159, 34.419372],
          [41.383965, 35.628317],
          [41.289707, 36.358815],
          [41.837064, 36.605854],
          [42.349591, 37.229873],
          [42.779126, 37.385264],
          [43.942259, 37.256228],
          [44.293452, 37.001514],
          [44.772699, 37.170445],
          [45.420618, 35.977546],
        ],
      ],
    },
    id: "IRQ",
  },
  {
    type: "Feature",
    properties: {
      name: "Iceland",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-14.508695, 66.455892],
          [-14.739637, 65.808748],
          [-13.609732, 65.126671],
```

```
          [-14.909834, 64.364082],
          [-17.794438, 63.678749],
          [-18.656246, 63.496383],
          [-19.972755, 63.643635],
          [-22.762972, 63.960179],
          [-21.778484, 64.402116],
          [-23.955044, 64.89113],
          [-22.184403, 65.084968],
          [-22.227423, 65.378594],
          [-24.326184, 65.611189],
          [-23.650515, 66.262519],
          [-22.134922, 66.410469],
          [-20.576284, 65.732112],
          [-19.056842, 66.276601],
          [-17.798624, 65.993853],
          [-16.167819, 66.526792],
          [-14.508695, 66.455892],
        ],
      ],
    },
    id: "ISL",
  },
  {
    type: "Feature",
    properties: {
      name: "Israel",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [35.719918, 32.709192],
          [35.545665, 32.393992],
          [35.18393, 32.532511],
          [34.974641, 31.866582],
          [35.225892, 31.754341],
          [34.970507, 31.616778],
          [34.927408, 31.353435],
          [35.397561, 31.489086],
          [35.420918, 31.100066],
          [34.922603, 29.501326],
          [34.265433, 31.219361],
          [34.556372, 31.548824],
          [34.488107, 31.605539],
          [34.752587, 32.072926],
          [34.955417, 32.827376],
          [35.098457, 33.080539],
          [35.126053, 33.0909],
          [35.460709, 33.08904],
          [35.552797, 33.264275],
          [35.821101, 33.277426],
          [35.836397, 32.868123],
          [35.700798, 32.716014],
          [35.719918, 32.709192],
        ],
      ],
    },
    id: "ISR",
  },
  {
    type: "Feature",
    properties: {
      name: "Italy",
    },
    geometry: {
      type: "MultiPolygon",
```

```
        coordinates: [
          [
            [
              [15.520376, 38.231155],
              [15.160243, 37.444046],
              [15.309898, 37.134219],
              [15.099988, 36.619987],
              [14.335229, 36.996631],
              [13.826733, 37.104531],
              [12.431004, 37.61295],
              [12.570944, 38.126381],
              [13.741156, 38.034966],
              [14.761249, 38.143874],
              [15.520376, 38.231155],
            ],
          ],
          [
            [
              [9.210012, 41.209991],
              [9.809975, 40.500009],
              [9.669519, 39.177376],
              [9.214818, 39.240473],
              [8.806936, 38.906618],
              [8.428302, 39.171847],
              [8.388253, 40.378311],
              [8.159998, 40.950007],
              [8.709991, 40.899984],
              [9.210012, 41.209991],
            ],
          ],
          [
            [
              [12.376485, 46.767559],
              [13.806475, 46.509306],
              [13.69811, 46.016778],
              [13.93763, 45.591016],
              [13.141606, 45.736692],
              [12.328581, 45.381778],
              [12.383875, 44.885374],
              [12.261453, 44.600482],
              [12.589237, 44.091366],
              [13.526906, 43.587727],
              [14.029821, 42.761008],
              [15.14257, 41.95514],
              [15.926191, 41.961315],
              [16.169897, 41.740295],
              [15.889346, 41.541082],
              [16.785002, 41.179606],
              [17.519169, 40.877143],
              [18.376687, 40.355625],
              [18.480247, 40.168866],
              [18.293385, 39.810774],
              [17.73838, 40.277671],
              [16.869596, 40.442235],
              [16.448743, 39.795401],
              [17.17149, 39.4247],
              [17.052841, 38.902871],
              [16.635088, 38.843572],
              [16.100961, 37.985899],
              [15.684087, 37.908849],
              [15.687963, 38.214593],
              [15.891981, 38.750942],
              [16.109332, 38.964547],
              [15.718814, 39.544072],
              [15.413613, 40.048357],
              [14.998496, 40.172949],
```

```
            [14.703268, 40.60455],
            [14.060672, 40.786348],
            [13.627985, 41.188287],
            [12.888082, 41.25309],
            [12.106683, 41.704535],
            [11.191906, 42.355425],
            [10.511948, 42.931463],
            [10.200029, 43.920007],
            [9.702488, 44.036279],
            [8.888946, 44.366336],
            [8.428561, 44.231228],
            [7.850767, 43.767148],
            [7.435185, 43.693845],
            [7.549596, 44.127901],
            [7.007562, 44.254767],
            [6.749955, 45.028518],
            [7.096652, 45.333099],
            [6.802355, 45.70858],
            [6.843593, 45.991147],
            [7.273851, 45.776948],
            [7.755992, 45.82449],
            [8.31663, 46.163642],
            [8.489952, 46.005151],
            [8.966306, 46.036932],
            [9.182882, 46.440215],
            [9.922837, 46.314899],
            [10.363378, 46.483571],
            [10.442701, 46.893546],
            [11.048556, 46.751359],
            [11.164828, 46.941579],
            [12.153088, 47.115393],
            [12.376485, 46.767559],
          ],
        ],
      ],
    },
    id: "ITA",
  },
  {
    type: "Feature",
    properties: {
      name: "Jamaica",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-77.569601, 18.490525],
          [-76.896619, 18.400867],
          [-76.365359, 18.160701],
          [-76.199659, 17.886867],
          [-76.902561, 17.868238],
          [-77.206341, 17.701116],
          [-77.766023, 17.861597],
          [-78.337719, 18.225968],
          [-78.217727, 18.454533],
          [-77.797365, 18.524218],
          [-77.569601, 18.490525],
        ],
      ],
    },
    id: "JAM",
  },
  {
    type: "Feature",
    properties: {
```

```
        name: "Jordan",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [35.545665, 32.393992],
            [35.719918, 32.709192],
            [36.834062, 32.312938],
            [38.792341, 33.378686],
            [39.195468, 32.161009],
            [39.004886, 32.010217],
            [37.002166, 31.508413],
            [37.998849, 30.5085],
            [37.66812, 30.338665],
            [37.503582, 30.003776],
            [36.740528, 29.865283],
            [36.501214, 29.505254],
            [36.068941, 29.197495],
            [34.956037, 29.356555],
            [34.922603, 29.501326],
            [35.420918, 31.100066],
            [35.397561, 31.489086],
            [35.545252, 31.782505],
            [35.545665, 32.393992],
          ],
        ],
      },
      id: "JOR",
    },
    {
      type: "Feature",
      properties: {
        name: "Japan",
      },
      geometry: {
        type: "MultiPolygon",
        coordinates: [
          [
            [
              [134.638428, 34.149234],
              [134.766379, 33.806335],
              [134.203416, 33.201178],
              [133.79295, 33.521985],
              [133.280268, 33.28957],
              [133.014858, 32.704567],
              [132.363115, 32.989382],
              [132.371176, 33.463642],
              [132.924373, 34.060299],
              [133.492968, 33.944621],
              [133.904106, 34.364931],
              [134.638428, 34.149234],
            ],
          ],
          [
            [
              [140.976388, 37.142074],
              [140.59977, 36.343983],
              [140.774074, 35.842877],
              [140.253279, 35.138114],
              [138.975528, 34.6676],
              [137.217599, 34.606286],
              [135.792983, 33.464805],
              [135.120983, 33.849071],
              [135.079435, 34.596545],
              [133.340316, 34.375938],
```

```
            [132.156771, 33.904933],
            [130.986145, 33.885761],
            [132.000036, 33.149992],
            [131.33279, 31.450355],
            [130.686318, 31.029579],
            [130.20242, 31.418238],
            [130.447676, 32.319475],
            [129.814692, 32.61031],
            [129.408463, 33.296056],
            [130.353935, 33.604151],
            [130.878451, 34.232743],
            [131.884229, 34.749714],
            [132.617673, 35.433393],
            [134.608301, 35.731618],
            [135.677538, 35.527134],
            [136.723831, 37.304984],
            [137.390612, 36.827391],
            [138.857602, 37.827485],
            [139.426405, 38.215962],
            [140.05479, 39.438807],
            [139.883379, 40.563312],
            [140.305783, 41.195005],
            [141.368973, 41.37856],
            [141.914263, 39.991616],
            [141.884601, 39.180865],
            [140.959489, 38.174001],
            [140.976388, 37.142074],
          ],
        ],
        [
          [
            [143.910162, 44.1741],
            [144.613427, 43.960883],
            [145.320825, 44.384733],
            [145.543137, 43.262088],
            [144.059662, 42.988358],
            [143.18385, 41.995215],
            [141.611491, 42.678791],
            [141.067286, 41.584594],
            [139.955106, 41.569556],
            [139.817544, 42.563759],
            [140.312087, 43.333273],
            [141.380549, 43.388825],
            [141.671952, 44.772125],
            [141.967645, 45.551483],
            [143.14287, 44.510358],
            [143.910162, 44.1741],
          ],
        ],
      ],
    },
    id: "JPN",
  },
  {
    type: "Feature",
    properties: {
      name: "Kazakhstan",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [70.962315, 42.266154],
          [70.388965, 42.081308],
          [69.070027, 41.384244],
          [68.632483, 40.668681],
```

```
            [68.259896, 40.662325],
            [67.985856, 41.135991],
            [66.714047, 41.168444],
            [66.510649, 41.987644],
            [66.023392, 41.994646],
            [66.098012, 42.99766],
            [64.900824, 43.728081],
            [63.185787, 43.650075],
            [62.0133, 43.504477],
            [61.05832, 44.405817],
            [60.239972, 44.784037],
            [58.689989, 45.500014],
            [58.503127, 45.586804],
            [55.928917, 44.995858],
            [55.968191, 41.308642],
            [55.455251, 41.259859],
            [54.755345, 42.043971],
            [54.079418, 42.324109],
            [52.944293, 42.116034],
            [52.50246, 41.783316],
            [52.446339, 42.027151],
            [52.692112, 42.443895],
            [52.501426, 42.792298],
            [51.342427, 43.132975],
            [50.891292, 44.031034],
            [50.339129, 44.284016],
            [50.305643, 44.609836],
            [51.278503, 44.514854],
            [51.316899, 45.245998],
            [52.16739, 45.408391],
            [53.040876, 45.259047],
            [53.220866, 46.234646],
            [53.042737, 46.853006],
            [52.042023, 46.804637],
            [51.191945, 47.048705],
            [50.034083, 46.60899],
            [49.10116, 46.39933],
            [48.593241, 46.561034],
            [48.694734, 47.075628],
            [48.057253, 47.743753],
            [47.315231, 47.715847],
            [46.466446, 48.394152],
            [47.043672, 49.152039],
            [46.751596, 49.356006],
            [47.54948, 50.454698],
            [48.577841, 49.87476],
            [48.702382, 50.605128],
            [50.766648, 51.692762],
            [52.328724, 51.718652],
            [54.532878, 51.02624],
            [55.716941, 50.621717],
            [56.777961, 51.043551],
            [58.363291, 51.063653],
            [59.642282, 50.545442],
            [59.932807, 50.842194],
            [61.337424, 50.79907],
            [61.588003, 51.272659],
            [59.967534, 51.96042],
            [60.927269, 52.447548],
            [60.739993, 52.719986],
            [61.699986, 52.979996],
            [60.978066, 53.664993],
            [61.436591, 54.006265],
            [65.178534, 54.354228],
            [65.666876, 54.601267],
            [68.1691, 54.970392],
```

```
          [69.068167, 55.38525],
          [70.865267, 55.169734],
          [71.180131, 54.133285],
          [72.22415, 54.376655],
          [73.508516, 54.035617],
          [73.425679, 53.48981],
          [74.384845, 53.546861],
          [76.8911, 54.490524],
          [76.525179, 54.177003],
          [77.800916, 53.404415],
          [80.03556, 50.864751],
          [80.568447, 51.388336],
          [81.945986, 50.812196],
          [83.383004, 51.069183],
          [83.935115, 50.889246],
          [84.416377, 50.3114],
          [85.11556, 50.117303],
          [85.54127, 49.692859],
          [86.829357, 49.826675],
          [87.35997, 49.214981],
          [86.598776, 48.549182],
          [85.768233, 48.455751],
          [85.720484, 47.452969],
          [85.16429, 47.000956],
          [83.180484, 47.330031],
          [82.458926, 45.53965],
          [81.947071, 45.317027],
          [79.966106, 44.917517],
          [80.866206, 43.180362],
          [80.18015, 42.920068],
          [80.25999, 42.349999],
          [79.643645, 42.496683],
          [79.142177, 42.856092],
          [77.658392, 42.960686],
          [76.000354, 42.988022],
          [75.636965, 42.8779],
          [74.212866, 43.298339],
          [73.645304, 43.091272],
          [73.489758, 42.500894],
          [71.844638, 42.845395],
          [71.186281, 42.704293],
          [70.962315, 42.266154],
        ],
      ],
    },
    id: "KAZ",
  },
  {
    type: "Feature",
    properties: {
      name: "Kenya",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [40.993, -0.85829],
          [41.58513, -1.68325],
          [40.88477, -2.08255],
          [40.63785, -2.49979],
          [40.26304, -2.57309],
          [40.12119, -3.27768],
          [39.80006, -3.68116],
          [39.60489, -4.34653],
          [39.20222, -4.67677],
          [37.7669, -3.67712],
```

```
          [37.69869, -3.09699],
          [34.07262, -1.05982],
          [33.903711, -0.95],
          [33.893569, 0.109814],
          [34.18, 0.515],
          [34.6721, 1.17694],
          [35.03599, 1.90584],
          [34.59607, 3.05374],
          [34.47913, 3.5556],
          [34.005, 4.249885],
          [34.620196, 4.847123],
          [35.298007, 5.506],
          [35.817448, 5.338232],
          [35.817448, 4.776966],
          [36.159079, 4.447864],
          [36.855093, 4.447864],
          [38.120915, 3.598605],
          [38.43697, 3.58851],
          [38.67114, 3.61607],
          [38.89251, 3.50074],
          [39.559384, 3.42206],
          [39.85494, 3.83879],
          [40.76848, 4.25702],
          [41.1718, 3.91909],
          [41.855083, 3.918912],
          [40.98105, 2.78452],
          [40.993, -0.85829],
        ],
      ],
    },
    id: "KEN",
  },
  {
    type: "Feature",
    properties: {
      name: "Kyrgyzstan",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [70.962315, 42.266154],
          [71.186281, 42.704293],
          [71.844638, 42.845395],
          [73.489758, 42.500894],
          [73.645304, 43.091272],
          [74.212866, 43.298339],
          [75.636965, 42.8779],
          [76.000354, 42.988022],
          [77.658392, 42.960686],
          [79.142177, 42.856092],
          [79.643645, 42.496683],
          [80.25999, 42.349999],
          [80.11943, 42.123941],
          [78.543661, 41.582243],
          [78.187197, 41.185316],
          [76.904484, 41.066486],
          [76.526368, 40.427946],
          [75.467828, 40.562072],
          [74.776862, 40.366425],
          [73.822244, 39.893973],
          [73.960013, 39.660008],
          [73.675379, 39.431237],
          [71.784694, 39.279463],
          [70.549162, 39.604198],
          [69.464887, 39.526683],
```

```
          [69.55961, 40.103211],
          [70.648019, 39.935754],
          [71.014198, 40.244366],
          [71.774875, 40.145844],
          [73.055417, 40.866033],
          [71.870115, 41.3929],
          [71.157859, 41.143587],
          [70.420022, 41.519998],
          [71.259248, 42.167711],
          [70.962315, 42.266154],
        ],
      ],
    },
    id: "KGZ",
  },
  {
    type: "Feature",
    properties: {
      name: "Cambodia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [103.49728, 10.632555],
          [103.09069, 11.153661],
          [102.584932, 12.186595],
          [102.348099, 13.394247],
          [102.988422, 14.225721],
          [104.281418, 14.416743],
          [105.218777, 14.273212],
          [106.043946, 13.881091],
          [106.496373, 14.570584],
          [107.382727, 14.202441],
          [107.614548, 13.535531],
          [107.491403, 12.337206],
          [105.810524, 11.567615],
          [106.24967, 10.961812],
          [105.199915, 10.88931],
          [104.334335, 10.486544],
          [103.49728, 10.632555],
        ],
      ],
    },
    id: "KHM",
  },
  {
    type: "Feature",
    properties: {
      name: "Kosovo",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [20.76216, 42.05186],
          [20.71731, 41.84711],
          [20.59023, 41.85541],
          [20.52295, 42.21787],
          [20.28374, 42.32025],
          [20.0707, 42.58863],
          [20.25758, 42.81275],
          [20.49679, 42.88469],
          [20.63508, 43.21671],
          [20.81448, 43.27205],
          [20.95651, 43.13094],
```

```
          [21.143395, 43.068685],
          [21.27421, 42.90959],
          [21.43866, 42.86255],
          [21.63302, 42.67717],
          [21.77505, 42.6827],
          [21.66292, 42.43922],
          [21.54332, 42.32025],
          [21.576636, 42.245224],
          [21.3527, 42.2068],
          [20.76216, 42.05186],
        ],
      ],
    },
    id: "OSA",
  },
  {
    type: "Feature",
    properties: {
      name: "Kuwait",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [47.974519, 29.975819],
          [48.183189, 29.534477],
          [48.093943, 29.306299],
          [48.416094, 28.552004],
          [47.708851, 28.526063],
          [47.459822, 29.002519],
          [46.568713, 29.099025],
          [47.302622, 30.05907],
          [47.974519, 29.975819],
        ],
      ],
    },
    id: "KWT",
  },
  {
    type: "Feature",
    properties: {
      name: "Laos",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [105.218777, 14.273212],
          [105.544338, 14.723934],
          [105.589039, 15.570316],
          [104.779321, 16.441865],
          [104.716947, 17.428859],
          [103.956477, 18.240954],
          [103.200192, 18.309632],
          [102.998706, 17.961695],
          [102.413005, 17.932782],
          [102.113592, 18.109102],
          [101.059548, 17.512497],
          [101.035931, 18.408928],
          [101.282015, 19.462585],
          [100.606294, 19.508344],
          [100.548881, 20.109238],
          [100.115988, 20.41785],
          [100.329101, 20.786122],
          [101.180005, 21.436573],
          [101.270026, 21.201652],
```

```
          [101.80312, 21.174367],
          [101.652018, 22.318199],
          [102.170436, 22.464753],
          [102.754896, 21.675137],
          [103.203861, 20.766562],
          [104.435, 20.758733],
          [104.822574, 19.886642],
          [104.183388, 19.624668],
          [103.896532, 19.265181],
          [105.094598, 18.666975],
          [105.925762, 17.485315],
          [106.556008, 16.604284],
          [107.312706, 15.908538],
          [107.564525, 15.202173],
          [107.382727, 14.202441],
          [106.496373, 14.570584],
          [106.043946, 13.881091],
          [105.218777, 14.273212],
        ],
      ],
    },
    id: "LAO",
  },
  {
    type: "Feature",
    properties: {
      name: "Lebanon",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [35.821101, 33.277426],
          [35.552797, 33.264275],
          [35.460709, 33.08904],
          [35.126053, 33.0909],
          [35.482207, 33.90545],
          [35.979592, 34.610058],
          [35.998403, 34.644914],
          [36.448194, 34.593935],
          [36.61175, 34.201789],
          [36.06646, 33.824912],
          [35.821101, 33.277426],
        ],
      ],
    },
    id: "LBN",
  },
  {
    type: "Feature",
    properties: {
      name: "Liberia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-7.712159, 4.364566],
          [-7.974107, 4.355755],
          [-9.004794, 4.832419],
          [-9.91342, 5.593561],
          [-10.765384, 6.140711],
          [-11.438779, 6.785917],
          [-11.199802, 7.105846],
          [-11.146704, 7.396706],
          [-10.695595, 7.939464],
```

```
          [-10.230094, 8.406206],
          [-10.016567, 8.428504],
          [-9.755342, 8.541055],
          [-9.33728, 7.928534],
          [-9.403348, 7.526905],
          [-9.208786, 7.313921],
          [-8.926065, 7.309037],
          [-8.722124, 7.711674],
          [-8.439298, 7.686043],
          [-8.485446, 7.395208],
          [-8.385452, 6.911801],
          [-8.60288, 6.467564],
          [-8.311348, 6.193033],
          [-7.993693, 6.12619],
          [-7.570153, 5.707352],
          [-7.539715, 5.313345],
          [-7.635368, 5.188159],
          [-7.712159, 4.364566],
        ],
      ],
    },
    id: "LBR",
  },
  {
    type: "Feature",
    properties: {
      name: "Libya",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [14.8513, 22.86295],
          [14.143871, 22.491289],
          [13.581425, 23.040506],
          [11.999506, 23.471668],
          [11.560669, 24.097909],
          [10.771364, 24.562532],
          [10.303847, 24.379313],
          [9.948261, 24.936954],
          [9.910693, 25.365455],
          [9.319411, 26.094325],
          [9.716286, 26.512206],
          [9.629056, 27.140953],
          [9.756128, 27.688259],
          [9.683885, 28.144174],
          [9.859998, 28.95999],
          [9.805634, 29.424638],
          [9.48214, 30.307556],
          [9.970017, 30.539325],
          [10.056575, 30.961831],
          [9.950225, 31.37607],
          [10.636901, 31.761421],
          [10.94479, 32.081815],
          [11.432253, 32.368903],
          [11.488787, 33.136996],
          [12.66331, 32.79278],
          [13.08326, 32.87882],
          [13.91868, 32.71196],
          [15.24563, 32.26508],
          [15.71394, 31.37626],
          [16.61162, 31.18218],
          [18.02109, 30.76357],
          [19.08641, 30.26639],
          [19.57404, 30.52582],
          [20.05335, 30.98576],
```

```
          [19.82033, 31.75179],
          [20.13397, 32.2382],
          [20.85452, 32.7068],
          [21.54298, 32.8432],
          [22.89576, 32.63858],
          [23.2368, 32.19149],
          [23.60913, 32.18726],
          [23.9275, 32.01667],
          [24.92114, 31.89936],
          [25.16482, 31.56915],
          [24.80287, 31.08929],
          [24.95762, 30.6616],
          [24.70007, 30.04419],
          [25, 29.238655],
          [25, 25.6825],
          [25, 22],
          [25, 20.00304],
          [23.85, 20],
          [23.83766, 19.58047],
          [19.84926, 21.49509],
          [15.86085, 23.40972],
          [14.8513, 22.86295],
        ],
      ],
    },
    id: "LBY",
  },
  {
    type: "Feature",
    properties: {
      name: "Sri Lanka",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [81.787959, 7.523055],
          [81.637322, 6.481775],
          [81.21802, 6.197141],
          [80.348357, 5.96837],
          [79.872469, 6.763463],
          [79.695167, 8.200843],
          [80.147801, 9.824078],
          [80.838818, 9.268427],
          [81.304319, 8.564206],
          [81.787959, 7.523055],
        ],
      ],
    },
    id: "LKA",
  },
  {
    type: "Feature",
    properties: {
      name: "Lesotho",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [28.978263, -28.955597],
          [29.325166, -29.257387],
          [29.018415, -29.743766],
          [28.8484, -30.070051],
          [28.291069, -30.226217],
          [28.107205, -30.545732],
```

```
          [27.749397, -30.645106],
          [26.999262, -29.875954],
          [27.532511, -29.242711],
          [28.074338, -28.851469],
          [28.5417, -28.647502],
          [28.978263, -28.955597],
        ],
      ],
    },
    id: "LSO",
  },
  {
    type: "Feature",
    properties: {
      name: "Lithuania",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [22.731099, 54.327537],
          [22.651052, 54.582741],
          [22.757764, 54.856574],
          [22.315724, 55.015299],
          [21.268449, 55.190482],
          [21.0558, 56.031076],
          [22.201157, 56.337802],
          [23.878264, 56.273671],
          [24.860684, 56.372528],
          [25.000934, 56.164531],
          [25.533047, 56.100297],
          [26.494331, 55.615107],
          [26.588279, 55.167176],
          [25.768433, 54.846963],
          [25.536354, 54.282423],
          [24.450684, 53.905702],
          [23.484128, 53.912498],
          [23.243987, 54.220567],
          [22.731099, 54.327537],
        ],
      ],
    },
    id: "LTU",
  },
  {
    type: "Feature",
    properties: {
      name: "Luxembourg",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [6.043073, 50.128052],
          [6.242751, 49.902226],
          [6.18632, 49.463803],
          [5.897759, 49.442667],
          [5.674052, 49.529484],
          [5.782417, 50.090328],
          [6.043073, 50.128052],
        ],
      ],
    },
    id: "LUX",
  },
  {
```

```
      type: "Feature",
      properties: {
        name: "Latvia",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [21.0558, 56.031076],
            [21.090424, 56.783873],
            [21.581866, 57.411871],
            [22.524341, 57.753374],
            [23.318453, 57.006236],
            [24.12073, 57.025693],
            [24.312863, 57.793424],
            [25.164594, 57.970157],
            [25.60281, 57.847529],
            [26.463532, 57.476389],
            [27.288185, 57.474528],
            [27.770016, 57.244258],
            [27.855282, 56.759326],
            [28.176709, 56.16913],
            [27.10246, 55.783314],
            [26.494331, 55.615107],
            [25.533047, 56.100297],
            [25.000934, 56.164531],
            [24.860684, 56.372528],
            [23.878264, 56.273671],
            [22.201157, 56.337802],
            [21.0558, 56.031076],
          ],
        ],
      },
      id: "LVA",
    },
    {
      type: "Feature",
      properties: {
        name: "Morocco",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-5.193863, 35.755182],
            [-4.591006, 35.330712],
            [-3.640057, 35.399855],
            [-2.604306, 35.179093],
            [-2.169914, 35.168396],
            [-1.792986, 34.527919],
            [-1.733455, 33.919713],
            [-1.388049, 32.864015],
            [-1.124551, 32.651522],
            [-1.307899, 32.262889],
            [-2.616605, 32.094346],
            [-3.06898, 31.724498],
            [-3.647498, 31.637294],
            [-3.690441, 30.896952],
            [-4.859646, 30.501188],
            [-5.242129, 30.000443],
            [-6.060632, 29.7317],
            [-7.059228, 29.579228],
            [-8.674116, 28.841289],
            [-8.66559, 27.656426],
            [-8.817809, 27.656426],
            [-8.817828, 27.656426],
```

```
          [-8.794884, 27.120696],
          [-9.413037, 27.088476],
          [-9.735343, 26.860945],
          [-10.189424, 26.860945],
          [-10.551263, 26.990808],
          [-11.392555, 26.883424],
          [-11.71822, 26.104092],
          [-12.030759, 26.030866],
          [-12.500963, 24.770116],
          [-13.89111, 23.691009],
          [-14.221168, 22.310163],
          [-14.630833, 21.86094],
          [-14.750955, 21.5006],
          [-17.002962, 21.420734],
          [-17.020428, 21.42231],
          [-16.973248, 21.885745],
          [-16.589137, 22.158234],
          [-16.261922, 22.67934],
          [-16.326414, 23.017768],
          [-15.982611, 23.723358],
          [-15.426004, 24.359134],
          [-15.089332, 24.520261],
          [-14.824645, 25.103533],
          [-14.800926, 25.636265],
          [-14.43994, 26.254418],
          [-13.773805, 26.618892],
          [-13.139942, 27.640148],
          [-13.121613, 27.654148],
          [-12.618837, 28.038186],
          [-11.688919, 28.148644],
          [-10.900957, 28.832142],
          [-10.399592, 29.098586],
          [-9.564811, 29.933574],
          [-9.814718, 31.177736],
          [-9.434793, 32.038096],
          [-9.300693, 32.564679],
          [-8.657476, 33.240245],
          [-7.654178, 33.697065],
          [-6.912544, 34.110476],
          [-6.244342, 35.145865],
          [-5.929994, 35.759988],
          [-5.193863, 35.755182],
        ],
      ],
    },
    id: "MAR",
  },
  {
    type: "Feature",
    properties: {
      name: "Moldova",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [26.619337, 48.220726],
          [26.857824, 48.368211],
          [27.522537, 48.467119],
          [28.259547, 48.155562],
          [28.670891, 48.118149],
          [29.122698, 47.849095],
          [29.050868, 47.510227],
          [29.415135, 47.346645],
          [29.559674, 46.928583],
          [29.908852, 46.674361],
```

```
          [29.83821, 46.525326],
          [30.024659, 46.423937],
          [29.759972, 46.349988],
          [29.170654, 46.379262],
          [29.072107, 46.517678],
          [28.862972, 46.437889],
          [28.933717, 46.25883],
          [28.659987, 45.939987],
          [28.485269, 45.596907],
          [28.233554, 45.488283],
          [28.054443, 45.944586],
          [28.160018, 46.371563],
          [28.12803, 46.810476],
          [27.551166, 47.405117],
          [27.233873, 47.826771],
          [26.924176, 48.123264],
          [26.619337, 48.220726],
        ],
      ],
    },
    id: "MDA",
  },
  {
    type: "Feature",
    properties: {
      name: "Madagascar",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [49.543519, -12.469833],
          [49.808981, -12.895285],
          [50.056511, -13.555761],
          [50.217431, -14.758789],
          [50.476537, -15.226512],
          [50.377111, -15.706069],
          [50.200275, -16.000263],
          [49.860606, -15.414253],
          [49.672607, -15.710204],
          [49.863344, -16.451037],
          [49.774564, -16.875042],
          [49.498612, -17.106036],
          [49.435619, -17.953064],
          [49.041792, -19.118781],
          [48.548541, -20.496888],
          [47.930749, -22.391501],
          [47.547723, -23.781959],
          [47.095761, -24.94163],
          [46.282478, -25.178463],
          [45.409508, -25.601434],
          [44.833574, -25.346101],
          [44.03972, -24.988345],
          [43.763768, -24.460677],
          [43.697778, -23.574116],
          [43.345654, -22.776904],
          [43.254187, -22.057413],
          [43.433298, -21.336475],
          [43.893683, -21.163307],
          [43.89637, -20.830459],
          [44.374325, -20.072366],
          [44.464397, -19.435454],
          [44.232422, -18.961995],
          [44.042976, -18.331387],
          [43.963084, -17.409945],
          [44.312469, -16.850496],
```

```
          [44.446517, -16.216219],
          [44.944937, -16.179374],
          [45.502732, -15.974373],
          [45.872994, -15.793454],
          [46.312243, -15.780018],
          [46.882183, -15.210182],
          [47.70513, -14.594303],
          [48.005215, -14.091233],
          [47.869047, -13.663869],
          [48.293828, -13.784068],
          [48.84506, -13.089175],
          [48.863509, -12.487868],
          [49.194651, -12.040557],
          [49.543519, -12.469833],
        ],
      ],
    },
    id: "MDG",
  },
  {
    type: "Feature",
    properties: {
      name: "Mexico",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-97.140008, 25.869997],
          [-97.528072, 24.992144],
          [-97.702946, 24.272343],
          [-97.776042, 22.93258],
          [-97.872367, 22.444212],
          [-97.699044, 21.898689],
          [-97.38896, 21.411019],
          [-97.189333, 20.635433],
          [-96.525576, 19.890931],
          [-96.292127, 19.320371],
          [-95.900885, 18.828024],
          [-94.839063, 18.562717],
          [-94.42573, 18.144371],
          [-93.548651, 18.423837],
          [-92.786114, 18.524839],
          [-92.037348, 18.704569],
          [-91.407903, 18.876083],
          [-90.77187, 19.28412],
          [-90.53359, 19.867418],
          [-90.451476, 20.707522],
          [-90.278618, 20.999855],
          [-89.601321, 21.261726],
          [-88.543866, 21.493675],
          [-87.658417, 21.458846],
          [-87.05189, 21.543543],
          [-86.811982, 21.331515],
          [-86.845908, 20.849865],
          [-87.383291, 20.255405],
          [-87.621054, 19.646553],
          [-87.43675, 19.472403],
          [-87.58656, 19.04013],
          [-87.837191, 18.259816],
          [-88.090664, 18.516648],
          [-88.300031, 18.499982],
          [-88.490123, 18.486831],
          [-88.848344, 17.883198],
          [-89.029857, 18.001511],
          [-89.150909, 17.955468],
```

```
            [-89.14308, 17.808319],
            [-90.067934, 17.819326],
            [-91.00152, 17.817595],
            [-91.002269, 17.254658],
            [-91.453921, 17.252177],
            [-91.08167, 16.918477],
            [-90.711822, 16.687483],
            [-90.600847, 16.470778],
            [-90.438867, 16.41011],
            [-90.464473, 16.069562],
            [-91.74796, 16.066565],
            [-92.229249, 15.251447],
            [-92.087216, 15.064585],
            [-92.20323, 14.830103],
            [-92.22775, 14.538829],
            [-93.359464, 15.61543],
            [-93.875169, 15.940164],
            [-94.691656, 16.200975],
            [-95.250227, 16.128318],
            [-96.053382, 15.752088],
            [-96.557434, 15.653515],
            [-97.263592, 15.917065],
            [-98.01303, 16.107312],
            [-98.947676, 16.566043],
            [-99.697397, 16.706164],
            [-100.829499, 17.171071],
            [-101.666089, 17.649026],
            [-101.918528, 17.91609],
            [-102.478132, 17.975751],
            [-103.50099, 18.292295],
            [-103.917527, 18.748572],
            [-104.99201, 19.316134],
            [-105.493038, 19.946767],
            [-105.731396, 20.434102],
            [-105.397773, 20.531719],
            [-105.500661, 20.816895],
            [-105.270752, 21.076285],
            [-105.265817, 21.422104],
            [-105.603161, 21.871146],
            [-105.693414, 22.26908],
            [-106.028716, 22.773752],
            [-106.90998, 23.767774],
            [-107.915449, 24.548915],
            [-108.401905, 25.172314],
            [-109.260199, 25.580609],
            [-109.444089, 25.824884],
            [-109.291644, 26.442934],
            [-109.801458, 26.676176],
            [-110.391732, 27.162115],
            [-110.641019, 27.859876],
            [-111.178919, 27.941241],
            [-111.759607, 28.467953],
            [-112.228235, 28.954409],
            [-112.271824, 29.266844],
            [-112.809594, 30.021114],
            [-113.163811, 30.786881],
            [-113.148669, 31.170966],
            [-113.871881, 31.567608],
            [-114.205737, 31.524045],
            [-114.776451, 31.799532],
            [-114.9367, 31.393485],
            [-114.771232, 30.913617],
            [-114.673899, 30.162681],
            [-114.330974, 29.750432],
            [-113.588875, 29.061611],
            [-113.424053, 28.826174],
```

```
            [-113.271969, 28.754783],
            [-113.140039, 28.411289],
            [-112.962298, 28.42519],
            [-112.761587, 27.780217],
            [-112.457911, 27.525814],
            [-112.244952, 27.171727],
            [-111.616489, 26.662817],
            [-111.284675, 25.73259],
            [-110.987819, 25.294606],
            [-110.710007, 24.826004],
            [-110.655049, 24.298595],
            [-110.172856, 24.265548],
            [-109.771847, 23.811183],
            [-109.409104, 23.364672],
            [-109.433392, 23.185588],
            [-109.854219, 22.818272],
            [-110.031392, 22.823078],
            [-110.295071, 23.430973],
            [-110.949501, 24.000964],
            [-111.670568, 24.484423],
            [-112.182036, 24.738413],
            [-112.148989, 25.470125],
            [-112.300711, 26.012004],
            [-112.777297, 26.32196],
            [-113.464671, 26.768186],
            [-113.59673, 26.63946],
            [-113.848937, 26.900064],
            [-114.465747, 27.14209],
            [-115.055142, 27.722727],
            [-114.982253, 27.7982],
            [-114.570366, 27.741485],
            [-114.199329, 28.115003],
            [-114.162018, 28.566112],
            [-114.931842, 29.279479],
            [-115.518654, 29.556362],
            [-115.887365, 30.180794],
            [-116.25835, 30.836464],
            [-116.721526, 31.635744],
            [-117.12776, 32.53534],
            [-115.99135, 32.61239],
            [-114.72139, 32.72083],
            [-114.815, 32.52528],
            [-113.30498, 32.03914],
            [-111.02361, 31.33472],
            [-109.035, 31.34194],
            [-108.24194, 31.34222],
            [-108.24, 31.754854],
            [-106.50759, 31.75452],
            [-106.1429, 31.39995],
            [-105.63159, 31.08383],
            [-105.03737, 30.64402],
            [-104.70575, 30.12173],
            [-104.45697, 29.57196],
            [-103.94, 29.27],
            [-103.11, 28.97],
            [-102.48, 29.76],
            [-101.6624, 29.7793],
            [-100.9576, 29.38071],
            [-100.45584, 28.69612],
            [-100.11, 28.11],
            [-99.52, 27.54],
            [-99.3, 26.84],
            [-99.02, 26.37],
            [-98.24, 26.06],
            [-97.53, 25.84],
            [-97.140008, 25.869997],
```

```
          ],
        ],
      },
      id: "MEX",
    },
    {
      type: "Feature",
      properties: {
        name: "Macedonia",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [20.59023, 41.85541],
            [20.71731, 41.84711],
            [20.76216, 42.05186],
            [21.3527, 42.2068],
            [21.576636, 42.245224],
            [21.91708, 42.30364],
            [22.380526, 42.32026],
            [22.881374, 41.999297],
            [22.952377, 41.337994],
            [22.76177, 41.3048],
            [22.597308, 41.130487],
            [22.055378, 41.149866],
            [21.674161, 40.931275],
            [21.02004, 40.842727],
            [20.60518, 41.08622],
            [20.46315, 41.51509],
            [20.59023, 41.85541],
          ],
        ],
      },
      id: "MKD",
    },
    {
      type: "Feature",
      properties: {
        name: "Mali",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-12.17075, 14.616834],
            [-11.834208, 14.799097],
            [-11.666078, 15.388208],
            [-11.349095, 15.411256],
            [-10.650791, 15.132746],
            [-10.086846, 15.330486],
            [-9.700255, 15.264107],
            [-9.550238, 15.486497],
            [-5.537744, 15.50169],
            [-5.315277, 16.201854],
            [-5.488523, 16.325102],
            [-5.971129, 20.640833],
            [-6.453787, 24.956591],
            [-4.923337, 24.974574],
            [-1.550055, 22.792666],
            [1.823228, 20.610809],
            [2.060991, 20.142233],
            [2.683588, 19.85623],
            [3.146661, 19.693579],
            [3.158133, 19.057364],
            [4.267419, 19.155265],
```

```
          [4.27021, 16.852227],
          [3.723422, 16.184284],
          [3.638259, 15.56812],
          [2.749993, 15.409525],
          [1.385528, 15.323561],
          [1.015783, 14.968182],
          [0.374892, 14.928908],
          [-0.266257, 14.924309],
          [-0.515854, 15.116158],
          [-1.066363, 14.973815],
          [-2.001035, 14.559008],
          [-2.191825, 14.246418],
          [-2.967694, 13.79815],
          [-3.103707, 13.541267],
          [-3.522803, 13.337662],
          [-4.006391, 13.472485],
          [-4.280405, 13.228444],
          [-4.427166, 12.542646],
          [-5.220942, 11.713859],
          [-5.197843, 11.375146],
          [-5.470565, 10.95127],
          [-5.404342, 10.370737],
          [-5.816926, 10.222555],
          [-6.050452, 10.096361],
          [-6.205223, 10.524061],
          [-6.493965, 10.411303],
          [-6.666461, 10.430811],
          [-6.850507, 10.138994],
          [-7.622759, 10.147236],
          [-7.89959, 10.297382],
          [-8.029944, 10.206535],
          [-8.335377, 10.494812],
          [-8.282357, 10.792597],
          [-8.407311, 10.909257],
          [-8.620321, 10.810891],
          [-8.581305, 11.136246],
          [-8.376305, 11.393646],
          [-8.786099, 11.812561],
          [-8.905265, 12.088358],
          [-9.127474, 12.30806],
          [-9.327616, 12.334286],
          [-9.567912, 12.194243],
          [-9.890993, 12.060479],
          [-10.165214, 11.844084],
          [-10.593224, 11.923975],
          [-10.87083, 12.177887],
          [-11.036556, 12.211245],
          [-11.297574, 12.077971],
          [-11.456169, 12.076834],
          [-11.513943, 12.442988],
          [-11.467899, 12.754519],
          [-11.553398, 13.141214],
          [-11.927716, 13.422075],
          [-12.124887, 13.994727],
          [-12.17075, 14.616834],
        ],
      ],
    },
    id: "MLI",
  },
  {
    type: "Feature",
    properties: {
      name: "Myanmar",
    },
    geometry: {
```

```
      type: "Polygon",
      coordinates: [
        [
          [99.543309, 20.186598],
          [98.959676, 19.752981],
          [98.253724, 19.708203],
          [97.797783, 18.62708],
          [97.375896, 18.445438],
          [97.859123, 17.567946],
          [98.493761, 16.837836],
          [98.903348, 16.177824],
          [98.537376, 15.308497],
          [98.192074, 15.123703],
          [98.430819, 14.622028],
          [99.097755, 13.827503],
          [99.212012, 13.269294],
          [99.196354, 12.804748],
          [99.587286, 11.892763],
          [99.038121, 10.960546],
          [98.553551, 9.93296],
          [98.457174, 10.675266],
          [98.764546, 11.441292],
          [98.428339, 12.032987],
          [98.509574, 13.122378],
          [98.103604, 13.64046],
          [97.777732, 14.837286],
          [97.597072, 16.100568],
          [97.16454, 16.928734],
          [96.505769, 16.427241],
          [95.369352, 15.71439],
          [94.808405, 15.803454],
          [94.188804, 16.037936],
          [94.533486, 17.27724],
          [94.324817, 18.213514],
          [93.540988, 19.366493],
          [93.663255, 19.726962],
          [93.078278, 19.855145],
          [92.368554, 20.670883],
          [92.303234, 21.475485],
          [92.652257, 21.324048],
          [92.672721, 22.041239],
          [93.166128, 22.27846],
          [93.060294, 22.703111],
          [93.286327, 23.043658],
          [93.325188, 24.078556],
          [94.106742, 23.850741],
          [94.552658, 24.675238],
          [94.603249, 25.162495],
          [95.155153, 26.001307],
          [95.124768, 26.573572],
          [96.419366, 27.264589],
          [97.133999, 27.083774],
          [97.051989, 27.699059],
          [97.402561, 27.882536],
          [97.327114, 28.261583],
          [97.911988, 28.335945],
          [98.246231, 27.747221],
          [98.68269, 27.508812],
          [98.712094, 26.743536],
          [98.671838, 25.918703],
          [97.724609, 25.083637],
          [97.60472, 23.897405],
          [98.660262, 24.063286],
          [98.898749, 23.142722],
          [99.531992, 22.949039],
          [99.240899, 22.118314],
```

```
          [99.983489, 21.742937],
          [100.416538, 21.558839],
          [101.150033, 21.849984],
          [101.180005, 21.436573],
          [100.329101, 20.786122],
          [100.115988, 20.41785],
          [99.543309, 20.186598],
        ],
      ],
    },
    id: "MMR",
  },
  {
    type: "Feature",
    properties: {
      name: "Montenegro",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [19.801613, 42.500093],
          [19.738051, 42.688247],
          [19.30449, 42.19574],
          [19.37177, 41.87755],
          [19.16246, 41.95502],
          [18.88214, 42.28151],
          [18.45, 42.48],
          [18.56, 42.65],
          [18.70648, 43.20011],
          [19.03165, 43.43253],
          [19.21852, 43.52384],
          [19.48389, 43.35229],
          [19.63, 43.21378],
          [19.95857, 43.10604],
          [20.3398, 42.89852],
          [20.25758, 42.81275],
          [20.0707, 42.58863],
          [19.801613, 42.500093],
        ],
      ],
    },
    id: "MNE",
  },
  {
    type: "Feature",
    properties: {
      name: "Mongolia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [87.751264, 49.297198],
          [88.805567, 49.470521],
          [90.713667, 50.331812],
          [92.234712, 50.802171],
          [93.104219, 50.49529],
          [94.147566, 50.480537],
          [94.815949, 50.013433],
          [95.814028, 49.977467],
          [97.259728, 49.726061],
          [98.231762, 50.422401],
          [97.82574, 51.010995],
          [98.861491, 52.047366],
          [99.981732, 51.634006],
```

```
          [100.88948, 51.516856],
          [102.065223, 51.259921],
          [102.255909, 50.510561],
          [103.676545, 50.089966],
          [104.621552, 50.275329],
          [105.886591, 50.406019],
          [106.888804, 50.274296],
          [107.868176, 49.793705],
          [108.475167, 49.282548],
          [109.402449, 49.292961],
          [110.662011, 49.130128],
          [111.581231, 49.377968],
          [112.89774, 49.543565],
          [114.362456, 50.248303],
          [114.96211, 50.140247],
          [115.485695, 49.805177],
          [116.678801, 49.888531],
          [116.191802, 49.134598],
          [115.485282, 48.135383],
          [115.742837, 47.726545],
          [116.308953, 47.85341],
          [117.295507, 47.697709],
          [118.064143, 48.06673],
          [118.866574, 47.74706],
          [119.772824, 47.048059],
          [119.66327, 46.69268],
          [118.874326, 46.805412],
          [117.421701, 46.672733],
          [116.717868, 46.388202],
          [115.985096, 45.727235],
          [114.460332, 45.339817],
          [113.463907, 44.808893],
          [112.436062, 45.011646],
          [111.873306, 45.102079],
          [111.348377, 44.457442],
          [111.667737, 44.073176],
          [111.829588, 43.743118],
          [111.129682, 43.406834],
          [110.412103, 42.871234],
          [109.243596, 42.519446],
          [107.744773, 42.481516],
          [106.129316, 42.134328],
          [104.964994, 41.59741],
          [104.522282, 41.908347],
          [103.312278, 41.907468],
          [101.83304, 42.514873],
          [100.845866, 42.663804],
          [99.515817, 42.524691],
          [97.451757, 42.74889],
          [96.349396, 42.725635],
          [95.762455, 43.319449],
          [95.306875, 44.241331],
          [94.688929, 44.352332],
          [93.480734, 44.975472],
          [92.133891, 45.115076],
          [90.94554, 45.286073],
          [90.585768, 45.719716],
          [90.970809, 46.888146],
          [90.280826, 47.693549],
          [88.854298, 48.069082],
          [88.013832, 48.599463],
          [87.751264, 49.297198],
        ],
      ],
    },
    id: "MNG",
```

```
    },
    {
      type: "Feature",
      properties: {
        name: "Mozambique",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [34.559989, -11.52002],
            [35.312398, -11.439146],
            [36.514082, -11.720938],
            [36.775151, -11.594537],
            [37.471284, -11.568751],
            [37.827645, -11.268769],
            [38.427557, -11.285202],
            [39.52103, -10.896854],
            [40.316589, -10.317096],
            [40.478387, -10.765441],
            [40.437253, -11.761711],
            [40.560811, -12.639177],
            [40.59962, -14.201975],
            [40.775475, -14.691764],
            [40.477251, -15.406294],
            [40.089264, -16.100774],
            [39.452559, -16.720891],
            [38.538351, -17.101023],
            [37.411133, -17.586368],
            [36.281279, -18.659688],
            [35.896497, -18.84226],
            [35.1984, -19.552811],
            [34.786383, -19.784012],
            [34.701893, -20.497043],
            [35.176127, -21.254361],
            [35.373428, -21.840837],
            [35.385848, -22.14],
            [35.562546, -22.09],
            [35.533935, -23.070788],
            [35.371774, -23.535359],
            [35.60747, -23.706563],
            [35.458746, -24.12261],
            [35.040735, -24.478351],
            [34.215824, -24.816314],
            [33.01321, -25.357573],
            [32.574632, -25.727318],
            [32.660363, -26.148584],
            [32.915955, -26.215867],
            [32.83012, -26.742192],
            [32.071665, -26.73382],
            [31.985779, -26.29178],
            [31.837778, -25.843332],
            [31.752408, -25.484284],
            [31.930589, -24.369417],
            [31.670398, -23.658969],
            [31.191409, -22.25151],
            [32.244988, -21.116489],
            [32.508693, -20.395292],
            [32.659743, -20.30429],
            [32.772708, -19.715592],
            [32.611994, -19.419383],
            [32.654886, -18.67209],
            [32.849861, -17.979057],
            [32.847639, -16.713398],
            [32.328239, -16.392074],
            [31.852041, -16.319417],
```

```
          [31.636498, -16.07199],
          [31.173064, -15.860944],
          [30.338955, -15.880839],
          [30.274256, -15.507787],
          [30.179481, -14.796099],
          [33.214025, -13.97186],
          [33.7897, -14.451831],
          [34.064825, -14.35995],
          [34.459633, -14.61301],
          [34.517666, -15.013709],
          [34.307291, -15.478641],
          [34.381292, -16.18356],
          [35.03381, -16.8013],
          [35.339063, -16.10744],
          [35.771905, -15.896859],
          [35.686845, -14.611046],
          [35.267956, -13.887834],
          [34.907151, -13.565425],
          [34.559989, -13.579998],
          [34.280006, -12.280025],
          [34.559989, -11.52002],
        ],
      ],
    },
    id: "MOZ",
  },
  {
    type: "Feature",
    properties: {
      name: "Mauritania",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-12.17075, 14.616834],
          [-12.830658, 15.303692],
          [-13.435738, 16.039383],
          [-14.099521, 16.304302],
          [-14.577348, 16.598264],
          [-15.135737, 16.587282],
          [-15.623666, 16.369337],
          [-16.12069, 16.455663],
          [-16.463098, 16.135036],
          [-16.549708, 16.673892],
          [-16.270552, 17.166963],
          [-16.146347, 18.108482],
          [-16.256883, 19.096716],
          [-16.377651, 19.593817],
          [-16.277838, 20.092521],
          [-16.536324, 20.567866],
          [-17.063423, 20.999752],
          [-16.845194, 21.333323],
          [-12.929102, 21.327071],
          [-13.118754, 22.77122],
          [-12.874222, 23.284832],
          [-11.937224, 23.374594],
          [-11.969419, 25.933353],
          [-8.687294, 25.881056],
          [-8.6844, 27.395744],
          [-4.923337, 24.974574],
          [-6.453787, 24.956591],
          [-5.971129, 20.640833],
          [-5.488523, 16.325102],
          [-5.315277, 16.201854],
          [-5.537744, 15.50169],
```

```
          [-9.550238, 15.486497],
          [-9.700255, 15.264107],
          [-10.086846, 15.330486],
          [-10.650791, 15.132746],
          [-11.349095, 15.411256],
          [-11.666078, 15.388208],
          [-11.834208, 14.799097],
          [-12.17075, 14.616834],
        ],
      ],
    },
    id: "MRT",
  },
  {
    type: "Feature",
    properties: {
      name: "Malawi",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [34.559989, -11.52002],
          [34.280006, -12.280025],
          [34.559989, -13.579998],
          [34.907151, -13.565425],
          [35.267956, -13.887834],
          [35.686845, -14.611046],
          [35.771905, -15.896859],
          [35.339063, -16.10744],
          [35.03381, -16.8013],
          [34.381292, -16.18356],
          [34.307291, -15.478641],
          [34.517666, -15.013709],
          [34.459633, -14.61301],
          [34.064825, -14.35995],
          [33.7897, -14.451831],
          [33.214025, -13.97186],
          [32.688165, -13.712858],
          [32.991764, -12.783871],
          [33.306422, -12.435778],
          [33.114289, -11.607198],
          [33.31531, -10.79655],
          [33.485688, -10.525559],
          [33.231388, -9.676722],
          [32.759375, -9.230599],
          [33.739729, -9.417151],
          [33.940838, -9.693674],
          [34.280006, -10.16],
          [34.559989, -11.52002],
        ],
      ],
    },
    id: "MWI",
  },
  {
    type: "Feature",
    properties: {
      name: "Malaysia",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [101.075516, 6.204867],
```

```
        [101.154219, 5.691384],
        [101.814282, 5.810808],
        [102.141187, 6.221636],
        [102.371147, 6.128205],
        [102.961705, 5.524495],
        [103.381215, 4.855001],
        [103.438575, 4.181606],
        [103.332122, 3.726698],
        [103.429429, 3.382869],
        [103.502448, 2.791019],
        [103.854674, 2.515454],
        [104.247932, 1.631141],
        [104.228811, 1.293048],
        [103.519707, 1.226334],
        [102.573615, 1.967115],
        [101.390638, 2.760814],
        [101.27354, 3.270292],
        [100.695435, 3.93914],
        [100.557408, 4.76728],
        [100.196706, 5.312493],
        [100.30626, 6.040562],
        [100.085757, 6.464489],
        [100.259596, 6.642825],
        [101.075516, 6.204867],
      ],
    ],
    [
      [
        [118.618321, 4.478202],
        [117.882035, 4.137551],
        [117.015214, 4.306094],
        [115.865517, 4.306559],
        [115.519078, 3.169238],
        [115.134037, 2.821482],
        [114.621355, 1.430688],
        [113.80585, 1.217549],
        [112.859809, 1.49779],
        [112.380252, 1.410121],
        [111.797548, 0.904441],
        [111.159138, 0.976478],
        [110.514061, 0.773131],
        [109.830227, 1.338136],
        [109.66326, 2.006467],
        [110.396135, 1.663775],
        [111.168853, 1.850637],
        [111.370081, 2.697303],
        [111.796928, 2.885897],
        [112.995615, 3.102395],
        [113.712935, 3.893509],
        [114.204017, 4.525874],
        [114.659596, 4.007637],
        [114.869557, 4.348314],
        [115.347461, 4.316636],
        [115.4057, 4.955228],
        [115.45071, 5.44773],
        [116.220741, 6.143191],
        [116.725103, 6.924771],
        [117.129626, 6.928053],
        [117.643393, 6.422166],
        [117.689075, 5.98749],
        [118.347691, 5.708696],
        [119.181904, 5.407836],
        [119.110694, 5.016128],
        [118.439727, 4.966519],
        [118.618321, 4.478202],
      ],
```

```
        ],
      ],
    },
    id: "MYS",
  },
  {
    type: "Feature",
    properties: {
      name: "Namibia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [16.344977, -28.576705],
          [15.601818, -27.821247],
          [15.210472, -27.090956],
          [14.989711, -26.117372],
          [14.743214, -25.39292],
          [14.408144, -23.853014],
          [14.385717, -22.656653],
          [14.257714, -22.111208],
          [13.868642, -21.699037],
          [13.352498, -20.872834],
          [12.826845, -19.673166],
          [12.608564, -19.045349],
          [11.794919, -18.069129],
          [11.734199, -17.301889],
          [12.215461, -17.111668],
          [12.814081, -16.941343],
          [13.462362, -16.971212],
          [14.058501, -17.423381],
          [14.209707, -17.353101],
          [18.263309, -17.309951],
          [18.956187, -17.789095],
          [21.377176, -17.930636],
          [23.215048, -17.523116],
          [24.033862, -17.295843],
          [24.682349, -17.353411],
          [25.07695, -17.578823],
          [25.084443, -17.661816],
          [24.520705, -17.887125],
          [24.217365, -17.889347],
          [23.579006, -18.281261],
          [23.196858, -17.869038],
          [21.65504, -18.219146],
          [20.910641, -18.252219],
          [20.881134, -21.814327],
          [19.895458, -21.849157],
          [19.895768, -24.76779],
          [19.894734, -28.461105],
          [19.002127, -28.972443],
          [18.464899, -29.045462],
          [17.836152, -28.856378],
          [17.387497, -28.783514],
          [17.218929, -28.355943],
          [16.824017, -28.082162],
          [16.344977, -28.576705],
        ],
      ],
    },
    id: "NAM",
  },
  {
    type: "Feature",
    properties: {
```

```
        name: "New Caledonia",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [165.77999, -21.080005],
            [166.599991, -21.700019],
            [167.120011, -22.159991],
            [166.740035, -22.399976],
            [166.189732, -22.129708],
            [165.474375, -21.679607],
            [164.829815, -21.14982],
            [164.167995, -20.444747],
            [164.029606, -20.105646],
            [164.459967, -20.120012],
            [165.020036, -20.459991],
            [165.460009, -20.800022],
            [165.77999, -21.080005],
          ],
        ],
      },
      id: "NCL",
    },
    {
      type: "Feature",
      properties: {
        name: "Niger",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [2.154474, 11.94015],
            [2.177108, 12.625018],
            [1.024103, 12.851826],
            [0.993046, 13.33575],
            [0.429928, 13.988733],
            [0.295646, 14.444235],
            [0.374892, 14.928908],
            [1.015783, 14.968182],
            [1.385528, 15.323561],
            [2.749993, 15.409525],
            [3.638259, 15.56812],
            [3.723422, 16.184284],
            [4.27021, 16.852227],
            [4.267419, 19.155265],
            [5.677566, 19.601207],
            [8.572893, 21.565661],
            [11.999506, 23.471668],
            [13.581425, 23.040506],
            [14.143871, 22.491289],
            [14.8513, 22.86295],
            [15.096888, 21.308519],
            [15.471077, 21.048457],
            [15.487148, 20.730415],
            [15.903247, 20.387619],
            [15.685741, 19.95718],
            [15.300441, 17.92795],
            [15.247731, 16.627306],
            [13.972202, 15.684366],
            [13.540394, 14.367134],
            [13.956699, 13.996691],
            [13.954477, 13.353449],
            [14.595781, 13.330427],
            [14.495787, 12.859396],
```

```
          [14.213531, 12.802035],
          [14.181336, 12.483657],
          [13.995353, 12.461565],
          [13.318702, 13.556356],
          [13.083987, 13.596147],
          [12.302071, 13.037189],
          [11.527803, 13.32898],
          [10.989593, 13.387323],
          [10.701032, 13.246918],
          [10.114814, 13.277252],
          [9.524928, 12.851102],
          [9.014933, 12.826659],
          [7.804671, 13.343527],
          [7.330747, 13.098038],
          [6.820442, 13.115091],
          [6.445426, 13.492768],
          [5.443058, 13.865924],
          [4.368344, 13.747482],
          [4.107946, 13.531216],
          [3.967283, 12.956109],
          [3.680634, 12.552903],
          [3.61118, 11.660167],
          [2.848643, 12.235636],
          [2.490164, 12.233052],
          [2.154474, 11.94015],
        ],
      ],
    },
    id: "NER",
  },
  {
    type: "Feature",
    properties: {
      name: "Nigeria",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [8.500288, 4.771983],
          [7.462108, 4.412108],
          [7.082596, 4.464689],
          [6.698072, 4.240594],
          [5.898173, 4.262453],
          [5.362805, 4.887971],
          [5.033574, 5.611802],
          [4.325607, 6.270651],
          [3.57418, 6.2583],
          [2.691702, 6.258817],
          [2.749063, 7.870734],
          [2.723793, 8.506845],
          [2.912308, 9.137608],
          [3.220352, 9.444153],
          [3.705438, 10.06321],
          [3.60007, 10.332186],
          [3.797112, 10.734746],
          [3.572216, 11.327939],
          [3.61118, 11.660167],
          [3.680634, 12.552903],
          [3.967283, 12.956109],
          [4.107946, 13.531216],
          [4.368344, 13.747482],
          [5.443058, 13.865924],
          [6.445426, 13.492768],
          [6.820442, 13.115091],
          [7.330747, 13.098038],
```

```
          [7.804671, 13.343527],
          [9.014933, 12.826659],
          [9.524928, 12.851102],
          [10.114814, 13.277252],
          [10.701032, 13.246918],
          [10.989593, 13.387323],
          [11.527803, 13.32898],
          [12.302071, 13.037189],
          [13.083987, 13.596147],
          [13.318702, 13.556356],
          [13.995353, 12.461565],
          [14.181336, 12.483657],
          [14.577178, 12.085361],
          [14.468192, 11.904752],
          [14.415379, 11.572369],
          [13.57295, 10.798566],
          [13.308676, 10.160362],
          [13.1676, 9.640626],
          [12.955468, 9.417772],
          [12.753672, 8.717763],
          [12.218872, 8.305824],
          [12.063946, 7.799808],
          [11.839309, 7.397042],
          [11.745774, 6.981383],
          [11.058788, 6.644427],
          [10.497375, 7.055358],
          [10.118277, 7.03877],
          [9.522706, 6.453482],
          [9.233163, 6.444491],
          [8.757533, 5.479666],
          [8.500288, 4.771983],
        ],
      ],
    },
    id: "NGA",
  },
  {
    type: "Feature",
    properties: {
      name: "Nicaragua",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-85.71254, 11.088445],
          [-86.058488, 11.403439],
          [-86.52585, 11.806877],
          [-86.745992, 12.143962],
          [-87.167516, 12.458258],
          [-87.668493, 12.90991],
          [-87.557467, 13.064552],
          [-87.392386, 12.914018],
          [-87.316654, 12.984686],
          [-87.005769, 13.025794],
          [-86.880557, 13.254204],
          [-86.733822, 13.263093],
          [-86.755087, 13.754845],
          [-86.520708, 13.778487],
          [-86.312142, 13.771356],
          [-86.096264, 14.038187],
          [-85.801295, 13.836055],
          [-85.698665, 13.960078],
          [-85.514413, 14.079012],
          [-85.165365, 14.35437],
          [-85.148751, 14.560197],
```

```
          [-85.052787, 14.551541],
          [-84.924501, 14.790493],
          [-84.820037, 14.819587],
          [-84.649582, 14.666805],
          [-84.449336, 14.621614],
          [-84.228342, 14.748764],
          [-83.975721, 14.749436],
          [-83.628585, 14.880074],
          [-83.489989, 15.016267],
          [-83.147219, 14.995829],
          [-83.233234, 14.899866],
          [-83.284162, 14.676624],
          [-83.182126, 14.310703],
          [-83.4125, 13.970078],
          [-83.519832, 13.567699],
          [-83.552207, 13.127054],
          [-83.498515, 12.869292],
          [-83.473323, 12.419087],
          [-83.626104, 12.32085],
          [-83.719613, 11.893124],
          [-83.650858, 11.629032],
          [-83.85547, 11.373311],
          [-83.808936, 11.103044],
          [-83.655612, 10.938764],
          [-83.895054, 10.726839],
          [-84.190179, 10.79345],
          [-84.355931, 10.999226],
          [-84.673069, 11.082657],
          [-84.903003, 10.952303],
          [-85.561852, 11.217119],
          [-85.71254, 11.088445],
        ],
      ],
    },
    id: "NIC",
  },
  {
    type: "Feature",
    properties: {
      name: "Netherlands",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [6.074183, 53.510403],
          [6.90514, 53.482162],
          [7.092053, 53.144043],
          [6.84287, 52.22844],
          [6.589397, 51.852029],
          [5.988658, 51.851616],
          [6.156658, 50.803721],
          [5.606976, 51.037298],
          [4.973991, 51.475024],
          [4.047071, 51.267259],
          [3.314971, 51.345755],
          [3.830289, 51.620545],
          [4.705997, 53.091798],
          [6.074183, 53.510403],
        ],
      ],
    },
    id: "NLD",
  },
  {
    type: "Feature",
```

```
      properties: {
        name: "Norway",
      },
      geometry: {
        type: "MultiPolygon",
        coordinates: [
          [
            [
              [28.165547, 71.185474],
              [31.293418, 70.453788],
              [30.005435, 70.186259],
              [31.101079, 69.55808],
              [29.399581, 69.156916],
              [28.59193, 69.064777],
              [29.015573, 69.766491],
              [27.732292, 70.164193],
              [26.179622, 69.825299],
              [25.689213, 69.092114],
              [24.735679, 68.649557],
              [23.66205, 68.891247],
              [22.356238, 68.841741],
              [21.244936, 69.370443],
              [20.645593, 69.106247],
              [20.025269, 69.065139],
              [19.87856, 68.407194],
              [17.993868, 68.567391],
              [17.729182, 68.010552],
              [16.768879, 68.013937],
              [16.108712, 67.302456],
              [15.108411, 66.193867],
              [13.55569, 64.787028],
              [13.919905, 64.445421],
              [13.571916, 64.049114],
              [12.579935, 64.066219],
              [11.930569, 63.128318],
              [11.992064, 61.800362],
              [12.631147, 61.293572],
              [12.300366, 60.117933],
              [11.468272, 59.432393],
              [11.027369, 58.856149],
              [10.356557, 59.469807],
              [8.382, 58.313288],
              [7.048748, 58.078884],
              [5.665835, 58.588155],
              [5.308234, 59.663232],
              [4.992078, 61.970998],
              [5.9129, 62.614473],
              [8.553411, 63.454008],
              [10.527709, 64.486038],
              [12.358347, 65.879726],
              [14.761146, 67.810642],
              [16.435927, 68.563205],
              [19.184028, 69.817444],
              [21.378416, 70.255169],
              [23.023742, 70.202072],
              [24.546543, 71.030497],
              [26.37005, 70.986262],
              [28.165547, 71.185474],
            ],
          ],
          [
            [
              [24.72412, 77.85385],
              [22.49032, 77.44493],
              [20.72601, 77.67704],
              [21.41611, 77.93504],
```

```
            [20.8119, 78.25463],
            [22.88426, 78.45494],
            [23.28134, 78.07954],
            [24.72412, 77.85385],
          ],
        ],
        [
          [
            [18.25183, 79.70175],
            [21.54383, 78.95611],
            [19.02737, 78.5626],
            [18.47172, 77.82669],
            [17.59441, 77.63796],
            [17.1182, 76.80941],
            [15.91315, 76.77045],
            [13.76259, 77.38035],
            [14.66956, 77.73565],
            [13.1706, 78.02493],
            [11.22231, 78.8693],
            [10.44453, 79.65239],
            [13.17077, 80.01046],
            [13.71852, 79.66039],
            [15.14282, 79.67431],
            [15.52255, 80.01608],
            [16.99085, 80.05086],
            [18.25183, 79.70175],
          ],
        ],
        [
          [
            [25.447625, 80.40734],
            [27.407506, 80.056406],
            [25.924651, 79.517834],
            [23.024466, 79.400012],
            [20.075188, 79.566823],
            [19.897266, 79.842362],
            [18.462264, 79.85988],
            [17.368015, 80.318896],
            [20.455992, 80.598156],
            [21.907945, 80.357679],
            [22.919253, 80.657144],
            [25.447625, 80.40734],
          ],
        ],
      ],
    },
    id: "NOR",
  },
  {
    type: "Feature",
    properties: {
      name: "Nepal",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [88.120441, 27.876542],
          [88.043133, 27.445819],
          [88.174804, 26.810405],
          [88.060238, 26.414615],
          [87.227472, 26.397898],
          [86.024393, 26.630985],
          [85.251779, 26.726198],
          [84.675018, 27.234901],
          [83.304249, 27.364506],
```

```
          [81.999987, 27.925479],
          [81.057203, 28.416095],
          [80.088425, 28.79447],
          [80.476721, 29.729865],
          [81.111256, 30.183481],
          [81.525804, 30.422717],
          [82.327513, 30.115268],
          [83.337115, 29.463732],
          [83.898993, 29.320226],
          [84.23458, 28.839894],
          [85.011638, 28.642774],
          [85.82332, 28.203576],
          [86.954517, 27.974262],
          [88.120441, 27.876542],
        ],
      ],
    },
    id: "NPL",
  },
  {
    type: "Feature",
    properties: {
      name: "New Zealand",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [173.020375, -40.919052],
            [173.247234, -41.331999],
            [173.958405, -40.926701],
            [174.247587, -41.349155],
            [174.248517, -41.770008],
            [173.876447, -42.233184],
            [173.22274, -42.970038],
            [172.711246, -43.372288],
            [173.080113, -43.853344],
            [172.308584, -43.865694],
            [171.452925, -44.242519],
            [171.185138, -44.897104],
            [170.616697, -45.908929],
            [169.831422, -46.355775],
            [169.332331, -46.641235],
            [168.411354, -46.619945],
            [167.763745, -46.290197],
            [166.676886, -46.219917],
            [166.509144, -45.852705],
            [167.046424, -45.110941],
            [168.303763, -44.123973],
            [168.949409, -43.935819],
            [169.667815, -43.555326],
            [170.52492, -43.031688],
            [171.12509, -42.512754],
            [171.569714, -41.767424],
            [171.948709, -41.514417],
            [172.097227, -40.956104],
            [172.79858, -40.493962],
            [173.020375, -40.919052],
          ],
        ],
        [
          [
            [174.612009, -36.156397],
            [175.336616, -37.209098],
            [175.357596, -36.526194],
```

```
            [175.808887, -36.798942],
            [175.95849, -37.555382],
            [176.763195, -37.881253],
            [177.438813, -37.961248],
            [178.010354, -37.579825],
            [178.517094, -37.695373],
            [178.274731, -38.582813],
            [177.97046, -39.166343],
            [177.206993, -39.145776],
            [176.939981, -39.449736],
            [177.032946, -39.879943],
            [176.885824, -40.065978],
            [176.508017, -40.604808],
            [176.01244, -41.289624],
            [175.239567, -41.688308],
            [175.067898, -41.425895],
            [174.650973, -41.281821],
            [175.22763, -40.459236],
            [174.900157, -39.908933],
            [173.824047, -39.508854],
            [173.852262, -39.146602],
            [174.574802, -38.797683],
            [174.743474, -38.027808],
            [174.697017, -37.381129],
            [174.292028, -36.711092],
            [174.319004, -36.534824],
            [173.840997, -36.121981],
            [173.054171, -35.237125],
            [172.636005, -34.529107],
            [173.007042, -34.450662],
            [173.551298, -35.006183],
            [174.32939, -35.265496],
            [174.612009, -36.156397],
          ],
        ],
      ],
    },
    id: "NZL",
  },
  {
    type: "Feature",
    properties: {
      name: "Oman",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [58.861141, 21.114035],
            [58.487986, 20.428986],
            [58.034318, 20.481437],
            [57.826373, 20.243002],
            [57.665762, 19.736005],
            [57.7887, 19.06757],
            [57.694391, 18.94471],
            [57.234264, 18.947991],
            [56.609651, 18.574267],
            [56.512189, 18.087113],
            [56.283521, 17.876067],
            [55.661492, 17.884128],
            [55.269939, 17.632309],
            [55.2749, 17.228354],
            [54.791002, 16.950697],
            [54.239253, 17.044981],
            [53.570508, 16.707663],
```

```
            [53.108573, 16.651051],
            [52.782184, 17.349742],
            [52.00001, 19.000003],
            [54.999982, 19.999994],
            [55.666659, 22.000001],
            [55.208341, 22.70833],
            [55.234489, 23.110993],
            [55.525841, 23.524869],
            [55.528632, 23.933604],
            [55.981214, 24.130543],
            [55.804119, 24.269604],
            [55.886233, 24.920831],
            [56.396847, 24.924732],
            [56.84514, 24.241673],
            [57.403453, 23.878594],
            [58.136948, 23.747931],
            [58.729211, 23.565668],
            [59.180502, 22.992395],
            [59.450098, 22.660271],
            [59.80806, 22.533612],
            [59.806148, 22.310525],
            [59.442191, 21.714541],
            [59.282408, 21.433886],
            [58.861141, 21.114035],
          ],
        ],
        [
          [
            [56.391421, 25.895991],
            [56.261042, 25.714606],
            [56.070821, 26.055464],
            [56.362017, 26.395934],
            [56.485679, 26.309118],
            [56.391421, 25.895991],
          ],
        ],
      ],
    },
    id: "OMN",
  },
  {
    type: "Feature",
    properties: {
      name: "Pakistan",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [75.158028, 37.133031],
          [75.896897, 36.666806],
          [76.192848, 35.898403],
          [77.837451, 35.49401],
          [76.871722, 34.653544],
          [75.757061, 34.504923],
          [74.240203, 34.748887],
          [73.749948, 34.317699],
          [74.104294, 33.441473],
          [74.451559, 32.7649],
          [75.258642, 32.271105],
          [74.405929, 31.692639],
          [74.42138, 30.979815],
          [73.450638, 29.976413],
          [72.823752, 28.961592],
          [71.777666, 27.91318],
          [70.616496, 27.989196],
```

```
          [69.514393, 26.940966],
          [70.168927, 26.491872],
          [70.282873, 25.722229],
          [70.844699, 25.215102],
          [71.04324, 24.356524],
          [68.842599, 24.359134],
          [68.176645, 23.691965],
          [67.443667, 23.944844],
          [67.145442, 24.663611],
          [66.372828, 25.425141],
          [64.530408, 25.237039],
          [62.905701, 25.218409],
          [61.497363, 25.078237],
          [61.874187, 26.239975],
          [63.316632, 26.756532],
          [63.233898, 27.217047],
          [62.755426, 27.378923],
          [62.72783, 28.259645],
          [61.771868, 28.699334],
          [61.369309, 29.303276],
          [60.874248, 29.829239],
          [62.549857, 29.318572],
          [63.550261, 29.468331],
          [64.148002, 29.340819],
          [64.350419, 29.560031],
          [65.046862, 29.472181],
          [66.346473, 29.887943],
          [66.381458, 30.738899],
          [66.938891, 31.304911],
          [67.683394, 31.303154],
          [67.792689, 31.58293],
          [68.556932, 31.71331],
          [68.926677, 31.620189],
          [69.317764, 31.901412],
          [69.262522, 32.501944],
          [69.687147, 33.105499],
          [70.323594, 33.358533],
          [69.930543, 34.02012],
          [70.881803, 33.988856],
          [71.156773, 34.348911],
          [71.115019, 34.733126],
          [71.613076, 35.153203],
          [71.498768, 35.650563],
          [71.262348, 36.074388],
          [71.846292, 36.509942],
          [72.920025, 36.720007],
          [74.067552, 36.836176],
          [74.575893, 37.020841],
          [75.158028, 37.133031],
        ],
      ],
    },
    id: "PAK",
  },
  {
    type: "Feature",
    properties: {
      name: "Panama",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-77.881571, 7.223771],
          [-78.214936, 7.512255],
          [-78.429161, 8.052041],
```

```
          [-78.182096, 8.319182],
          [-78.435465, 8.387705],
          [-78.622121, 8.718124],
          [-79.120307, 8.996092],
          [-79.557877, 8.932375],
          [-79.760578, 8.584515],
          [-80.164481, 8.333316],
          [-80.382659, 8.298409],
          [-80.480689, 8.090308],
          [-80.00369, 7.547524],
          [-80.276671, 7.419754],
          [-80.421158, 7.271572],
          [-80.886401, 7.220541],
          [-81.059543, 7.817921],
          [-81.189716, 7.647906],
          [-81.519515, 7.70661],
          [-81.721311, 8.108963],
          [-82.131441, 8.175393],
          [-82.390934, 8.292362],
          [-82.820081, 8.290864],
          [-82.850958, 8.073823],
          [-82.965783, 8.225028],
          [-82.913176, 8.423517],
          [-82.829771, 8.626295],
          [-82.868657, 8.807266],
          [-82.719183, 8.925709],
          [-82.927155, 9.07433],
          [-82.932891, 9.476812],
          [-82.546196, 9.566135],
          [-82.187123, 9.207449],
          [-82.207586, 8.995575],
          [-81.808567, 8.950617],
          [-81.714154, 9.031955],
          [-81.439287, 8.786234],
          [-80.947302, 8.858504],
          [-80.521901, 9.111072],
          [-79.9146, 9.312765],
          [-79.573303, 9.61161],
          [-79.021192, 9.552931],
          [-79.05845, 9.454565],
          [-78.500888, 9.420459],
          [-78.055928, 9.24773],
          [-77.729514, 8.946844],
          [-77.353361, 8.670505],
          [-77.474723, 8.524286],
          [-77.242566, 7.935278],
          [-77.431108, 7.638061],
          [-77.753414, 7.70984],
          [-77.881571, 7.223771],
        ],
      ],
    },
    id: "PAN",
  },
  {
    type: "Feature",
    properties: {
      name: "Peru",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-69.590424, -17.580012],
          [-69.858444, -18.092694],
          [-70.372572, -18.347975],
```

```
[-71.37525, -17.773799],
[-71.462041, -17.363488],
[-73.44453, -16.359363],
[-75.237883, -15.265683],
[-76.009205, -14.649286],
[-76.423469, -13.823187],
[-76.259242, -13.535039],
[-77.106192, -12.222716],
[-78.092153, -10.377712],
[-79.036953, -8.386568],
[-79.44592, -7.930833],
[-79.760578, -7.194341],
[-80.537482, -6.541668],
[-81.249996, -6.136834],
[-80.926347, -5.690557],
[-81.410943, -4.736765],
[-81.09967, -4.036394],
[-80.302561, -3.404856],
[-80.184015, -3.821162],
[-80.469295, -4.059287],
[-80.442242, -4.425724],
[-80.028908, -4.346091],
[-79.624979, -4.454198],
[-79.205289, -4.959129],
[-78.639897, -4.547784],
[-78.450684, -3.873097],
[-77.837905, -3.003021],
[-76.635394, -2.608678],
[-75.544996, -1.56161],
[-75.233723, -0.911417],
[-75.373223, -0.152032],
[-75.106625, -0.057205],
[-74.441601, -0.53082],
[-74.122395, -1.002833],
[-73.659504, -1.260491],
[-73.070392, -2.308954],
[-72.325787, -2.434218],
[-71.774761, -2.16979],
[-71.413646, -2.342802],
[-70.813476, -2.256865],
[-70.047709, -2.725156],
[-70.692682, -3.742872],
[-70.394044, -3.766591],
[-69.893635, -4.298187],
[-70.794769, -4.251265],
[-70.928843, -4.401591],
[-71.748406, -4.593983],
[-72.891928, -5.274561],
[-72.964507, -5.741251],
[-73.219711, -6.089189],
[-73.120027, -6.629931],
[-73.724487, -6.918595],
[-73.723401, -7.340999],
[-73.987235, -7.52383],
[-73.571059, -8.424447],
[-73.015383, -9.032833],
[-73.226713, -9.462213],
[-72.563033, -9.520194],
[-72.184891, -10.053598],
[-71.302412, -10.079436],
[-70.481894, -9.490118],
[-70.548686, -11.009147],
[-70.093752, -11.123972],
[-69.529678, -10.951734],
[-68.66508, -12.5613],
[-68.88008, -12.899729],
```

```
          [-68.929224, -13.602684],
          [-68.948887, -14.453639],
          [-69.339535, -14.953195],
          [-69.160347, -15.323974],
          [-69.389764, -15.660129],
          [-68.959635, -16.500698],
          [-69.590424, -17.580012],
        ],
      ],
    },
    id: "PER",
  },
  {
    type: "Feature",
    properties: {
      name: "Philippines",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [126.376814, 8.414706],
            [126.478513, 7.750354],
            [126.537424, 7.189381],
            [126.196773, 6.274294],
            [125.831421, 7.293715],
            [125.363852, 6.786485],
            [125.683161, 6.049657],
            [125.396512, 5.581003],
            [124.219788, 6.161355],
            [123.93872, 6.885136],
            [124.243662, 7.36061],
            [123.610212, 7.833527],
            [123.296071, 7.418876],
            [122.825506, 7.457375],
            [122.085499, 6.899424],
            [121.919928, 7.192119],
            [122.312359, 8.034962],
            [122.942398, 8.316237],
            [123.487688, 8.69301],
            [123.841154, 8.240324],
            [124.60147, 8.514158],
            [124.764612, 8.960409],
            [125.471391, 8.986997],
            [125.412118, 9.760335],
            [126.222714, 9.286074],
            [126.306637, 8.782487],
            [126.376814, 8.414706],
          ],
        ],
        [
          [
            [123.982438, 10.278779],
            [123.623183, 9.950091],
            [123.309921, 9.318269],
            [122.995883, 9.022189],
            [122.380055, 9.713361],
            [122.586089, 9.981045],
            [122.837081, 10.261157],
            [122.947411, 10.881868],
            [123.49885, 10.940624],
            [123.337774, 10.267384],
            [124.077936, 11.232726],
            [123.982438, 10.278779],
          ],
```

```
      ],
      [
        [
          [118.504581, 9.316383],
          [117.174275, 8.3675],
          [117.664477, 9.066889],
          [118.386914, 9.6845],
          [118.987342, 10.376292],
          [119.511496, 11.369668],
          [119.689677, 10.554291],
          [119.029458, 10.003653],
          [118.504581, 9.316383],
        ],
      ],
      [
        [
          [121.883548, 11.891755],
          [122.483821, 11.582187],
          [123.120217, 11.58366],
          [123.100838, 11.165934],
          [122.637714, 10.741308],
          [122.00261, 10.441017],
          [121.967367, 10.905691],
          [122.03837, 11.415841],
          [121.883548, 11.891755],
        ],
      ],
      [
        [
          [125.502552, 12.162695],
          [125.783465, 11.046122],
          [125.011884, 11.311455],
          [125.032761, 10.975816],
          [125.277449, 10.358722],
          [124.801819, 10.134679],
          [124.760168, 10.837995],
          [124.459101, 10.88993],
          [124.302522, 11.495371],
          [124.891013, 11.415583],
          [124.87799, 11.79419],
          [124.266762, 12.557761],
          [125.227116, 12.535721],
          [125.502552, 12.162695],
        ],
      ],
      [
        [
          [121.527394, 13.06959],
          [121.26219, 12.20556],
          [120.833896, 12.704496],
          [120.323436, 13.466413],
          [121.180128, 13.429697],
          [121.527394, 13.06959],
        ],
      ],
      [
        [
          [121.321308, 18.504065],
          [121.937601, 18.218552],
          [122.246006, 18.47895],
          [122.336957, 18.224883],
          [122.174279, 17.810283],
          [122.515654, 17.093505],
          [122.252311, 16.262444],
          [121.662786, 15.931018],
          [121.50507, 15.124814],
```

```
            [121.728829, 14.328376],
            [122.258925, 14.218202],
            [122.701276, 14.336541],
            [123.950295, 13.782131],
            [123.855107, 13.237771],
            [124.181289, 12.997527],
            [124.077419, 12.536677],
            [123.298035, 13.027526],
            [122.928652, 13.55292],
            [122.671355, 13.185836],
            [122.03465, 13.784482],
            [121.126385, 13.636687],
            [120.628637, 13.857656],
            [120.679384, 14.271016],
            [120.991819, 14.525393],
            [120.693336, 14.756671],
            [120.564145, 14.396279],
            [120.070429, 14.970869],
            [119.920929, 15.406347],
            [119.883773, 16.363704],
            [120.286488, 16.034629],
            [120.390047, 17.599081],
            [120.715867, 18.505227],
            [121.321308, 18.504065],
          ],
        ],
      ],
    },
    id: "PHL",
  },
  {
    type: "Feature",
    properties: {
      name: "Papua New Guinea",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [155.880026, -6.819997],
            [155.599991, -6.919991],
            [155.166994, -6.535931],
            [154.729192, -5.900828],
            [154.514114, -5.139118],
            [154.652504, -5.042431],
            [154.759991, -5.339984],
            [155.062918, -5.566792],
            [155.547746, -6.200655],
            [156.019965, -6.540014],
            [155.880026, -6.819997],
          ],
        ],
        [
          [
            [151.982796, -5.478063],
            [151.459107, -5.56028],
            [151.30139, -5.840728],
            [150.754447, -6.083763],
            [150.241197, -6.317754],
            [149.709963, -6.316513],
            [148.890065, -6.02604],
            [148.318937, -5.747142],
            [148.401826, -5.437756],
            [149.298412, -5.583742],
            [149.845562, -5.505503],
```

```
        [149.99625, -5.026101],
        [150.139756, -5.001348],
        [150.236908, -5.53222],
        [150.807467, -5.455842],
        [151.089672, -5.113693],
        [151.647881, -4.757074],
        [151.537862, -4.167807],
        [152.136792, -4.14879],
        [152.338743, -4.312966],
        [152.318693, -4.867661],
        [151.982796, -5.478063],
      ],
    ],
    [
      [
        [147.191874, -7.388024],
        [148.084636, -8.044108],
        [148.734105, -9.104664],
        [149.306835, -9.071436],
        [149.266631, -9.514406],
        [150.038728, -9.684318],
        [149.738798, -9.872937],
        [150.801628, -10.293687],
        [150.690575, -10.582713],
        [150.028393, -10.652476],
        [149.78231, -10.393267],
        [148.923138, -10.280923],
        [147.913018, -10.130441],
        [147.135443, -9.492444],
        [146.567881, -8.942555],
        [146.048481, -8.067414],
        [144.744168, -7.630128],
        [143.897088, -7.91533],
        [143.286376, -8.245491],
        [143.413913, -8.983069],
        [142.628431, -9.326821],
        [142.068259, -9.159596],
        [141.033852, -9.117893],
        [141.017057, -5.859022],
        [141.00021, -2.600151],
        [142.735247, -3.289153],
        [144.583971, -3.861418],
        [145.27318, -4.373738],
        [145.829786, -4.876498],
        [145.981922, -5.465609],
        [147.648073, -6.083659],
        [147.891108, -6.614015],
        [146.970905, -6.721657],
        [147.191874, -7.388024],
      ],
    ],
    [
      [
        [153.140038, -4.499983],
        [152.827292, -4.766427],
        [152.638673, -4.176127],
        [152.406026, -3.789743],
        [151.953237, -3.462062],
        [151.384279, -3.035422],
        [150.66205, -2.741486],
        [150.939965, -2.500002],
        [151.479984, -2.779985],
        [151.820015, -2.999972],
        [152.239989, -3.240009],
        [152.640017, -3.659983],
        [153.019994, -3.980015],
```

```
            [153.140038, -4.499983],
          ],
        ],
      ],
    },
    id: "PNG",
  },
  {
    type: "Feature",
    properties: {
      name: "Poland",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [15.016996, 51.106674],
          [14.607098, 51.745188],
          [14.685026, 52.089947],
          [14.4376, 52.62485],
          [14.074521, 52.981263],
          [14.353315, 53.248171],
          [14.119686, 53.757029],
          [14.8029, 54.050706],
          [16.363477, 54.513159],
          [17.622832, 54.851536],
          [18.620859, 54.682606],
          [18.696255, 54.438719],
          [19.66064, 54.426084],
          [20.892245, 54.312525],
          [22.731099, 54.327537],
          [23.243987, 54.220567],
          [23.484128, 53.912498],
          [23.527536, 53.470122],
          [23.804935, 53.089731],
          [23.799199, 52.691099],
          [23.199494, 52.486977],
          [23.508002, 52.023647],
          [23.527071, 51.578454],
          [24.029986, 50.705407],
          [23.922757, 50.424881],
          [23.426508, 50.308506],
          [22.51845, 49.476774],
          [22.776419, 49.027395],
          [22.558138, 49.085738],
          [21.607808, 49.470107],
          [20.887955, 49.328772],
          [20.415839, 49.431453],
          [19.825023, 49.217125],
          [19.320713, 49.571574],
          [18.909575, 49.435846],
          [18.853144, 49.49623],
          [18.392914, 49.988629],
          [17.649445, 50.049038],
          [17.554567, 50.362146],
          [16.868769, 50.473974],
          [16.719476, 50.215747],
          [16.176253, 50.422607],
          [16.238627, 50.697733],
          [15.490972, 50.78473],
          [15.016996, 51.106674],
        ],
      ],
    },
    id: "POL",
  },
```

```
    {
      type: "Feature",
      properties: {
        name: "Puerto Rico",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-66.282434, 18.514762],
            [-65.771303, 18.426679],
            [-65.591004, 18.228035],
            [-65.847164, 17.975906],
            [-66.599934, 17.981823],
            [-67.184162, 17.946553],
            [-67.242428, 18.37446],
            [-67.100679, 18.520601],
            [-66.282434, 18.514762],
          ],
        ],
      },
      id: "PRI",
    },
    {
      type: "Feature",
      properties: {
        name: "Portugal",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-9.034818, 41.880571],
            [-8.671946, 42.134689],
            [-8.263857, 42.280469],
            [-8.013175, 41.790886],
            [-7.422513, 41.792075],
            [-7.251309, 41.918346],
            [-6.668606, 41.883387],
            [-6.389088, 41.381815],
            [-6.851127, 41.111083],
            [-6.86402, 40.330872],
            [-7.026413, 40.184524],
            [-7.066592, 39.711892],
            [-7.498632, 39.629571],
            [-7.098037, 39.030073],
            [-7.374092, 38.373059],
            [-7.029281, 38.075764],
            [-7.166508, 37.803894],
            [-7.537105, 37.428904],
            [-7.453726, 37.097788],
            [-7.855613, 36.838269],
            [-8.382816, 36.97888],
            [-8.898857, 36.868809],
            [-8.746101, 37.651346],
            [-8.839998, 38.266243],
            [-9.287464, 38.358486],
            [-9.526571, 38.737429],
            [-9.446989, 39.392066],
            [-9.048305, 39.755093],
            [-8.977353, 40.159306],
            [-8.768684, 40.760639],
            [-8.790853, 41.184334],
            [-8.990789, 41.543459],
            [-9.034818, 41.880571],
          ],
```

```
        ],
      },
      id: "PRT",
    },
    {
      type: "Feature",
      properties: {
        name: "Paraguay",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-62.685057, -22.249029],
            [-62.291179, -21.051635],
            [-62.265961, -20.513735],
            [-61.786326, -19.633737],
            [-60.043565, -19.342747],
            [-59.115042, -19.356906],
            [-58.183471, -19.868399],
            [-58.166392, -20.176701],
            [-57.870674, -20.732688],
            [-57.937156, -22.090176],
            [-56.88151, -22.282154],
            [-56.473317, -22.0863],
            [-55.797958, -22.35693],
            [-55.610683, -22.655619],
            [-55.517639, -23.571998],
            [-55.400747, -23.956935],
            [-55.027902, -24.001274],
            [-54.652834, -23.839578],
            [-54.29296, -24.021014],
            [-54.293476, -24.5708],
            [-54.428946, -25.162185],
            [-54.625291, -25.739255],
            [-54.788795, -26.621786],
            [-55.695846, -27.387837],
            [-56.486702, -27.548499],
            [-57.60976, -27.395899],
            [-58.618174, -27.123719],
            [-57.63366, -25.603657],
            [-57.777217, -25.16234],
            [-58.807128, -24.771459],
            [-60.028966, -24.032796],
            [-60.846565, -23.880713],
            [-62.685057, -22.249029],
          ],
        ],
      },
      id: "PRY",
    },
    {
      type: "Feature",
      properties: {
        name: "Qatar",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [50.810108, 24.754743],
            [50.743911, 25.482424],
            [51.013352, 26.006992],
            [51.286462, 26.114582],
            [51.589079, 25.801113],
            [51.6067, 25.21567],
```

```
          [51.389608, 24.627386],
          [51.112415, 24.556331],
          [50.810108, 24.754743],
        ],
      ],
    },
    id: "QAT",
  },
  {
    type: "Feature",
    properties: {
      name: "Romania",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [22.710531, 47.882194],
          [23.142236, 48.096341],
          [23.760958, 47.985598],
          [24.402056, 47.981878],
          [24.866317, 47.737526],
          [25.207743, 47.891056],
          [25.945941, 47.987149],
          [26.19745, 48.220881],
          [26.619337, 48.220726],
          [26.924176, 48.123264],
          [27.233873, 47.826771],
          [27.551166, 47.405117],
          [28.12803, 46.810476],
          [28.160018, 46.371563],
          [28.054443, 45.944586],
          [28.233554, 45.488283],
          [28.679779, 45.304031],
          [29.149725, 45.464925],
          [29.603289, 45.293308],
          [29.626543, 45.035391],
          [29.141612, 44.82021],
          [28.837858, 44.913874],
          [28.558081, 43.707462],
          [27.970107, 43.812468],
          [27.2424, 44.175986],
          [26.065159, 43.943494],
          [25.569272, 43.688445],
          [24.100679, 43.741051],
          [23.332302, 43.897011],
          [22.944832, 43.823785],
          [22.65715, 44.234923],
          [22.474008, 44.409228],
          [22.705726, 44.578003],
          [22.459022, 44.702517],
          [22.145088, 44.478422],
          [21.562023, 44.768947],
          [21.483526, 45.18117],
          [20.874313, 45.416375],
          [20.762175, 45.734573],
          [20.220192, 46.127469],
          [21.021952, 46.316088],
          [21.626515, 46.994238],
          [22.099768, 47.672439],
          [22.710531, 47.882194],
        ],
      ],
    },
    id: "ROU",
  },
```

```
{
  type: "Feature",
  properties: {
    name: "Russia",
  },
  geometry: {
    type: "MultiPolygon",
    coordinates: [
      [
        [
          [143.648007, 50.7476],
          [144.654148, 48.976391],
          [143.173928, 49.306551],
          [142.558668, 47.861575],
          [143.533492, 46.836728],
          [143.505277, 46.137908],
          [142.747701, 46.740765],
          [142.09203, 45.966755],
          [141.906925, 46.805929],
          [142.018443, 47.780133],
          [141.904445, 48.859189],
          [142.1358, 49.615163],
          [142.179983, 50.952342],
          [141.594076, 51.935435],
          [141.682546, 53.301966],
          [142.606934, 53.762145],
          [142.209749, 54.225476],
          [142.654786, 54.365881],
          [142.914616, 53.704578],
          [143.260848, 52.74076],
          [143.235268, 51.75666],
          [143.648007, 50.7476],
        ],
      ],
      [
        [
          [22.731099, 54.327537],
          [20.892245, 54.312525],
          [19.66064, 54.426084],
          [19.888481, 54.86616],
          [21.268449, 55.190482],
          [22.315724, 55.015299],
          [22.757764, 54.856574],
          [22.651052, 54.582741],
          [22.731099, 54.327537],
        ],
      ],
      [
        [
          [-175.01425, 66.58435],
          [-174.33983, 66.33556],
          [-174.57182, 67.06219],
          [-171.85731, 66.91308],
          [-169.89958, 65.97724],
          [-170.89107, 65.54139],
          [-172.53025, 65.43791],
          [-172.555, 64.46079],
          [-172.95533, 64.25269],
          [-173.89184, 64.2826],
          [-174.65392, 64.63125],
          [-175.98353, 64.92288],
          [-176.20716, 65.35667],
          [-177.22266, 65.52024],
          [-178.35993, 65.39052],
          [-178.90332, 65.74044],
          [-178.68611, 66.11211],
```

```
        [-179.88377, 65.87456],
        [-179.43268, 65.40411],
        [-180, 64.979709],
        [-180, 68.963636],
        [-177.55, 68.2],
        [-174.92825, 67.20589],
        [-175.01425, 66.58435],
      ],
    ],
    [
      [
        [180, 70.832199],
        [178.903425, 70.78114],
        [178.7253, 71.0988],
        [180, 71.515714],
        [180, 70.832199],
      ],
    ],
    [
      [
        [-178.69378, 70.89302],
        [-180, 70.832199],
        [-180, 71.515714],
        [-179.871875, 71.55762],
        [-179.02433, 71.55553],
        [-177.577945, 71.26948],
        [-177.663575, 71.13277],
        [-178.69378, 70.89302],
      ],
    ],
    [
      [
        [143.60385, 73.21244],
        [142.08763, 73.20544],
        [140.038155, 73.31692],
        [139.86312, 73.36983],
        [140.81171, 73.76506],
        [142.06207, 73.85758],
        [143.48283, 73.47525],
        [143.60385, 73.21244],
      ],
    ],
    [
      [
        [150.73167, 75.08406],
        [149.575925, 74.68892],
        [147.977465, 74.778355],
        [146.11919, 75.17298],
        [146.358485, 75.49682],
        [148.22223, 75.345845],
        [150.73167, 75.08406],
      ],
    ],
    [
      [
        [145.086285, 75.562625],
        [144.3, 74.82],
        [140.61381, 74.84768],
        [138.95544, 74.61148],
        [136.97439, 75.26167],
        [137.51176, 75.94917],
        [138.831075, 76.13676],
        [141.471615, 76.09289],
        [145.086285, 75.562625],
      ],
    ],
```

```
[
  [
    [57.535693, 70.720464],
    [56.944979, 70.632743],
    [53.677375, 70.762658],
    [53.412017, 71.206662],
    [51.601895, 71.474759],
    [51.455754, 72.014881],
    [52.478275, 72.229442],
    [52.444169, 72.774731],
    [54.427614, 73.627548],
    [53.50829, 73.749814],
    [55.902459, 74.627486],
    [55.631933, 75.081412],
    [57.868644, 75.60939],
    [61.170044, 76.251883],
    [64.498368, 76.439055],
    [66.210977, 76.809782],
    [68.15706, 76.939697],
    [68.852211, 76.544811],
    [68.180573, 76.233642],
    [64.637326, 75.737755],
    [61.583508, 75.260885],
    [58.477082, 74.309056],
    [56.986786, 73.333044],
    [55.419336, 72.371268],
    [55.622838, 71.540595],
    [57.535693, 70.720464],
  ],
],
[
  [
    [106.97013, 76.97419],
    [107.24, 76.48],
    [108.1538, 76.72335],
    [111.07726, 76.71],
    [113.33151, 76.22224],
    [114.13417, 75.84764],
    [113.88539, 75.32779],
    [112.77918, 75.03186],
    [110.15125, 74.47673],
    [109.4, 74.18],
    [110.64, 74.04],
    [112.11919, 73.78774],
    [113.01954, 73.97693],
    [113.52958, 73.33505],
    [113.96881, 73.59488],
    [115.56782, 73.75285],
    [118.77633, 73.58772],
    [119.02, 73.12],
    [123.20066, 72.97122],
    [123.25777, 73.73503],
    [125.38, 73.56],
    [126.97644, 73.56549],
    [128.59126, 73.03871],
    [129.05157, 72.39872],
    [128.46, 71.98],
    [129.71599, 71.19304],
    [131.28858, 70.78699],
    [132.2535, 71.8363],
    [133.85766, 71.38642],
    [135.56193, 71.65525],
    [137.49755, 71.34763],
    [138.23409, 71.62803],
    [139.86983, 71.48783],
    [139.14791, 72.41619],
```

```
            [140.46817, 72.84941],
            [149.5, 72.2],
            [150.35118, 71.60643],
            [152.9689, 70.84222],
            [157.00688, 71.03141],
            [158.99779, 70.86672],
            [159.83031, 70.45324],
            [159.70866, 69.72198],
            [160.94053, 69.43728],
            [162.27907, 69.64204],
            [164.05248, 69.66823],
            [165.94037, 69.47199],
            [167.83567, 69.58269],
            [169.57763, 68.6938],
            [170.81688, 69.01363],
            [170.0082, 69.65276],
            [170.45345, 70.09703],
            [173.64391, 69.81743],
            [175.72403, 69.87725],
            [178.6, 69.4],
            [180, 68.963636],
            [180, 64.979709],
            [179.99281, 64.97433],
            [178.7072, 64.53493],
            [177.41128, 64.60821],
            [178.313, 64.07593],
            [178.90825, 63.25197],
            [179.37034, 62.98262],
            [179.48636, 62.56894],
            [179.22825, 62.3041],
            [177.3643, 62.5219],
            [174.56929, 61.76915],
            [173.68013, 61.65261],
            [172.15, 60.95],
            [170.6985, 60.33618],
            [170.33085, 59.88177],
            [168.90046, 60.57355],
            [166.29498, 59.78855],
            [165.84, 60.16],
            [164.87674, 59.7316],
            [163.53929, 59.86871],
            [163.21711, 59.21101],
            [162.01733, 58.24328],
            [162.05297, 57.83912],
            [163.19191, 57.61503],
            [163.05794, 56.15924],
            [162.12958, 56.12219],
            [161.70146, 55.28568],
            [162.11749, 54.85514],
            [160.36877, 54.34433],
            [160.02173, 53.20257],
            [158.53094, 52.95868],
            [158.23118, 51.94269],
            [156.78979, 51.01105],
            [156.42, 51.7],
            [155.99182, 53.15895],
            [155.43366, 55.38103],
            [155.91442, 56.76792],
            [156.75815, 57.3647],
            [156.81035, 57.83204],
            [158.36433, 58.05575],
            [160.15064, 59.31477],
            [161.87204, 60.343],
            [163.66969, 61.1409],
            [164.47355, 62.55061],
            [163.25842, 62.46627],
```

```
[162.65791, 61.6425],
[160.12148, 60.54423],
[159.30232, 61.77396],
[156.72068, 61.43442],
[154.21806, 59.75818],
[155.04375, 59.14495],
[152.81185, 58.88385],
[151.26573, 58.78089],
[151.33815, 59.50396],
[149.78371, 59.65573],
[148.54481, 59.16448],
[145.48722, 59.33637],
[142.19782, 59.03998],
[138.95848, 57.08805],
[135.12619, 54.72959],
[136.70171, 54.60355],
[137.19342, 53.97732],
[138.1647, 53.75501],
[138.80463, 54.25455],
[139.90151, 54.18968],
[141.34531, 53.08957],
[141.37923, 52.23877],
[140.59742, 51.23967],
[140.51308, 50.04553],
[140.06193, 48.44671],
[138.55472, 46.99965],
[138.21971, 46.30795],
[136.86232, 45.1435],
[135.51535, 43.989],
[134.86939, 43.39821],
[133.53687, 42.81147],
[132.90627, 42.79849],
[132.27807, 43.28456],
[130.93587, 42.55274],
[130.78, 42.22],
[130.64, 42.395],
[130.633866, 42.903015],
[131.144688, 42.92999],
[131.288555, 44.11152],
[131.02519, 44.96796],
[131.883454, 45.321162],
[133.09712, 45.14409],
[133.769644, 46.116927],
[134.11235, 47.21248],
[134.50081, 47.57845],
[135.026311, 48.47823],
[133.373596, 48.183442],
[132.50669, 47.78896],
[130.98726, 47.79013],
[130.582293, 48.729687],
[129.397818, 49.4406],
[127.6574, 49.76027],
[127.287456, 50.739797],
[126.939157, 51.353894],
[126.564399, 51.784255],
[125.946349, 52.792799],
[125.068211, 53.161045],
[123.57147, 53.4588],
[122.245748, 53.431726],
[121.003085, 53.251401],
[120.177089, 52.753886],
[120.725789, 52.516226],
[120.7382, 51.96411],
[120.18208, 51.64355],
[119.27939, 50.58292],
[119.288461, 50.142883],
```

```
            [117.879244, 49.510983],
            [116.678801, 49.888531],
            [115.485695, 49.805177],
            [114.96211, 50.140247],
            [114.362456, 50.248303],
            [112.89774, 49.543565],
            [111.581231, 49.377968],
            [110.662011, 49.130128],
            [109.402449, 49.292961],
            [108.475167, 49.282548],
            [107.868176, 49.793705],
            [106.888804, 50.274296],
            [105.886591, 50.406019],
            [104.62158, 50.27532],
            [103.676545, 50.089966],
            [102.25589, 50.51056],
            [102.06521, 51.25991],
            [100.88948, 51.516856],
            [99.981732, 51.634006],
            [98.861491, 52.047366],
            [97.82574, 51.010995],
            [98.231762, 50.422401],
            [97.25976, 49.72605],
            [95.81402, 49.97746],
            [94.815949, 50.013433],
            [94.147566, 50.480537],
            [93.10421, 50.49529],
            [92.234712, 50.802171],
            [90.713667, 50.331812],
            [88.805567, 49.470521],
            [87.751264, 49.297198],
            [87.35997, 49.214981],
            [86.829357, 49.826675],
            [85.54127, 49.692859],
            [85.11556, 50.117303],
            [84.416377, 50.3114],
            [83.935115, 50.889246],
            [83.383004, 51.069183],
            [81.945986, 50.812196],
            [80.568447, 51.388336],
            [80.03556, 50.864751],
            [77.800916, 53.404415],
            [76.525179, 54.177003],
            [76.8911, 54.490524],
            [74.38482, 53.54685],
            [73.425679, 53.48981],
            [73.508516, 54.035617],
            [72.22415, 54.376655],
            [71.180131, 54.133285],
            [70.865267, 55.169734],
            [69.068167, 55.38525],
            [68.1691, 54.970392],
            [65.66687, 54.60125],
            [65.178534, 54.354228],
            [61.4366, 54.00625],
            [60.978066, 53.664993],
            [61.699986, 52.979996],
            [60.739993, 52.719986],
            [60.927269, 52.447548],
            [59.967534, 51.96042],
            [61.588003, 51.272659],
            [61.337424, 50.79907],
            [59.932807, 50.842194],
            [59.642282, 50.545442],
            [58.36332, 51.06364],
            [56.77798, 51.04355],
```

```
            [55.71694, 50.62171],
            [54.532878, 51.02624],
            [52.328724, 51.718652],
            [50.766648, 51.692762],
            [48.702382, 50.605128],
            [48.577841, 49.87476],
            [47.54948, 50.454698],
            [46.751596, 49.356006],
            [47.043672, 49.152039],
            [46.466446, 48.394152],
            [47.31524, 47.71585],
            [48.05725, 47.74377],
            [48.694734, 47.075628],
            [48.59325, 46.56104],
            [49.10116, 46.39933],
            [48.64541, 45.80629],
            [47.67591, 45.64149],
            [46.68201, 44.6092],
            [47.59094, 43.66016],
            [47.49252, 42.98658],
            [48.58437, 41.80888],
            [47.987283, 41.405819],
            [47.815666, 41.151416],
            [47.373315, 41.219732],
            [46.686071, 41.827137],
            [46.404951, 41.860675],
            [45.7764, 42.09244],
            [45.470279, 42.502781],
            [44.537623, 42.711993],
            [43.93121, 42.55496],
            [43.75599, 42.74083],
            [42.3944, 43.2203],
            [40.92219, 43.38215],
            [40.076965, 43.553104],
            [39.955009, 43.434998],
            [38.68, 44.28],
            [37.53912, 44.65721],
            [36.67546, 45.24469],
            [37.40317, 45.40451],
            [38.23295, 46.24087],
            [37.67372, 46.63657],
            [39.14767, 47.04475],
            [39.1212, 47.26336],
            [38.223538, 47.10219],
            [38.255112, 47.5464],
            [38.77057, 47.82562],
            [39.738278, 47.898937],
            [39.89562, 48.23241],
            [39.67465, 48.78382],
            [40.080789, 49.30743],
            [40.06904, 49.60105],
            [38.594988, 49.926462],
            [38.010631, 49.915662],
            [37.39346, 50.383953],
            [36.626168, 50.225591],
            [35.356116, 50.577197],
            [35.37791, 50.77394],
            [35.022183, 51.207572],
            [34.224816, 51.255993],
            [34.141978, 51.566413],
            [34.391731, 51.768882],
            [33.7527, 52.335075],
            [32.715761, 52.238465],
            [32.412058, 52.288695],
            [32.15944, 52.06125],
            [31.78597, 52.10168],
```

```
[31.540018, 52.742052],
[31.305201, 53.073996],
[31.49764, 53.16743],
[32.304519, 53.132726],
[32.693643, 53.351421],
[32.405599, 53.618045],
[31.731273, 53.794029],
[31.791424, 53.974639],
[31.384472, 54.157056],
[30.757534, 54.811771],
[30.971836, 55.081548],
[30.873909, 55.550976],
[29.896294, 55.789463],
[29.371572, 55.670091],
[29.229513, 55.918344],
[28.176709, 56.16913],
[27.855282, 56.759326],
[27.770016, 57.244258],
[27.288185, 57.474528],
[27.716686, 57.791899],
[27.42015, 58.72457],
[28.131699, 59.300825],
[27.98112, 59.47537],
[29.1177, 60.02805],
[28.07, 60.50352],
[30.211107, 61.780028],
[31.139991, 62.357693],
[31.516092, 62.867687],
[30.035872, 63.552814],
[30.444685, 64.204453],
[29.54443, 64.948672],
[30.21765, 65.80598],
[29.054589, 66.944286],
[29.977426, 67.698297],
[28.445944, 68.364613],
[28.59193, 69.064777],
[29.39955, 69.15692],
[31.10108, 69.55811],
[32.13272, 69.90595],
[33.77547, 69.30142],
[36.51396, 69.06342],
[40.29234, 67.9324],
[41.05987, 67.45713],
[41.12595, 66.79158],
[40.01583, 66.26618],
[38.38295, 65.99953],
[33.91871, 66.75961],
[33.18444, 66.63253],
[34.81477, 65.90015],
[34.878574, 65.436213],
[34.94391, 64.41437],
[36.23129, 64.10945],
[37.01273, 63.84983],
[37.14197, 64.33471],
[36.539579, 64.76446],
[37.17604, 65.14322],
[39.59345, 64.52079],
[40.4356, 64.76446],
[39.7626, 65.49682],
[42.09309, 66.47623],
[43.01604, 66.41858],
[43.94975, 66.06908],
[44.53226, 66.75634],
[43.69839, 67.35245],
[44.18795, 67.95051],
[43.45282, 68.57079],
```

```
            [46.25, 68.25],
            [46.82134, 67.68997],
            [45.55517, 67.56652],
            [45.56202, 67.01005],
            [46.34915, 66.66767],
            [47.89416, 66.88455],
            [48.13876, 67.52238],
            [50.22766, 67.99867],
            [53.71743, 68.85738],
            [54.47171, 68.80815],
            [53.48582, 68.20131],
            [54.72628, 68.09702],
            [55.44268, 68.43866],
            [57.31702, 68.46628],
            [58.802, 68.88082],
            [59.94142, 68.27844],
            [61.07784, 68.94069],
            [60.03, 69.52],
            [60.55, 69.85],
            [63.504, 69.54739],
            [64.888115, 69.234835],
            [68.51216, 68.09233],
            [69.18068, 68.61563],
            [68.16444, 69.14436],
            [68.13522, 69.35649],
            [66.93008, 69.45461],
            [67.25976, 69.92873],
            [66.72492, 70.70889],
            [66.69466, 71.02897],
            [68.54006, 71.9345],
            [69.19636, 72.84336],
            [69.94, 73.04],
            [72.58754, 72.77629],
            [72.79603, 72.22006],
            [71.84811, 71.40898],
            [72.47011, 71.09019],
            [72.79188, 70.39114],
            [72.5647, 69.02085],
            [73.66787, 68.4079],
            [73.2387, 67.7404],
            [71.28, 66.32],
            [72.42301, 66.17267],
            [72.82077, 66.53267],
            [73.92099, 66.78946],
            [74.18651, 67.28429],
            [75.052, 67.76047],
            [74.46926, 68.32899],
            [74.93584, 68.98918],
            [73.84236, 69.07146],
            [73.60187, 69.62763],
            [74.3998, 70.63175],
            [73.1011, 71.44717],
            [74.89082, 72.12119],
            [74.65926, 72.83227],
            [75.15801, 72.85497],
            [75.68351, 72.30056],
            [75.28898, 71.33556],
            [76.35911, 71.15287],
            [75.90313, 71.87401],
            [77.57665, 72.26717],
            [79.65202, 72.32011],
            [81.5, 71.75],
            [80.61071, 72.58285],
            [80.51109, 73.6482],
            [82.25, 73.85],
            [84.65526, 73.80591],
```

```
            [86.8223, 73.93688],
            [86.00956, 74.45967],
            [87.16682, 75.11643],
            [88.31571, 75.14393],
            [90.26, 75.64],
            [92.90058, 75.77333],
            [93.23421, 76.0472],
            [95.86, 76.14],
            [96.67821, 75.91548],
            [98.92254, 76.44689],
            [100.75967, 76.43028],
            [101.03532, 76.86189],
            [101.99084, 77.28754],
            [104.3516, 77.69792],
            [106.06664, 77.37389],
            [104.705, 77.1274],
            [106.97013, 76.97419],
          ],
        ],
        [
          [
            [105.07547, 78.30689],
            [99.43814, 77.921],
            [101.2649, 79.23399],
            [102.08635, 79.34641],
            [102.837815, 79.28129],
            [105.37243, 78.71334],
            [105.07547, 78.30689],
          ],
        ],
        [
          [
            [51.136187, 80.54728],
            [49.793685, 80.415428],
            [48.894411, 80.339567],
            [48.754937, 80.175468],
            [47.586119, 80.010181],
            [46.502826, 80.247247],
            [47.072455, 80.559424],
            [44.846958, 80.58981],
            [46.799139, 80.771918],
            [48.318477, 80.78401],
            [48.522806, 80.514569],
            [49.09719, 80.753986],
            [50.039768, 80.918885],
            [51.522933, 80.699726],
            [51.136187, 80.54728],
          ],
        ],
        [
          [
            [99.93976, 78.88094],
            [97.75794, 78.7562],
            [94.97259, 79.044745],
            [93.31288, 79.4265],
            [92.5454, 80.14379],
            [91.18107, 80.34146],
            [93.77766, 81.0246],
            [95.940895, 81.2504],
            [97.88385, 80.746975],
            [100.186655, 79.780135],
            [99.93976, 78.88094],
          ],
        ],
      ],
    },
```

```
      id: "RUS",
    },
    {
      type: "Feature",
      properties: {
        name: "Rwanda",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [30.419105, -1.134659],
            [30.816135, -1.698914],
            [30.758309, -2.28725],
            [30.469696, -2.413858],
            [29.938359, -2.348487],
            [29.632176, -2.917858],
            [29.024926, -2.839258],
            [29.117479, -2.292211],
            [29.254835, -2.21511],
            [29.291887, -1.620056],
            [29.579466, -1.341313],
            [29.821519, -1.443322],
            [30.419105, -1.134659],
          ],
        ],
      },
      id: "RWA",
    },
    {
      type: "Feature",
      properties: {
        name: "Western Sahara",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-8.794884, 27.120696],
            [-8.817828, 27.656426],
            [-8.66559, 27.656426],
            [-8.665124, 27.589479],
            [-8.6844, 27.395744],
            [-8.687294, 25.881056],
            [-11.969419, 25.933353],
            [-11.937224, 23.374594],
            [-12.874222, 23.284832],
            [-13.118754, 22.77122],
            [-12.929102, 21.327071],
            [-16.845194, 21.333323],
            [-17.063423, 20.999752],
            [-17.020428, 21.42231],
            [-17.002962, 21.420734],
            [-14.750955, 21.5006],
            [-14.630833, 21.86094],
            [-14.221168, 22.310163],
            [-13.89111, 23.691009],
            [-12.500963, 24.770116],
            [-12.030759, 26.030866],
            [-11.71822, 26.104092],
            [-11.392555, 26.883424],
            [-10.551263, 26.990808],
            [-10.189424, 26.860945],
            [-9.735343, 26.860945],
            [-9.413037, 27.088476],
            [-8.794884, 27.120696],
```

```
        ],
      ],
    },
    id: "ESH",
  },
  {
    type: "Feature",
    properties: {
      name: "Saudi Arabia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [42.779332, 16.347891],
          [42.649573, 16.774635],
          [42.347989, 17.075806],
          [42.270888, 17.474722],
          [41.754382, 17.833046],
          [41.221391, 18.6716],
          [40.939341, 19.486485],
          [40.247652, 20.174635],
          [39.801685, 20.338862],
          [39.139399, 21.291905],
          [39.023696, 21.986875],
          [39.066329, 22.579656],
          [38.492772, 23.688451],
          [38.02386, 24.078686],
          [37.483635, 24.285495],
          [37.154818, 24.858483],
          [37.209491, 25.084542],
          [36.931627, 25.602959],
          [36.639604, 25.826228],
          [36.249137, 26.570136],
          [35.640182, 27.37652],
          [35.130187, 28.063352],
          [34.632336, 28.058546],
          [34.787779, 28.607427],
          [34.83222, 28.957483],
          [34.956037, 29.356555],
          [36.068941, 29.197495],
          [36.501214, 29.505254],
          [36.740528, 29.865283],
          [37.503582, 30.003776],
          [37.66812, 30.338665],
          [37.998849, 30.5085],
          [37.002166, 31.508413],
          [39.004886, 32.010217],
          [39.195468, 32.161009],
          [40.399994, 31.889992],
          [41.889981, 31.190009],
          [44.709499, 29.178891],
          [46.568713, 29.099025],
          [47.459822, 29.002519],
          [47.708851, 28.526063],
          [48.416094, 28.552004],
          [48.807595, 27.689628],
          [49.299554, 27.461218],
          [49.470914, 27.109999],
          [50.152422, 26.689663],
          [50.212935, 26.277027],
          [50.113303, 25.943972],
          [50.239859, 25.60805],
          [50.527387, 25.327808],
          [50.660557, 24.999896],
          [50.810108, 24.754743],
```

```
          [51.112415, 24.556331],
          [51.389608, 24.627386],
          [51.579519, 24.245497],
          [51.617708, 24.014219],
          [52.000733, 23.001154],
          [55.006803, 22.496948],
          [55.208341, 22.70833],
          [55.666659, 22.000001],
          [54.999982, 19.999994],
          [52.00001, 19.000003],
          [49.116672, 18.616668],
          [48.183344, 18.166669],
          [47.466695, 17.116682],
          [47.000005, 16.949999],
          [46.749994, 17.283338],
          [46.366659, 17.233315],
          [45.399999, 17.333335],
          [45.216651, 17.433329],
          [44.062613, 17.410359],
          [43.791519, 17.319977],
          [43.380794, 17.579987],
          [43.115798, 17.08844],
          [43.218375, 16.66689],
          [42.779332, 16.347891],
        ],
      ],
    },
    id: "SAU",
  },
  {
    type: "Feature",
    properties: {
      name: "Sudan",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [33.963393, 9.464285],
          [33.824963, 9.484061],
          [33.842131, 9.981915],
          [33.721959, 10.325262],
          [33.206938, 10.720112],
          [33.086766, 11.441141],
          [33.206938, 12.179338],
          [32.743419, 12.248008],
          [32.67475, 12.024832],
          [32.073892, 11.97333],
          [32.314235, 11.681484],
          [32.400072, 11.080626],
          [31.850716, 10.531271],
          [31.352862, 9.810241],
          [30.837841, 9.707237],
          [29.996639, 10.290927],
          [29.618957, 10.084919],
          [29.515953, 9.793074],
          [29.000932, 9.604232],
          [28.966597, 9.398224],
          [27.97089, 9.398224],
          [27.833551, 9.604232],
          [27.112521, 9.638567],
          [26.752006, 9.466893],
          [26.477328, 9.55273],
          [25.962307, 10.136421],
          [25.790633, 10.411099],
          [25.069604, 10.27376],
```

```
          [24.794926, 9.810241],
          [24.537415, 8.917538],
          [24.194068, 8.728696],
          [23.88698, 8.61973],
          [23.805813, 8.666319],
          [23.459013, 8.954286],
          [23.394779, 9.265068],
          [23.55725, 9.681218],
          [23.554304, 10.089255],
          [22.977544, 10.714463],
          [22.864165, 11.142395],
          [22.87622, 11.38461],
          [22.50869, 11.67936],
          [22.49762, 12.26024],
          [22.28801, 12.64605],
          [21.93681, 12.58818],
          [22.03759, 12.95546],
          [22.29658, 13.37232],
          [22.18329, 13.78648],
          [22.51202, 14.09318],
          [22.30351, 14.32682],
          [22.56795, 14.94429],
          [23.02459, 15.68072],
          [23.88689, 15.61084],
          [23.83766, 19.58047],
          [23.85, 20],
          [25, 20.00304],
          [25, 22],
          [29.02, 22],
          [32.9, 22],
          [36.86623, 22],
          [37.18872, 21.01885],
          [36.96941, 20.83744],
          [37.1147, 19.80796],
          [37.48179, 18.61409],
          [37.86276, 18.36786],
          [38.41009, 17.998307],
          [37.904, 17.42754],
          [37.16747, 17.26314],
          [36.85253, 16.95655],
          [36.75389, 16.29186],
          [36.32322, 14.82249],
          [36.42951, 14.42211],
          [36.27022, 13.56333],
          [35.86363, 12.57828],
          [35.26049, 12.08286],
          [34.83163, 11.31896],
          [34.73115, 10.91017],
          [34.25745, 10.63009],
          [33.96162, 9.58358],
          [33.963393, 9.464285],
        ],
      ],
    },
    id: "SDN",
  },
  {
    type: "Feature",
    properties: {
      name: "South Sudan",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [33.963393, 9.464285],
```

```
          [33.97498, 8.68456],
          [33.8255, 8.37916],
          [33.2948, 8.35458],
          [32.95418, 7.78497],
          [33.56829, 7.71334],
          [34.0751, 7.22595],
          [34.25032, 6.82607],
          [34.70702, 6.59422],
          [35.298007, 5.506],
          [34.620196, 4.847123],
          [34.005, 4.249885],
          [33.39, 3.79],
          [32.68642, 3.79232],
          [31.88145, 3.55827],
          [31.24556, 3.7819],
          [30.83385, 3.50917],
          [29.95349, 4.1737],
          [29.715995, 4.600805],
          [29.159078, 4.389267],
          [28.696678, 4.455077],
          [28.428994, 4.287155],
          [27.979977, 4.408413],
          [27.374226, 5.233944],
          [27.213409, 5.550953],
          [26.465909, 5.946717],
          [26.213418, 6.546603],
          [25.796648, 6.979316],
          [25.124131, 7.500085],
          [25.114932, 7.825104],
          [24.567369, 8.229188],
          [23.88698, 8.61973],
          [24.194068, 8.728696],
          [24.537415, 8.917538],
          [24.794926, 9.810241],
          [25.069604, 10.27376],
          [25.790633, 10.411099],
          [25.962307, 10.136421],
          [26.477328, 9.55273],
          [26.752006, 9.466893],
          [27.112521, 9.638567],
          [27.833551, 9.604232],
          [27.97089, 9.398224],
          [28.966597, 9.398224],
          [29.000932, 9.604232],
          [29.515953, 9.793074],
          [29.618957, 10.084919],
          [29.996639, 10.290927],
          [30.837841, 9.707237],
          [31.352862, 9.810241],
          [31.850716, 10.531271],
          [32.400072, 11.080626],
          [32.314235, 11.681484],
          [32.073892, 11.97333],
          [32.67475, 12.024832],
          [32.743419, 12.248008],
          [33.206938, 12.179338],
          [33.086766, 11.441141],
          [33.206938, 10.720112],
          [33.721959, 10.325262],
          [33.842131, 9.981915],
          [33.824963, 9.484061],
          [33.963393, 9.464285],
        ],
      ],
    },
    id: "SDS",
```

```
    },
    {
      type: "Feature",
      properties: {
        name: "Senegal",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-16.713729, 13.594959],
            [-17.126107, 14.373516],
            [-17.625043, 14.729541],
            [-17.185173, 14.919477],
            [-16.700706, 15.621527],
            [-16.463098, 16.135036],
            [-16.12069, 16.455663],
            [-15.623666, 16.369337],
            [-15.135737, 16.587282],
            [-14.577348, 16.598264],
            [-14.099521, 16.304302],
            [-13.435738, 16.039383],
            [-12.830658, 15.303692],
            [-12.17075, 14.616834],
            [-12.124887, 13.994727],
            [-11.927716, 13.422075],
            [-11.553398, 13.141214],
            [-11.467899, 12.754519],
            [-11.513943, 12.442988],
            [-11.658301, 12.386583],
            [-12.203565, 12.465648],
            [-12.278599, 12.35444],
            [-12.499051, 12.33209],
            [-13.217818, 12.575874],
            [-13.700476, 12.586183],
            [-15.548477, 12.62817],
            [-15.816574, 12.515567],
            [-16.147717, 12.547762],
            [-16.677452, 12.384852],
            [-16.841525, 13.151394],
            [-15.931296, 13.130284],
            [-15.691001, 13.270353],
            [-15.511813, 13.27857],
            [-15.141163, 13.509512],
            [-14.712197, 13.298207],
            [-14.277702, 13.280585],
            [-13.844963, 13.505042],
            [-14.046992, 13.794068],
            [-14.376714, 13.62568],
            [-14.687031, 13.630357],
            [-15.081735, 13.876492],
            [-15.39877, 13.860369],
            [-15.624596, 13.623587],
            [-16.713729, 13.594959],
          ],
        ],
      },
      id: "SEN",
    },
    {
      type: "Feature",
      properties: {
        name: "Solomon Islands",
      },
      geometry: {
        type: "MultiPolygon",
```

```
      coordinates: [
        [
          [
            [162.119025, -10.482719],
            [162.398646, -10.826367],
            [161.700032, -10.820011],
            [161.319797, -10.204751],
            [161.917383, -10.446701],
            [162.119025, -10.482719],
          ],
        ],
        [
          [
            [160.852229, -9.872937],
            [160.462588, -9.89521],
            [159.849447, -9.794027],
            [159.640003, -9.63998],
            [159.702945, -9.24295],
            [160.362956, -9.400304],
            [160.688518, -9.610162],
            [160.852229, -9.872937],
          ],
        ],
        [
          [
            [161.679982, -9.599982],
            [161.529397, -9.784312],
            [160.788253, -8.917543],
            [160.579997, -8.320009],
            [160.920028, -8.320009],
            [161.280006, -9.120011],
            [161.679982, -9.599982],
          ],
        ],
        [
          [
            [159.875027, -8.33732],
            [159.917402, -8.53829],
            [159.133677, -8.114181],
            [158.586114, -7.754824],
            [158.21115, -7.421872],
            [158.359978, -7.320018],
            [158.820001, -7.560003],
            [159.640003, -8.020027],
            [159.875027, -8.33732],
          ],
        ],
        [
          [
            [157.538426, -7.34782],
            [157.33942, -7.404767],
            [156.90203, -7.176874],
            [156.491358, -6.765943],
            [156.542828, -6.599338],
            [157.14, -7.021638],
            [157.538426, -7.34782],
          ],
        ],
      ],
    },
    id: "SLB",
  },
  {
    type: "Feature",
    properties: {
      name: "Sierra Leone",
```

```
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-11.438779, 6.785917],
            [-11.708195, 6.860098],
            [-12.428099, 7.262942],
            [-12.949049, 7.798646],
            [-13.124025, 8.163946],
            [-13.24655, 8.903049],
            [-12.711958, 9.342712],
            [-12.596719, 9.620188],
            [-12.425929, 9.835834],
            [-12.150338, 9.858572],
            [-11.917277, 10.046984],
            [-11.117481, 10.045873],
            [-10.839152, 9.688246],
            [-10.622395, 9.26791],
            [-10.65477, 8.977178],
            [-10.494315, 8.715541],
            [-10.505477, 8.348896],
            [-10.230094, 8.406206],
            [-10.695595, 7.939464],
            [-11.146704, 7.396706],
            [-11.199802, 7.105846],
            [-11.438779, 6.785917],
          ],
        ],
      },
      id: "SLE",
    },
    {
      type: "Feature",
      properties: {
        name: "El Salvador",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [-87.793111, 13.38448],
            [-87.904112, 13.149017],
            [-88.483302, 13.163951],
            [-88.843228, 13.259734],
            [-89.256743, 13.458533],
            [-89.812394, 13.520622],
            [-90.095555, 13.735338],
            [-90.064678, 13.88197],
            [-89.721934, 14.134228],
            [-89.534219, 14.244816],
            [-89.587343, 14.362586],
            [-89.353326, 14.424133],
            [-89.058512, 14.340029],
            [-88.843073, 14.140507],
            [-88.541231, 13.980155],
            [-88.503998, 13.845486],
            [-88.065343, 13.964626],
            [-87.859515, 13.893312],
            [-87.723503, 13.78505],
            [-87.793111, 13.38448],
          ],
        ],
      },
      id: "SLV",
    },
```

```
    {
      type: "Feature",
      properties: {
        name: "Somaliland",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [48.93813, 9.451749],
            [48.486736, 8.837626],
            [47.78942, 8.003],
            [46.948328, 7.996877],
            [43.67875, 9.18358],
            [43.296975, 9.540477],
            [42.92812, 10.02194],
            [42.55876, 10.57258],
            [42.776852, 10.926879],
            [43.145305, 11.46204],
            [43.47066, 11.27771],
            [43.666668, 10.864169],
            [44.117804, 10.445538],
            [44.614259, 10.442205],
            [45.556941, 10.698029],
            [46.645401, 10.816549],
            [47.525658, 11.127228],
            [48.021596, 11.193064],
            [48.378784, 11.375482],
            [48.948206, 11.410622],
            [48.942005, 11.394266],
            [48.938491, 10.982327],
            [48.938233, 9.9735],
            [48.93813, 9.451749],
          ],
        ],
      },
      id: "ABV",
    },
    {
      type: "Feature",
      properties: {
        name: "Somalia",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [49.72862, 11.5789],
            [50.25878, 11.67957],
            [50.73202, 12.0219],
            [51.1112, 12.02464],
            [51.13387, 11.74815],
            [51.04153, 11.16651],
            [51.04531, 10.6409],
            [50.83418, 10.27972],
            [50.55239, 9.19874],
            [50.07092, 8.08173],
            [49.4527, 6.80466],
            [48.59455, 5.33911],
            [47.74079, 4.2194],
            [46.56476, 2.85529],
            [45.56399, 2.04576],
            [44.06815, 1.05283],
            [43.13597, 0.2922],
            [42.04157, -0.91916],
            [41.81095, -1.44647],
```

```
          [41.58513, -1.68325],
          [40.993, -0.85829],
          [40.98105, 2.78452],
          [41.855083, 3.918912],
          [42.12861, 4.23413],
          [42.76967, 4.25259],
          [43.66087, 4.95755],
          [44.9636, 5.00162],
          [47.78942, 8.003],
          [48.486736, 8.837626],
          [48.93813, 9.451749],
          [48.938233, 9.9735],
          [48.938491, 10.982327],
          [48.942005, 11.394266],
          [48.948205, 11.410617],
          [49.26776, 11.43033],
          [49.72862, 11.5789],
        ],
      ],
    },
    id: "SOM",
  },
  {
    type: "Feature",
    properties: {
      name: "Serbia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [20.874313, 45.416375],
          [21.483526, 45.18117],
          [21.562023, 44.768947],
          [22.145088, 44.478422],
          [22.459022, 44.702517],
          [22.705726, 44.578003],
          [22.474008, 44.409228],
          [22.65715, 44.234923],
          [22.410446, 44.008063],
          [22.500157, 43.642814],
          [22.986019, 43.211161],
          [22.604801, 42.898519],
          [22.436595, 42.580321],
          [22.545012, 42.461362],
          [22.380526, 42.32026],
          [21.91708, 42.30364],
          [21.576636, 42.245224],
          [21.54332, 42.32025],
          [21.66292, 42.43922],
          [21.77505, 42.6827],
          [21.63302, 42.67717],
          [21.43866, 42.86255],
          [21.27421, 42.90959],
          [21.143395, 43.068685],
          [20.95651, 43.13094],
          [20.81448, 43.27205],
          [20.63508, 43.21671],
          [20.49679, 42.88469],
          [20.25758, 42.81275],
          [20.3398, 42.89852],
          [19.95857, 43.10604],
          [19.63, 43.21378],
          [19.48389, 43.35229],
          [19.21852, 43.52384],
          [19.454, 43.5681],
```

```
          [19.59976, 44.03847],
          [19.11761, 44.42307],
          [19.36803, 44.863],
          [19.00548, 44.86023],
          [19.390476, 45.236516],
          [19.072769, 45.521511],
          [18.82982, 45.90888],
          [19.596045, 46.17173],
          [20.220192, 46.127469],
          [20.762175, 45.734573],
          [20.874313, 45.416375],
        ],
      ],
    },
    id: "SRB",
  },
  {
    type: "Feature",
    properties: {
      name: "Suriname",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-57.147436, 5.97315],
          [-55.949318, 5.772878],
          [-55.84178, 5.953125],
          [-55.03325, 6.025291],
          [-53.958045, 5.756548],
          [-54.478633, 4.896756],
          [-54.399542, 4.212611],
          [-54.006931, 3.620038],
          [-54.181726, 3.18978],
          [-54.269705, 2.732392],
          [-54.524754, 2.311849],
          [-55.097587, 2.523748],
          [-55.569755, 2.421506],
          [-55.973322, 2.510364],
          [-56.073342, 2.220795],
          [-55.9056, 2.021996],
          [-55.995698, 1.817667],
          [-56.539386, 1.899523],
          [-57.150098, 2.768927],
          [-57.281433, 3.333492],
          [-57.601569, 3.334655],
          [-58.044694, 4.060864],
          [-57.86021, 4.576801],
          [-57.914289, 4.812626],
          [-57.307246, 5.073567],
          [-57.147436, 5.97315],
        ],
      ],
    },
    id: "SUR",
  },
  {
    type: "Feature",
    properties: {
      name: "Slovakia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [18.853144, 49.49623],
```

```
          [18.909575, 49.435846],
          [19.320713, 49.571574],
          [19.825023, 49.217125],
          [20.415839, 49.431453],
          [20.887955, 49.328772],
          [21.607808, 49.470107],
          [22.558138, 49.085738],
          [22.280842, 48.825392],
          [22.085608, 48.422264],
          [21.872236, 48.319971],
          [20.801294, 48.623854],
          [20.473562, 48.56285],
          [20.239054, 48.327567],
          [19.769471, 48.202691],
          [19.661364, 48.266615],
          [19.174365, 48.111379],
          [18.777025, 48.081768],
          [18.696513, 47.880954],
          [17.857133, 47.758429],
          [17.488473, 47.867466],
          [16.979667, 48.123497],
          [16.879983, 48.470013],
          [16.960288, 48.596982],
          [17.101985, 48.816969],
          [17.545007, 48.800019],
          [17.886485, 48.903475],
          [17.913512, 48.996493],
          [18.104973, 49.043983],
          [18.170498, 49.271515],
          [18.399994, 49.315001],
          [18.554971, 49.495015],
          [18.853144, 49.49623],
        ],
      ],
    },
    id: "SVK",
  },
  {
    type: "Feature",
    properties: {
      name: "Slovenia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [13.806475, 46.509306],
          [14.632472, 46.431817],
          [15.137092, 46.658703],
          [16.011664, 46.683611],
          [16.202298, 46.852386],
          [16.370505, 46.841327],
          [16.564808, 46.503751],
          [15.768733, 46.238108],
          [15.67153, 45.834154],
          [15.323954, 45.731783],
          [15.327675, 45.452316],
          [14.935244, 45.471695],
          [14.595109, 45.634941],
          [14.411968, 45.466166],
          [13.71506, 45.500324],
          [13.93763, 45.591016],
          [13.69811, 46.016778],
          [13.806475, 46.509306],
        ],
      ],
```

```
      },
      id: "SVN",
    },
    {
      type: "Feature",
      properties: {
        name: "Sweden",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [22.183173, 65.723741],
            [21.213517, 65.026005],
            [21.369631, 64.413588],
            [19.778876, 63.609554],
            [17.847779, 62.7494],
            [17.119555, 61.341166],
            [17.831346, 60.636583],
            [18.787722, 60.081914],
            [17.869225, 58.953766],
            [16.829185, 58.719827],
            [16.44771, 57.041118],
            [15.879786, 56.104302],
            [14.666681, 56.200885],
            [14.100721, 55.407781],
            [12.942911, 55.361737],
            [12.625101, 56.30708],
            [11.787942, 57.441817],
            [11.027369, 58.856149],
            [11.468272, 59.432393],
            [12.300366, 60.117933],
            [12.631147, 61.293572],
            [11.992064, 61.800362],
            [11.930569, 63.128318],
            [12.579935, 64.066219],
            [13.571916, 64.049114],
            [13.919905, 64.445421],
            [13.55569, 64.787028],
            [15.108411, 66.193867],
            [16.108712, 67.302456],
            [16.768879, 68.013937],
            [17.729182, 68.010552],
            [17.993868, 68.567391],
            [19.87856, 68.407194],
            [20.025269, 69.065139],
            [20.645593, 69.106247],
            [21.978535, 68.616846],
            [23.539473, 67.936009],
            [23.56588, 66.396051],
            [23.903379, 66.006927],
            [22.183173, 65.723741],
          ],
        ],
      },
      id: "SWE",
    },
    {
      type: "Feature",
      properties: {
        name: "Swaziland",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
```

```
          [32.071665, -26.73382],
          [31.86806, -27.177927],
          [31.282773, -27.285879],
          [30.685962, -26.743845],
          [30.676609, -26.398078],
          [30.949667, -26.022649],
          [31.04408, -25.731452],
          [31.333158, -25.660191],
          [31.837778, -25.843332],
          [31.985779, -26.29178],
          [32.071665, -26.73382],
        ],
      ],
    },
    id: "SWZ",
  },
  {
    type: "Feature",
    properties: {
      name: "Syria",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [38.792341, 33.378686],
          [36.834062, 32.312938],
          [35.719918, 32.709192],
          [35.700798, 32.716014],
          [35.836397, 32.868123],
          [35.821101, 33.277426],
          [36.06646, 33.824912],
          [36.61175, 34.201789],
          [36.448194, 34.593935],
          [35.998403, 34.644914],
          [35.905023, 35.410009],
          [36.149763, 35.821535],
          [36.41755, 36.040617],
          [36.685389, 36.259699],
          [36.739494, 36.81752],
          [37.066761, 36.623036],
          [38.167727, 36.90121],
          [38.699891, 36.712927],
          [39.52258, 36.716054],
          [40.673259, 37.091276],
          [41.212089, 37.074352],
          [42.349591, 37.229873],
          [41.837064, 36.605854],
          [41.289707, 36.358815],
          [41.383965, 35.628317],
          [41.006159, 34.419372],
          [38.792341, 33.378686],
        ],
      ],
    },
    id: "SYR",
  },
  {
    type: "Feature",
    properties: {
      name: "Chad",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
```

```
          [14.495787, 12.859396],
          [14.595781, 13.330427],
          [13.954477, 13.353449],
          [13.956699, 13.996691],
          [13.540394, 14.367134],
          [13.97217, 15.68437],
          [15.247731, 16.627306],
          [15.300441, 17.92795],
          [15.685741, 19.95718],
          [15.903247, 20.387619],
          [15.487148, 20.730415],
          [15.47106, 21.04845],
          [15.096888, 21.308519],
          [14.8513, 22.86295],
          [15.86085, 23.40972],
          [19.84926, 21.49509],
          [23.83766, 19.58047],
          [23.88689, 15.61084],
          [23.02459, 15.68072],
          [22.56795, 14.94429],
          [22.30351, 14.32682],
          [22.51202, 14.09318],
          [22.18329, 13.78648],
          [22.29658, 13.37232],
          [22.03759, 12.95546],
          [21.93681, 12.58818],
          [22.28801, 12.64605],
          [22.49762, 12.26024],
          [22.50869, 11.67936],
          [22.87622, 11.38461],
          [22.864165, 11.142395],
          [22.231129, 10.971889],
          [21.723822, 10.567056],
          [21.000868, 9.475985],
          [20.059685, 9.012706],
          [19.094008, 9.074847],
          [18.81201, 8.982915],
          [18.911022, 8.630895],
          [18.389555, 8.281304],
          [17.96493, 7.890914],
          [16.705988, 7.508328],
          [16.456185, 7.734774],
          [16.290562, 7.754307],
          [16.106232, 7.497088],
          [15.27946, 7.421925],
          [15.436092, 7.692812],
          [15.120866, 8.38215],
          [14.979996, 8.796104],
          [14.544467, 8.965861],
          [13.954218, 9.549495],
          [14.171466, 10.021378],
          [14.627201, 9.920919],
          [14.909354, 9.992129],
          [15.467873, 9.982337],
          [14.923565, 10.891325],
          [14.960152, 11.555574],
          [14.89336, 12.21905],
          [14.495787, 12.859396],
        ],
      ],
    },
    id: "TCD",
  },
  {
    type: "Feature",
    properties: {
```

```
        name: "Togo",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [1.865241, 6.142158],
            [1.060122, 5.928837],
            [0.836931, 6.279979],
            [0.570384, 6.914359],
            [0.490957, 7.411744],
            [0.712029, 8.312465],
            [0.461192, 8.677223],
            [0.365901, 9.465004],
            [0.36758, 10.191213],
            [-0.049785, 10.706918],
            [0.023803, 11.018682],
            [0.899563, 10.997339],
            [0.772336, 10.470808],
            [1.077795, 10.175607],
            [1.425061, 9.825395],
            [1.463043, 9.334624],
            [1.664478, 9.12859],
            [1.618951, 6.832038],
            [1.865241, 6.142158],
          ],
        ],
      },
      id: "TGO",
    },
    {
      type: "Feature",
      properties: {
        name: "Thailand",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [102.584932, 12.186595],
            [101.687158, 12.64574],
            [100.83181, 12.627085],
            [100.978467, 13.412722],
            [100.097797, 13.406856],
            [100.018733, 12.307001],
            [99.478921, 10.846367],
            [99.153772, 9.963061],
            [99.222399, 9.239255],
            [99.873832, 9.207862],
            [100.279647, 8.295153],
            [100.459274, 7.429573],
            [101.017328, 6.856869],
            [101.623079, 6.740622],
            [102.141187, 6.221636],
            [101.814282, 5.810808],
            [101.154219, 5.691384],
            [101.075516, 6.204867],
            [100.259596, 6.642825],
            [100.085757, 6.464489],
            [99.690691, 6.848213],
            [99.519642, 7.343454],
            [98.988253, 7.907993],
            [98.503786, 8.382305],
            [98.339662, 7.794512],
            [98.150009, 8.350007],
            [98.25915, 8.973923],
```

```
          [98.553551, 9.93296],
          [99.038121, 10.960546],
          [99.587286, 11.892763],
          [99.196354, 12.804748],
          [99.212012, 13.269294],
          [99.097755, 13.827503],
          [98.430819, 14.622028],
          [98.192074, 15.123703],
          [98.537376, 15.308497],
          [98.903348, 16.177824],
          [98.493761, 16.837836],
          [97.859123, 17.567946],
          [97.375896, 18.445438],
          [97.797783, 18.62708],
          [98.253724, 19.708203],
          [98.959676, 19.752981],
          [99.543309, 20.186598],
          [100.115988, 20.41785],
          [100.548881, 20.109238],
          [100.606294, 19.508344],
          [101.282015, 19.462585],
          [101.035931, 18.408928],
          [101.059548, 17.512497],
          [102.113592, 18.109102],
          [102.413005, 17.932782],
          [102.998706, 17.961695],
          [103.200192, 18.309632],
          [103.956477, 18.240954],
          [104.716947, 17.428859],
          [104.779321, 16.441865],
          [105.589039, 15.570316],
          [105.544338, 14.723934],
          [105.218777, 14.273212],
          [104.281418, 14.416743],
          [102.988422, 14.225721],
          [102.348099, 13.394247],
          [102.584932, 12.186595],
        ],
      ],
    },
    id: "THA",
  },
  {
    type: "Feature",
    properties: {
      name: "Tajikistan",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [71.014198, 40.244366],
          [70.648019, 39.935754],
          [69.55961, 40.103211],
          [69.464887, 39.526683],
          [70.549162, 39.604198],
          [71.784694, 39.279463],
          [73.675379, 39.431237],
          [73.928852, 38.505815],
          [74.257514, 38.606507],
          [74.864816, 38.378846],
          [74.829986, 37.990007],
          [74.980002, 37.41999],
          [73.948696, 37.421566],
          [73.260056, 37.495257],
          [72.63689, 37.047558],
```

```
          [72.193041, 36.948288],
          [71.844638, 36.738171],
          [71.448693, 37.065645],
          [71.541918, 37.905774],
          [71.239404, 37.953265],
          [71.348131, 38.258905],
          [70.806821, 38.486282],
          [70.376304, 38.138396],
          [70.270574, 37.735165],
          [70.116578, 37.588223],
          [69.518785, 37.608997],
          [69.196273, 37.151144],
          [68.859446, 37.344336],
          [68.135562, 37.023115],
          [67.83, 37.144994],
          [68.392033, 38.157025],
          [68.176025, 38.901553],
          [67.44222, 39.140144],
          [67.701429, 39.580478],
          [68.536416, 39.533453],
          [69.011633, 40.086158],
          [69.329495, 40.727824],
          [70.666622, 40.960213],
          [70.45816, 40.496495],
          [70.601407, 40.218527],
          [71.014198, 40.244366],
        ],
      ],
    },
    id: "TJK",
  },
  {
    type: "Feature",
    properties: {
      name: "Turkmenistan",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [61.210817, 35.650072],
          [61.123071, 36.491597],
          [60.377638, 36.527383],
          [59.234762, 37.412988],
          [58.436154, 37.522309],
          [57.330434, 38.029229],
          [56.619366, 38.121394],
          [56.180375, 37.935127],
          [55.511578, 37.964117],
          [54.800304, 37.392421],
          [53.921598, 37.198918],
          [53.735511, 37.906136],
          [53.880929, 38.952093],
          [53.101028, 39.290574],
          [53.357808, 39.975286],
          [52.693973, 40.033629],
          [52.915251, 40.876523],
          [53.858139, 40.631034],
          [54.736845, 40.951015],
          [54.008311, 41.551211],
          [53.721713, 42.123191],
          [52.91675, 41.868117],
          [52.814689, 41.135371],
          [52.50246, 41.783316],
          [52.944293, 42.116034],
          [54.079418, 42.324109],
```

```
          [54.755345, 42.043971],
          [55.455251, 41.259859],
          [55.968191, 41.308642],
          [57.096391, 41.32231],
          [56.932215, 41.826026],
          [57.78653, 42.170553],
          [58.629011, 42.751551],
          [59.976422, 42.223082],
          [60.083341, 41.425146],
          [60.465953, 41.220327],
          [61.547179, 41.26637],
          [61.882714, 41.084857],
          [62.37426, 40.053886],
          [63.518015, 39.363257],
          [64.170223, 38.892407],
          [65.215999, 38.402695],
          [66.54615, 37.974685],
          [66.518607, 37.362784],
          [66.217385, 37.39379],
          [65.745631, 37.661164],
          [65.588948, 37.305217],
          [64.746105, 37.111818],
          [64.546479, 36.312073],
          [63.982896, 36.007957],
          [63.193538, 35.857166],
          [62.984662, 35.404041],
          [62.230651, 35.270664],
          [61.210817, 35.650072],
        ],
      ],
    },
    id: "TKM",
  },
  {
    type: "Feature",
    properties: {
      name: "East Timor",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [124.968682, -8.89279],
          [125.086246, -8.656887],
          [125.947072, -8.432095],
          [126.644704, -8.398247],
          [126.957243, -8.273345],
          [127.335928, -8.397317],
          [126.967992, -8.668256],
          [125.925885, -9.106007],
          [125.08852, -9.393173],
          [125.07002, -9.089987],
          [124.968682, -8.89279],
        ],
      ],
    },
    id: "TLS",
  },
  {
    type: "Feature",
    properties: {
      name: "Trinidad and Tobago",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
```

```
        [
          [-61.68, 10.76],
          [-61.105, 10.89],
          [-60.895, 10.855],
          [-60.935, 10.11],
          [-61.77, 10],
          [-61.95, 10.09],
          [-61.66, 10.365],
          [-61.68, 10.76],
        ],
      ],
    },
    id: "TTO",
  },
  {
    type: "Feature",
    properties: {
      name: "Tunisia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [9.48214, 30.307556],
          [9.055603, 32.102692],
          [8.439103, 32.506285],
          [8.430473, 32.748337],
          [7.612642, 33.344115],
          [7.524482, 34.097376],
          [8.140981, 34.655146],
          [8.376368, 35.479876],
          [8.217824, 36.433177],
          [8.420964, 36.946427],
          [9.509994, 37.349994],
          [10.210002, 37.230002],
          [10.18065, 36.724038],
          [11.028867, 37.092103],
          [11.100026, 36.899996],
          [10.600005, 36.41],
          [10.593287, 35.947444],
          [10.939519, 35.698984],
          [10.807847, 34.833507],
          [10.149593, 34.330773],
          [10.339659, 33.785742],
          [10.856836, 33.76874],
          [11.108501, 33.293343],
          [11.488787, 33.136996],
          [11.432253, 32.368903],
          [10.94479, 32.081815],
          [10.636901, 31.761421],
          [9.950225, 31.37607],
          [10.056575, 30.961831],
          [9.970017, 30.539325],
          [9.48214, 30.307556],
        ],
      ],
    },
    id: "TUN",
  },
  {
    type: "Feature",
    properties: {
      name: "Turkey",
    },
    geometry: {
      type: "MultiPolygon",
```

```
        coordinates: [
          [
            [
              [36.913127, 41.335358],
              [38.347665, 40.948586],
              [39.512607, 41.102763],
              [40.373433, 41.013673],
              [41.554084, 41.535656],
              [42.619549, 41.583173],
              [43.582746, 41.092143],
              [43.752658, 40.740201],
              [43.656436, 40.253564],
              [44.400009, 40.005],
              [44.79399, 39.713003],
              [44.109225, 39.428136],
              [44.421403, 38.281281],
              [44.225756, 37.971584],
              [44.772699, 37.170445],
              [44.293452, 37.001514],
              [43.942259, 37.256228],
              [42.779126, 37.385264],
              [42.349591, 37.229873],
              [41.212089, 37.074352],
              [40.673259, 37.091276],
              [39.52258, 36.716054],
              [38.699891, 36.712927],
              [38.167727, 36.90121],
              [37.066761, 36.623036],
              [36.739494, 36.81752],
              [36.685389, 36.259699],
              [36.41755, 36.040617],
              [36.149763, 35.821535],
              [35.782085, 36.274995],
              [36.160822, 36.650606],
              [35.550936, 36.565443],
              [34.714553, 36.795532],
              [34.026895, 36.21996],
              [32.509158, 36.107564],
              [31.699595, 36.644275],
              [30.621625, 36.677865],
              [30.391096, 36.262981],
              [29.699976, 36.144357],
              [28.732903, 36.676831],
              [27.641187, 36.658822],
              [27.048768, 37.653361],
              [26.318218, 38.208133],
              [26.8047, 38.98576],
              [26.170785, 39.463612],
              [27.28002, 40.420014],
              [28.819978, 40.460011],
              [29.240004, 41.219991],
              [31.145934, 41.087622],
              [32.347979, 41.736264],
              [33.513283, 42.01896],
              [35.167704, 42.040225],
              [36.913127, 41.335358],
            ],
          ],
          [
            [
              [27.192377, 40.690566],
              [26.358009, 40.151994],
              [26.043351, 40.617754],
              [26.056942, 40.824123],
              [26.294602, 40.936261],
              [26.604196, 41.562115],
```

```
            [26.117042, 41.826905],
            [27.135739, 42.141485],
            [27.99672, 42.007359],
            [28.115525, 41.622886],
            [28.988443, 41.299934],
            [28.806438, 41.054962],
            [27.619017, 40.999823],
            [27.192377, 40.690566],
          ],
        ],
      ],
    },
    id: "TUR",
  },
  {
    type: "Feature",
    properties: {
      name: "Taiwan",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [121.777818, 24.394274],
          [121.175632, 22.790857],
          [120.74708, 21.970571],
          [120.220083, 22.814861],
          [120.106189, 23.556263],
          [120.69468, 24.538451],
          [121.495044, 25.295459],
          [121.951244, 24.997596],
          [121.777818, 24.394274],
        ],
      ],
    },
    id: "TWN",
  },
  {
    type: "Feature",
    properties: {
      name: "Tanzania",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [33.903711, -0.95],
          [34.07262, -1.05982],
          [37.69869, -3.09699],
          [37.7669, -3.67712],
          [39.20222, -4.67677],
          [38.74054, -5.90895],
          [38.79977, -6.47566],
          [39.44, -6.84],
          [39.47, -7.1],
          [39.19469, -7.7039],
          [39.25203, -8.00781],
          [39.18652, -8.48551],
          [39.53574, -9.11237],
          [39.9496, -10.0984],
          [40.31659, -10.3171],
          [39.521, -10.89688],
          [38.427557, -11.285202],
          [37.82764, -11.26879],
          [37.47129, -11.56876],
          [36.775151, -11.594537],
```

```
          [36.514082, -11.720938],
          [35.312398, -11.439146],
          [34.559989, -11.52002],
          [34.28, -10.16],
          [33.940838, -9.693674],
          [33.73972, -9.41715],
          [32.759375, -9.230599],
          [32.191865, -8.930359],
          [31.556348, -8.762049],
          [31.157751, -8.594579],
          [30.74, -8.34],
          [30.2, -7.08],
          [29.62, -6.52],
          [29.419993, -5.939999],
          [29.519987, -5.419979],
          [29.339998, -4.499983],
          [29.753512, -4.452389],
          [30.11632, -4.09012],
          [30.50554, -3.56858],
          [30.75224, -3.35931],
          [30.74301, -3.03431],
          [30.52766, -2.80762],
          [30.46967, -2.41383],
          [30.758309, -2.28725],
          [30.816135, -1.698914],
          [30.419105, -1.134659],
          [30.76986, -1.01455],
          [31.86617, -1.02736],
          [33.903711, -0.95],
        ],
      ],
    },
    id: "TZA",
  },
  {
    type: "Feature",
    properties: {
      name: "Uganda",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [31.86617, -1.02736],
          [30.76986, -1.01455],
          [30.419105, -1.134659],
          [29.821519, -1.443322],
          [29.579466, -1.341313],
          [29.587838, -0.587406],
          [29.8195, -0.2053],
          [29.875779, 0.59738],
          [30.086154, 1.062313],
          [30.468508, 1.583805],
          [30.85267, 1.849396],
          [31.174149, 2.204465],
          [30.77332, 2.33989],
          [30.83385, 3.50917],
          [31.24556, 3.7819],
          [31.88145, 3.55827],
          [32.68642, 3.79232],
          [33.39, 3.79],
          [34.005, 4.249885],
          [34.47913, 3.5556],
          [34.59607, 3.05374],
          [35.03599, 1.90584],
          [34.6721, 1.17694],
```

```
          [34.18, 0.515],
          [33.893569, 0.109814],
          [33.903711, -0.95],
          [31.86617, -1.02736],
        ],
      ],
    },
    id: "UGA",
  },
  {
    type: "Feature",
    properties: {
      name: "Ukraine",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [31.785998, 52.101678],
          [32.159412, 52.061267],
          [32.412058, 52.288695],
          [32.715761, 52.238465],
          [33.7527, 52.335075],
          [34.391731, 51.768882],
          [34.141978, 51.566413],
          [34.224816, 51.255993],
          [35.022183, 51.207572],
          [35.377924, 50.773955],
          [35.356116, 50.577197],
          [36.626168, 50.225591],
          [37.39346, 50.383953],
          [38.010631, 49.915662],
          [38.594988, 49.926462],
          [40.069058, 49.601055],
          [40.080789, 49.30743],
          [39.674664, 48.783818],
          [39.895632, 48.232405],
          [39.738278, 47.898937],
          [38.770585, 47.825608],
          [38.255112, 47.5464],
          [38.223538, 47.10219],
          [37.425137, 47.022221],
          [36.759855, 46.6987],
          [35.823685, 46.645964],
          [34.962342, 46.273197],
          [35.020788, 45.651219],
          [35.510009, 45.409993],
          [36.529998, 45.46999],
          [36.334713, 45.113216],
          [35.239999, 44.939996],
          [33.882511, 44.361479],
          [33.326421, 44.564877],
          [33.546924, 45.034771],
          [32.454174, 45.327466],
          [32.630804, 45.519186],
          [33.588162, 45.851569],
          [33.298567, 46.080598],
          [31.74414, 46.333348],
          [31.675307, 46.706245],
          [30.748749, 46.5831],
          [30.377609, 46.03241],
          [29.603289, 45.293308],
          [29.149725, 45.464925],
          [28.679779, 45.304031],
          [28.233554, 45.488283],
          [28.485269, 45.596907],
```

```
          [28.659987, 45.939987],
          [28.933717, 46.25883],
          [28.862972, 46.437889],
          [29.072107, 46.517678],
          [29.170654, 46.379262],
          [29.759972, 46.349988],
          [30.024659, 46.423937],
          [29.83821, 46.525326],
          [29.908852, 46.674361],
          [29.559674, 46.928583],
          [29.415135, 47.346645],
          [29.050868, 47.510227],
          [29.122698, 47.849095],
          [28.670891, 48.118149],
          [28.259547, 48.155562],
          [27.522537, 48.467119],
          [26.857824, 48.368211],
          [26.619337, 48.220726],
          [26.19745, 48.220881],
          [25.945941, 47.987149],
          [25.207743, 47.891056],
          [24.866317, 47.737526],
          [24.402056, 47.981878],
          [23.760958, 47.985598],
          [23.142236, 48.096341],
          [22.710531, 47.882194],
          [22.64082, 48.15024],
          [22.085608, 48.422264],
          [22.280842, 48.825392],
          [22.558138, 49.085738],
          [22.776419, 49.027395],
          [22.51845, 49.476774],
          [23.426508, 50.308506],
          [23.922757, 50.424881],
          [24.029986, 50.705407],
          [23.527071, 51.578454],
          [24.005078, 51.617444],
          [24.553106, 51.888461],
          [25.327788, 51.910656],
          [26.337959, 51.832289],
          [27.454066, 51.592303],
          [28.241615, 51.572227],
          [28.617613, 51.427714],
          [28.992835, 51.602044],
          [29.254938, 51.368234],
          [30.157364, 51.416138],
          [30.555117, 51.319503],
          [30.619454, 51.822806],
          [30.927549, 52.042353],
          [31.785998, 52.101678],
        ],
      ],
    },
    id: "UKR",
  },
  {
    type: "Feature",
    properties: {
      name: "Uruguay",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [-57.625133, -30.216295],
          [-56.976026, -30.109686],
```

```
          [-55.973245, -30.883076],
          [-55.60151, -30.853879],
          [-54.572452, -31.494511],
          [-53.787952, -32.047243],
          [-53.209589, -32.727666],
          [-53.650544, -33.202004],
          [-53.373662, -33.768378],
          [-53.806426, -34.396815],
          [-54.935866, -34.952647],
          [-55.67409, -34.752659],
          [-56.215297, -34.859836],
          [-57.139685, -34.430456],
          [-57.817861, -34.462547],
          [-58.427074, -33.909454],
          [-58.349611, -33.263189],
          [-58.132648, -33.040567],
          [-58.14244, -32.044504],
          [-57.874937, -31.016556],
          [-57.625133, -30.216295],
        ],
      ],
    },
    id: "URY",
  },
  {
    type: "Feature",
    properties: {
      name: "USA",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [-155.54211, 19.08348],
            [-155.68817, 18.91619],
            [-155.93665, 19.05939],
            [-155.90806, 19.33888],
            [-156.07347, 19.70294],
            [-156.02368, 19.81422],
            [-155.85008, 19.97729],
            [-155.91907, 20.17395],
            [-155.86108, 20.26721],
            [-155.78505, 20.2487],
            [-155.40214, 20.07975],
            [-155.22452, 19.99302],
            [-155.06226, 19.8591],
            [-154.80741, 19.50871],
            [-154.83147, 19.45328],
            [-155.22217, 19.23972],
            [-155.54211, 19.08348],
          ],
        ],
        [
          [
            [-156.07926, 20.64397],
            [-156.41445, 20.57241],
            [-156.58673, 20.783],
            [-156.70167, 20.8643],
            [-156.71055, 20.92676],
            [-156.61258, 21.01249],
            [-156.25711, 20.91745],
            [-155.99566, 20.76404],
            [-156.07926, 20.64397],
          ],
        ],
```

```
        [
          [
            [-156.75824, 21.17684],
            [-156.78933, 21.06873],
            [-157.32521, 21.09777],
            [-157.25027, 21.21958],
            [-156.75824, 21.17684],
          ],
        ],
        [
          [
            [-157.65283, 21.32217],
            [-157.70703, 21.26442],
            [-157.7786, 21.27729],
            [-158.12667, 21.31244],
            [-158.2538, 21.53919],
            [-158.29265, 21.57912],
            [-158.0252, 21.71696],
            [-157.94161, 21.65272],
            [-157.65283, 21.32217],
          ],
        ],
        [
          [
            [-159.34512, 21.982],
            [-159.46372, 21.88299],
            [-159.80051, 22.06533],
            [-159.74877, 22.1382],
            [-159.5962, 22.23618],
            [-159.36569, 22.21494],
            [-159.34512, 21.982],
          ],
        ],
        [
          [
            [-94.81758, 49.38905],
            [-94.64, 48.84],
            [-94.32914, 48.67074],
            [-93.63087, 48.60926],
            [-92.61, 48.45],
            [-91.64, 48.14],
            [-90.83, 48.27],
            [-89.6, 48.01],
            [-89.272917, 48.019808],
            [-88.378114, 48.302918],
            [-87.439793, 47.94],
            [-86.461991, 47.553338],
            [-85.652363, 47.220219],
            [-84.87608, 46.900083],
            [-84.779238, 46.637102],
            [-84.543749, 46.538684],
            [-84.6049, 46.4396],
            [-84.3367, 46.40877],
            [-84.14212, 46.512226],
            [-84.091851, 46.275419],
            [-83.890765, 46.116927],
            [-83.616131, 46.116927],
            [-83.469551, 45.994686],
            [-83.592851, 45.816894],
            [-82.550925, 45.347517],
            [-82.337763, 44.44],
            [-82.137642, 43.571088],
            [-82.43, 42.98],
            [-82.9, 42.43],
            [-83.12, 42.08],
            [-83.142, 41.975681],
```

```
            [-83.02981, 41.832796],
            [-82.690089, 41.675105],
            [-82.439278, 41.675105],
            [-81.277747, 42.209026],
            [-80.247448, 42.3662],
            [-78.939362, 42.863611],
            [-78.92, 42.965],
            [-79.01, 43.27],
            [-79.171674, 43.466339],
            [-78.72028, 43.625089],
            [-77.737885, 43.629056],
            [-76.820034, 43.628784],
            [-76.5, 44.018459],
            [-76.375, 44.09631],
            [-75.31821, 44.81645],
            [-74.867, 45.00048],
            [-73.34783, 45.00738],
            [-71.50506, 45.0082],
            [-71.405, 45.255],
            [-71.08482, 45.30524],
            [-70.66, 45.46],
            [-70.305, 45.915],
            [-69.99997, 46.69307],
            [-69.237216, 47.447781],
            [-68.905, 47.185],
            [-68.23444, 47.35486],
            [-67.79046, 47.06636],
            [-67.79134, 45.70281],
            [-67.13741, 45.13753],
            [-66.96466, 44.8097],
            [-68.03252, 44.3252],
            [-69.06, 43.98],
            [-70.11617, 43.68405],
            [-70.645476, 43.090238],
            [-70.81489, 42.8653],
            [-70.825, 42.335],
            [-70.495, 41.805],
            [-70.08, 41.78],
            [-70.185, 42.145],
            [-69.88497, 41.92283],
            [-69.96503, 41.63717],
            [-70.64, 41.475],
            [-71.12039, 41.49445],
            [-71.86, 41.32],
            [-72.295, 41.27],
            [-72.87643, 41.22065],
            [-73.71, 40.931102],
            [-72.24126, 41.11948],
            [-71.945, 40.93],
            [-73.345, 40.63],
            [-73.982, 40.628],
            [-73.952325, 40.75075],
            [-74.25671, 40.47351],
            [-73.96244, 40.42763],
            [-74.17838, 39.70926],
            [-74.90604, 38.93954],
            [-74.98041, 39.1964],
            [-75.20002, 39.24845],
            [-75.52805, 39.4985],
            [-75.32, 38.96],
            [-75.071835, 38.782032],
            [-75.05673, 38.40412],
            [-75.37747, 38.01551],
            [-75.94023, 37.21689],
            [-76.03127, 37.2566],
            [-75.72205, 37.93705],
```

```
            [-76.23287, 38.319215],
            [-76.35, 39.15],
            [-76.542725, 38.717615],
            [-76.32933, 38.08326],
            [-76.989998, 38.239992],
            [-76.30162, 37.917945],
            [-76.25874, 36.9664],
            [-75.9718, 36.89726],
            [-75.86804, 36.55125],
            [-75.72749, 35.55074],
            [-76.36318, 34.80854],
            [-77.397635, 34.51201],
            [-78.05496, 33.92547],
            [-78.55435, 33.86133],
            [-79.06067, 33.49395],
            [-79.20357, 33.15839],
            [-80.301325, 32.509355],
            [-80.86498, 32.0333],
            [-81.33629, 31.44049],
            [-81.49042, 30.72999],
            [-81.31371, 30.03552],
            [-80.98, 29.18],
            [-80.535585, 28.47213],
            [-80.53, 28.04],
            [-80.056539, 26.88],
            [-80.088015, 26.205765],
            [-80.13156, 25.816775],
            [-80.38103, 25.20616],
            [-80.68, 25.08],
            [-81.17213, 25.20126],
            [-81.33, 25.64],
            [-81.71, 25.87],
            [-82.24, 26.73],
            [-82.70515, 27.49504],
            [-82.85526, 27.88624],
            [-82.65, 28.55],
            [-82.93, 29.1],
            [-83.70959, 29.93656],
            [-84.1, 30.09],
            [-85.10882, 29.63615],
            [-85.28784, 29.68612],
            [-85.7731, 30.15261],
            [-86.4, 30.4],
            [-87.53036, 30.27433],
            [-88.41782, 30.3849],
            [-89.18049, 30.31598],
            [-89.593831, 30.159994],
            [-89.413735, 29.89419],
            [-89.43, 29.48864],
            [-89.21767, 29.29108],
            [-89.40823, 29.15961],
            [-89.77928, 29.30714],
            [-90.15463, 29.11743],
            [-90.880225, 29.148535],
            [-91.626785, 29.677],
            [-92.49906, 29.5523],
            [-93.22637, 29.78375],
            [-93.84842, 29.71363],
            [-94.69, 29.48],
            [-95.60026, 28.73863],
            [-96.59404, 28.30748],
            [-97.14, 27.83],
            [-97.37, 27.38],
            [-97.38, 26.69],
            [-97.33, 26.21],
            [-97.14, 25.87],
```

```
[-97.53, 25.84],
[-98.24, 26.06],
[-99.02, 26.37],
[-99.3, 26.84],
[-99.52, 27.54],
[-100.11, 28.11],
[-100.45584, 28.69612],
[-100.9576, 29.38071],
[-101.6624, 29.7793],
[-102.48, 29.76],
[-103.11, 28.97],
[-103.94, 29.27],
[-104.45697, 29.57196],
[-104.70575, 30.12173],
[-105.03737, 30.64402],
[-105.63159, 31.08383],
[-106.1429, 31.39995],
[-106.50759, 31.75452],
[-108.24, 31.754854],
[-108.24194, 31.34222],
[-109.035, 31.34194],
[-111.02361, 31.33472],
[-113.30498, 32.03914],
[-114.815, 32.52528],
[-114.72139, 32.72083],
[-115.99135, 32.61239],
[-117.12776, 32.53534],
[-117.295938, 33.046225],
[-117.944, 33.621236],
[-118.410602, 33.740909],
[-118.519895, 34.027782],
[-119.081, 34.078],
[-119.438841, 34.348477],
[-120.36778, 34.44711],
[-120.62286, 34.60855],
[-120.74433, 35.15686],
[-121.71457, 36.16153],
[-122.54747, 37.55176],
[-122.51201, 37.78339],
[-122.95319, 38.11371],
[-123.7272, 38.95166],
[-123.86517, 39.76699],
[-124.39807, 40.3132],
[-124.17886, 41.14202],
[-124.2137, 41.99964],
[-124.53284, 42.76599],
[-124.14214, 43.70838],
[-124.020535, 44.615895],
[-123.89893, 45.52341],
[-124.079635, 46.86475],
[-124.39567, 47.72017],
[-124.68721, 48.184433],
[-124.566101, 48.379715],
[-123.12, 48.04],
[-122.58736, 47.096],
[-122.34, 47.36],
[-122.5, 48.18],
[-122.84, 49],
[-120, 49],
[-117.03121, 49],
[-116.04818, 49],
[-113, 49],
[-110.05, 49],
[-107.05, 49],
[-104.04826, 48.99986],
[-100.65, 49],
```

```
            [-97.22872, 49.0007],
            [-95.15907, 49],
            [-95.15609, 49.38425],
            [-94.81758, 49.38905],
          ],
        ],
        [
          [
            [-153.006314, 57.115842],
            [-154.00509, 56.734677],
            [-154.516403, 56.992749],
            [-154.670993, 57.461196],
            [-153.76278, 57.816575],
            [-153.228729, 57.968968],
            [-152.564791, 57.901427],
            [-152.141147, 57.591059],
            [-153.006314, 57.115842],
          ],
        ],
        [
          [
            [-165.579164, 59.909987],
            [-166.19277, 59.754441],
            [-166.848337, 59.941406],
            [-167.455277, 60.213069],
            [-166.467792, 60.38417],
            [-165.67443, 60.293607],
            [-165.579164, 59.909987],
          ],
        ],
        [
          [
            [-171.731657, 63.782515],
            [-171.114434, 63.592191],
            [-170.491112, 63.694975],
            [-169.682505, 63.431116],
            [-168.689439, 63.297506],
            [-168.771941, 63.188598],
            [-169.52944, 62.976931],
            [-170.290556, 63.194438],
            [-170.671386, 63.375822],
            [-171.553063, 63.317789],
            [-171.791111, 63.405846],
            [-171.731657, 63.782515],
          ],
        ],
        [
          [
            [-155.06779, 71.147776],
            [-154.344165, 70.696409],
            [-153.900006, 70.889989],
            [-152.210006, 70.829992],
            [-152.270002, 70.600006],
            [-150.739992, 70.430017],
            [-149.720003, 70.53001],
            [-147.613362, 70.214035],
            [-145.68999, 70.12001],
            [-144.920011, 69.989992],
            [-143.589446, 70.152514],
            [-142.07251, 69.851938],
            [-140.985988, 69.711998],
            [-140.985988, 69.711998],
            [-140.992499, 66.000029],
            [-140.99777, 60.306397],
            [-140.012998, 60.276838],
            [-139.039, 60.000007],
```

```
[-138.34089, 59.56211],
[-137.4525, 58.905],
[-136.47972, 59.46389],
[-135.47583, 59.78778],
[-134.945, 59.27056],
[-134.27111, 58.86111],
[-133.355549, 58.410285],
[-132.73042, 57.69289],
[-131.70781, 56.55212],
[-130.00778, 55.91583],
[-129.979994, 55.284998],
[-130.53611, 54.802753],
[-131.085818, 55.178906],
[-131.967211, 55.497776],
[-132.250011, 56.369996],
[-133.539181, 57.178887],
[-134.078063, 58.123068],
[-135.038211, 58.187715],
[-136.628062, 58.212209],
[-137.800006, 58.499995],
[-139.867787, 59.537762],
[-140.825274, 59.727517],
[-142.574444, 60.084447],
[-143.958881, 59.99918],
[-145.925557, 60.45861],
[-147.114374, 60.884656],
[-148.224306, 60.672989],
[-148.018066, 59.978329],
[-148.570823, 59.914173],
[-149.727858, 59.705658],
[-150.608243, 59.368211],
[-151.716393, 59.155821],
[-151.859433, 59.744984],
[-151.409719, 60.725803],
[-150.346941, 61.033588],
[-150.621111, 61.284425],
[-151.895839, 60.727198],
[-152.57833, 60.061657],
[-154.019172, 59.350279],
[-153.287511, 58.864728],
[-154.232492, 58.146374],
[-155.307491, 57.727795],
[-156.308335, 57.422774],
[-156.556097, 56.979985],
[-158.117217, 56.463608],
[-158.433321, 55.994154],
[-159.603327, 55.566686],
[-160.28972, 55.643581],
[-161.223048, 55.364735],
[-162.237766, 55.024187],
[-163.069447, 54.689737],
[-164.785569, 54.404173],
[-164.942226, 54.572225],
[-163.84834, 55.039431],
[-162.870001, 55.348043],
[-161.804175, 55.894986],
[-160.563605, 56.008055],
[-160.07056, 56.418055],
[-158.684443, 57.016675],
[-158.461097, 57.216921],
[-157.72277, 57.570001],
[-157.550274, 58.328326],
[-157.041675, 58.918885],
[-158.194731, 58.615802],
[-158.517218, 58.787781],
[-159.058606, 58.424186],
```

```
            [-159.711667, 58.93139],
            [-159.981289, 58.572549],
            [-160.355271, 59.071123],
            [-161.355003, 58.670838],
            [-161.968894, 58.671665],
            [-162.054987, 59.266925],
            [-161.874171, 59.633621],
            [-162.518059, 59.989724],
            [-163.818341, 59.798056],
            [-164.662218, 60.267484],
            [-165.346388, 60.507496],
            [-165.350832, 61.073895],
            [-166.121379, 61.500019],
            [-165.734452, 62.074997],
            [-164.919179, 62.633076],
            [-164.562508, 63.146378],
            [-163.753332, 63.219449],
            [-163.067224, 63.059459],
            [-162.260555, 63.541936],
            [-161.53445, 63.455817],
            [-160.772507, 63.766108],
            [-160.958335, 64.222799],
            [-161.518068, 64.402788],
            [-160.777778, 64.788604],
            [-161.391926, 64.777235],
            [-162.45305, 64.559445],
            [-162.757786, 64.338605],
            [-163.546394, 64.55916],
            [-164.96083, 64.446945],
            [-166.425288, 64.686672],
            [-166.845004, 65.088896],
            [-168.11056, 65.669997],
            [-166.705271, 66.088318],
            [-164.47471, 66.57666],
            [-163.652512, 66.57666],
            [-163.788602, 66.077207],
            [-161.677774, 66.11612],
            [-162.489715, 66.735565],
            [-163.719717, 67.116395],
            [-164.430991, 67.616338],
            [-165.390287, 68.042772],
            [-166.764441, 68.358877],
            [-166.204707, 68.883031],
            [-164.430811, 68.915535],
            [-163.168614, 69.371115],
            [-162.930566, 69.858062],
            [-161.908897, 70.33333],
            [-160.934797, 70.44769],
            [-159.039176, 70.891642],
            [-158.119723, 70.824721],
            [-156.580825, 71.357764],
            [-155.06779, 71.147776],
          ],
        ],
      ],
    },
    id: "USA",
  },
  {
    type: "Feature",
    properties: {
      name: "Uzbekistan",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
```

```
        [
          [66.518607, 37.362784],
          [66.54615, 37.974685],
          [65.215999, 38.402695],
          [64.170223, 38.892407],
          [63.518015, 39.363257],
          [62.37426, 40.053886],
          [61.882714, 41.084857],
          [61.547179, 41.26637],
          [60.465953, 41.220327],
          [60.083341, 41.425146],
          [59.976422, 42.223082],
          [58.629011, 42.751551],
          [57.78653, 42.170553],
          [56.932215, 41.826026],
          [57.096391, 41.32231],
          [55.968191, 41.308642],
          [55.928917, 44.995858],
          [58.503127, 45.586804],
          [58.689989, 45.500014],
          [60.239972, 44.784037],
          [61.05832, 44.405817],
          [62.0133, 43.504477],
          [63.185787, 43.650075],
          [64.900824, 43.728081],
          [66.098012, 42.99766],
          [66.023392, 41.994646],
          [66.510649, 41.987644],
          [66.714047, 41.168444],
          [67.985856, 41.135991],
          [68.259896, 40.662325],
          [68.632483, 40.668681],
          [69.070027, 41.384244],
          [70.388965, 42.081308],
          [70.962315, 42.266154],
          [71.259248, 42.167711],
          [70.420022, 41.519998],
          [71.157859, 41.143587],
          [71.870115, 41.3929],
          [73.055417, 40.866033],
          [71.774875, 40.145844],
          [71.014198, 40.244366],
          [70.601407, 40.218527],
          [70.45816, 40.496495],
          [70.666622, 40.960213],
          [69.329495, 40.727824],
          [69.011633, 40.086158],
          [68.536416, 39.533453],
          [67.701429, 39.580478],
          [67.44222, 39.140144],
          [68.176025, 38.901553],
          [68.392033, 38.157025],
          [67.83, 37.144994],
          [67.075782, 37.356144],
          [66.518607, 37.362784],
        ],
      ],
    },
    id: "UZB",
  },
  {
    type: "Feature",
    properties: {
      name: "Venezuela",
    },
    geometry: {
```

```
      type: "Polygon",
      coordinates: [
        [
          [-71.331584, 11.776284],
          [-71.360006, 11.539994],
          [-71.94705, 11.423282],
          [-71.620868, 10.96946],
          [-71.633064, 10.446494],
          [-72.074174, 9.865651],
          [-71.695644, 9.072263],
          [-71.264559, 9.137195],
          [-71.039999, 9.859993],
          [-71.350084, 10.211935],
          [-71.400623, 10.968969],
          [-70.155299, 11.375482],
          [-70.293843, 11.846822],
          [-69.943245, 12.162307],
          [-69.5843, 11.459611],
          [-68.882999, 11.443385],
          [-68.233271, 10.885744],
          [-68.194127, 10.554653],
          [-67.296249, 10.545868],
          [-66.227864, 10.648627],
          [-65.655238, 10.200799],
          [-64.890452, 10.077215],
          [-64.329479, 10.389599],
          [-64.318007, 10.641418],
          [-63.079322, 10.701724],
          [-61.880946, 10.715625],
          [-62.730119, 10.420269],
          [-62.388512, 9.948204],
          [-61.588767, 9.873067],
          [-60.830597, 9.38134],
          [-60.671252, 8.580174],
          [-60.150096, 8.602757],
          [-59.758285, 8.367035],
          [-60.550588, 7.779603],
          [-60.637973, 7.415],
          [-60.295668, 7.043911],
          [-60.543999, 6.856584],
          [-61.159336, 6.696077],
          [-61.139415, 6.234297],
          [-61.410303, 5.959068],
          [-60.733574, 5.200277],
          [-60.601179, 4.918098],
          [-60.966893, 4.536468],
          [-62.08543, 4.162124],
          [-62.804533, 4.006965],
          [-63.093198, 3.770571],
          [-63.888343, 4.02053],
          [-64.628659, 4.148481],
          [-64.816064, 4.056445],
          [-64.368494, 3.79721],
          [-64.408828, 3.126786],
          [-64.269999, 2.497006],
          [-63.422867, 2.411068],
          [-63.368788, 2.2009],
          [-64.083085, 1.916369],
          [-64.199306, 1.492855],
          [-64.611012, 1.328731],
          [-65.354713, 1.095282],
          [-65.548267, 0.789254],
          [-66.325765, 0.724452],
          [-66.876326, 1.253361],
          [-67.181294, 2.250638],
          [-67.447092, 2.600281],
```

```
          [-67.809938, 2.820655],
          [-67.303173, 3.318454],
          [-67.337564, 3.542342],
          [-67.621836, 3.839482],
          [-67.823012, 4.503937],
          [-67.744697, 5.221129],
          [-67.521532, 5.55687],
          [-67.34144, 6.095468],
          [-67.695087, 6.267318],
          [-68.265052, 6.153268],
          [-68.985319, 6.206805],
          [-69.38948, 6.099861],
          [-70.093313, 6.960376],
          [-70.674234, 7.087785],
          [-71.960176, 6.991615],
          [-72.198352, 7.340431],
          [-72.444487, 7.423785],
          [-72.479679, 7.632506],
          [-72.360901, 8.002638],
          [-72.439862, 8.405275],
          [-72.660495, 8.625288],
          [-72.78873, 9.085027],
          [-73.304952, 9.152],
          [-73.027604, 9.73677],
          [-72.905286, 10.450344],
          [-72.614658, 10.821975],
          [-72.227575, 11.108702],
          [-71.973922, 11.608672],
          [-71.331584, 11.776284],
        ],
      ],
    },
    id: "VEN",
  },
  {
    type: "Feature",
    properties: {
      name: "Vietnam",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [108.05018, 21.55238],
          [106.715068, 20.696851],
          [105.881682, 19.75205],
          [105.662006, 19.058165],
          [106.426817, 18.004121],
          [107.361954, 16.697457],
          [108.269495, 16.079742],
          [108.877107, 15.276691],
          [109.33527, 13.426028],
          [109.200136, 11.666859],
          [108.36613, 11.008321],
          [107.220929, 10.364484],
          [106.405113, 9.53084],
          [105.158264, 8.59976],
          [104.795185, 9.241038],
          [105.076202, 9.918491],
          [104.334335, 10.486544],
          [105.199915, 10.88931],
          [106.24967, 10.961812],
          [105.810524, 11.567615],
          [107.491403, 12.337206],
          [107.614548, 13.535531],
          [107.382727, 14.202441],
```

```
          [107.564525, 15.202173],
          [107.312706, 15.908538],
          [106.556008, 16.604284],
          [105.925762, 17.485315],
          [105.094598, 18.666975],
          [103.896532, 19.265181],
          [104.183388, 19.624668],
          [104.822574, 19.886642],
          [104.435, 20.758733],
          [103.203861, 20.766562],
          [102.754896, 21.675137],
          [102.170436, 22.464753],
          [102.706992, 22.708795],
          [103.504515, 22.703757],
          [104.476858, 22.81915],
          [105.329209, 23.352063],
          [105.811247, 22.976892],
          [106.725403, 22.794268],
          [106.567273, 22.218205],
          [107.04342, 21.811899],
          [108.05018, 21.55238],
        ],
      ],
    },
    id: "VNM",
  },
  {
    type: "Feature",
    properties: {
      name: "Vanuatu",
    },
    geometry: {
      type: "MultiPolygon",
      coordinates: [
        [
          [
            [167.844877, -16.466333],
            [167.515181, -16.59785],
            [167.180008, -16.159995],
            [167.216801, -15.891846],
            [167.844877, -16.466333],
          ],
        ],
        [
          [
            [167.107712, -14.93392],
            [167.270028, -15.740021],
            [167.001207, -15.614602],
            [166.793158, -15.668811],
            [166.649859, -15.392704],
            [166.629137, -14.626497],
            [167.107712, -14.93392],
          ],
        ],
      ],
    },
    id: "VUT",
  },
  {
    type: "Feature",
    properties: {
      name: "West Bank",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
```

```
        [
          [35.545665, 32.393992],
          [35.545252, 31.782505],
          [35.397561, 31.489086],
          [34.927408, 31.353435],
          [34.970507, 31.616778],
          [35.225892, 31.754341],
          [34.974641, 31.866582],
          [35.18393, 32.532511],
          [35.545665, 32.393992],
        ],
      ],
    },
    id: "PSE",
  },
  {
    type: "Feature",
    properties: {
      name: "Yemen",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [53.108573, 16.651051],
          [52.385206, 16.382411],
          [52.191729, 15.938433],
          [52.168165, 15.59742],
          [51.172515, 15.17525],
          [49.574576, 14.708767],
          [48.679231, 14.003202],
          [48.238947, 13.94809],
          [47.938914, 14.007233],
          [47.354454, 13.59222],
          [46.717076, 13.399699],
          [45.877593, 13.347764],
          [45.62505, 13.290946],
          [45.406459, 13.026905],
          [45.144356, 12.953938],
          [44.989533, 12.699587],
          [44.494576, 12.721653],
          [44.175113, 12.58595],
          [43.482959, 12.6368],
          [43.222871, 13.22095],
          [43.251448, 13.767584],
          [43.087944, 14.06263],
          [42.892245, 14.802249],
          [42.604873, 15.213335],
          [42.805015, 15.261963],
          [42.702438, 15.718886],
          [42.823671, 15.911742],
          [42.779332, 16.347891],
          [43.218375, 16.66689],
          [43.115798, 17.08844],
          [43.380794, 17.579987],
          [43.791519, 17.319977],
          [44.062613, 17.410359],
          [45.216651, 17.433329],
          [45.399999, 17.333335],
          [46.366659, 17.233315],
          [46.749994, 17.283338],
          [47.000005, 16.949999],
          [47.466695, 17.116682],
          [48.183344, 18.166669],
          [49.116672, 18.616668],
          [52.00001, 19.000003],
```

```
          [52.782184, 17.349742],
          [53.108573, 16.651051],
        ],
      ],
    },
    id: "YEM",
  },
  {
    type: "Feature",
    properties: {
      name: "South Africa",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [31.521001, -29.257387],
          [31.325561, -29.401978],
          [30.901763, -29.909957],
          [30.622813, -30.423776],
          [30.055716, -31.140269],
          [28.925553, -32.172041],
          [28.219756, -32.771953],
          [27.464608, -33.226964],
          [26.419452, -33.61495],
          [25.909664, -33.66704],
          [25.780628, -33.944646],
          [25.172862, -33.796851],
          [24.677853, -33.987176],
          [23.594043, -33.794474],
          [22.988189, -33.916431],
          [22.574157, -33.864083],
          [21.542799, -34.258839],
          [20.689053, -34.417175],
          [20.071261, -34.795137],
          [19.616405, -34.819166],
          [19.193278, -34.462599],
          [18.855315, -34.444306],
          [18.424643, -33.997873],
          [18.377411, -34.136521],
          [18.244499, -33.867752],
          [18.25008, -33.281431],
          [17.92519, -32.611291],
          [18.24791, -32.429131],
          [18.221762, -31.661633],
          [17.566918, -30.725721],
          [17.064416, -29.878641],
          [17.062918, -29.875954],
          [16.344977, -28.576705],
          [16.824017, -28.082162],
          [17.218929, -28.355943],
          [17.387497, -28.783514],
          [17.836152, -28.856378],
          [18.464899, -29.045462],
          [19.002127, -28.972443],
          [19.894734, -28.461105],
          [19.895768, -24.76779],
          [20.165726, -24.917962],
          [20.758609, -25.868136],
          [20.66647, -26.477453],
          [20.889609, -26.828543],
          [21.605896, -26.726534],
          [22.105969, -26.280256],
          [22.579532, -25.979448],
          [22.824271, -25.500459],
          [23.312097, -25.26869],
```

```
          [23.73357, -25.390129],
          [24.211267, -25.670216],
          [25.025171, -25.71967],
          [25.664666, -25.486816],
          [25.765849, -25.174845],
          [25.941652, -24.696373],
          [26.485753, -24.616327],
          [26.786407, -24.240691],
          [27.11941, -23.574323],
          [28.017236, -22.827754],
          [29.432188, -22.091313],
          [29.839037, -22.102216],
          [30.322883, -22.271612],
          [30.659865, -22.151567],
          [31.191409, -22.25151],
          [31.670398, -23.658969],
          [31.930589, -24.369417],
          [31.752408, -25.484284],
          [31.837778, -25.843332],
          [31.333158, -25.660191],
          [31.04408, -25.731452],
          [30.949667, -26.022649],
          [30.676609, -26.398078],
          [30.685962, -26.743845],
          [31.282773, -27.285879],
          [31.86806, -27.177927],
          [32.071665, -26.73382],
          [32.83012, -26.742192],
          [32.580265, -27.470158],
          [32.462133, -28.301011],
          [32.203389, -28.752405],
          [31.521001, -29.257387],
        ],
        [
          [28.978263, -28.955597],
          [28.5417, -28.647502],
          [28.074338, -28.851469],
          [27.532511, -29.242711],
          [26.999262, -29.875954],
          [27.749397, -30.645106],
          [28.107205, -30.545732],
          [28.291069, -30.226217],
          [28.8484, -30.070051],
          [29.018415, -29.743766],
          [29.325166, -29.257387],
          [28.978263, -28.955597],
        ],
      ],
    },
    id: "ZAF",
  },
  {
    type: "Feature",
    properties: {
      name: "Zambia",
    },
    geometry: {
      type: "Polygon",
      coordinates: [
        [
          [32.759375, -9.230599],
          [33.231388, -9.676722],
          [33.485688, -10.525559],
          [33.31531, -10.79655],
          [33.114289, -11.607198],
          [33.306422, -12.435778],
```

```
          [32.991764, -12.783871],
          [32.688165, -13.712858],
          [33.214025, -13.97186],
          [30.179481, -14.796099],
          [30.274256, -15.507787],
          [29.516834, -15.644678],
          [28.947463, -16.043051],
          [28.825869, -16.389749],
          [28.467906, -16.4684],
          [27.598243, -17.290831],
          [27.044427, -17.938026],
          [26.706773, -17.961229],
          [26.381935, -17.846042],
          [25.264226, -17.73654],
          [25.084443, -17.661816],
          [25.07695, -17.578823],
          [24.682349, -17.353411],
          [24.033862, -17.295843],
          [23.215048, -17.523116],
          [22.562478, -16.898451],
          [21.887843, -16.08031],
          [21.933886, -12.898437],
          [24.016137, -12.911046],
          [23.930922, -12.565848],
          [24.079905, -12.191297],
          [23.904154, -11.722282],
          [24.017894, -11.237298],
          [23.912215, -10.926826],
          [24.257155, -10.951993],
          [24.314516, -11.262826],
          [24.78317, -11.238694],
          [25.418118, -11.330936],
          [25.75231, -11.784965],
          [26.553088, -11.92444],
          [27.16442, -11.608748],
          [27.388799, -12.132747],
          [28.155109, -12.272481],
          [28.523562, -12.698604],
          [28.934286, -13.248958],
          [29.699614, -13.257227],
          [29.616001, -12.178895],
          [29.341548, -12.360744],
          [28.642417, -11.971569],
          [28.372253, -11.793647],
          [28.49607, -10.789884],
          [28.673682, -9.605925],
          [28.449871, -9.164918],
          [28.734867, -8.526559],
          [29.002912, -8.407032],
          [30.346086, -8.238257],
          [30.740015, -8.340007],
          [31.157751, -8.594579],
          [31.556348, -8.762049],
          [32.191865, -8.930359],
          [32.759375, -9.230599],
        ],
      ],
    },
    id: "ZMB",
  },
  {
    type: "Feature",
    properties: {
      name: "Zimbabwe",
    },
    geometry: {
```

```
        type: "Polygon",
        coordinates: [
          [
            [31.191409, -22.25151],
            [30.659865, -22.151567],
            [30.322883, -22.271612],
            [29.839037, -22.102216],
            [29.432188, -22.091313],
            [28.794656, -21.639454],
            [28.02137, -21.485975],
            [27.727228, -20.851802],
            [27.724747, -20.499059],
            [27.296505, -20.39152],
            [26.164791, -19.293086],
            [25.850391, -18.714413],
            [25.649163, -18.536026],
            [25.264226, -17.73654],
            [26.381935, -17.846042],
            [26.706773, -17.961229],
            [27.044427, -17.938026],
            [27.598243, -17.290831],
            [28.467906, -16.4684],
            [28.825869, -16.389749],
            [28.947463, -16.043051],
            [29.516834, -15.644678],
            [30.274256, -15.507787],
            [30.338955, -15.880839],
            [31.173064, -15.860944],
            [31.636498, -16.07199],
            [31.852041, -16.319417],
            [32.328239, -16.392074],
            [32.847639, -16.713398],
            [32.849861, -17.979057],
            [32.654886, -18.67209],
            [32.611994, -19.419383],
            [32.772708, -19.715592],
            [32.659743, -20.30429],
            [32.508693, -20.395292],
            [32.244988, -21.116489],
            [31.191409, -22.25151],
          ],
        ],
      },
      id: "ZWE",
    },
    {
      type: "Feature",
      properties: {
        name: "Korea",
      },
      geometry: {
        type: "Polygon",
        coordinates: [
          [
            [128.349716, 38.612243],
            [129.21292, 37.432392],
            [129.46045, 36.784189],
            [129.468304, 35.632141],
            [129.091377, 35.082484],
            [128.18585, 34.890377],
            [127.386519, 34.475674],
            [126.485748, 34.390046],
            [126.37392, 34.93456],
            [126.559231, 35.684541],
            [126.117398, 36.725485],
            [126.860143, 36.893924],
```

```
            [126.174759, 37.749686],
            [126.237339, 37.840378],
            [126.68372, 37.804773],
            [127.073309, 38.256115],
            [127.780035, 38.304536],
            [128.205746, 38.370397],
            [128.349716, 38.612243],
            [130.640016, 42.395009],
            [130.780007, 42.220007],
            [130.400031, 42.280004],
            [129.965949, 41.941368],
            [129.667362, 41.601104],
            [129.705189, 40.882828],
            [129.188115, 40.661808],
            [129.0104, 40.485436],
            [128.633368, 40.189847],
            [127.967414, 40.025413],
            [127.533436, 39.75685],
            [127.50212, 39.323931],
            [127.385434, 39.213472],
            [127.783343, 39.050898],
            [128.349716, 38.612243],
            [128.205746, 38.370397],
            [127.780035, 38.304536],
            [127.073309, 38.256115],
            [126.68372, 37.804773],
            [126.237339, 37.840378],
            [126.174759, 37.749686],
            [125.689104, 37.94001],
            [125.568439, 37.752089],
            [125.27533, 37.669071],
            [125.240087, 37.857224],
            [124.981033, 37.948821],
            [124.712161, 38.108346],
            [124.985994, 38.548474],
            [125.221949, 38.665857],
            [125.132859, 38.848559],
            [125.38659, 39.387958],
            [125.321116, 39.551385],
            [124.737482, 39.660344],
            [124.265625, 39.928493],
            [125.079942, 40.569824],
            [126.182045, 41.107336],
            [126.869083, 41.816569],
            [127.343783, 41.503152],
            [128.208433, 41.466772],
            [128.052215, 41.994285],
            [129.596669, 42.424982],
            [129.994267, 42.985387],
            [130.640016, 42.395009],
          ],
        ],
      },
      id: "KOR",
    },
  ],
};
```

FILE: frontend/src/stores/useCartStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios";
import { toast } from "react-hot-toast";
import { loadStripe } from "@stripe/stripe-js";
import { Elements, useStripe, useElements, CardElement } from "@stripe/react-stripe-js";
```

```
// Load your Stripe publishable key
const stripePromise =
loadStripe("pk_test_51PMjcCIw69kb65LMm6zd49cWEi1zj4wFnwVbF9mxurJg1JlgoH0S7tOPdJglr0YmyejIYYfDTHhVTFC

export const useCartStore = create((set, get) => ({
  cart: [],
  orderId: null,
  isCouponApplied: false,
  isOrderChanged: false,
  coupon: null, // Selected coupon
  userCoupons: [], // List of user coupons
  totalAmount: 0, // Total amount before discount
  discountAmount: 0, // Discount applied
  totalAfterDiscount: 0, // Total after discount
  isPaymentProcessing: false, // Flag for payment processing status
  paymentMessage: '',
  routeDetails: [], // Stores the route details for delivery
  totalDistance: 0, // Total distance for the delivery
  stripeClientSecret: "", // Stores the Stripe client secret for the payment
  userLocation:"",
  // Prepare delivery: Get the route details and shipment data

  setUserLocation: (location) => set({ userLocation: location }),

  resetRouteDetails: () => set({ routeDetails: [] }),

  processPayment: async (paymentMethod, paymentGateway, currency) => {
    const { orderId } = get(); // Get the current orderId and total amount

    if (!orderId) {
      toast.error("Order not found. Please try again.");
      return;
    }

    set({ isPaymentProcessing: true, paymentMessage: '' }); // Set loading state
    console.log(orderId);
    const formData = {
      orderId: orderId,
      payment_method: paymentMethod,
      gateway: paymentGateway,
      currency: currency,
    };

    try {
      const response = await axiosInstance.post('/payment/process', formData);
      set({ paymentMessage: 'ok' });
      if (response.data.success) {
        toast.success('Payment processed successfully!');
    } else {
        toast.error(`Error: ${response.data.error}`);
      }
    } catch (error) {
      set({ paymentMessage: 'An error occurred while processing the payment.' });
      toast.error('An error occurred while processing the payment.');
    } finally {
      set({ isPaymentProcessing: false });
    }
  },
  prepareDelivery: async (userLocation) => {
    const { orderId } = get(); // Get the current orderId
    if (!orderId) {
      toast.error("Order not found. Cannot prepare delivery.");
      return;
    }

    try {
```

```
      // Prepare the delivery data, passing the orderId and userLocation
      const response = await axiosInstance.post('/payment/delivery/prepare', { orderId,
userLocation });
      console.log(response.data);
      // Update the route details in the state
      set({
        routeDetails: response.data.route_details,
        totalDistance: response.data.total_distance,
      });

      toast.success("Delivery prepared successfully!");
    } catch (error) {
      console.error("Error preparing delivery:", error);
      toast.error(error.response?.data?.message || "Failed to prepare delivery. Please try
again.");
    } finally {
      set({ isPaymentProcessing: false }); // Reset the loading state
    }
  },

  processStripePayment: async (paymentMethod) => {
    const { orderId, stripeClientSecret } = get(); // Get the current orderId and
stripeClientSecret
    const stripe = await stripePromise;
    const elements = useElements();

    if (!stripe || !elements) {
      toast.error("Stripe is not loaded correctly. Please try again later.");
      return;
    }

    set({ isPaymentProcessing: true, paymentMessage: '' });

    try {
      // Use Stripe's confirmCardPayment function to handle payment processing
      const { error, paymentIntent } = await stripe.confirmCardPayment(stripeClientSecret, {
        payment_method: {
          card: elements.getElement(CardElement),
        },
      });

      if (error) {
        set({ paymentMessage: error.message });
        toast.error(error.message);
      } else if (paymentIntent.status === "succeeded") {
          toast.success("Payment processed successfully!");
      }
    } catch (error) {
      set({ paymentMessage: 'An error occurred while processing the payment.' });
      toast.error('An error occurred while processing the payment.');
    } finally {
      set({ isPaymentProcessing: false });
    }
  },

  createStripePaymentIntent: async (currency,totalAfterDiscount) => {
    const { orderId } = get();
    if (!orderId) {
      toast.error("Order not found. Cannot create payment intent.");
      return;
    }

    try {
      // Create a Stripe payment intent on the backend
      const response = await axiosInstance.post("/create-payment-intent", {
        orderId,
```

```
        amount: totalAfterDiscount, // Send the amount after discount
        currency: currency,
      });

      if (response.data.clientSecret) {
        set({ stripeClientSecret: response.data.clientSecret });
      }

      toast.success("Stripe payment intent created successfully.");
    } catch (error) {
      toast.error("Failed to create payment intent. Please try again.");
    }
  },

  // Fetch all user coupons and include the cart items
  getMyCoupons: async () => {
    try {
      const { cart } = get(); // Get the current cart items
      if (!cart || cart.length === 0) {
        toast.error("Cart is empty. Unable to fetch coupons.");
        return;
      }

      const productIds = cart.map(item => item.products_id); // Extract the product IDs from cart
items
      const response = await axiosInstance.post(`/coupon`, { productIds });

      set({ userCoupons: response.data, isCouponApplied: false }); // Store the list of user
coupons
    } catch (error) {
      console.error("Error fetching coupons:", error);
    }
  },

  // Apply a coupon, update cart and totals
  applyCoupon: async (couponId) => {
    try {
      const { cart, orderId } = get(); // Get the current cart and orderId
      if (!cart || cart.length === 0) {
        toast.error("Cart is empty. Cannot apply coupon.");
        return;
      }

      const productIds = cart.map(item => item.products_id); // Extract the product IDs from cart
items

      // Send the coupon id, product IDs, and orderId to validate the coupon
      const response = await axiosInstance.post("/coupon/apply", { couponId, productIds, orderId
});

      // Update the state with the coupon data, and the updated cart and totals
      set({
        cart: response.data.order_items, // Update the cart with the new items (with the coupon
applied)
        orderId: response.data.order_id, // Ensure the orderId is updated correctly
        isCouponApplied: true, // Mark coupon as applied
        coupon: response.data.coupon, // Set the applied coupon
        totalAmount: response.data.total_amount, // Set the total amount
        discountAmount: response.data.discount_amount, // Set the discount amount
        totalAfterDiscount: response.data.total_after_discount, // Set the total after discount
      });

      toast.success("Coupon applied successfully");
    } catch (error) {
      console.error("Error applying coupon:", error);
      toast.error(error.response?.data?.message || "Failed to apply coupon. Please try again.");
```

```
    }
  },

  // Clear the cart, reset orderId, and reset coupon states
  clearCart: () => {
    set({
      cart: [], // Clear the cart
      orderId: null, // Reset the orderId
      isCouponApplied: false, // Reset coupon applied status
      coupon: null, // Reset coupon
      userCoupons: [], // Optionally clear user coupons
      totalAmount: 0, // Reset totalAmount
      discountAmount: 0, // Reset discountAmount
      totalAfterDiscount: 0, // Reset totalAfterDiscount
    });

    toast.success("Cart cleared successfully");
  },

  // Add a product to the cart
  addToCart: async (productId) => {
    try {
      const { orderId } = get(); // Get the current orderId
      const response = await axiosInstance.post("/cart", { productId, orderId }); // Send orderId
with productId
      const { orderId: newOrderId, orderItems, totalAmount } = response.data;

      set({
        orderId: newOrderId, // Update the orderId if changed
        cart: orderItems, // Update the cart with the new items
        isOrderChanged: !get().isOrderChanged, // Toggle the `isOrderChanged` flag
        totalAmount: totalAmount,
        totalAfterDiscount: totalAmount,
      });

      toast.success("Product added to cart");
    } catch (error) {
      console.error("Error adding product to cart:", error);
      toast.error(error.response?.data?.message || "Failed to add product to cart. Please try
again.");
    }
  },

  // Remove a product from the cart
  removeFromCart: async (productId) => {
    try {
      const { orderId } = get(); // Get the current orderId
      const response = await axiosInstance.delete("/cart", { data: { productId, orderId } }); //
Send data as part of request body
      const { orderId: newOrderId, orderItems, totalAmount } = response.data;

      set({
        orderId: newOrderId, // Update the orderId if changed
        cart: orderItems, // Update the cart with the new items
        isOrderChanged: !get().isOrderChanged, // Toggle the `isOrderChanged` flag
        totalAmount: totalAmount,
        totalAfterDiscount: totalAmount,
      });

      toast.success("Product removed from cart");
    } catch (error) {
      console.error("Error removing product from cart:", error);
      toast.error(error.response?.data?.message || "Failed to remove product from cart. Please try
again.");
    }
  },
```

```
  // Update quantity of a product in the cart
  updateQuantity: async (productId, isIncrement) => {
    try {
      const { orderId } = get(); // Get the current orderId
      const response = await axiosInstance.put("/cart/quantity", { productId, orderId, isIncrement
}); // Sending the isIncrement flag
      const { orderId: newOrderId, orderItems, totalAmount } = response.data;

      set({
        orderId: newOrderId, // Update the orderId if changed
        cart: orderItems, // Update the cart with the new items
        isOrderChanged: !get().isOrderChanged, // Toggle the `isOrderChanged` flag
        totalAmount: totalAmount,
        totalAfterDiscount:totalAmount
      });

      toast.success(isIncrement ? "Quantity increased" : "Quantity decreased");
    } catch (error) {
      console.error("Error updating quantity:", error);
      toast.error(error.response?.data?.message || "Failed to update quantity. Please try again.");
    }
  },
}));
```

FILE: frontend/src/stores/useChatBotStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Assuming you have axiosInstance configured
import { toast } from "react-hot-toast";

// Define the store
export const useChatBotStore = create((set) => ({
  chats: [], // Store the chat data here
  chat: null, // Store the current chat data
  isLoading: false,
  isError: false,
  errorMessage: "",
  chatbotContext: null, // Store the chatbot context data
  // Fetch user chats
  fetchUserChats: async () => {
    set({ isLoading: true, isError: false, errorMessage: "" });
    try {
      const response = await axiosInstance.get("/userchats");
      const chats = response.data;
      set({ chats });
    } catch (err) {
      const message = err.response?.data?.message || "Error fetching chats.";
      set({ isError: true, errorMessage: message });
      toast.error(message);
    } finally {
      set({ isLoading: false });
    }
  },

  // Create a new chat
  createChat: async (text, navigate) => {
    set({ isLoading: true, isError: false, errorMessage: "" });
    try {
      const response = await axiosInstance.post("/chats",{ text });
      const chat = response.data;
      set((state) => ({ chats: [...state.chats, chat] }));
      toast.success("Chat created successfully!");
      navigate(`/chatbot/chats/${chat.id}`);
    } catch (err) {
      const message = err.response?.data?.message || "Error creating chat.";
```

```
      set({ isError: true, errorMessage: message });
      toast.error(message);
    } finally {
      set({ isLoading: false });
    }
  },

  // Fetch a specific chat
  fetchChat: async (chatId) => {
    set({ isLoading: true, isError: false, errorMessage: "" });
    try {
      const response = await axiosInstance.get(`/chats/${chatId}`);
      const chat = response.data;
      set({ chat });
    } catch (err) {
      const message = err.response?.data?.message || "Error fetching chat.";
      set({ isError: true, errorMessage: message });
      toast.error(message);
    } finally {
      set({ isLoading: false });
    }
  },

  // Update an existing chat
  updateChat: async (chatId, question, answer) => {
    set({ isLoading: true, isError: false, errorMessage: "" });
    try {
      const response = await axiosInstance.put(`/chats/${chatId}`,{ question, answer });
      const updatedChat = response.data;
      toast.success("Chat updated successfully!");
    } catch (err) {
      const message = err.response?.data?.message || "Error updating chat.";
      set({ isError: true, errorMessage: message });
      toast.error(message);
      throw new Error(message);
    } finally {
      set({ isLoading: false });
    }
  },
  fetchChatbotContext: async () => {
    set({ isLoading: true, isError: false, errorMessage: "" });
    try {
        const response = await axiosInstance.get("/chatbot/context"); // Adjust API endpoint as
necessary
        const contextData = response.data;

        let cumulativeText = ""; // Initialize the cumulativeText

        // Arrays to store unique values
        const uniqueComments = new Set();
        const uniqueCoupons = new Set();
        const uniquePosts = new Set();
        const uniqueProducts = new Set();
        const uniqueRoles = new Set();

        // Check if contextData is an object and process it
        if (typeof contextData === "object" && contextData !== null) {
            // Loop through each key in the contextData object
            Object.keys(contextData).forEach((key) => {
                const data = contextData[key];

                // Process comments if available (limit to first 50)
                if (key === 'comments' && Array.isArray(data)) {
                    data.slice(0, 50).forEach((c) => { // Limit to first 50 comments
                        if (!uniqueComments.has(c.content)) {
                            uniqueComments.add(c.content);  // Add to the Set to ensure uniqueness
```

```
                            cumulativeText += " " + c.content;
                        }
                    });
                }

                // Process coupons if available (limit to first 50)
                if (key === 'coupons' && Array.isArray(data)) {
                    data.slice(0, 50).forEach((c) => { // Limit to first 50 coupons
                        if (!uniqueCoupons.has(c.discount)) {
                            uniqueCoupons.add(c.discount);  // Add to the Set to ensure uniqueness
                            cumulativeText += " " + c.discount;
                        }
                    });
                }

                // Process posts if available (limit to first 50)
                if (key === 'posts' && Array.isArray(data)) {
                    data.slice(0, 50).forEach((p) => { // Limit to first 50 posts
                        if (!uniquePosts.has(p.content)) {
                            uniquePosts.add(p.content);  // Add to the Set to ensure uniqueness
                            cumulativeText += " " + p.content;
                        }
                    });
                }

                // Process products if available (limit to first 50)
                if (key === 'products' && Array.isArray(data)) {
                    data.slice(0, 50).forEach((p) => { // Limit to first 50 products
                        if (!uniqueProducts.has(p.description)) {
                            uniqueProducts.add(p.description);  // Add to the Set to ensure
uniqueness
                            cumulativeText += " " + p.description;
                        }
                    });
                }

                // Process roles if available (limit to first 50)
                if (key === 'roles' && Array.isArray(data)) {
                    data.slice(0, 50).forEach((r) => { // Limit to first 50 roles
                        const roleDescription = r.role_name + " " + r.description;
                        if (!uniqueRoles.has(roleDescription)) {
                            uniqueRoles.add(roleDescription);  // Add to the Set to ensure
uniqueness
                            cumulativeText += " " + roleDescription;
                        }
                    });
                }
            });

            console.log(cumulativeText.trim());

            set({ chatbotContext: cumulativeText.trim() });

            toast.success("Chatbot context loaded successfully!");
        } else {
            // Handle the case where contextData is not a valid object
            toast.error("Invalid data format received from server.");
            console.error("Received invalid data format: ", contextData);
        }
        } catch (err) {
            const message = err.response?.data?.message || "Error fetching chatbot context.";
            set({ isError: true, errorMessage: message });
            toast.error(message);
        } finally {
            set({ isLoading: false });
        }
```

```
    },



  // Reset store state
  reset: () => set({
    chats: [],
    chat: null,
    isLoading: false,
    isError: false,
    errorMessage: "",
    chatbotContext: null, // Reset chatbot context as well
  }),
}));

export default useChatBotStore;
```

FILE: frontend/src/stores/useChatStore.js

```
// useChatStore.js
import { create } from "zustand";
import { toast } from "react-hot-toast";
import axiosInstance from "../lib/axios";

export const useChatStore = create((set, get) => ({
  conversations: [],
  selectedConversation: null,
  localMessages: [],
  noMoreMessages: false,
  scrollFromBottom: 0,
  isLoading: false,
  isError: false,
  errorMessage: '',

  // Group deletion handler
  deleteGroup: async (groupId) => {
    try {
      // Perform the API request to delete the group
      const response = await axiosInstance.delete(`/group/destroy/${groupId}`);
      toast.success("Group deleted successfully");

      // Optionally update state or fetch the updated list of conversations/groups
      set({ conversations: get().conversations.filter((conv) => conv.id !== groupId) });

      // Optionally emit an event for successful deletion (if needed)
      // emit("group.deleted", groupId);

    } catch (error) {
      const message = error.response?.data?.message || "Error deleting group.";
      toast.error(message);
    }
  },

  // Send message
  sendMessage: async (newMessage, chosenFiles, conversation) => {
    const { messageCreated } = get();

    // Prepare FormData for the API call
    const formData = new FormData();
    chosenFiles.forEach((file) => {
      formData.append("attachments[]", file.file);
    });
    formData.append("message", newMessage);
    if (conversation.is_user) {
      formData.append("receiver_id", conversation.id);
    } else if (conversation.is_group) {
```

```
      formData.append("group_id", conversation.id);
    }

    try {
      // Replace `route("message.store")` with the actual API endpoint
      const response = await axiosInstance.post("/message/store", formData, {
        onUploadProgress: (progressEvent) => {
          const progress = Math.round(
            (progressEvent.loaded / progressEvent.total) * 100
          );
          console.log(progress);
        },
      });

      messageCreated(response.data); // Update the messages in the store
      toast.success("Message sent successfully");
    } catch (error) {
      const message = error?.response?.data?.message || "An error occurred while sending the
message";
      toast.error(message);
    }
  },

  // Message deletion handler
  deleteMessage: async (messageId) => {
    set({ isLoading: true, isError: false, errorMessage: '' });
    try {
      // Perform the API request to delete the message
      await axiosInstance.delete(`/messages/${messageId}`);

      // Update the localMessages by removing the deleted message
      const { localMessages } = get();
      const updatedMessages = localMessages.filter((message) => message.id !== messageId);
      set({ localMessages: updatedMessages });

      toast.success("Message deleted successfully");

    } catch (error) {
      const message = error.response?.data?.message || "Error deleting message.";
      set({ isError: true, errorMessage: message });
      toast.error(message);
    } finally {
      set({ isLoading: false });
    }
  },

  createOrUpdateGroup: async (data, groupId = null) => {
    try {
      const response = groupId
        ? await axiosInstance.put(`/group/update/${groupId}`, data) // Update group
        : await axiosInstance.post("/group/store", data); // Create new group

      // Assuming response.data contains the group data
      toast.success(`Group "${data.name}" ${groupId ? 'updated' : 'created'}`);
      // Optionally update state with the new group data, e.g. by fetching updated conversations.
    } catch (err) {
      const message = err.response?.data?.message || "Error processing group request.";
      toast.error(message);
    }
  },
  // Fetch conversations
  fetchConversations: async () => {
    set({ isLoading: true, isError: false, errorMessage: '' });
    try {
      const response = await axiosInstance.get("/user/conversations");
      console.log(response.data)
```

```
      set({ conversations: response.data });
    } catch (error) {
      const message = error.response?.data?.message || "Error fetching conversations.";
      set({ isError: true, errorMessage: message });
      toast.error(message);
    } finally {
      set({ isLoading: false });
    }
  },

  // Fetch conversation data
  fetchConversationData: async (conversationId, isUser) => {
    set({ isLoading: true, isError: false, errorMessage: '' });
    try {
      // Assuming isUser should be passed as a query parameter
      const response = await axiosInstance.get(`/conversations/${conversationId}/messages`, {
        params: { isUser }  // Pass isUser as a query parameter
      });
      console.log(response.data)
      set({ localMessages: response.data.reverse()});
    } catch (error) {
      const message = error.response?.data?.message || "Error fetching conversation data.";
      set({ isError: true, errorMessage: message });
      toast.error(message);
    } finally {
      set({ isLoading: false });
    }
  },


  // Message creation handler
  messageCreated: (message) => {
    const { selectedConversation, localMessages } = get();
    if (selectedConversation) {
      if (
        selectedConversation.is_group && selectedConversation.id === message.group_id
      ) {
        set({ localMessages: [...localMessages, message] });
      } else if (
        selectedConversation.is_user &&
        (selectedConversation.id === message.sender_id || selectedConversation.id ===
message.receiver_id)
      ) {
        set({ localMessages: [...localMessages, message] });
      }
    }
  },

  // Message deletion handler
  messageDeleted: (message) => {
    const { selectedConversation, localMessages } = get();
    if (selectedConversation) {
      const updatedMessages = localMessages.filter(m => m.id !== message.id);
      set({ localMessages: updatedMessages });
    }
  },

  // Load more messages
  loadMoreMessages: async () => {
    const { localMessages, noMoreMessages } = get();
    if (noMoreMessages) return;

    const firstMessage = localMessages[0];
    try {
      const { data } = await axiosInstance.get(`/messages/load-older/${firstMessage.id}`);
      if (data.length === 0) {
```

```
        set({ noMoreMessages: true });
      } else {
        const scrollHeight = document.querySelector('.messages-container').scrollHeight;
        const scrollTop = document.querySelector('.messages-container').scrollTop;
        const clientHeight = document.querySelector('.messages-container').clientHeight;
        const tmpScrollFromBottom = scrollHeight - scrollTop - clientHeight;

        set({ scrollFromBottom: tmpScrollFromBottom });
        set({ localMessages: [...data.reverse(), ...localMessages] });
        toast.success('More messages loaded successfully');

      }
    } catch (error) {
      const message = error.response?.data?.message || "Error loading older messages.";
      set({ isError: true, errorMessage: message });
      toast.error(message);
    }
  },

  // Set selected conversation
  setSelectedConversation: (conversation) => {
    set({ selectedConversation: conversation, noMoreMessages: false, scrollFromBottom: 0 });
  },

  // Reset store (optional)
  reset: () => set({
    conversations: [],
    selectedConversation: null,
    localMessages: [],
    noMoreMessages: false,
    scrollFromBottom: 0,
    isLoading: false,
    isError: false,
    errorMessage: '',
  }),
}));

export default useChatStore;
```

FILE: frontend/src/stores/useCouponStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Ensure this is the correct path to your axiosInstance
import { toast } from "react-hot-toast";

export const useCouponStore = create((set, get) => ({
  // State for creating a coupon
  couponData: null, // Store the created coupon data
  isProcessingCoupon: false, // Flag to track if the coupon is being created
  isErrorCoupon: false, // Flag to track if there's an error during coupon creation
  errorMessageCoupon: "", // Error message if any
  isSuccessCoupon: false, // Flag to track if the coupon was created successfully

  // State for fetching user coupons
  userCoupons: [], // Store the user's coupons
  isProcessingGetCoupons: false, // Flag to track if fetching coupons is in progress
  isErrorGetCoupons: false, // Flag to track if there's an error while fetching coupons
  errorMessageGetCoupons: "", // Error message if any
  isSuccessGetCoupons: false, // Flag to track if the coupons were fetched successfully

  // Function to create a coupon and assign it to the authenticated user and products
  createCoupon: async (couponData) => {
    const { isProcessingCoupon, userCoupons } = get();

    // Prevent creating coupon if one is already being processed
    if (isProcessingCoupon) return;
```

```
    set({
      isProcessingCoupon: true,
      isErrorCoupon: false,
      errorMessageCoupon: "",
      isSuccessCoupon: false,
    });

    try {
      // Send the coupon data to the backend to create it
      const response = await axiosInstance.post("/create-coupon", couponData); // Replace with your
backend URL

      set({
        couponData: response.data, // Store the created coupon data
        isProcessingCoupon: false,
        isSuccessCoupon: true,
        userCoupons: [...userCoupons, response.data], // Correctly add the new coupon to the list
      });

      toast.success("Coupon created successfully!");
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error creating coupon";
      set({
        isProcessingCoupon: false,
        isErrorCoupon: true,
        errorMessageCoupon: errorMessage,
      });

      toast.error(errorMessage);
    }
  },

  // Function to fetch the coupons of the authenticated user
  getUserCoupons: async () => {
    const { isProcessingGetCoupons } = get();

    // Prevent fetching if already in progress
    if (isProcessingGetCoupons) return;

    set({
      isProcessingGetCoupons: true,
      isErrorGetCoupons: false,
      errorMessageGetCoupons: "",
      isSuccessGetCoupons: false,
    });

    try {
      // Send request to fetch user's coupons from the backend
      const response = await axiosInstance.get("/get-my-coupons"); // Replace with your backend URL

      set({
        userCoupons: response.data, // Store the fetched coupons
        isProcessingGetCoupons: false,
        isSuccessGetCoupons: true,
      });

      toast.success("Coupons fetched successfully!");
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error fetching coupons";
      set({
        isProcessingGetCoupons: false,
        isErrorGetCoupons: true,
        errorMessageGetCoupons: errorMessage,
      });
```

```
      toast.error(errorMessage);
    }
  },

  // Function to reset the state
  reset: () => set({
    couponData: null,
    isProcessingCoupon: false,
    isErrorCoupon: false,
    errorMessageCoupon: "",
    isSuccessCoupon: false,

    userCoupons: [],
    isProcessingGetCoupons: false,
    isErrorGetCoupons: false,
    errorMessageGetCoupons: "",
    isSuccessGetCoupons: false,
  }),
}));

export default useCouponStore;
```

FILE: frontend/src/stores/useDeliveryStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Assuming you have axiosInstance configured
import { toast } from "react-hot-toast";

export const useDeliveryStore = create((set, get) => ({
  shipments: [],
  vehicle: null,  // Store the vehicle data here
  isLoading: false,
  isError: false,
  errorMessage: '',

  // Function to fetch shipments with related route optimizations, route details, and route
conditions
  fetchShipmentsWithRouteDetails: async () => {
    set({ isLoading: true, isError: false, errorMessage: '' }); // Start loading
    try {
      const response = await axiosInstance.get('/shipments/with-details'); // Assuming this route
exists on the backend
      const shipments = response.data; // Destructure shipments from the response
      console.log(shipments);
      set({
        shipments: shipments, // Store shipments in the state
      });
    } catch (err) {
      const message = err.response?.data?.message || 'Error fetching shipments.';
      set({ isError: true, errorMessage: message }); // Set error state
      toast.error(message); // Display error toast
    } finally {
      set({ isLoading: false }); // Set loading state to false
    }
  },

  // Function to save the vehicle ID (consistent with other actions in the store)
  handleSaveVehicleId: async (routeDetailId, assignedVehicleId) => {
    set({ isLoading: true, isError: false, errorMessage: '' }); // Start loading
    try {
      await axiosInstance.put(`/route-details/${routeDetailId}`, { vehicle_id: assignedVehicleId
});
      toast.success('Vehicle assigned successfully');
    } catch (error) {
      const message = error.response?.data?.message || 'Error updating vehicle ID.';
      set({ isError: true, errorMessage: message }); // Set error state
```

```
      toast.error(message); // Display error toast
    } finally {
      set({ isLoading: false }); // Set loading state to false
    }
  },

  // Function to fetch the vehicle for the authenticated user
  getVehicleForAuthenticatedUser: async () => {
    set({ isLoading: true, isError: false, errorMessage: '' }); // Start loading
    try {
      const response = await axiosInstance.get('/user/vehicle'); // Assuming this is the API
endpoint
      const vehicle = response.data.vehicle; // Extract vehicle from the response
      set({
        vehicle: vehicle, // Store vehicle data in the state
      });
      toast.success('Vehicle fetched successfully');
    } catch (error) {
      const message = error.response?.data?.message || 'Error fetching vehicle.';
      set({ isError: true, errorMessage: message }); // Set error state
      toast.error(message); // Display error toast
    } finally {
      set({ isLoading: false }); // Set loading state to false
    }
  },

  // Function to reset the store values (if needed)
  reset: () => set({
    shipments: [],
    vehicle: null, // Reset vehicle data as well
    isLoading: false,
    isError: false,
    errorMessage: '',
  }),
}));

export default useDeliveryStore;
```

FILE: frontend/src/stores/useEditorStore.js

```
import { create } from "zustand";
import { toast } from "sonner";
import axiosInstance from "../lib/axios";

export const useEditorStore = create((set, get) => ({
  layers: [],
  layerComparisonMode: false,
  comparedLayers: [],
  tags: [],
  activeTag: '',
  activeColor: '',
  activeLayer: {
    id: '',
    url: '',
    width: 0,
    height: 0,
    name: "default",
    publicId: "",
    format: "",
    resourceType: "default",
    tags: [],
  },
  removingBackgroundGenerating: false,
  replacingBackgroundGenerating: false,
  extractingImageGenerating: false,
  genFillingGenerating: false,
```

```
  genRemovingGenerating: false,
  recoloringImageGenerating: false,
  croppingVideoGenerating: false,
  transcribingGenerating: false,
  uploadingImageGenerating: false,
  uploadingVideoGenerating: false,

  removeBackgroundError: null,
  replaceBackgroundError: null,
  extractImageError: null,
  genFillError: null,
  genRemoveError: null,
  recolorImageError: null,
  cropVideoError: null,
  initiateTranscriptionError: null,
  uploadImageError: null,
  uploadVideoError: null,

  // General state setters
  setTags: (tags) => set({ tags }),
  setActiveTag: (tag) => set({ activeTag: tag }),
  setActiveColor: (color) => set({ activeColor: color }),

  // Layer management
  addLayer: (layer) => set((state) => ({ layers: [...state.layers, { ...layer }] })),
  removeLayer: (id) => set((state) => ({ layers: state.layers.filter((l) => l.id !== id) })),
  setActiveLayer: (id) => set((state) => ({ activeLayer: state.layers.find((l) => l.id === id) ||
state.layers[0] })),
  updateLayer: (layer) => set((state) => ({ layers: state.layers.map((l) => (l.id === layer.id ?
layer : l)) })),
  setPoster: (id, posterUrl) => set((state) => ({ layers: state.layers.map((l) => (l.id === id ? {
...l, poster: posterUrl } : l)) })),
  setTranscription: (id, transcriptionURL) => set((state) => ({ layers: state.layers.map((l) =>
(l.id === id ? { ...l, transcriptionURL } : l)) })),

  // Layer comparison
  setLayerComparisonMode: (mode) => set(() => ({ layerComparisonMode: mode, comparedLayers: mode ?
[] : [] })),
  setComparedLayers: (layers) => set(() => ({ comparedLayers: layers, layerComparisonMode:
layers.length > 0 })),
  toggleComparedLayer: (id) => set((state) => {
    const newComparedLayers = state.comparedLayers.includes(id)
      ? state.comparedLayers.filter((layerId) => layerId !== id)
      : [...state.comparedLayers, id].slice(-2);
    return { comparedLayers: newComparedLayers, layerComparisonMode: newComparedLayers.length > 0
};
  }),

  // Remove background
  removeBackground: async (imageUrl, format, id) => {
    set({ removingBackgroundGenerating: true, removeBackgroundError: null });
    try {
      const response = await axiosInstance.post("/remove-background", { activeImage: imageUrl,
format });
      if (response.data.success) {
        toast.success("Background removed successfully!");
        set({ activeLayer: { ...response.data.success, id } });
      } else {
        set({ removeBackgroundError: "Failed to remove background" });
        toast.error("Failed to remove background");
      }
    } catch (e) {
      set({ removeBackgroundError: "Error removing background" });
      toast.error("Error removing background");
      console.error("Error:", e);
    } finally {
```

```
      set({ removingBackgroundGenerating: false });
    }
  },

  // Replace background
  replaceBackground: async (imageUrl, prompt = '', id) => {
    set({ replacingBackgroundGenerating: true, replaceBackgroundError: null });
    try {
      const response = await axiosInstance.post("/replace-background", { activeImage: imageUrl,
prompt });
      if (response.data.success) {
        toast.success("Background replaced successfully!");
        set({ activeLayer: { ...response.data.success, id } });
      } else {
        set({ replaceBackgroundError: "Failed to replace background" });
        toast.error("Failed to replace background");
      }
    } catch (e) {
      set({ replaceBackgroundError: "Error replacing background" });
      toast.error("Error replacing background");
      console.error("Error:", e);
    } finally {
      set({ replacingBackgroundGenerating: false });
    }
  },

  // Extract objects
  extractImage: async (imageUrl, prompts, multiple = false, mode = 'default', invert = false,
format = 'jpg', id) => {
    set({ extractingImageGenerating: true, extractImageError: null });
    try {
      const response = await axiosInstance.post("/extract-image", { activeImage: imageUrl, prompts,
multiple, mode, invert, format });
      if (response.data.success) {
        toast.success("Image extraction completed!");
        set({ activeLayer: { ...response.data.success, id } });
      } else {
        set({ extractImageError: "Failed to extract objects from image" });
        toast.error("Failed to extract objects from image");
      }
    } catch (e) {
      set({ extractImageError: "Error extracting image" });
      toast.error("Error extracting image");
      console.error("Error:", e);
    } finally {
      set({ extractingImageGenerating: false });
    }
  },

  // Generative fill
  genFill: async (imageUrl, aspect, width, height, id) => {
    set({ genFillingGenerating: true, genFillError: null });
    try {
      const response = await axiosInstance.post("/gen-fill", { activeImage: imageUrl, aspect,
width, height });
      if (response.data.success) {
        toast.success("Generative fill completed!");
        set({ activeLayer: { ...response.data.success, id } });
      } else {
        set({ genFillError: "Failed to generate fill" });
        toast.error("Failed to generate fill");
      }
    } catch (e) {
      set({ genFillError: "Error generating fill" });
      toast.error("Error generating fill");
      console.error("Error:", e);
```

```
    } finally {
      set({ genFillingGenerating: false });
    }
  },

  // Generative remove
  genRemove: async (imageUrl, prompt, id) => {
    set({ genRemovingGenerating: true, genRemoveError: null });
    try {
      const response = await axiosInstance.post("/gen-remove", { activeImage: imageUrl, prompt });
      if (response.data.success) {
        toast.success("Generative remove completed!");
        set({ activeLayer: { ...response.data.success, id } });
      } else {
        set({ genRemoveError: "Failed to generate remove" });
        toast.error("Failed to generate remove");
      }
    } catch (e) {
      set({ genRemoveError: "Error generating remove" });
      toast.error("Error generating remove");
      console.error("Error:", e);
    } finally {
      set({ genRemovingGenerating: false });
    }
  },

  // Recolor image
  recolorImage: async (imageUrl, tag, color, id) => {
    set({ recoloringImageGenerating: true, recolorImageError: null });
    try {
      const response = await axiosInstance.post("/recolor-image", { activeImage: imageUrl, tag,
color });
      if (response.data.success) {
        toast.success("Image recolored successfully!");
        set({ activeLayer: { ...response.data.success, id } });
      } else {
        set({ recolorImageError: "Failed to recolor image" });
        toast.error("Failed to recolor image");
      }
    } catch (e) {
      set({ recolorImageError: "Error recoloring image" });
      toast.error("Error recoloring image");
      console.error("Error:", e);
    } finally {
      set({ recoloringImageGenerating: false });
    }
  },

  // Crop video
  cropVideo: async (videoUrl, aspect, height, id) => {
    set({ croppingVideoGenerating: true, cropVideoError: null });
    try {
      const response = await axiosInstance.post("/crop-video", { activeVideo: videoUrl, aspect,
height });
      if (response.data.success) {
        toast.success("Video cropped successfully!");
        set({ activeLayer: { ...response.data.success, id } });
      } else {
        set({ cropVideoError: "Failed to crop video" });
        toast.error("Failed to crop video");
      }
    } catch (e) {
      set({ cropVideoError: "Error cropping video" });
      toast.error("Error cropping video");
      console.error("Error:", e);
    } finally {
```

```
      set({ croppingVideoGenerating: false });
    }
  },

  // Initiate transcription
  initiateTranscription: async (publicId, id) => {
    set({ transcribingGenerating: true, initiateTranscriptionError: null });
    try {
      const response = await axiosInstance.post("/initiate-transcription", { publicId });
      if (response.data.success) {
        toast.success("Transcription initiated successfully!");
        set({ activeLayer: { ...response.data.success, id } });
      } else {
        set({ initiateTranscriptionError: "Failed to initiate transcription" });
        toast.error("Failed to initiate transcription");
      }
    } catch (e) {
      set({ initiateTranscriptionError: "Error initiating transcription" });
      toast.error("Error initiating transcription");
      console.error("Error:", e);
    } finally {
      set({ transcribingGenerating: false });
    }
  },

  // Upload image
  uploadImage: async (imageFile, id) => {
    set({ uploadingImageGenerating: true, uploadImageError: null });
    try {
      const formData = new FormData();
      formData.append('image', imageFile);
      const response = await axiosInstance.post("/upload-image", formData);
      if (response.data.success) {
        toast.success("Image uploaded successfully!");
        set({ activeLayer: { ...response.data.success, id } });
      } else {
        set({ uploadImageError: "Failed to upload image" });
        toast.error("Failed to upload image");
      }
    } catch (e) {
      set({ uploadImageError: "Error uploading image" });
      toast.error("Error uploading image");
      console.error("Error:", e);
    } finally {
      const currentState = get(); // if using Zustand, replace with correct getter
      console.log(currentState.activeLayer);
      set({ uploadingImageGenerating: false });
    }
  },

  // Upload video
  uploadVideo: async (videoFile, id) => {
    set({ uploadingVideoGenerating: true, uploadVideoError: null });
    try {
      const formData = new FormData();
      formData.append("video", videoFile);
      const response = await axiosInstance.post("/upload-video", formData);
      if (response.data.success) {
        toast.success("Video uploaded successfully!");
        set({ activeLayer: { ...response.data.success, id } });
      } else {
        set({ uploadVideoError: "Failed to upload video" });
        toast.error("Failed to upload video");
      }
    } catch (e) {
      set({ uploadVideoError: "Error uploading video" });
```

```
      toast.error("Error uploading video");
      console.error("Error:", e);
    } finally {
      set({ uploadingVideoGenerating: false });
    }
  },
}));
```

FILE: frontend/src/stores/useExpensesStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Ensure this is the correct path to your axiosInstance
instance
import { toast } from "react-hot-toast";

export const useExpensesStore = create((set, get) => ({
  expenses: [],
  isLoading: false,
  isError: false,
  errorMessage: "",
  startDate: "",
  endDate: "",

  fetchExpenses: async () => {
    set({ isLoading: true, isError: false, errorMessage: "" });

    const { startDate, endDate } = get();
    console.log("Start Date:", startDate);
    console.log("End Date:", endDate);

    try {
      const response = await axiosInstance.post("/expense", {
        start_date: startDate || undefined,
        end_date: endDate || undefined,
      });

      // Convert the response data (which is an object) into an array
      const expensesArray = Object.values(response.data).map(item => ({
        name: item.name,
        amount: item.amount,
      }));

      set({ expenses: expensesArray, isLoading: false });
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Failed to fetch expenses";
      set({ isLoading: false, isError: true, errorMessage });
      toast.error(errorMessage);
    }
  },

  setStartDate: (date) => set({ startDate: date }),
  setEndDate: (date) => set({ endDate: date }),
  resetFilters: () => set({ startDate: "", endDate: "", expenses: [] }),
}));
```

FILE: frontend/src/stores/useFeedbackFormStore.js

```
import { create } from "zustand";
import { toast } from "react-hot-toast";
import axiosInstance from "../lib/axios";

export const useFeedbackFormStore = create((set, get) => ({
  currentFeedbackForm: null,
  isLoadingForm: false,
  isError: false,
```

```
  errorMessage: "",
  questionTypes: ['text', "select", "radio", "checkbox", "textarea"],
  feedbackForms: [],
  // Get all feedback forms associated with a specific order
  getFeedbackFormsForOrder: async (orderId) => {
    set({ isLoadingForm: true, isError: false, errorMessage: "" });

    try {
      const response = await axiosInstance.get(`/feedback-forms/${orderId}`);
      console.log(response.data.feedback_forms);
      set({
        feedbackForms: response.data.feedback_forms, // Assuming the response includes a
'feedback_forms' array
        isLoadingForm: false,
      });
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Failed to fetch feedback forms for
this order";
      set({ isLoadingForm: false, isError: true, errorMessage });
      toast.error(errorMessage);
    }
  },
  // Create a new feedback form
  createFeedbackForm: async (feedbackForm) => {
    set({ isLoadingForm: true, isError: false, errorMessage: "" });

    try {
      const response = await axiosInstance.post("/feedback-forms", feedbackForm);
      toast.success("Feedback form created successfully!");
      set({ isLoadingForm: false });
      return response.data; // Return the created feedback form
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Failed to create feedback form";
      set({ isLoadingForm: false, isError: true, errorMessage });
      toast.error(errorMessage);
    }
  },

  // Update a feedback form by ID
  updateFeedbackForm: async (id, feedbackForm) => {
    set({ isLoadingForm: true, isError: false, errorMessage: "" });

    try {
      const response = await axiosInstance.put(`/feedback-forms/${id}`, feedbackForm);
      toast.success("Feedback form updated successfully!");
      set({ isLoadingForm: false });
      return response.data;
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Failed to update feedback form";
      set({ isLoadingForm: false, isError: true, errorMessage });
      toast.error(errorMessage);
    }
  },

  // Submit answers to a feedback form
  submitFeedbackForm: async (id, answers) => {
    set({ isLoadingForm: true, isError: false, errorMessage: "" });

    try {
      await axiosInstance.post(`/feedback-forms/${id}/answer`, { answers });
      toast.success("Feedback submitted successfully!");
      set({ isLoadingForm: false });
      return true;
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Failed to submit feedback";
      set({ isLoadingForm: false, isError: true, errorMessage });
```

```
      toast.error(errorMessage);
      return false;
    }
  },


  // Reset all store data
  resetStore: () => set({ currentFeedbackForm: null, isLoadingForm: false, isError: false,
errorMessage: "" }),
}));
```

FILE: frontend/src/stores/useGeographyStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Ensure this is the correct path to your axiosInstance
instance
import { toast } from "react-hot-toast";

export const useGeographyStore = create((set, get) => ({
  geographyData: [],   // To store aggregated warehouse count by country
  isLoading: false,     // To track loading state
  isError: false,       // To track error state
  errorMessage: "",     // Error message

  // Fetch geography data
  fetchGeography: async () => {
    set({ isLoading: true, isError: false, errorMessage: "" });

    try {
      // Make the API request to fetch data
      const response = await axiosInstance.get("/warehouse/geography");
      console.log(response.data);
      // Check if the response data is valid and structure it correctly
      if (response.data && Array.isArray(response.data)) {
        set({ geographyData: response.data, isLoading: false });
      } else {
        throw new Error("Invalid data format received from server");
      }
    } catch (error) {
      // Handle the error gracefully
      let errorMessage = "Failed to fetch geography data";

      // Check if the error is related to the response
      if (error.response) {
        errorMessage = error.response?.data?.message || errorMessage;
      } else if (error.message) {
        // Handle cases where the error is thrown manually (e.g., invalid data format)
        errorMessage = error.message;
      }

      // Update the state with the error and show a toast notification
      set({ isLoading: false, isError: true, errorMessage });
      toast.error(errorMessage);
    }
  },
}));
```

FILE: frontend/src/stores/useGiftStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios";
import { toast } from "react-hot-toast";

export const useGiftStore = create((set, get) => ({
  stripeClientSecret: "", // Stores the Stripe client secret for the payment
```

```
  isPaymentProcessing: false, // State to manage if the payment is being processed

  createStripePaymentIntent: async (currency, totalAfterDiscount) => {
    try {
      set({ isPaymentProcessing: true }); // Set isPaymentProcessing to true when the request
starts

      // Create a Stripe payment intent on the backend
      const response = await axiosInstance.post("/create-payment-intent-gift", {
        amount: totalAfterDiscount, // Send the amount after discount
        currency: currency,
      });

      if (response.data.clientSecret) {
        set({ stripeClientSecret: response.data.clientSecret });
      }

      toast.success("Stripe payment intent created successfully.");
    } catch (error) {
      toast.error("Failed to create payment intent. Please try again.");
    } finally {
      set({ isPaymentProcessing: false }); // Set isPaymentProcessing to false when done or on
error
    }
  },
}));
```

FILE: frontend/src/stores/useInventoryStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios";
import { toast } from "react-hot-toast";

export const useInventoryStore = create((set, get) => ({
  isSubmitting: false,
  isError: false,
  stock: 0,
  weightPerUnit: 0,
  productDetails: {
    name: '',
    description: '',
    price: '',
    image: null,
    category: '',
    isFeatured: false,
  },
  warehouse: {
    totalWeight: 0,
    capacity: 0,
  },
  setProductDetails: (details) => set({ productDetails: { ...get().productDetails, ...details } }),
  setInventoryDetails: (details) => set({ ...details }),

  // Function to create inventory
  createInventory: async () => {
    set({ isSubmitting: true });
    const { productDetails, stock, weightPerUnit } = get();

    const formData = new FormData();
    // Add product details to FormData
    Object.keys(productDetails).forEach((key) => {
      formData.append(`product[${key}]`, productDetails[key]);
    });
    formData.append('inventory[stock]', stock);
    formData.append('inventory[weightPerUnit]', weightPerUnit);
    if (productDetails.image) {
```

```
      formData.append('product[image]', productDetails.image);
    }
    try {
      const response = await axiosInstance.post('/warehouse/create-inventory', formData);
      set({
        stock: 0,
        weightPerUnit: 0,
        productDetails: {
          name: '', description: '', price: '', image: null, category: '', isFeatured: false
        },
      });
      toast.success('Product and inventory created successfully!');
    } catch (err) {
      const errorMessage = err.response?.data?.message || 'Error creating product and inventory.';
      toast.error(errorMessage);
    } finally {
      set({ isSubmitting: false });
    }
  },

  // Function to get the total inventory weight and warehouse capacity
  getTotalInventoryWeightForUserWarehouse: async () => {
    set({ isSubmitting: true });
    try {
      const response = await axiosInstance.get('/warehouse/capacity');
      const { total_weight, capacity } = response.data;

      // Update the store with the fetched warehouse data
      set({
        warehouse: {
          totalWeight: total_weight,
          capacity: capacity,
        },
      });

    } catch (err) {
      const errorMessage = err.response?.data?.message || 'Error fetching warehouse data.';
      toast.error(errorMessage);
    } finally {
      set({ isSubmitting: false });
    }
  },

  // Function to reset store values
  reset: () => set({
    productDetails: { name: '', description: '', price: '', image: null, category: '', isFeatured:
false },
    stock: 0,
    weightPerUnit: 0,
    warehouse: { totalWeight: 0, capacity: 0 },
  }),
}));

export default useInventoryStore;
```

FILE: frontend/src/stores/useLicensePlateStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios";
import { toast } from "react-hot-toast"; // Assuming react-hot-toast is used for toast
notifications

export const useLicensePlateStore = create((set) => ({
  licensePlates: [], // Array to store multiple license plates
  isLoading: false,
  isError: false,
```

```
  errorMessage: '',

  // Function to process the image and get the license plates
  processLicensePlate: async (image) => {
    set({ isLoading: true, isError: false, errorMessage: '' }); // Start loading

    try {
      // Create FormData to send the image as a file to the backend
      const formData = new FormData();
      formData.append('image', dataURLtoFile(image, 'captured.jpg')); // Append image as 'image'
field

      // Make the POST request to the backend (Laravel)
      const response = await axiosInstance.post('/process-license-plate',formData);

      // Assuming the backend returns a response with the 'license_plate_results' array
      const { license_plate_results } = response.data;
      if (license_plate_results && license_plate_results.length > 0) {
        set({ licensePlates: license_plate_results });
      } else {
        set({ licensePlates: [] });
      }
    } catch (error) {
      console.error('Error calling backend:', error);
      set({ isError: true, errorMessage: 'Error with OCR' });
      toast.error('Error processing the image');
    } finally {
      set({ isLoading: false }); // End loading
    }
  },

  // Reset the state if needed
  reset: () => set({ licensePlates: [], isLoading: false, isError: false, errorMessage: '' }),
}));

// Utility function to convert Base64 image to File
const dataURLtoFile = (dataurl, filename) => {
  const [meta, base64] = dataurl.split(',');
  const mime = meta.match(/:(.*?);/)[1];
  const byteString = atob(base64);
  const ab = new ArrayBuffer(byteString.length);
  const ia = new Uint8Array(ab);
  for (let i = 0; i < byteString.length; i++) {
    ia[i] = byteString.charCodeAt(i);
  }
  return new File([ab], filename, { type: mime });
};

export default useLicensePlateStore;
```

FILE: frontend/src/stores/useLineChartStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Ensure this is the correct path to your axiosInstance
instance
import { toast } from "react-hot-toast";

// Utility function to format date to 'yyyy-mm-dd'
const formatDate = (date) => {
  const d = new Date(date);
  const year = d.getFullYear();
  const month = String(d.getMonth() + 1).padStart(2, '0'); // Ensure two digits
  const day = String(d.getDate()).padStart(2, '0'); // Ensure two digits
  return `${year}-${month}-${day}`;
};
```

```
export const useLineChartStore = create((set, get) => ({
  data: [],
  loading: false,
  isError: false,
  errorMessage: "",
  totalShipmentAmount: 0,
  totalMaintenanceCost: 0,
  totalProductExpenses: 0,
  startDate: null,
  endDate: null,

  // Fetch line chart data
  fetchLineChartData: async (startDate = null, endDate = null) => {
    set({
      loading: true,
      isError: false,
      errorMessage: "",
      startDate,
      endDate,
    });

    try {
      const response = await axiosInstance.post("/warehouse/line-chart", { start_date: startDate,
end_date: endDate });

      // Group the data by date (ensure dates are formatted as yyyy-mm-dd)
      const groupedData = response.data.reduce((acc, item) => {
        const date = formatDate(item.date); // Format date to 'yyyy-mm-dd'

        // Initialize the accumulator object for the date if it doesn't exist
        if (!acc[date]) {
          acc[date] = {
            date,
            total_shipment_amount: 0,
            total_maintenance_cost: 0,
            total_product_expenses: 0,
          };
        }

        // Accumulate the values for each date
        acc[date].total_shipment_amount += item.total_shipment_amount;
        acc[date].total_maintenance_cost += item.total_maintenance_cost;
        acc[date].total_product_expenses += item.total_product_expenses;

        return acc;
      }, {});

      // Convert the object back to an array and accumulate totals
      const accumulatedData = Object.values(groupedData).map((item, index, arr) => {
        const previousItem = arr[index - 1] || {
          total_shipment_amount: 0,
          total_maintenance_cost: 0,
          total_product_expenses: 0
        };

        return {
          ...item,
          total_shipment_amount: previousItem.total_shipment_amount + item.total_shipment_amount,
          total_maintenance_cost: previousItem.total_maintenance_cost +
item.total_maintenance_cost,
          total_product_expenses: previousItem.total_product_expenses +
item.total_product_expenses,
        };
      });

      set({
```

```
        data: accumulatedData,
        loading: false,
        totalShipmentAmount: accumulatedData[accumulatedData.length - 1]?.total_shipment_amount ||
0,
        totalMaintenanceCost: accumulatedData[accumulatedData.length - 1]?.total_maintenance_cost
|| 0,
        totalProductExpenses: accumulatedData[accumulatedData.length - 1]?.total_product_expenses
|| 0,
      });
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Failed to fetch line chart data";
      set({ loading: false, isError: true, errorMessage });
      toast.error(errorMessage);
    }
  },

  // Reset the store to its initial state
  reset: () => set({
    data: [],
    loading: false,
    isError: false,
    errorMessage: "",
    totalShipmentAmount: 0,
    totalMaintenanceCost: 0,
    totalProductExpenses: 0,
    startDate: null,
    endDate: null,
  }),

}));

export default useLineChartStore;
```

FILE: frontend/src/stores/useLiveStreamStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Ensure this is the correct path to your axiosInstance
import { toast } from "react-hot-toast";

export const useLiveStreamStore = create((set, get) => ({
  // Block-related state
  blockedUsers: null,
  isProcessingBlock: false,
  isErrorBlock: false,
  errorMessageBlock: "",

  // Follow-related state
  followedUsers: null,
  isProcessingFollow: false,
  isErrorFollow: false,
  errorMessageFollow: "",

  // Stream-related state
  streamData: null, // Store a single stream
  streams: [], // Store many streams
  isProcessingStream: false,
  isErrorStream: false,
  errorMessageStream: "",


  isStoppingStream: false,
  isErrorStoppingStream: false,
  errorMessageStoppingStream: "",

  // Update-related state
  isProcessingUpdate: false,
```

```
  isErrorUpdate: false,
  errorMessageUpdate: "",

  isBlocked: false,
  viewerToken:"",
  // Function to create a new stream
  createStream: async (formData) => {
    const { isProcessingStream } = get();
    if (isProcessingStream) return;

    set({ isProcessingStream: true, isErrorStream: false, errorMessageStream: "" });

    try {
      // Modify formData for boolean fields before sending it to the server
      const booleanKeys = ['isChatEnabled', 'isChatDelayed', 'isChatFollowersOnly'];

      for (let [key, value] of formData.entries()) {
        // Check if the key is one of the boolean keys
        if (booleanKeys.includes(key)) {
          // Convert the value to a boolean (true or false) before setting it back in FormData
          const booleanValue = value === 'true' ? true : (value === 'false' ? false : value);
          formData.set(key, booleanValue); // Update the form data with the correct boolean value
        }
      }

      // Iterate over FormData and log key-value pairs
      for (let [key, value] of formData.entries()) {
        console.log(`${key}:`, value);
      }
      // Send the modified formData
      const response = await axiosInstance.post("/create-stream", formData);

      set({ streams: [...get().streams, response.data], isProcessingStream: false });
      toast.success("Stream created successfully!");
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error creating stream";
      set({ isProcessingStream: false, isErrorStream: true, errorMessageStream: errorMessage });
      toast.error(errorMessage);
    }
  },

  stopStream: async (streamId = null) => {
    const { isProcessingStream } = get();
    if (isProcessingStream) return;

    set({ isStoppingStream: true, isErrorStoppingStream: false, errorMessageStoppingStream: "" });

    try {
      // Make the API request with the optional streamId parameter
      const requestPayload = streamId ? { stream_id: streamId } : {}; // If streamId is passed,
include it in the request
      const response = await axiosInstance.post("/stop-stream", requestPayload);

      // Filter out the stopped stream from the list of active streams
      set({
        streams: get().streams.filter(stream => stream.id !== response.data.id),
        isStoppingStream: false,
      });

      toast.success("Stream stopped successfully!");
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error stopping stream";
      set({ isStoppingStream: false, isErrorStream: true, errorMessageStoppingStream: errorMessage
});
      toast.error(errorMessage);
    }
```

```
  },


  // Function to update user's headline and about
  updateUserHeadlineAndAbout: async (headline, about) => {
    const { isProcessingUpdate } = get();
    if (isProcessingUpdate) return;

    set({ isProcessingUpdate: true, isErrorUpdate: false, errorMessageUpdate: "" });

    try {
      const response = await axiosInstance.put("/user/headline-about", {
        headline,
        about,
      });
      set({ isProcessingUpdate: false });
      toast.success("Profile updated successfully!");
      return response.data; // Optionally return updated user data if needed
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error updating profile";
      set({ isProcessingUpdate: false, isErrorUpdate: true, errorMessageUpdate: errorMessage });
      toast.error(errorMessage);
    }
  },

  // Function to block a user
  blockUser: async (id) => {
    const { isProcessingBlock } = get();
    if (isProcessingBlock) return;

    set({ isProcessingBlock: true, isErrorBlock: false, errorMessageBlock: "" });

    try {
      const response = await axiosInstance.post(`/block/${id}`);
      set({ blockedUsers: response.data, isProcessingBlock: false });
      toast.success("User blocked successfully!");
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error blocking user";
      set({ isProcessingBlock: false, isErrorBlock: true, errorMessageBlock: errorMessage });
      toast.error(errorMessage);
    }
  },

  // Function to unblock a user
  unblockUser: async (id) => {
    const { isProcessingBlock } = get();
    if (isProcessingBlock) return;

    set({ isProcessingBlock: true, isErrorBlock: false, errorMessageBlock: "" });

    try {
      const response = await axiosInstance.post(`/unblock/${id}`);
      set({ blockedUsers: response.data, isProcessingBlock: false });
      toast.success("User unblocked successfully!");
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error unblocking user";
      set({ isProcessingBlock: false, isErrorBlock: true, errorMessageBlock: errorMessage });
      toast.error(errorMessage);
    }
  },

  // Function to get the list of blocked users
  getBlockedUsers: async () => {
    try {
      const response = await axiosInstance.get("/blocked-users");
      set({ blockedUsers: response.data });
```

```
    } catch (error) {
      toast.error("Error fetching blocked users");
    }
  },

  // Function to check if a user is blocked
  isBlockedByUser: async (id) => {
    try {
      const response = await axiosInstance.get(`/is-blocked/${id}`);
      set({ isBlocked: response.data });
    } catch (error) {
      toast.error("Error checking if user is blocked");
    }
  },

  // Function to follow a user
  followUser: async (id) => {
    const { isProcessingFollow } = get();
    if (isProcessingFollow) return;

    set({ isProcessingFollow: true, isErrorFollow: false, errorMessageFollow: "" });

    try {
      const response = await axiosInstance.post(`/follow/${id}`);
      set({ followedUsers: response.data, isProcessingFollow: false });
      toast.success("User followed successfully!");
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error following user";
      set({ isProcessingFollow: false, isErrorFollow: true, errorMessageFollow: errorMessage });
      toast.error(errorMessage);
    }
  },

  // Function to unfollow a user
  unfollowUser: async (id) => {
    const { isProcessingFollow } = get();
    if (isProcessingFollow) return;

    set({ isProcessingFollow: true, isErrorFollow: false, errorMessageFollow: "" });

    try {
      const response = await axiosInstance.post(`/unfollow/${id}`);
      set({ followedUsers: response.data, isProcessingFollow: false });
      toast.success("User unfollowed successfully!");
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error unfollowing user";
      set({ isProcessingFollow: false, isErrorFollow: true, errorMessageFollow: errorMessage });
      toast.error(errorMessage);
    }
  },

  // Function to get the list of followed users
  getFollowedUsers: async () => {
    try {
      const response = await axiosInstance.get("/followed-users");
      set({ followedUsers: response.data });
    } catch (error) {
      toast.error("Error fetching followed users");
    }
  },

  // Function to update stream
  updateStream: async (streamData) => {
    const { isProcessingStream } = get();
    if (isProcessingStream) return;
```

```
    set({ isProcessingStream: true, isErrorStream: false, errorMessageStream: "" });

    try {
      const response = await axiosInstance.post("/stream/update", streamData);
      set({ streamData: response.data, isProcessingStream: false });
      toast.success("Stream updated successfully!");
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error updating stream";
      set({ isProcessingStream: false, isErrorStream: true, errorMessageStream: errorMessage });
      toast.error(errorMessage);
    }
  },

  // Function to get the list of streams
  getStreams: async () => {
    try {
      const response = await axiosInstance.get("/streams");
      console.log(response.data);
      set({ streams: response.data });
    } catch (error) {
      toast.error("Error fetching streams");
    }
  },

  // Function to search streams
  searchStreams: async (query) => {
    try {
      const response = await axiosInstance.get(`/search-streams?q=${query}`);
      set({ streams: response.data });
    } catch (error) {
      toast.error("Error searching streams");
    }
  },

  // Function to create viewer token
  createViewerToken: async (hostIdentity) => {
    try {
      const response = await axiosInstance.get(`/livekit/create-viewer-token/${hostIdentity}`);
      set({ viewerToken: response.data });
    } catch (error) {
      toast.error("Error creating viewer token");
    }
  },

  // Reset state
  reset: () => set({
    blockedUsers: null,
    followedUsers: null,
    streamData: null, // Reset single stream
    streams: [], // Reset multiple streams
    isProcessingBlock: false,
    isProcessingFollow: false,
    isProcessingStream: false,
    isErrorBlock: false,
    isErrorFollow: false,
    isErrorStream: false,
    errorMessageBlock: "",
    errorMessageFollow: "",
    errorMessageStream: "",
    isProcessingUpdate: false,
    isErrorUpdate: false,
    errorMessageUpdate: "",
    isBlocked: false,
    viewerToken:"",
    isStoppingStream: false,
    isErrorStoppingStream: false,
```

```
    errorMessageStoppingStream: "",
  }),
}));

export default useLiveStreamStore;
```

FILE: frontend/src/stores/useMailStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Ensure this is the correct path to your axiosInstance
instance
import { toast } from "react-hot-toast";

export const useMailStore = create((set, get) => ({
  orderData: null, // Store the order data
  isProcessingMail: false, // Flag to track if the order is being processed
  isErrorMail: false, // Flag to track if there's an error
  errorMessageMail: "", // Error message if any
  isSuccessMail: false, // Flag to track if the order was processed successfully

  // Function to process the order and send the email
  sendMail: async (orderData) => {
    const { isProcessingMail } = get();

    // Prevent sending mail if one is already being processed
    if (isProcessingMail) return;

    set({ isProcessingMail: true, isErrorMail: false, errorMessageMail: "", isSuccessMail: false
});

    try {
      // Send the order data to the backend
      const response = await axiosInstance.post("/send-email", orderData); // Replace with your
backend URL

      set({
        orderData: response.data, // Store the order data in the state
        isProcessingMail: false,
        isSuccessMail: true,
      });
      toast.success("Email sent successfully!");

    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error sending mail";
      set({
        isProcessingMail: false,
        isErrorMail: true,
        errorMessageMail: errorMessage,
      });
      toast.error(errorMessage);
    }
  },

  reset: () => set({
    orderData: null,
    isProcessingMail: false,
    isErrorMail: false,
    errorMessageMail: "",
    isSuccessMail: false,
  }),
}));

export default useMailStore;
```

FILE: frontend/src/stores/useOrderStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Assuming you have axiosInstance configured
import { toast } from "react-hot-toast";
export const useOrderStore = create((set, get) => ({
  orders: [],
  isLoading: false,
  isError: false,
  errorMessage: '',
  orderStatus: null, // To hold the status of a specific order

  // Function to fetch orders for the user warehouse
  fetchOrders: async () => {
    set({ isLoading: true, isError: false, errorMessage: '' }); // Start loading
    try {
      const response = await axiosInstance.get('/warehouse/orders');
      set({ orders: response.data }); // Store orders in the state
    } catch (err) {
      const message = err.response?.data?.message || 'Error fetching orders.';
      set({ isError: true, errorMessage: message }); // Set error state
      toast.error(message); // Display error toast
    } finally {
      set({ isLoading: false }); // Set loading state to false
    }
  },

  // Function to update the status of an order
  updateOrderStatus: async (orderId, status) => {
    set({ isLoading: true, isError: false, errorMessage: '' }); // Start loading
    try {
      const response = await axiosInstance.post(`/orders/${orderId}/status`, { status });

      // Update the local orders state with the new order data
      set((state) => {
        const updatedOrders = state.orders.map((order) =>
          order.id === orderId ? { ...order, status } : order
        );
        return { orders: updatedOrders };
      });

      // Show success toast
      toast.success('Order status updated successfully!');
    } catch (err) {
      const message = err.response?.data?.message || 'Error updating order status.';
      set({ isError: true, errorMessage: message }); // Set error state
      toast.error(message); // Display error toast
    } finally {
      set({ isLoading: false }); // Set loading state to false
    }
  },

  // Function to get the status of an order by its ID
  getOrderStatusById: async (orderId) => {
    set({ isLoading: true, isError: false, errorMessage: '' }); // Start loading
    try {
      const response = await axiosInstance.get(`/orders/${orderId}/status`);

      // Store the status in the state
      set({ orderStatus: response.data });

      // Optionally show success toast
      toast.success('Order status fetched successfully!');
    } catch (err) {
      const message = err.response?.data?.message || 'Error fetching order status.';
      set({ isError: true, errorMessage: message }); // Set error state
      toast.error(message); // Display error toast
    } finally {
```

```
      set({ isLoading: false }); // Set loading state to false
    }
  },

  // Function to listen for real-time order status updates using WebSockets
  listenForOrderStatusUpdates: (orderId) => {
    console.log(orderId)
    const channel = Echo.channel(`order-status.${orderId}`);
    console.log(channel)
    // Listen for OrderStatusUpdated event
    channel.listen('OrderStatusUpdated', (event) => {
        console.log(event);
      set({ orderStatus: event.status });
      toast.success(`Order status updated to: ${event.status}`);
    });

    return () => {
      // Cleanup: Stop listening when the component unmounts
      channel.stopListening('OrderStatusUpdated');
    };
  },

  // Function to reset the store values (if needed)
  reset: () => set({
    orders: [],
    isLoading: false,
    isError: false,
    errorMessage: '',
    orderStatus: null, // Reset orderStatus as well
  }),
}));

export default useOrderStore;
```

FILE: frontend/src/stores/useProductStore.js

```
import { create } from "zustand";
import toast from "react-hot-toast";
import axiosInstance from "../lib/axios";

export const useProductStore = create((set) => ({
    products: [],
    loading: false,
    currentProduct: null, // Add a state for the current product
    inventories: [],
    setProducts: (products) => set({ products }),

    fetchProductById: async (id) => {
        set({ loading: true });
        try {
          const response = await axiosInstance.get(`/products/single/${id}`);
          set({ currentProduct: response.data, loading: false });
        } catch (error) {
          set({ loading: false });
          toast.error(error.response?.data?.error || "Failed to fetch product details");
        }
      },

    createProduct: async (productData) => {
        set({ loading: true });
        try {
            const res = await axiosInstance.post("/products", productData);
            set((prevState) => ({
                products: [...prevState.products, res.data],
                loading: false,
            }));
```

```
        } catch (error) {
            toast.error(error.response.data.error);
            set({ loading: false });
        }
    },
    fetchAllProducts: async () => {
        set({ loading: true });
        try {
            const response = await axiosInstance.get("/products");
            set({ products: response.data.products, loading: false });
        } catch (error) {
            set({ error: "Failed to fetch products", loading: false });
            toast.error(error.response.data.error || "Failed to fetch products");
        }
    },
    getInventoriesForWarehouse: async (product_name, category) => {
        set({ loading: true });
        try {
            // Prepare the filters object to be sent in the POST request
            const filters = {
                product_name,
                category
            };

            // Make the API call with the filters in the request body
            const response = await axiosInstance.post("/warehouse/inventories", filters);

            // Set the response data in the store
            set({ inventories: response.data, loading: false });
            console.log(inventories);
        } catch (error) {
            // Handle errors and show appropriate message
            set({ error: "Failed to fetch products", loading: false });
        }
    },


    updateInventoriesForWarehouse: async (inventoryUpdates) => {
        set({ loading: true });
        try {
            const response = await axiosInstance.put("/warehouse/inventories", {
                inventory_updates: inventoryUpdates,
            });

            // Update the inventories by replacing the updated ones
            set((state) => {
                const updatedInventories = [...state.inventories];

                // Loop through the updated inventories from the server response
                response.data.updated_inventories.forEach((updatedInventory) => {
                    // Find the index of the inventory that matches the updated inventory ID
                    const index = updatedInventories.findIndex(
                        (inventory) => inventory.id === updatedInventory.id
                    );

                    // If a match is found, replace the inventory with the updated one
                    if (index !== -1) {
                        updatedInventories[index] = updatedInventory;
                    }
                });

                // Return updated state with the updated inventories
                return {
                    inventories: updatedInventories,
                    loading: false,
                };
```

```
            });

            toast.success("Inventory updated successfully!");
        } catch (error) {
            set({ loading: false });
            toast.error(error.response?.data?.error || "Failed to update inventories");
        }
    },


    fetchProductsByCategory: async (categories) => {
        set({ loading: true });
        try {
            const response = await axiosInstance.post("/products/categories",{
                categories: categories,
            });
            set({ products: response.data.products, loading: false });
        } catch (error) {
            set({ error: "Failed to fetch products", loading: false });
            toast.error(error.response.data.error || "Failed to fetch products");
        }
    },
    deleteProduct: async (productId) => {
        set({ loading: true });
        try {
            await axiosInstance.delete(`/products/${productId}`);
            set((prevProducts) => ({
                products: prevProducts.products.filter((product) => product._id !== productId),
                loading: false,
            }));
        } catch (error) {
            set({ loading: false });
            toast.error(error.response.data.error || "Failed to delete product");
        }
    },
    toggleFeaturedProduct: async (productId) => {
        set({ loading: true });
        try {
            const response = await axiosInstance.patch(`/products/${productId}`);
            // this will update the isFeatured prop of the product
            set((prevProducts) => ({
                products: prevProducts.products.map((product) =>
                    product._id === productId ? { ...product, isFeatured: response.data.isFeatured
} : product
                ),
                loading: false,
            }));
        } catch (error) {
            set({ loading: false });
            toast.error(error.response.data.error || "Failed to update product");
        }
    },
    fetchFeaturedProducts: async () => {
        set({ loading: true });
        try {
            const response = await axiosInstance.get("/products/featured");
            set({ products: response.data, loading: false });
        } catch (error) {
            set({ error: "Failed to fetch products", loading: false });
            console.log("Error fetching featured products:", error);
        }
    },
}));
```

FILE: frontend/src/stores/useRatingStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Ensure this is the correct path to your axiosInstance
instance
import { toast } from "react-hot-toast";

export const useRatingStore = create((set, get) => ({
  ratings: [],
  ratingLoading: false,
  isSubmitting: false,
  isError: false,
  errorMessage: "",
  selectedProductId: null, // For storing the currently selected product ID
  feedback: "",
  ratingValue: 0,

  fetchRatings: async (productId) => {
    set({ ratingLoading: true, isError: false, errorMessage: "" });

    try {
      const response = await axiosInstance.get(`/products/${productId}/ratings`);
      set({ ratings: response.data, ratingLoading: false });
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Failed to fetch ratings";
      set({ ratingLoading: false, isError: true, errorMessage });
      toast.error(errorMessage);
    }
  },

  submitRating: async (productId, ratingValue, feedback, shipmentId) => {
    set({ isSubmitting: true });

    try {
      const response = await axiosInstance.post(`/products/${productId}/ratings`, {
        rating_value: ratingValue,
        feedback,
        shipments_id: shipmentId, // Assuming you have shipmentId available
      });

      set((state) => ({
        ratings: [...state.ratings, response.data.rating], // Update the ratings list
        feedback: "", // Clear feedback after submission
        ratingValue: 0, // Reset rating value
      }));
      toast.success("Feedback submitted successfully!");
    } catch (error) {
      const errorMessage = error.response?.data?.message || "Failed to submit feedback";
      toast.error(errorMessage);
    } finally {
      set({ isSubmitting: false });
    }
  },

  setFeedback: (feedback) => set({ feedback }),
  setRatingValue: (ratingValue) => set({ ratingValue }),
  setSelectedProductId: (productId) => set({ selectedProductId: productId }),

  reset: () => set({
    ratings: [],
    feedback: "",
    ratingValue: 0,
    selectedProductId: null,
  }),

}));

export default useRatingStore;
```

FILE: frontend/src/stores/useSocialMediaStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios";
import { toast } from "react-hot-toast";

export const useSocialMediaStore = create((set, get) => ({
  posts: [],
  recommendedUsers: [],
  connectionRequests: [],
  connections: [],
  notifications: [],
  post: null,
  connectionStatuses: [],

  isLoadingPosts: false,
  isLoadingRecommendedUsers: false,
  isLoadingConnectionRequests: false,
  isLoadingConnections: false,
  isLoadingNotifications: false,
  isLoadingConnectionStatus: false,
  isLoadingPost: false,

  isErrorPosts: false,
  isErrorRecommendedUsers: false,
  isErrorConnectionRequests: false,
  isErrorConnections: false,
  isErrorNotifications: false,
  isErrorPost: false,
  isErrorConnectionStatus: false,

  errorMessageConnectionStatus: "",
  errorMessage: '',
  errorMessagePost: "",

  fetchConnectionStatus: async (userId) => {
    set({ isLoadingConnectionStatus: true, isErrorConnectionStatus: false,
errorMessageConnectionStatus: "" });

    try {
      const response = await axiosInstance.get(`/connections/status/${userId}`);

      // Use get to retrieve current connection statuses array
      const currentConnectionStatuses = get().connectionStatuses || [];

      // Check if the user already has a status in the array
      const userIndex = currentConnectionStatuses.findIndex(status => status.userId === userId);
      if (userIndex >= 0) {

        // Update the existing user status in the array
        currentConnectionStatuses[userIndex] = { userId, status: response.data };
      } else {
        // If not found, add a new status object for the user
        currentConnectionStatuses.push({ userId, status: response.data });
      }

      // Update state with the modified array
      set({ connectionStatuses: currentConnectionStatuses });

    } catch (error) {
      set({
        isErrorConnectionStatus: true,
        errorMessageConnectionStatus: error.response?.data?.error || "Error fetching connection
status",
      });
      toast.error(error.response?.data?.error || "Error fetching connection status");
```

```
    } finally {
      set({ isLoadingConnectionStatus: false });
    }
  },



  // Send connection request
  sendConnectionRequest: async (userId) => {
    try {
      await axiosInstance.post(`/connections/request/${userId}`);
      toast.success("Connection request sent successfully");
      // Invalidate or refetch the connection status
      set({ connectionStatus: { status: "pending" } });
    } catch (error) {
      toast.error(error.response?.data?.error || "An error occurred");
    }
  },

  // Accept connection request
  acceptConnectionRequestForRecommendedUser: async (requestId) => {
    try {
      await axiosInstance.put(`/connections/accept/${requestId}`);
      toast.success("Connection request accepted");
      // Update connection status
      set({ connectionStatus: { status: "connected" } });
    } catch (error) {
      toast.error(error.response?.data?.error || "An error occurred");
    }
  },

  // Reject connection request
  rejectConnectionRequestForRecommendedUser: async (requestId) => {
    try {
      await axiosInstance.put(`/connections/reject/${requestId}`);
      toast.success("Connection request rejected");
      // Update connection status
      set({ connectionStatus: { status: "not_connected" } });
    } catch (error) {
      toast.error(error.response?.data?.error || "An error occurred");
    }
  },

  // Fetch single post
  fetchPost: async (postId) => {
    set({ isLoadingPost: true, isErrorPost: false, errorMessagePost: "" });

    try {
      const response = await axiosInstance.get(`/posts/${postId}`);
      set({ post: response.data });
    } catch (err) {
      set({ isErrorPost: true, errorMessagePost: err.response?.data?.message || "Error fetching
post" });
      toast.error(err.response?.data?.message || "Error fetching post");
    } finally {
      set({ isLoadingPost: false });
    }
  },
  // Create post action
  createPost: async (postData, queryClient) => {
      const res = await axiosInstance.post("/posts/create", postData);
      toast.success("Post created successfully");

      // Invalidate posts query to refresh data
      queryClient.invalidateQueries({ queryKey: ["posts"] });
  },
```

```
  // Add new actions for post, comment, like and delete
  likePost: async (postId, authUserId) => {
    try {
      await axiosInstance.post(`/posts/${postId}/like`);
      // Optimistically update the posts state after liking
      set((state) => ({
        posts: state.posts.map(post =>
          post.id === postId
            ? { ...post, likes: [...post.likes, authUserId] }
            : post
        ),
      }));
      toast.success("Post liked!");
    } catch (err) {
      toast.error(err.response?.data?.message || 'Error liking post.');
    }
  },

  createComment: async (postId, newComment, authUser) => {
    try {
      const response = await axiosInstance.post(`/posts/${postId}/comment`, { content: newComment
});
      // Add the new comment to the state
      set((state) => ({
        posts: state.posts.map(post =>
          post.id === postId
            ? {
                ...post,
                comments: [
                  ...post.comments,
                  {
                    content: newComment,
                    user: authUser,
                    created_at: new Date(),
                    id: response.data.id,  // Assuming the response contains the new comment's id
                  },
                ],
              }
            : post
        ),
      }));
      toast.success("Comment added!");
    } catch (err) {
      toast.error(err.response?.data?.message || 'Error adding comment.');
    }
  },

  deletePost: async (postId) => {
    try {
      await axiosInstance.delete(`/posts/delete/${postId}`);
      set((state) => ({
        posts: state.posts.filter((post) => post.id !== postId),
      }));
      toast.success("Post deleted!");
    } catch (err) {
      toast.error(err.response?.data?.message || 'Error deleting post.');
    }
  },

  // Fetching posts (if required to be used in the store)
  fetchPosts: async () => {
    try {
      const response = await axiosInstance.get('/posts');
      set({ posts: response.data });
    } catch (err) {
      toast.error(err.response?.data?.message || 'Error fetching posts.');
```

```
    }
  },
  // Fetch notifications
  fetchNotifications: async () => {
    set({ isLoadingNotifications: true, isErrorNotifications: false, errorMessage: '' });
    try {
      const response = await axiosInstance.get('/notifications');
      set({ notifications: response.data });
    } catch (err) {
      set({ isErrorNotifications: true, errorMessage: err.response?.data?.message || 'Error
fetching notifications.' });
      toast.error(err.response?.data?.message || 'Error fetching notifications.');
    } finally {
      set({ isLoadingNotifications: false });
    }
  },

  // Mark notification as read
  markAsRead: async (id) => {
    try {
      await axiosInstance.put(`/notifications/${id}/read`);
      set((state) => ({
        notifications: state.notifications.map((notif) =>
          notif.id === id ? { ...notif, read: true } : notif
        ),
      }));
    } catch (err) {
      toast.error(err.response?.data?.error || 'Error marking notification as read.');
    }
  },

  // Delete notification
  deleteNotification: async (id) => {
    try {
      await axiosInstance.delete(`/notifications/${id}`);
      set((state) => ({
        notifications: state.notifications.filter((notif) => notif.id !== id),
      }));
      toast.success("Notification deleted");
    } catch (err) {
      toast.error(err.response?.data?.error || 'Error deleting notification.');
    }
  },
  // Fetch posts
  fetchPosts: async () => {
    set({ isLoadingPosts: true, isErrorPosts: false, errorMessage: '' });
    try {
      const response = await axiosInstance.get('/posts');
      set({ posts: response.data });
    } catch (err) {
      set({ isErrorPosts: true, errorMessage: err.response?.data?.message || 'Error fetching
posts.' });
      toast.error(err.response?.data?.message || 'Error fetching posts.');
    } finally {
      set({ isLoadingPosts: false });
    }
  },

  // Fetch recommended users
  fetchRecommendedUsers: async () => {
    set({ isLoadingRecommendedUsers: true, isErrorRecommendedUsers: false, errorMessage: '' });
    try {
      const response = await axiosInstance.get('/users/suggestions');
      set({ recommendedUsers: response.data });
    } catch (err) {
      set({ isErrorRecommendedUsers: true, errorMessage: err.response?.data?.message || 'Error
```

```
fetching recommended users.' });
      toast.error(err.response?.data?.message || 'Error fetching recommended users.');
    } finally {
      set({ isLoadingRecommendedUsers: false });
    }
  },

  // Fetch connection requests
  fetchConnectionRequests: async () => {
    set({ isLoadingConnectionRequests: true, isErrorConnectionRequests: false, errorMessage: '' });
    try {
      const response = await axiosInstance.get('/connections/requests');
      set({ connectionRequests: response.data });
    } catch (err) {
      set({ isErrorConnectionRequests: true, errorMessage: err.response?.data?.message || 'Error
fetching connection requests.' });
      toast.error(err.response?.data?.message || 'Error fetching connection requests.');
    } finally {
      set({ isLoadingConnectionRequests: false });
    }
  },

  // Fetch connections
  fetchConnections: async () => {
    set({ isLoadingConnections: true, isErrorConnections: false, errorMessage: '' });
    try {
      const response = await axiosInstance.get('/connections');
      set({ connections: response.data });
    } catch (err) {
      set({ isErrorConnections: true, errorMessage: err.response?.data?.message || 'Error fetching
connections.' });
      toast.error(err.response?.data?.message || 'Error fetching connections.');
    } finally {
      set({ isLoadingConnections: false });
    }
  },

  // Accept a connection request
  acceptConnectionRequest: async (requestId) => {
    try {
      await axiosInstance.put(`/connections/accept/${requestId}`);
      toast.success("Connection request accepted");
      set((state) => ({
        connectionRequests: state.connectionRequests.filter((request) => request.id !== requestId),
      }));
    } catch (err) {
      toast.error(err.response?.data?.error || 'Error accepting connection request.');
    }
  },

  // Reject a connection request
  rejectConnectionRequest: async (requestId) => {
    try {
      await axiosInstance.put(`/connections/reject/${requestId}`);
      toast.success("Connection request rejected");
      set((state) => ({
        connectionRequests: state.connectionRequests.filter((request) => request.id !== requestId),
      }));
    } catch (err) {
      toast.error(err.response?.data?.error || 'Error rejecting connection request.');
    }
  },

  // Reset the store
  reset: () => set({
    posts: [],
```

```
    recommendedUsers: [],
    connectionRequests: [],
    connections: [],
    isLoadingPosts: false,
    isLoadingRecommendedUsers: false,
    isLoadingConnectionRequests: false,
    isLoadingConnections: false,
    isErrorPosts: false,
    isErrorRecommendedUsers: false,
    isErrorConnectionRequests: false,
    isErrorConnections: false,
    errorMessage: '',
    notifications: [],
    isLoadingNotifications: false,
    isErrorNotifications: false,
  }),
}));

export default useSocialMediaStore;
```

FILE: frontend/src/stores/useStreamMessageStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Ensure this is the correct path to your axiosInstance
import { toast } from "react-hot-toast";

export const useStreamMessageStore = create((set, get) => ({
  // Initial state for the stream messages store
  messages: [], // Store all messages
  isProcessingMessage: false, // Flag to track if the message is being sent
  isErrorMessage: false, // Flag to track if there's an error sending the message
  errorMessageMessage: "", // Error message if any
  isSuccessMessage: false, // Flag to track if the message was sent successfully

  // Function to send a message to a viewer
  sendMessage: async (message, creatorId, viewerId, streamId) => {
    const { isProcessingMessage } = get();

    // Prevent sending a message if one is already being processed
    if (isProcessingMessage) return;

    set({ isProcessingMessage: true, isErrorMessage: false, errorMessageMessage: "",
isSuccessMessage: false });

    try {
      // Send the message data to the backend
      const response = await axiosInstance.post("/stream-message", {
        'message' : message,
        'creator_id' : creatorId,
        'viewer_id' : viewerId,
        'stream_id' : streamId,
      }); // Replace with your backend URL

      set({
        isProcessingMessage: false,
        isSuccessMessage: true,
      });
      toast.success("Message sent successfully!");

    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error sending message";
      set({
        isProcessingMessage: false,
        isErrorMessage: true,
        errorMessageMessage: errorMessage,
      });
```

```
      toast.error(errorMessage);
    }
  },

  // Function to get all messages sent by a specific creator
  getMessagesByCreator: async (creatorId) => {
    const { isProcessingMessage } = get();

    // Prevent making multiple requests if one is already in progress
    if (isProcessingMessage) return;

    set({ isProcessingMessage: true, isErrorMessage: false, errorMessageMessage: "",
isSuccessMessage: false });

    try {
      // Get messages from the backend
      const response = await axiosInstance.get(`/stream-messages/creator/${creatorId}`); // Adjust
with the correct backend route

      set({
        messages: response.data, // Store the messages in the state
        isProcessingMessage: false,
        isSuccessMessage: true,
      });

    } catch (error) {
      const errorMessage = error.response?.data?.message || "Error retrieving messages";
      set({
        isProcessingMessage: false,
        isErrorMessage: true,
        errorMessageMessage: errorMessage,
      });
      toast.error(errorMessage);
    }
  },
  listenForNewMessage: (streamId) => {
    // Listen for new messages in the specified stream
    Echo.channel(`stream-chat.${streamId}`)
      .listen('StreamChatMessageSent', (event) => {
        console.log(event.message);
        // When a new message is received, append it to the messages array
        set((state) => ({
          messages: [...state.messages, event],
        }));
        toast.success("New message received!");
      });
  },
  // Function to reset the store
  reset: () => set({
    messages: [],
    isProcessingMessage: false,
    isErrorMessage: false,
    errorMessageMessage: "",
    isSuccessMessage: false,
  }),
}));

export default useStreamMessageStore;
```

FILE: frontend/src/stores/useTwoFactorAuthenticationStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Ensure this is the correct path to your axiosInstance
instance
import { toast } from "react-hot-toast";
```

```
export const useTwoFactorAuthenticationStore = create((set, get) => ({
  sendLoading: false,          // Loading state for sending 2FA code
  verifyLoading: false,        // Loading state for verifying 2FA code
  sendError: null,             // Error state for sending 2FA code
  verifyError: null,           // Error state for verifying 2FA code
  is2FAComplete: false,        // Track if the 2FA process has been completed

  // Send 2FA code to the user
  sendTwoFactorCode: async (phoneNumber) => {
    set({ sendLoading: true, sendError: null });

    try {
      const res = await axiosInstance.post("/send-2fa-code", { phone_number: phoneNumber });

      toast.success("Verification code sent to your phone!");
      set({ sendLoading: false }); // Reset send loading state

    } catch (error) {
      set({ sendLoading: false, sendError: error.response?.data?.message || "An error occurred
while sending the 2FA code" });
      toast.error(error.response?.data?.message || "An error occurred while sending the 2FA code");
    }
  },

  // Verify 2FA code entered by the user
  verifyTwoFactorCode: async (code) => {
    set({ verifyLoading: true, verifyError: null });

    try {
      const res = await axiosInstance.post("/verify-2fa-code", { code });

      set({ verifyLoading: false, is2FAComplete: true });  // Mark 2FA as complete
      toast.success("2FA code verified successfully!");

    } catch (error) {
      set({ verifyLoading: false, verifyError: error.response?.data?.message || "An error occurred
while verifying the 2FA code" });
      toast.error(error.response?.data?.message || "An error occurred while verifying the 2FA
code");
    }
  },

  // Reset 2FA errors and loading state
  resetTwoFactorAuthentication: () => {
    set({ sendLoading: false, verifyLoading: false, sendError: null, verifyError: null,
is2FAComplete: false });
  }
}));
```

FILE: frontend/src/stores/useUserStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Ensure this is the correct path to your axiosInstance
instance
import { toast } from "react-hot-toast";

export const useUserStore = create((set, get) => ({
  user: null,
  loading: false,
  checkingAuth: true,
  userAddresses: [], // Store user addresses
  userRoles: [],  // Add userRoles to the state
  userPermissions: [],  // Add userPermissions to the state
  userOrders: [],  // Add userOrders to the state
  userInvoices: [],
  // Signup user
```

```
  signup: async ({ first_name, last_name, phone_number, email, password, confirmPassword, language
= "en" }) => {
    set({ loading: true });

    // Basic password confirmation validation
    if (password !== confirmPassword) {
      set({ loading: false });
      return toast.error("Passwords do not match");
    }

    try {
      // Make the API call for signup
      const res = await axiosInstance.post("/signup", {
        first_name,
        last_name,
        phone_number,
        email,
        password,
        language
      });

      // Save the access token (you may adjust to store refresh token if needed in the future)
      localStorage.setItem("access_token", res.data.token);

      // Update the user state with the user data returned from the API
      set({ user: res.data.user, loading: false });

      // Show success message
      toast.success("Account created successfully!");
    } catch (error) {
      set({ loading: false });

      // Show error message from API response
      toast.error(error.response?.data?.message || "An error occurred during signup");
    }
  },

  // Login user
  login: async (email, password) => {
    set({ loading: true });

    try {
      // Make the API call for login
      const res = await axiosInstance.post("/login", { email, password });

      // Save the access token (store refresh token if necessary in the future)
      localStorage.setItem("access_token", res.data.token);

      // Update the user state with the user data returned from the API
      set({ user: res.data.user, loading: false });

      // Show success message
      toast.success("Logged in successfully!");
    } catch (error) {
      set({ loading: false });

      // Show error message from API response
      toast.error(error.response?.data?.message || "An error occurred during login");
    }
  },

  logout: async () => {
    try {
      // Step 1: Send logout request to the server
      const response = await axiosInstance.post("/logout");
```

```
      // Step 2: If the server responds successfully, remove the token
      if (response.status === 200) {
        localStorage.removeItem("access_token");
        set({ user: null }); // Update user state to null

        // Show success message
        toast.success("Logged out successfully!");
      } else {
        // Handle unexpected response
        toast.error("Logout failed. Please try again.");
      }
    } catch (error) {
      // Handle any error that occurs during logout
      toast.error(error.response?.data?.message || "An error occurred during logout");
    }
  },

  checkAuth: async () => {
    set({ checkingAuth: true }); // Set checkingAuth to true when checking authentication

    const token = localStorage.getItem("access_token");

    if (!token) {
      set({ checkingAuth: false }); // Set checkingAuth to false if no token exists
      return; // No token, can't check auth
    }

    try {
      // Fetch the user data, including roles, permissions, and orders
      const res = await axiosInstance.get("/me");
      console.log(res.data)
      // Assuming the API response includes user roles, permissions, and orders
      set({
        user: res.data.user,             // Set the user data
        userRoles: res.data.roles,       // Set the user roles
        userPermissions: res.data.permissions, // Set the user permissions
        userOrders: res.data.orders,     // Set the user orders
        userInvoices: res.data.invoices, // Set the user invoices (if provided)
        checkingAuth: false,             // Set checkingAuth to false
      });
    } catch (error) {
      localStorage.removeItem("access_token");
      set({ user: null, checkingAuth: false }); // Set user to null and checkingAuth to false
    }
  },


  updateProfile: async (updatedData) => {
    set({ loading: true });
    try {
      const res = await axiosInstance.put("/user/profile", updatedData);
      set({ user: res.data, loading: false });
      toast.success("Profile updated successfully!");
    } catch (error) {
      set({ loading: false });
      toast.error(error.response?.data?.message || "An error occurred while updating the profile");
    }
  },

  changePassword: async (currentPassword, newPassword, confirmPassword) => {
    set({ loading: true });
    if (newPassword !== confirmPassword) {
      set({ loading: false });
      return toast.error("Passwords do not match");
    }
    try {
```

```
      const res = await axiosInstance.post("/user/change-password", { currentPassword, newPassword
});
      toast.success("Password changed successfully!");
      set({ loading: false });
    } catch (error) {
      set({ loading: false });
      toast.error(error.response?.data?.message || "An error occurred while changing the
password");
    }
  },

  // Get user addresses
  getUserAddresses: async () => {
    set({ loading: true });
    try {
      const res = await axiosInstance.get("/user/addresses");
      set({ userAddresses: res.data, loading: false });
    } catch (error) {
      set({ loading: false });
      toast.error(error.response?.data?.message || "An error occurred while fetching addresses");
    }
  },

  // Store user address
  storeUserAddress: async (addressData) => {
    set({ loading: true });
    try {
      const res = await axiosInstance.post("/user/addresses", addressData);
      set((state) => ({
        userAddresses: [...state.userAddresses, res.data], // Add the new address to the state
        loading: false
      }));
      toast.success("Address added successfully!");
    } catch (error) {
      set({ loading: false });
      toast.error(error.response?.data?.message || "An error occurred while adding the address");
    }
  },

  // Update user address
  updateUserAddress: async (addressId, updatedData) => {
    set({ loading: true });
    try {
      const res = await axiosInstance.put(`/user/addresses/${addressId}`, updatedData);
      set((state) => ({
        userAddresses: state.userAddresses.map((address) =>
          address.id === addressId ? res.data : address // Replace the updated address in the list
        ),
        loading: false
      }));
      toast.success("Address updated successfully!");
    } catch (error) {
      set({ loading: false });
      toast.error(error.response?.data?.message || "An error occurred while updating the address");
    }
  },

  // Set language for the user
  setLanguage: async (language) => {
    set({ loading: true });

    try {
      // Make the API call to update the language
      const res = await axiosInstance.put("/user/set-language", { language });

      // Update the user state with the new language
```

```
      set({
        user: res.data, // Assuming the API response includes the updated language
        loading: false,
      });

      toast.success("Language updated successfully!");
    } catch (error) {
      set({ loading: false });
      toast.error(error.response?.data?.message || "An error occurred while updating the
language");
    }
  },
}));
```

FILE: frontend/src/stores/useVehicleStore.js

```
import { create } from "zustand";
import axiosInstance from "../lib/axios"; // Assuming you have axiosInstance configured
import { toast } from "react-hot-toast";

export const useVehicleStore = create((set, get) => ({
  vehicles: [],
  routeDetailsWithCoordinates: [], // New state to store the route details by orderId
  isLoading: false,
  isError: false,
  errorMessage: '',

  // Function to fetch vehicles and user data
  fetchVehicles: async () => {
    set({ isLoading: true, isError: false, errorMessage: '' }); // Start loading
    try {
      const response = await axiosInstance.get('/vehicles');
      const vehicles = response.data; // Destructure user and vehicles from the response
      console.log(vehicles);
      set({
        vehicles: vehicles, // Store vehicles in the state
      });
    } catch (err) {
      const message = err.response?.data?.message || 'Error fetching vehicles.';
      set({ isError: true, errorMessage: message }); // Set error state
      toast.error(message); // Display error toast
    } finally {
      set({ isLoading: false }); // Set loading state to false
    }
  },

  // Function to update a vehicle
  updateVehicle: async (vehicleId, vehicleData) => {
    set({ isLoading: true, isError: false, errorMessage: '' }); // Start loading
    console.log(vehicleData);
    try {
      // Send the update request
      const response = await axiosInstance.put(`/vehicles/${vehicleId}`, vehicleData);

      // Update the vehicle in the store
      const updatedVehicle = response.data;
      set((state) => {
        const updatedVehicles = state.vehicles.map((vehicle) =>
          vehicle.id === updatedVehicle.id ? updatedVehicle : vehicle
        );
        return { vehicles: updatedVehicles }; // Update the specific vehicle in the state
      });

      toast.success('Vehicle updated successfully'); // Success toast
    } catch (err) {
      const message = err.response?.data?.message || 'Error updating vehicle.';
```

```
      set({ isError: true, errorMessage: message }); // Set error state
      toast.error(message); // Display error toast
    } finally {
      set({ isLoading: false }); // Set loading state to false
    }
  },

  // New function to fetch route details by orderId
  getRouteDetailsByOrderId: async (orderId) => {
    set({ isLoading: true, isError: false, errorMessage: '' }); // Start loading
    try {
      const response = await axiosInstance.get(`/route-details/${orderId}`);
      const routeDetailsWithCoordinates = response.data; // Assuming route details are in the
response body
      console.log(routeDetailsWithCoordinates);
      set({
        routeDetailsWithCoordinates: routeDetailsWithCoordinates, // Store route details in the
state
      });

      toast.success('Route details fetched successfully'); // Success toast
    } catch (err) {
      const message = err.response?.data?.message || 'Error fetching route details.';
      set({ isError: true, errorMessage: message }); // Set error state
      toast.error(message); // Display error toast
    } finally {
      set({ isLoading: false }); // Set loading state to false
    }
  },

  // Function to reset the store values (if needed)
  reset: () => set({
    vehicles: [],
    routeDetailsWithCoordinates: [], // Reset route details as well
    isLoading: false,
    isError: false,
    errorMessage: '',
  }),
}));

export default useVehicleStore;
```

FILE: frontend/src/stores/useViewerTokenStore.js

```
import { create } from "zustand";
import { toast } from "sonner";
import { jwtDecode } from "jwt-decode";
import axiosInstance from "../lib/axios"; // Ensure this is the correct axios instance

export const useViewerTokenStore = create((set, get) => ({
  token: "",
  name: "",
  identity: "",

  fetchViewerToken: async (hostIdentity) => {
    set({ token: "", name: "", identity: "" });  // Reset state before fetching

    try {
      // Make a GET request to fetch the viewer token, passing hostIdentity as a query parameter
      const response = await axiosInstance.get(`/livekit/create-viewer-token/${hostIdentity}`);


      const viewerToken = response.data;
      console.log(viewerToken);

      const decodedToken = jwtDecode(viewerToken);
```

```
      console.log(decodedToken);
      const name = decodedToken?.name;
      const identity = decodedToken?.jti;


      // Set the token, name, and identity all at once
      set({
        token: viewerToken,
        name,
        identity,
      });

    } catch (e) {
      toast.error("Failed to create token");
      console.error("Error fetching viewer token:", e); // Log the error for debugging
    }
  },
}));
```

FILE: frontend/src/stripe.js

```
// src/stripe.js
import { loadStripe } from '@stripe/stripe-js';

// Add your Stripe public key here
const stripePromise =
loadStripe('pk_test_51PMjcCIw69kb65LMm6zd49cWEi1zj4wFnwVbF9mxurJg1JlgoH0S7tOPdJglr0YmyejIYYfDTHhVTFC

export { stripePromise };
```

FILE: frontend/tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
export default {
    content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
    theme: {
        extend: {},
    },
    plugins: [],
};
```

FILE: frontend/vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

// https://vitejs.dev/config/
export default defineConfig({
    plugins: [react()],
    server: {
        proxy: {
            "/api": {
                target: "http://localhost:8000",
            },
        },
    },
    define: {
        global: 'globalThis', // Use globalThis instead of global
    },
    resolve: {
        alias: {
          stream: 'stream-browserify',
        },
      },
```

```
});
```

FILE: postcss.config.js

```
export default {
    plugins: {
        tailwindcss: {},
        autoprefixer: {},
    },
};
```

FILE: public/index.php

```
<?php

use Illuminate\Http\Request;

define('LARAVEL_START', microtime(true));

// Determine if the application is in maintenance mode...
if (file_exists($maintenance = __DIR__.'/../storage/framework/maintenance.php')) {
    require $maintenance;
}

// Register the Composer autoloader...
require __DIR__.'/../vendor/autoload.php';

// Bootstrap Laravel and handle the request...
(require_once __DIR__.'/../bootstrap/app.php')
    ->handleRequest(Request::capture());
```

FILE: resources/js/echo.js

```
import Echo from 'laravel-echo';

import Pusher from 'pusher-js';
window.Pusher = Pusher;

window.Echo = new Echo({
    broadcaster: 'reverb',
    key: import.meta.env.VITE_REVERB_APP_KEY,
    wsHost: import.meta.env.VITE_REVERB_HOST,
    wsPort: import.meta.env.VITE_REVERB_PORT ?? 80,
    wssPort: import.meta.env.VITE_REVERB_PORT ?? 443,
    forceTLS: (import.meta.env.VITE_REVERB_SCHEME ?? 'https') === 'https',
    enabledTransports: ['ws', 'wss'],
});
```

FILE: resources/views/emails/order_confirmation.blade.php

```
<!-- resources/views/emails/order_confirmation.blade.php -->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Order Confirmation</title>
    <style>
        body {
            font-family: Arial, sans-serif;
```

```
        color: #333;
        background-color: #f4f4f4;
        margin: 0;
        padding: 0;
    }

    .email-container {
        max-width: 600px;
        margin: 20px auto;
        background-color: #fff;
        padding: 20px;
        border-radius: 8px;
        box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
    }

    h1 {
        color: #007BFF;
        font-size: 24px;
        text-align: center;
    }

    h3 {
        color: #333;
        font-size: 20px;
        border-bottom: 2px solid #007BFF;
        padding-bottom: 10px;
        margin-bottom: 20px;
    }

    p, ul {
        font-size: 16px;
        line-height: 1.6;
        color: #555;
    }

    strong {
        color: #333;
    }

    ul {
        padding-left: 20px;
        list-style-type: none;
    }

    ul li {
        padding: 10px 0;
        border-bottom: 1px solid #eee;
    }

    .total {
        font-weight: bold;
        color: #007BFF;
    }

    .product-image {
        width: 80px;
        height: auto;
        border-radius: 5px;
    }

    .order-summary, .route-details {
        margin-top: 30px;
    }

    .route-details ul {
        padding-left: 20px;
```

```
        }

        .footer {
            font-size: 14px;
            text-align: center;
            margin-top: 20px;
            color: #777;
        }

        .footer a {
            color: #007BFF;
            text-decoration: none;
        }

        .email-header {
            text-align: center;
            padding: 10px;
            background-color: #007BFF;
            color: #fff;
            border-radius: 8px 8px 0 0;
        }
    </style>
</head>
<body>

    <div class="email-container">
        <div class="email-header">
            <h1>Order Confirmation</h1>
        </div>

        <p>Thank you for your order! Below are your order details:</p>

        <div class="order-summary">
            <h3>Order ID: {{ $orderData['orderSummary']['orderId'] }}</h3>
            <p><strong>Total Amount:</strong> ${{ $orderData['orderSummary']['totalAmount'] }}</p>
            <p><strong>Discount Applied:</strong> -${{ $orderData['orderSummary']['discountAmount'] }}</p>
            <p><strong>Total After Discount:</strong> ${{ $orderData['orderSummary']['totalAfterDiscount'] }}</p>
        </div>

        <div class="product-list">
            <h4>Products:</h4>
            <ul>
                @foreach ($orderData['cartItems'] as $item)
                    <li>
                        <div style="display: flex; align-items: center;">
                            <img class="product-image" src="{{ $item['image'] }}" alt="{{ $item['productName'] }}">
                            <div style="margin-left: 15px;">
                                <strong>{{ $item['productName'] }}</strong><br>
                                Quantity: {{ $item['quantity'] }} x ${{ $item['price'] }}<br>
                                <span class="total">Total: ${{ $item['totalAmount'] }}</span>
                            </div>
                        </div>
                    </li>
                @endforeach
            </ul>
        </div>

        <div class="route-details">
            <h4>Route Details:</h4>
            <ul>
                @foreach ($orderData['routeDetails'] as $route)
                    <li>
                        <strong>Route Name:</strong> {{ $route['routeName'] }}<br>
                        <strong>Start Location:</strong> {{ $route['supplier'] }}<br>
                        <strong>Warehouse:</strong> {{ $route['warehouse'] }}<br>
                        <strong>Destination Location:</strong> {{ $route['destination'] }}<br>
```

```
                        <strong>Estimated Time:</strong> {{ $route['estimatedTime'] }}<br>
                        <strong>Distance:</strong> {{ $route['distance'] }} km
                    </li>
                @endforeach
            </ul>
        </div>

        <div class="footer">
            <p>Thank you for your purchase! We hope you enjoy your order.</p>
            <p>If you have any questions, feel free to <a href="mailto:support@company.com">contact us</a>.</p>
        </div>
    </div>

</body>
</html>
```

FILE: routes/api.php

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\AuthController;
use App\Http\Controllers\CartController;
use App\Http\Controllers\CouponController;
use App\Http\Controllers\CheckoutController;
use App\Http\Controllers\ProductController;
use App\Http\Controllers\PaymentController;
use App\Http\Controllers\WarehouseController;
use App\Http\Controllers\UserProfileController;
use App\Http\Controllers\RatingController;
use App\Http\Controllers\OrderController;
use App\Http\Controllers\VehicleManagerController;
use App\Http\Controllers\DeliveryController;
use App\Http\Controllers\StripeController;
use App\Http\Controllers\LicensePlateController;
use App\Http\Controllers\ConnectionRequestController;
use App\Http\Controllers\NotificationController;
use App\Http\Controllers\PostController;
use App\Http\Controllers\UserController;
use App\Http\Controllers\FeedbackFormController;
use App\Http\Controllers\GroupController;
use App\Http\Controllers\MessageController;
use App\Http\Controllers\ChatBotController;
use App\Http\Controllers\MailController;
use App\Http\Controllers\VideoChatController;
use App\Http\Controllers\BlockController;
use App\Http\Controllers\FollowController;
use App\Http\Controllers\StreamController;
use App\Http\Controllers\ViewerController;
use App\Http\Controllers\StreamMessageController;
use App\Http\Controllers\TwoFactorController;
use App\Http\Controllers\EditorController;


// Apply CORS middleware globally on all routes in this file
Route::middleware(['custom_cors'])->group(function () {

    // Auth Routes
    Route::post('/signup', [AuthController::class, 'signup']);
    Route::post('/login', [AuthController::class, 'login']);
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::middleware(['auth:sanctum',
'role:Administration,WarehouseManager,DeliveryDriver,Customer,CustomerSupportStaff,FinanceManager,ProductSaler,VehicleManager,ShipmentManag
>group(function () {
```

```
        Route::get('/me', [UserProfileController::class, 'getCurrentUser']);
        Route::put('/user/profile', [UserProfileController::class, 'updateDetails']);
        Route::post('/user/change-password', [UserProfileController::class, 'updatePassword']);
        Route::get('/products/{productId}/ratings', [RatingController::class, 'getProductRatings']);
        Route::post('/products/{productId}/ratings', [RatingController::class, 'storeRating']);
        Route::get('/user/addresses', [UserProfileController::class, 'getUserAddresses']);  // Get all addresses
        Route::post('/user/addresses', [UserProfileController::class, 'storeUserAddress']); // Create a new address
        Route::put('/user/addresses/{id}', [UserProfileController::class, 'updateUserAddress']); // Update an existing address

        Route::get('/user/conversations', [MessageController::class, 'getConversationsForSidebar']);
        Route::get('/conversations/{conversationId}/messages', [MessageController::class, 'getMessages']);
        Route::post('/message/store', [MessageController::class, 'store']);
        Route::delete('/messages/{messageId}', [MessageController::class, 'deleteMessage']);
        Route::get('/messages/load-older/{firstMessageId}', [MessageController::class, 'loadOlder']);
        Route::post('/group/store', [GroupController::class, 'store']);
        Route::put('/group/update/{groupId}', [GroupController::class, 'update']);
        Route::delete('/group/destroy/{groupId}', [GroupController::class, 'destroy']);

        Route::post('/chats', [ChatBotController::class, 'storeChat']);
        Route::get('/userchats', [ChatBotController::class, 'getUserChats']);
        Route::get('/chats/{id}', [ChatBotController::class, 'getChatById']);
        Route::put('/chats/{id}', [ChatBotController::class, 'updateChat']);
        Route::get('/chatbot/context', [ChatbotController::class, 'gatherChatbotContext']);
        Route::post('/send-email', [MailController::class, 'sendEMail']);


        Route::post('send-2fa-code', [TwoFactorController::class, 'sendCode']);
        Route::post('verify-2fa-code', [TwoFactorController::class, 'verifyCode']);

        Route::post('/user/set-language', [UserController::class, 'setLanguage']);

    });

    // Route::middleware(['role:Administration,Customer,Customer Support Staff,Product Saler'])->group(function () {
    // });
    Route::get('/products/single/{id}', [ProductController::class, 'fetchProductById']);
    Route::get('/products/featured', [ProductController::class, 'getFeaturedProducts']);
    Route::post('/products/categories', [ProductController::class, 'getProductsByCategory']);
    Route::get('/products/recommendations', [ProductController::class, 'getRecommendedProducts']);

    // Protected Routes
    Route::middleware(['auth:sanctum','role:Administration,Customer,CustomerSupportStaff,ProductSaler'])->group(function () {
        Route::post('/refresh-token', [AuthController::class, 'refreshToken']);
        Route::post('/cart', [CartController::class, 'addToCart']);
        Route::delete('/cart', [CartController::class, 'removeCartItem']);
        Route::put('/cart/quantity', [CartController::class, 'updateOrderItemQuantity']);

        Route::post('/coupon', [CouponController::class, 'getMyCoupon']);
        Route::post('/coupon/apply', [CouponController::class, 'applyCoupon']);
        Route::post('/create-coupon', [CouponController::class, 'create']);
        Route::get('/get-my-coupons', [CouponController::class, 'getAllCoupons']);

        Route::post('/payment/process', [PaymentController::class, 'processPayment']);
        Route::post('/payment/delivery/prepare', [PaymentController::class, 'prepareDelivery']);
        Route::get('/orders/{orderId}/status', [OrderController::class, 'getOrderStatusById']);

        Route::post('/create-payment-intent', [StripeController::class, 'createPaymentIntent']);
        Route::post('/create-payment-intent-gift', [StripeController::class, 'createPaymentIntentGift']);

        Route::get('/route-details/{orderId}', [PaymentController::class, 'getRouteDetailsByOrderId']);

        Route::post('/connections/request/{userId}', [ConnectionRequestController::class, 'sendConnectionRequest']);
        Route::put('/connections/accept/{requestId}', [ConnectionRequestController::class, 'acceptConnectionRequest']);
        Route::put('/connections/reject/{requestId}', [ConnectionRequestController::class, 'rejectConnectionRequest']);
        Route::get('/connections/requests', [ConnectionRequestController::class, 'getConnectionRequests']);
        Route::get('/connections', [ConnectionRequestController::class, 'getUserConnections']);
```

```
        Route::delete('/connections/{userId}', [ConnectionRequestController::class, 'removeConnection']);
        Route::get('/connections/status/{userId}', [ConnectionRequestController::class, 'getConnectionStatus']);
        Route::get('/notifications', [NotificationController::class, 'getUserNotifications']);
        Route::put('/notifications/{id}/read', [NotificationController::class, 'markNotificationAsRead']);
        Route::delete('/notifications/{id}', [NotificationController::class, 'deleteNotification']);
        Route::get('/posts', [PostController::class, 'getFeedPosts']);
        Route::post('/posts/create', [PostController::class, 'createPost']);
        Route::delete('/posts/delete/{id}', [PostController::class, 'deletePost']);
        Route::get('/posts/{id}', [PostController::class, 'getPostById']);
        Route::post('/posts/{id}/comment', [PostController::class, 'createComment']);
        Route::post('/posts/{id}/like', [PostController::class, 'likePost']);
        Route::get('/users/suggestions', [UserController::class, 'getSuggestedConnections']);

        Route::get('/feedback-forms/view/{id}', [FeedbackFormController::class, 'showFeedbackForm']);
        Route::post('/feedback-forms/{id}/answer', [FeedbackFormController::class, 'storeAnswer']);

        Route::post('/block/{id}', [BlockController::class, 'blockUser']);
        Route::post('/unblock/{id}', [BlockController::class, 'unblockUser']);
        Route::get('/blocked-users', [BlockController::class, 'getBlockedUsers']);
        Route::get('/is-blocked/{id}', [BlockController::class, 'isBlockedByUser']);
        Route::post('/follow/{id}', [FollowController::class, 'followUser']);
        Route::post('/unfollow/{id}', [FollowController::class, 'unfollowUser']);
        Route::get('/is-following/{id}', [FollowController::class, 'isFollowingUser']);
        Route::get('/followed-users', [FollowController::class, 'getFollowedUsers']);
        Route::post('create-stream', [StreamController::class, 'createStream']);  // Create a new stream
        Route::post('stop-stream', [StreamController::class, 'stopStream']); // Stop the stream
        Route::post('/stream/update', [StreamController::class, 'updateStream']);
        Route::get('/streams', [StreamController::class, 'getStreams']);
        Route::get('/recommended-users/stream', [StreamController::class, 'getRecommendedUsers']);
        Route::get('/search-streams', [StreamController::class, 'searchStreams']);
        Route::get('/user/stream', [StreamController::class, 'fetchUserStream']);
        Route::get('/livekit/create-viewer-token/{hostIdentity}', [ViewerController::class, 'createViewerToken']);
        Route::put('/user/headline-about', [UserController::class, 'updateHeadlineAndAbout']);
        Route::post('/stream-message', [StreamMessageController::class, 'sendMessage']);
        Route::get('/stream-messages/creator/{creatorId}', [StreamMessageController::class, 'getMessagesByCreator']);

        Route::post('/remove-background', [EditorController::class, 'removeBackground']);
        Route::post('/replace-background', [EditorController::class, 'replaceBackground']);
        Route::post('/extract-image', [EditorController::class, 'extractImage']);
        Route::post('/gen-fill', [EditorController::class, 'genFill']);
        Route::post('/gen-remove', [EditorController::class, 'genRemove']);
        Route::post('/recolor-image', [EditorController::class, 'recolorImage']);
        Route::post('/crop-video', [EditorController::class, 'cropVideo']);
        Route::post('/initiate-transcription', [EditorController::class, 'initiateTranscription']);
        Route::post('/upload-video', [EditorController::class, 'uploadVideo']);
        Route::post('/upload-image', [EditorController::class, 'uploadImage']);

    });
    Route::middleware(['auth:sanctum','role:Administration,ProductSaler'])->group(function () {
        Route::post('/feedback-forms', [FeedbackFormController::class, 'storeFeedbackForm']);
        Route::put('/feedback-forms/{id}', [FeedbackFormController::class, 'updateFeedbackForm']);
        Route::get('/feedback-forms/{orderId}', [FeedbackFormController::class, 'getOrderWithFeedbackForms']);
    });

    // Product Routes (Admin only)
    Route::get('/products', [ProductController::class, 'getAllProducts']);
    Route::post('/products', [ProductController::class, 'createProduct']);
    Route::delete('/products/{id}', [ProductController::class, 'deleteProduct']);
    Route::patch('/products/{id}', [ProductController::class, 'toggleFeaturedProduct']);

    Route::middleware(['auth:sanctum','role:Administration,WarehouseManager'])->group(function () {
        Route::post('/expense', [WarehouseController::class, 'filterExpenses']);
        Route::post('/warehouse/inventories', [WarehouseController::class, 'getInventoriesForWarehouse']);
        Route::put('/warehouse/inventories', [WarehouseController::class, 'updateInventory']);
        Route::post('/warehouse/line-chart', [WarehouseController::class, 'lineChart']);
        Route::get('/warehouse/geography', [WarehouseController::class, 'getGeography']);
```

```
        Route::post('/warehouse/create-inventory', [WarehouseController::class, 'createInventory']);
        Route::get('/warehouse/capacity', [WarehouseController::class, 'getTotalInventoryWeightForUserWarehouse']);
        Route::get('/warehouse/orders', [WarehouseController::class, 'getOrdersByUserWarehouse']);
        Route::post('/orders/{orderId}/status', [WarehouseController::class, 'updateOrderStatus']);
        Route::post('/process-license-plate', [LicensePlateController::class, 'processLicensePlate']);
    });

    Route::middleware(['auth:sanctum','role:Administration,VehicleManager'])->group(function () {
        Route::get('/vehicles', [VehicleManagerController::class, 'getVehiclesWithVehicleManagement']);
        Route::put('/vehicles/{vehicleId}', [VehicleManagerController::class, 'updateVehicleWithManagement']);

    });

    Route::middleware(['auth:sanctum','role:Administration,DeliveryMan'])->group(function () {
        Route::get('/shipments/with-details', [DeliveryController::class, 'getShipmentsWithRouteDetails']);
        Route::put('/route-details/{routeDetailId}', [DeliveryController::class, 'assignVehicle']);
        Route::get('/user/vehicle', [DeliveryController::class, 'getVehicleForAuthenticatedUser']);
    });
});
```

FILE: routes/auth.php

```
<?php

use App\Http\Controllers\Auth\AuthenticatedSessionController;
use App\Http\Controllers\Auth\EmailVerificationNotificationController;
use App\Http\Controllers\Auth\NewPasswordController;
use App\Http\Controllers\Auth\PasswordResetLinkController;
use App\Http\Controllers\Auth\RegisteredUserController;
use App\Http\Controllers\Auth\VerifyEmailController;
use Illuminate\Support\Facades\Route;

Route::post('/register', [RegisteredUserController::class, 'store'])
    ->middleware('guest')
    ->name('register');

Route::post('/login', [AuthenticatedSessionController::class, 'store'])
    ->middleware('guest')
    ->name('login');

Route::post('/forgot-password', [PasswordResetLinkController::class, 'store'])
    ->middleware('guest')
    ->name('password.email');

Route::post('/reset-password', [NewPasswordController::class, 'store'])
    ->middleware('guest')
    ->name('password.store');

Route::get('/verify-email/{id}/{hash}', VerifyEmailController::class)
    ->middleware(['auth', 'signed', 'throttle:6,1'])
    ->name('verification.verify');

Route::post('/email/verification-notification', [EmailVerificationNotificationController::class, 'store'])
    ->middleware(['auth', 'throttle:6,1'])
    ->name('verification.send');

Route::post('/logout', [AuthenticatedSessionController::class, 'destroy'])
    ->middleware('auth')
    ->name('logout');
```

FILE: routes/channels.php

```
<?php

use App\Http\Resources\UserResource;
use Illuminate\Support\Facades\Broadcast;
use App\Models\User;

Broadcast::channel('online', function (User $user) {
    return $user ? new UserResource($user) : null;
});

Broadcast::channel('message.user.{userId1}-{userId2}', function (User $user, int $userId1, int $userId2) {
    return $user->id === $userId1 || $user->id === $userId2 ? $user : null;
});

Broadcast::channel('message.group.{groupId}', function (User $user, int $groupId) {
    return $user->groups->contains('id', $groupId) ? $user : null;
});

Broadcast::channel('group.deleted.{groupId}', function (User $user, int $groupId) {
    return $user->groups->contains('id', $groupId);
});
```

FILE: routes/console.php

```
<?php

use Illuminate\Foundation\Inspiring;
use Illuminate\Support\Facades\Artisan;

Artisan::command('inspire', function () {
    $this->comment(Inspiring::quote());
})->purpose('Display an inspiring quote')->hourly();
```

FILE: routes/web.php

```
<?php

use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return ['Laravel' => app()->version()];
});

require __DIR__.'/auth.php';
```

FILE: tailwind.config.js

```
import defaultTheme from 'tailwindcss/defaultTheme';

/** @type {import('tailwindcss').Config} */
export default {
    content: [
        './vendor/laravel/framework/src/Illuminate/Pagination/resources/views/*.blade.php',
        './storage/framework/views/*.php',
        './resources/**/*.blade.php',
        './resources/**/*.js',
        './resources/**/*.vue',
    ],
```

```
    theme: {
        extend: {
            fontFamily: {
                sans: ['Figtree', ...defaultTheme.fontFamily.sans],
            },
        },
    },
    plugins: [],
};
```

FILE: tests/Feature/Auth/AuthenticationTest.php

```
<?php

use App\Models\User;

test('users can authenticate using the login screen', function () {
    $user = User::factory()->create();

    $response = $this->post('/login', [
        'email' => $user->email,
        'password' => 'password',
    ]);

    $this->assertAuthenticated();
    $response->assertNoContent();
});

test('users can not authenticate with invalid password', function () {
    $user = User::factory()->create();

    $this->post('/login', [
        'email' => $user->email,
        'password' => 'wrong-password',
    ]);

    $this->assertGuest();
});

test('users can logout', function () {
    $user = User::factory()->create();

    $response = $this->actingAs($user)->post('/logout');

    $this->assertGuest();
    $response->assertNoContent();
});
```

FILE: tests/Feature/Auth/EmailVerificationTest.php

```
<?php

use App\Models\User;
use Illuminate\Auth\Events\Verified;
use Illuminate\Support\Facades\Event;
use Illuminate\Support\Facades\URL;

test('email can be verified', function () {
    $user = User::factory()->unverified()->create();

    Event::fake();
```

```
    $verificationUrl = URL::temporarySignedRoute(
        'verification.verify',
        now()->addMinutes(60),
        ['id' => $user->id, 'hash' => sha1($user->email)]
    );

    $response = $this->actingAs($user)->get($verificationUrl);

    Event::assertDispatched(Verified::class);
    expect($user->fresh()->hasVerifiedEmail())->toBeTrue();
    $response->assertRedirect(config('app.frontend_url').'/dashboard?verified=1');
});

test('email is not verified with invalid hash', function () {
    $user = User::factory()->unverified()->create();

    $verificationUrl = URL::temporarySignedRoute(
        'verification.verify',
        now()->addMinutes(60),
        ['id' => $user->id, 'hash' => sha1('wrong-email')]
    );

    $this->actingAs($user)->get($verificationUrl);

    expect($user->fresh()->hasVerifiedEmail())->toBeFalse();
});
```

FILE: tests/Feature/Auth/PasswordResetTest.php

```
<?php

use App\Models\User;
use Illuminate\Auth\Notifications\ResetPassword;
use Illuminate\Support\Facades\Notification;

test('reset password link can be requested', function () {
    Notification::fake();

    $user = User::factory()->create();

    $this->post('/forgot-password', ['email' => $user->email]);

    Notification::assertSentTo($user, ResetPassword::class);
});

test('password can be reset with valid token', function () {
    Notification::fake();

    $user = User::factory()->create();

    $this->post('/forgot-password', ['email' => $user->email]);

    Notification::assertSentTo($user, ResetPassword::class, function (object $notification) use ($user) {
        $response = $this->post('/reset-password', [
            'token' => $notification->token,
            'email' => $user->email,
            'password' => 'password',
            'password_confirmation' => 'password',
        ]);

        $response
            ->assertSessionHasNoErrors()
            ->assertStatus(200);
```

```
        return true;
    });
});
```

FILE: tests/Feature/Auth/RegistrationTest.php

```
<?php

test('new users can register', function () {
    $response = $this->post('/register', [
        'name' => 'Test User',
        'email' => 'test@example.com',
        'password' => 'password',
        'password_confirmation' => 'password',
    ]);

    $this->assertAuthenticated();
    $response->assertNoContent();
});
```

FILE: tests/Feature/ExampleTest.php

```
<?php

test('example', function () {
    $response = $this->get('/');

    $response->assertStatus(200);
});
```

FILE: tests/Pest.php

```
<?php

/*
|--------------------------------------------------------------------------
| Test Case
|--------------------------------------------------------------------------
|
| The closure you provide to your test functions is always bound to a specific PHPUnit test
| case class. By default, that class is "PHPUnit\Framework\TestCase". Of course, you may
| need to change it using the "pest()" function to bind a different classes or traits.
|
*/

pest()->extend(Tests\TestCase::class)
    ->use(Illuminate\Foundation\Testing\RefreshDatabase::class)
    ->in('Feature');

/*
|--------------------------------------------------------------------------
| Expectations
|--------------------------------------------------------------------------
|
| When you're writing tests, you often need to check that values meet certain conditions. The
| "expect()" function gives you access to a set of "expectations" methods that you can use
| to assert different things. Of course, you may extend the Expectation API at any time.
|
```

```
*/

expect()->extend('toBeOne', function () {
    return $this->toBe(1);
});

/*
|--------------------------------------------------------------------------
| Functions
|--------------------------------------------------------------------------
|
| While Pest is very powerful out-of-the-box, you may have some testing code specific to your
| project that you don't want to repeat in every file. Here you can also expose helpers as
| global functions to help you to reduce the number of lines of code in your test files.
|
*/

function something()
{
    // ..
}
```

FILE: tests/TestCase.php

```
<?php

namespace Tests;

use Illuminate\Foundation\Testing\TestCase as BaseTestCase;

abstract class TestCase extends BaseTestCase
{
    //
}
```

FILE: tests/Unit/ExampleTest.php

```
<?php

test('example', function () {
    expect(true)->toBeTrue();
});
```